

Manual de Maxima

Manual de Maxima

Maxima es un sistema de cálculo simbólico escrito en Lisp.

Maxima descende del sistema Macsyma, desarrollado en el MIT (Massachusetts Institute of Technology) entre los años 1968 y 1982 como parte del proyecto MAC. El MIT pasó una copia del código fuente al DOE (Department of Energy) en 1982, en una versión conocida como DOE-Macsyma. Una de estas copias fue mantenida por el Profesor William F. Schelter de la Universidad de Texas desde el año 1982 hasta su fallecimiento en 2001. En 1998 Schelter había obtenido del Departamento de Energía permiso para distribuir el código fuente de DOE-Macsyma bajo licencia GNU-GPL, iniciando en el año 2000 el proyecto Maxima en SourceForge con el fin de mantener y seguir desarrollando DOE-Macsyma, ahora con el nombre de Maxima.

Notas de la traducción española.

La primera traducción al español del manual de Maxima se debió a Juan Pablo Romero Bernal y colaboradores del proyecto GLUD-CLog (<http://glud.udistrital.edu.co/clog>) del Grupo Linux Universidad Distrital (Colombia). Parte de la versión actual se debe a ellos. Actualmente este documento lo mantiene Mario Rodríguez Riotorto (<http://www.biomates.net>); para comentarios, sugerencias y colaboraciones, contactar en la dirección de correo electrónico:

mario ARROBA edu PUNTO xunta PUNTO es

1 Introducción a Maxima

Se puede iniciar Maxima con el comando "maxima". Maxima desplegará alguna información importante acerca de la versión que se está usando y un prompt. Cada comando que vaya a ser ejecutado por Maxima debe terminar con un punto y coma. Para finalizar una sesión en Maxima se emplea el comando "quit();". A continuación se presenta un breve ejemplo de sesión:

```
[wfs@chromium]$ maxima
Maxima 5.9.1 http://maxima.sourceforge.net
Using Lisp CMU Common Lisp 19a
Distributed under the GNU Public License. See the file COPYING.
Dedicated to the memory of William Schelter.
This is a development version of Maxima. The function bug_report()
provides bug reporting information.
(%i1) factor(10!);

              8 4 2
              2 3 5 7
(%o1)
(%i2) expand ((x + y)^6);
              6      5      2 4      3 3      4 2      5      6
(%o2) y  + 6 x y  + 15 x y  + 20 x y  + 15 x y  + 6 x y  + x
(%i3) factor (x^6 - 1);

              2      2
              (x - 1) (x + 1) (x - x + 1) (x + x + 1)
(%o3)
(%i4) quit();
[wfs@chromium]$
```

Maxima puede buscar en las páginas info. Para ello puede usar el comando *describe* que mostrará todos los comandos y variables asociadas con la cadena o argumento de este comando. Opcionalmente puede mostrar la documentación respectiva. El signo de cierre de interrogación ? es una abreviación del comando *describe*:

```
(%i1) ? integ

0: (maxima.info)Introduction to Elliptic Functions and Integrals.
1: Definitions for Elliptic Integrals.
2: Integration.
3: Introduction to Integration.
4: Definitions for Integration.
5: askinteger :Definitions for Simplification.
6: integerp :Definitions for Miscellaneous Options.
7: integrate :Definitions for Integration.
8: integrate_use_rootsof :Definitions for Integration.
9: integration_constant_counter :Definitions for Integration.
Enter space-separated numbers, 'all' or 'none': 6 5

Info from file /usr/local/info/maxima.info:
- Function: integerp (<expr>)
  Returns 'true' if <expr> is an integer, otherwise 'false'.
```

- Function: askinteger (expr, integer)
 - Function: askinteger (expr)
 - Function: askinteger (expr, even)
 - Function: askinteger (expr, odd)
- 'askinteger (expr, integer)' attempts to determine from the 'assume' database whether 'expr' is an integer. 'askinteger' will ask the user if it cannot tell otherwise, and attempt to install the information in the database if possible. 'askinteger (expr)' is equivalent to 'askinteger (expr, integer)'.

'askinteger (expr, even)' and 'askinteger (expr, odd)' likewise attempt to determine if 'expr' is an even integer or odd integer, respectively.

(%o1) false

Para usar un resultado de forma posterior, usted puede asignar dicho resultado a una variable o referirse a él por medio de la etiqueta asociada (%i* o %o*). Adicionalmente puede usar % para referirse al último resultado obtenido.

```
(%i1) u: expand ((x + y)^6);
      6      5      2 4      3 3      4 2      5      6
(%o1) y  + 6 x y  + 15 x  y  + 20 x  y  + 15 x  y  + 6 x  y + x
(%i2) diff (u, x);
      5      4      2 3      3 2      4      5
(%o2) 6 y  + 30 x y  + 60 x  y  + 60 x  y  + 30 x  y + 6 x
(%i3) factor (%o2);
      5
(%o3) 6 (y + x)
```

Maxima manipula sin ningún problema números complejos y constantes numéricas:

```
(%i1) cos(%pi);
(%o1) - 1
(%i2) exp(%i*%pi);
(%o2) - 1
```

Maxima puede hacer derivadas e integrales:

```
(%i1) u: expand ((x + y)^6);
      6      5      2 4      3 3      4 2      5      6
(%o1) y  + 6 x y  + 15 x  y  + 20 x  y  + 15 x  y  + 6 x  y + x
(%i2) diff (% , x);
      5      4      2 3      3 2      4      5
(%o2) 6 y  + 30 x y  + 60 x  y  + 60 x  y  + 30 x  y + 6 x
(%i3) integrate (1/(1 + x^3), x);
      2 x - 1
      2      atan(-----)
      log(x  - x + 1)      sqrt(3)      log(x + 1)
(%o3) - ---- + ---- + ----
      6      sqrt(3)      3
```

Maxima puede resolver sistemas de ecuaciones lineales y cúbicas:

```
(%i1) linsolve ([3*x + 4*y = 7, 2*x + a*y = 13], [x, y]);
```

```

(%o1)          7 a - 52          25
      [x = -----, y = -----]
          3 a - 8          3 a - 8
(%i2) solve (x^3 - 3*x^2 + 5*x = 15, x);
(%o2) [x = - sqrt(5) %i, x = sqrt(5) %i, x = 3]

```

Maxima puede resolver sistemas de ecuaciones no lineales. Tenga en cuenta que si usted no desea que el resultado sea impreso, puede finalizar el comando con \$ en vez de ;.

```

(%i1) eq_1: x^2 + 3*x*y + y^2 = 0$
(%i2) eq_2: 3*x + y = 1$
(%i3) solve ([eq_1, eq_2]);
(%o3) [[y = - (3 sqrt(5) + 7) / 2, x = (sqrt(5) + 3) / 2],
      [y = (3 sqrt(5) - 7) / 2, x = (sqrt(5) - 3) / 2]]

```

Bajo un sistema X window, (es decir que posea interfaz gráfica), Maxima puede generar gráficas de una o más funciones:

```

(%i1) eq_1: x^2 + 3*x*y + y^2 = 0$
(%i2) eq_2: 3*x + y = 1$
(%i3) solve ([eq_1, eq_2]);
(%o3) [[y = - (3 sqrt(5) + 7) / 2, x = (sqrt(5) + 3) / 2],
      [y = (3 sqrt(5) - 7) / 2, x = (sqrt(5) - 3) / 2]]

(%i4) kill(labels);
(%o0) done
(%i1) plot2d (sin(x)/x, [x, -20, 20]);
(%o1)
(%i2) plot2d ([atan(x), erf(x), tanh(x)], [x, -5, 5]);
(%o2)
(%i3) plot3d (sin(sqrt(x^2 + y^2))/sqrt(x^2 + y^2), [x, -12, 12], [y, -12, 12]);
(%o3)

```


2 Reporte y Detección de Bugs

2.1 Introducción a la detección de Bugs y su reporte

Como todos los grandes programas, Maxima contiene tanto bugs conocidos como desconocidos. Este capítulo describe las facilidades internas para ejecutar la suite de evaluación de Maxima (Maxima test suite) además de reportar nuevos bugs.

2.2 Definiciones para la detección de Bugs y su reporte::

run_testsuite ()	Función
run_testsuite (<i>boolean</i>)	Función
run_testsuite (<i>boolean</i> , <i>boolean</i>)	Función
run_testsuite (<i>boolean</i> , <i>boolean</i> , <i>list</i>)	Función

Ejecuta la suite de evaluación de Maxima. Los tests que producen las respuestas deseadas son considerados como “pruebas superadas”, como los tests que no producen las respuestas deseadas, son marcados como bugs conocidos.

run_testsuite () muestra sólo los test que no pasaron la prueba.

run_testsuite (**true**) muestra los tests que son marcados como bugs conocidos, además de fallos.

run_testsuite (**true**, **true**) muestra todos los tests.

Si se incluye el tercer argumento opcional, se ejecutará una parte de los tests. Los tests a ejecutar se especifican formando una lista con los nombres de los mismos. La lista completa de tests está en **testsuite_files**.

run_testsuite cambia el entorno de Maxima. Típicamente un script de test ejecuta **kill** para establecer un entorno conocido (llámese uno sin funciones ni variables definidas por el usuario) y entonces define una serie de funciones y variables apropiadas para el test.

run_testsuite retorna **done**.

testsuite_files	Variable opcional
------------------------	-------------------

testsuite_files es el conjunto de tests a ejecutar por **run_testsuite**. Se trata de una lista con los nombres de los ficheros que contienen los tests a ejecutar. Si se sabe que alguno de los tests de un fichero falla, entonces en lugar de listar el nombre del fichero, se utiliza una lista que contiene el nombre del fichero y los números de los tests que fallan.

Por ejemplo, esta es una parte de los tests por defecto:

```
["rtest13s", ["rtest14", 57, 63]]
```

Con esto se especifica que el conjunto de tests está formado por los ficheros "rtest13s" y "rtest14", pero que "rtest14" contiene dos tests que se sabe que causan fallos, el 57 y el 63.

bug_report () Función

Imprime las versiones de Maxima y de Lisp y proporciona un enlace a la página web de reporte de bugs del proyecto Maxima. La información respecto a las versiones es la misma que reporta la función **build_info**.

Cuando un bug es reportado, es de gran ayuda que se copie la información relacionada con la versión de Maxima y de Lisp usada, dentro del reporte de bug.

bug_report retorna una cadena vacía "".

build_info () Función

Imprime un resumen de los parámetros que se usaron para construir la versión de Maxima que se está usando.

build_info retorna una cadena vacía "".

3 Ayuda

3.1 Introducción a la Ayuda

El comando de ayuda más útil es `describe`, el cual es invocado por el signo de cierre de interrogación `?` a través del prompt interactivo. `? foo` (con espacio entre `?` y `foo`) es equivalente a `describe ("foo")`, donde `foo` es el nombre o parte del nombre de alguna función o tema; entonces `describe` encontrará todos los documentos que contengan la cadena `foo` en sus títulos y los desplegará como opciones. Si hay más de una opción, Maxima preguntará al usuario para que seleccione las opciones que desee consultar.

```
(%i1) ? integ
0: (maxima.info)Introduction to Elliptic Functions and Integrals.
1: Definitions for Elliptic Integrals.
2: Integration.
3: Introduction to Integration.
4: Definitions for Integration.
5: askinteger :Definitions for Simplification.
6: integerp :Definitions for Miscellaneous Options.
7: integrate :Definitions for Integration.
8: integrate_use_rootsof :Definitions for Integration.
9: integration_constant_counter :Definitions for Integration.
Enter space-separated numbers, 'all' or 'none': 7 8
```

```
Info from file /use/local/maxima/doc/info/maxima.info:
- Function: integrate (expr, var)
- Function: integrate (expr, var, a, b)
  Attempts to symbolically compute the integral of 'expr' with
  respect to 'var'. 'integrate (expr, var)' is an indefinite
  integral, while 'integrate (expr, var, a, b)' is a definite
  integral, [...]
```

En este ejemplo las opciones 7 y 8 fueron seleccionadas. Todas o ninguna de las opciones pueden ser seleccionadas escribiendo `all` o `none`, las cuales pueden ser abreviadas por `a` o `n`, respectivamente.

3.2 Lisp y Maxima

Maxima fue escrito en Lisp, y es muy fácil tener acceso a funciones y variables Lisp desde Maxima y viceversa. Los símbolos Lisp y los símbolos Maxima están claramente diferenciados por medio de una convención de nombres. Un símbolo Lisp el cual comienza con un signo pesos `$` corresponde a un símbolo Maxima sin el signo pesos. Un símbolo Maxima el cual comienza con un signo de cierre de interrogación `?` corresponde a un símbolo Lisp sin dicho signo. Por ejemplo, el símbolo Maxima `foo` corresponde a el símbolo Lisp `$foo`, mientras que el símbolo Maxima `?foo` corresponde a el símbolo Lisp `foo`, tenga en cuenta que `?foo` esta escrito sin espacio entre `?` y `foo`; de otra manera se estaría invocando a `describe ("foo")`.

El guión -, asterisco *, u otros caracteres especiales en símbolos Lisp deben ser escritos mediante un backslash \ si aparecen en código Maxima. Por ejemplo, el identificador Lisp `*foo-bar*` se debe escribir `?\*foo\-bar\*` en Maxima.

Se puede ejecutar código Lisp desde una sesión de Maxima. Una línea Lisp (que contenga una o más formas) puede ser ejecutada por medio de un comando especial `:lisp`. Por ejemplo,

```
(%i1) :lisp (foo $x $y)
```

se llama a la función Lisp `foo` con variables Maxima `x` y `y` como argumentos. La instrucción `:lisp` puede aparecer en el prompt interactivo o en un archivo que sea procesado por `batch` o `demo`, pero no en un archivo que sea procesado por `load`, `batchload`, `translate_file` o `compile_file`.

La función `to_lisp()` abre una sesión interactiva con el interprete Lisp. Escribiendo `(to-maxima)` se cierra la sesión con Lisp y se retorna a Maxima.

Las funciones y variables Lisp las cuales esten para ser visibles en Maxima como funciones y variables con nombres ordinarios (sin una puntuación especial), deben tener nombres tipo Lisp que comiencen con el signo pesos \$.

Maxima es case-sensitive, distingue entre letras minúsculas y mayúsculas en identificadores, mientras que Lisp no. Existen algunas reglas que gobiernan la traducción de nombres entre Lisp y Maxima.

1. Un identificador Lisp que no se encuentra encerrado en barras verticales corresponde a un identificador Maxima in minúscula. Que el identificador Lisp este en mayúscula, minúscula o una combinación de las dos, no afecta en nada. E.g., los identificadores Lisp `$foo`, `$FOO`, y `$Foo`, todos corresponden al identificador Maxima `foo`.
2. Unn identificador Lisp el cual se encuentre todo en mayúscula o todo en minúscula y encerrado entre barras verticales corresponde a un identificador Maxima con el caso contrario. Esto es, de mayúsculas cambia a minúsculas y de minúsculas cambia a mayúsculas. E.g., el identificador Lisp `|$FOO|` y `|$foo|` corresponden los identificadores Maxima `foo` y `FOO`, respectivamente.
3. Un identificador Lisp el cual esta escrito mezclando letras mayúsculas y minúsculas y se encuentra entre barras verticales corresponde a un identificador Maxima con la misma escritura. E.g., el identificador Lisp `|$Foo|` corresponde a el identificador Maxima `Foo`.

La macro Lisp `#$` permite el uso de expresiones Maxima dentro de código Lisp. `#$expr$` extiende a una expresión Lisp equivalente a la expresión Maxima `expr`.

```
(msetq $foo #$(x, y)$)
```

Esto tiene el mismo efecto que:

```
(%i1) foo: [x, y];
```

La función Lisp `displa` imprime una expresión en formato Maxima.

```
(%i1) :lisp #$(x, y, z)$
((MLIST SIMP) $X $Y $Z)
(%i1) :lisp (displa '((MLIST SIMP) $X $Y $Z))
[x, y, z]
NIL
```

Las funciones definidas en Maxima no son funciones Lisp ordinarias. La función Lisp `mfuncall` llama a una función Maxima. Por ejemplo:

```
(%i1) foo(x,y) := x*y$
(%i2) :lisp (mfuncall '$foo 'a 'b)
(MTIMES SIMP) A B)
```

Algunas funciones Lisp son compartidas en el paquete Maxima, las cuales se listan a continuación:

`complement`, `continue`, `//`, `float`, `functionp`, `array`, `exp`, `listen`, `signum`, `atan`, `asin`, `acos`, `asinh`, `acosh`, `atanh`, `tanh`, `cosh`, `sinh`, `tan`, `break`, y `gcd`.

3.3 Recolector de Basura

La computación simbólica tiende a crear una buena cantidad de basura, y un manejo efectivo de esto puede ser crucial para el término exitoso de algunos programas.

Bajo GCL (GNU Common Lisp), en los sistemas UNIX donde la llamada al sistema `mprotect` esta disponible (incluyendo SUN OS 4.0 y algunas variantes de BSD) un recolector de basura estratificado está disponible. Estos límites de colección para memoria virtual, han sido escritos recientemente. Mire la documentación de GCL bajo `ALLOCATE` y `GBC`. En el nivel lisp haga `(setq si::*notify-gbc* t)` eso le ayudará a determinar cuales áreas necesitan más espacio.

3.4 Documentación

El manual en línea del usuario de Maxima puede ser visto en diferentes formas. Desde el prompt interactivo de Maxima, el manual de usuario es visto como texto plano por medio del comando `?` (i.e., la función `describe`). El manual de usuario también puede ser visto como hipertexto tipo `info` por medio del programa `info` y como una página web a través de cualquier navegador.

El comando `example` muestra ejemplos para muchas funciones Maxima. Por ejemplo:

```
(%i1) example (integrate);
produce:
(%i2) test(f):=block([u],u:integrate(f,x),ratsimp(f-diff(u,x)))
(%o2) test(f) := block([u], u : integrate(f, x),

                                     ratsimp(f - diff(u, x)))

(%i3) test(sin(x))
(%o3)                                     0
(%i4) test(1/(x+1))
(%o4)                                     0
(%i5) test(1/(x^2+1))
(%o5)                                     0
```

y salidas adicionales.

3.5 Definiciones para la Ayuda

demo (*archivo*)

Función

Evalúa las expresiones Maxima contenidas en *archivo* y muestra los resultados. `demo` hace pausas después de evaluar cada expresión y continúa después de que el usuario

ingrese un retorno de carro. (Si se ejecuta en Xmaxima, `demo` puede que necesite un punto y coma ; a continuación del retorno de carro.)

`demo` busca la lista de directorios `file_search_demo` para encontrar `archivo`. Si el archivo tiene el sufijo `dem`, el sufijo puede ser omitido. Ver también `file_search`.

`demo` evalúa su argumento. `demo` retorna el nombre del archivo demostración.

Ejemplo:

```
(%i1) demo ("disol");

batching /home/wfs/maxima/share/simplification/disol.dem
At the _ prompt, type ';' followed by enter to get next demo
(%i2)                                load(disol)

-
(%i3)          exp1 : a (e (g + f) + b (d + c))
(%o3)          a (e (g + f) + b (d + c))

-
(%i4)          disolate(exp1, a, b, e)
(%t4)          d + c

(%t5)          g + f

(%o5)          a (%t5 e + %t4 b)

-
(%i5) demo ("rncomb");

batching /home/wfs/maxima/share/simplification/rncomb.dem
At the _ prompt, type ';' followed by enter to get next demo
(%i6)                                load(rncomb)

-
(%i7)          exp1 : 
$$\frac{z}{y + x} + \frac{x}{2 (y + x)}$$

(%o7)          
$$\frac{z}{y + x} + \frac{x}{2 (y + x)}$$


-
(%i8)          combine(exp1)
(%o8)          
$$\frac{z}{y + x} + \frac{x}{2 (y + x)}$$


-
(%i9)          rncombine(%)
(%o9)          
$$2 z + x$$

```

```
(%o9)
-----
2 (y + x)

-
(%i10)
exp2 : - + - + - + -
      3  3  2  2
      d  c  b  a
      - + - + - + -
      3  3  2  2

(%o10)
-----
2 d + 2 c + 3 (b + a)

-
(%i11)
combine(exp2)
-----
6

-
(%i12)
rncombine(exp2)
-----
6

-
(%i13)
```

describe (*cadena*)

Función

Encuentra todos los documentos los cuales contengan *cadena* es sus títulos y los despliega como opciones. Si hay más de una opción, Maxima preguntará al usuario para que seleccione las opciones que desee consultar. En el prompt interactivo, `? foo` (con espacio entre `?` y `foo`) es equivalente a `describe ("foo")`.

`describe ("")` produce una lista de todos los temas documentados en el manual en línea.

`describe` no evalúa su argumento. `describe` siempre retorna `false`.

Ejemplo:

```
(%i1) ? integ
0: (maxima.info)Introduction to Elliptic Functions and Integrals.
1: Definitions for Elliptic Integrals.
2: Integration.
3: Introduction to Integration.
4: Definitions for Integration.
5: askinteger :Definitions for Simplification.
6: integerp :Definitions for Miscellaneous Options.
7: integrate :Definitions for Integration.
8: integrate_use_rootsof :Definitions for Integration.
9: integration_constant_counter :Definitions for Integration.
Enter space-separated numbers, 'all' or 'none': 7 8
```

Info from file /use/local/maxima/doc/info/maxima.info:

- Function: integrate (expr, var)
 - Function: integrate (expr, var, a, b)
- Attempts to symbolically compute the integral of 'expr' with respect to 'var'. 'integrate (expr, var)' is an indefinite integral, while 'integrate (expr, var, a, b)' is a definite integral, [...]

En este ejemplo las opciones 7 y 8 fueron seleccionadas. Todas o ninguna de las opciones pueden ser seleccionadas escribiendo `all` o `none`, las cuales pueden ser abreviadas por `a` o `n`, respectivamente.

example (*tema*)

Función

example ()

Función

example (*tema*) mostrará algunos ejemplos de *tema*, el cual es un símbolo (no una cadena). La mayoría de los temas son nombres de funciones. **example** () retorna la lista de todos los temas conocidos.

El nombre del archivo que contiene los ejemplos esta dado por la variable global `manual_demo`, cuyo valor por defecto es `"manual_demo"`.

example no evalua su argumento. **example** retorna `done` a menos que haya un error o que no haya argumento, en cuyo caso **example** retornará la lista de todos los temas conocidos.

Ejemplos:

```
(%i1) example (append);
(%i2) append([x+y,0,-3.2],[2.5E+20,x])
(%o2) [y + x, 0, - 3.2, 2.5E+20, x]
(%o2) done
(%i3) example (coeff);
(%i4) coeff(b+tan(x)+2*a*tan(x) = 3+5*tan(x),tan(x))
(%o4) 2 a + 1 = 5
(%i5) coeff(1+x*%e^x+y,x,0)
(%o5) y + 1
(%o5) done
```

4 Línea de Comandos

4.1 Introducción a la Línea de Comandos

""

Operador

El operador comilla sencilla ' evita la evaluación.

Aplicado a un símbolo, la comilla sencilla evita la evaluación del símbolo.

Aplicado a la llamada de una función, la comilla sencilla evita la evaluación de la función llamada, aunque los argumentos de la función son evaluados (siempre y cuando la evaluación no se evite de otra manera). El resultado es una forma de nombre de la función llamada.

Aplicado a una expresión con paréntesis, la comilla simple evita la evaluación de todos los símbolos y llamadas a funciones que hayan en la expresión. E.g., '(f(x))' significa que no se evalúa la expresión f(x). 'f(x)' (con la comilla simple aplicada a f en cambio de a f(x)) significa el retorno de la forma de nombre de f aplicada a [x].

La comilla simple no evita la simplificación.

Cuando el interruptor global `noundisp` es `true`, los nombres se muestran con una comilla simple. Este interruptor siempre tiene como valor `true` cuando se muestran definiciones de funciones.

Ver también los operadores comilla-comilla '' y `nouns`.

Ejemplos:

Aplicado a un símbolo, la comilla sencilla evita la evaluación del símbolo.

```
(%i1) aa: 1024;
(%o1)                                     1024
(%i2) aa^2;
(%o2)                                   1048576
(%i3) 'aa^2;
                                           2
(%o3)                                   aa
(%i4) ''%;
(%o4)                                   1048576
```

Aplicado a la llamada de una función, la comilla sencilla evita la evaluación de la función llamada, aunque los argumentos de la función son evaluados (siempre y cuando la evaluación no se evite de otra manera). El resultado es una forma de nombre de la función llamada.

```
(%i1) x0: 5;
(%o1)                                     5
(%i2) x1: 7;
(%o2)                                     7
(%i3) integrate (x^2, x, x0, x1);
                                           218
(%o3)                                     ---
                                           3
```



```
(%i4) 'integrate (x^2, x, x0, x1);
                                     7
                                     /
                                     [  2
(%o4)                               I  x  dx
                                     ]
                                     /
                                     5

(%i5) %, nouns;
(%o5)                               218
                                     ---
                                     3
```

Aplicado a una expresión con paréntesis, la comilla simple evita la evaluación de todos los símbolos y llamadas a funciones que hayan en la expresión.

```
(%i1) aa: 1024;
(%o1)                               1024
(%i2) bb: 19;
(%o2)                               19
(%i3) sqrt(aa) + bb;
(%o3)                               51
(%i4) '(sqrt(aa) + bb);
(%o4)                               bb + sqrt(aa)
(%i5) ''%;
(%o5)                               51
```

La comilla sencilla no evita la simplificación.

```
(%i1) sin (17 * %pi) + cos (17 * %pi);
(%o1)                               - 1
(%i2) '(sin (17 * %pi) + cos (17 * %pi));
(%o2)                               - 1
```

""

Operador

El operador '' (doble comilla simple) causa que una evaluación extra tenga lugar. E.g., ''%i4 reevaluará la línea de entrada %i4. ''(f(x)) significa evaluar la expresión f(x) en tiempo extra. ''f(x) (con la doble comilla simple aplicada a f en vez de a f(x)) significa el retorno de la forma verbal de f aplicada a [x].

4.2 Definiciones para la Línea de Comandos

alias (*new_name_1, old_name_1, ..., new_name_n, old_name_n*) Función
 provee un nombre alternativo para una (bien sea definida por el usuario o por el sistema) función, variable, arreglo, etc. Cualquier número par de argumentos puede ser usado.

debugmode Variable opcional
 Valor por defecto: `false`

Cuando en Maxima ocurre un error, Maxima inicializará el depurador si `debugmode` tiene el valor `true`. El usuario puede ingresar comandos para examinar la pila de

llamadas, los puntos de interrupción; en pocas palabras ir a través del código de Maxima. Vea `debugging` para una lista de los comandos del depurador.

Habilitando `debugmode` no se capturarán los errores tipo Lisp.

ev (*expr*, *arg-1*, ..., *arg-n*)

Función

Evalúa la expresión *expr* en el entorno especificado por los argumentos *arg-1*, ..., *arg-n*. Los argumentos son interruptores (Variables Booleanas), variables de asignación, ecuaciones y funciones. **ev** retorna el resultado (otra expresión) de la evaluación.

La evaluación se realiza por etapas, como sigue:

1. Primero se configura el entorno de acuerdo a los argumentos los cuales pueden ser algunos o todos de la siguiente lista:
 - **simp** causa que *expr* sea simplificada sin importar el valor de la variable interruptor **simp** la cual inhibe la simplificación cuando su valor es **false**.
 - **noeval** suprime la fase de evaluación de **ev** (Vea el paso (4) más adelante). Esto es muy útil en conjunción con otras variables interruptor y causan en *expr* que sea resimplificada sin ser reevaluada.
 - **nouns** causa que la evaluación de las formas de nombre (típicamente funciones sin evaluar tales como `'integrate` or `'diff`) en *expr*.
 - **expand** causa expansión.
 - **expand** (*m*, *n*) causa expansión, asignando los valores de **maxposex** y **maxnegex** a *m* y *n*, respectivamente.
 - **detout** causa que cualesquiera matrices inversas computadas en *expr* conserven su determinante fuera de la inversa en vez de que divida a cada elemento.
 - **diif** causa que todas las diferenciaciones indicadas en *expr* sean llevadas a cabo.
 - **derivlist** (*x*, *y*, *z*, ...) causa que sólo las diferenciaciones con respecto a las variables indicadas se lleven a cabo.
 - **float** causa que los números racionales no-enteros sean convertidos a números de tipo flotante.
 - **numer** causa que algunas funciones matemáticas (incluyendo potenciación) con argumentos numéricos sean evaluados como punto flotante. Esto causa que las variables en *expr* las cuales hayan sido declaradas como variables numéricas sean reemplazadas por sus respectivos valores. Esto también configura la variable interruptor **float** a **true**.
 - **pred** causa que los predicados (expresiones las cuales se evalúan a **true** o **false**) sean evaluados.
 - **eval** causa que una post-evaluación extra de *expr* ocurra. (Vea el paso (5) más adelante).
 - **A**, donde **A** es un átomo declarado como una variable de tipo interruptor, (Vea **evflag**) causa que **A** tenga como valor **true** durante la evaluación de *expr*.

- **V: expresion** (o alternativamente **V=expresion**) causa que **V** tenga el valor de **expresion** durante la evaluación de **expr**. Notese que si **V** es una opción Maxima, entonces **expresion** se usa como su valor durante la evaluación de **expr**. Si más de un argumento de **ev** es de este tipo entonces el vínculo se hace en paralelo. Si **V** es una expresión no atómica entonces se hace una sustitución más que un vínculo.
- **F** donde **F**, un nombre de función, ha sido declarado para ser una función de evaluación (Vea **evfun**) causa que **F** sea aplicada a **expr**.
- Cualquier otro nombre de función (e.g., **sum**) causa la evaluación de las ocurrencias de esos nombres en **expr** como si ellos fueran verbos.
- En adición de que una función ocurra en **expr** (digamos **F(x)**) puede ser definida localmente para el propósito de esta evaluación de **expr** pasando **F(x) := expresion** como un argumento a **ev**.
- Si un átomo no mencionado anteriormente o una variable o expresión con subíndices fueran pasadas como un argumento, esta es evaluada y si el resultado es una ecuación o una asignación entonces el vínculo o sustitución se llevará a cabo. Si el resultado es una lista entonces los miembros de la lista tratados como si ellos fueran argumentos adicionales pasados a **ev**. Esto permite que una lista de argumentos sea pasada (e.g., **[X=1, Y=A**2]**) o una lista de nombres de ecuaciones (e.g., **[%t1, %t2]** donde **%t1** y **%t2** son ecuaciones) tal como lo que es retornado por **solve**.

Los argumentos de **ev** pueden ser pasados en cualquier orden con excepción de la sustitución de ecuaciones las cuales son manipuladas en secuencia, de izquierda a derecha y las funciones de evaluación las cuales son compuestas, e.g., **ev (expr, ratsimp, realpart)** es manipulada como **realpart (ratsimp (expr))**.

Los interruptores **simp**, **numer**, **float** y **pred** pueden también ser configurados localmente en una sentencia block, o globalmente en Maxima para que su efecto permanezca hasta que sean reconfiguradas.

Si **expr** es una Expresión Racional Canónica (CRE, por sus siglas en inglés), entonces la expresión retornada por **ev** es también de tipo CRE, siempre que los interruptores **numer** y **float** no sean **true**.

2. Durante el paso (1), se fabrica una lista de las variables que no contienen subíndices que aparecen en el lado izquierdo de las ecuaciones en los argumentos o en el valor de algunos argumentos si el valor es una ecuación. Las variables (variables que contienen subíndices las cuales no tienen asociado un arreglo de funciones como también las variables que no contienen subíndices) en la expresión **expr** son reemplazadas por sus valores globales, excepto por aquellos que aparezcan en esa lista. Usualmente, **expr** es sólo una etiqueta o un **%** (como en **%i2** en el ejemplo de más abajo) así que este paso simplemente recupera la expresión a la que hace referencia la etiqueta y así **ev** puede trabajarla.
3. Si algunas sustituciones son indicadas por los argumentos, ellas serán llevadas a cabo ahora.
4. La expresión resultante es también reevaluada (a menos que uno de los argumentos fuese **noeval**) y simplificada de acuerdo a los argumentos. Notese que cualquier llamada a una función en **expr** será llevada a cabo después de que

las variables sean evaluadas en ella y que `ev(F(x))` pueda comportarse como `F(ev(x))`.

5. Si uno de los argumentos fuese `eval`, los pasos (3) y (4) se repiten.

Ejemplos

```
(%i1) sin(x) + cos(y) + (w+1)^2 + 'diff (sin(w), w);
                                d
(%o1)          cos(y) + sin(x) + -- (sin(w)) + (w + 1)
                                dw
(%i2) ev (%, sin, expand, diff, x=2, y=1);
                                2
(%o2)          cos(w) + w  + 2 w + cos(1) + 1.909297426825682
```

Una sintaxis alternativa de alto nivel ha sido proveida para `ev`, por medio de la cual uno puede escribir solamente sus argumentos, sin el comando `ev()`. Esto es, una forma sencilla de escritura:

`expr, arg_1, ..., arg_n`

Esto no es permitido como parte de otra expresión, e.g., en funciones, sentencias `block`, etc.

Nótese el proceso de vínculo en paralelo en el siguiente ejemplo:

```
(%i3) programmode: false;
(%o3)          false
(%i4) x+y, x: a+y, y: 2;
(%o4)          y + a + 2
(%i5) 2*x - 3*y = 3$
(%i6) -3*x + 2*y = -4$
(%i7) solve ([%o5, %o6]);
Solución

                                1
(%t7)          y = - -
                                5

                                6
(%t8)          x = -
                                5
(%o8)          [[%t7, %t8]]
(%i8) %o6, %o8;
(%o8)          - 4 = - 4
(%i9) x + 1/x > gamma (1/2);
                                1
(%o9)          x + - > sqrt(%pi)
                                x
(%i10) %, numer, x=1/2;
(%o10)          2.5 > 1.772453850905516
(%i11) %, pred;
(%o11)          true
```

evflag

Propiedad

Algunos interruptores booleanos tienen la propiedad **evflag**. **ev** trata de forma especial tales interruptores. Un interruptor con la propiedad **evflag** será puesto a **true** durante la ejecución de **ev** si este es mencionado en la llamada a **ev**. Por ejemplo, **demoivre** y **ratfac** son puestos a **true** durante la llamada a **ev** (**%**, **demoivre**, **ratfac**).

Los interruptores que tienen la propiedad **evflag** son:

algebraic, **cauchysum**, **demoivre**, **dotscrules**, **%emode**, **%enumer**, **exponentialize**, **exptisolate**, **factorflag**, **float**, **halfangles**, **infeval**, **isolate_wrt_times**, **keepfloat**, **letrat**, **listarith**, **logabs**, **logarc**, **logexpand**, **lognegint**, **lognumer**, **mipbranch**, **numer_pbranch**, **programmode**, **radexpand**, **ratalgdenom**, **ratfac**, **ratmx**, **ratsimpexpons**, **simp**, **simplsum**, **sumexpand**, y **trigexpand**.

La instrucción **:lisp (putprop '\$foo t 'evflag)** da la propiedad **evflag** a la variable **foo**, así que **foo** es puesta a **true** durante la llamada a **ev** (**%**, **foo**). De forma equivalente, **ev (\$, foo:true)** tiene el mismo efecto.

evfun

Propiedad

Algunas funciones tienen la propiedad **evfun**. **ev** trata tales funciones de forma especial. Una función con la propiedad **evfun** será aplicada durante la ejecución de **ev** si ésta es mencionada en la llamada a **ev**. Por ejemplo, **ratsimp** y **radcan** serán aplicadas durante la llamada a **ev** (**%**, **ratsimp**, **radcan**).

Las funciones que tienen la propiedad **evfun** son:

bfloat, **factor**, **fullratsimp**, **logcontract**, **polarform**, **radcan**, **ratexpand**, **ratsimp**, **rectform**, **rootscontract**, **trigexpand**, y **trigreduce**.

La instrucción **:lisp (putprop '\$foo t 'evfun)** da la propiedad **evfun** a la función **foo**, así que **foo** es aplicada durante la llamada a **ev** (**%**, **foo**). De forma equivalente, **foo (ev(%))** tiene el mismo efecto.

infeval

Variable opcional

Habilita el modo de "evaluación infinita". **ev** repetidamente evalúa una expresión hasta que se interrumpa la acción. Para prevenir que una variable, digamos **X**, sea evaluada sin parar en este modo, simplemente incluya **X='X** como argumento de **ev**. Esta claro que expresiones como: **ev (X, X=X+1, infeval)** generarán un bucle infinito.

kill (<i>symbol_1</i> , ..., <i>symbol_n</i>)	Función
kill (<i>labels</i>)	Función
kill (<i>inlabels</i> , <i>outlabels</i> , <i>linelabels</i>)	Función
kill (<i>n</i>)	Función
kill (<i>[m, n]</i>)	Función
kill (<i>values</i> , <i>functions</i> , <i>arrays</i> , ...)	Función
kill (<i>all</i>)	Función
kill (<i>allbut</i> (<i>symbol_1</i> , ..., <i>symbol_n</i>))	Función

Remueve todos los vínculos (valor, función, arreglo o regla) de los argumentos *symbol_1*, ..., *symbol_n*. Un argumento puede ser un simple arreglo de elementos o funciones con subíndices.

Varios argumentos especiales son reconocidos. Diferentes clases de argumentos pueden ser combinados, e.g., `kill (inlabels, functions, allbut (foo, bar))`.

`kill (labels)` elimina todas las etiquetas de entrada, salida y expresiones intermedias creadas hasta el momento.

`kill (inlabels)` elimina sólo las etiquetas de entrada las cuales empiezan con el valor de `inchar` en ese momento.

Igualmente, `kill (outlabels)` elimina sólo las etiquetas de salida las cuales empiezan con el valor de `outchar` de ese momento, y `kill (linelabels)` elimina sólo las etiquetas de expresiones intermedias las cuales empiezan con el valor de `linchar` de ese momento.

`kill (n)`, donde *n* es un entero, elimina las *n* etiquetas de entrada y salida más recientes.

`kill ([m, n]` elimina las etiquetas de entrada y salida *m* hasta *n*.

`kill (infolist)`, donde *infolist* es cualquier item de `infolists` (tales como `values`, `functions` o `arrays`), elimina todos los items en *infolist*. Véase también `infolists`.

`kill (all)` elimina todos los items de todas las listas de información (`infolists`). `kill (all)` no reconfigura las variables globales a sus valores por defecto; vea `reset` para más información sobre este punto.

`kill (allbut (symbol_1, ..., symbol_n))` elimina todos los items de todas las listas de información (`infolists`) excepto para *symbol_1*, ..., *symbol_n*.

`kill (allbut (infolist))` elimina todos los items excepto para los que aparezcan en *infolist*, donde *infolist* es `values`, `functions`, `arrays`, etc.

`kill` no evalúa sus argumentos. El operador de doble comilla simple, `' '`, rechaza

La función `kill` no evalúa sus argumentos. El operador de dobles comillas simples, `' '`, hace que los argumentos se evalúen.

La llamada `kill (symbol)` elimina todas las propiedades de *symbol*. En contraste, `remvalue`, `remfunction`, `remarray` y `remrule` elimina a propiedad en específico.

`kill` siempre retorna `done`, incluso si un argumento no fue eliminado.

`labels (symbol)`

Función

`labels`

Variable del sistema

Retorna la lista de etiquetas de entrada, salida o de expresiones intermedias las cuales empiezan con *symbol*. Típicamente *symbol* es el valor de las variables `inchar`, `outchar` o `linechar`. El carácter de etiqueta puede ser pasado con o sin signo de porcentaje, así, por ejemplo, `i` y `%i` producen el mismo resultado.

Si ninguna etiqueta empieza con *symbol*, `labels` retorna a una lista vacía.

La función `labels` no evalúa su argumento. Por ejemplo, `labels (' 'inchar)` retorna las etiquetas de entrada las cuales empiezan con el carácter de etiqueta de entrada actual.

La variable `labels` es una lista de las etiquetas de entrada, salida y expresiones intermedias, incluyendo todas las etiquetas anteriores en el caso de que `inchar`, `outchar` o `linechar` hayan sido redefinidas.

Por defecto, Maxima muestra el resultado de cada expresión introducida por el usuario, asignando al resultado una etiqueta de salida. La salida (es decir el resultado) puede ser suprimida terminando la expresión de entrada con un `$` (signo de dólar) en vez de un `;` (punto y coma). En este caso, se crea la etiqueta de salida y se le asigna el resultado, aunque éste no se muestre; aún así, la etiqueta puede ser referenciada de la misma forma que se hace con aquéllas cuyos resultados sí son mostrados.

Véanse también: `%`, `%%` y `%th`.

Las etiquetas de expresiones intermedias pueden ser generadas por algunas funciones. El interruptor `programmode` controla si `solve` y algunas otras funciones generan etiquetas de expresiones intermedias en vez de retornar una lista de expresiones. Algunas otras funciones, tales como `ldisplay`, siempre generan etiquetas de expresiones intermedias.

Véase también: `inchar`, `outchar`, `linechar` y `infolists`.

linenum

Variable del sistema

El número de la línea del par de expresiones de entrada y salida actuales.

myoptions

Variable del sistema

Valor por defecto: `[]`

`myoptions` es la lista de todas las opciones que nunca fueron reconfiguradas por el usuario, aunque éstas hayan sido reconfiguradas a su valor por defecto.

nolabels

Variable opcional

Valor por defecto: `false`

Cuando `nolabels` vale `true`, las etiquetas de entrada y salida (`%i` y `%o`, respectivamente) son mostradas, pero a éstas no se les asignan los resultados; además, las etiquetas no se incorporan a la lista `labels`. Puesto que a las etiquetas no se les asignan resultados, el colector de basura puede recuperar la memoria ocupada por los éstos.

En el caso contrario, a las etiquetas de entrada y salida se les asignan los resultados correspondientes y son añadidas a la lista `labels`.

Las etiquetas de expresiones intermedias (`%t`) no se ven afectadas por la variable `nolabels`; independientemente de que `nolabels` valga `true` o `false`, a las etiquetas de expresiones intermedias se les asignan siempre valores, además de ser añadidas a la lista `labels`.

Véanse también `batch`, `batchload` y `labels`.

optionset

Variable opcional

Valor por defecto: `false`

Cuando `optionset` tiene como valor `true`, Maxima imprime un mensaje cada vez que una opción de Maxima es reconfigurada. Esto es muy útil si el usuario duda con frecuencia de la correctitud de alguna opción y quiere estar seguro de la variable a la que él asignó un valor fue verdaderamente una variable opción (o interruptor).

playback ()	Función
playback (<i>n</i>)	Función
playback ([<i>m</i> , <i>n</i>])	Función
playback ([<i>m</i>])	Función
playback (<i>input</i>)	Función
playback (<i>slow</i>)	Función
playback (<i>time</i>)	Función
playback (<i>grind</i>)	Función

Muestra las entradas, salidas y expresiones intermedias sin recalcularlas. **playback** sólo muestra las expresiones asociadas con etiquetas; cualquier otra salida (tal como texto impreso por **print** o **describe**, o mensajes de error) no es mostrada. Véase también: **labels**.

playback no evalúa sus argumentos. **playback** siempre retorna **done**.

playback () (sin argumentos) muestra todas las entradas, salidas y expresiones intermedias generadas hasta el momento. Una expresión de salida es mostrada incluso si ésta fue suprimida por el carácter de terminación \$, cuando fue originalmente calculada.

playback (*n*) muestra las *n* expresiones más recientes. Cada entrada, salida y expresión intermedia cuenta como una.

playback ([*m*, *n*]) muestra entradas, salidas y expresiones intermedias con los números desde *m* hasta *n*, ambos inclusive.

playback ([*m*]) es equivalente a **playback** ([*m*, *m*]); esto usualmente imprime un par de expresiones de entrada y salida.

playback (*input*) muestra todas las expresiones de entrada generadas hasta el momento.

playback (*slow*) hace pausas entre expresiones y espera a que el usuario pulse la tecla **enter** para continuar. Esto es un comportamiento similar a **demo**.

playback (*slow*) es muy útil en conjunción con **save** o **stringout** cuando se crea un archivo secundario de almacenamiento con el objetivo de elegir cuidadosamente las expresiones realmente útiles.

playback (*time*) muestra el tiempo de computo por cada expresión.

playback (*grind*) muestra las expresiones de entrada en el mismo formato como la función **grind**. Las expresiones de salida no se ven afectadas por la opción **grind**. Vea **grind**.

Los argumentos pueden ser combinandos, e.g., **playback** ([5, 10], **grind**, **time**, **slow**).

printprops (<i>a</i> , <i>i</i>)	Función
printprops ([<i>a_1</i> , ..., <i>a_n</i>], <i>i</i>)	Función
printprops (<i>all</i> , <i>i</i>)	Función

Muestra la propiedad con el indicador *i* asociado con el átomo *a*. *a* pueden ser también una lista de átomos o el átomo **all** en cuyo caso todos los átomos a los cuales se les haya dado esa propiedad serán usados. Por ejemplo, **printprops** ([**f**, **g**], **atvalue**). **printprops** es para propiedades que no pueden ser mostradas de otra manera, i.e. para **atvalue**, **atomgrad**, **gradef**, y **matchdeclare**.

- prompt** Variable opcional
 Valor por defecto: `_`
`prompt` es el símbolo del prompt de la función `demo`, del modo `playback (slow)` y del bucle de interrupción de Maxima (el que se invoca con `break`).
- quit ()** Función
 Termina una sesión de Maxima. Nótese que la función debe ser invocada como `quit()`; o `quit()$`, no como `quit`.
 Para parar un cálculo muy demorado pulse **Control-C**. La acción por defecto es retornar a prompt de Maxima. Si `*debugger-hook*` tiene como valor `nil`, pulsar **Control-C** abrirá el depurador de Lisp. Vea también: `debugging`.
- remfunction (*f₁*, ..., *f_n*)** Función
remfunction (*all*) Función
 Desliga las definiciones de función de sus símbolos *f₁*, ..., *f_n*. Los argumentos pueden ser nombres de funciones ordinarias (creadas con `:=` o `define`) o de funciones macro (creadas con `::=`).
 La instrucción **remfunction** (*all*) desliga todas las definiciones de funciones.
 La función **remfunction** no evalúa sus argumentos.
 La función **remfunction** devuelve una lista con los símbolos para los que la definición de función fue desligada. Devuelve `false` en el lugar de cualquier símbolo para el que no hay función definida.
- reset ()** Función
 Reconfigura muchas variables y opciones globales y algunas otras variables a sus valores por defecto.
reset procesa las variables que se encuentran en la lista Lisp `*variable-initial-values*`. La macro Lisp `defmvar` pone las variables en ésta lista (entre otras acciones). Muchas, pero no todas, las variables y opciones globales son definidas por `defmvar`, y algunas variables definidas por `defmvar` no son ni variables ni opciones globales.
- showtime** Variable opcional
 Valor por defecto: `false`
 Cuando **showtime** tiene como valor `true`, el tiempo de cálculo y el tiempo de retardo se imprimen junto con la salida de cada expresión.
 El tiempo de cálculo se almacena siempre, de manera que `time` y `playback` puedan mostrar el tiempo de cálculo incluso cuando **showtime** vale `false`.
 Véase también `timer`.
- sstatus (*feature*, *package*)** Función
 Configura el estado de *feature* en *package*. Después de que **sstatus** (*feature*, *package*) es ejecutado **status** (*feature*, *package*) retorna `true`. Esto puede ser muy útil para escritores de paquetes, con el objetivo de conservar las pistas de que características han cargado estos.

to_lisp () Function
Entra en el intérprete Lisp bajo Maxima. (**to-maxima**) retorna de nuevo a Maxima.

values Variable del sistema
Valor inicial: []
values es una lista de todas las variables que el usuario ha creado (no constituye las opciones Maxima ni los interruptores). La lista comprende los símbolos : , ::, o :=.

5 Operadores

5.1 n-arios

Un operador de tipo **nary** (n-ario, en Maxima **nary**) es usado para denotar una función cuyo número de argumentos es arbitrario; los argumentos se separa por el símbolo del operador, como en $A+B$ o $A+B+C$. La función **nary**("x") es una extensión de la función **syntax** para declarar a x como un operador n-ario. Las funciones pueden ser declaradas para ser **nary**. Si se hace **declare(J,nary);**, se está diciendo al simplificador que haga la reducción de $j(j(a,b),j(c,d))$ a $j(a, b, c, d)$.

Funciones relacionadas: **syntax**.

5.2 no-fijos

Los operadores de tipo **nofix** (no-fijos, en Maxima **nofix**) son usados para denotar funciones que no reciben argumentos. La mera presencia de tal operador en un comando causará que la correspondiente función sea evaluada. Por ejemplo, cuando se escribe "exit;" para salir de una interrupción de Maxima, "exit" se está comportando de forma similar a un operador no-fijo (**nofix**). La función **nofix**("x") es una extensión de la función **syntax** la cual declara a x como un operador **nofix**.

Funciones relacionadas **syntax**.

5.3 operadores

Véase **operadores**.

5.4 postfijos

Un operador de tipo **postfix** (postfijo), al contrario de los de tipo **prefix**, denota funciones de un solo argumento, pero en este caso el argumento precede inmediatamente la ocurrencia del operador en la cadena de entrada, como en $3!$. La función **postfix**("x") es una extensión de la función **syntax** para declarar a x como un operador **postfix**.

Funciones relacionadas: **syntax**.

5.5 prefijos

Un operador de tipo **prefix** (prefijo, en Maxima **prefix**) indica una función de un (1) argumento, dicho argumento viene inmediatamente después de una ocurrencia del operador. La función **prefix**("x") es una extensión de la función **syntax** para declarar a x como un operador de **prefix**.

Funciones relacionadas: **syntax**.

5.6 Definiciones para Operadores

"!"

Operador

El operador factorial. Para cualquier número complejo x (incluyendo enteros, racionales y números reales) excepto para enteros negativos, $x!$ se define como $\text{gamma}(x+1)$.

Para un entero x , $x!$ simplifica el producto de los enteros desde 1 hasta x . $0!$ simplifica a 1. Para a un número de punto flotante x , $x!$ calcula al valor de $\text{gamma}(x+1)$. Para x igual a $n/2$ donde n es un entero impar, $x!$ simplifica a un factor racional por $\text{sqrt}(\pi)$ (donde $\text{gamma}(1/2)$ es igual a $\text{sqrt}(\pi)$). Si x es cualquier otra cosa, $x!$ no se simplifica.

Las variables `factlim`, `minfactorial` y `factcomb` controlan la simplificación de expresiones que contienen factoriales.

Las funciones `gamma`, `bffac` y `cbffac` son variaciones la función `gamma`. `makegamma` substituye a `gamma` para factoriales y funciones relacionadas.

Funciones relacionadas: `binomial`

- El factorial de un entero, semi-entero o de punto flotante es simplificado a menos que el operando sea mayor que `factlim`.

```
(%i1) factlim: 10$
(%i2) [0!, (7/2)!, 4.77!, 8!, 20!];
      105 sqrt(%pi)
(%o2) [1, -----, 81.44668037931193, 40320, 20!]
      16
```

- El factorial de un número complejo, constante conocida (por ejemplo `%e`) o una expresión general no es simplificado.

Sin embargo puede ser posible simplificar el factorial después evaluar el operando.

```
(%i1) [(%i + 1)!, %pi!, %e!, (cos(1) + sin(1))!];
(%o1) [(%i + 1)!, %pi!, %e!, (sin(1) + cos(1))!]
(%i2) ev (% , numer, %enumer);
(%o2) [(%i + 1)!, 7.188082728976031, 4.260820476357003,
                                           1.227580202486819]
```

- El factorial de un símbolo no se simplifica.

```
(%i1) kill (foo)$
(%i2) foo!;
(%o2)                                     foo!
```

- Los factoriales son simplificados no evaluados. Así $x!$ puede ser reemplazado en una expresión antecedida por el operador comilla.

```
(%i1) '([0!, (7/2)!, 4.77!, 8!, 20!]);
      105 sqrt(%pi)
(%o1) [1, -----, 81.44668037931193, 40320, 20!]
      16
```

"!!"

Operador

El operador doble factorial.

Para un número entero, de punto flotante o racional n , $n!!$ se evaluará como el producto de n $(n-2)$ $(n-4)$ $(n-6)$... $(n - 2(k-1))$ donde k es igual a $\text{entier}(n/2)$, que es, el mayor entero menor o igual a $n/2$. Note que esta definición no coincide con otras definiciones publicadas para argumentos, los cuales no son enteros.

Para un entero par (o impar) n , $n!$ se evalúa el producto de todos los enteros pares (o impares) consecutivos desde 2 (o 1) por *n*inclusive.

Para un argumento n el cual no es un número entero, punto flotante o racional, $n!!$ produce una forma de nombre `genfact (n, n/2, 2)`.

#

Operador

Representa la negación de la igualdad sintáctica $=$.

Nótese que debido a las reglas de evaluación de expresiones de tipo predicado (en concreto debido a que `not expr` obliga a la evaluación previa de `expr`), `not a = b` no será equivalente a `a # b` en algunos casos.

Ejemplos:

```
(%i1) a = b;
(%o1)                                     a = b
(%i2) is (a = b);
(%o2)                                     false
(%i3) a # b;
(%o3)                                     a # b
(%i4) not a = b;
(%o4)                                     true
(%i5) is (a # b);
(%o5)                                     true
(%i6) is (not a = b);
(%o6)                                     true
```

."

Operador

El operador punto, para multiplicación de matrices (no-conmutativo). Cuando "." se usa de esta forma, se dejarán espacios a ambos lados de éste, como en `A . B`. Así se evita que se confunda con el punto decimal de los números.

Véanse: `dot`, `dot0nscsimp`, `dot0simp`, `dot1simp`, `dotassoc`, `dotconstrules`, `dotdistrib`, `dotexptsimp`, `dotident` y `dotscrules`.

":

Operador

El operador de asignación. Por ejemplo, `A:3` se asigna a la variable `A` el valor de 3.

"::"

Operador

Operador de asignación. `::` asigna el valor de una expresión en su derecha a el valor de la cantidad en su izquierda, la cual debe evaluarse a una variable de tipo atómica o una variable subindicada.

":="

Operador

El operador de definición de macros `::=` define una función (llamada macro por razones históricas) que no evalúa sus argumentos, siendo la expresión que retorna


```

(%i6) ' '%;
(2) x is equal to %pi
(%o6)                                     %pi
macroexpand devuelve la macroexpansión; macroexpand (foo (x)) seguida de ' '%
es equivalente a foo (x) si foo es una función macro.
(%i1) x: %pi;
(%o1)                                     %pi
(%i2) y: 1234;
(%o2)                                     1234
(%i3) z: 1729 * w;
(%o3)                                     1729 w
(%i4) g (x) ::= buildq ([x], print ("x is equal to", x));
(%o4)      g(x) ::= buildq([x], print("x is equal to", x))
(%i5) macroexpand (g (y - z));
(%o5)      print(x is equal to, y - z)
(%i6) ' '%;
x is equal to 1234 - 1729 w
(%o6)      1234 - 1729 w
(%i7) g (y - z);
x is equal to 1234 - 1729 w
(%o7)      1234 - 1729 w

```

":=" operator
 El operador de definición de funciones. Por ejemplo, $f(x) := \sin(x)$ define una función f .

"=" Operador
 Denota una ecuación para Maxima. El patrón de emparejamiento en Maxima denota una relación total que se sostiene entre dos expresiones si y sólo si las expresiones son sintácticamente idénticas.
 La negación de $=$ se representa por $\#$. Nótese que debido a las reglas de evaluación de expresiones de tipo predicado (en concreto debido a que `not expr` obliga a la evaluación previa de `expr`), `not a = b` no será equivalente a `a # b` en algunos casos.

and Operador
 Operador de conjunción lógica. El operador `and` es un operador infijo n -ario; sus operandos son expresiones booleanas y su resultado es un valor lógico.
 El operador `and` impone la evaluación (igual que `is`) de uno o más operandos, y puede forzar la evaluación de todos los operandos.
 Los operandos se evalúan en el orden en el que aparecen; `and` sólo evalúa tantos operandos como sean necesarios para determinar el resultado. Si un operando vale `false`, el resultado es `false` y ya no se evalúan más operandos.
 La variable global `prederror` controla el comportamiento de `and` cuando la evaluación de un operando no da como resultado `true` o `false`; `and` imprime un mensaje de error cuando `prederror` vale `true`. En caso contrario, `and` devuelve `unknown`.
 El operador `and` no es conmutativo: `a and b` puede no ser igual a `b and a` debido al tratamiento de operandos indeterminados.

or Operador

Operador de disyunción lógica. El operador **or** es un operador infijo *n*-ario; sus operandos son expresiones booleanas y su resultado es un valor lógico.

El operador **or** impone la evaluación (igual que **is**) de uno o más operandos, y puede forzar la evaluación de todos los operandos.

Los operandos se evalúan en el orden en el que aparecen; **or** sólo evalúa tantos operandos como sean necesarios para determinar el resultado. Si un operando vale **true**, el resultado es **true** y ya no se evalúan más operandos.

La variable global **prederror** controla el comportamiento de **or** cuando la evaluación de un operando no da como resultado **true** o **false**; **or** imprime un mensaje de error cuando **prederror** vale **true**. En caso contrario, **and** devuelve **unknown**.

El operador **or** no es conmutativo: **a or b** puede no ser igual a **b or a** debido al tratamiento de operandos indeterminados.

not Operador

Operador de negación lógica. El operador **not** es un operador prefijo; su operando es una expresión booleana y su resultado es un valor lógico.

El operador **not** impone la evaluación (igual que **is**) de su operando.

La variable global **prederror** controla el comportamiento de **not** cuando la evaluación de su operando no da como resultado **true** o **false**; **not** imprime un mensaje de error cuando **prederror** vale **true**. En caso contrario, **not** devuelve **unknown**.

abs (expr) Función

Devuelve el valor absoluto de *expr*. Si la expresión es compleja, retorna el módulo de *expr*.

additive Clave

Si **declat**(*f*,**additive**) ha sido ejecutado, entonces:

(1) Si *f* es univariado, cada vez que el simplificador encuentre *f* aplicada a una suma, *f* será distribuida bajo esta suma. Por ejemplo, *f*(*x*+*y*) se simplificará a *f*(*x*)+*f*(*y*).

(2) Si *f* es una función de 2 o más argumentos, aditivamente es definida como aditiva en el primer argumento de *f*, como en el caso de **sum** o **integrate**. Por ejemplo, *f*(*h*(*x*)+*g*(*x*),*x*) se simplificará a *f*(*h*(*x*),*x*)+*f*(*g*(*x*),*x*). Esta simplificación no ocurre cuando *f* se aplica a expresiones de la forma **sum**(*x*[*i*],*i*,**lower-limit**,**upper-limit**).

allbut Clave

Opera con los comandos **part** (como **part**, **inpart**, **substpart**, **substinpart**, **dpart** y **lpart**). Por ejemplo:

```
(%i1) expr: e+d+c+b+a$
(%i2) part (expr, [2, 5]);
(%o2)                d + a
```

mientras que:

```
(%i3) part (expr, allbut (2, 5));
(%o3)          e + c + b
```

También trabaja con el comando `kill`,

```
kill (allbut (name_1, ..., name_k))
```

lo cual hará un `kill(all)` excepto a aquellos nombres especificados a `kill`, en este caso. Nota: `name_i` representa un nombre de función tal como `u`, `f`, `foo` o `g`, no una lista como `functions`.

antisymmetric

Declaración

Si `declare(h,antisymmetric)` es ejecutado, esto dice al simplificador que `h` es antisimétrico. E.g. `h(x,z,y)` será simplificado a `-h(x,y,z)`. Que es, el producto de $(-1)^n$ por el resultado dado por `symmetric` o `commutative`, donde n es el número de intercambios necesarios de dos argumentos para convertirle a esta forma.

cabs (*expr*)

Función

Devuelve el valor absoluto complejo (módulo complejo) de *expr*.

ceiling (*x*)

Función

Si *x* es un número real, devuelve el menor entero mayor o igual que *x*.

Si *x* es una expresión constante (por ejemplo, `10 * %pi`), `ceiling` evalúa *x* haciendo uso de números grandes en coma flotante (big floats), aplicando a continuación `ceiling` al número decimal obtenido. Puesto que `ceiling` hace evaluaciones en coma flotante, es posible, pero improbable, que esta función devuelva un valor erróneo para entradas constantes. Para evitar estos errores, la evaluación en punto flotante se lleva a cabo utilizando tres valores para `fpprec`.

Para argumentos no constantes, `ceiling` intenta devolver un valor simplificado. Aquí se presentan algunos ejemplos sobre las simplificaciones que `ceiling` es capaz de hacer:

```
(%i1) ceiling(ceiling(x));
(%o1) ceiling(x)
(%i2) ceiling(floor(x));
(%o2) floor(x)
(%i3) declare(n,integer)$
(%i4) [ceiling(n), ceiling(abs(n)), ceiling(max(n,6))];
(%o4) [n, abs(n), max(n,6)]
(%i5) assume(x > 0, x < 1)$
(%i6) ceiling(x);
(%o6) 1
(%i7) tex(ceiling(a));
$$\left \lceil a \right \rceil$$
```

La función `ceiling` no se extiende automáticamente a los elementos de listas y matrices. Por último, para todos los argumentos que tengan una forma compleja, `ceiling` devuelve una forma nominal.

Si el rango de una función es subconjunto de los números enteros, entonces puede ser declarada como `integervalued`. Tanto `ceiling` como `floor` son funciones que hacen uso de esta información; por ejemplo:

```
(%i1) declare(f,integervalued)$
(%i2) floor(f(x));
(%o2) f(x)
(%i3) ceiling(f(x) -1);
(%o3) f(x)-1
```

charfun (*p*)

Función

Devuelve 0 cuando el predicado *p* toma el valor **false**, y devuelve 1 cuando vale **true**. Si el predicado toma un valor diferente de **true** y **false** (desconocido), entonces devuelve una forma nominal.

Ejemplos:

```
(%i1) charfun(x<1);
(%o1) charfun(x<1)
(%i2) subst(x=-1,%);
(%o2) 1
(%i3) e : charfun('"and"(-1 < x, x < 1))$
(%i4) [subst(x=-1,e), subst(x=0,e), subst(x=1,e)];
(%o4) [0,1,0]
```

commutative

Declaración

Si `declare(h,commutative)` es ejecutado, le dice al simplificador que *h* es una función conmutativa. Por ejemplo, *h*(*x,z,y*) se simplificará a *h*(*x,y,z*). Esto es lo mismo que `symmetric`.

compare (*x, y*)

Función

Devuelve un operador de comparación *op* (<, <=, >, >=, = o #) de manera que `is (x op y)` tome el valor **true**; cuando tanto *x* como *y* dependan de *%i* y *x* # *y*, devuelve **notcomparable**; cuando no exista tal operador o Maxima sea incapaz de determinarlo, devolverá **unknown**.

Ejemplos:

```
(%i1) compare(1,2);
(%o1) <
(%i2) compare(1,x);
(%o2) unknown
(%i3) compare(%i,%i);
(%o3) =
(%i4) compare(%i,%i+1);
(%o4) notcomparable
(%i5) compare(1/x,0);
(%o5) #
(%i6) compare(x,abs(x));
(%o6) <=
```

La función `compare` no intenta determinar si los dominios reales de sus argumentos son conjuntos no vacíos; así,

```
(%i1) compare(acos(x^2+1), acos(x^2+1) + 1);
(%o1) <
```

Aquí, el dominio real de `acos (x2 + 1)` es el conjunto vacío.

entier (*x*)

Función

Devuelve el mayor entero menor o igual a *x*, siendo *x* numérico. La función **fix** (como en **fixnum**) es un sinónimo, de modo que **fix**(*x*) hace justamente lo mismo.

equal (*expr-1*, *expr-2*)

Función

Utilizado junto con **is**, devuelve **true** (o **false**) si y sólo si *expr-1* y *expr-2* son iguales (o no) para todos los posibles valores de sus variables (tal como los calcula **ratsimp**). Así, **is** (**equal** ((*x* + 1)², *x*² + 2**x* + 1)) devuelve **true**, mientras que si *x* no tiene valor asignado, **is** ((*x* + 1)² = *x*² + 2**x* + 1) devuelve **false**. Nótese también que **is**(**rat**(0)=0) da **false** pero **is** (**equal** (**rat**(0), 0)) da **true**.

Si algún cálculo no se puede hacer, **is** (**equal** (*a*, *b*)) devuelve una expresión simplificada y equivalente, mientras que **is** (*a*=*b*) siempre devuelve **true** o **false**.

Todas las variables presentes en *expr-1* y *expr-2* se suponen números reales.

La negación de **equal** es **notequal**. Nótese que debido a las reglas de evaluación de expresiones de tipo predicado (en concreto debido a que **not** *expr* obliga a la evaluación previa de *expr*), **notequal** no será equivalente a **not equal** en algunos casos.

La llamada **ev** (*expr*, *pred*) equivale a **is** (*expr*).

```
(%i1) is (x^2 >= 2*x - 1);
(%o1)                                     true
(%i2) assume (a > 1);
(%o2)                                     [a > 1]
(%i3) is (log (log (a+1) + 1) > 0 and a^2 + 1 > 2*a);
(%o3)                                     true
```

floor (*x*)

Función

Si *x* es un número real, devuelve el mayor entero menor o igual que *x*.

Si *x* es una expresión constante (por ejemplo, 10 * %pi), **floor** evalúa *x* haciendo uso de números grandes en coma flotante (big floats), aplicando a continuación **floor** al número decimal obtenido. Puesto que **floor** hace evaluaciones en coma flotante, es posible, pero improbable, que esta función devuelva un valor erróneo para entradas constantes. Para evitar estos errores, la evaluación en punto flotante se lleva a cabo utilizando tres valores para **fpprec**.

Para argumentos no constantes, **floor** intenta devolver un valor simplificado. Aquí se presentan algunos ejemplos sobre las simplificaciones que **floor** es capaz de hacer:

```
(%i1) floor(ceiling(x));
(%o1) ceiling(x)
(%i2) floor(floor(x));
(%o2) floor(x)
(%i3) declare(n,integer)$
(%i3) [floor(n), floor(abs(n)), floor(min(n,6))];
(%o4) [n,abs(n),min(n,6)]
(%i4) assume(x > 0, x < 1)$
(%i5) floor(x);
(%o5) 0
```

```
(%i6) tex(floor(a);
      $$\left \lfloor a \right \rfloor$$
```

La función `floor` no se extiende automáticamente a los elementos de listas y matrices. Por último, para todos los argumentos que tengan una forma compleja, `floor` devuelve una forma nominal.

Si el rango de una función es subconjunto de los números enteros, entonces puede ser declarada como `integervalued`. Tanto `ceiling` como `floor` son funciones que hacen uso de esta información; por ejemplo:

```
(%i1) declare(f,integervalued)$
(%i2) floor(f(x));
(%o2) f(x)
(%i3) ceiling(f(x) -1);
(%o3) f(x)-1
```

notequal (*expr-1*, *expr-2*)

Función

Representa la negación de `equal` (*expr-1*, *expr-2*).

Nótese que debido a las reglas de evaluación de expresiones de tipo predicado (en concreto debido a que `not expr` obliga a la evaluación previa de `expr`), `notequal` no será equivalente a `not equal` en algunos casos.

Ejemplos:

```
(%i1) equal (a, b);
(%o1) equal(a, b)
(%i2) maybe (equal (a, b));
(%o2) unknown
(%i3) notequal (a, b);
(%o3) notequal(a, b)
(%i4) not equal (a, b);
'macsyma' was unable to evaluate the predicate:
equal(a, b)
-- an error. Quitting. To debug this try debugmode(true);
(%i5) maybe (notequal (a, b));
(%o5) unknown
(%i6) maybe (not equal (a, b));
(%o6) unknown
(%i7) assume (a > b);
(%o7) [a > b]
(%i8) equal (a, b);
(%o8) equal(a, b)
(%i9) maybe (equal (a, b));
(%o9) false
(%i10) notequal (a, b);
(%o10) notequal(a, b)
(%i11) not equal (a, b);
(%o11) true
(%i12) maybe (notequal (a, b));
(%o12) true
(%i13) maybe (not equal (a, b));
(%o13) true
```

eval Operador
 El operador **eval** realiza una evaluación extra de una expresión *expr*. Véase **ev**.

evenp (*expr*) Función
 Devuelve **true** si *expr* es un entero par y **false** en cualquier otro caso.

fix (*x*) Función
 Es un sinónimo de **entier** (*x*).

fullmap (*f*, *expr_1*, ...) Función
 Similar a **map**, pero conservará el mapeado descendente de todas las subexpresiones hasta que los operadores principales ya no sean los mismos.

La función **fullmap** es utilizada por el simplificador de Maxima en algunas transformaciones matriciales, por lo que Maxima generará en algunas ocasiones mensajes de error relacionados con **fullmap** aunque el usuario no haya invocado explícitamente esta función.

```
(%i1) a + b*c$
(%i2) fullmap (g, %);
(%o2)          g(b) g(c) + g(a)
(%i3) map (g, %th(2));
(%o3)          g(b c) + g(a)
```

fullmapl (*f*, *list_1*, ...) Function
 Similar a **fullmap**, pero **fullmapl** sólo hace mapeo sobre listas y matrices.

```
(%i1) fullmapl ("+", [3, [4, 5]], [[a, 1], [0, -1.5]]);
(%o1)          [[a + 3, 4], [4, 3.5]]
```

is (*expr*) Función
 Intenta determinar si el predicado *expr* se puede deducir de los hechos almacenados en la base de datos gestionada por **assume**.

Si el predicado se reduce a **true** o **false**, **is** devuelve **true** o **false**, respectivamente. En otro caso, el valor devuelto está controlado por la variable global **prederror**. Si **prederror** vale **false**, **is** devuelve **unknown** si el predicado no se puede reducir a un valor de verdad lógica, retornando un mensaje de error si **prederror** vale **true**.

Véanse también **assume**, **facts** y **maybe**.

Ejemplos:

is hace evaluación de predicados.

```
(%i1) %pi > %e;
(%o1)          %pi > %e
(%i2) is (%pi > %e);
(%o2)          true
```

is evalúa predicados a partir de la base de datos de **assume**.

```
(%i1) assume (a > b);
(%o1) [a > b]
(%i2) assume (b > c);
(%o2) [b > c]
(%i3) is (a < b);
(%o3) false
(%i4) is (a > c);
(%o4) true
(%i5) is (equal (a, c));
(%o5) false
```

Si `is` no puede evaluar el valor lógico del predicado a partir de la base de datos gestionada por `assume`, la variable global `prederror` controla el comportamiento de `is`.

```
(%i1) assume (a > b);
(%o1) [a > b]
(%i2) prederror: true$
(%i3) is (a > 0);
'macsyma' was unable to evaluate the predicate:
a > 0
-- an error. Quitting. To debug this try debugmode(true);
(%i4) prederror: false$
(%i5) is (a > 0);
(%o5) unknown
```

maybe (*expr*)

Función

Intenta determinar si el predicado *expr* se puede deducir de los hechos almacenados en la base de datos gestionada por `assume`.

Si el predicado se reduce a `true` o `false`, `maybe` devuelve `true` o `false`, respectivamente. En otro caso, `maybe` devuelve `unknown`.

La función `maybe` es funcionalmente equivalente a `is` con `prederror: false`, pero el resultado se calcula sin asignar valor alguno a `prederror`.

Véanse también `assume`, `facts` y `is`.

Ejemplos:

```
(%i1) maybe (x > 0);
(%o1) unknown
(%i2) assume (x > 1);
(%o2) [x > 1]
(%i3) maybe (x > 0);
(%o3) true
```

isqrt (*x*)

Función

Devuelve la "raíz cuadrada entera" del valor absoluto de *x*, el cual debe ser un entero.

lmax (*L*)

Función

Si *L* es una lista o conjunto, devuelve `apply ('max, args (L))`. Si *L* no es una lista o conjunto, envía un mensaje de error.

lmin (*L*)

Función

Si *L* es una lista o conjunto, devuelve `apply ('min, args (L))`. Si *L* no es una lista o conjunto, envía un mensaje de error.

max (*x₁*, ..., *x_n*)

Función

Devuelve un valor simplificado de la mayor de las expresiones desde *x₁* hasta *x_n*. Si `get (trylevel, maxmin)` es 2 o más, `max` aplica la simplificación `max (e, -e) --> |e|`. Si `get (trylevel, maxmin)` es 3 o más, `max` intenta eliminar las expresiones que estén entre otros dos de los argumentos dados; por ejemplo, `max (x, 2*x, 3*x) --> max (x, 3*x)`. Para asignar el valor 2 a `trylevel` se puede hacer `put (trylevel, 2, maxmin)`.

min (*x₁*, ..., *x_n*)

Función

Devuelve un valor simplificado de la menor de las expresiones desde *x₁* hasta *x_n*. Si `get (trylevel, maxmin)` es 2 o más, `min` aplica la simplificación `min (e, -e) --> |e|`. Si `get (trylevel, maxmin)` es 3 o más, `min` intenta eliminar las expresiones que estén entre otros dos de los argumentos dados; por ejemplo, `min (x, 2*x, 3*x) --> min (x, 3*x)`. Para asignar el valor 2 a `trylevel` se puede hacer `put (trylevel, 2, maxmin)`.

polymod (*p*)

Función

polymod (*p*, *m*)

Función

Convierte el polinomio *p* a una representación modular respecto del módulo actual, que es el valor almacenado en la variable `modulus`.

La llamada `polymod (p, m)` especifica un módulo *m* para ser utilizado en lugar de valor almacenado en `modulus`.

Véase `modulus`.

mod (*x*, *y*)

Función

Si *x* e *y* son números reales e *y* es distinto de cero, devuelve `x - y * floor(x / y)`. Para todos los reales *x*, se tiene `mod (x, 0) = x`. Para información sobre la definición de `mod (x, 0) = x`, véase la sección 3.4 de "Concrete Mathematics", by Graham, Knuth, and Patashnik. La función `mod (x, 1)` es de diente de sierra con periodo unidad y con `mod (1, 1) = 0` y `mod (0, 1) = 0`.

Para encontrar el argumento principal (un número del intervalo $(-\pi, \pi]$) de un número complejo, hágase uso de la función `x |-> %pi - mod (%pi - x, 2*%pi)`, donde *x* es un argumento.

Si *x* e *y* son expresiones constantes (por ejemplo, `10 * %pi`), `mod` utiliza el mismo esquema de evaluación basado en números grandes en coma flotante (big floats) que `floor` y `ceiling`. También es posible, pero improbable, que `mod` pueda retornar un valor erróneo en tales casos.

Para argumentos no numéricos *x* o *y*, `mod` aplica algunas reglas de simplificación:

```
(%i1) mod(x,0);
(%o1) x
(%i2) mod(a*x,a*y);
```



```

(%o3)                                     768
(%i4) random (9573684);
(%o4)                                     7657880
(%i5) random (2^75);
(%o5)                                     11804491615036831636390
(%i6) s2: make_random_state (false)$
(%i7) random (1.0);
(%o7)                                     .2310127244107132
(%i8) random (10.0);
(%o8)                                     4.394553645870825
(%i9) random (100.0);
(%o9)                                     32.28666704056853
(%i10) set_random_state (s2);
(%o10)                                    done
(%i11) random (1.0);
(%o11)                                     .2310127244107132
(%i12) random (10.0);
(%o12)                                     4.394553645870825
(%i13) random (100.0);
(%o13)                                     32.28666704056853

```

rationalize (*expr*)

Función

Convierte todos los números en coma flotante de doble precisión y grandes (big float) presentes en una expresión *expr* de Maxima a a sus formas racionales exactas equivalentes. Si el usuario no está familiarizado con la representación binaria de números en coma flotante, le puede extrañar que **rationalize** (0.1) no sea igual que 1/10. Este comportamiento no es único de Maxima, ya que el número 1/10 en su forma binaria es periódico y no exacto.

```

(%i1) rationalize(0.5);
(%o1) 1/2
(%i2) rationalize(0.1);
(%o2) 3602879701896397/36028797018963968
(%i3) fpprec : 5$
(%i4) rationalize(0.1b0);
(%o4) 209715/2097152
(%i5) fpprec : 20$
(%i6) rationalize(0.1b0);
(%o6) 236118324143482260685/2361183241434822606848
(%i7) rationalize(sin(0.1 * x + 5.6));
(%o7) sin((3602879701896397*x)/36028797018963968+3152519739159347/562949953421.

```

Ejemplo de uso:

```

unitfrac(r) := block([uf : [], q],
  if not(ratnump(r)) then error("The input to 'unitfrac' must be a rational n
  while r # 0 do (
    uf : cons(q : 1/ceiling(1/r), uf),
    r : r - q),
  reverse(uf));

```

```
(%i2) unitfrac(9/10);
(%o2) [1/2,1/3,1/15]
(%i3) apply("+",%);
(%o3) 9/10
(%i4) unitfrac(-9/10);
(%o4) [-1,1/10]
(%i5) apply("+",%);
(%o5) -9/10
(%i6) unitfrac(36/37);
(%o6) [1/2,1/3,1/8,1/69,1/6808]
(%i7) apply("+",%);
(%o7) 36/37
```

sign (*expr*) Función
 Intenta determinar el signo de *expr* en base a los hechos almacenados en la base de datos. Devuelve una de las siguientes respuestas: **pos** (positivo), **neg** (negativo), **zero** (cero), **pz** (positivo o cero), **nz** (negativo o cero), **pn** (positivo o negativo), o **pnz** (positivo, negativo o cero, lo que significa que el signo es desconocido).

signum (*x*) Función
 Para *x* numérico, devuelve 0 si *x* es 0, en caso contrario devuelve -1 o +1, según que *x* sea menor o mayor que 0, respectivamente.
 Si *x* no es numérico, entonces se devuelve una forma simplificada equivalente. Por ejemplo, **signum**(-*x*) devuelve -**signum**(*x*).

sort (*list*, *p*) Función
sort (*list*) Función
 Ordena *list* de acuerdo con el predicado *p* de dos argumentos, como "<" o **orderlessp**.
 La llamada **sort** (*list*) ordena *list* de acuerdo con la ordenación preestablecida en Maxima.
 El argumento *list* puede contener elementos numéricos o no numéricos o de de ambos tipos.

sqrt (*x*) Función
 Raíz cuadrada de *x*. Se representa internamente por $x^{(1/2)}$. Véase también **rootscontract**.
 Si la variable **radexpand** vale **true** hará que las raíces *n*-ésimas de los factores de un producto que sean potencias de *n* se extraídas del radical; por ejemplo, **sqrt**(16*x^2) se convertirá en 4*x sólo si **radexpand** vale **true**.

sqrtdispflag Variable opcional
 Valor por defecto: **true**
 Si **sqrtdispflag** vale **false**, hará que **sqrt** se muestre con el exponente 1/2.

sublis (*list*, *expr*) Función

Realiza sustituciones múltiples en paralelo en una expresión.

La variable `sublis_apply_lambda` controla la simplificación después de `sublis`.

Ejemplo:

```
(%i1) sublis ([a=b, b=a], sin(a) + cos(b));
(%o1) sin(b) + cos(a)
```

sublist (*list*, *p*) Función

Devuelve la lista de elementos de *list* para los cuales el predicado *p* retorna `true`.

Ejemplo:

```
(%i1) L: [1, 2, 3, 4, 5, 6]$
(%i2) sublist (L, evenp);
(%o2) [2, 4, 6]
```

sublis_apply_lambda Variable opcional

Valor por defecto: `true`

Controla si los `lambda` sustituidos son aplicados en la simplificación después de invocar a `sublis`, o si se tiene que hacer un `ev` para hacerlo. Si `sublis_apply_lambda` vale `true`, significa que se ejecute la aplicación.

subst (*a*, *b*, *c*) Función

Sustituye *a* por *b* en *c*. El argumento *b* debe ser un átomo o una subexpresión completa de *c*. Por ejemplo, `x+y+z` es una subexpresión completa de `2*(x+y+z)/w` mientras que `x+y` no lo es. Cuando *b* no cumple esta característica, se puede utilizar en algunos casos `substpart` o `ratsubst` (ver más abajo). Alternativamente, si *b* no es de la forma `e/f` entonces se puede usar `subst (a*f, e, c)`, pero si *b* es de la forma `e^(1/f)` se debe usar `subst (a^f, e, c)`. La instrucción `subst` también reconoce `x^y` en `x^-y`, de manera que `subst (a, sqrt(x), 1/sqrt(x))` da `1/a`. Los argumentos *a* y *b* también pueden ser operadores de una expresión acotados por comillas dobles " o nombres de funciones. Si se quiere sustituir la variable independiente en expresiones con derivadas se debe utilizar la función `at` (ver más abajo).

La función `subst` es sinónimo de `substitute`.

La llamada `subst (eq_1, expr)` o `subst ([eq_1, ..., eq_k], expr)` están permitidas. Las `eq_i` son ecuaciones que indican las sustituciones a realizar. Para cada ecuación, el miembro derecho será sustituida por la expresión del miembro izquierdo en *expr*.

Si la variable `exptsubst` vale `true` se permiten ciertas sustituciones de exponentes; por ejemplo, sustituir `y` por `%e^x` en `%e^(a*x)`.

Si `opsubst` vale `false`, `subst` no intentará sustituir un operador de una expresión. Por ejemplo, `(opsubst: false, subst (x^2, r, r+r[0]))` trabajará sin problemas.

Ejemplos:

```
(%i1) subst (a, x+y, x + (x+y)^2 + y);
(%o1) y + x + a
```

```
(%i2) subst (-%i, %i, a + b*%i);
(%o2)          a - %i b
```

Para más ejemplos, ejecútese `example (subst)`.

substinpart (*x, expr, n_1, ..., n_k*)

Función

Es similar a `substpart`, pero `substinpart` trabaja con la representación interna de `expr`.

```
(%i1) x . 'diff (f(x), x, 2);
          2
          D
(%o1)      x . ---- (f(x))
          2
          dx
(%i2) substinpart (d^2, %, 2);
          2
(%o2)      x . d
(%i3) substinpart (f1, f[1](x+1), 0);
(%o3)      f1(x + 1)
```

Si el último argumento de la función `part` es una lista de índices, entonces se toman varias subexpresiones, cada una de las cuales en correspondencia con un índice de la lista. Así,

```
(%i1) part (x+y+z, [1, 3]);
(%o1)      z + x
```

`piece` guarda el valor de la última expresión seleccionada cada vez que se utiliza la función `part`. Esta asignación se hace durante la ejecución de la función, con lo que puede ser referenciada en la propia función tal como se muestra más abajo. Si `partswitch` vale `true` entonces se devuelve `end` cuando la parte seleccionada de una expresión no existe, en caso contrario se muestra un mensaje de error.

```
(%i1) expr: 27*y^3 + 54*x*y^2 + 36*x^2*y + y + 8*x^3 + x + 1;
          3      2      2      3
(%o1)      27 y  + 54 x y  + 36 x  y + y + 8 x  + x + 1
(%i2) part (expr, 2, [1, 3]);
          2
(%o2)      54 y
(%i3) sqrt (piece/54);
(%o3)      abs(y)
(%i4) substpart (factor (piece), expr, [1, 2, 3, 5]);
          3
(%o4)      (3 y + 2 x)  + y + x + 1
(%i5) expr: 1/x + y/x - 1/z;
          1      y      1
(%o5)      - - + - + -
          z      x      x
(%i6) substpart (xthru (piece), expr, [2, 3]);
          y + 1      1
(%o6)      ----- - -
          x          z
```

Además, darle a la opción `inflag` el valor `true` y llamar a `part` o a `substpart` es lo mismo que llamar a `inpart` o a `substinpart`.

substpart (*x*, *expr*, *n_1*, ..., *n_k*) Función

Sustituye *x* por la subexpresión determinada por el resto de argumentos, según el esquema de `part`. Devuelve el nuevo valor de *expr*. El argumento *x* puede ser un operador a ser sustituido por un operador de *expr*. En algunos casos *x* necesita estar acotado por comillas dobles `"`, como en `substpart ("+", a*b, 0)` para que retorne `b + a`.

```
(%i1) 1/(x^2 + 2);
(%o1)
      1
-----
      2
      x  + 2
(%i2) substpart (3/2, %, 2, 1, 2);
(%o2)
      1
-----
      3/2
      x  + 2
(%i3) a*x + f (b, y);
(%o3)
      a x + f(b, y)
(%i4) substpart ("+", %, 1, 0);
(%o4)
      x + f(b, y) + a
```

Además, darle a la opción `inflag` el valor `true` y llamar a `part` o a `substpart` es lo mismo que llamar a `inpart` o a `substinpart`.

subvarp (*expr*) Función

Devuelve `true` si *expr* es una variable subindicada, como `a[i]`.

symbolp (*expr*) Función

Devuelve `true` si *expr* es un símbolo y `false` en caso contrario. La llamada `symbolp(x)` equivale al predicado `atom(x)` and not `numberp(x)`.

Véase también `Identifiers`.

unorder () Función

Desactiva las asociaciones creadas por la última utilización de los comandos de ordenación `ordergreat` y `orderless`, los cuales no pueden ser utilizados más de una vez sin invocar a `unorder`. Véase también `ordergreat` y `orderless`.

```
(%i1) unorder();
(%o1)
[]
(%i2) b*x + a^2;
(%o2)
      2
      b x + a
(%i3) ordergreat (a);
(%o3)
done
(%i4) b*x + a^2;
(%o4)
      2
```

```
(%o4)          a  + b x
(%i5) %th(1) - %th(3);
              2    2
(%o5)          a  - a
(%i6) unordered();
(%o6)          [a]
```

vectorpotential (*gencurl*)

Función

Devuelve el vector potencial de un vector rotacional en el sistema de coordenadas actual. **potentialzeroloc** tiene un rol similar al de **potential**, pero el orden del miembro izquierdo de las ecuaciones debe ser una permutación cíclica de las coordenadas.

xthru (*expr*)

Función

Combina todos los términos de *expr* (la cual debe ser una suma) sobre un común denominador sin expandir productos ni sumas elevadas a exponentes al modo que lo hace **ratsimp**. La función **xthru** cancela factores comunes en el numerador y denominador de expresiones racionales, pero sólo si los factores son explícitos.

En ocasiones puede ser útil el uso de **xthru** antes de la llamada a **ratsimp** a fin de cancelar factores explícitos del máximo común divisor del numerador y denominador y así simplificar la expresión a la que se va a aplicar **ratsimp**.

```
(%i1) ((x+2)^20 - 2*y)/(x+y)^20 + (x+y)^(-19) - x/(x+y)^20;
              20
              1      (x + 2)  - 2 y      x
(%o1)  ----- + ----- - -----
              19      20      20
      (y + x)  (y + x)  (y + x)
(%i2) xthru (%);
              20
      (x + 2)  - y
(%o2)  -----
              20
      (y + x)
```

zeroequiv (*expr*, *v*)

Función

Analiza si la expresión *expr* de variable *v* equivale a cero, devolviendo **true**, **false** o **dontknow**.

La función **zeroequiv** tiene estas restricciones:

1. No utilizar funciones que Maxima no sepa derivar y evaluar.
2. Si la expresión tiene polos en la recta real, pueden aparecer errores en el resultado, aunque es poco probable.
3. Si la expresión contiene funciones que no son soluciones de ecuaciones diferenciales ordinarias de primer orden (como las funciones de Bessel) pueden presentarse resultados incorrectos.
4. El algoritmo utiliza evaluaciones en puntos aleatoriamente seleccionados. Esto conlleva un riesgo, aunque el algoritmo intenta minimizar el error.

Por ejemplo, `zeroequiv (sin(2*x) - 2*sin(x)*cos(x), x)` devuelve `true` y `zeroequiv (%e^x + x, x)` devuelve `false`. Por otro lado `zeroequiv (log(a*b) - log(a) - log(b), a)` devuelve `dontknow` debido a la presencia del parámetro `b`.

6 Expresiones

6.1 Introducción a las Expresiones

Existe un cierto número de palabras reservadas que no pueden utilizarse como nombres de variables. Su uso podría causar errores críticos de sintaxis.

integrate	next	from	diff
in	at	limit	sum
for	and	elseif	then
else	do	or	if
unless	product	while	thru
step			

La mayoría de los objetos en Maxima son expresiones. Una secuencia de expresiones puede constituir una expresión, separándolas por comas y colocando paréntesis alrededor de ellas. Esto es similar a las *expresiones con coma* en **C**.

```
(%i1) x: 3$
(%i2) (x: x+1, x: x^2);
(%o2)                                     16
(%i3) (if (x > 17) then 2 else 4);
(%o3)                                     4
(%i4) (if (x > 17) then x: 2 else y: 4, y+x);
(%o4)                                     20
```

Incluso los bucles en Maxima son expresiones, aunque el valor que retornan (**done**) no es muy útil.

```
(%i1) y: (x: 1, for i from 1 thru 10 do (x: x*i))$
(%i2) y;
(%o2)                                     done
```

pero quizás se quiera incluir un tercer término en la *expresión con coma* para que devuelva el valor de interés.

```
(%i3) y: (x: 1, for i from 1 thru 10 do (x: x*i), x)$
(%i4) y;
(%o4)                                     3628800
```

6.2 Asignación

Existen dos operadores de asignación en Maxima, **:** y **::**. Por ejemplo, **a: 3** asigna a la variable **a** el valor de 3. Por otro lado, **::** asigna el valor de la expresión de su derecha al valor de la cantidad de su izquierda, el cual debe evaluarse a una variable de tipo átomo o una variable subindicada.

6.3 Expresiones complejas

Una expresión compleja se especifica en Maxima añadiendo a la parte real de la expresión el producto de **%i** por la parte imaginaria. Así las raíces de la ecuación $x^2 - 4x + 13 = 0$ son $2 + 3\%i$ y $2 - 3\%i$. Nótese que la simplificación de productos de expresiones complejas

puede ser efectuado expandiendo el producto. La simplificación de cocientes, raíces y otras funciones que contengan expresiones complejas pueden normalmente conseguirse a través de las funciones `realpart`, `imagpart`, `rectform`, `polarform`, `abs` o `carg`.

6.4 Nombres y verbos

Maxima distingue entre operadores que son "nombres" y operadores que son "verbos". Un verbo es un operador que puede ser ejecutado. Un nombre es un operador que aparece como un símbolo en una expresión pero sin ser ejecutado. Por defecto, los nombres de funciones son verbos. Un verbo puede transformarse en nombre utilizando el apóstrofo o aplicando la función `nounify`. Un nombre puede transformarse en verbo aplicando la función `verbify`. La variable `nouns` hace que `ev` evalúe los nombres presentes en una expresión.

La forma verbal se distingue mediante la precedencia del carácter dólar \$ al correspondiente símbolo de Lisp. Por otro lado, la forma nominal se distingue mediante la precedencia del carácter porcentaje % al correspondiente símbolo de Lisp. Algunos nombres gozan de propiedades especiales para su representación, como `'integrate` o `'derivative` (devuelto por `diff`), pero la mayoría no. Por defecto, las formas nominal y verbal de una función son idénticas cuando se muestran en un terminal. La variable global `noundisp` hace que Maxima muestre los nombres precedidos del apóstrofo `'`.

Véanse también `noun`, `nouns`, `nounify` y `verbify`.

Ejemplos:

```
(%i1) foo (x) := x^2;
                                2
(%o1)          foo(x) := x
(%i2) foo (42);
(%o2)          1764
(%i3) 'foo (42);
(%o3)          foo(42)
(%i4) 'foo (42), nouns;
(%o4)          1764
(%i5) declare (bar, noun);
(%o5)          done
(%i6) bar (x) := x/17;
                                x
(%o6)          ''bar(x) := --
                                17
(%i7) bar (52);
(%o7)          bar(52)
(%i8) bar (52), nouns;
                                52
(%o8)          --
                                17
(%i9) integrate (1/x, x, 1, 42);
(%o9)          log(42)
(%i10) 'integrate (1/x, x, 1, 42);
                                42
```

```

                                /
                                [ 1
(%o10)                        I  - dx
                                ]  x
                                /
                                1
(%i11) ev (% , nouns);
(%o11)                        log(42)

```

6.5 Identificadores

En Maxima, los identificadores pueden contener caracteres alfabéticos, números del 0 al 9 y cualquier otro carácter precedido de la barra invertida \.

Un identificador puede comenzar con un carácter numérico si éste va precedido de la barra invertida \. Los caracteres numéricos que ocupen la segunda posición o posterior no necesitan ir precedidos de la barra invertida.

Un carácter especial puede declararse alfabético con la función **declare**. Así declarado, no necesita ir precedido de la barra invertida en un identificador. En principio, los caracteres alfabéticos son las letras de A a Z y a a z, junto con % y _.

Maxima distingue minúsculas y mayúsculas. Los identificadores `foo`, `F00` y `Foo` son distintos. Véase [\[Lisp y Maxima\]](#), página [\[Lisp y Maxima\]](#) para más información.

Un identificador en Maxima es un símbolo Lisp que comienza con el símbolo dólar \$. Cualquier otro símbolo de Lisp va precedido de la interrogación ? cuando aparece en Maxima. Véase [\[Lisp y Maxima\]](#), página [\[Lisp y Maxima\]](#) para más información.

Ejemplos:

```

(%i1) %an_ordinary_identifier42;
(%o1) %an_ordinary_identifier42
(%i2) embedded\ spaces\ in\ an\ identifier;
(%o2) embedded spaces in an identifier
(%i3) symbolp (%);
(%o3) true
(%i4) [foo+bar, foo\+bar];
(%o4) [foo + bar, foo+bar]
(%i5) [1729, \1729];
(%o5) [1729, 1729]
(%i6) [symbolp (foo\+bar), symbolp (\1729)];
(%o6) [true, true]
(%i7) [is (foo\+bar = foo+bar), is (\1729 = 1729)];
(%o7) [false, false]
(%i8) baz~quux;
(%o8) baz~quux
(%i9) declare ("~", alphabetic);
(%o9) done
(%i10) baz~quux;
(%o10) baz~quux
(%i11) [is (foo = F00), is (F00 = Foo), is (Foo = foo)];
(%o11) [false, false, false]

```

```
(%i12) :lisp (defvar *my-lisp-variable* '$foo)
*MY-LISP-VARIABLE*
(%i12) ?\*my\-lisp\-variable\*;
(%o12)                                     foo
```

6.6 Desigualdades

Maxima dispone de los operadores de desigualdad `<`, `<=`, `>=`, `>`, `#` y `notequal`. Véase `if` para una descripción de las expresiones condicionales.

6.7 Sintaxis

Es posible definir nuevos operadores con una precedencia especificada, o eliminar o redefinir la precedencia de operadores ya existentes. Un operador puede ser de tipo prefijo unario o postfijo unario, infijo binario, infijo *n*-ario, "bi-fijo" (`matchfix`) o "no-fijo"; "bi-fijo" se refiere a un par de símbolos que encierran su o sus argumentos, y "no-fijo" es un operador que no necesita argumentos. A continuación ejemplos sobre los diferentes tipos de operadores.

prefijo unario

negación `- a`

postfijo unario

factorial `a!`

infijo binario

exponenciación `a^b`

n-ary infix suma `a + b`

"bi-fijo" construcción de una lista `[a, b]`

(Maxima no incluye operadores "no-fijos", pero se puede ver un ejemplo en `nofix`.)

El mecanismo para definir un nuevo operador es sencillo. Tan solo es necesario declarar una función como operador; la función operador puede estar definida o no.

Un ejemplo de operador definido por el usuario es el siguiente. Nótese que la llamada a función `"dd" (a)` equivale a `dd a`, de igual manera que `"<-" (a, b)` también equivale a `a <- b`. Nótese también que las funciones `"dd"` y `"<-"` no están definidas en este ejemplo.

```
(%i1) prefix ("dd");
(%o1)                                     dd
(%i2) dd a;
(%o2)                                     dd a
(%i3) "dd" (a);
(%o3)                                     dd a
(%i4) infix ("<-");
(%o4)                                     <-
(%i5) a <- dd b;
(%o5)                                     a <- dd b
(%i6) "<-" (a, "dd" (b));
(%o6)                                     a <- dd b
```

Las funciones de Maxima que definen nuevos operadores se resumen en esta tabla, en la que se establecen las fuerzas de enlace a izquierda (lbp, de *left binding power*) y a derecha (rbp, de *right binding power*) por defecto. (La fuerza de enlace determina la precedencia del operador. Sin embargo, puesto que las fuerzas de enlace a izquierda y derecha pueden ser diferentes, la fuerza de enlace es algo más que la simple precedencia.) Algunas de las funciones para definir operadores toman argumentos adicionales; véanse las descripciones de estas funciones para más detalles.

```

prefix      rbp=180
postfix     lbp=180
infix       lbp=180, rbp=180
nary        lbp=180, rbp=180
matchfix    (la fuerza de enlace no se aplica aquí)
nofix       (la fuerza de enlace no se aplica aquí)

```

A efectos comparativos, aquí se presentan algunos operadores de Maxima junto con sus fuerzas de enlace a izquierda y derecha.

Operator	lbp	rbp
:	180	20
::	180	20
:=	180	20
::=	180	20
!	160	
!!	160	
^	140	139
.	130	129
*	120	
/	120	120
+	100	100
-	100	134
=	80	80
#	80	80
>	80	80
>=	80	80
<	80	80
<=	80	80
not		70
and	65	
or	60	
,	10	
\$	-1	
;	-1	

Las funciones `remove` y `kill` eliminan propiedades de operadores de un átomo. La llamada `remove ("a", op)` sólo elimina las propiedades de operador de `a`. La llamada `kill ("a")` elimina todas las propiedades de `a`, incluidas las propiedades de operador. Nótese que el nombre del operador debe ir entre comillas.

```

(%i1) infix ("@");
(%o1)                                     @
(%i2) "@ (a, b) := a^b;
                                     b
(%o2)                                a @ b := a
(%i3) 5 @ 3;
(%o3)                                125
(%i4) remove ("@", op);
(%o4)                                done
(%i5) 5 @ 3;
Incorrect syntax: @ is not an infix operator
5 @
~
(%i5) "@ (5, 3);
(%o5)                                125
(%i6) infix ("@");
(%o6)                                @
(%i7) 5 @ 3;
(%o7)                                125
(%i8) kill ("@");
(%o8)                                done
(%i9) 5 @ 3;
Incorrect syntax: @ is not an infix operator
5 @
~
(%i9) "@ (5, 3);
(%o9)                                @(5, 3)

```

6.8 Definiciones para Expresiones

at (*expr*, [*eqn_1*, ..., *eqn_n*]) Función

at (*expr*, *eqn*) Función

Evalúa la expresión *expr* asignando a las variables los valores especificados para ellas en la lista de ecuaciones [*eqn_1*, ..., *eqn_n*] o en la ecuación simple *eqn*.

Si una subexpresión depende de cualquiera de las variables para la cual se especifica un valor, pero no puede ser evaluado, entonces **at** devuelve una forma nominal.

La función **at** realiza múltiples sustituciones en serie, no en paralelo.

Véase también **atvalue**. Para otras funciones que también llevan a cabo sustituciones, consúltense **subst** y **ev**.

Ejemplos:

```

(%i1) atvalue (f(x,y), [x = 0, y = 1], a^2);
                                     2
(%o1)                                a
(%i2) atvalue ('diff (f(x,y), x), x = 0, 1 + y);
                                     @2 + 1
(%o2)                                @2 + 1
(%i3) printprops (all, atvalue);
                                     !

```

```

      d
      --- (f(@1, @2))!      = @2 + 1
      d@1                    !
                          !@1 = 0

                                2
      f(0, 1) = a

(%o3)                                done
(%i4) diff (4*f(x, y)^2 - u(x, y)^2, x);
      d
      8 f(x, y) (--- (f(x, y))) - 2 u(x, y) (--- (u(x, y)))
      dx                      dx
(%i5) at (% , [x = 0, y = 1]);
                                !
                                !
      2      d
      16 a  - 2 u(0, 1) (--- (u(x, y)))!
      dx                      !
                                !x = 0, y = 1

```

box (*expr*) Función
box (*expr*, *a*) Función

Devuelve *expr* encerrada en una caja. El valor devuelto es una expresión con **box** como operador y *expr* como argumento. Se dibujará una caja cuando `display2d` valga `true`.

La llamada **box** (*expr*, *a*) encierra *expr* en una caja etiquetada con el símbolo *a*. La etiqueta se recorta si es más larga que el ancho de la caja.

La expresión encerrada no se evalúa, siendo excluida de los cálculos.

La variable **boxchar** guarda el carácter a utilizar para dibujar la caja en las funciones **box**, **dpart** y **lpart**.

Ejemplos:

```

(%i1) box (a^2 + b^2);
      " 2 2 "
(%o1)  "b  + a "
      " 2 2 "
(%i2) box (a^2 + b^2, term_1);
      term_1""
      " 2 2 "
(%o2)  "b  + a "
      " 2 2 "
(%i3) 1729 - box (1729);
      " 1729 "
(%o3) 1729 - "1729"
      " 1729 "
(%i4) boxchar: "-";
(%o4) -

```



```
(%i5) box (sin(x) + cos(y));
-----
(%o5)      -COS(y) + SIN(x)-
-----
(%i6)
```

boxchar

Variable opcional

Valor por defecto: "

La variable **boxchar** guarda el carácter a utilizar para dibujar la caja en las funciones **box**, **dpart** y **lpart**.

Todas las cajas en una expresión se dibujan con el valor actual de **boxchar**, carácter que no se almacena con las expresión encerrada.

carg (*z*)

Función

Devuelve el argumento complejo de *z*. El argumento complejo es un ángulo **theta** en $(-\pi, \pi]$ tal que $r \exp(i\theta) = z$ donde *r* es la magnitud de *z*.

La función **carg** es computacional, no simplificativa.

La función **carg** ignora la declaración **declare** (*x*, **complex**), y trata a *x* como una variable real. Se trata de un fallo conocido en Maxima.

Véanse también **abs** (módulo complejo), **polarform**, **rectform**, **realpart** y **imagpart**.

Ejemplos:

```
(%i1) carg (1);
(%o1)      0
(%i2) carg (1 + %i);
(%o2)      %pi
-----
          4
(%i3) carg (exp (%i));
(%o3)      1
(%i4) carg (exp (%pi * %i));
(%o4)      %pi
(%i5) carg (exp (3/2 * %pi * %i));
(%o5)      %pi
-----
          2
(%i6) carg (17 * exp (2 * %i));
(%o6)      2
```

constant

Operator especial

La llamada **declare** (*a*, **constant**) declara *a* como constante. Véase **declare**.

constantp (*expr*)

Función

Devuelve **true** si *expr* es una expresión constante y **false** en caso contrario.

Una expresión se considera constante si sus argumentos son números (incluidos los números racionales que se muestran con **/R/**), constantes simbólicas como **%pi**, **%e** o

`%i`, variables con valor constante o declarada como constante por `declare`, o funciones cuyos argumentos son constantes.

La función `constantp` evalúa sus argumentos.

Ejemplos:

```
(%i1) constantp (7 * sin(2));
(%o1)      TRUE
(%i2) constantp (rat (17/29));
(%o2)      TRUE
(%i3) constantp (%pi * sin(%e));
(%o3)      TRUE
(%i4) constantp (exp (x));
(%o4)      FALSE
(%i5) declare (x, constant);
(%o5)      DONE
(%i6) constantp (exp (x));
(%o6)      TRUE
(%i7) constantp (foo (x) + bar (%e) + baz (2));
(%o7)      FALSE
(%i8)
```

declare (*a₁*, *f₁*, *a₂*, *f₂*, ...)

Función

Asigna al átomo *a_i* la característica *f_i*. Las *a_i* y las *f_i* también pueden ser listas de átomos y características, respectivamente, en cuyo caso cada átomo toma todas las características.

La función `declare` no evalúa sus argumentos y devuelve siempre `done`.

Las posibles características y sus significados son:

`constant` convierte a *a_i* en una constante, como `%pi`.

`mainvar` convierte a *a_i* en `mainvar`. Ordenación de los átomos: números < constantes (como `%e`, `%pi`) < escalares < otras variables < `mainvars`.

`scalar` convierte a *a_i* en escalar.

`nonscalar` hace que *a_i* se comporte como una lista o matriz ante el operador punto.

`noun` convierte la función *a_i* en un nombre, de manera que no se evalúe automáticamente.

`evfun` hace que *a_i* sea reconocida por la función `ev` de manera que se aplique cuando se mencione su nombre. Véase `evfun`.

`evflag` hace que *a_i* sea reconocida por la función `ev` de manera que tome el valor `true` durante la ejecución de `ev` si es mencionada. Véase `evflag`.

`bindtest` hace que *a_i* envíe un error si en algún momento es utilizada en un cálculo sin habersele asignado un valor.

Maxima reconoce y utiliza las siguientes propiedades de objetos:

```
even, odd, integer, rational, irrational, real, imaginary,
and complex
```

Propiedades útiles para funciones son:

increasing,
 decreasing, oddfun (odd function), evenfun (even function),
 commutative (or symmetric), antisymmetric, lassociative and
 rassociative

La llamada `featurep (object, feature)` determina si *object* ha sido declarado con la propiedad *feature*.

Véase también `features`.

disolate (*expr*, *x_1*, ..., *x_n*)

Función

Es similar a `isolate (expr, x)`, excepto que permite al usuario aislar más de una variable simultáneamente. Puede ser útil para hacer un cambio de variables en integrales múltiples en las que tales variables dependan de de dos o más variables de integración. Esta función se carga automáticamente desde `'simplification/disol.mac'`. Se dispone de una demostración en `demo("disol")$`.

dispform (*expr*)

Función

Devuelve la representación externa de *expr* con respecto a su operador principal. Es útil cuando se usa conjuntamente con `part`, que también trata de la representación externa. Si póngase que *expr* es $-A$. Entonces la representación interna de *expr* es `"*(-1,A)`, mientras que la representación externa es `"-(A)`. La llamada `dispform (expr, all)` convierte la expresión entera al formato externo. Por ejemplo, si `expr: sin (sqrt (x))`, entonces `freeof (sqrt, expr)` y `freeof (sqrt, dispform (expr))` dan `true`, mientras que `freeof (sqrt, dispform (expr, all))` da `false`.

distrib (*expr*)

Función

Diistribuye sumas sobre productos. Difiere de `expand` en que trabaja sólo al nivel superior de una expresión, siendo más rápida que `expand`. Difiere de `multthru` en que expande todas las sumas del nivel superior.

Ejemplos:

```
(%i1) distrib ((a+b) * (c+d));
(%o1)          b d + a d + b c + a c
(%i2) multthru ((a+b) * (c+d));
(%o2)          (b + a) d + (b + a) c
(%i3) distrib (1/((a+b) * (c+d)));
(%o3)          1
              -----
              (b + a) (d + c)
(%i4) expand (1/((a+b) * (c+d)), 1, 0);
(%o4)          1
              -----
              b d + a d + b c + a c
```

dpart (*expr*, *n_1*, ..., *n_k*)

Función

Selecciona la misma expresión que `part`, pero en lugar de devolver esa expresión como su valor, devuelve la expresión completa con la subexpresión seleccionada dentro de una caja. La caja es parte de la expresión.

```
(%i1) dpart (x+y/z^2, 1, 2, 1);
(%o1)          y
          ---- + x
              2
          """"
          "z"
          """"
```

exp (*x*) Función

Representa la función exponencial. La expresión **exp** (*x*) en la entrada se simplifica en e^x ; **exp** no aparece en expresiones simplificadas.

Si la variable **demoivre** vale **true** hace que $e^{(a + b i)}$ se simplifique a $e^a (\cos(b) + i \sin(b))$ si *b* no contiene a *i*. Véase **demoivre**.

Si la variable **%emode** vale **true**, hace que $e^{(\pi i x)}$ se simplifique. Véase **%emode**.

Si la variable **%enumer** vale **true** hace que *e* se reemplace por 2.718... siempre que **numer** valga **true**. Véase **%enumer**.

%emode Variable opcional

Valor por defecto: **true**

Si **%emode** vale **true**, $e^{(\pi i x)}$ se simplifica como sigue.

$e^{(\pi i x)}$ se simplifica a $\cos(\pi x) + i \sin(\pi x)$ si *x* es un entero o múltiplo de 1/2, 1/3, 1/4 o 1/6, y luego se sigue simplificando.

Para otros valores numéricos de *x*, $e^{(\pi i x)}$ se simplifica a $e^{(\pi i y)}$ donde *y* es $x - 2k$ para algún entero *k* tal que $\text{abs}(y) < 1$.

Si **%emode** vale **false**, no se realizan simplificaciones especiales a $e^{(\pi i x)}$.

%enumer Variable opcional

Valor por defecto: **false**

Si la variable **%enumer** vale **true** hace que *e* se reemplace por 2.718... siempre que **numer** valga **true**.

Si **%enumer** vale **false**, esta sustitución se realiza sólo si el exponente en e^x tiene un valor numérico.

Véanse también **ev** y **numer**.

exptsubst Variable opcional

Valor por defecto: **false**

Si **exptsubst** vale **true** permite la sustitución y por e^x en $e^{(a x)}$.

freeof (*x₁*, ..., *x_n*, *expr*) Función

freeof (*x₁*, *expr*) Devuelve **true** si ninguna subexpresión de *expr* es igual a *x₁* o si *x₁* aparece como variable muda en *expr*, devolviendo **false** en otro caso.

La llamada **freeof** (*x₁*, ..., *x_n*, *expr*) equivale a **freeof** (*x₁*, *expr*) and ... and **freeof** (*x_n*, *expr*).

Los argumentos $x_1, \dots, x_n$ pueden ser nombres de funciones y variables, nombres subindicados, operadores (encerrados entre comillas dobles) o expresiones generales. La función `freeof` evalúa sus argumentos.

Una variable es una variable muda en una expresión si no tiene valor asignado fuera de la expresión. Variable mudas reconocidas por `freeof` son el índice de una suma o producto, la variable límite en `limit`, la variable de integración en la versión de integral definida de `integrate`, la variable original en `laplace`, variables formales en expresiones `at` y los argumentos de las expresiones `lambda`. Las variables locales en `block` no son reconocidas por `freeof` como variables mudas; esto es un fallo de Maxima.

La versión indefinida de `integrate` no está libre de su variable de integración.

- Los argumentos son nombres de funciones, variables, nombres subindicados, operadores y expresiones. La llamada `freeof (a, b, expr)` equivale a `freeof (a, expr)` and `freeof (b, expr)`.

```
(%i1) expr: z^3 * cos (a[1]) * b^(c+d);
                                d + c  3
(%o1)          cos(a ) b      z
                                1

(%i2) freeof (z, expr);
(%o2)                                false
(%i3) freeof (cos, expr);
(%o3)                                false
(%i4) freeof (a[1], expr);
(%o4)                                false
(%i5) freeof (cos (a[1]), expr);
(%o5)                                false
(%i6) freeof (b^(c+d), expr);
(%o6)                                false
(%i7) freeof ("^", expr);
(%o7)                                false
(%i8) freeof (w, sin, a[2], sin (a[2]), b*(c+d), expr);
(%o8)                                true
```

- `freeof` evalúa sus argumentos.

```
(%i1) expr: (a+b)^5$
(%i2) c: a$
(%i3) freeof (c, expr);
(%o3)                                false
```

- `freeof` no considera funciones equivalentes. La simplificación puede dar una expresión equivalente pero diferente.

```
(%i1) expr: (a+b)^5$
(%i2) expand (expr);
      5      4      2 3      3 2      4      5
(%o2)    b  + 5 a b  + 10 a  b  + 10 a  b  + 5 a  b + a
(%i3) freeof (a+b, %);
(%o3)                                true
(%i4) freeof (a+b, expr);
(%o4)                                false
```

```
(%i5) exp (x);
                                x
(%o5)                          %e
(%i6) freeof (exp, exp (x));
(%o6)                          true
```

- Un sumatorio o integral definida está libre de su variable muda. Una integral indefinida de `integrate` no está libre de su variable de integración

```
(%i1) freeof (i, 'sum (f(i), i, 0, n));
(%o1)                          true
(%i2) freeof (x, 'integrate (x^2, x, 0, 1));
(%o2)                          true
(%i3) freeof (x, 'integrate (x^2, x));
(%o3)                          false
```

genfact (*x, y, z*) Función

Devuelve el factorial generalizado, definido como $x (x-z) (x-2z) \dots (x-(y-1)z)$. Así, para el entero x , `genfact (x, x, 1) = x!` y `genfact (x, x/2, 2) = x!!`.

imagpart (*expr*) Función

Devuelve la parte imaginaria de la expresión *expr*.

La función `imagpart` es computacional, no simplificativa.

Véanse también `abs`, `carg`, `polarform`, `rectform` y `realpart`.

infix (*op*) Función

infix (*op, lbp, rbp*) Función

infix (*op, lbp, rbp, lpos, rpos, pos*) Función

Declara *op* como operador infijo. Un operador infijo es una función de dos argumentos, con el nombre de la función escrito entre sus argumentos. Por ejemplo, el operador de sustracción `-` es un operador infijo.

`infix (op)` declara *op* como operador infijo con fuerzas de ligadura por la izquierda y por la derecha iguales a 180, que es el valor por defecto, y partes izquierda y derecha iguales a `any`.

`infix (op, lbp, rbp)` declara *op* como operador infijo con fuerzas de ligadura por la izquierda y por la derecha declaradas en los argumentos, siendo las partes izquierda y derecha iguales a `any`.

`infix (op, lbp, rbp, lpos, rpos, pos)` declara *op* como operador infijo con fuerzas de ligadura por la izquierda y por la derecha y partes declaradas en los argumentos.

La precedencia de *op* con respecto a otros operadores deriva de las fuerzas de ligadura de los operadores en cuestión. Si las fuerzas de ligadura a izquierda y derecha de *op* son ambas mayores que las fuerzas de ligadura a izquierda y derecha de otro operador, entonces *op* tiene preferencia sobre el otro operador. Si las fuerzas de ligadura no son ambas mayores o menores, se aplican otras relaciones más complejas.

La asociatividad de *op* depende de las fuerzas de ligadura. Una mayor fuerza de ligadura a la izquierda (*lbp*) implica que *op* sea evaluado antes que otros operadores a su izquierda en la expresión, mientras que mayor fuerza de ligadura a la derecha

(*rbp*) implica que *op* sea evaluado antes que otros operadores a su derecha en la expresión. Así, si *lbp* es mayor, *op* es asociativo por la derecha, mientras que si *rbp* es mayor, *op* es asociativo por la izquierda.

Véase también *Syntax*.

Ejemplos:

- Si las fuerzas de ligadura a izquierda y derecha de *op* son ambas mayores que las fuerzas de ligadura a izquierda y derecha de otro operador, entonces *op* tiene preferencia sobre el otro operador.

```
(%i1) "@(a, b) := sconcat("(", a, ",", b, ")")$
(%i2) :lisp (get '$+ 'lbp)
100
(%i2) :lisp (get '$+ 'rbp)
100
(%i2) infix ("@", 101, 101)$
(%i3) 1 + a@b + 2;
(%o3) (a,b) + 3
(%i4) infix ("@", 99, 99)$
(%i5) 1 + a@b + 2;
(%o5) (a+1,b+2)
```

- Mayor *lbp* hace a *op* asociativo por la derecha, mientras que mayor *rbp* hace a *op* asociativo por la izquierda.

```
(%i1) "@(a, b) := sconcat("(", a, ",", b, ")")$
(%i2) infix ("@", 100, 99)$
(%i3) foo @ bar @ baz;
(%o3) (foo,(bar,baz))
(%i4) infix ("@", 100, 101)$
(%i5) foo @ bar @ baz;
(%o5) ((foo,bar),baz)
```

inflag

Variable opcional

Valor por defecto: `false`

Si *inflag* vale `true`, las funciones para la extracción de partes inspeccionan la forma interna de *expr*.

Nótese que el simplificador reordena expresiones. Así, `first (x + y)` devuelve `x` si *inflag* vale `true` y `y` si *inflag* vale `false`. (`first (y + x)` devuelve el mismo resultado.)

Además, dándole a *inflag* el valor `true` y llamando a `part` o a `substpart` es lo mismo que llamar a `inpart` o a `substinpart`.

Las funciones que se ven afectadas por el valor de *inflag* son: `part`, `substpart`, `first`, `rest`, `last`, `length`, la construcción `for ... in`, `map`, `fullmap`, `maplist`, `reveal` y `pickapart`.

inpart (expr, n_1, ..., n_k)

Función

Similar a `part`, pero trabaja con la representación interna de la expresión, siendo más rápida. Se debe tener cuidado con el orden de subexpresiones en sumas y productos,

pues el orden de las variables en la forma interna es normalmente diferente al que se muestra por el terminal, y cuando se trata con el signo menos unario, resta y división, pues estos operadores desaparecen de la expresión. Las llamadas `part (x+y, 0)` o `inpart (x+y, 0)` devuelven `+`, siendo necesario encerrar el operador entre comillas dobles cuando se haga referencia a él. Por ejemplo, `... if inpart (%o9,0) = "+" then ...`

Ejemplos:

```
(%i1) x + y + w*z;
(%o1)          w z + y + x
(%i2) inpart (%, 3, 2);
(%o2)          z
(%i3) part (%th (2), 1, 2);
(%o3)          z
(%i4) 'limit (f(x)^g(x+1), x, 0, minus);
              g(x + 1)
(%o4)          limit  f(x)
              x -> 0-
(%i5) inpart (%, 1, 2);
(%o5)          g(x + 1)
```

isolate (*expr*, *x*)

Función

Devuelve *expr* con subexpresiones que son sumas y que no contienen variables reemplazadas por etiquetas de expresiones intermedias (tales etiquetas son símbolos atómicos como `%t1`, `%t2`, ...). Esta función es de utilidad para evitar la expansión innecesaria de subexpresiones que no contienen la variable de interés. Puesto que las etiquetas intermedias toman el valor de subexpresiones pueden ser todas sustituidas evaluando la expresión en la que aparecen.

Si la variable `exptisolate`, cuyo valor por defecto es `false`, vale `true` hará que `isolate` busque exponentes de átomos (como `%e`) que contengan la variable.

Si `isolate_wrt_times` vale `true`, entonces `isolate` también aislará respecto de productos. Véase `isolate_wrt_times`.

Para ejemplos, ejecútese `example (isolate)`.

isolate_wrt_times

Variable opcional

Valor por defecto: `false`

Si `isolate_wrt_times` vale `true`, entonces `isolate` también aislará respecto de productos. Compárese el comportamiento de `isolate` al cambiar el valor de esta variable global en el siguiente ejemplo,

```
(%i1) isolate_wrt_times: true$
(%i2) isolate (expand ((a+b+c)^2), c);

(%t2)          2 a

(%t3)          2 b
```



```

(%t4)

$$b^2 + 2 a b + a^2$$


(%o4)

$$c^2 + \%t3 c + \%t2 c + \%t4$$

(%i4) isolate_wrt_times: false$
(%i5) isolate (expand ((a+b+c)^2), c);

(%o5)

$$c^2 + 2 b c + 2 a c + \%t4$$


```

listconstvars

Variable opcional

Valor por defecto: `false`

Si `listconstvars` vale `true`, hará que `listofvars` incluya `%e`, `%pi`, `%i` y cualquier otra variable que sea declarada constante de las que aparezcan en el argumento de `listofvars`. Estas constantes se omiten por defecto.

listdummyvars

Variable opcional

Valor por defecto: `true`

Si `listdummyvars` vale `false`, las "variables mudas" de la expresión no serán incluidas en la lista devuelta por `listofvars`. (La definición de "variables mudas" se encuentra en la descripción de `freeof`. "Variables mudas" son objetos matemáticos como el índice de un sumatorio o producto, una variable límite o la variable de una integración definida.) Ejemplo:

```

(%i1) listdummyvars: true$
(%i2) listofvars ('sum(f(i), i, 0, n));
(%o2)

$$[i, n]$$

(%i3) listdummyvars: false$
(%i4) listofvars ('sum(f(i), i, 0, n));
(%o4)

$$[n]$$


```

listofvars (expr)

Función

Devuelve una lista con las variables presentes en `expr`.

Si la variable `listconstvars` vale `true` entonces `listofvars` incluirá `%e`, `%pi`, `%i` y cualquier otra variable declarada constante de las que aparezcan en `expr`. Estas constantes se omiten por defecto.

```

(%i1) listofvars (f (x[1]+y) / g^(2+a));
(%o1)

$$[g, a, x_1, y]$$


```

lfreeof (list, expr)

Función

Para cada miembro `m` de `list`, realiza la llamada `freeof (m, expr)`. Devuelve `false` si alguna de estas llamadas a `freeof` retornó `false`, y `true` en caso contrario.

lopow (expr, x)

Función

Devuelve el mínimo exponente de `x` que aparece explícitamente en `expr`. Así,

```
(%i1) lopow ((x+y)^2 + (x+y)^a, x+y);
(%o1) min(a, 2)
```

lpart (*label, expr, n_1, ..., n_k*)

Función

Similar a **dpart** pero utiliza una caja etiquetada. Una caja etiquetada es similar a la que produce **dpart**, pero con un nombre en la línea superior.

multthru (*expr*)

Función

multthru (*expr_1, expr_2*)

Función

Multiplica un factor (que debería ser una suma) de *expr* por los otros factores de *expr*. Esto es, *expr* es $f_1 f_2 \dots f_n$, donde al menos un factor, por ejemplo f_i , es una suma de términos. Cada término en esta suma se multiplica por los otros factores del producto, excepto el propio f_i . La función **multthru** no expande sumas elevadas a exponentes, siendo el método más rápido para distribuir productos (sean o no conmutativos) sobre sumas. Puesto que los cocientes se representan como productos, puede utilizarse **multthru** para dividir sumas entre productos.

La llamada **multthru** (*expr_1, expr_2*) multiplica cada término de *expr_2* (que debería ser una suma o una ecuación) por *expr_1*. Si *expr_1* no es ella misma una suma, entonces la llamada es equivalente a **multthru** (*expr_1*expr_2*).

```
(%i1) x/(x-y)^2 - 1/(x-y) - f(x)/(x-y)^3;
(%o1)
      1      x      f(x)
      - ---- + ---- - ----
      x - y      (x - y)^2      (x - y)^3

(%i2) multthru ((x-y)^3, %);
(%o2)
      2
      - (x - y) + x (x - y) - f(x)

(%i3) ratexpand (%);
(%o3)
      2
      - y + x y - f(x)

(%i4) ((a+b)^10*s^2 + 2*a*b*s + (a*b)^2)/(a*b*s^2);
(%o4)
      10 2      2 2
      (b + a) s + 2 a b s + a b
      -----
      2
      a b s

(%i5) multthru (%); /* note that this does not expand (b+a)^10 */
(%o5)
      2      a b      (b + a)
      - + ---- + ----
      s      2      a b
      s

(%i6) multthru (a.(b+c.(d+e)+f));
(%o6)
      a . f + a . c . (e + d) + a . b

(%i7) expand (a.(b+c.(d+e)+f));
(%o7)
      a . f + a . c . e + a . c . d + a . b
```

nounify (*f*)

Función

Devuelve la forma nominal de la función cuyo nombre es *f*. Puede ser útil cuando se quiera hacer referencia al nombre de una función sin que ésta se ejecute. Nótese que algunas funciones verbales devolverán su forma nominal si no pueden ser evaluadas para ciertos argumentos. Esta es también la expresión que se obtiene cuando la llamada a una función va precedida por del apóstrofo.

nterms (*expr*)

Función

Devuelve el número de términos que *expr* llegaría a tener si fuese completamente expandida y no hubiesen cancelaciones ni combinaciones de términos semejantes. Nótese que expresiones como **sin** (*expr*), **sqrt** (*expr*), **exp** (*expr*), etc. cuentan como un sólo término, independientemente de cuántos términos tenga a su vez *expr* en caso de tratarse de una suma.

op (*expr*)

Función

Devuelve el operador principal de la expresión *expr*. La llamada **op** (*expr*) equivale a **part** (*expr*, 0).

La función **op** devuelve una cadena si el operador principal es un operador prefijo, infijo (binario o n-ario), postfijo, "bi-fijo" o "no-fijo" ("bi-fijo" se refiere a un par de símbolos que encierran su o sus argumentos, y "no-fijo" es un operador que no necesita argumentos). En cualquier otro caso **op** devuelve un símbolo.

La función **op** observa el valor de la variable global **inflag**.

La función **op** evalúa sus argumentos.

Véase también **args**.

Ejemplos:

```
(%i1) ?stringdisp: true$
(%i2) op (a * b * c);
(%o2)      "*"
(%i3) op (a * b + c);
(%o3)      "+"
(%i4) op ('sin (a + b));
(%o4)      sin
(%i5) op (a!);
(%o5)      "!"
(%i6) op (-a);
(%o6)      "-"
(%i7) op ([a, b, c]);
(%o7)      "["
(%i8) op ('(if a > b then c else d));
(%o8)      "if"
(%i9) op ('foo (a));
(%o9)      foo
(%i10) prefix (foo);
(%o10)     "foo"
(%i11) op (foo a);
(%o11)     "foo"
```

- operatorp** (*expr*, *op*) Función
operatorp (*expr*, [*op_1*, ..., *op_n*]) Función
 La llamada **operatorp** (*expr*, *op*) devuelve **true** si *op* es igual al operador de *expr*.
 La llamada **operatorp** (*expr*, [*op_1*, ..., *op_n*]) devuelve **true** si algún elemento *op_1*, ..., *op_n* es igual al operador de *expr*.
- optimize** (*expr*) Función
 Devuelve una expresión que produce el mismo valor y efectos secundarios que *expr*, pero de forma más eficiente al evitar recalcular subexpresiones comunes. La función **optimize** también tiene el efecto secundario de colapsar su argumento de manera que se compartan todas sus subexpresiones comunes. Hágase **example (optimize)** para ver ejemplos.
- optimprefix** Variable opcional
 Valor por defecto: %
 La variable **optimprefix** es el prefijo utilizado para los símbolos generados por la instrucción **optimize**.
- ordergreat** (*v_1*, ..., *v_n*) Función
 Establece una ordenación para las variables *v_1*, ..., *v_n* de manera que *v_1* > *v_2* > ... > *v_n* y *v_n* > cualquier otra variable no mencionada como argumento.
 Véase también **orderless**.
- ordergreatp** (*expr_1*, *expr_2*) Función
 Devuelve **true** si *expr_2* precede a *expr_1* según el orden establecido con la función **ordergreat**.
- orderless** (*v_1*, ..., *v_n*) Función
 Establece una ordenación para las variables *v_1*, ..., *v_n* de manera que *v_1* < *v_2* < ... < *v_n* y *v_n* < cualquier otra variable no mencionada como argumento.
 Así, la ordenación completa de objetos es: constantes numéricas < constantes declaradas < escalares declarados < primer argumento de **orderless** < ... < último argumento de **orderless** < variables que empiezan por A < ... < variables que empiezan por Z < último argumento de **ordergreat** < ... < primer argumento de **ordergreat** < objetos declarados como **mainvar**.
 Véanse también **ordergreat** y **mainvar**.
- orderlessp** (*expr_1*, *expr_2*) Función
 Devuelve **true** si *expr_1* precede a *expr_2* según el orden establecido con la función **orderless**.
- part** (*expr*, *n_1*, ..., *n_k*) Función
 Devuelve partes de la forma mostrada de **expr**. Obtiene la parte de **expr** que se especifica por los índices *n_1*, ..., *n_k*. Primero se obtiene la parte *n_1* de **expr**,

después la parte n_2 del resultado anterior, y así sucesivamente. El resultado que se obtiene es la parte n_k de ... la parte n_2 de la parte n_1 de **expr**.

La función **part** se puede utilizar para obtener un elemento de una lista, una fila de una matriz, etc.

Si el último argumento de la función **part** es una lista de índices, entonces se toman varias subexpresiones, cada una de las cuales correspondiente a un índice de la lista. Así, **part** ($x + y + z$, [1, 3]) devuelve $z+x$.

La variable **piece** guarda la última expresión seleccionada con la función **part**. Se actualiza durante la ejecución de la función, por lo que puede ser referenciada en la misma función.

Si **partswitch** vale **true** entonces devuelve **end** cuando no exista la parte seleccionada de una expresión, si vale **false** se mostrará un mensaje de error.

Ejemplo: **part** ($z+2*y$, 2, 1) devuelve 2.

La instrucción **example** (**part**) muestra más ejemplos.

partition (*expr*, *x*)

Función

Devuelve una lista con dos expresiones, que son: (1) los factores de *expr* si es un producto, los términos de *expr* si es una suma, o los elementos de *expr*, si es una lista, que no contengan a *x*, (2) los factores, términos o lista que contengan a *x*.

```
(%i1) partition (2*a*x*f(x), x);
(%o1)          [2 a, x f(x)]
(%i2) partition (a+b, x);
(%o2)          [b + a, 0]
(%i3) partition ([a, b, f(a), c], a);
(%o3)          [[b, c], [a, f(a)]]
```

partswitch

Variable opcional

Valor por defecto: **false**

Si **partswitch** vale **true** entonces devuelve **end** cuando no exista la parte seleccionada de una expresión, si vale **false** se mostrará un mensaje de error.

pickapart (*expr*, *n*)

Función

Asigna etiquetas de expresiones intermedias a subexpresiones de *expr* al nivel de profundidad *n*, que es un entero. A las subexpresiones a un nivel de profundidad mayor o menor no se les asignan etiquetas. La función **pickapart** devuelve una expresión en términos de expresiones intermedias equivalente a la expresión original *expr*.

Véanse también **part**, **dpart**, **lpart**, **inpart** y **reveal**.

Ejemplos:

```
(%i1) expr: (a+b)/2 + sin (x^2)/3 - log (1 + sqrt(x+1));
(%o1)          - log(sqrt(x + 1) + 1) +  $\frac{\sin(x)}{3}$  +  $\frac{b + a}{2}$ 
(%i2) pickapart (expr, 0);
```

```

(%t2)      - log(sqrt(x + 1) + 1) +  $\frac{\sin^2(x)}{3}$  +  $\frac{b + a}{2}$ 

(%o2)      %t2
(%i3) pickapart (expr, 1);

(%t3)      - log(sqrt(x + 1) + 1)

(%t4)       $\frac{\sin^2(x)}{3}$ 

(%t5)       $\frac{b + a}{2}$ 

(%o5)      %t5 + %t4 + %t3
(%i5) pickapart (expr, 2);

(%t6)      log(sqrt(x + 1) + 1)

(%t7)       $\sin^2(x)$ 

(%t8)      b + a

(%o8)       $\frac{\%t8}{2} + \frac{\%t7}{3} - \%t6$ 
(%i8) pickapart (expr, 3);

(%t9)      sqrt(x + 1) + 1

(%t10)     x2

(%o10)      $\frac{b + a}{2} - \log(\%t9) + \frac{\sin(\%t10)}{3}$ 
(%i10) pickapart (expr, 4);

```

```
(%t11)                                sqrt(x + 1)

                                2
                                sin(x )  b + a
(%o11)  ----- + ----- - log(%t11 + 1)
                                3          2
(%i11) pickapart (expr, 5);

(%t12)                                x + 1

                                2
                                sin(x )  b + a
(%o12)  ----- + ----- - log(sqrt(%t12) + 1)
                                3          2
(%i12) pickapart (expr, 6);

                                2
                                sin(x )  b + a
(%o12)  ----- + ----- - log(sqrt(x + 1) + 1)
                                3          2
```

piece

System variable

Guarda la última expresión seleccionada por las funciones **part**.

polarform (*expr*)

Función

Devuelve una expresión de la forma $r e^{(i \theta)}$ equivalente a *expr*, con *r* y *theta* son reales.

powers (*expr*, *x*)

Función

Devuelve las potencias de *x* dentro de *expr*.

La instrucción **load** (**powers**) carga esta función.

product (*expr*, *i*, *i_0*, *i_1*)

Función

Representa el producto de los valores de **expr** según el índice *i* varía de *i_0* hasta *i_1*. La forma nominal '**product**' se presenta en forma de letra pi mayúscula.

La función **product** evalúa *expr* y los límites inferior y superior, *i_0* y *i_1*, pero no evalúa el índice *i*.

Si la diferencia entre los límites superior e inferior es un número entero, la expresión *expr* se evalúa para cada valor del índice *i*, siendo el resultado un producto en forma explícita.

En caso contrario, el rango del índice no está definido, aplicándose entonces algunas reglas que permitan simplificar el producto. Cuando la variable global **simplproduct** valga **true**, se aplicarán reglas adicionales. En ciertos casos, la simplificación dará lugar a un resultado que ya no tenga el formato del producto; en caso contrario se devolverá una forma nominal '**product**'.

Véanse también **nouns** y **evflag**.

Ejemplos:

```

(%i1) product (x + i*(i+1)/2, i, 1, 4);
(%o1)          (x + 1) (x + 3) (x + 6) (x + 10)
(%i2) product (i^2, i, 1, 7);
(%o2)          25401600
(%i3) product (a[i], i, 1, 7);
(%o3)          a a a a a a a
                1 2 3 4 5 6 7
(%i4) product (a(i), i, 1, 7);
(%o4)          a(1) a(2) a(3) a(4) a(5) a(6) a(7)
(%i5) product (a(i), i, 1, n);
                n
                /===\
                ! !
(%o5)          ! ! a(i)
                ! !
                i = 1
(%i6) product (k, k, 1, n);
                n
                /===\
                ! !
(%o6)          ! ! k
                ! !
                k = 1
(%i7) product (k, k, 1, n), simpproduct;
(%o7)          n!
(%i8) product (integrate (x^k, x, 0, 1), k, 1, n);
                n
                /===\
                ! !      1
(%o8)          ! ! -----
                ! ! k + 1
                k = 1
(%i9) product (if k <= 5 then a^k else b^k, k, 1, 10);
                15 40
(%o9)          a   b

```

realpart (*expr*) Función

Devuelve la parte real de *expr*. Las funciones **realpart** y **imagpart** operan también con expresiones que contengan funciones trigonométricas e hiperbólicas, raíces cuadradas, logaritmos y exponentes.

rectform (*expr*) Función

Devuelve una expresión de la forma $a + b \%i$ equivalente a *expr*, con *a* y *b* reales.

rembox (*expr*, *unlabelled*) Función

rembox (*expr*, *label*) Función

rembox (*expr*) Función

Elimina cajas de *expr*.

La llamada `rembox (expr, unlabelled)` elimina todas las cajas no etiquetadas de `expr`.

La llamada `rembox (expr, label)` sólo elimina las cajas etiquetadas con `label`.

La llamada `rembox (expr)` elimina todas las cajas, independientemente de que estén etiquetadas o no.

Las cajas son dibujadas por las funciones `box`, `dpart` y `lpart`.

Ejemplos:

```
(%i1) expr: (a*d - b*c)/h^2 + sin(%pi*x);
(%o1)
          a d - b c
      sin(%pi x) + ----
                  2
                  h

(%i2) dpart (dpart (expr, 1, 1), 2, 2);
(%o2)
          "a d - b c"
      sin("%pi x") + ----
          " 2"
          "h "
          "    "

(%i3) expr2: lpart (BAR, lpart (FOO, %, 1), 2);
FOO"          " BAR"          "
"          " "a d - b c"
"sin("%pi x")" + "-----"
"          " "    "
"          " " 2" "
"          " "h " "
"          " "    "
"          "

(%i4) rembox (expr2, unlabelled);
          BAR"          "
FOO"          " "a d - b c"
"sin(%pi x)" + "-----"
"          " 2 "
"          " h "
"          "

(%i5) rembox (expr2, FOO);
          BAR"          "
          "a d - b c"
sin("%pi x") + "-----"
          "    "
          " 2" "
          "h " "
          "    "
          "

(%i6) rembox (expr2, BAR);
FOO"          "
"          " a d - b c
"sin("%pi x")" + -----
```

```

"          "      "
"          "      " 2"
"          "      "h "
"          "      ""

(%i7) rembox (expr2);

(%o7)
          a d - b c
sin(%pi x) + -----
              2
              h

```

sum (*expr*, *i*, *i_0*, *i_1*) Función

Representa la suma de los valores de **expr** según el índice *i* varía de *i_0* hasta *i_1*. La forma nominal **'sum** se presenta en forma de letra sigma mayúscula.

La función **sum** evalúa su sumando *expr* y los límites inferior y superior, *i_0* y *i_1*, pero no evalúa el índice *i*.

Si la diferencia entre los límites superior e inferior es un número entero, el sumando *expr* se evalúa para cada valor del índice *i*, siendo el resultado una suma en forma explícita.

En caso contrario, el rango del índice no está definido, aplicándose entonces algunas reglas que permitan simplificar la suma. Cuando la variable global **simplsum** valga **true**, se aplicarán reglas adicionales. En ciertos casos, la simplificación dará lugar a un resultado que ya no tenga el formato del sumatorio; en caso contrario se devolverá una forma nominal **'product**.

Cuando **cauchysum** vale **true**, el producto de sumatorios se expresa como un producto de Cauchy, en cuyo caso el índice del sumatorio interior es función del índice del exterior, en lugar de variar independientemente.

La variable global **genindex** guarda el prefijo alfabético a utilizar cuando sea necesario generar automáticamente el siguiente índice de sumatorio.

La variable global **gensumnum** guarda el sufijo numérico a utilizar cuando sea necesario generar automáticamente el siguiente índice de sumatorio. Si **gensumnum** vale **false**, un índice generado automáticamente constará sólo de **genindex**, sin sufijo numérico.

Véanse también **sumcontract**, **intosum**, **bashindices**, **niceindices**, **nouns** y **evflag**.

Ejemplos:

```

(%i1) sum (i^2, i, 1, 7);
(%o1)
140
(%i2) sum (a[i], i, 1, 7);
(%o2)
a  + a  + a  + a  + a  + a  + a
 7   6   5   4   3   2   1
(%i3) sum (a(i), i, 1, 7);
(%o3)
a(7) + a(6) + a(5) + a(4) + a(3) + a(2) + a(1)
(%i4) sum (a(i), i, 1, n);
(%o4)
      n
      ====
      \
      >  a(i)

```

```

/
=====
i = 1
(%i5) sum (2^i + i^2, i, 0, n);
n
=====
\      i      2
>      (2  + i )
/
=====
i = 0
(%i6) sum (2^i + i^2, i, 0, n), simpsum;
3      2
n + 1  2 n  + 3 n  + n
(%o6)  2      + ----- - 1
6
(%i7) sum (1/3^i, i, 1, inf);
inf
=====
\      1
>      --
/      i
===== 3
i = 1
(%i8) sum (1/3^i, i, 1, inf), simpsum;
1
(%o8)  -
2
(%i9) sum (i^2, i, 1, 4) * sum (1/i^2, i, 1, inf);
inf
=====
\      1
30 >  --
/      2
===== i
i = 1
(%i10) sum (i^2, i, 1, 4) * sum (1/i^2, i, 1, inf), simpsum;
2
(%o10) 5 %pi
(%i11) sum (integrate (x^k, x, 0, 1), k, 1, n);
n
=====
\      1
>      ----
/      k + 1
=====
k = 1
(%i12) sum (if k <= 5 then a^k else b^k, k, 1, 10));
Incorrect syntax: Too many )'s

```

```

else b^k, k, 1, 10))
      ^
(%i12) linenum:11;
(%o11)
      11
(%i12) sum (integrate (x^k, x, 0, 1), k, 1, n);
      n
      ====
      \      1
      >  -----
      /      k + 1
      ====
      k = 1
(%i13) sum (if k <= 5 then a^k else b^k, k, 1, 10);
      10   9   8   7   6   5   4   3   2
(%o13)  b  + b + b + b + b + a + a + a + a + a

```

lsum (*expr*, *x*, *L*)

Función

Representa la suma de *expr* para cada elemento *x* en *L*.

Se retornará la forma nominal 'lsum si el argumento *L* no es una lista.

Ejemplos:

```

(%i1) lsum (x^i, i, [1, 2, 7]);
      7   2
(%o1)  x  + x  + x
(%i2) lsum (i^2, i, rootsof (x^3 - 1));
      ====
      \      2
      >      i
      /
      ====
      3
      i in rootsof(x  - 1)

```

verbify (*f*)

Función

Devuelve la forma verbal del nombre de función *f*.

Véanse también **verb**, **noun** y **nounify**.

Ejemplos:

```

(%i1) verbify ('foo);
(%o1)  foo
(%i2) :lisp $%
$F00
(%i2) nounify (foo);
(%o2)  foo
(%i3) :lisp $%
%F00

```


7 Simplificación

7.1 Definiciones para Simplificación

askexp

Variable del sistema

Cuando se invoca a **asksign**, la expresión que se va a analizar es precisamente **askexp**.

askinteger (*expr*, *integer*)

Función

askinteger (*expr*)

Función

askinteger (*expr*, *even*)

Función

askinteger (*expr*, *odd*)

Función

La llamada **askinteger** (*expr*, *integer*) intenta determinar a partir de la base de datos de **assume** si *expr* es un entero. La función **askinteger** pide más información al usuario si no encuentra la respuesta, tratando de almacenar la nueva información en la base de datos si es posible. La llamada **askinteger** (*expr*) equivale a **askinteger** (*expr*, *integer*).

La llamadas **askinteger** (*expr*, *even*) ay **askinteger** (*expr*, *odd*) intentan determinar si *expr* es un entero par o impar, respectivamente.

asksign (*expr*)

Función

Primero intenta determinar si la expresión especificada es positiva, negativa o cero. Si no lo consigue, planteará al usuario preguntas que le ayuden a completar la deducción. Las respuestas del usuario son almacenadas en la base de datos durante el tiempo que dure este cálculo. El valor que al final devuelva **asksign** será **pos**, **neg** o **zero**.

demoivre (*expr*)

Función

demoivre

Variable opcional

La función **demoivre** (*expr*) convierte una expresión sin modificar la variable global **demoivre**.

Cuando **demoivre** vale **true**, los exponenciales complejos se convierten en expresiones equivalentes pero en términos de las funciones trigonométricas: **exp** (*a* + *b**%i) se reduce a %e^a * (cos(*b*) + %i*sin(*b*)) si *b* no contiene a %i. Las expresiones *a* y *b* no se expanden.

El valor por defecto de **demoivre** es **false**.

La función **exponentialize** convierte funciones trigonométricas e hiperbólicas a la forma exponencial, por lo que **demoivre** y **exponentialize** no pueden valer **true** al mismo tiempo.

domain

Variable opcional

Valor por defecto: **real**

Si **domain** vale **complex**, **sqrt** (*x*²) permanecerá como **sqrt** (*x*²) en lugar de devolver **abs**(*x*).

expand (*expr*)

Función

expand (*expr*, *p*, *n*)

Función

Expande la expresión *expr*. Los productos de sumas y de sumas con exponentes se multiplican, los numeradores de las expresiones racionales que son sumas se separan en sus respectivos términos, y las multiplicaciones (tanto las que son conmutativas como las que no) se distribuyen sobre las sumas en todos los niveles de *expr*.

En el caso de los polinomios es más aconsejable utilizar **ratexpand**, que utiliza un algoritmo más eficiente.

Las variables **maxnegex** y **maxposex** controlan los máximos exponentes negativos y positivos que se van a expandir.

La llamada **expand** (*expr*, *p*, *n*) expande *expr* asignando a **maxposex** el valor *p* y a **maxnegex** el *n*. Esto es útil para expandir sólo parte de la expresión.

La variable **expon** guarda el mayor exponente negativo que será expandido automáticamente, independientemente de **expand**. Por ejemplo, si **expon** vale 4 entonces $(x+1)^{-5}$ no se expandirá automáticamente.

La variable **expop** guarda el mayor exponente positivo que será expandido automáticamente. Así, $(x+1)^3$ se expandirá automáticamente sólo si **expop** es mayor o igual que 3. Si se quiere expandir $(x+1)^n$, siendo *n* mayor que **expop**, entonces **expand** $((x+1)^n)$ se desarrollará sólo si **maxposex** no es menor que *n*.

La variable **expand** utilizada con **ev** provocará una expansión.

El fichero '**simplification/facexp.mac**' contiene algunas funciones relacionadas con **expand** (en concreto, **facsum**, **factorfacsum** y **collectterms**, que se cargan automáticamente) y variables (**nextlayerfactor** y **facsum_combine**) que permiten al usuario estructurar las expresiones controlando la expansión. En '**simplification/facexp.usg**' se pueden encontrar breves descripciones de estas funciones. Se accederá a una demostración con la instrucción **demo("facexp")**.

expandwrt (*expr*, *x_1*, ..., *x_n*)

Función

Expande la expresión *expr* con respecto a las variables *x_1*, ..., *x_n*. Todos los productos que contengan a las variables aparecen explícitamente. El resultado que se obtenga no tendrá productos de sumas de expresiones que contengan a las variables. Los argumentos *x_1*, ..., *x_n* pueden ser variables, operadores o expresiones.

Por defecto, no se expanden los denominadores, pero esto puede cambiarse mediante el uso de la variable **expandwrt_denom**.

Esta función se carga automáticamente de '**simplification/stopex.mac**'.

expandwrt_denom

Variable opcional

Valor por defecto: **false**

La variable **expandwrt_denom** controla el tratamiento de las expresiones racionales por parte de **expandwrt**. Si vale **true**, se expandirán tanto el numerador como el denominador de la expresión respecto de los argumentos de **expandwrt**, pero si **expandwrt_denom** vale **false**, sólo se expandirá el numerador.

expandwrt_factored (*expr*, *x_1*, ..., *x_n*) Función

Es similar a **expandwrt**, pero trata a las expresiones que son productos de una forma algo diferente. La función **expandwrt_factored** expande sólo aquellos factores de **expr** que contienen a las variables *x_1*, ..., *x_n*.

Esta función se carga automáticamente de 'simplification/stopex.mac'.

expon Variable opcional

Valor por defecto: 0

La variable **expon** guarda el mayor exponente negativo que será expandido automáticamente, independientemente de **expand**. Por ejemplo, si **expon** vale 4 entonces $(x+1)^{-5}$ no se expandirá automáticamente.

exponentialize (*expr*) Función**exponentialize** Variable opcional

La función **exponentialize** (*expr*) convierte las funciones trigonométricas e hiperbólicas de *expr* a exponenciales, sin alterar la variable global **exponentialize**.

Cuando la variable **exponentialize** vale **true**, todas las funciones trigonométricas e hiperbólicas se convierten a forma exponencial. El valor por defecto es **false**.

La función **demoivre** convierte funciones trigonométricas e hiperbólicas a la forma exponencial, por lo que **demoivre** y **exponentialize** no pueden valer **true** al mismo tiempo.

expop Variable opcional

Valor por defecto: 0

La variable **expop** guarda el mayor exponente positivo que será expandido automáticamente. Así, $(x+1)^3$ se expandirá automáticamente sólo si **expop** es mayor o igual que 3. Si se quiere expandir $(x+1)^n$, siendo *n* mayor que **expop**, entonces **expand** $((x+1)^n)$ se desarrollará sólo si **maxposex** no es menor que *n*.

factlim Variable opcional

Valor por defecto: -1

La variable **factlim** especifica el mayor factorial que será expandido automáticamente. Si su valor es -1, entonces se expandirán todos los enteros.

intosum (*expr*) Función

Mueve los factores multiplicativos que están fuera de un sumatorio hacia dentro de éste. Si el índice aparece en la expresión exterior, entonces **intosum** busca un índice razonable, lo mismo que hace con **sumcontract**. Se trata de la operación contraria a extraer factores comunes de los sumatorios.

En algunos casos puede ser necesario hacer **scanmap** (**multthru**, *expr*) antes que **intosum**.

lassociative Declaración

La instrucción **declare** (*g*, **lassociative**) le indica al simplificador de Maxima que *g* es asociativo por la izquierda. Por ejemplo, **g** (**g** (*a*, *b*), **g** (*c*, *d*)) se reduce a **g** (**g** (*a*, *b*), *c*), *d*.

linear

Declaración

Es una de las propiedades de operadores de Maxima. Si la función univariante f se declara lineal, la expansión de $f(x + y)$ produce $f(x) + f(y)$, $f(ax)$ produce $a*f(x)$ si a es una constante. Si la función tiene dos o más argumentos, la linealidad se interpreta como la de `sum` o `integrate`, esto es, $f(ax + b, x)$ produce $a*f(x, x) + b*f(1, x)$ si a y b no contienen a x .

`linear` equivale a `additive` y `outative`. Véase también `opproperties`.

mainvar

Declaración

Se pueden declarar variables de tipo `mainvar`. El orden de los átomos es: números < constantes (como `%e` o `%pi`) < escalares < otras variables < "mainvars". Por ejemplo, compárese `expand((X+Y)^4)` con `(declare(x, mainvar), expand((x+y)^4))`. (Nota: Se debe tener cuidado si se quiere hacer uso de esta declaración. Por ejemplo, si se resta una expresión en la que x ha sido declarada como `mainvar` de otra en la que x no es `mainvar`, puede ser necesario volver a simplificar, `ev(expr, simp)`, a fin de obtener cancelaciones. Además, si se guarda una expresión en la que x es `mainvar`, quizás sea necesario guardar también x .)

maxapplydepth

Variable opcional

Valor por defecto: 10000

La variable `maxapplydepth` es la máxima profundidad a la que van a introducirse `apply1` y `apply2`.

maxapplyheight

Variable opcional

Valor por defecto: 10000

La variable `maxapplyheight` es la máxima altura a la que escalará `applyb1` antes de detenerse.

maxnegex

Variable opcional

Valor por defecto: 1000

La variable `maxnegex` es el mayor exponente negativo que expandirá la función `expand`. Véase también `maxposex`.

maxposex

Variable opcional

Valor por defecto: 1000

La variable `maxposex` es el mayor exponente que expandirá la función `expand`. Véase también `maxnegex`.

multiplicative

Declaración

La instrucción `declare(f, multiplicative)` indica al simplificador de Maxima que f es multiplicativa.

1. Si f es univariante, cada vez que el simplificador encuentre a f aplicada a un producto, f se distribuirá sobre ese producto. Por ejemplo, $f(x*y)$ se reducirá a $f(x)*f(y)$.

2. Si f es una función de 2 o más argumentos, la multiplicabilidad se define como multiplicabilidad para el primer argumento de f , de modo que $f(g(x) * h(x), x)$ se reduciría a $f(g(x), x) * f(h(x), x)$.

Esta transformación no se realiza cuando f se aplica a expresiones de la forma `product(x[i], i, m, n)`.

negdistrib

Variable opcional

Valor por defecto: `true`

Si `negdistrib` vale `true`, -1 se distribuye sobre una expresión. Por ejemplo, $-(x + y)$ se transforma en $-y - x$. Dándole el valor `false` se mostrará $-(x + y)$ tal cual. Esto puede ser útil, pero también peligroso; al igual que el indicador `simp`, no conviene asignarle el valor `false`.

negsumdispflag

Variable opcional

Valor por defecto: `true`

Si `negsumdispflag` vale `true`, $x - y$ se muestra como $x - y$ en lugar de $-y + x$. Dándole el valor `false` se realiza un análisis adicional para que no se representen de forma muy diferente dos expresiones similares. Una aplicación puede ser para que $a + i*b$ y $a - i*b$ se representen ambas de la misma manera.

noeval

Símbolo especial

El símbolo `noeval` evita la fase de evaluación de `ev`. Es útil conjuntamente con otras variables globales y para poder volver a simplificar expresiones sin tener que evaluarlas otra vez.

noun

Declaración

El símbolo `noun` es una de las opciones de la instrucción `declare`. Hace que una función se declare como "nombre", lo que significa que no se evaluará automáticamente.

noundisp

Variable opcional

Valor por defecto: `false`

Si `noundisp` vale `true`, los nombres se muestran precedidos de un apóstrofo. Siempre debe valer `true` cuando se quiera representar la definición de funciones.

nouns

Símbolo especial

El símbolo `nouns` es una `evflag`, lo que significa que cuando se utilice como una opción de la instrucción `ev`, todas las formas nominales que aparezcan en una expresión las convierte en verbales, esto es, las evalúa. Véanse también `noun`, `nounify`, `verb` y `verbify`.

numer

Símbolo especial

El símbolo `numer` hace que algunas funciones matemáticas, incluida la exponenciación, con argumentos numéricos se evalúen como decimales de punto flotante. También reemplaza las variables que haya en `expr`, y a las cuales se les ha asignado previamente un valor, por estos mismo valores. Además, activa la variable `float`.

numeval (*x_1, expr_1, ..., var_n, expr_n*) Función

Declara las variables *x_1, ..., x_n* asignándoles los valores numéricos *expr_1, ..., expr_n*. El valor numérico se evalúa y sustituye a la variable en cualquier expresión en la que ésta aparezca si *numeval* toma el valor *true*. Véase también *ev*.

Las expresiones *expr_1, ..., expr_n* pueden ser expresiones no necesariamente numéricas.

opproperties Variable del sistema

La variable *opproperties* es la lista con las propiedades especiales de los operadores reconocidas por el simplificador de Maxima: *linear, additive, multiplicative, outative, evenfun, oddfun, commutative, symmetric, antisymmetric, nary, lassociative, rassociative*.

opsubst Variable opcional

Valor por defecto: *true*

Si *opsubst* vale *false*, *subst* no sustituye el operador de una expresión, de manera que (*opsubst: false, subst (x^2, r, r+r[0])*) trabajará correctamente.

outative Declaración

La instrucción *declare (f, outative)* le indica al simplificador de Maxima que los factores constantes del argumento de la función *f* pueden ser extraídos.

1. Si *f* es univariante, cada vez que el simplificador se encuentra con *f* aplicada a un producto, éste será particionado en factores que son constantes y factores que no lo son, siendo entonces los constantes extraídos de la función. Por ejemplo, *f(a*x)* se reducirá a *a*f(x)* siendo *a* una constante. Las constantes no atómicas no serán extraídas.
2. Si *f* es una función de 2 o más argumentos, esta propiedad se define como en *sum* o *integrate*, esto es, *f(a*g(x), x)* se reducirá a *a * f(g(x), x)* si *a* no contiene a *x*.

Las funciones *sum, integrate* y *limit* han sido todas declaradas con la propiedad *outative*.

posfun Declaración

La instrucción *declare (f, posfun)* declara a *f* como función positiva, de forma que *is (f(x) > 0)* devolverá *true*.

radcan (*expr*) Función

Simplifica la expresión *expr*, que puede contener logaritmos, exponenciales y radicales, convirtiéndola a una forma canónica, lo que significa que todas las expresiones funcionalmente equivalentes se reducen a una forma única. Ciertas expresiones, sin embargo, son reducidas por *radcan* a una forma regular, lo que significa que dos expresiones equivalentes no tienen necesariamente el mismo aspecto, pero su diferencia puede ser reducida por *radcan* a cero.

Con algunas expresiones *radcan* puede consumir mucho tiempo. Este es el coste por explorar ciertas relaciones entre las componentes de la expresión para simplificaciones basadas en factorizaciones y expansiones parciales de fracciones de exponentes.

Si `%e_to_numlog` vale `true`, `%e^(r*log(expr))` se reduce a `expr^r` si `r` es un número racional.

Si `radexpand` vale `false`, ciertas transformaciones se inhiben; `radcan (sqrt (1-x))` se mantiene como `sqrt (1-x)` y no se reduce a `%i sqrt (x-1)`, o `radcan (sqrt (x^2 - 2*x + 1))` se mantiene como `sqrt (x^2 - 2*x + 1)` sin reducirse a `x - 1`.

La instrucción `example (radcan)` muestra algunos ejemplos.

radexpand

Variable opcional

Valor por defecto: `true`

La variable `radexpand` controla algunas simplificaciones de radicales.

Si `radexpand` vale `all`, las raíces n -ésimas de los factores de un producto que sean potencias de n se extraen del símbolo radical. Por ejemplo, si `radexpand` vale `all`, `sqrt (16*x^2)` se reduce a `4*x`.

Más concretamente, considérese `sqrt (x^2)`.

- Si `radexpand` vale `all` o se ha ejecutado `assume (x > 0)`, `sqrt(x^2)` se reduce a `x`.
- Si `radexpand` vale `true` y `domain` es `real` (su valor por defecto), `sqrt(x^2)` se reduce a `abs(x)`.
- Si `radexpand` vale `false` o `radexpand` vale `true` y `domain` es `complex`, `sqrt(x^2)` no se simplifica.

Nótese que `domain` sólo se tiene en cuenta si `radexpand` vale `true`.

radsubstflag

Variable opcional

Valor por defecto: `false`

Si `radsubstflag` vale `true` se permite a `ratsubst` hacer la sustitución `u` por `sqrt (x)` in `x`.

rassociative

Declaración

La instrucción `declare (g, rassociative)` le indica al simplificador de Maxima que `g` es asociativa por la derecha. Por ejemplo, `g(g(a, b), g(c, d))` se reduce a `g(a, g(b, g(c, d)))`.

scsimp (*expr, rule_1, ..., rule_n*)

Función

Es el "Sequential Comparative Simplification" (método debido a Stoute). La función `scsimp` intenta simplificar `expr` de acuerdo con las reglas `rule_1, ..., rule_n`. Si se obtiene una expresión más pequeña, el proceso se repite. En caso contrario, después de que se hayan intentado todas las simplificaciones, devuelve la respuesta original.

La instrucción `example (scsimp)` muestra algunos ejemplos.

simpsum

Variable opcional

Valor por defecto: `false`

Si `simpsum` vale `true`, se simplifica el resultado de un sumatorio `sum`. Esta simplificación podrá producir en ocasiones una expresión compacta. Si `simpsum` vale `false` o si se utiliza la forma apostrofada `'sum`, el valor es una forma nominal que representa la notación sigma habitual en matemáticas.

sumcontract (*expr*)

Función

Combina todos los sumatorios de una suma cuyos límites inferiores y superiores difieren por constantes. El resultado es una expresión que contiene un sumatorio para conjunto de tales sumatorios. La función **sumcontract** combina todos los sumatorios compatibles y utiliza uno de los índices de uno de los sumatorios si puede, si no formará un índice que sea razonable.

Puede ser necesario hacer **intosum** (*expr*) antes que **sumcontract**.

sumexpand

Variable opcional

Valor por defecto: **false**

Si **sumexpand** vale **true**, productos de sumatorios y de sumatorios con exponentes se reducen a sumatorios anidados.

Véase también **cauchysum**.

Ejemplos:

```
(%i1) sumexpand: true$
(%i2) sum (f (i), i, 0, m) * sum (g (j), j, 0, n);
      m      n
      ====  ====
      \      \
      >      >      f(i1) g(i2)
      /      /
      ====  ====
      i1 = 0 i2 = 0
(%i3) sum (f (i), i, 0, m)^2;
      m      m
      ====  ====
      \      \
      >      >      f(i3) f(i4)
      /      /
      ====  ====
      i3 = 0 i4 = 0
```

sumsplitfact

Variable opcional

Valor por defecto: **true**

Si **sumsplitfact** vale **false**, **minfactorial** se aplica después de **factcomb**.

symmetric

Declaración

La instrucción **declare** (**h**, **symmetric**) le indica al simplificador de Maxima que **h** es una función simétrica. Por ejemplo, **h** (**x**, **z**, **y**) se reduce a **h** (**x**, **y**, **z**).

El nombre **commutative** es sinónimo de **symmetric**.

unknown (*expr*)

Función

Devuelve **true** si y sólo si *expr* contiene un operador o función no reconocido por el simplificador de Maxima.

8 Gráficos

8.1 Definiciones para Gráficos

in_netmath

Variable opcional

Valor por defecto: `false`

Si `in_netmath` vale `true`, `plot3d` imprime salida de OpenMath en la consola si `plot_format` vale `openmath`, en caso contrario, `in_netmath` (incluso si vale `true`) deja de tener efecto alguno.

La variable `in_netmath` no afecta a `plot2d`.

openplot_curves (*list*, *rest_options*)

Función

Toma una lista de curvas como

```
[[x1, y1, x2, y2, ...], [u1, v1, u2, v2, ...], ...]
```

o

```
[[[x1, y1], [x2, y2], ...], ...]
```

y las dibuja. Es similar a `xgraph_curves`, pero utiliza las rutinas de "open plot". Se le puede dar argumentos adicionales como "`{xrange -3 4}`". El siguiente ejemplo dibuja dos curvas con puntos grandes, etiquetando el primero con `jim` y el segundo con `jane`.

```
openplot_curves ([["{plotpoints 1} {pointsize 6} {label jim}
  {xlabel {joe is nice}}"], [1, 2, 3, 4, 5, 6, 7, 8],
  ["{label jane} {color pink }"], [3, 1, 4, 2, 5, 7]]);
```

Otros símbolos de elementos importantes son `xfun`, `color`, `plotpoints`, `linecolors`, `pointsize`, `nolines`, `bargraph`, `labelposition`, `xaxislabel` y `yaxislabel`.

plot2d (*expr*, *range*, ..., *options*, ...)

Función

plot2d (*parametric_expr*)

Función

plot2d (*discrete_expr*)

Función

plot2d (*[expr_1, ..., expr_n]*, *x_range*, *y_range*)

Función

plot2d (*[expr_1, ..., expr_n]*, *x_range*)

Función

plot2d (*expr*, *x_range*, *y_range*)

Función

plot2d (*expr*, *x_range*)

Función

plot2d (*[name_1, ..., name_n]*, *x_range*, *y_range*)

Función

plot2d (*[name_1, ..., name_n]*, *x_range*)

Función

plot2d (*name*, *x_range*, *y_range*)

Función

plot2d (*name*, *x_range*)

Función

Muestra un gráfico de una o más expresiones como función de una variable.

En todos los casos, *expr* es una expresión a ser representada en el eje vertical como función de una variable. El argumento *x_range*, que es el rango del eje horizontal, es una lista de la forma [*variable*, *min*, *max*], donde *variable* es una variable que aparece en *expr*. También *y_range*, el rango del eje vertical, es otra lista de la forma [*y*, *min*, *max*].

La llamada `plot2d (expr, x_range)` dibuja plots *expr* como función de la variable nombrada en *x_range*, en el rango especificado por *x_range*. Si el rango vertical

no se especifica explícitamente por medio de `set_plot_option`, se escogerá automáticamente. Todas las opciones toman sus valores por defecto a menos que se especifiquen de otra manera con `set_plot_option`.

La llamada `plot2d (expr, x_range, y_range)` dibuja `expr` como función de la variable nombrada en `x_range`, en el rango especificado por `x_range`. El rango vertical se ajusta a `y_range`. Todas las opciones toman sus valores por defecto a menos que se especifiquen de otra manera con `set_plot_option`.

La llamada `plot2d ([expr_1, ..., expr_n], x_range)` dibuja `expr_1, ..., expr_n` como función de la variable nombrada en `x_range`, en el rango especificado por `x_range`. Si el rango vertical no se especifica explícitamente por medio de `set_plot_option`, se escogerá automáticamente. Todas las opciones toman sus valores por defecto a menos que se especifiquen de otra manera con `set_plot_option`.

La llamada `plot2d ([expr_1, ..., expr_n], x_range, y_range)` dibuja `expr_1, ..., expr_n` como función de la variable nombrada en `x_range`, en el rango especificado por `x_range`. El rango vertical se ajusta a `y_range`. Todas las opciones toman sus valores por defecto a menos que se especifiquen de otra manera con `set_plot_option`.

Cuando la función a representar ha sido definida en Maxima mediante `:=` o `define`, o en Lisp por `DEFUN` o `DEFMFUN`, entonces se podrá especificar por su nombre. Las funciones definidas a nivel de LISP por `DEFMSPEC`, las funciones de simplificación, junto con muchas otras funciones, no pueden especificarse directamente por su nombre.

Ejemplos:

Gráfico de una expresión y especificación de algunos parámetros de uso común.

```
(%i1) plot2d (sin(x), [x, -5, 5])$
(%i2) plot2d (sec(x), [x, -2, 2], [y, -20, 20], [nticks, 200])$
```

Representando funciones por nombre.

```
(%i1) F(x) := x^2 $

(%i2) :lisp (defun |$g| (x) (m* x x x))

$g
(%i2) H(x) := if x < 0 then x^4 - 1 else 1 - x^5 $

(%i3) plot2d (F, [u, -1, 1])$

(%i4) plot2d ([F, G, H], [u, -1, 1])$
```

Allá donde vaya una expresión ordinaria, también puede ir una expresión paramétrica en su lugar: *parametric_expr* es una lista de la forma `[parametric, x_expr, y_expr, t_range, options]`. Aquí `x_expr` y `y_expr` son expresiones de una variable `var` que a su vez es el primer elemento del rango `trange`. El dibujo que se obtiene es el lugar geométrico de los pares `[x_expr, y_expr]` cuando `var` varía según `trange`.

En el siguiente ejemplo se dibuja un círculo, luego se repite lo mismo con pocos puntos, con lo que se obtiene una estrella, para finalmente dibujar todo junto con una función ordinaria en `x`.

Ejemplos:

- Un círculo como curva paramétrica.

```
(%i1) plot2d ([parametric, cos(t), sin(t), [t, -%pi*2, %pi*2],
[nticks, 80]])$
```
- Una estrella: la curva anterior con sólo ocho puntos.

```
(%i2) plot2d ([parametric, cos(t), sin(t), [t, -%pi*2, %pi*2],
[nticks, 8]])$
```
- Un polinomio cúbico como curva ordinaria y un círculo como curva paramétrica.

```
(%i3) plot2d ([x^3 + 2, [parametric, cos(t), sin(t), [t, -5, 5],
[nticks, 80]]], [x, -3, 3])$
```

También se pueden utilizar expresiones discretas en lugar de ordinarias o paramétricas: *discrete_expr* es una lista de la forma `[discrete, x_list, y_list]` o `[discrete, xy_list]`, siendo *xy_list* una lista de pares `[x,y]`.

Ejemplos:

- Se crean algunas listas.

```
(%i1) xx:makelist(x,x,0,10)$
(%i2) yy:makelist(exp(-x*1.0),x,0,10)$
(%i3) xy:makelist([x,x*x],x,0,5)$
```
- Dibujo con segmentos lineales.

```
(%i4) plot2d([discrete,xx,yy])$
```
- Dibujo con segmentos lineales, utilizando una lista de pares.

```
(%i5) plot2d([discrete,xy])$
```
- Dibujo con puntos.

```
(%i6) plot2d([discrete,xx,yy],[gnuplot_curve_styles,["with points"]])$
```
- Dibujo de la curva `cos(x)` con segmentos y `(xx,yy)` con puntos.

```
plot2d([cos(x),[discrete,xx,yy]],[x,0,10],[gnuplot_curve_styles,["with line"]])$
```

Véase también `plot_options`, que describe las opciones gráficas y tiene más ejemplos.

xgraph_curves (*list*)

Función

Dibuja el conjunto de puntos de la lista del argumento *list* con el programa `xgraph`.

El conjunto de puntos puede ser de la forma

```
[x0, y0, x1, y1, x2, y2, ...]
```

o

```
[[x0, y0], [x1, y1], ...]
```

Un conjunto de puntos también puede contener símbolos con etiquetas u otra información.

```
xgraph_curves ([pt_set1, pt_set2, pt_set3]);
```

dibuja los tres conjuntos de puntos como tres curvas.

```
pt_set: append (["NoLines: True", "LargePixels: true"], [x0, y0, x1, y1, ...])
```

construye el conjunto de puntos, declara que no haya segmentos rectilíneos entre ellos y que se utilicen píxeles grandes. Véase el manual de `xgraph` para más opciones.

`pt_set: append ([concat ("\"", "x^2+y"), [x0, y0, x1, y1, ...]);`
 construye una etiqueta con el contenido "x^2+y" para este conjunto particular de puntos. Las comillas dobles " al comienzo son las que le indican a xgraph que se trata de una etiqueta.

`pt_set: append ([concat ("TitleText: Datos muestrales"), [x0, ...])$`
 establece el título principal del gráfico como "Datos muestrales" en lugar de "Maxima Plot".

Para hacer un gráfico de barras con columnas de 0.2 unidades de ancho y para dibujar dos diagramas diferentes de este tipo:

```
xgraph_curves ([append ([\"BarGraph: true\", \"NoLines: true\", \"BarWidth: .2\"],
  create_list ([i - .2, i^2], i, 1, 3)),
  append ([\"BarGraph: true\", \"NoLines: true\", \"BarWidth: .2\"],
  create_list ([i + .2, .7*i^2], i, 1, 3))]);
```

Se utiliza un fichero temporal 'xgraph-out'.

plot_options

Variable del sistema

Los elementos de esta lista establecen las opciones por defecto para los gráficos. Si una opción está presente en una llamada a `plot2d` o a `plot3d`, este valor adquiere prevalencia sobre las opciones por defecto. En otro caso se utilizará el valor que tenga en `plot_options`. Las opciones por defecto se asignan mediante la función `set_plot_option`.

Cada elemento de `plot_options` es una lista de dos o más elementos, el primero de los cuales es el nombre de la opción, siendo los siguientes los valores de aquélla. En algunos casos el valor asignado es a su vez una lista, que puede contener varios elementos.

Las opciones gráficas que reconocen `plot2d` y `plot3d` son:

- Opción: `plot_format` determina qué módulo gráfico van a utilizar `plot2d` y `plot3d`.
 - Valor por defecto: `gnuplot` Gnuplot es el programa por defecto y el más avanzado. Requiere de una instalación externa de gnuplot.
 - Valor: `mgnuplot` Mgnuplot es una interface para gnuplot basada en Tk. Se incluye en la distribución de Maxima. Mgnuplot ofrece una interface gráfica de usuario rudimentaria para gnuplot, pero tiene algunas mejoras respecto de la interface propia de gnuplot. Mgnuplot requiere de una instalación externa de gnuplot y de Tcl/Tk.
 - Valor: `openmath` Openmath es un programa gráfico escrito en Tcl/Tk. Se incluye en la distribución de Maxima.
 - Valor: `ps` Genera ficheros PostScript directamente desde Maxima. Se tendrán resultados PostScript mejores utilizando gnuplot, dejando la opción `plot_format` sin especificar (aceptando la que tiene por defecto), y dándole a `gnuplot_term` el valor `ps`.
- Opción: `run_viewer` controla si el visor apropiado para la salida gráfica debe ejecutarse o no.
 - Valor por defecto: `true`, ejecuta el visor.

- Valor: **false**, no ejecuta el visor.
- **gnuplot_term** establece el terminal de gnuplot.
 - Valor por defecto: **default** La salida de gnuplot se muestra en una ventana gráfica.
 - Valor: **dumb** GLa salida de gnuplot se muestra en la consola de Maxima con caracteres ASCII.
 - Valor: **ps** Gnuplot genera comandos en lenguaje PostScript. Si la opción **gnuplot_out_file** vale *filename*, gnuplot escribe los comandos PostScript en *filename*. En otro caso, los comandos se imprimen en la consola de Maxima.
- Opción: **gnuplot_out_file** escribe la salida de gnuplot en un fichero.
 - Valor por defecto: **false** No se especifica ningún archivo de salida.
 - Valor: *filename* Ejemplo: `[gnuplot_out_file, "myplot.ps"]` Este ejemplo envía código PostScript al fichero `myplot.ps` cuando se utiliza conjuntamente con el terminal PostScript de gnuplot.
- Opción: **x** Rango horizontal por defecto.

`[x, - 3, 3]`

Establece el rango horizontal como `[-3, 3]`.
- Opción: **y** Rango vertical por defecto.

`[y, - 3, 3]`

Establece el rango vertical como `[-3, 3]`.
- Opción: **t** El rango por defecto para el parámetro de las representaciones paramétricas.

`[t, 0, 10]`

Establece el rango de la variable paramétrica como `[0, 10]`.
- Opción: **nticks** Número inicial de puntos a utilizar por el algoritmo adaptativo de representación gráfica.

`[nticks, 20]`

El valor por defecto para **nticks** es 10.
- Opción: **adapt_depth** Número máximo de particiones utilizado por el algoritmo adaptativo de representación gráfica.

`[adapt_depth, 5]`

El valor por defecto para **adapt_depth** es 10.
- Opción: **grid** Establece el número de puntos de la retícula a utilizar en las direcciones x e y en los gráficos de tres dimensiones.

`[grid, 50, 50]`

establece la retícula en 50 por 50 puntos. El valor por defecto es 30 por 30.
- Opción: **transform_xy** Permite que se realicen transformaciones en los gráficos de tres dimensiones.

`[transform_xy, false]`

El valor por defecto de **transform_xy** es **false**. Cuando vale **false**, da el resultado de

```
make_transform ([x, y, z], f1(x, y, z), f2(x, y, z), f3(x, y, z))$
```

La transformación `polar_xy` está definida en Maxima. Devuelve la misma transformación que

```
make_transform ([r, th, z], r*cos(th), r*sin(th), z)$
```

- Opción: `colour_z` es específico del formato gráfico `ps`.

```
[colour_z, true]
```

El valor por defecto de `colour_z` es `false`.

- Opción: `view_direction` Específico del formato gráfico `ps`.

```
[view_direction, 1, 1, 1]
```

El valor por defecto de `view_direction` es `[1, 1, 1]`.

Hay varias opciones gráficas que son específicas de `gnuplot`. Todas ellas (excepto `gnuplot_pm3d`) son comandos propios de `gnuplot` que se especifican como cadenas alfanuméricas. Consúltase la documentación de `gnuplot` para más detalles.

- Opción: `gnuplot_pm3d` Controla la utilización del modo PM3D, que tiene capacidades avanzadas para gráficos tridimensionales. PM3D sólo está disponible en versiones de `gnuplot` posteriores a la 3.7. El valor por defecto de `gnuplot_pm3d` es `false`.

Ejemplo:

```
[gnuplot_pm3d, true]
```

- Opción: `gnuplot_preamble` Introduce instrucciones de `gnuplot` antes de que se haga el dibujo. Puede utilizarse cualquier comando válido de `gnuplot`. Si interesa introducir varios comandos se separarán con punto y coma. El ejemplo que se muestra produce un gráfico en escala logarítmica. El valor por defecto de `gnuplot_preamble` es la cadena vacía `""`.

Ejemplo:

```
[gnuplot_preamble, "set log y"]
```

- Opción: `gnuplot_curve_titles`

Controla los títulos dados a la clave del gráfico. El valor por defecto es `[default]`, el cual establece automáticamente los títulos para cada curva representada. Si no es `[default]`, `gnuplot_curve_titles` debe contener una lista de cadenas, cada una de las cuales es `"title 'title_string'"`. (Para desactivar la clave del gráfico, añádase `"set nokey"` a `gnuplot_preamble`.)

Ejemplo:

```
[gnuplot_curve_titles, ["title 'My first function'", "title 'My second function'"]]
```

- Opción: `gnuplot_curve_styles` Es una lista de cadenas que controlan el aspecto de las curvas, como el color, el ancho, la discontinuidad, etc., y que deben enviarse al comando `plot` de `gnuplot`. El valor por defecto es `["with lines 3", "with lines 1", "with lines 2", "with lines 5", "with lines 4", "with lines 6", "with lines 7"]`, que realiza un ciclo sobre un conjunto de colores diferentes. Consúltase la documentación de `gnuplot` sobre `plot` para más información.

Ejemplo:

```
[gnuplot_curve_styles, ["with lines 7", "with lines 2"]]
```

- Opción: `gnuplot_default_term_command` Comando de gnuplot para establecer el tipo de terminal para el terminal por defecto. El valor por defecto es la cadena vacía "", esto es, utiliza el valor por defecto de gnuplot.

Ejemplo:

```
[gnuplot_default_term_command, "set term x11"]
```

- Opción: `gnuplot_dumb_term_command` Comando de gnuplot para establecer el tipo de terminal para el terminal oculto. El valor por defecto es "set term dumb 79 22", que da una salida de texto de 79 por 22 caracteres.

Ejemplo:

```
[gnuplot_dumb_term_command, "set term dumb 132 50"]
```

- Opción: `gnuplot_ps_term_command` Comando de gnuplot para establecer el tipo de terminal para el terminal PostScript. El valor por defecto es "set size 1.5, 1.5;set term postscript eps enhanced color solid 24", que establece un tamaño de 1.5 veces el valor por defecto de gnuplot, junto con un tamaño de fuente de 24, entre otras cosas. Consúltase la documentación de gnuplot para más información sobre `set term postscript`.

Ejemplo:

```
[gnuplot_ps_term_command, "set term postscript eps enhanced color solid 18"]
```

Ejemplos:

- Almacena un gráfico de $\sin(x)$ en el fichero `sin.eps`.

```
plot2d (sin(x), [x, 0, 2*%pi], [gnuplot_term, ps], [gnuplot_out_file, "sin.eps"])
```

- Utiliza la opción `gnuplot_preamble` para saltarse las singularidades, así como la opción `gnuplot_preamble` para colocar la clave en la parte inferior del dibujo.

```
plot2d ([gamma(x), 1/gamma(x)], [x, -4.5, 5], [y, -10, 10], [gnuplot_preamble,
```

- Utiliza un `gnuplot_preamble` bastante complicado a fin de producir etiquetas en el eje x. (Nótese que la cadena de `gnuplot_preamble` debe introducirse sin saltos de línea.)

```
my_preamble: "set xzeroaxis; set xtics ('-2pi' -6.283, '-3pi/2' -4.712, '-pi' -3.14159);
plot2d ([cos(x), sin(x), tan(x), cot(x)], [x, -2*%pi, 2*%pi],
[y, -2, 2], [gnuplot_preamble, my_preamble]);
```

- Utiliza un `gnuplot_preamble` bastante complicado a fin de producir etiquetas en el eje x, produciendo una salida PostScript que aprovecha el formateo avanzado de texto disponible en gnuplot. (Nótese que la cadena de `gnuplot_preamble` debe introducirse sin saltos de línea.)

```
my_preamble: "set xzeroaxis; set xtics ('-2{/Symbol p}' -6.283, '-3{/Symbol p}' -4.712, '-pi' -3.14159);
plot2d ([cos(x), sin(x), tan(x)], [x, -2*%pi, 2*%pi], [y, -2, 2],
[gnuplot_preamble, my_preamble], [gnuplot_term, ps], [gnuplot_out_file, "t.eps"])
```

- Un gráfico tridimensional utilizando el terminal `pm3d` de gnuplot.

```
plot3d (atan (-x^2 + y^3/4), [x, -4, 4], [y, -4, 4], [grid, 50, 50], [gnuplot_term,
```

- Un gráfico tridimensional sin malla y con contornos proyectados sobre el plano inferior.

```
my_preamble: "set pm3d at s;unset surface;set contour;set cntrparam levels 20;
plot3d (atan (-x^2 + y^3/4), [x, -4, 4], [y, -4, 4], [grid, 50, 50],
[gnuplot_pm3d, true], [gnuplot_preamble, my_preamble])$
```

- Un gráfico en el que el eje z sólo se representa por el color. (Nótese que la cadena de `gnuplot_preamble` debe introducirse sin saltos de línea.)

```
plot3d (cos (-x^2 + y^3/4), [x, -4, 4], [y, -4, 4],
[gnuplot_preamble, "set view map; unset surface"], [gnuplot_pm3d, true], [
```

plot3d (<i>expr</i> , <i>x_range</i> , <i>y_range</i> , ..., <i>options</i> , ...)	Función
plot3d ([<i>expr_1</i> , <i>expr_2</i> , <i>expr_3</i>], <i>x_range</i> , <i>y_range</i> , ..., <i>options</i> , ...)	Función
plot3d ([<i>expr_1</i> , <i>expr_2</i> , <i>expr_3</i>], <i>x_rge</i> , <i>y_rge</i>)	Function
plot3d ([<i>name_1</i> , <i>name_2</i> , <i>name_3</i>], <i>x_range</i> , <i>y_range</i> , ..., <i>options</i> , ...)	Function

Representa gráficamente una o tres expresiones como funciones de dos variables.

```
plot3d (2^(-u^2 + v^2), [u, -5, 5], [v, -7, 7]);
```

dibuja $z = 2^{-(u^2+v^2)}$ con u y v variando en $[-5,5]$ y $[-7,7]$ respectivamente, y con u sobre el eje x, y con v sobre el eje y.

Un ejemplo del tercer patrón de argumentos es

```
plot3d ([cos(x)*(3 + y*cos(x/2)), sin(x)*(3 + y*cos(x/2)), y*sin(x/2)],
[x, -%pi, %pi], [y, -1, 1], ['grid, 50, 15]);
```

que dibuja una banda de Moebius, parametrizada por las tres expresiones dadas como primer argumento a `plot3d`. Un argumento opcional `['grid, 50, 15]` da el número de intervalos en las direcciones x e y, respectivamente.

Cuando la función a representar ha sido definida en Maxima mediante `:=` o `define`, o en Lisp por `DEFUN` o `DEFMFUN`, entonces se podrá especificar por su nombre. Las funciones definidas a nivel de LISP por `DEFMSPEC`, las funciones de simplificación, junto con muchas otras funciones, no pueden especificarse directamente por su nombre.

Este ejemplo muestra un gráfico de la parte real de $z^{1/3}$.

```
plot3d (r^.33*cos(th/3), [r, 0, 1], [th, 0, 6*%pi],
['grid, 12, 80], ['plot_format, ps],
['transform_xy, polar_to_xy], ['view_direction, 1, 1, 1.4],
['colour_z, true]);
```

Aquí la opción `view_direction` indica la dirección desde la que se hace la proyección. Se hace esto desde una posición infinita pero paralela a la línea que va desde `view_direction` hasta el origen. Esto sólo se utiliza cuando `plot_format` está en `ps`, puesto que los otros visores permiten la rotación interactiva del objeto.

Otro ejemplo es la botella de Klein:

```
expr_1: 5*cos(x)*(cos(x/2)*cos(y) + sin(x/2)*sin(2*y) + 3.0) - 10.0;
expr_2: -5*sin(x)*(cos(x/2)*cos(y) + sin(x/2)*sin(2*y) + 3.0);
expr_3: 5*(-sin(x/2)*cos(y) + cos(x/2)*sin(2*y));
```

```
plot3d ([expr_1, expr_2, expr_3], [x, -%pi, %pi], [y, -%pi, %pi], ['grid, 40,
```

o un toro

```
expr_1: cos(y)*(10.0+6*cos(x));
expr_2: sin(y)*(10.0+6*cos(x));
expr_3: -6*sin(x);
```

```
plot3d ([expr_1, expr_2, expr_3], [x, 0, 2*pi], [y, 0, 2*pi], ['grid, 40, 40]
```

También se puede hacer con `gnuplot`:

```
plot3d (2^(x^2 - y^2), [x, -1, 1], [y, -2, 2], [plot_format, gnuplot]);■
```

En ocasiones puede ser necesario definir una función para representarla. Todos los argumentos de `plot3d` se evalúan, de manera que puede ser difícil escribir una expresión que haga lo que el usuario realmente quiere; en tales casos facilita las cosas definir previamente la función.

```
M: matrix([1, 2, 3, 4], [1, 2, 3, 2], [1, 2, 3, 4], [1, 2, 3, 3])$
f(x, y) := float (M [round(x), round(y)])$
plot3d (f, [x, 1, 4], [y, 1, 4], ['grid, 4, 4])$
```

Véase `plot_options` para más ejemplos.

make_transform (*vars, fx, fy, fz*)

Función

Devuelve una función apropiada para la función de transformación de `plot3d`. Debe usarse con la opción gráfica `transform_xy`.

```
make_transform ([r, th, z], r*cos(th), r*sin(th), z)$
```

es una transformación para pasar a coordenadas polares.

plot2d_ps (*expr, range*)

Función

Escribe en el *pstream* una secuencia de comandos PostScript que dibujan *expr* sobre *range*.

El argumento *expr* es una expresión y *range* es una lista de la forma `[x, min, max]` en la cual *x* es una variable que aparece en *expr*.

Véase también `closeps`.

closeps ()

Función

Esta función debería invocarse al final de una secuencia de comandos gráficos. Cierra el *pstream* y le asigna `nil`. También puede ser invocado antes de empezar un dibujo, para asegurar que se cierre el *pstream* si estaba abierto. Todas las instrucciones que se envían al *pstream* lo abren si es necesario. La función `closeps` es independiente de otras instrucciones gráficas, ya que si se quiere dibujar dos rangos o sobreponer varios gráficos, *pstream* debe permanecer abierto.

set_plot_option (*option*)

Función

Asigna un valor a una de las variables globales que controlan los gráficos. El argumento *option* se especifica como una lista de dos o más elementos, en la que el primero es el nombre de una de las opciones de la lista `plot_options`.

La función `set_plot_option` evalúa sus argumentos y devuelve `plot_options` tal como queda después de la actualización.

Véanse también `plot_options`, `plot2d` y `plot3d`.

Ejemplos:

Se modifican los valores de `grid` y `x`. Si a un nombre de opción de `plot_options` tiene ya un valor asignado, hacerlo preceder de un apóstrofo para evitar su evaluación.

```
(%i1) set_plot_option ([grid, 30, 40]);
(%o1) [[x, - 1.755559702014E+305, 1.755559702014E+305],
[y, - 1.755559702014E+305, 1.755559702014E+305], [t, - 3, 3],
[grid, 30, 40], [view_direction, 1, 1, 1], [colour_z, false],
[transform_xy, false], [run_viewer, true],
[plot_format, gnuplot], [gnuplot_term, default],
[gnuplot_out_file, false], [nticks, 10], [adapt_depth, 10],
[gnuplot_pm3d, false], [gnuplot_preamble, ],
[gnuplot_curve_titles, [default]],
[gnuplot_curve_styles, [with lines 3, with lines 1,
with lines 2, with lines 5, with lines 4, with lines 6,
with lines 7]], [gnuplot_default_term_command, ],
[gnuplot_dumb_term_command, set term dumb 79 22],
[gnuplot_ps_term_command, set size 1.5, 1.5;set term postscript #
eps enhanced color solid 24]]
(%i2) x: 42;
(%o2)
42
(%i3) set_plot_option ([x, -100, 100]);
(%o3) [[x, - 100.0, 100.0], [y, - 1.755559702014E+305,
1.755559702014E+305], [t, - 3, 3], [grid, 30, 40],
[view_direction, 1, 1, 1], [colour_z, false],
[transform_xy, false], [run_viewer, true],
[plot_format, gnuplot], [gnuplot_term, default],
[gnuplot_out_file, false], [nticks, 10], [adapt_depth, 10],
[gnuplot_pm3d, false], [gnuplot_preamble, ],
[gnuplot_curve_titles, [default]],
[gnuplot_curve_styles, [with lines 3, with lines 1,
with lines 2, with lines 5, with lines 4, with lines 6,
with lines 7]], [gnuplot_default_term_command, ],
[gnuplot_dumb_term_command, set term dumb 79 22],
[gnuplot_ps_term_command, set size 1.5, 1.5;set term postscript #
eps enhanced color solid 24]]
```

psdraw_curve (*ptlist*)

Función

Dibuja una curva uniendo los puntos de *ptlist*, que puede ser de la forma `[x0, y0, x1, y1, ...]` o `[[x0, y0], [x1, y1], ...]`

La función `join` se puede utilizar para tomar una lista con las *x* y otra con las *y* para luego formar pares.

La función `psdraw_curve` tan sólo llama a la función `pscurve`. Esta es su definición:

```
(defun $psdraw_curve (lis)
  (p "newpath")
  ($pscurve lis)
  (p "stroke"))
```

pscom (*cmd*)

Función

El argumento *cmd* se añade al fichero PostScript. Ejemplo:

```
pscom ("4.5 72 mul 5.5 72 mul translate 14 14 scale");
```


9 Entrada y Salida

9.1 Introducción a Entrada y Salida

9.2 Archivos

Un archivo no es más que una área de un cierto dispositivo de almacenamiento que contiene datos o texto. Los archivos se agrupan en los discos en "directorios", que son listas de archivos. Instrucciones que operan con archivos son: `save`, `load`, `loadfile`, `stringout`, `batch`, `demo`, `writefile`, `closefile` y `appendfile`.

9.3 Definiciones para Entrada y Salida

-- Variable del sistema

`__` es la expresión de entrada que está siendo actualmente evaluada. Esto es, mientras se está evaluando una expresión de entrada, `__` es igual a `expr`.

A `__` se le asigna la expresión de entrada antes de que ésta sea simplificada o evaluada. Sin embargo, el valor de `__` es simplificado, pero no evaluado, cuando su valor es mostrado en el terminal.

La variable `__` es reconocida por `batch`, pero no por `load`.

Véanse también `_` y `%`.

Ejemplos:

```
(%i1) print ("I was called as", __);
I was called as print(I was called as, __)
(%o1)          print(I was called as, __)
(%i2) foo (__);
(%o2)          foo(foo(__))
(%i3) g (x) := (print ("Current input expression =", __), 0);
(%o3) g(x) := (print("Current input expression =", __), 0)
(%i4) [aa : 1, bb : 2, cc : 3];
(%o4)          [1, 2, 3]
(%i5) (aa + bb + cc)/(dd + ee + g(x));
              cc + bb + aa
Current input expression = -----
              g(x) + ee + dd
                  6
(%o5)          -----
              ee + dd
```

- Variable del sistema

El símbolo `_` representa la última expresión de entrada (esto es, `%i1`, `%i2`, `%i3`, ...).

Al símbolo `_` se le asigna la expresión de entrada antes de que ésta sea simplificada o evaluada. Sin embargo, el valor de `_` se simplifica (pero no se evalúa) cuando se muestra en el terminal.

El símbolo `_` es reconocido por `batch`, pero no por `load`.

Véanse también `--` y `%`.

Ejemplos:

```
(%i1) 13 + 29;
(%o1)                                     42
(%i2) :lisp $_
((MPLUS) 13 29)
(%i2) _;
(%o2)                                     42
(%i3) sin (%pi/2);
(%o3)                                     1
(%i4) :lisp $_
((%SIN) ((MQUOTIENT) $%PI 2))
(%i4) _;
(%o4)                                     1
(%i5) a: 13$
(%i6) b: 29$
(%i7) a + b;
(%o7)                                     42
(%i8) :lisp $_
((MPLUS) $A $B)
(%i8) _;
(%o8)                                     b + a
(%i9) a + b;
(%o9)                                     42
(%i10) ev (_);
(%o10)                                    42
```

`%`

Variable del sistema

El símbolo `%` representa la expresión de salida (esto es, `%o1`, `%o2`, `%o3`, ...) más reciente calculada por Maxima, independientemente de que la haya mostrado o no.

El símbolo `%` es reconocido por `batch`, pero no por `load`.

Véanse también `_`, `%%` y `%th`.

`%%`

Variable del sistema

En una sentencia compuesta, como `block`, `lambda` o `(s1, ..., sn)`, `%%` es el valor de la sentencia previa. Por ejemplo,

```
block (integrate (x^5, x), ev (%%, x=2) - ev (%%, x=1));
block ([prev], prev: integrate (x^5, x), ev (prev, x=2) - ev (prev, x=1));
```

devuelven el mismo resultado $21/2$.

Una sentencia compuesta puede contener otras sentencias compuestas. Independientemente de que una sentencia sea simple o compuesta, `%%` es el valor de la sentencia previa. Por ejemplo,

```
block (block (a^n, %%*42), %%/6)
```

devuelve $7*a^n$.

Dentro de una sentencia compuesta, el valor de `%%` puede inspeccionarse en un punto de interrupción que se abra ejecutando la función `break`. Por ejemplo, en el punto de interrupción abierto por

```
block (a: 42, break ())$
```

introduciendo `%%`; se obtiene 42.

En la primera sentencia de una sentencia compuesta, o fuera de una sentencia compuesta, `%%` no está definido.

El símbolo `%%` es reconocido tanto por `batch` como por `load`.

Véase también `%`.

%edispflag

Variable opcional

Valor por defecto: `false`

Si `%edispflag` vale `true`, Maxima muestra `%e` elevado a un exponente negativo como un cociente. Por ejemplo, `%e-x` se muestra como `1/%ex`.

%th (i)

Función

Es el valor de la expresión de salida previa i -ésima. Esto es, si la siguiente expresión a calcular es la salida n -ésima, `%th (m)` es la salida $(n - m)$ -ésima.

La función `%th` es útil en archivos tipo `batch` o para hacer referencia a un grupo de expresiones de salida. Por ejemplo,

```
block (s: 0, for i:1 thru 10 do s: s + %th (i))$
```

asigna a `s` la suma de las diez últimas expresiones de salida.

La función `%th` es reconocida por `batch`, pero no por `load`.

Véase también `%`.

"?"

Special symbol

Como prefijo de una función o nombre de variable, `?` significa que el nombre es de Lisp, no de Maxima. Por ejemplo, `?round` representa la función de Lisp `ROUND`. Véase `Lisp` y `Maxima` para más información.

La notación `? word` (un símbolo de interrogación seguido de una palabra y separados por un espacio) equivale a `describe ("word")`.

absboxchar

Variable opcional

Valor por defecto: `!`

La variable `absboxchar` es el carácter utilizado para representar el valor absoluto de una expresión que ocupa más de una línea de altura.

file_output_append

Variable opcional

Valor por defecto: `false`

La variable `file_output_append` controla si las funciones de escritura de ficheros añaden información o sustituyen el fichero de salida. Cuando `file_output_append` toma el valor `true`, estas funciones amplían el contenido de sus ficheros de salida; en otro caso, sustituyen el fichero anterior de igual nombre por otro con el nuevo contenido.

Las funciones `save`, `stringout` y `with_stdout` se ven afectadas por el valor que tome la variable `file_output_append`. Otras funciones que también escriben en ficheros de salida no tienen en cuenta este valor; en concreto, las funciones para la representación de gráficos y las de traducción siempre sustituyen el fichero anterior por uno nuevo de igual nombre, mientras que las funciones `tex` y `appendfile` siempre añaden información al fichero de salida sin eliminar la información anterior.

appendfile (*filename*)

Función

Añade información de la consola a *filename*, de igual manera que lo hace `writefile`, pero con la salvedad de que si el archivo ya existe la información queda añadida al final de su contenido.

La función `closefile` cierra los archivos abiertos por `appendfile` o `writefile`.

batch (*filename*)

Función

Lee expresiones de Maxima desde *filename* y las evalúa. La función `batch` busca *filename* en la lista `file_search_maxima`. Véase `file_search`.

El contenido de *filename* debe ser una secuencia de expresiones de Maxima, cada una de las cuales termina en `;` o `$`. La variable especial `%` y la función `%th` se refieren a resultados previos dentro del archivo. El archivo puede incluir construcciones del tipo `:lisp`. Espacios, tabulaciones y saltos de línea en el archivo se ignoran. Un archivo de entrada válido puede crearse con un editor de texto o con la función `stringout`.

La función `batch` lee las expresiones del archivo *filename*, muestra las entradas en la consola, realiza los cálculos solicitados y muestra las expresiones de los resultados. A las expresiones de entrada se les asignan etiquetas, así como a las de salida. La función `batch` evalúa todas las expresiones de entrada del archivo a menos que se produzca un error. Si se le solicita información al usuario (con `asksign` o `askinteger`, por ejemplo) `batch` se detiene para leer la nueva información para luego continuar.

Es posible detener `batch` tecleando `control-C` desde la consola. El efecto de `control-C` depende del entorno Lisp instalado.

La función `batch` tiene diversas aplicaciones, tales como servir de almacén de líneas de instrucciones, suministrar demostraciones libres de errores o ayudar a organizar el trabajo del usuario en la resolución de problemas complejos.

La función `batch` evalúa su argumento.

Véanse también `load`, `batchload` y `demo`.

batchload (*filename*)

Función

Lee expresiones de Maxima desde *filename* y las evalúa sin mostrar las entradas ni las salidas y sin asignarles etiquetas. Sin embargo, las salidas producidas por `print` o `describe` sí se muestran.

La variable especial `%` y la función `%th` se refieren a resultados previos del intérprete interactivo, no a los del propio archivo. El archivo no puede incluir construcciones del tipo `:lisp`.

La función `batchload` devuelve la ruta de *filename* en formato de cadena.

La función `batchload` evalúa sus argumentos.

Véanse también `batch` y `load`.

closefile () Función
 La función `closefile` cierra los archivos abiertos por `appendfile` o `writefile`.

concat (*arg-1*, *arg-2*, ...) Función
 Concatena sus argumentos, que deben ser todos átomos. El valor devuelto es un símbolo si el primer argumento es a su vez un símbolo, o una cadena de Maxima en caso contrario.

La función `concat` evalúa sus argumentos. El apátrofo ' evita la evaluación.

```
(%i1) y: 7$
(%i2) z: 88$
(%i3) concat (y, z/2);
(%o3)
744
(%i4) concat ('y, z/2);
(%o4)
y44
```

A un símbolo construido por `concat` se le puede asignar un valor y ser utilizado posteriormente en expresiones. La asignación con el operador `::` evalúa su expresión izquierda.

```
(%i5) a: concat ('y, z/2);
(%o5)
y44
(%i6) a:: 123;
(%o6)
123
(%i7) y44;
(%o7)
123
(%i8) b^a;
(%o8)
y44
b
(%i9) %, numer;
(%o9)
123
b
```

Nótese que aunque `concat (1, 2)` parezca un número, se trata de una cadena de Maxima.

```
(%i10) concat (1, 2) + 3;
(%o10)
12 + 3
```

sconcat (*arg-1*, *arg-2*, ...) Función
 Concatena sus argumentos para producir una cadena. Al contrario que `concat`, sus argumentos *no* necesitan ser átomos.
 El resultado es una cadena de Lisp.

```
(%i1) sconcat ("xx[" , 3, "]" : , expand ((x+y)^3));
(%o1)
xx[3]:y^3+3*x*y^2+3*x^2*y+x^3
```

disp (*expr-1*, *expr-2*, ...) Función
 Es como `display` pero sólo se muestran los valores de los argumentos, no las ecuaciones. Es útil para argumentos complicados que no tienen nombre o en situaciones en las que solamente es de interés el valor del argumento pero no su nombre.

dispcon (*tensor_1*, *tensor_2*, ...)

Función

dispcon (*all*)

Función

Muestra las propiedades contractivas de sus argumentos tal como fueron asignadas por **defcon**. La llamada **dispcon** (*all*) muestra todas propiedades contractivas que fueron definidas.

display (*expr_1*, *expr_2*, ...)

Función

Muestra las ecuaciones cuyos miembros izquierdos son *expr_i* sin evaluar y cuyos miembros derechos son los valores de las expresiones. Esta función es útil en los bloques y en las sentencias **for** para mostrar resultados intermedios. Los argumentos de **display** suelen ser átomos, variables subíndicadas o llamadas a funciones. Véase también **disp**.

```
(%i1) display(B[1,2]);
```

$$B_{1,2} = X - X^2$$

```
(%o1)
```

```
done
```

display2d

Variable opcional

Valor por defecto: **true**

Si **display2d** vale **false**, la salida por consola es una cadena unidimensional, en lugar de una expresión bidimensional.

display_format_internal

Variable opcional

Valor por defecto: **false**

Si **display_format_internal** vale **true**, las expresiones se muestran sin ser transformadas de manera que oculten su representación matemática interna. Se representa lo que la función **inpart** devolvería, en oposición a **part**.

Ejemplos:

User	part	inpart
a-b;	A - B	A + (- 1) B
a/b;	$\frac{A}{B}$	$\frac{A}{B} - 1$
sqrt(x);	sqrt(X)	$X^{1/2}$
X*4/3;	$\frac{4X}{3}$	$\frac{4}{3} X$

dispterm (*expr*)

Función

Muestra *expr* en partes, una debajo de la otra. Esto es, primero se muestra el operador de *expr*, luego cada término si se trata de una suma, o cada factor si es un producto, o si no se muestra separadamente la parte de una expresión más general. Es útil

si *expr* es demasiado grande para representarla de otra forma. Por ejemplo, si *P1*, *P2*, ... son expresiones muy grandes, entonces el programa de representación puede superar el espacio de almacenamiento tratando de mostrar *P1 + P2 + ...* todo junto. Sin embargo, `dispterm`s (*P1 + P2 + ...*) muestra *P1*, debajo *P2*, etc. Cuando una expresión exponencial es demasiado ancha para ser representada como A^B , si no se utiliza `dispterm`s, entonces aparecerá como `expt (A, B)` (o como `ncexpt (A, B)`, en lugar de $A^{^B}$).

error_size

Variable opcional

Valor por defecto: 10

La variable `error_size` modifica los mensajes de error de acuerdo con el tamaño de las expresiones que aparecen en él. Si el tamaño de una expresión (tal como lo determina la función Lisp `ERROR-SIZE`) es mayor que `error_size`, la expresión se reemplaza en el mensaje por un símbolo, asignándole a éste una expresión. Los símbolos se toman de la lista `error_syms`.

En caso contrario, si la expresión es menor que `error_size`, la expresión se muestra en el propio mensaje.

Véanse también `error` y `error_syms`.

Ejemplo:

El tamaño de *U*, tal como lo determina `ERROR-SIZE`, es 24.

```
(%i1) U: (C^D^E + B + A)/(cos(X-1) + 1)$
```

```
(%i2) error_size: 20$
```

```
(%i3) error ("Example expression is", U);
```

```
Example expression is errexp1
```

```
-- an error. Quitting. To debug this try debugmode(true);
```

```
(%i4) errexp1;
```

```
(%o4)
      E
      D
      C  + B + A
      -----
      cos(X - 1) + 1
```

```
(%i5) error_size: 30$
```

```
(%i6) error ("Example expression is", U);
```

```

      E
      D
      C  + B + A
Example expression is -----
      cos(X - 1) + 1
```

```
-- an error. Quitting. To debug this try debugmode(true);
```

error_syms

Variable opcional

Valor por defecto: [`errexp1`, `errexp2`, `errexp3`]

En los mensajes de error, las expresiones mayores que `error_size` son reemplazadas por símbolos a los cuales se les asignan estas expresiones. Los símbolos se toman de la lista `error_syms`. La primera expresión que resulte ser demasiado larga se reemplaza por `error_syms[1]`, la segunda por `error_syms[2]` y así sucesivamente.

Si hay más expresiones largas que elementos en `error_syms`, los símbolos se construyen automáticamente, siendo el n -ésimo símbolo equivalente a `concat ('errexpr', n)`.

Véanse también `error` y `error_size`.

expt (*a*, *b*) Función

Si una expresión exponencial es demasiado ancha para ser mostrada como a^b aparecerá como `expt (a, b)` (o como `ncexpt (a, b)` en lugar de a^{b^b}).

Las funciones `expt` y `ncexpt` no se reconocen en una entrada.

exptdispflag Variable opcional

Valor por defecto: `true`

Si `exptdispflag` vale `true`, Maxima muestra las expresiones con exponentes negativos como cocientes; por ejemplo, X^{-1} se muestra como $1/X$.

filename_merge (*path*, *filename*) Función

Construye una ruta modificada a partir de *path* y *filename*. Si la componente final de *path* es de la forma `###.something`, la componente se reemplaza con *filename.something*. En otro caso, la componente final se reemplaza simplemente por *filename*.

file_search (*filename*) Función

file_search (*filename*, *pathlist*) Función

La función `file_search` busca el archivo *filename* y devuelve su ruta como una cadena; si no lo encuentra, `file_search` devuelve `false`. La llamada `file_search (filename)` busca en los directorios de búsqueda por defecto, que son los especificados por las variables `file_search_maxima`, `file_search_lisp` y `file_search_demo`.

La función `file_search` analiza primero si el nombre del argumento existe antes de hacerlo coincidir con los comodines de los patrones de búsqueda de archivos. Véase `file_search_maxima` para más información sobre patrones de búsqueda de archivos.

El argumento *filename* puede ser una ruta con nombre de archivo, o simplemente el nombre del archivo, o, si el directorio de búsqueda de archivo incluye un patrón de búsqueda, es suficiente con el nombre de archivo sin extensión. Por ejemplo,

```
file_search ("/home/wfs/special/zeta.mac");
file_search ("zeta.mac");
file_search ("zeta");
```

todos buscan el mismo archivo, dando por hecho que el archivo existe y que `/home/wfs/special/###.mac` está en `file_search_maxima`.

La llamada `file_search (filename, pathlist)` busca solamente en los directorios especificados por *pathlist*, que es una lista de cadenas. El argumento *pathlist* ignora los directorios de búsqueda por defecto, de manera que si se da la lista de rutas,

`file_search` busca solamente en ellas y no en los directorios por defecto. Incluso si hay un único directorio en *pathlist*, debe ser suministrado como una lista de un único elemento.

El usuario puede modificar los directorios de búsqueda por defecto; véase para ello See `file_search_maxima`.

La función `file_search` es llamada por `load` con los directorios de búsqueda `file_search_maxima` y `file_search_lisp`.

file_search_maxima	Variable opcional
file_search_lisp	Variable opcional
file_search_demo	Variable opcional

Estas variables especifican listas de directorios en los que deben buscar la funciones `load`, `demo` y algunas otras. Los valores por defecto de estas variables nombran directorios de la instalación de Maxima.

El usuario puede modificar estas variables, bien reemplazando los valores por defecto, bien añadiendo nuevos directorios. Por ejemplo,

```
file_search_maxima: ["/usr/local/foo/###.mac",
"/usr/local/bar/###.mac"]$
```

reemplaza el valor por defecto de `file_search_maxima`, mientras que

```
file_search_maxima: append (file_search_maxima,
"/usr/local/foo/###.mac", "/usr/local/bar/###.mac")$
```

añade dos directorios más. Puede ser conveniente colocar una expresión como esta en el archivo `maxima-init.mac`, de manera que la ruta de búsqueda de ficheros se asigne automáticamente cada vez que arranca Maxima.

Se pueden especificar varias extensiones de archivos y rutas con comodines especiales. La cadena `###` representa el nombre del archivo buscado y una lista separada de comas y encerrada entre llaves, `{foo,bar,baz}` representa múltiples cadenas. Por ejemplo, suponiendo que se busca el nombre `neumann`,

```
"/home/{wfs,gcj}/###.{lisp,mac}"
```

se interpreta como `/home/wfs/neumann.lisp`, `/home/gcj/neumann.lisp`, `/home/wfs/neumann.mac` y `/home/gcj/neumann.mac`.

file_type (<i>filename</i>)	Función
--------------------------------------	---------

Devuelve una descripción del contenido de *filename* basada en la extensión, sin intentar abrir el archivo para inspeccionar su contenido.

El valor que la función retorna puede ser cualquiera de los siguientes: `object`, `lisp` o `maxima`. Si la extensión comienza por `m` o `d`, `file_type` devuelve `maxima`. Si la extensión comienza por `l`, `file_type` devuelve `lisp`. En cualquier otro caso, `file_type` devuelve `object`.

grind (<i>expr</i>)	Función
grind	Variable opcional

La función `grind` imprime *expr* en la consola en un formato admisible como entrada para Maxima. La función `grind` devuelve siempre `done`.

Cuando *expr* es el nombre de una función o macro, **grind** muestra la definición de la función o de la macro en lugar de sólo su nombre.

Véase también **string**, que devuelve una cadena en lugar de imprimir la salida. La función **grind** intenta imprimir la expresión de forma que sea algo más sencilla de leer que la salida de **string**.

Cuando la variable **grind** vale **true**, la salida de **string** y **stringout** tienen el mismo formato que la de **grind**; en caso contrario no se formatea la salida de esas funciones. El valor por defecto de la variable **grind** es **false**.

La variable **grind** también se puede utilizar como argumento en **playback**. Si **grind** está presente, **playback** imprime las expresiones de entrada en el mismo formato que lo hace la función **grind**; en caso contrario no se formatean la expresiones de entrada.

La función **grind** evalúa sus argumentos.

Ejemplos:

```
(%i1) aa + 1729;
(%o1) aa + 1729
(%i2) grind (%);
aa+1729$
(%o2) done
(%i3) [aa, 1729, aa + 1729];
(%o3) [aa, 1729, aa + 1729]
(%i4) grind (%);
[aa,1729,aa+1729]$
(%o4) done
(%i5) matrix ([aa, 17], [29, bb]);
[ aa 17 ]
(%o5) [
[ 29 bb ]
]
(%i6) grind (%);
matrix([aa,17],[29,bb])$
(%o6) done
(%i7) set (aa, 17, 29, bb);
(%o7) {17, 29, aa, bb}
(%i8) grind (%);
{17,29,aa,bb}$
(%o8) done
(%i9) exp (aa / (bb + 17)^29);
aa
-----
29
(bb + 17)
(%o9) %e
(%i10) grind (%);
%e^(aa/(bb+17)^29)$
(%o10) done
(%i11) expr: expand ((aa + bb)^10);
10 9 2 8 3 7 4 6
(%o11) bb + 10 aa bb + 45 aa bb + 120 aa bb + 210 aa bb
```

```

      5 5      6 4      7 3      8 2
+ 252 aa bb + 210 aa bb + 120 aa bb + 45 aa bb
      9      10
+ 10 aa bb + aa
(%i12) grind (expr);
bb^10+10*aa*bb^9+45*aa^2*bb^8+120*aa^3*bb^7+210*aa^4*bb^6
+252*aa^5*bb^5+210*aa^6*bb^4+120*aa^7*bb^3+45*aa^8*bb^2
+10*aa^9*bb+aa^10$
(%o12)
done
(%i13) string (expr);
(%o13) bb^10+10*aa*bb^9+45*aa^2*bb^8+120*aa^3*bb^7+210*aa^4*bb^6\
+252*aa^5*bb^5+210*aa^6*bb^4+120*aa^7*bb^3+45*aa^8*bb^2+10*aa^9*\
bb+aa^10
(%i14) cholesky (A):= block ([n : length (A), L : copymatrix (A),
p : makelist (0, i, 1, length (A))], for i thru n do for j : i thru n do
(x : L[i, j], x : x - sum (L[j, k] * L[i, k], k, 1, i - 1), if i = j then
p[i] : 1 / sqrt(x) else L[j, i] : x * p[i]), for i thru n do L[i, i] : 1 / p[i]
for i thru n do for j : i + 1 thru n do L[i, j] : 0, L)$
(%i15) grind (cholesky);
cholesky(A):=block(
[n:length(A),L:copymatrix(A),
p:makelist(0,i,1,length(A))],
for i thru n do
(for j from i thru n do
(x:L[i,j],x:x-sum(L[j,k]*L[i,k],k,1,i-1),
if i = j then p[i]:1/sqrt(x)
else L[j,i]:x*p[i])),
for i thru n do L[i,i]:1/p[i],
for i thru n do (for j from i+1 thru n do L[i,j]:0),L)$
(%o15)
done
(%i16) string (fundef (cholesky));
(%o16) cholesky(A):=block([n:length(A),L:copymatrix(A),p:makelis\
t(0,i,1,length(A))],for i thru n do (for j from i thru n do (x:L\
[i,j],x:x-sum(L[j,k]*L[i,k],k,1,i-1),if i = j then p[i]:1/sqrt(x)\
) else L[j,i]:x*p[i])),for i thru n do L[i,i]:1/p[i],for i thru \
n do (for j from i+1 thru n do L[i,j]:0),L)

```

ibase

Variable opcional

Valor por defecto: 10

Los enteros que se escriban en Maxima se interpretan como dados en base **ibase**.

A la variable **ibase** se le puede asignar cualquier entero entre 2 y 35 (decimal), ambos inclusive. Si **ibase** es mayor que 10, las cifras serán los números decimales entre 0 y 9, más las letras mayúsculas A, B, C, ..., según sea necesario. Las cifras en base 35, la mayor base que acepta Maxima, son los dígitos de 0 a 9 y las letras de A a Y.

Véase también **obase**.**inchar**

Variable opcional

Valor por defecto: %i

La variable `inchar` es el prefijo de las etiquetas de las expresiones introducidas por el usuario. Maxima crea automáticamente una etiqueta para cada expresión de entrada concatenando `inchar` y `linenum`; a `inchar` se le puede asignar cualquier símbolo o cadena, no necesariamente un carácter sencillo.

```
(%i1) inchar: "input";
(%o1)                                     input
(input1) expand ((a+b)^3);
                                     3      2      2      3
(%o1)                                b  + 3 a b  + 3 a  b + a
(input2)
```

Véase también `labels`.

ldisp (*expr_1*, ..., *expr_n*)

Función

Muestra las expresiones *expr_1*, ..., *expr_n* en la consola con el formato de salida; `ldisp` asigna una etiqueta a cada argumento y devuelve la lista de etiquetas.

Véase también `disp`.

```
(%i1) e: (a+b)^3;
(%o1)                                     3
                                     (b + a)
(%i2) f: expand (e);
(%o2)                                3      2      2      3
                                     b  + 3 a b  + 3 a  b + a
(%i3) ldisp (e, f);
(%t3)                                3
                                     (b + a)
(%t4)                                3      2      2      3
                                     b  + 3 a b  + 3 a  b + a
(%o4)                                [%t3, %t4]
(%i4) %t3;
(%o4)                                3
                                     (b + a)
(%i5) %t4;
(%o5)                                3      2      2      3
                                     b  + 3 a b  + 3 a  b + a
```

ldisplay (*expr_1*, ..., *expr_n*)

Función

Muestra las expresiones *expr_1*, ..., *expr_n* en la consola con el formato de salida. Cada expresión se muestra como una ecuación de la forma `lhs = rhs` en la que `lhs` es uno de los argumentos de `ldisplay` y `rhs` su valor. Normalmente, cada argumento será el nombre de una variable. La función `ldisp` asigna una etiqueta a cada ecuación y devuelve la lista de etiquetas.

Véase también `display`.

```
(%i1) e: (a+b)^3;
(%o1)                                3
                                     (b + a)
```

```
(%i2) f: expand (e);
(%o2)          3      2      2      3
          b  + 3 a b  + 3 a  b + a
(%i3) ldisplay (e, f);
(%t3)          3
          e = (b + a)
(%t4)          3      2      2      3
          f = b  + 3 a b  + 3 a  b + a
(%o4)          [%t3, %t4]
(%i4) %t3;
(%o4)          3
          e = (b + a)
(%i5) %t4;
(%o5)          3      2      2      3
          f = b  + 3 a b  + 3 a  b + a
```

linechar

Variable opcional

Valor por defecto: %t

La variable **linechar** es el prefijo de las etiquetas que genera Maxima para expresiones intermedias. Cuando sea necesario, Maxima creará una etiqueta para cada expresión intermedia concatenando **linechar** y **linenum**. A **linechar** se le puede asignar cualquier cadena o símbolo, no necesariamente un carácter simple.

Las expresiones intermedias pueden ser mostradas o no. Véanse también **programmode** y **labels**.

linel

Variable opcional

Valor por defecto: 79

La variable **linel** es la anchura (medida en número de caracteres) de la consola que se le da a Maxima para que muestre las expresiones. A **linel** se le puede asignar cualquier valor, pero si éste es muy pequeño o grande resultará de poca utilidad. El texto que impriman las funciones internas de Maxima, como los mensajes de error y las salidas de la función **describe**, no se ve afectado por el valor de **linel**.

lispdisp

Variable opcional

Valor por defecto: false

Si **lispdisp** vale **true**, los símbolos de Lisp se muestran precedidos del carácter de interrogación ?. En caso contrario, los símbolos de Lisp se muestran sin esta marca.

Ejemplos:

```
(%i1) lispdisp: false$
(%i2) ?foo + ?bar;
(%o2)          foo + bar
(%i3) lispdisp: true$
(%i4) ?foo + ?bar;
(%o4)          ?foo + ?bar
```

load (*filename*)

Función

Evalúa las expresiones del archivo *filename*, trayendo variables, funciones y otros objetos a Maxima. Una asignación hecha previamente a una variable en Maxima será destruida por otra asignación que se le haga en *filename*. Para encontrar el fichero, **load** llama a **file_search** con **file_search_maxima** y **file_search_lisp** como directorios de búsqueda. Si la llamada a **load** funciona correctamente, devuelve el nombre del fichero; en caso contrario, **load** muestra un mensaje de error.

La función **load** trabaja indistintamente con código Lisp y Maxima. Los ficheros creados con **save**, **translate_file** y **compile_file**, que crea código Lisp, y **stringout**, que crea código Maxima, todos ellos pueden ser procesados por **load**. La función **load** llama a **loadfile** para cargar archivos en Lisp y a **batchload** para cargar archivos en Maxima.

Véanse también **loadfile**, **batch**, **batchload** y **demo**; **loadfile** procesa archivos en Lisp; **batch**, **batchload** y **demo** procesan archivos en Maxima.

Véase **file_search** para más detalles sobre el mecanismo de búsqueda de archivos.

La función **load** evalúa sus argumentos.

loadfile (*filename*)

Función

Evalúa las expresiones Lisp del archivo *filename*. La función **loadfile** no llama a **file_search**, de manera que *filename* debe incluir la extensión del archivo y su ruta completa.

La función **loadfile** puede procesar ficheros creados por **save**, **translate_file** y **compile_file**. Puede ser más conveniente utilizar **load** en lugar de **loadfile**.

La función **loadfile** no evalúa su argumento, de manera que *filename* debe ser una cadena, no una variable que la contenga. La utilización de doble comilla simple sí permite la evaluación.

loadprint

Variable opcional

Valor por defecto: **true**

La variable **loadprint** indica si mostrar un mensaje cuando se carga un archivo.

- Si **loadprint** vale **true**, se muestra siempre un mensaje.
- Si **loadprint** vale **'loadfile**, muestra un mensaje sólo si el archivo es cargado con la función **loadfile**.
- Si **loadprint** vale **'autoload**, muestra un mensaje sólo cuando un archivo se carga automáticamente. Véase **setup_autoload**.
- Si **loadprint** vale **false**, nunca mostrará mensajes.

obase

Variable opcional

Valor por defecto: 10

La variable **obase** es la base de los enteros mostrados por Maxima; se le puede asignar cualquier entero entre 2 y 35 (decimal), ambos inclusive. Si **obase** es mayor que 10, las cifras serán los números decimales entre 0 y 9, más las letras mayúsculas A, B, C, ..., según sea necesario. Las cifras en base 35, la mayor base que acepta Maxima, son los dígitos de 0 a 9 y las letras de A a Y.

Véase también **ibase**.

outchar

Variable opcional

Valor por defecto: %o

La variable **outchar** es el prefijo de las etiquetas de las expresiones calculadas por Maxima. Maxima crea automáticamente una etiqueta para cada expresión calculada concatenando **outchar** y **linenum**; a **outchar** se le puede asignar cualquier símbolo o cadena, no necesariamente un carácter sencillo.

```
(%i1) outchar: "output";
(output1)                                     output
(%i2) expand ((a+b)^3);
                                     3      2      2      3
(output2)                             b  + 3 a b  + 3 a  b  + a
(%i3)
```

Véase también **labels**.**packagefile**

Variable opcional

Valor por defecto: **false**

Los desarrolladores de paquetes que utilizan **save** o **translate** para crear paquetes (ficheros) que van a ser utilizados por terceros pueden hacer **packagefile: true** para evitar que se añada información a la listas de información de Maxima (por ejemplo, **values**, **functions**) excepto allí donde sea necesario cuando el archivo sea cargado en memoria.

pformat

Variable opcional

Valor por defecto: **false**

Si **pformat** vale **true**, una fracción de enteros será mostrada con el carácter de barra inclinada / entre ellos.

```
(%i1) pformat: false$
(%i2) 2^16/7^3;
                                     65536
(%o2) -----
                                     343
(%i3) (a+b)/8;
                                     b + a
(%o3) -----
                                     8
(%i4) pformat: true$
(%i5) 2^16/7^3;
                                     65536/343
(%i6) (a+b)/8;
                                     1/8 (b + a)
(%o6)
```

print (*expr_1*, ..., *expr_n*)

Función

Evalúa y muestra las expresiones *expr_1*, ..., *expr_n* secuencialmente de izquierda a derecha, comenzando la impresión por el borde izquierdo de la consola.

El valor devuelto por **print** es el valor de su último argumento. La función **print** no genera etiquetas para las expresiones intermedias.

Véanse también `display`, `disp`, `ldisplay` y `ldisp`, que muestran una expresión por línea, mientras que `print` trata de mostrar dos o más expresiones por línea.

Para mostrar el contenido de un archivo véase `printfile`.

```
(%i1) r: print ("(a+b)^3 is", expand ((a+b)^3), "log (a^10/b) is", radcan (log
      3      2      2      3
(a+b)^3 is b  + 3 a b  + 3 a  b + a  log (a^10/b) is

      10 log(a) - log(b)

(%i2) r;
(%o2)      10 log(a) - log(b)
(%i3) disp ("(a+b)^3 is", expand ((a+b)^3), "log (a^10/b) is", radcan (log (a^
      (a+b)^3 is

      3      2      2      3
b  + 3 a b  + 3 a  b + a

      log (a^10/b) is

      10 log(a) - log(b)
```

read (*expr_1*, ..., *expr_n*)

Función

Imprime *expr_1*, ..., *expr_n* y a continuación lee una expresión desde la consola y devuelve la expresión evaluada. La expresión termina con un punto y coma ; o con el símbolo de dólar \$.

Véase también `readonly`.

```
(%i1) foo: 42$
(%i2) foo: read ("foo vale", foo, " -- nuevo valor.")$
foo vale 42 -- nuevo valor.
(a+b)^3;
(%i3) foo;

      3
(%o3) (b + a)
```

readonly (*expr_1*, ..., *expr_n*)

Función

Imprime *expr_1*, ..., *expr_n* y a continuación lee una expresión desde la consola y devuelve la expresión sin evaluar. La expresión termina con un punto y coma ; o con el símbolo de dólar \$.

```
(%i1) aa: 7$
(%i2) foo: readonly ("Introducir expresion:");
Introducir expresion:
2^aa;

      aa
(%o2) 2
(%i3) foo: read ("Introducir expresion:");
Introducir expresion:
2^aa;
(%o3) 128
```

Véase también `read`.

reveal (*expr*, *nivel*)

Función

Reemplaza partes de *expr* al *nivel* especificado y las sustituye por descripciones cortas.

- Las sumas y restas se reemplazan por **sum**(*n*), siendo *n* el número de términos de la suma.
- Los productos se reemplazan por **product**(*n*), siendo *n* el número de factores del producto.
- Las potencias se reemplazan por **expt**.
- Los cocientes se reemplazan por **quotient**.
- El símbolo negativo se reemplaza por **negterm**.

Si el entero *depth* es mayor o igual que la profundidad máxima de *expr*, **reveal** (*expr*, *depth*) devuelve *expr* sin modificar.

La función **reveal** evalúa sus argumentos y devuelve la expresión con las modificaciones solicitadas.

Ejemplo:

```
(%i1) e: expand ((a - b)^2)/expand ((exp(a) + exp(b))^2);
```

```
(%o1)
      2      2
      b  - 2 a b + a
-----
      2      2      2
      b + a  2 b  2 a
2 %e      + %e  + %e
```

```
(%i2) reveal (e, 1);
```

```
(%o2) quotient
```

```
(%i3) reveal (e, 2);
```

```
(%o3)
      sum(3)
      -----
      sum(3)
```

```
(%i4) reveal (e, 3);
```

```
(%o4)
      expt + negterm + expt
      -----
      product(2) + expt + expt
```

```
(%i5) reveal (e, 4);
```

```
(%o5)
      2      2
      b  - product(3) + a
      -----
      2      2      2
      product(2) product(2)
2 expt + %e      + %e
```

```
(%i6) reveal (e, 5);
```

```
(%o6)
      2      2
      b  - 2 a b + a
      -----
      sum(2)      2 b  2 a
2 %e      + %e  + %e
```

```
(%i7) reveal (e, 6);
```

```
(%o7)
      2      2
      b  - 2 a b + a
      -----
```

$$\frac{b^2 + a^2}{2} \%e + \frac{2b^2}{2} \%e + \frac{2a^2}{2} \%e$$

rmxchar

Variable opcional

Valor por defecto:]

La variable **rmxchar** es el carácter que se dibuja al lado derecho de una matriz.Véase también **lmxchar**.**save** (*filename*, *name_1*, *name_2*, *name_3*, ...)

Función

save (*filename*, *values*, *functions*, *labels*, ...)

Función

save (*filename*, [*m*, *n*])

Función

save (*filename*, *name_1=expr_1*, ...)

Función

save (*filename*, *all*)

Función

Almacena los valores actuales de *name_1*, *name_2*, *name_3*, ..., en el archivo *filename*.

Los argumentos son nombres de variables, funciones u otros objetos. Si un nombre no tiene un valor o una función asociado a él, entonces se ignora.

La función **save** devuelve *filename*.La función **save** almacena datos en forma de expresiones Lisp. Los datos almacenados por **save** pueden recuperarse con **load** (*filename*). El resultado de ejecutar **save** cuando *filename* ya existe depende del soporte Lisp implementado; el archivo puede ser sobrescrito o que **save** envíe un mensaje de error.La llamada **save** (*filename*, *values*, *functions*, *labels*, ...) almacena los elementos cuyos nombres son *values*, *functions*, *labels*, etc. Los nombres pueden ser cualesquiera de los especificados por la variable **infolists**; *values* incluye todas las variables definidas por el usuario.La llamada **save** (*filename*, [*m*, *n*]) almacena los valores de las etiquetas de entrada y salida desde *m* hasta *n*. Nótese que *m* y *n* deben ser números o nombres de variables precedidas de doble comilla simple. Las etiquetas de entrada y salida también se pueden almacenar una a una, por ejemplo, **save** ("foo.1", %i42, %o42). La llamada **save** (*filename*, *labels*) almacena todas las etiquetas de entrada y salida. Cuando las etiquetas almacenadas en el archivo sean posteriormente recuperadas, se sobrescribirán las activas en ese momento.La llamada **save** (*filename*, *name_1=expr_1*, *name_2=expr_2*, ...) almacena los valores de *expr_1*, *expr_2*, ..., con los nombres *name_1*, *name_2*, Es útil hacer este tipo de llamada para con etiquetas de entrada y salida, por ejemplo, **save** ("foo.1", aa=%o88). El miembro derecho de la igualdad puede ser cualquier expresión, que será evaluada. Esta llamada a la función **save** no incorpora nuevos nombres a la sesión actual de Maxima, simplemente los almacena en el archivo *filename*.Todas estas formas de llamar a la función **save** se pueden combinar a voluntad. Por ejemplo, **save** (*filename*, aa, bb, cc=42, *functions*, [11, 17]).La llamada **save** (*filename*, *all*) almacena el estado actual de Maxima, lo que incluye todas las variables definidas por el usuario, funciones, arreglos, etc., así como algunos objetos definidos automáticamente. Los elementos almacenados incluyen variables del sistema, como **file_search_maxima** o **showtime**, si han sido modificadas por el usuario. Véase **myoptions**.

La función `save` no evalúa sus argumentos. El argumento *filename* debe ser una cadena, no una variable que contenga una cadena. Si se especifican números de etiquetas, éstos deben ser enteros. El operador de doble comilla simple evalúa una variable de cadena devolviendo su valor, por ejemplo, `s: "foo.1"$ save (''s, all)$`, y de las variables enteras devuelve sus valores enteros, como en `m: 5$ n: 12$ save ("foo.1", [''m, ''n])$`.

savedef

Variable opcional

Valor por defecto: `true`

Si `savedef` vale `true`, se mantiene la versión Máxima de una función definida por el usuario cuando ésta se traduce, lo que permite mostrar su código con `dispfun` y que la función pueda ser editada.

Si `savedef` vale `false`, los nombres de las funciones traducidas se eliminan de la lista `functions`.

show (expr)

Función

Muestra `expr` con sus objetos indexados que tengan índices covariantes como subíndices y los contravariantes como superíndices. Los índices derivados se muestran como subíndices, separados por una coma de los covariantes.

showratvars (expr)

Función

Devuelve una lista de las variables de expresiones canónicas racionales (CRE) en la expresión `expr`.

Véase también `ratvars`.

stardisp

Variable opcional

Valor por defecto: `false`

Si `stardisp` vale `true`, la multiplicación se muestra con un asterisco `*` entre los operandos.

string (expr)

Función

Convierte `expr` a la notación lineal de Maxima, tal como si fuese tecleada.

El valor que retorna la función `string` es una cadena, por lo que no puede ser utilizada en los cálculos.

stringdisp

Variable de Lisp

Valor por defecto: `false`

Si `?stringdisp` vale `true`, las cadenas alfanuméricas se muestran encerradas entre comillas dobles. En caso contrario, no se muestran las comillas.

La variable `?stringdisp` vale siempre `true` cuando se muestra la definición de una función.

La variable `?stringdisp` es una variable Lisp, por lo que debe ser escrita precedida de un signo de interrogación `?`.

Ejemplos:

```
(%i1) ?stringdisp: false$
(%i2) "This is an example string.";
(%o2)          This is an example string.
(%i3) foo () := print ("This is a string in a function definition.");
(%o3) foo() :=
          print("This is a string in a function definition.")
(%i4) ?stringdisp: true$
(%i5) "This is an example string.";
(%o5)          "This is an example string."
```

stringout (<i>filename</i> , <i>expr_1</i> , <i>expr_2</i> , <i>expr_3</i> , ...)	Función
stringout (<i>filename</i> , [<i>m</i> , <i>n</i>])	Función
stringout (<i>filename</i> , <i>input</i>)	Función
stringout (<i>filename</i> , <i>functions</i>)	Función
stringout (<i>filename</i> , <i>values</i>)	Función

La función **stringout** escribe expresiones en un archivo de la misma forma en que se escribirían como expresiones de entrada. El archivo puede ser utilizado entonces como entrada a las funciones **batch** o **demo**, y puede ser editado para cualquier otro propósito.

La forma general de **stringout** escribe los valores de una o más expresiones en el archivo de salida. Nótese que si una expresión es una variable, solamente se escribirá el valor de la variable y no el nombre de ésta. Como caso especial, y muy útil en algunas ocasiones, las expresiones pueden ser etiquetas de entrada (%i1, %i2, %i3, ...) o de salida (%o1, %o2, %o3, ...).

Si **grind** vale **true**, **stringout** formatea la salida utilizando **grind**. En caso contrario, se utilizará el formato **string**. Véanse **grind** y **string**.

La forma especial **stringout** (*filename*, [*m*, *n*]) escribe los valores de las etiquetas de entrada desde la *m* hasta la *n*, ambas inclusive.

La forma especial **stringout** (*filename*, *input*) escribe todas las etiquetas de entrada en el archivo.

La forma especial **stringout** (*filename*, *functions*) escribe todas las funciones definidas por el usuario, contenidas en la lista global **functions**, en el archivo.

La forma especial **stringout** (*filename*, *values*) escribe todas las variables asignadas por el usuario, contenidas en la lista global **values**, en el archivo. Cada variable se escribe como una sentencia de asignación, con el nombre de la variable seguida de dos puntos y a continuación su valor. Nótese que la forma general de **stringout** no escribe las variables como sentencias de asignación.

tex (<i>expr</i>)	Función
tex (<i>label</i>)	Función
tex (<i>expr</i> , <i>filename</i>)	Función
tex (<i>label</i> , <i>filename</i>)	Función

Devuelve la expresión en un formato apropiado para ser incorporado a un documento basado en TeX. El resultado que se obtiene es un fragmento de código que puede incluirse en un documento mayor, pero que no puede ser procesado aisladamente.

La instrucción `tex (expr)` imprime en la consola la representación en TeX de `expr`.

La instrucción `tex (label)` imprime en la consola la representación en TeX de la expresión a la que hace referencia la etiqueta `label`, asignándole a su vez una etiqueta de ecuación que será mostrada al lado izquierdo de la misma. La etiqueta de la expresión en TeX es la misma que la de Maxima.

La instrucción `tex (expr, filename)` añade la representación TeX de `expr` al fichero `filename`. La función `tex` no evalúa el argumento `filename`, pero la doble comilla simple `''` fuerza su evaluación.

La instrucción `tex (label, filename)` añade la representación TeX de la expresión a la que hace referencia la etiqueta `label`, junto con una etiqueta, al fichero `filename`. La función `tex` no evalúa el argumento `filename`, pero la doble comilla simple `''` fuerza su evaluación.

La función `tex` evalúa sus argumentos tras comprobar si se trata de una etiqueta. La doble comilla simple `''` fuerza la evaluación del argumento, anulando la comprobación sobre la etiqueta.

Véase también `texput`.

Ejemplos:

```
(%i1) integrate (1/(1+x^3), x);
                                     2 x - 1
                                     atan(-----)
                                     sqrt(3)
(%o1)  - ---- + ---- + ----
        6          sqrt(3)      3
(%i2) tex (%o1);
$$-{\log \left(x^2-x+1\right)}\over{6}}+{\{\arctan \left({2\,x-1\over\sqrt{3}}\right)}\over{\sqrt{3}}}+{\log \left(x+1\right)}\over{3}}\leqno{\tt (\%o1)}$$
(%o2)                                     (%o1)
(%i3) tex (integrate (sin(x), x));
$$-\cos x$$
(%o3)                                     false
(%i4) tex (%o1, "foo.tex");
(%o4)                                     (%o1)
```

texput (<i>a</i> , <i>s</i>)	Función
texput (<i>a</i> , <i>s</i> , <i>operator_type</i>)	Función
texput (<i>a</i> , [<i>s</i> ₁ , <i>s</i> ₂], <i>matchfix</i>)	Función
texput (<i>a</i> , [<i>s</i> ₁ , <i>s</i> ₂ , <i>s</i> ₃], <i>matchfix</i>)	Función

Establece el formato en TeX del átomo *a*, el cual puede ser un símbolo o el nombre de un operador.

La instrucción `texput (a, s)` hace que la función `tex` introduzca *s* en la salida TeX en el lugar de *a*.

La instrucción `texput (a, s, operator_type)`, en la que *operator_type* es `prefix`, `infix` o `postfix`, hace que la función `tex` introduzca *s* en la salida TeX en el lugar de *a*, colocándolo en el lugar correcto.

La instrucción `texput (a, [s_1, s_2], matchfix)` hace que la función `tex` introduzca s_1 y s_2 en la salida TeX a los lados de los argumentos de a . Si son más de uno, los argumentos se separan por comas.

La instrucción `texput (a, [s_1, s_2, s_3], matchfix)` hace que la función `tex` introduzca s_1 y s_2 en la salida TeX a los lados de los argumentos de a , con s_3 separando los argumentos.

Ejemplos:

```
(%i1) texput (me,"\\mu_e");
(%o1)                                     \mu_e
(%i2) tex (me);
$$\mu_e$$
(%o2)                                     false
(%i3) texput (lcm, "\\mathrm{lcm}");
(%o3)                                     \mathrm{lcm}
(%i4) tex (lcm (a, b));
$$\mathrm{lcm}\left(a , b\right)$$
(%o4)                                     false
(%i5) prefix ("grad");
(%o5)                                     grad
(%i6) texput ("grad", " \\nabla ", prefix);
(%o6)                                     180
(%i7) tex (grad f);
$$ \nabla f$$
(%o7)                                     false
(%i8) infix ("~");
(%o8)                                     ~
(%i9) texput ("~", " \\times ", infix);
(%o9)                                     180
(%i10) tex (a ~ b);
$$a \times b$$
(%o10)                                    false
(%i11) postfix ("@");
(%o11)                                    @
(%i12) texput ("@", "!!", postfix);
(%o12)                                    160
(%i13) tex (x @);
$$x!!$$
(%o13)                                    false
(%i14) matchfix ("<<", ">>");
(%o14)                                    <<
(%i15) texput ("<<", [" \\langle ", " \\rangle "], matchfix);
(%o15)                                    \langle ( \rangle , false)
(%i16) tex (<<a>>);
$$ \langle a \rangle $$
(%o16)                                    false
(%i17) tex (<<a, b>>);
$$ \langle a , b \rangle $$
(%o17)                                    false
```

```
(%i18) texput ("<<", [" \\langle ", " \\rangle ", " \\, | \\,", matchfix);
(%o18)          \langle \rangle , \, | \,
(%i19) tex (<<a>);
$$ \langle a \rangle $$
(%o19)          false
(%i20) tex (<<a, b>);
$$ \langle a \, | \, b \rangle $$
(%o20)          false
```

system (*command*)

Función

Ejecuta la instrucción *command* como un proceso independiente de Maxima. La instrucción se le pasa a la consola del sistema para su ejecución. La función **system** no está soportada por todos los sistemas operativos, pero suele estarlo en todos los entornos Unix y similares.

Suponiendo que `_hist.out` es una lista de frecuencias que se quieren representar en un diagrama de barras utilizando el programa **xgraph**,

```
(%i1) (with_stdout("_hist.out",
      for i:1 thru length(hist) do (
        print(i,hist[i]))),
      system("xgraph -bar -brw .7 -nl < _hist.out"));
```

A fin de hacer el diagrama y eliminar el archivo temporal posteriormente, hágase:

```
system("(xgraph -bar -brw .7 -nl < _hist.out; rm -f _hist.out)&")
```

ttyoff

Variable opcional

Valor por defecto: `false`

Si **ttyoff** vale `true`, no se muestran las expresiones resultantes, pero éstas se calculan de todos modos y se les asignan etiquetas. Véase **labels**.

El texto que escriban las funciones internas de Maxima, tales como los mensajes de error y las salidas de **describe**, no se ven afectadas por **ttyoff**.

with_stdout (*filename, expr_1, expr_2, expr_3, ...*)

Macro

Abre el archivo *filename* y a continuación evalúa *expr_1*, *expr_2*, *expr_3*, Los valores de los argumentos no se almacenan *filename*, pero cualquier resultado a imprimir por **print**, **display**, **disp** o **grind**, va a parar al archivo *filename* en lugar de a la consola.

La macro **with_stdout** devuelve el valor de su último argumento.

Véase también **writefile**.

```
(%i1) with_stdout ("tmp.out", for i:5 thru 10 do print (i, "! yields", i!))$
(%i2) printfile ("tmp.out")$
5 ! yields 120
6 ! yields 720
7 ! yields 5040
8 ! yields 40320
9 ! yields 362880
10 ! yields 3628800
```


writefile (*filename*)

Función

Comienza escribiendo una transcripción de la sesión de Maxima en el archivo *filename*. Cualquier interacción entre Maxima y el usuario se almacena también en este archivo, tal como aparece en la consola.

Puesto que la transcripción se escribe en el formato de salida a la consola, su contenido no es interpretable por Maxima. Para hacer un archivo que contenga expresiones que puedan ser nuevamente cargadas en Maxima, véanse **save** y **stringout**; la función **save** almacena expresiones en formato Lisp, mientras que **stringout** lo hace en formato Maxima.

El resultado de ejecutar **writefile** cuando el archivo *filename* ya existe depende del entorno Lisp operativo; el contenido anterior puede ser sobrescrito o ampliado con la sesión actual. La función **appendfile** siempre añade la sesión al contenido actual.

Puede ser útil ejecutar **playback** después de **writefile** para guardar las interacciones previas de la sesión. Puesto que **playback** muestra solamente las variables de entrada y salida (%i1, %o1, etc.), cualquier salida generada por una sentencia de impresión desde dentro de una función no es mostrada por **playback**.

La función **closefile** cierra los archivos abiertos por **writefile** o **appendfile**.

10 Aritmética de punto flotante

10.1 Definiciones para aritmética de punto flotante

bffac (*expr*, *n*) Función

Versión para "bigfloat" de la función factorial (Gamma desplazada). El segundo argumento indica cuántos dígitos se conservan y devuelven, pudiendo utilizarse para obtener algunas cifras extra.

La instrucción `load ("bffac")` carga esta función.

algepsilon Variable optativa

Valor por defecto: 10^8

El valor de `algepsilon` es usado por `algsys`.

bfloat (*expr*) Función

Convierte todos los números y funciones numéricas a números decimales de punto flotante grandes ("bigfloats"). El número de dígitos significativos de los "bigfloats" resultantes se especifica mediante la variable global `fpprec`.

Si `float2bf` vale `false` se mostrará un mensaje de aviso cuando un número en punto flotante se convierte a decimal de tipo "bigfloats", puesto que tal transformación puede conllevar pérdida de precisión.

bfloatp (*expr*) Función

Devuelve `true` si *expr* es un número decimal en punto flotante grande ("bigfloats"), en caso contrario devuelve `false`.

bfpsi (*n*, *z*, *fpprec*) Función

bfpsi0 (*z*, *fpprec*) Función

La función `bfpsi` es la poligamma de argumento real *z* y de orden el entero *n*. La función `bfpsi0` es la digamma. La llamada `bfpsi0 (z, fpprec)` equivale a `bfpsi (0, z, fpprec)`.

Estas funciones devuelven valores "bigfloat". La variable `fpprec` es la precisión "bigfloat" del valor de retorno.

La instrucción `load ("bffac")` carga estas funciones.

bftorat Variable optativa

Valor por defecto: `false`

La variable `bftorat` controla la conversión de números decimales de punto flotante grandes ("bigfloats") a números racionales. Si `bftorat` vale `false`, se utilizará `ratepsilon` para controlar la conversión (lo cual resulta en números racionales relativamente pequeños). Si `bftorat` vale `true`, el número racional generado representará exactamente al número decimal de punto flotante grande ("bigfloat").

- bftrunc** Variable optativa
 Valor por defecto: `true`
 La variable `bftrunc` provoca la eliminación de ceros en números decimales grandes no nulos para que no se muestren. Así, si `bftrunc` vale `false`, `bfloat (1)` se muestra como `1.000000000000000B0`. En otro caso, se mostrará como `1.0B0`.
- cbffac** (*z*, *fpprec*) Función
 Calcula el factorial de números complejos de punto flotante grandes.
 La instrucción `load ("bffac")` carga esta función.
- float** (*expr*) Función
 Convierte los enteros, números racionales y los decimales de punto flotante grandes ("`bigfloats`") que están presentes en *expr* a números de punto flotante. También actúa como símbolo `evflag`.
- float2bf** Variable optativa
 Valor por defecto: `false`
 Si `float2bf` vale `false` se mostrará un mensaje de aviso cuando un número en punto flotante se convierte a decimal de tipo "`bigfloats`", puesto que tal transformación puede conllevar pérdida de precisión.
- floatnump** (*expr*) Función
 Devuelve `true` si *expr* es un número de punto flotante, en caso contrario retorna `false`.
- fpprec** Variable optativa
 Valor por defecto: 16
 La variable `fpprec` guarda el número de dígitos significativos en la aritmética con números decimales de punto flotante grandes ("`bigfloats`"). La variable `fpprec` no afecta a los cálculos con números decimales de punto flotante ordinarios.
 Véanse también `bfloat` y `fpprintprec`.
- fpprintprec** Variable optativa
 Valor por defecto: 0
 La variable `fpprintprec` guarda el número de dígitos a imprimir cuando se muestren por terminal números decimales de punto flotante grandes, haciendo así que sea posible realizar cálculos con gran precisión pero mostrando los resultados con menor número de dígitos.
 Si `fpprintprec` vale 0, o es mayor o igual que `fpprec`, entonces será el valor de `fpprec` el que controle el número de dígitos a presentar.
 Si `fpprintprec` toma un valor entre 2 y `fpprec - 1`, entonces controlará el número de dígitos a utilizar. (El número mínimo de dígitos es 2, uno a la izquierda del punto decimal y otro a la derecha.)
 No se admite el valor 1 para `fpprintprec`.

?round (*x*)

Función Lisp

?round (*x*, *divisor*)

Función Lisp

Redondea el número decimal en punto flotante *x* al entero más próximo. El argumento debe ser un decimal flotante ordinario, no grande ("bigfloat"). El símbolo ? al comienzo indica que se trata de una función Lisp.

```
(%i1) ?round (-2.8);
(%o1) - 3
```

?truncate (*x*)

Función Lisp

?truncate (*x*, *divisor*)

Función Lisp

Trunca el número decimal en punto flotante *x* en dirección al 0, para convertirlo en un entero. El argumento debe ser un decimal flotante ordinario, no grande ("bigfloat"). El símbolo ? al comienzo indica que se trata de una función Lisp.

```
(%i1) ?truncate (-2.8);
(%o1) - 2
(%i2) ?truncate (2.4);
(%o2) 2
(%i3) ?truncate (2.8);
(%o3) 2
```


11 Contextos

11.1 Definiciones para Contextos

activate (*context_1*, ..., *context_n*)

Función

Activa los contextos *context_1*, ..., *context_n*. Los hechos en estos contextos están disponibles para hacer deducciones y extraer información. Los hechos en estos contextos no se listan al invocar **facts** ().

La variable **activecontexts** es la lista de contextos que se han activado por medio de la función **activate**.

activecontexts

Variable del sistema

Valor por defecto: []

La variable **activecontexts** es la lista de contextos que se han activado por medio de la función **activate**, pero que no se han activado por ser subcontextos del contexto actual.

assume (*pred_1*, ..., *pred_n*)

Función

Añade los predicados *pred_1*, ..., *pred_n* al contexto actual. Si un predicado es inconsistente o redundante con los otros predicados del contexto actual, entonces no es añadido al contexto. El contexto va acumulando predicados con cada llamada a **assume**.

La función **assume** devuelve una lista cuyos miembros son los predicados que han sido añadidos al contexto, o los átomos **redundant** o **inconsistent** si fuere necesario.

Los predicados *pred_1*, ..., *pred_n* tan solo pueden ser expresiones formadas con los operadores relacionales **<**, **<=**, **equal**, **notequal**, **>=** y **>**. Los predicados no pueden estar formados por expresiones que sean del tipo igualdad **=** ni del tipo desigualdad **#**, ni tampoco pueden ser funciones de predicado como **integerp**.

En cambio, sí se reconocen predicados compuestos de la forma *pred_1 and ... and pred_n*, pero no *pred_1 or ... or pred_n*. También se reconoce **not pred_k** si *pred_k* es un predicado relacional. Expresiones de la forma **not (pred_1 and pred_2)** y **not (pred_1 or pred_2)** no son reconocidas.

El mecanismo deductivo de Maxima no es muy potente; existen muchas consecuencias que, siendo obvias, no pueden ser obtenidas por **is**. Se trata de una debilidad reconocida.

La función **assume** evalúa sus argumentos.

Véanse también **is**, **facts**, **forget**, **context** y **declare**.

Ejemplos:

```
(%i1) assume (xx > 0, yy < -1, zz >= 0);
(%o1) [xx > 0, yy < - 1, zz >= 0]
(%i2) assume (aa < bb and bb < cc);
(%o2) [bb > aa, cc > bb]
(%i3) facts ();
(%o3) [xx > 0, - 1 > yy, zz >= 0, bb > aa, cc > bb]
```

```

(%i4) is (xx > yy);
(%o4)                                     true
(%i5) is (yy < -yy);
(%o5)                                     true
(%i6) is (sinh (bb - aa) > 0);
(%o6)                                     true
(%i7) forget (bb > aa);
(%o7)                                     [bb > aa]
(%i8) prederror : false;
(%o8)                                     false
(%i9) is (sinh (bb - aa) > 0);
(%o9)                                     unknown
(%i10) is (bb^2 < cc^2);
(%o10)                                    unknown

```

assumescalar

Variable opcional

Valor por defecto: **true**

La variable **assumescalar** ayuda a controlar si una expresión **expr** para la cual **nonscalarp (expr)** es **false** va a tener un comportamiento similar a un escalar bajo ciertas transformaciones.

Sea **expr** cualquier expresión distinta de una lista o matriz, y sea también **[1, 2, 3]** una lista o una matriz. Entonces, **expr . [1, 2, 3]** dará como resultado **[expr, 2 expr, 3 expr]** si **assumescalar** es **true**, o si **scalarp (expr)** es **true**, o si **constantp (expr)** es **true**.

Si **assumescalar** vale **true**, la expresión se comportará como un escalar sólo en operaciones conmutativas, pero no en el caso de la multiplicación no conmutativa o producto matricial ..

Si **assumescalar** vale **false**, la expresión se comportará como un no escalar.

Si **assumescalar** vale **all**, la expresión se comportará como un escalar para todas las operaciones.

assume_pos

Variable opcional

Valor por defecto: **false**

Si **assume_pos** vale **true** y el signo de un parámetro **x** no puede ser determinado a partir del contexto actual o de otras consideraciones, **sign** y **asksign (x)** devolverán **true**. Con esto se pueden evitar algunas preguntas al usuario que se generan automáticamente, como las que hacen **integrate** y otras funciones.

By default, a parameter is **x** such that **symbolp (x)** or **subvarp (x)**.

Por defecto, un parámetro **x** es aquel para el que **symbolp (x)** o **subvarp (x)** devuelven **true**. La clase de expresiones que se consideran parámetros se puede extender mediante la utilización de la variable **assume_pos_pred**.

Las funciones **sign** y **asksign** intentan deducir el signo de una expresión a partir de los signos de los operandos que contiene. Por ejemplo, si **a** y **b** son ambos positivos, entonces **a + b** también es positivo.

Sin embargo, no es posible obviar todas las preguntas que hace **asksign**. En particular, cuando el argumento de **asksign** es una diferencia **x - y** o un logaritmo **log(x)**,

`asksign` siempre solicita una respuesta por parte del usuario, incluso cuando `assume_pos` vale `true` y `assume_pos_pred` es una función que devuelve `true` para todos los argumentos.

`assume_pos_pred`

Variable opcional

Valor por defecto: `false`

Cuando a `assume_pos_pred` se le asigna el nombre de una función o una expresión lambda de un único argumento `x`, ésta será invocada para determinar si `x` se considera un parámetro por `assume_pos`. La variable `assume_pos_pred` se ignora cuando `assume_pos` vale `false`.

La función `assume_pos_pred` es invocada por `sign` y por `asksign` con un argumento `x`, el cual puede ser un átomo, una variable subindicada o una expresión de llamada a una función. Si la función `assume_pos_pred` devuelve `true`, `x` será considerada como un parámetro por `assume_pos`.

Por defecto, un parámetro `x` es aquel para el que `symbolp (x)` o `subvarp (x)` devuelven `true`.

Véanse también `assume` y `assume_pos`.

Ejemplos:

```
(%i1) assume_pos: true$
(%i2) assume_pos_pred: symbolp$
(%i3) sign (a);
(%o3)                                     pos
(%i4) sign (a[1]);
(%o4)                                     pnz
(%i5) assume_pos_pred: lambda ([x], display (x), true)$
(%i6) asksign (a);
                                     x = a

(%o6)                                     pos
(%i7) asksign (a[1]);
                                     x = a
                                     1

(%o7)                                     pos
(%i8) asksign (foo (a));
                                     x = foo(a)

(%o8)                                     pos
(%i9) asksign (foo (a) + bar (b));
                                     x = foo(a)

                                     x = bar(b)

(%o9)                                     pos
(%i10) asksign (log (a));
                                     x = a
```



```
Is a - 1 positive, negative, or zero?
```

```
p;
(%o10)                                     pos
(%i11) asksign (a - b);
                                     x = a
                                     x = b
                                     x = a
                                     x = b
```

```
Is b - a positive, negative, or zero?
```

```
p;
(%o11)                                     neg
```

context

Variable opcional

Valor por defecto: `initial`

La variable **context** da nombre al conjunto de hechos establecidos desde **assume** y **forget**. La función **assume** añade nuevos hechos al conjunto nombrado por **context**, mientras que **forget** los va eliminando. Asignando a **context** un nuevo nombre *foo* cambia el contexto actual a *foo*. Si el contexto *foo* no existe todavía, se crea automáticamente mediante una llamada a **newcontext**.

Véase **contexts** para una descripción general del mecanismo que siguen los contextos.

contexts

Variable opcional

Valor por defecto: `[initial, global]`

La variable **contexts** es una lista que contiene los contextos existentes, incluyendo el actualmente activo.

The context mechanism makes it possible for a user to bind together and name a collection of facts, called a context.

El mecanismo que siguen los contextos permiten al usuario agrupar y nombrar un conjunto de hechos, que recibe el nombre de contexto. Una vez hecho esto, el usuario puede hacer que Maxima tenga en cuenta o que olvide cualquier número de hechos sin más que activar o desactivar su contexto.

Cualquier átomo simbólico puede ser el nombre de un contexto, y los hechos contenidos en tal contexto pueden ser almacenados hasta que se destruyan uno a uno mediante llamadas a la función **forget**, o que se destruyan conjuntamente invocando a **kill** para eliminar el contexto al que pertenecen.

Los contextos tienen estructura jerárquica, siendo su raíz el contexto **global**, el cual contiene información sobre Maxima que necesitan algunas funciones. Cuando en un contexto todos los hechos están activos (lo que significa que están siendo utilizados en deducciones) lo estarán también en cualquier subcontexto del contexto actual.

Cuando se comienza una sesión de Maxima, el usuario estará trabajando en un contexto llamado `initial`, el cual tiene un subcontexto de nombre `global`.

Véanse también `facts`, `newcontext`, `supcontext`, `killcontext`, `activate`, `deactivate`, `assume` y `forget`.

deactivate (*contexto_1*, ..., *contexto_n*) Función
Desactiva los contextos especificados *contexto_1*, ..., *contexto_n*.

facts (*item*) Función
facts () Función

Si *item* es el nombre de un contexto, **facts** (*item*) devuelve una lista con los hechos asociados al contexto especificado.

Si *item* no es el nombre de un contexto, **facts** (*item*) devuelve una lista con los hechos conocidos acerca de *item* en el contexto actual. Los hechos que estén activos en contextos diferentes no aparecen en la lista.

La llamada **facts** (), sin argumentos, muestra el contexto actual.

features Declaración

Maxima reconoce ciertas propiedades matemáticas sobre funciones y variables.

La llamada **declare** (*x*, *foo*) asocia la propiedad *foo* a la función o variable *x*.

La llamada **declare** (*foo*, *feature*) declara una nueva propiedad *foo*. Por ejemplo, **declare** ([rojo, verde, azul], *feature*) declara tres nuevas propiedades, rojo, verde y azul.

El predicado **featurep** (*x*, *foo*) devuelve `true` si *x* goza de la propiedad *foo*, y `false` en caso contrario.

La lista **features** contiene las propiedades que reconoce Maxima; a saber, `integer`, `noninteger`, `even`, `odd`, `rational`, `irrational`, `real`, `imaginary`, `complex`, `analytic`, `increasing`, `decreasing`, `oddfun`, `evenfun`, `posfun`, `commutative`, `lassociative`, `rassociative`, `symmetric`, and `antisymmetric`, junto con las definidas por el usuario.

La lista **features** sólo contiene propiedades matemáticas. Hay otra lista con propiedades no matemáticas; Véase **status**.

forget (*pred_1*, ..., *pred_n*) Función

forget (*L*) Función

Borra los predicados establecidos por **assume**. Los predicados pueden ser expresiones equivalentes, pero no necesariamente idénticas, a las establecidas por **assume**. The predicates may be expressions equivalent to (but not necessarily identical to) those previously assumed.

La llamada **forget** (*L*), siendo *L* una lista de predicados, borra todos los predicados contenidos en ella.

killcontext (*contexto_1*, ..., *contexto_n*) Función

Elimina los contextos *contexto_1*, ..., *contexto_n*.

Si alguno de estos contextos es el actual, el nuevo contexto activo será el primer subcontexto disponible del actual que no haya sido eliminado. Si el primer contexto no eliminado disponible es `global` entonces `initial` será usado en su lugar. Si el contexto `initial` es eliminado, se creará un nuevo contexto `initial` completamente vacío.

La función `killcontext` no elimina un contexto actualmente activo si es un subcontexto del contexto actual, o si se hace uso de la función `activate`.

La función `killcontext` evalúa sus argumentos y devuelve `done`.

newcontext (*nombre*)

Función

Crea un nuevo contexto vacío *nombre*, el cual tiene a `global` como su único subcontexto. El recién creado contexto pasa a ser el contexto actualmente activo.

La función `newcontext` evalúa sus argumentos y devuelve *nombre*.

supcontext (*nombre*, *contexto*)

Función

supcontext (*nombre*)

Función

Crea un nuevo contexto *nombre*, que tiene a *contexto* como subcontexto. El argumento *contexto* debe existir ya.

Si no se especifica *contexto*, se tomará como tal el actual.

12 Polinomios

12.1 Introducción a los Polinomios

Los polinomios se almacenan en Maxima, bien en un formato general, bien en una forma conocida como canónica (Canonical Rational Expressions, CRE). La última corresponde al formato estándar y se utiliza internamente para realizar operaciones como `factor`, `ratsimp` y demás.

Las Expresiones Racionales Canónicas (CRE) constituyen un tipo de representación que es especialmente apropiado para expandir polinomios y funciones racionales (así como para polinomios parcialmente factorizados y funciones racionales cuando a la variable `ratfac` se le asigna el valor `true`). En esta forma CRE las variables se ordenan de mayor a menor. Los polinomios se representan recursivamente como una lista compuesta por la variable principal seguida por una serie de pares de expresiones, una por cada término del polinomio. El primer miembro de cada par es el exponente de la variable principal en ese término y el segundo miembro es el coeficiente de ese término, el cual puede ser un número o un polinomio en otra variable representado también de esta forma. Así, la parte principal de la forma CRE de $3X^2-1$ es $(X\ 2\ 3\ 0\ -1)$ y la de $2XY+X-3$ es $(Y\ 1\ (X\ 1\ 2)\ 0\ (X\ 1\ 1\ 0\ -3))$ asumiendo que Y es la variable principal, y será $(X\ 1\ (Y\ 1\ 2\ 0\ 1)\ 0\ -3)$ si se asume que la variable principal es X . Qué variable se considera "principal" se determina en orden alfabético inverso. Las "variables" de la expresión CRE no son necesariamente atómicas. De hecho cualquier subexpresión cuyo operador principal no es $+$ $-$ $*$ $/$ ni $^$ con potencia entera puede ser considerada como una "variable" de la expresión (en forma CRE) en el cual aparezca. Por ejemplo las variables CRE de la expresión $X+\sin(X+1)+2\sqrt{X}+1$ son X , $\sqrt{X}$ y $\sin(X+1)$. Si el usuario no especifica una ordenación de las variables mediante la función `ratvars` Maxima escogerá una alfabéticamente. En general, las CRE representan expresiones racionales, esto es, fracciones de polinomios, donde el numerador y el denominador no tienen factores comunes, siendo el denominador es positivo. La forma interna es esencialmente un par de polinomios (el numerador y el denominador) precedida por la lista de variables ordenadas. Si una expresión a ser mostrada está en la forma CRE o contiene alguna subexpresión en forma de CRE, el símbolo `/R/` será seguido por la etiqueta de la línea de comando. Véase la función `rat` para convertir una expresión a la forma CRE. Una extensión de la forma CRE se utiliza para la representación de las series de Taylor. La noción de una expresión racional se extiende de manera que los exponentes de las variables pueden ser números racionales positivos o negativos y no sólo enteros positivos y los coeficientes pueden ser también expresiones racionales y no sólo polinomios. Estas expresiones se representan internamente por una forma polinomial recursiva que es similar a la forma CRE, pero que la generaliza, aportando información adicional como el grado de truncamiento. Como con la forma CRE, el símbolo `/T/` sigue la etiqueta de línea de comando en la que se encuentra dicha expresión.

12.2 Definiciones para Polinomios

algebraic

Valor por defecto: `false`

Variable opcional

La variable **algebraic** debe valer **true** para que se pueda hacer la simplificación de enteros algebraicos.

berlefact

Variable opcional

Valor por defecto: **true**

Si **berlefact** vale **false** entonces se utiliza el algoritmo de factorización de Kronecker, en caso contrario se utilizará el algoritmo de Berlekamp, que es el que se aplica por defecto.

bezout (*p1*, *p2*, *x*)

Función

Es una alternativa a la función **resultant**. Devuelve una matriz.

bothcoef (*expr*, *x*)

Función

Devuelve una lista cuyo primer miembro es el coeficiente de *x* en *expr* (que coincide con el que devuelve **ratcoef** si *expr* está en formato CRE, o el que devuelve **coeff** si no está en este formato) y cuyo segundo miembro es la parte restante de *expr*. Esto es, [*A*, *B*] donde *expr* = *A***x* + *B*.

Ejemplo:

```
(%i1) islinear (expr, x) := block ([c],
      c: bothcoef (rat (expr, x), x),
      is (freeof (x, c) and c[1] # 0))$
(%i2) islinear ((r^2 - (x - r)^2)/x, x);
(%o2) true
```

coeff (*expr*, *x*, *n*)

Función

Devuelve el coeficiente de x^n en *expr*. El argumento *n* puede omitirse si es igual a la unidad. El argumento *x* puede ser un átomo o una subexpresión completa de *expr*, por ejemplo **sin(x)**, **a[i+1]**, **x + y**, etc. (En este último caso, la expresión (**x + y**) debe aparecer en *expr*). En ocasiones puede ser necesario expandir o factorizar *expr* para hacer x^n explícito, lo cual no hace **coeff** automáticamente.

Ejemplos:

```
(%i1) coeff (2*a*tan(x) + tan(x) + b = 5*tan(x) + 3, tan(x));
(%o1) 2 a + 1 = 5
(%i2) coeff (y + x*e^x + 1, x, 0);
(%o2) y + 1
```

combine (*expr*)

Función

Simplifica la suma *expr* combinando términos de con igual denominador reduciéndolos a un único término.

content (*p_1*, *x_1*, ..., *x_n*)

Función

Devuelve una lista cuyo primer miembro es el máximo común divisor de los coeficientes de los términos del polinomio *p_1* de variable *x_n* (este es el contenido) y cuyo segundo miembro es el polinomio *p_1* dividido por el contenido.

Ejemplos:

```
(%i1) content (2*x*y + 4*x^2*y^2, y);
          2
(%o1)          [2 x, 2 x y  + y]
```

denom (*expr*)

Función

Devuelve el denominador de la expresión racional *expr*.**divide** (*p-1*, *p-2*, *x-1*, ..., *x-n*)

Función

Calcula el cociente y el resto del polinomio *p-1* dividido por el polinomio *p-2*, siendo la variable principal *x-n*. Las otras funciones son como en la función **ratvars**. El resultado es una lista cuyo primer miembro es el cociente y el segundo miembro el resto.

Ejemplos:

```
(%i1) divide (x + y, x - y, x);
(%o1)          [1, 2 y]
(%i2) divide (x + y, x - y);
(%o2)          [- 1, 2 x]
```

Nótese que *y* es la variable principal en el segundo ejemplo.**eliminate** (*[eqn-1, ..., eqn-n]*, *[x-1, ..., x-k]*)

Función

Elimina variables de ecuaciones (o de expresiones que se supone valen cero) tomando resultantes sucesivas. Devuelve una lista con *n - k* expresiones y *k* variables *x-1*, ..., *x-k* eliminadas. Primero se elimina *x-1* dando *n - 1* expresiones, después se elimina *x-2*, etc. Si *k = n* entonces se devuelve una lista con una única expresión, libre de las variables *x-1*, ..., *x-k*. En este caso se llama a **solve** para resolver la última resultante para la última variable.

Ejemplo:

```
(%i1) expr1: 2*x^2 + y*x + z;
          2
(%o1)          z + x y + 2 x
(%i2) expr2: 3*x + 5*y - z - 1;
(%o2)          - z + 5 y + 3 x - 1
(%i3) expr3: z^2 + x - y^2 + 5;
          2      2
(%o3)          z - y + x + 5
(%i4) eliminate ([expr3, expr2, expr1], [y, z]);
          8      7      6      5      4
(%o4) [7425 x  - 1170 x  + 1299 x  + 12076 x  + 22887 x
          3      2
      - 5154 x  - 1291 x  + 7688 x + 15376]
```

ezgcd (*p-1*, *p-2*, *p-3*, ...)

Función

Devuelve una lista cuyo primer elemento es el máximo común divisor (mcd) de los polinomios *p-1*, *p-2*, *p-3*, ... siendo los miembros restantes los mismos polinomios divididos por el mcd. Se utiliza siempre el algoritmo **ezgcd**.

facexpand

Variable opcional

Valor por defecto: `true`

La variable `facexpand` controla si los factores irreducibles devueltos por `factor` están en formato expandido (por defecto) o recursivo (CRE normal).

factcomb (*expr*)

Función

Trata de combinar los coeficientes de los factoriales de *expr* con los mismos factoriales, convirtiendo, por ejemplo, $(n + 1) * n!$ en $(n + 1)!$.

Si la variable `sumsplitfact` vale `false` hará que `minfactorial` se aplique después de `factcomb`.

factor (*expr*)

Función

Factoriza la expresión *expr*, que puede contener cualquier número de variables o funciones, en factores irreducibles respecto de los enteros. La llamada `factor (expr, p)` factoriza *expr* en el campo de los enteros con un elemento añadido cuyo polinomio mínimo es *p*.

Si la variable `factorflag` vale `false` suprime la factorización de los factores enteros en las expresiones racionales.

La variable `dontfactor` puede contener una lista de variables con respecto a las cuales no se factorizará (inicialmente está vacía). Tampoco se factorizará respecto de cualesquiera otra variables que sean menos importantes (según la ordenación que se sigue en el formato CRE) que aquellas que se encuentran en la lista `dontfactor`.

Si la variable `savefactors` vale `true`, los factores de una expresión en forma de producto se guardarán por ciertas funciones a fin de acelerar posteriores factorizaciones de expresiones que contengan algunos de estos mismos factores.

Si `berlefact` vale `false` entonces se utiliza el algoritmo de factorización de Kronecker, en caso contrario se utilizará el algoritmo de Berlekamp, que es el que se aplica por defecto.

La variable `intfaclim` guarda el mayor divisor que se ensayará al factorizar un número `bignum`. Si vale `false` (este es el caso cuando el usuario llama a `factor` explícitamente), o si el entero ocupa dos bytes, se intentará una factorización completa del entero. El valor dado por el usuario a `intfaclim` se utiliza en las llamadas internas a `factor`. Así, puede ser necesario cambiar `intfaclim` a fin de evitar que Maxima consuma un tiempo excesivo intentando factorizar números enteros grandes.

Ejemplos:

```
(%i1) factor (2^63 - 1);
              2
(%o1)          7  73 127 337 92737 649657
(%i2) factor (-8*y - 4*x + z^2*(2*y + x));
(%o2)          (2 y + x) (z - 2) (z + 2)
(%i3) -1 - 2*x - x^2 + y^2 + 2*x*y^2 + x^2*y^2;
              2  2          2      2      2
(%o3)          x  y  + 2 x y  + y  - x  - 2 x - 1
(%i4) block ([dontfactor: [x]], factor (%/36/(1 + 2*y + y^2)));
              2
```

```

(%o4) 
$$\frac{(x^2 + 2x + 1)(y - 1)}{36(y + 1)}$$

(%i5) factor (1 + %e^(3*x));
(%o5) 
$$(\%e^x + 1)(\%e^{2x} - \%e^x + 1)$$

(%i6) factor (1 + x^4, a^2 - 2);
(%o6) 
$$(x^2 - ax + 1)(x^2 + ax + 1)$$

(%i7) factor (-y^2*z^2 - x*z^2 + x^2*y^2 + x^3);
(%o7) 
$$-(y^2 + x)(z - x)(z + x)$$

(%i8) (2 + x)/(3 + x)/(b + x)/(c + x)^2;
(%o8) 
$$\frac{x + 2}{(x + 3)(x + b)(x + c)^2}$$

(%i9) ratsimp (%);
(%o9) 
$$(x + 2)/(x^4 + (2c + b + 3)x^3 + (c^2 + (2b + 6)c + 3b)x^2 + ((b + 3)c^2 + 6bc)x + 3b^2c)$$

(%i10) partfrac (%, x);
(%o10) 
$$-(c^2 - 4c - b + 6)/((c^4 + (-2b - 6)c^3 + (b^2 + 12b + 9)c^2 + (-6b^2 - 18b)c + 9b^2)(x + c))$$


$$- \frac{c - 2}{(c^2 + (-b - 3)c + 3b)(x + c)^2}$$


$$+ \frac{b - 2}{((b - 3)c^2 + (6b - 2b^2)c + b^3 - 3b^2)(x + b)}$$


$$- \frac{1}{((b - 3)c^2 + (18 - 6b)c + 9b - 27)(x + 3)}$$

(%i11) map ('factor, %);
(%o11) 
$$-\frac{c^2 - 4c - b + 6}{2} - \frac{c - 2}{2}$$


```


$$\frac{(c-3)(c-b)(x+c)}{b-2} - \frac{(c-3)(c-b)(x+c)}{1}$$

```

(%i12) ratsimp ((x^5 - 1)/(x - 1));
(%o12)
      4      3      2
      x  + x  + x  + x + 1
(%i13) subst (a, x, %);
(%o13)
      4      3      2
      a  + a  + a  + a + 1
(%i14) factor (%th(2), %);
(%o14)
      2      3      3      2
      (x - a) (x - a) (x - a) (x + a + a + a + 1)
(%i15) factor (1 + x^12);
(%o15)
      4      8      4
      (x  + 1) (x  - x  + 1)
(%i16) factor (1 + x^99);
(%o16)
      2      6      3
      (x + 1) (x  - x + 1) (x  - x + 1)

      10      9      8      7      6      5      4      3      2
      (x  - x  + x  - x  + x  - x  + x  - x  + x  - x + 1)

      20      19      17      16      14      13      11      10      9      7      6
      (x  + x  - x  - x  + x  + x  - x  - x  - x  + x  + x

      4      3      60      57      51      48      42      39      33
      - x  - x  + x + 1) (x  + x  - x  - x  + x  + x  - x

      30      27      21      18      12      9      3
      - x  - x  + x  + x  - x  - x  + x + 1)

```

factorflag

Variable opcional

Valor por defecto: `false`

Si `factorflag` vale `false` se evita la factorización de factores enteros de expresiones racionales.

factorout (*expr*, *x-1*, *x-2*, ...)

Función

Reorganiza la suma *expr* como una suma de términos de la forma `f (x-1, x-2, ...)*g` donde *g* es un producto de expresiones que no contienen ningún *x.i* y *f* se factoriza.

factorsum (*expr*)

Función

Intenta agrupar términos en los factores de *expr* que son sumas en grupos de términos tales que su suma sea factorizable. La función `factorsum` puede restablecer el recuperar de `expand ((x + y)^2 + (z + w)^2)` pero no puede recuperar `expand ((x + 1)^2 + (x + y)^2)` porque los términos tienen variables comunes.

Ejemplo:

```
(%i1) expand ((x + 1)*((u + v)^2 + a*(w + z)^2));
(%o1) a x z + a z + 2 a w x z + 2 a w z + a w x + v x
      2      2      2      2      2      2
      + 2 u v x + u x + a w + v + 2 u v + u
(%i2) factorsum (%);
(%o2) (x + 1) (a (z + w) + (v + u) )
      2      2
```

fasttimes (*p_1*, *p_2*)

Función

Calcula el producto de los polinomios *p_1* y *p_2* utilizando un algoritmo especial. Los polinomios *p_1* y *p_2* deben ser multivariantes, densos y aproximadamente del mismo tamaño. La multiplicación clásica es de orden *n_1 n_2* donde *n_1* es el grado de *p_1* y *n_2* el grado de *p_2*. La función **fasttimes** es de orden $\max(n_1, n_2)^{1.585}$.

fullratsimp (*expr*)

Función

Aplica repetidamente **ratsimp** a una expresión, seguida de simplificaciones no racionales, hasta que no se obtienen más transformaciones; entonces devuelve el resultado.

En presencia de expresiones no racionales, una llamada a **ratsimp** seguida de una simplificación no racional ("general") puede no ser suficiente para conseguir un resultado simplificado. En ocasiones serán necesarias más de una llamada a **ratsimp**, que es lo que hace precisamente **fullratsimp**.

Ejemplo:

```
(%i1) expr: (x^(a/2) + 1)^2*(x^(a/2) - 1)^2/(x^a - 1);
      a/2      2      a/2      2
      (x  - 1) (x  + 1)
(%o1) -----
      a
      x  - 1
(%i2) ratsimp (expr);
      2 a      a
      x  - 2 x  + 1
(%o2) -----
      a
      x  - 1
(%i3) fullratsimp (expr);
      a
      x  - 1
(%o3) -----
      a/2 4      a/2 2
      (x  ) - 2 (x  ) + 1
(%o4) /R/ -----
      a
      x  - 1
```

fullratsubst (*a*, *b*, *c*)

Función

Similar a **ratsubst** excepto por el hecho de que se llama a í misma recursivamente hasta que el resultado deja de cambiar. Esta función es útil cuando la expresión a sustituir y la que la sustituye tienen variables comunes.

La función **fullratsubst** también acepta sus argumentos en el formato de **lratsubst**. Es necesario ejecutar **load ("lrats")** para cargar **fullratsubst** y **lratsubst**.

Ejemplos:

- ```
(%i1) load ("lrats")$
```
- **subst** puede hacer sustituciones múltiples; **lratsubst** es análoga a **subst**.
 

```
(%i2) subst ([a = b, c = d], a + c);
```

```
(%o2) d + b
```

```
(%i3) lratsubst ([a^2 = b, c^2 = d], (a + e)*c*(a + c));
```

```
(%o3) (d + a c) e + a d + b c
```
  - Si sólo se quiere una sustitución, entonces se puede dar una única ecuación como primer argumento.
 

```
(%i4) lratsubst (a^2 = b, a^3);
```

```
(%o4) a b
```
  - **fullratsubst** equivale a **ratsubst**, excepto por el hecho de que se llama a í misma recursivamente hasta que el resultado deja de cambiar.
 

```
(%i5) ratsubst (b*a, a^2, a^3);
```

```
(%o5) a^2 b
```

```
(%i6) fullratsubst (b*a, a^2, a^3);
```

```
(%o6) a^2 b
```
  - **fullratsubst** también acepta una lista de ecuaciones o una sólo ecuación como primer argumento.
 

```
(%i7) fullratsubst ([a^2 = b, b^2 = c, c^2 = a], a^3*b*c);
```

```
(%o7) b
```

```
(%i8) fullratsubst (a^2 = b*a, a^3);
```

```
(%o8) a^2 b
```
  - **fullratsubst** puede caer en una recursión infinita.
 

```
(%i9) errcatch (fullratsubst (b*a^2, a^2, a^3));
```

\*\*\* - Lisp stack overflow. RESET

**gcd** (*p-1*, *p-2*, *x-1*, ...)

Función

Devuelve el máximo común divisor de *p-1* y *p-2*. La variable **gcd** determina qué algoritmo se va a utilizar. Asignándole a **gcd** los valores **ez**, **eez**, **subres**, **red** o **spmod**, se seleccionan los algoritmos **ezgcd**, nuevo **eez gcd**, subresultante **prs**, reducido o modular, respectivamente. Si **gcd** vale **false** entonces **gcd(p1,p2,var)** devolverá siempre 1 para cualquier **var**. Muchas funciones (por ejemplo, **ratsimp**, **factor**, etc.) hacen uso de **gcd** implícitamente. En caso de polinomios homogéneos se recomienda darle a **gcd** el valor **subres**. Para calcular un máximo común divisor en presencia

de raíces, como en `gcd(x^2-2*sqrt(2)*x+2,x-sqrt(2));`, la variable `algebraic` debe igualarse previamente a `true` y `gcd` no puede ser `ez`. Se recomienda utilizar el algoritmo `subres` en lugar de `red`, por ser aquél más nuevo.

Cuando la variable `gcd`, cuyo valor por defecto es `subres`, vale `false`, no se calculará el máximo común divisor cuando las expresiones se conviertan a su forma canónica (CRE), lo que redundará en ocasiones en mayor rapidez de cálculo.

**gcdex** (*f*, *g*)

Función

**gcdex** (*f*, *g*, *x*)

Función

Devuelve una lista [*a*, *b*, *u*] en la que *u* es el máximo común divisor (mcd) de *f* y *g*, e igual a *a f* + *b g*. Los argumentos *f* y *g* deben ser polinomios univariantes, o indicarles la variable principal *x* en caso de ser multivariantes.

La función `gcdex` implementa el algoritmo de Euclides, en el que tenemos una secuencia de *L*[*i*]: [*a*[*i*], *b*[*i*], *r*[*i*]] todos ellos ortogonales a [*f*, *g*, -1] siendo el siguiente calculado a partir de *q* = `quotient(r[i]/r[i+1])` y *L*[*i*+2]: *L*[*i*] - *q L*[*i*+1]; el proceso termina en *L*[*i*+1] cuando el resto *r*[*i*+2] se anula.

```
(%i1) gcdex (x^2 + 1, x^3 + 4);
 2
 x + 4 x - 1 x + 4
(%o1)/R/ [- ----, ----, 1]
 17 17
(%i2) % . [x^2 + 1, x^3 + 4, -1];
(%o2)/R/ 0
```

**gcfactor** (*n*)

Función

Factoriza el entero gaussiano *n* como producto, a su vez, de enteros gaussianos, (un entero gaussiano es de la forma *a* + *b %i* donde *a* y *b* son números enteros). Los factores se normalizan de manera que tanto la parte real como imaginaria sean no negativas.

**gfactor** (*expr*)

Función

Factoriza el polinomio *expr* sobre los enteros gaussianos (un entero gaussiano es de la forma *a* + *b %i* donde *a* y *b* son números enteros). Es como `factor (expr, a^2+1)` donde *a* vale *%i*.

Ejemplo:

```
(%i1) gfactor (x^4 - 1);
(%o1) (x - 1) (x + 1) (x - %i) (x + %i)
```

**gfactorsum** (*expr*)

Función

Esta función es similar a `factorsum` pero aplica `gfactor` en lugar de `factor`.

**hipow** (*expr*, *x*)

Función

Devuelve el mayor exponente explícito de *x* en *expr*. El argumento *x* puede ser una variable o una expresión general. Si *x* no aparece en *expr*, `hipow` devuelve 0.

La función **hipow** no tiene en cuenta expresiones equivalentes a **expr**. En particular, **hipow** no expande **expr**, de manera que **hipow (expr, x)** y **hipow (expand (expr, x))** pueden dar resultados diferentes.

Ejemplos:

```
(%i1) hipow (y^3 * x^2 + x * y^4, x);
(%o1)
2
(%i2) hipow ((x + y)^5, x);
(%o2)
1
(%i3) hipow (expand ((x + y)^5), x);
(%o3)
5
(%i4) hipow ((x + y)^5, x + y);
(%o4)
5
(%i5) hipow (expand ((x + y)^5), x + y);
(%o5)
0
```

### **intfaclim**

Variable opcional

Valor por defecto: 1000

La variable **intfaclim** guarda el mayor divisor que se ensayará al factorizar un número **bignum**. Si vale **false** (este es el caso cuando el usuario llama a **factor** explícitamente), o si el entero ocupa dos bytes, se intentará una factorización completa del entero.

A la variable **intfaclim** se le asigna el valor **false** cuando los factores se calculan desde las funciones **divsum**, **totient** y **primep**.

Las llamadas internas a **factor** respetan el valor dado por el usuario a **intfaclim**. Asignando a **intfaclim** valores pequeños se puede reducir el tiempo de factorización de números enteros grandes.

### **keepfloat**

Variable opcional

Valor por defecto: **false**

Si **keepfloat** vale **true**, los números decimales en coma flotante no se racionalizan cuando las expresiones que los contienen se convierten al formato canónico racional (CRE).

### **lratsubst** (*L*, *expr*)

Función

Esta función es similar a **subst** (*L*, *expr*), excepto por el hecho de que utiliza **ratsubst** en lugar de **subst**.

El primer argumento de **lratsubst** es una ecuación o lista de ecuaciones idénticas en formato a las aceptadas por **subst**. Las sustituciones se hacen en el orden dado por la lista de ecuaciones, esto es, de izquierda a derecha.

La instrucción **load ("lrats")** carga **fullratsubst** y **lratsubst**.

Ejemplos:

```
(%i1) load ("lrats")$
```

- **subst** can carry out multiple substitutions. **lratsubst** is analogous to **subst**.

```
(%i2) subst ([a = b, c = d], a + c);
(%o2) d + b
(%i3) lratsubst ([a^2 = b, c^2 = d], (a + e)*c*(a + c));
(%o3) (d + a c) e + a d + b c
```

- If only one substitution is desired, then a single equation may be given as first argument.

```
(%i4) lratsubst (a^2 = b, a^3);
(%o4) a b
```

## modulus

Variable opcional

Valor por defecto: `false`

Si `modulus` es un número positivo  $p$ , las operaciones con números racionales (como los devueltos por `rat` y funciones asociadas) se realizan módulo  $p$ , utilizando el llamado sistema de módulo balanceado, en el que  $n$  módulo  $p$  se define como un entero  $k$  de  $[-(p-1)/2, \dots, 0, \dots, (p-1)/2]$  si  $p$  es impar, o de  $[-(p/2 - 1), \dots, 0, \dots, p/2]$  si  $p$  es par, de tal manera que  $a p + k$  es igual a  $n$  para algún entero  $a$ .

Normalmente a `modulus` se le asigna un número primo. Se acepta que a `modulus` se le asigne un entero positivo no primo, pero se obtendrá un mensaje de aviso. Maxima permitirá que a `modulus` se le asigne cero o un entero negativo, aunque no esté clara su utilidad.

## num (expr)

Función

Devuelve el numerador de `expr` si se trata de una fracción. Si `expr` no es una fracción, se devuelve `expr`.

La función `num` evalúa su argumento.

## polydecomp (p, x)

Función

Descompone el polinomio  $p$  de variable  $x$  en una composición funcional de polinomios en  $x$ . La función `polydecomp` devuelve una lista  $[p_1, \dots, p_n]$  tal que

```
lambda ([x], p_1) (lambda ([x], p_2) (... (lambda ([x], p_n) (x)) ...))
```

es igual a  $p$ . El grado de  $p_i$  es mayor que 1 para  $i$  menor que  $n$ .

Esta descomposición no es única.

Ejemplos:

```
(%i1) polydecomp (x^210, x);
(%o1) 7 5 3 2
 [x , x , x , x]
(%i2) p : expand (subst (x^3 - x - 1, x, x^2 - a));
(%o2) 6 4 3 2
 x - 2 x - 2 x + x + 2 x - a + 1
(%i3) polydecomp (p, x);
(%o3) 2 3
 [x - a, x - x - 1]
```

La siguiente función compone  $L = [e_1, \dots, e_n]$  como funciones de  $x$ ; se trata de la inversa de `polydecomp`:

```
compose (L, x) :=
 block ([r : x], for e in L do r : subst (e, x, r), r) $
```

Se vuelve a obtener el resultado del ejemplo de más arriba haciendo uso de `compose`:

```
(%i3) polydecomp (compose ([x^2 - a, x^3 - x - 1], x), x);
(%o3) [x^2 - a, x^3 - x - 1]
```

Nótese que aunque `compose (polydecomp (p, x), x)` devuelve siempre  $p$  (sin expandir), `polydecomp (compose ([p_1, ..., p_n], x), x)` no devuelve necesariamente  $[p_1, ..., p_n]$ :

```
(%i4) polydecomp (compose ([x^2 + 2*x + 3, x^2], x), x);
(%o4) [x^2 + 2, x^2 + 1]
(%i5) polydecomp (compose ([x^2 + x + 1, x^2 + x + 1], x), x);
(%o5) [x^2 + 3, x^2 + 5, 2 x + 1]
```

**quotient** ( $p_1, p_2$ ) Función  
**quotient** ( $p_1, p_2, x_1, \dots, x_n$ ) Función

Devuelve el polinomio  $p_1$  dividido por el polinomio  $p_2$ . Los argumentos  $x_1, \dots, x_n$  se interpretan como en la función `ratvars`.

La función `quotient` devuelve el primer elemento de la lista devuelta por `divide`.

**rat** ( $expr$ ) Función  
**rat** ( $expr, x_1, \dots, x_n$ ) Función

Convierte  $expr$  al formato canónico racional (canonical rational expression o CRE) expandiendo y combinando todos los términos sobre un denominador común y cancelando el máximo común divisor del numerador y denominador, así como convirtiendo números decimales en coma flotante a números racionales dentro de la tolerancia indicada por `ratesilon`. Las variables se ordenan de acuerdo a  $x_1, \dots, x_n$  si se han especificado, como en la función `ratvars`.

En general, `rat` no simplifica otras funciones que no sean la suma  $+$ , resta  $-$ , multiplicación  $*$ , división  $/$  y exponenciación de exponente entero, mientras que `ratsimp` sí lo hace. Nótese que los átomos (números y variables) en expresiones en formato CRE no son los mismos que en el formato general. Por ejemplo, `rat(x) - x` devuelve `rat(0)`, que tiene una representación interna diferente de 0.

Si `ratprint` vale `false` no aparecerán mensajes informando al usuario sobre la conversión de números decimales en coma flotante a números racionales.

Si `keepfloat` vale `true` no se convertirán números decimales en coma flotante a números racionales.

Véanse también `ratexpand` y `ratsimp`.

Ejemplos:

```
(%i1) ((x - 2*y)^4/(x^2 - 4*y^2)^2 + 1)*(y + a)*(2*y + x)/(4*y^2 + x^2);
```

```

 (x - 2 y)
 (y + a) (2 y + x) (----- + 1)
 2 2 2
 (x - 4 y)
(%o1) -----
 2 2
 4 y + x
(%i2) rat (% , y, a, x);
 2 a + 2 y
(%o2) /R/ -----
 x + 2 y

```

**ratalgdenom**

Variable opcional

Valor por defecto: `true`

Si `ratalgdenom` vale `true`, se permite la racionalización de denominadores eliminando radicales. La variable `ratalgdenom` sólo tiene efecto cuando expresiones en formato canónico (CRE) están siendo utilizadas en modo algebraico.

**ratcoef** (*expr*, *x*, *n*)

Función

**ratcoef** (*expr*, *x*)

Función

Devuelve el coeficiente de la expresión  $x^n$  dentro de la expresión *expr*. Si se omite, *n* se considera igual a 1.

El valor devuelto está libre de las variables en *x*, excepto quizás en un sentido no racional. Si no existe un coeficiente de este tipo se devuelve 0.

La función `ratcoef` expande y simplifica racionalmente su primer argumento, por lo que puede dar una respuesta diferente a la dada por la función `coeff`, la cual tiene un carácter puramente sintáctico.

La llamada `ratcoef (expr, x, 0)`, siendo *expr* una suma, devuelve una suma formada por los términos que no contienen *x*.

Puesto que *expr* se simplifica racionalmente antes de ser examinada, algunos coeficientes puede que no aparezcan como en la expresión original.

Ejemplo:

```

(%i1) s: a*x + b*x + 5$
(%i2) ratcoef (s, a + b);
(%o2) x

```

**ratdenom** (*expr*)

Función

Devuelve el denominador de *expr*, después de transformar *expr* al formato canónico (CRE). El valor retornado está también en formato CRE.

El argumento *expr* se transforma al formato CRE por la función `rat`, a menos que ya esté en este formato. Esta conversión puede cambiar la forma de *expr* colocando todos sus términos sobre un denominador común.

La función `denom` es parecida, pero devuelve una expresión general en lugar de una CRE. Tampoco `denom` intenta colocar todos sus términos sobre un denominador común, de manera que algunas expresiones que son consideradas como divisiones por `ratdenom`, no son tales para `denom`.



**ratdenomdivide**

Variable opcional

Valor por defecto: `true`

Si `ratdenomdivide` vale `true`, la función `ratexpand` expande una fracción en la que el numerador es una suma en una suma de divisiones. En otro caso, `ratexpand` reduce una suma de divisiones a una única fracción, cuyo numerador es la suma de los denominadores de cada fracción.

Examples:

```
(%i1) expr: (x^2 + x + 1)/(y^2 + 7);
```

$$\frac{x^2 + x + 1}{y^2 + 7}$$

```
(%o1)
```

```
(%i2) ratdenomdivide: true$
```

```
(%i3) ratexpand (expr);
```

$$\frac{x^2}{y^2 + 7} + \frac{x}{y^2 + 7} + \frac{1}{y^2 + 7}$$

```
(%o3)
```

```
(%i4) ratdenomdivide: false$
```

```
(%i5) ratexpand (expr);
```

$$\frac{x^2 + x + 1}{y^2 + 7}$$

```
(%o5)
```

```
(%i6) expr2: a^2/(b^2 + 3) + b/(b^2 + 3);
```

$$\frac{b}{b^2 + 3} + \frac{a^2}{b^2 + 3}$$

```
(%o6)
```

```
(%i7) ratexpand (expr2);
```

$$\frac{b + a^2}{b^2 + 3}$$

```
(%o7)
```

**ratdiff** (*expr*, *x*)

Función

Deriva la expresión racional *expr* con respecto a *x*. El argumento *expr* debe ser una fracción algebraica o un polinomio en *x*. El argumento *x* puede ser una variable o una subexpresión de *expr*.

El resultado equivale al devuelto por `diff`, aunque es posible que se obtenga en una forma diferente. La función `ratdiff` puede ser más rápida que `diff` en expresiones racionales.

La función `ratdiff` devuelve una expresión en formato canónico o CRE si `expr` es también una expresión CRE. En otro caso, `ratdiff` devuelve una expresión general. La función `ratdiff` considera únicamente la dependencia de `expr` respecto de `x`, ignorando cualquier dependencia establecida por `depends`.

Ejemplo:

```
(%i1) expr: (4*x^3 + 10*x - 11)/(x^5 + 5);
 3
 4 x + 10 x - 11
(%o1) -----
 5
 x + 5

(%i2) ratdiff (expr, x);
 7 5 4 2
 8 x + 40 x - 55 x - 60 x - 50
(%o2) -----
 10 5
 x + 10 x + 25

(%i3) expr: f(x)^3 - f(x)^2 + 7;
 3 2
 f (x) - f (x) + 7
(%o3) -----
 2
 3 f (x) - 2 f(x)
(%i4) ratdiff (expr, f(x));
 2
 3 f (x) - 2 f(x)
(%o4) -----
 3 2
 (b + a) + (b + a)
(%i5) expr: (a + b)^3 + (a + b)^2;
 3 2
 (b + a) + (b + a)
(%i6) ratdiff (expr, a + b);
 2 2
 3 b + (6 a + 2) b + 3 a + 2 a
(%o6) -----
```

### **ratdisrep** (*expr*)

Función

Devuelve su argumento como una expresión general. Si `expr` es una expresión general, se devuelve sin cambios.

Normalmente se invoca a `ratdisrep` a fin de convertir una expresión en formato canónico (CRE) al formato general, lo que puede ser utilizado si se quiere parar el contagio que produce el formato CRE, o para utilizar funciones racionales en contextos no racionales.

Véase también `totaldisrep`.

### **ratepsilon**

Variable opcional

Valor por defecto: 2.0e-8

La variable `ratepsilon` guarda la tolerancia utilizada en la conversión de números decimales en coma flotante a números racionales.

### **ratexpand** (*expr*)

Función

### **ratexpand**

Variable opcional

Expande `expr` multiplicando productos de sumas y sumas con exponentes, combinando fracciones con común denominador, cancelando el máximo común divisor del

numerador y del denominador y luego dividiendo los sumandos del numerador por el denominador.

El valor que devuelve `ratexpand` es una expresión general, incluso cuando `expr` está en formato canónico o CRE.

Si la variable `ratexpand` vale `true` hará que las expresiones CRE se expandan completamente cuando se conviertan al formato general o se muestren en el terminal, mientras que si vale `false` se mostrarán de forma recursiva. Véase también `ratsimp`.

Si `ratdenomdivide` vale `true`, `ratexpand` expande una fracción en la que el numerador es una suma en una suma de fracciones, todas ellas con denominador común. En otro caso, `ratexpand` reduce una suma de fracciones en una única fracción, cuyo numerador es la suma de los numeradores de cada fracción.

Si `keepfloat` vale `true`, los números decimales en coma flotante no se racionalizan cuando las expresiones que los contienen se convierten al formato canónico racional (CRE).

Ejemplos:

```
(%i1) ratexpand ((2*x - 3*y)^3);
(%o1)
 3 2 2 3
 - 27 y + 54 x y - 36 x y + 8 x
(%i2) expr: (x - 1)/(x + 1)^2 + 1/(x - 1);
(%o2)
 x - 1 1
 ----- + -----
 2 x - 1
 (x + 1)
(%i3) expand (expr);
(%o3)
 x 1 1
 ----- - ----- + -----
 2 2 x - 1
 x + 2 x + 1 x + 2 x + 1
(%i4) ratexpand (expr);
(%o4)
 2 x 2
 ----- + -----
 3 2 3 2
 x + x - x - 1 x + x - x - 1
```

## ratfac

Variable opcional

Valor por defecto: `false`

Si `ratfac` vale `true`, las expresiones canónicas (CRE) se manipulan en una forma parcialmente factorizada.

Durante las operaciones racionales, las expresiones se mantienen completamente factorizadas tanto como sea posible sin llamar a `factor`. Esto debería ahorrar espacio y tiempo en algunos cálculos. El numerador y denominador se hacen primos relativos, por ejemplo `rat ((x^2 - 1)^4/(x + 1)^2)` devuelve  $(x - 1)^4 (x + 1)^2$ , pero los factores dentro de cada parte pueden no ser primos relativos.

En el paquete `ctensr` sobre manipulación de tensores por componentes, los tensores de Ricci, Einstein, Riemann y Weyl y la curvatura escalar se factorizan automáticamente

si **ratfac** vale **true**; **ratfac** debe activarse únicamente en aquellos casos en los que se sabe que el número de términos de las componentes tensoriales es pequeño.

Nota: Los esquemas de comportamiento basados en **ratfac** y **ratweight** son incompatibles y no se debe pretender usarlos al mismo tiempo.

**ratnumber** (*expr*) Función

Devuelve el numerador de *expr*, después de reducir *expr* a su forma canónica (CRE). El valor retornado está también en formato CRE.

El argumento *expr* se transforma al formato CRE por la función **rat**, a menos que ya esté en este formato. Esta conversión puede cambiar la forma de *expr* colocando todos sus términos sobre un denominador común.

Es parecida a la función **num**, pero devuelve una expresión general en lugar de una CRE. Además, **num** no intenta colocar todos los términos sobre un denominador común, de manera que algunas expresiones que son consideradas fracciones por **ratnumber** no se consideran como tales por **num**.

**ratnump** (*expr*) Función

Devuelve **true** si *expr* es un entero literal o una fracción de enteros literales, en caso contrario devuelve **false**.

**ratp** (*expr*) Función

Devuelve **true** si *expr* es una expresión canónica racional (canonical rational expression o CRE) o una CRE extendida, en caso contrario devuelve **false**.

Las expresiones CRE son creadas por **rat** y funciones asociadas. Las CRE extendidas son creadas por **taylor** y funciones asociadas.

**ratprint** Variable opcional

Valor por defecto: **true**

Si **ratprint** vale **true**, se muestra al usuario un mensaje dando cuenta de la conversión de números decimales en coma flotante a formato racional.

**ratsimp** (*expr*) Función

**ratsimp** (*expr*, *x\_1*, ..., *x\_n*) Función

Simplifica la expresión *expr* y todas sus subexpresiones, incluyendo los argumentos de funciones no racionales. El resultado es un cociente de dos polinomios en una forma recursiva, esto es, los coeficientes de la variable principal son polinomios respecto de las otras variables. Las variables pueden incluir funciones no racionales, como **sin** (*x*<sup>2</sup> + 1), y los argumentos de tales funciones son también racionalmente simplificados.

La llamada **ratsimp** (*expr*, *x\_1*, ..., *x\_n*) permite la simplificación racional con la especificación del orden de las variables, como en **ratvars**.

Si **ratsimpexpons** vale **true**, **ratsimp** se aplica a los exponentes de las expresiones durante la simplificación.

Véase también **ratexpand**. Nótese que **ratsimp** se ve afectada por algunas de las variables globales que controlan a **ratexpand**.

Ejemplos:

```
(%i1) sin (x/(x^2 + x)) = exp ((log(x) + 1)^2 - log(x)^2);
 2 2
(%o1) sin(-----) = %e
 x (log(x) + 1) - log (x)
 2
 x + x
(%i2) ratsimp (%);
 1 2
(%o2) sin(-----) = %e x
 x + 1
(%i3) ((x - 1)^(3/2) - (x + 1)*sqrt(x - 1))/sqrt((x - 1)*(x + 1));
 3/2
 (x - 1) - sqrt(x - 1) (x + 1)
(%o3) -----
 sqrt((x - 1) (x + 1))
(%i4) ratsimp (%);
 2 sqrt(x - 1)
(%o4) - -----
 2
 sqrt(x - 1)
(%i5) x^(a + 1/a), ratsimpexpons: true;
 2
 a + 1

 a
(%o5) x
```

**ratsimpexpons**

Variable opcional

Valor por defecto: `false`

Si `ratsimpexpons` vale `true`, `ratsimp` se aplica a los exponentes de las expresiones durante la simplificación.

**ratsubst** (*a*, *b*, *c*)

Función

Sustituye *b* por *a* en *c* y devuelve la expresión resultante. El argumento *b* puede ser una suma, un producto, una potencia, etc.

La función `ratsubst` reconoce el significado de las expresiones, mientras que `subst` tan solo realiza sustituciones sintácticas. Así por ejemplo, `subst (a, x + y, x + y + z)` devuelve `x + y + z` cuando `ratsubst` devuelve `z + a`.

Si `radsubstflag` vale `true`, `ratsubst` sustituye radicales en expresiones que no los contienen explícitamente.

Ejemplos:

```
(%i1) ratsubst (a, x*y^2, x^4*y^3 + x^4*y^8);
 3 4
(%o1) a x y + a
(%i2) cos(x)^4 + cos(x)^3 + cos(x)^2 + cos(x) + 1;
 4 3 2
(%o2) cos (x) + cos (x) + cos (x) + cos(x) + 1
```

```
(%i3) ratsubst (1 - sin(x)^2, cos(x)^2, %);
 4 2 2
(%o3) sin (x) - 3 sin (x) + cos(x) (2 - sin (x)) + 3
(%i4) ratsubst (1 - cos(x)^2, sin(x)^2, sin(x)^4);
 4 2
(%o4) cos (x) - 2 cos (x) + 1
(%i5) radsubstflag: false$
(%i6) ratsubst (u, sqrt(x), x);
(%o6) x
(%i7) radsubstflag: true$
(%i8) ratsubst (u, sqrt(x), x);
 2
(%o8) u
```

**ratvars** ( $x_1, \dots, x_n$ )

Función

**ratvars** ()

Función

**ratvars**

Variable del sistema

Declara como variables principales  $x_1, \dots, x_n$  en expresiones racionales. Si  $x_n$  está presente en una expresión racional, se considerará como variable principal. Si no está presente, entonces se considerará principal a la variable  $x_{[n-1]}$  si aparece en la expresión, se continúa así hasta  $x_1$ , que se considerará como variable principal sólo si ninguna de las variables que le siguen está presente en la expresión.

Si una variable de la expresión racional no está presente en la lista **ratvars**, se le dará una prioridad inferior a la de  $x_1$ .

Los argumentos de **ratvars** pueden ser tanto variables como funciones no racionales como **sin(x)**.

La variable **ratvars** es una lista que contiene los argumentos pasados a la función **ratvars** la última vez que fue invocada. Cada llamada a la función **ratvars** reinicializa la lista. La llamada **ratvars** () vacía la lista.

**ratweight** ( $x_1, w_1, \dots, x_n, w_n$ )

Función

**ratweight** ()

Función

Asigna un peso  $w_i$  a la variable  $x_i$ . Un término será reemplazado por 0 si su peso excede el valor de la variable **ratwtlvl** (por defecto no se realiza el truncamiento). El peso de un término es la suma de los productos de los pesos de las variables que lo forman multiplicados por sus exponentes. Por ejemplo, el peso de  $3 x_1^2 x_2$  es  $2 w_1 + w_2$ . El truncamiento basado en **ratwtlvl** solamente se lleva a cabo cuando se multiplican o se elevan a potencias expresiones canónicas (CRE).

La llamada **ratweight** () devuelve la lista acumulada de asignaciones de pesos.

Nota: Los esquemas de comportamiento basados en **ratfac** y **ratweight** son incompatibles y no se debe pretender usarlos al mismo tiempo.

Ejemplos:

```
(%i1) ratweight (a, 1, b, 1);
(%o1) [a, 1, b, 1]
(%i2) expr1: rat(a + b + 1)$
(%i3) expr1^2;
```

```

 2 2
(%o3)/R/ b + (2 a + 2) b + a + 2 a + 1
(%i4) ratwtlvl: 1$
(%i5) expr1^2;
(%o5)/R/ 2 b + 2 a + 1

```

**ratweights**

Variable del sistema

Valor por defecto: []

La variable **ratweights** es una lista que contiene los pesos asignados por **ratweight**. La lista es acumulativa, en el sentido de que cada llamada a **ratweight** añade nuevos elementos a la lista.

**ratwtlvl**

Variable opcional

Valor por defecto: **false**

La variable **ratwtlvl** se utiliza en combinación con la función **ratweight** para controlar el truncamiento de expresiones racionales canónicas (CRE). Con el valor por defecto, **false**, no se produce truncamiento alguno.

**remainder** (*p-1*, *p-2*)

Función

**remainder** (*p-1*, *p-2*, *x-1*, ..., *x-n*)

Función

Devuelve el resto de la división del polinomio *p-1* entre *p-2*. Los argumentos *x-1*, ..., *x-n* se interpretan como en **ratvars**.

La función **remainder** devuelve el segundo elemento de la lista retornada por **divide**.

**resultant** (*p-1*, *p-2*, *x*)

Función

**resultant**

Variable

Calcula la resultante de los dos polinomios *p-1* y *p-2*, eliminando la variable *x*. La resultante es un determinante de los coeficientes de *x* en *p-1* y *p-2*, que es igual a cero si sólo si *p-1* y *p-2* tienen un factor común no constante.

Si *p-1* o *p-2* pueden ser factorizados, puede ser necesario llamar a **factor** antes que invocar a **resultant**.

La variable **resultant** controla qué algoritmo será utilizado para calcular la resultante.

La función **bezout** toma los mismos argumentos que **resultant** y devuelve una matriz. El determinante del valor retornado es la resultante buscada.

**savefactors**

Variable opcional

Valor por defecto: **false**

Si **savefactors** vale **true**, los factores de una expresión producto se almacenan por ciertas funciones a fin de acelerar posteriores factorizaciones de expresiones que tengan algunos de estos factores.

**tellrat** (*p-1*, ..., *p-n*)

Función

**tellrat** ()

Función

Añade al anillo de enteros algebraicos conocidos por Maxima los elementos que son soluciones de los polinomios *p-1*, ..., *p-n*. Cada argumento *p-i* es un polinomio de coeficientes enteros.

La llamada `tellrat (x)` hace que se sustituya 0 por  $x$  en las funciones racionales.

La llamada `tellrat ()` devuelve una lista con las sustituciones actuales.

A la variable `algebraic` se le debe asignar el valor `true` a fin de poder realizar la simplificación de enteros algebraicos.

Maxima reconoce la unidad imaginaria `%i` y todas las raíces de los enteros.

La instrucción `untellrat` borra todas las propiedades de `tellrat`.

Es ambiguo aplicar `tellrat` a un polinomio multivariante tal como `tellrat (x^2 - y^2)`, pues no se sabe si sustituir  $y^2$  por  $x^2$  o al revés. Maxima sigue un cierto orden, pero si el usuario quiere especificar uno en concreto, puede hacerlo mediante la sintaxis `tellrat (y^2 = x^2)`, que indica que se ponga  $x^2$  en lugar de  $y^2$ .

Ejemplos:

```
(%i1) 10*(%i + 1)/(%i + 3^(1/3));
(%o1)

$$\frac{10 (\%i + 1)}{\%i + 3^{1/3}}$$

(%i2) ev (ratdisrep (rat(%)), algebraic);
(%o2)

$$\frac{(4^2/3 - 2^2/3 - 4) \%i + 2^2/3 + 4^2/3 - 2}{(1 + a + a^2)}$$

(%i3) tellrat (1 + a + a^2);
(%o3)

$$[a^2 + a + 1]$$

(%i4) 1/(a*sqrt(2) - 1) + a/(sqrt(3) + sqrt(2));
(%o4)

$$\frac{1}{\sqrt{2} a - 1} + \frac{a}{\sqrt{3} + \sqrt{2}}$$

(%i5) ev (ratdisrep (rat(%)), algebraic);
(%o5)

$$\frac{(7 \sqrt{3} - 10 \sqrt{2} + 2) a - 2 \sqrt{2} - 1}{7}$$

(%i6) tellrat (y^2 = x^2);
(%o6)

$$[y^2 - x^2, a^2 + a + 1]$$

```

### **totaldisrep** (*expr*)

Función

Convierte cada subexpresión de *expr* del formato canónico (CRE) al general y devuelve el resultado. Si *expr* está en formato CRE entonces `totaldisrep` es idéntico a `ratdisrep`.

La función `totaldisrep` puede ser útil para modificar expresiones como las ecuaciones, listas, matrices, etc., que tienen algunas subexpresiones en formato CRE.

### **untellrat** (*x\_1*, ..., *x\_n*)

Función

Elimina de *x\_1*, ..., *x\_n* las propiedades relacionadas con `tellrat`.





## 13 Constantes

### 13.1 Definiciones para Constantes

|                 |                                                                                     |           |
|-----------------|-------------------------------------------------------------------------------------|-----------|
| <b>%e</b>       | La base de los logaritmos naturales, $e$ , se representa en Maxima como <b>%e</b> . | Constante |
| <b>false</b>    | Constante lógica para "falso". (NIL en Lisp)                                        | Constante |
| <b>inf</b>      | Más infinito.                                                                       | Constante |
| <b>infinity</b> | Infinito complejo.                                                                  | Constante |
| <b>minf</b>     | Menos infinito.                                                                     | Constante |
| <b>%pi</b>      | El valor "pi" se representa en Maxima como <b>%pi</b> .                             | Constante |
| <b>true</b>     | Constante lógica para "verdadero". (T en Lisp)                                      | Constante |



## 14 Logaritmos

### 14.1 Definiciones para Logaritmos

#### `%e_to_numlog`

Variable opcional

Valor por defecto: `false`

Si `%e_to_numlog` vale `true`, `r` es un número racional y `x` una expresión, `%e(r*log(x))` se reduce a `xr`. Téngase en cuenta que la instrucción `radcan` también hace este tipo de transformaciones, así como otras más complicadas. La instrucción `logcontract` "contrae" expresiones que contienen algún `log`.

#### `li [s] (z)`

Función

Representa la función polilogarítmica de orden `s` y argumento `z`, definida por la serie infinita

$$\text{Li}_s(z) = \sum_{k=1}^{\infty} \frac{z^k}{k^s}$$

`li [1]` es  $-\log(1 - z)$ . `li [2]` y `li [3]` son las funciones di- y trilogaritmo, respectivamente.

Cuando el orden es 1, el polilogaritmo se simplifica a  $-\log(1 - z)$ , el cual a su vez se reduce a un valor numérico si `z` es un número real o complejo en coma flotante o si está presente el término `numer`.

Cuando el orden es 2 ó 3, el polilogaritmo se reduce a un valor numérico si `z` es un número real en coma flotante o si está presente el término `numer`.

Ejemplos:

```
(%i1) assume (x > 0);
(%o1) [x > 0]
(%i2) integrate ((log (1 - t)) / t, t, 0, x);
(%o2) - li (x)
 2
(%i3) li [2] (7);
(%o3) li (7)
 2
(%i4) li [2] (7), numer;
(%o4) 1.24827317833392 - 6.113257021832577 %i
(%i5) li [3] (7);
(%o5) li (7)
 3
(%i6) li [2] (7), numer;
(%o6) 1.24827317833392 - 6.113257021832577 %i
(%i7) L : makelist (i / 4.0, i, 0, 8);
```

```
(%o7) [0.0, 0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0]
(%i8) map (lambda ([x], li [2] (x)), L);
(%o8) [0, .2676526384986274, .5822405249432515,
.9784693966661848, 1.64493407, 2.190177004178597
- .7010261407036192 %i, 2.374395264042415
- 1.273806203464065 %i, 2.448686757245154
- 1.758084846201883 %i, 2.467401098097648
- 2.177586087815347 %i]
(%i9) map (lambda ([x], li [3] (x)), L);
(%o9) [0, .2584613953442624, 0.537213192678042,
.8444258046482203, 1.2020569, 1.642866878950322
- .07821473130035025 %i, 2.060877505514697
- .2582419849982037 %i, 2.433418896388322
- .4919260182322965 %i, 2.762071904015935
- .7546938285978846 %i]
```

**log (x)**

Función

Representa el logaritmo natural (en base  $e$ ) de  $x$ .

Maxima no tiene definida una función para el logaritmo de base 10 u otras bases. El usuario puede hacer uso de la definición  $\log_{10}(x) := \log(x) / \log(10)$ .

La simplificación y evaluación de logaritmos se controla cciertas variables globales:

**logexpand** - hace que  $\log(a^b)$  se convierta en  $b \cdot \log(a)$ . Si toma el valor **all**,  $\log(a \cdot b)$  también se reducirá a  $\log(a) + \log(b)$ . Si toma el valor **super**, entonces  $\log(a/b)$  también se reducirá a  $\log(a) - \log(b)$ , siendo  $a/b$  racional y  $a \neq 1$ , (la expresión  $\log(1/b)$ , para  $b$  entero, se simplifica siempre). Si toma el valor **false**, se desactivarán todas estas simplificaciones.

**logsimp** - si vale **false**, entonces no se transforma  $e$  a potencias que contengan logaritmos.

**lognumber** - si vale **true**, entonces los argumentos de **log** que sean números decimales negativos en coma flotante se convertirán siempre a su valor absoluto antes de aplicar **log**. Si **numer** vale también **true**, entonces los argumentos enteros negativos de **log** también se convertirán en su valor absoluto.

**lognegint** - si vale **true** se aplica la regla  $\log(-n) \rightarrow \log(n) + i \cdot \pi$  siendo  $n$  un entero positivo.

**%e\_to\_numlog** - si vale **true**,  $r$  es un número racional y  $x$  una expresión,  $e^{(r \cdot \log(x))}$  se reduce a  $x^r$ . Téngase en cuenta que la instrucción **radcan** también hace este tipo de transformaciones, así como otras más complicadas. La instrucción **logcontract** "contrae" expresiones que contienen algún **log**.

**logabs**

Variable opcional

Valor por defecto: **false**

Cuando se calculan integrales indefinidas en las que se generan logaritmos, como en **integrate(1/x,x)**, el resultado se devuelve de la forma  $\log(\text{abs}(\dots))$  si **logabs** vale **true**, o de la forma  $\log(\dots)$  si **logabs** vale **false**. En la integración definida se hace la asignación **logabs:true**, ya que aquí es normalmente necesario evaluar la integral indefinida en los extremos del intervalo de integración.

**logarc**

Variable opcional

Valor por defecto: `false`

Si `logarc` vale `true` hará que las funciones circulares e hiperbólicas inversas se conviertan a la forma logarítmica. La instrucción `logarc(exp)` hará esta conversión sólo en la expresión `exp` sin cambiar el valor de la variable global ni tener que volver a evaluar la expresión con `ev`.

**logconcoeffp**

Variable opcional

Valor por defecto: `false`

Controla qué coeficientes se contraen cuando se utiliza `logcontract`. Se le puede asignar el nombre de una función de predicado de un argumento; por ejemplo, si se quiere introducir raíces cuadradas, se puede hacer `logconcoeffp: 'logconfun$ logconfun(m):=featurep(m,integer) or ratnum(m)$`. Entonces `logcontract(1/2*log(x))`; devolverá `log(sqrt(x))`.

**logcontract (expr)**

Función

Analiza la expresión `expr` recursivamente, transformando subexpresiones de la forma `a1*log(b1) + a2*log(b2) + c` en `log(ratsimp(b1^a1 * b2^a2)) + c`

```
(%i1) 2*(a*log(x) + 2*a*log(y))$
(%i2) logcontract(%);

 2 4
(%o2) a log(x y)
```

Si se hace `declare(n,integer)`; entonces `logcontract(2*a*n*log(x))`; da `a*log(x^(2*n))`. Los coeficientes que se contraen de esta manera son aquellos que como el 2 y el n satisfacen `featurep(coeff,integer)`. El usuario puede controlar qué coeficientes se contraen asignándole a la variable global `logconcoeffp` el nombre de una función de predicado de un argumento; por ejemplo, si se quiere introducir raíces cuadradas, se puede hacer `logconcoeffp: 'logconfun$ logconfun(m):=featurep(m,integer) or ratnum(m)$`. Entonces `logcontract(1/2*log(x))`; devolverá `log(sqrt(x))`.

**logexpand**

Variable opcional

Valor por defecto: `true`

Si `logexpand` vale `true` hace que `log(a^b)` se convierta en `b*log(a)`. Si toma el valor `all`, `log(a*b)` también se reducirá a `log(a)+log(b)`. Si toma el valor `super`, entonces `log(a/b)` también se reducirá a `log(a)-log(b)`, siendo `a/b` racional y `a#1`, (la expresión `log(1/b)`, para `b` entero, se simplifica siempre). Si toma el valor `false`, se desactivarán todas estas simplificaciones.

**lognegint**

Variable opcional

Valor por defecto: `false`

Si `lognegint` vale `true` se aplica la regla `log(-n) -> log(n)+%i*pi` siendo `n` un entero positivo.

**lognumber**

Variable opcional

Valor por defecto: `false`

Si `lognumber` vale `true`, entonces los argumentos de `log` que sean números decimales negativos en coma flotante se convertirán siempre a su valor absoluto antes de aplicar `log`. Si `numer` vale también `true`, entonces los argumentos enteros negativos de `log` también se convertirán en su valor absoluto.

**logsimp**

Variable opcional

Valor por defecto: `true`

Si `logsimp` vale `false`, entonces no se transforma `%e` a potencias que contengan logaritmos.

**plog** (*x*)

Función

Representa la rama principal del logaritmo natural complejo con  $-\pi < \text{carg}(x) \leq \pi$ .

## 15 Trigonometría

### 15.1 Introducción a la Trigonometría

Maxima reconoce muchas funciones trigonométricas. No están programadas todas las identidades trigonométricas, pero el usuario puede añadir muchas de ellas haciendo uso de las técnicas basadas en patrones. Las funciones trigonométricas definidas en Maxima son: `acos`, `acosh`, `acot`, `acoth`, `acsc`, `acsch`, `asec`, `asech`, `asin`, `asinh`, `atan`, `atanh`, `cos`, `cosh`, `cot`, `coth`, `csc`, `csch`, `sec`, `sech`, `sin`, `sinh`, `tan` y `tanh`. Hay también un determinado número de instrucciones especiales para manipular funciones trigonométricas; véanse a este respecto `trigexpand`, `trigreduce` y la variable `trigsign`. Dos paquetes adicionales amplían las reglas de simplificación de Maxima, `ntrig` y `atrig1`. Ejecútense `describe(command)` para más detalles.

### 15.2 Definiciones para Trigonometría

|                                        |          |
|----------------------------------------|----------|
| <b><code>acos</code></b> ( <i>x</i> )  | Function |
| Arco coseno.                           |          |
| <b><code>acosh</code></b> ( <i>x</i> ) | Función  |
| Arco coseno hiperbólico.               |          |
| <b><code>acot</code></b> ( <i>x</i> )  | Función  |
| Arco cotangente.                       |          |
| <b><code>acoth</code></b> ( <i>x</i> ) | Función  |
| Arco cotangente hiperbólica.           |          |
| <b><code>acsc</code></b> ( <i>x</i> )  | Función  |
| Arco cosecante.                        |          |
| <b><code>acsch</code></b> ( <i>x</i> ) | Función  |
| Arco cosecante hiperbólica.            |          |
| <b><code>asec</code></b> ( <i>x</i> )  | Función  |
| Arco secante.                          |          |
| <b><code>asech</code></b> ( <i>x</i> ) | Función  |
| Arco secante hiperbólica.              |          |
| <b><code>asin</code></b> ( <i>x</i> )  | Función  |
| Arco seno.                             |          |
| <b><code>asinh</code></b> ( <i>x</i> ) | Función  |
| Arco seno hiperbólico.                 |          |



|                                                                                                                                                                                                                                                                                                                                                                                                                                     |         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| <b>atan</b> ( $x$ )                                                                                                                                                                                                                                                                                                                                                                                                                 | Función |
| Arco tangente.                                                                                                                                                                                                                                                                                                                                                                                                                      |         |
| <b>atan2</b> ( $y, x$ )                                                                                                                                                                                                                                                                                                                                                                                                             | Función |
| Calcula el valor de <b>atan</b> ( $y/x$ ) en el intervalo de $-\pi$ a $\pi$ .                                                                                                                                                                                                                                                                                                                                                       |         |
| <b>atanh</b> ( $x$ )                                                                                                                                                                                                                                                                                                                                                                                                                | Función |
| Arco tangente hiperbólica.                                                                                                                                                                                                                                                                                                                                                                                                          |         |
| <b>atrig1</b>                                                                                                                                                                                                                                                                                                                                                                                                                       | Paquete |
| El paquete <b>atrig1</b> contiene ciertas reglas de simplificación adicionales para las funciones trigonométricas inversas. Junto con las reglas que ya conoce Maxima, los siguientes ángulos están completamente implementados: $0$ , $\pi/6$ , $\pi/4$ , $\pi/3$ y $\pi/2$ . Los ángulos correspondientes en los otros tres cuadrantes también están disponibles. Para hacer uso de estas reglas, ejecútese <b>load(atrig1)</b> ; |         |
| <b>cos</b> ( $x$ )                                                                                                                                                                                                                                                                                                                                                                                                                  | Función |
| Coseno.                                                                                                                                                                                                                                                                                                                                                                                                                             |         |
| <b>cosh</b> ( $x$ )                                                                                                                                                                                                                                                                                                                                                                                                                 | Función |
| Coseno hiperbólico.                                                                                                                                                                                                                                                                                                                                                                                                                 |         |
| <b>cot</b> ( $x$ )                                                                                                                                                                                                                                                                                                                                                                                                                  | Función |
| Cotangente.                                                                                                                                                                                                                                                                                                                                                                                                                         |         |
| <b>coth</b> ( $x$ )                                                                                                                                                                                                                                                                                                                                                                                                                 | Función |
| Cotangente hiperbólica.                                                                                                                                                                                                                                                                                                                                                                                                             |         |
| <b>csc</b> ( $x$ )                                                                                                                                                                                                                                                                                                                                                                                                                  | Función |
| Cosecante.                                                                                                                                                                                                                                                                                                                                                                                                                          |         |
| <b>csch</b> ( $x$ )                                                                                                                                                                                                                                                                                                                                                                                                                 | Función |
| Cosecante hiperbólica.                                                                                                                                                                                                                                                                                                                                                                                                              |         |
| <b>ntrig</b>                                                                                                                                                                                                                                                                                                                                                                                                                        | Paquete |
| El paquete <b>ntrig</b> contiene un conjunto de reglas de simplificación que se pueden usar para simplificar funciones trigonométricas cuyos argumentos son de la forma $f(n\pi/10)$ donde $f$ es cualquiera de las funciones <b>sin</b> , <b>cos</b> , <b>tan</b> , <b>csc</b> , <b>sec</b> o <b>cot</b> .                                                                                                                         |         |
| <b>sec</b> ( $x$ )                                                                                                                                                                                                                                                                                                                                                                                                                  | Función |
| Secante.                                                                                                                                                                                                                                                                                                                                                                                                                            |         |
| <b>sech</b> ( $x$ )                                                                                                                                                                                                                                                                                                                                                                                                                 | Función |
| Secante hiperbólica.                                                                                                                                                                                                                                                                                                                                                                                                                |         |

**sin** (*x*) Función  
Seno.

**sinh** (*x*) Función  
Seno hiperbólico.

**tan** (*x*) Función  
Tangente.

**tanh** (*x*) Función  
Tangente hiperbólica.

**trigexpand** (*expr*) Función

Expande funciones trigonométricas e hiperbólicas de sumas de ángulos y de múltiplos de ángulos presentes en *expr*. Para mejorar los resultados, *expr* debería expandirse. Para facilitar el control por parte del usuario de las simplificaciones, esta función tan solo expande un nivel de cada vez, expandiendo sumas de ángulos o de múltiplos de ángulos. A fin de obtener una expansión completa en senos y coseno, se le dará a la variable **trigexpand** el valor **true**.

La función **trigexpand** está controlada por las siguientes variables:

**trigexpand**  
Si vale **true**, provoca la expansión de todas las expresiones que contengan senos y cosenos.

**trigexpandplus**  
Controla la regla de la suma para **trigexpand**, la expansión de una suma como  $\sin(x + y)$  se llevará a cabo sólo si **trigexpandplus** vale **true**.

**trigexpandtimes**  
Controla la regla del producto para **trigexpand**, la expansión de un producto como  $\sin(2x)$  se llevará a cabo sólo si **trigexpandtimes** vale **true**.

Ejemplos:

```
(%i1) x+sin(3*x)/sin(x),trigexpand=true,expand;
 2 2
(%o1) - sin (x) + 3 cos (x) + x
(%i2) trigexpand(sin(10*x+y));
(%o2) cos(10 x) sin(y) + sin(10 x) cos(y)
```

**trigexpandplus** Variable optativa

Valor por defecto: **true**

La variable **trigexpandplus** controla la regla de la suma para **trigexpand**. Así, si la instrucción **trigexpand** se utiliza o si la variable **trigexpand** vale **true**, se realizará la expansión de sumas como  $\sin(x+y)$  sólo si **trigexpandplus** vale **true**.

**trigexpandtimes**

Variable optativa

Valor por defecto: `true`

La variable `trigexpandtimes` controla la regla del producto para `trigexpand`. Así, si la instrucción `trigexpand` se utiliza o si la variable `trigexpand` vale `true`, se realizará la expansión de productos como `sin(2*x)` sólo si `trigexpandtimes` vale `true`.

**triginverses**

Variable optativa

Valor por defecto: `all`

La variable `triginverses` controla la simplificación de la composición de funciones trigonométricas e hiperbólicas con sus funciones inversas.

Si vale `all`, tanto `atan(tan(x))` como `tan(atan(x))` se reducen a `x`.

Si vale `true`, se desactiva la simplificación de `arcfun(fun(x))`.

Si vale `false`, se desactivan las simplificaciones de `arcfun(fun(x))` y `fun(arcfun(x))`.

**trigreduce (expr, x)**

Función

**trigreduce (expr)**

Función

Combina productos y potencias de senos y cosenos trigonométricos e hiperbólicos de `x`, transformándolos en otros que son múltiplos de `x`. También intenta eliminar estas funciones cuando aparecen en los denominadores. Si no se introduce el argumento `x`, entonces se utilizan todas las variables de `expr`.

Véase también `poissimp`.

```
(%i1) trigreduce(-sin(x)^2+3*cos(x)^2+x);
(%o1) cos(2 x) cos(2 x) 1 1
 ----- + 3 (----- + -) + x - -
 2 2 2 2
```

Las rutinas de simplificación trigonométrica utilizan información declarada en algunos casos sencillos. Las declaraciones sobre variables se utilizan como se indica a continuación:

```
(%i1) declare(j, integer, e, even, o, odd)$
(%i2) sin(x + (e + 1/2)*%pi);
(%o2) cos(x)
(%i3) sin(x + (o + 1/2)*%pi);
(%o3) - cos(x)
```

**trigsign**

Variable optativa

Valor por defecto: `true`

Si `trigsign` vale `true`, se permite la simplificación de argumentos negativos en funciones trigonométricas, como en `sin(-x)`, que se transformará en `-sin(x)` sólo si `trigsign` vale `true`.

**trigsimp (expr)**

Función

Utiliza las identidades  $\sin(x)^2 + \cos(x)^2 = 1$  y  $\cosh(x)^2 - \sinh(x)^2 = 1$  para simplificar expresiones que contienen `tan`, `sec`, etc., en expresiones con `sin`, `cos`, `sinh`, `cosh`.

Las funciones `trigreduce`, `ratsimp` y `radcan` pueden seguir siendo útiles para continuar el proceso de simplificación.

La instrucción `demo ("trgsmp.dem")` muestra algunos ejemplos de `trigsimp`.

### **trigrat** (*expr*)

Función

Devuelve una forma canónica simplificada cuasi-lineal de una expresión trigonométrica; *expr* es una fracción racional que contiene `sin`, `cos` o `tan`, cuyos argumentos son formas lineales respecto de ciertas variables (o kernels) y  $\pi/n$  ( $n$  entero) con coeficientes enteros. El resultado es una fracción simplificada con el numerador y denominador lineales respecto de `sin` y `cos`.

```
(%i1) trigrat(sin(3*a)/sin(a+%pi/3));
(%o1) sqrt(3) sin(2 a) + cos(2 a) - 1
```

El siguiente ejemplo se ha tomado de Davenport, Siret y Tournier, *Calcul Formel*, Masson (o en inglés, Addison-Wesley), sección 1.5.5, teorema de Morley.

```
(%i1) c: %pi/3 - a - b;
(%o1) - b - a + ---
 %pi
 3
(%i2) bc: sin(a)*sin(3*c)/sin(a+b);
(%o2) sin(a) sin(3 b + 3 a)

 sin(b + a)
(%i3) ba: bc, c=a, a=c$
(%i4) ac2: ba^2 + bc^2 - 2*bc*ba*cos(b);
(%o4) 2 2
 sin (a) sin (3 b + 3 a)

 2
 sin (b + a)

 2 sin(a) sin(3 a) cos(b) sin(b + a - ---) sin(3 b + 3 a)
 3

 %pi
 sin(a - ---) sin(b + a)
 3

 2 2 %pi
 sin (3 a) sin (b + a - ---)
 3
 + -----
 2 %pi
 sin (a - ---)
 3

(%i5) trigrat (ac2);
(%o5) - (sqrt(3) sin(4 b + 4 a) - cos(4 b + 4 a))
```

$$\begin{aligned}
& - 2 \sqrt{3} \sin(4 b + 2 a) + 2 \cos(4 b + 2 a) \\
& - 2 \sqrt{3} \sin(2 b + 4 a) + 2 \cos(2 b + 4 a) \\
& + 4 \sqrt{3} \sin(2 b + 2 a) - 8 \cos(2 b + 2 a) - 4 \cos(2 b - 2 a) \\
& + \sqrt{3} \sin(4 b) - \cos(4 b) - 2 \sqrt{3} \sin(2 b) + 10 \cos(2 b) \\
& + \sqrt{3} \sin(4 a) - \cos(4 a) - 2 \sqrt{3} \sin(2 a) + 10 \cos(2 a) \\
& - 9)/4
\end{aligned}$$

## 16 Funciones Especiales

### 16.1 Introducción a las Funciones Especiales

### 16.2 specint

El paquete `hypgeo` permite calcular transformadas de Laplace de funciones especiales, mientras que `hyp` es otro paquete destinado al cálculo de funciones hipergeométricas generalizadas.

Por otro lado, `specint` tiene por objeto calcular la integral definida (en el intervalo que va desde cero hasta infinito) de expresiones que contengan funciones especiales. Cuando el integrando contenga el factor `exp (-s t)`, el resultado no es otro que la transformada de Laplace.

La sintaxis es la siguiente:

```
specint (exp (-s*t) * expr, t);
```

donde  $t$  es la variable de integración y  $expr$  es una expresión que contiene funciones especiales.

Si `specint` no puede calcular la integral, la respuesta que se obtiene puede contener símbolos de Lisp, entre los que se incluyen `other-defint-to-follow-negtest`, `other-lt-exponential-to-follow`, `product-of-y-with-nofract-indices`, etc.; este es un fallo conocido del programa.

A continuación se especifican las notaciones correspondientes a las funciones especiales:

|                                     |                                                       |
|-------------------------------------|-------------------------------------------------------|
| <code>bessel_j (index, expr)</code> | Función de Bessel de primera especie                  |
| <code>bessel_y (index, expr)</code> | Función de Bessel de segunda especie                  |
| <code>bessel_i (index, expr)</code> | Función de Bessel modificada de primera especie       |
| <code>bessel_k (index, expr)</code> | Función de Bessel modificada de segunda especie       |
| <code>%he[n] (z)</code>             | Polinomio de Hermite (Ojo: he, no h. Ver A&S 22.5.18) |
| <code>%p[u,v] (z)</code>            | Función de Legendre de primera especie                |
| <code>%q[u,v] (z)</code>            | Función de Legendre de segunda especie                |
| <code>hstruve[n] (z)</code>         | Función H de Struve                                   |
| <code>lstruve[n] (z)</code>         | Función L de Struve                                   |
| <code>%f[p,q] ([], [], expr)</code> | Función hipergeométrica generalizada                  |
| <code>gamma()</code>                | Función Gamma                                         |
| <code>gammagreek(a,z)</code>        | Función Gamma incompleta                              |
| <code>gammaincomplete(a,z)</code>   | Extremo de la función Gamma incompleta                |
| <code>sloimmel</code>               |                                                       |
| <code>%m[u,k] (z)</code>            | Función de Whittaker de primera especie               |
| <code>%w[u,k] (z)</code>            | Función de Whittaker de segunda especie               |
| <code>erfc (z)</code>               | Complemento de la función de error, erf               |
| <code>ei (z)</code>                 | Integral exponencial (?)                              |
| <code>kelliptic (z)</code>          | Integral elíptica completa de primera especie (K)     |
| <code>%d [n] (z)</code>             | Función cilíndrica parabólica                         |

Con la instrucción `demo ("hypgeo")` se podrán ver algunos ejemplos de transformadas de Laplace calculadas por `specint`.

Este paquete aún está en desarrollo, por lo que algunos nombres de funciones pueden sufrir cambios.

## 16.3 Definiciones para las Funciones Especiales

### **airy** (*x*)

Función

Función Ai de Airy. Si el argumento *x* es un número, se obtiene el valor numérico de **airy** (*x*). En caso contrario, se devuelve la expresión no evaluada **airy** (*x*).

La ecuación de Airy  $\text{diff}(y(x), x, 2) - x y(x) = 0$  tiene dos soluciones linealmente independientes, llamadas **ai** y **bi**. Esta ecuación es muy utilizada para obtener aproximaciones a problemas complejos en muchos ámbitos de la física matemática.

La instrucción `load("airy")` carga las funciones **ai**, **bi**, **dai** y **dbi**.

El paquete **airy** contiene algoritmos para calcular **ai** y **bi**, así como sus derivadas **dai** y **dbi**. El resultado es un número decimal en coma flotante si el argumento es a su vez un número, en caso contrario será una expresión no evaluada.

Se producirá un error si el argumento es lo suficientemente grande como para que los exponenciales causen un desbordamiento (*overflow*), o una pérdida de precisión en el **sin** o en el **cos**. Con esto, el rango de validez es aproximadamente entre -2800 y  $10^{38}$  para **ai** y **dai**, y entre -2800 y 25 para **bi** y **dbi**.

Maxima reconoce las siguientes derivadas:

- `diff(ai(x), x)` devuelve **dai**(*x*),
- `diff(dai(x), x)` devuelve *x* **ai**(*x*),
- `diff(bi(x), x)` devuelve **dbi**(*x*),
- `diff(dbi(x), x)` devuelve *x* **bi**(*x*).

Los valores de las funciones se calculan a partir del desarrollo convergente de Taylor para  $\text{abs}(x) < 3$ , y de las expansiones asintóticas para  $x < -3$  o  $x > 3$ , según sea necesario. Con esto se consiguen discrepancias numéricas despreciables en  $x = 3$  y  $x = -3$ . Para detalles, véase Abramowitz y Stegun, *Handbook of Mathematical Functions*, Sección 10.4 y Tabla 10.11.

La llamada `ev(taylor(ai(x), x, 0, 9), infeval)` devuelve el desarrollo de Taylor con números decimales de la función **ai**. Se puede escribir una expresión similar para **bi**.

### **airy\_ai** (*x*)

Función

Función Ai de Airy, tal como la definen Abramowitz y Stegun, *Handbook of Mathematical Functions*, Sección 10.4.

La ecuación de Airy  $\text{diff}(y(x), x, 2) - x y(x) = 0$  tiene dos soluciones linealmente independientes,  $y = \text{Ai}(x)$  y  $y = \text{Bi}(x)$ . La derivada `diff(airy_ai(x), x)` es **airy\_dai**(*x*).

Si el argumento *x* es un número decimal real o complejo, se devolverá el valor numérico de **airy\_ai** siempre que sea posible.

Véanse **airy\_bi**, **airy\_dai** y **airy\_dbi**.

**airy\_dai** (*x*) Función  
 Es la derivada de la función Ai de Airy, `airy_ai(x)`.  
 Véase `airy_ai`.

**airy\_bi** (*x*) Función  
 Es la función Bi de Airy, tal como la definen Abramowitz y Stegun, *Handbook of Mathematical Functions*, Sección 10.4. Se trata de la segunda solución de la ecuación de Airy `diff (y(x), x, 2) - x y(x) = 0`.  
 Si el argumento *x* es un número decimal real o complejo, se devolverá el valor numérico de `airy_bi` siempre que sea posible. En los otros casos, se devuelve la expresión sin evaluar.  
 La derivada `diff (airy_bi(x), x)` es `airy_dbi(x)`.  
 Véanse `airy_ai` y `airy_dbi`.

**airy\_dbi** (*x*) Función  
 Es la derivada de la función Bi de Airy, `airy_bi(x)`.  
 Véanse `airy_ai` y `airy_bi`.

**asympa** Función  
 El paquete `asympa` contiene funciones de simplificación para realizar análisis asintótico, incluyendo las funciones “big O” y “little o”, de uso frecuente en el análisis complejo y numérico.  
 La instrucción `load ("asympa")` carga este paquete.

**bessel** (*z*, *a*) Función  
 Función de Bessel de primera especie.  
 Ya no se recomienda su uso. Utilícese `bessel_j (z, a)` en su lugar.

**bessel\_j** (*v*, *z*) Función  
 Función de Bessel de primera especie de orden *v* y argumento *z*.  
 La función `bessel_j` calcula el arreglo `besselarray` tal que `besselarray [i] = bessel_j [i + v - int(v)] (z)` para *i* desde cero hasta `int(v)`.  
 La función `bessel_j` se define como

$$\sum_{k=0}^{\infty} \frac{(-1)^k \left(\frac{z}{2}\right)^{v+2k}}{k! \Gamma(v+k+1)}$$

aunque la serie infinita no se utiliza en los cálculos.

**bessel\_y** (*v*, *z*) Función  
 Función de Bessel de segunda especie de orden *v* y argumento *z*.  
 La función `bessel_y` calcula el arreglo `besselarray` tal que `besselarray [i] = bessel_y [i + v - int(v)] (z)` para *i* desde cero hasta `int(v)`.  
 La función `bessel_y` se define como



$$\frac{\cos(\pi v) J_v(z) - J_{-v}(z)}{\sin(\pi v)}$$

si  $v$  no es un entero. En caso de que  $v$  sea un entero  $n$ , se calcula el límite cuando  $v$  se aproxima a  $n$ .

### **bessel\_i** ( $v, z$ )

Función

Función modificada de Bessel de primera especie de orden  $v$  y argumento  $z$ .

La función **bessel\_i** calcula el arreglo **besselarray** tal que **besselarray** [ $i$ ] = **bessel\_i** [ $i + v - \text{int}(v)$ ] ( $z$ ) para  $i$  desde cero hasta  $\text{int}(v)$ .

La función **bessel\_i** se define como

$$\sum_{k=0}^{\infty} \frac{1}{k! \Gamma(v + k + 1)} \left(\frac{z}{2}\right)^{v+2k}$$

aunque la serie infinita no se utiliza en los cálculos.

### **bessel\_k** ( $v, z$ )

Función

Función modificada de Bessel de segunda especie de orden  $v$  y argumento  $z$ .

La función **bessel\_k** calcula el arreglo **besselarray** tal que **besselarray** [ $i$ ] = **bessel\_k** [ $i + v - \text{int}(v)$ ] ( $z$ ) para  $i$  desde cero hasta  $\text{int}(v)$ .

La función **bessel\_k** se define como

$$\frac{\pi \csc(\pi v) (I_{-v}(z) - I_v(z))}{2}$$

si  $v$  no es un entero. Si  $v$  es igual al entero  $n$ , entonces se calcula el límite cuando  $v$  tiende a  $n$ .

### **besselexpand**

Variable optativa

Valor por defecto: **false**

Controla la expansión de las funciones de Bessel cuando el orden es la mitad de un entero impar. En tal caso, las funciones de Bessel se pueden expandir en términos de otras funciones elementales. Si **besselexpand** vale **true**, se expande la función de Bessel.

```
(%i1) besselexpand: false$
(%i2) bessel_j (3/2, z);

(%o2) 3
 bessel_j(-, z)
 2

(%i3) besselexpand: true$
(%i4) bessel_j (3/2, z);

(%o4) 2 z sin(z) cos(z)
 sqrt(----) (----- - ----)
 %pi 2 z
 z
```

|                                                                                                                                                                                                                                                                                                                                                                                    |                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| <b>j0</b> ( <i>x</i> )                                                                                                                                                                                                                                                                                                                                                             | Función           |
| Función de Bessel de primera especie de orden 0.                                                                                                                                                                                                                                                                                                                                   |                   |
| Ya no se recomienda su uso. Utilícese <code>bessel_j</code> (0, <i>x</i> ) en su lugar.                                                                                                                                                                                                                                                                                            |                   |
| <b>j1</b> ( <i>x</i> )                                                                                                                                                                                                                                                                                                                                                             | Función           |
| Función de Bessel de primera especie de orden 1.                                                                                                                                                                                                                                                                                                                                   |                   |
| Ya no se recomienda su uso. Utilícese <code>bessel_j</code> (1, <i>x</i> ) en su lugar.                                                                                                                                                                                                                                                                                            |                   |
| <b>jn</b> ( <i>x</i> , <i>n</i> )                                                                                                                                                                                                                                                                                                                                                  | Función           |
| Función de Bessel de primera especie de orden <i>n</i> .                                                                                                                                                                                                                                                                                                                           |                   |
| Ya no se recomienda su uso. Utilícese <code>bessel_j</code> ( <i>n</i> , <i>x</i> ) en su lugar.                                                                                                                                                                                                                                                                                   |                   |
| <b>i0</b> ( <i>x</i> )                                                                                                                                                                                                                                                                                                                                                             | Función           |
| Función modificada de Bessel de primera especie de orden 0.                                                                                                                                                                                                                                                                                                                        |                   |
| Ya no se recomienda su uso. Utilícese <code>bessel_i</code> (0, <i>x</i> ) en su lugar.                                                                                                                                                                                                                                                                                            |                   |
| <b>i1</b> ( <i>x</i> )                                                                                                                                                                                                                                                                                                                                                             | Función           |
| Función modificada de Bessel de primera especie de orden 1.                                                                                                                                                                                                                                                                                                                        |                   |
| Ya no se recomienda su uso. Utilícese <code>bessel_i</code> (1, <i>x</i> ) en su lugar.                                                                                                                                                                                                                                                                                            |                   |
| <b>beta</b> ( <i>x</i> , <i>y</i> )                                                                                                                                                                                                                                                                                                                                                | Función           |
| Función beta, definida como $\text{gamma}(x) \text{gamma}(y) / \text{gamma}(x + y)$ .                                                                                                                                                                                                                                                                                              |                   |
| <b>gamma</b> ( <i>x</i> )                                                                                                                                                                                                                                                                                                                                                          | Función           |
| Función gamma.                                                                                                                                                                                                                                                                                                                                                                     |                   |
| Véase también <code>makegamma</code> .                                                                                                                                                                                                                                                                                                                                             |                   |
| La variable <code>gammalim</code> controla la simplificación de la función gamma.                                                                                                                                                                                                                                                                                                  |                   |
| La constante de Euler-Mascheroni es <code>%gamma</code> .                                                                                                                                                                                                                                                                                                                          |                   |
| <b>gammalim</b>                                                                                                                                                                                                                                                                                                                                                                    | Variable optativa |
| Valor por defecto: 1000000                                                                                                                                                                                                                                                                                                                                                         |                   |
| La variable <code>gammalim</code> controla la simplificación de la función gamma con argumentos enteros o racionales. Si el valor absoluto del argumento no es mayor que <code>gammalim</code> , entonces se realizará la simplificación. Nótese que la variable <code>factlim</code> también controla la simplificación del resultado de <code>gamma</code> con argumento entero. |                   |
| <b>intopois</b> ( <i>a</i> )                                                                                                                                                                                                                                                                                                                                                       | Función           |
| Convierte <i>a</i> en un codificado Poisson.                                                                                                                                                                                                                                                                                                                                       |                   |
| <b>makefact</b> ( <i>expr</i> )                                                                                                                                                                                                                                                                                                                                                    | Función           |
| Transforma las funciones <code>binomial</code> , <code>gamma</code> y <code>beta</code> que aparecen en <i>expr</i> en su notación factorial.                                                                                                                                                                                                                                      |                   |
| Véase también <code>makegamma</code> .                                                                                                                                                                                                                                                                                                                                             |                   |

**makegamma** (*expr*) Función  
 Transforma las funciones `binomial`, `factorial` y `beta` que aparecen en *expr* en funciones `gamma`.

Véase también `makefact`.

**numfactor** (*expr*) Función  
 Devuelve el factor numérico que multiplica a la expresión *expr*, la cual debe tener un único término.

```
(%i1) gamma (7/2);
(%o1) 15 sqrt(%pi)

 8
(%i2) numfactor (%);
(%o2) 15
 --
 8
```

**outofpois** (*a*) Función  
 Convierte *a* desde codificado de Poisson a una representación general. Si *a* no está en forma de Poisson, `outofpois` hace la conversión, siendo entonces el valor retornado `outofpois (intopois (a))`. Esta función es un simplificador canónico para sumas de potencias de senos y cosenos.

**poisdiff** (*a*, *b*) Función  
 Deriva *a* con respecto a *b*. El argumento *b* debe aparecer sólo en los argumentos trigonométricos o sólo en los coeficientes.

**poisexpt** (*a*, *b*) Función  
 Idéntico a `intopois (a^b)`. El argumento *b* debe ser un entero positivo.

**poislim** Variable optativa  
 Valor por defecto: 5  
 La variable `poislim` determina el dominio de los coeficientes en los argumentos de las funciones trigonométricas. El valor por defecto 5 corresponde al intervalo  $[-2^{(5-1)+1}, 2^{(5-1)}]$ , o  $[-15, 16]$ , pero puede reasignarse para  $[-2^{(n-1)+1}, 2^{(n-1)}]$ .

**poisplus** (*a*, *b*) Función  
 Idéntico a `intopois (a + b)`.

**poissimp** (*a*) Función  
 Convierte *a* en una serie de Poisson para *a* en su representación general.

**poisson** Símbolo especial  
 El símbolo `/P/` sigue a la etiqueta de las líneas que contienen expresiones que son series de Poisson.

- poissubst** (*a*, *b*, *c*) Función  
 Sustituye *b* por *a* en *c*, donde *c* es una serie de Poisson.
- (1) Si *b* es una de las variables *u*, *v*, *w*, *x*, *y* o *z*, entonces *a* debe ser una expresión lineal en esas variables (por ejemplo, `6*u + 4*v`).
- (2) Si *b* no es ninguna de esas variables, entonces *a* no puede contener tampoco a ninguna de ellas, ni senos, ni cosenos.
- poistimes** (*a*, *b*) Función  
 Idéntico a `intopois (a*b)`.
- printpois** (*a*) Función  
 Presenta una serie de Poisson en un formato legible. Conjuntamente con `outofpois`, si es necesario convertirá a primero en una codificación de Poisson.
- psi** (*x*) Función  
**psi** [*n*](*x*) Función  
 Derivada de `log (gamma (x))`.  
 Maxima no calcula el valor numérico de **psi**. Sin embargo, la función **bfpsi** del paquete **bffac** puede calcular valores numéricos.



## 17 Funciones elípticas

### 17.1 Introducción a las funciones e integrales elípticas

Maxima da soporte para las funciones elípticas jacobianas y para las integrales elípticas completas e incompletas. Esto incluye la manipulación simbólica de estas funciones y su evaluación numérica. Las definiciones de estas funciones y de muchas de sus propiedades se pueden encontrar en Abramowitz y Stegun, capítulos 16–17, que es la fuente principal utilizada para su programación en Maxima, aunque existen algunas diferencias.

En particular, todas las funciones e integrales elípticas utilizan el parámetro  $m$  en lugar del módulo  $k$  o del ángulo  $\alpha$ . Esta es una de las diferencias con Abramowitz y Stegun, que utilizan el ángulo para las funciones elípticas. Las siguientes relaciones son válidas:

$$m = k^2$$

y

$$k = \sin \alpha.$$

Las funciones e integrales elípticas en Maxima tienen como objetivo primordial dar soporte al cálculo simbólico, de ahí que también estén incluidas la mayoría de las derivadas e integrales asociadas a estas funciones. No obstante lo anterior, si los argumentos dados a las funciones son decimales en coma flotante, los resultados también serán decimales.

Sin embargo, la mayoría de las propiedades no relacionadas con las derivadas de las funciones e integrales elípticas todavía no han sido programadas en Maxima.

Algunos ejemplos de funciones elípticas:

```
(%i1) jacobi_sn (u, m);
(%o1) jacobi_sn(u, m)
(%i2) jacobi_sn (u, 1);
(%o2) tanh(u)
(%i3) jacobi_sn (u, 0);
(%o3) sin(u)
(%i4) diff (jacobi_sn (u, m), u);
(%o4) jacobi_cn(u, m) jacobi_dn(u, m)
(%i5) diff (jacobi_sn (u, m), m);
(%o5) jacobi_cn(u, m) jacobi_dn(u, m)

 elliptic_e(asin(jacobi_sn(u, m)), m)
(u - -----)/(2 m)
 1 - m

 2
 jacobi_cn (u, m) jacobi_sn(u, m)
+ -----
 2 (1 - m)
```

Algunos ejemplos de integrales elípticas:

```

(%i1) elliptic_f (phi, m);
(%o1) elliptic_f(phi, m)
(%i2) elliptic_f (phi, 0);
(%o2) phi
(%i3) elliptic_f (phi, 1);
(%o3) log(tan(--- + ---))
 2 4
(%i4) elliptic_e (phi, 1);
(%o4) sin(phi)
(%i5) elliptic_e (phi, 0);
(%o5) phi
(%i6) elliptic_kc (1/2);
(%o6) elliptic_kc(---)
 2
(%i7) makegamma (%);
(%o7) gamma(---)
 4
(%i8) diff (elliptic_f (phi, m), phi);
(%o8) -----
 2
 sqrt(1 - m sin (phi))
(%i9) diff (elliptic_f (phi, m), m);
(%o9) (-----)
 m
 cos(phi) sin(phi)
 - -----)/(2 (1 - m))
 2
 sqrt(1 - m sin (phi))

```

El paquete para funciones e integrales elípticas fue programado por Raymond Toy. Se distribuye, igual que Maxima, bajo la General Public License (GPL).

## 17.2 Definiciones para funciones elípticas

|                                         |         |
|-----------------------------------------|---------|
| <b>jacobi_sn</b> ( $u, m$ )             | Función |
| Función elíptica jacobiana $sn(u, m)$ . |         |
| <b>jacobi_cn</b> ( $u, m$ )             | Función |
| Función elíptica jacobiana $cn(u, m)$ . |         |
| <b>jacobi_dn</b> ( $u, m$ )             | Función |
| Función elíptica jacobiana $dn(u, m)$ . |         |

|                                                             |         |
|-------------------------------------------------------------|---------|
| <b>jacobi_ns</b> ( $u, m$ )                                 | Función |
| Función elíptica jacobiana $ns(u, m) = 1/sn(u, m)$ .        |         |
| <b>jacobi_sc</b> ( $u, m$ )                                 | Función |
| Función elíptica jacobiana $sc(u, m) = sn(u, m)/cn(u, m)$ . |         |
| <b>jacobi_sd</b> ( $u, m$ )                                 | Función |
| Función elíptica jacobiana $sd(u, m) = sn(u, m)/dn(u, m)$ . |         |
| <b>jacobi_nc</b> ( $u, m$ )                                 | Función |
| Función elíptica jacobiana $nc(u, m) = 1/cn(u, m)$ .        |         |
| <b>jacobi_cs</b> ( $u, m$ )                                 | Función |
| Función elíptica jacobiana $cs(u, m) = cn(u, m)/sn(u, m)$ . |         |
| <b>jacobi_cd</b> ( $u, m$ )                                 | Función |
| Función elíptica jacobiana $cd(u, m) = cn(u, m)/dn(u, m)$ . |         |
| <b>jacobi_nd</b> ( $u, m$ )                                 | Función |
| Función elíptica jacobiana $nc(u, m) = 1/cn(u, m)$ .        |         |
| <b>jacobi_ds</b> ( $u, m$ )                                 | Función |
| Función elíptica jacobiana $ds(u, m) = dn(u, m)/sn(u, m)$ . |         |
| <b>jacobi_dc</b> ( $u, m$ )                                 | Función |
| Función elíptica jacobiana $dc(u, m) = dn(u, m)/cn(u, m)$ . |         |
| <b>inverse_jacobi_sn</b> ( $u, m$ )                         | Función |
| Inversa de la función elíptica jacobiana $sn(u, m)$ .       |         |
| <b>inverse_jacobi_cn</b> ( $u, m$ )                         | Función |
| Inversa de la función elíptica jacobiana $cn(u, m)$ .       |         |
| <b>inverse_jacobi_dn</b> ( $u, m$ )                         | Función |
| Inversa de la función elíptica jacobiana $dn(u, m)$ .       |         |
| <b>inverse_jacobi_ns</b> ( $u, m$ )                         | Función |
| Inversa de la función elíptica jacobiana $ns(u, m)$ .       |         |
| <b>inverse_jacobi_sc</b> ( $u, m$ )                         | Función |
| Inversa de la función elíptica jacobiana $sc(u, m)$ .       |         |
| <b>inverse_jacobi_sd</b> ( $u, m$ )                         | Función |
| Inversa de la función elíptica jacobiana $sd(u, m)$ .       |         |



|                                                       |         |
|-------------------------------------------------------|---------|
| <b>inverse_jacobi_nc</b> ( <i>u</i> , <i>m</i> )      | Función |
| Inversa de la función elíptica jacobiana $nc(u, m)$ . |         |
| <b>inverse_jacobi_cs</b> ( <i>u</i> , <i>m</i> )      | Función |
| Inversa de la función elíptica jacobiana $cs(u, m)$ . |         |
| <b>inverse_jacobi_cd</b> ( <i>u</i> , <i>m</i> )      | Función |
| Inversa de la función elíptica jacobiana $cd(u, m)$ . |         |
| <b>inverse_jacobi_nd</b> ( <i>u</i> , <i>m</i> )      | Función |
| Inversa de la función elíptica jacobiana $nc(u, m)$ . |         |
| <b>inverse_jacobi_ds</b> ( <i>u</i> , <i>m</i> )      | Función |
| Inversa de la función elíptica jacobiana $ds(u, m)$ . |         |
| <b>inverse_jacobi_dc</b> ( <i>u</i> , <i>m</i> )      | Función |
| Inversa de la función elíptica jacobiana $dc(u, m)$ . |         |

### 17.3 Definiciones para integrales elípticas

|                                                                |         |
|----------------------------------------------------------------|---------|
| <b>elliptic_f</b> ( <i>phi</i> , <i>m</i> )                    | Función |
| Integral elíptica incompleta de primera especie, definida como |         |

$$\int_0^\phi \frac{d\theta}{\sqrt{1 - m \sin^2 \theta}}$$

Véanse también **elliptic\_e** y **elliptic\_kc**.

|                                                                |         |
|----------------------------------------------------------------|---------|
| <b>elliptic_e</b> ( <i>phi</i> , <i>m</i> )                    | Función |
| Integral elíptica incompleta de segunda especie, definida como |         |

$$\int_0^\phi \sqrt{1 - m \sin^2 \theta} d\theta$$

Véanse también **elliptic\_e** y **elliptic\_ec**.

|                                                                |         |
|----------------------------------------------------------------|---------|
| <b>elliptic_eu</b> ( <i>u</i> , <i>m</i> )                     | Función |
| Integral elíptica incompleta de segunda especie, definida como |         |

$$\int_0^u \operatorname{dn}(v, m)^2 dv = \int_0^\tau \sqrt{\frac{1 - mt^2}{1 - t^2}} dt$$

donde  $\tau = \operatorname{sn}(u, m)$

Esto se relaciona con **elliptic\_e** mediante

$$EU(u, m) = E(\phi, m)$$

donde  $\phi = \sin^{-1} \operatorname{sn}(u, m)$

Véase también **elliptic\_e**.

**elliptic\_pi** (*n*, *phi*, *m*)

Función

Integral elíptica incompleta de tercera especie, definida como

$$\int_0^\phi \frac{d\theta}{(1 - n \sin^2 \theta) \sqrt{1 - m \sin^2 \theta}}$$

Maxima sólo conoce la derivada respecto de *phi*.**elliptic\_kc** (*m*)

Función

Integral elíptica completa de primera especie, definida como

$$\int_0^{\frac{\pi}{2}} \frac{d\theta}{\sqrt{1 - m \sin^2 \theta}}$$

Para algunos valores de *m*, se conoce el valor de la integral en términos de la función *Gamma*. Hágase uso de **makegamma** para realizar su cálculo.

**elliptic\_ec** (*m*)

Función

Integral elíptica completa de segunda especie, definida como

$$\int_0^{\frac{\pi}{2}} \sqrt{1 - m \sin^2 \theta} d\theta$$

Para algunos valores de *m*, se conoce el valor de la integral en términos de la función *Gamma*. Hágase uso de **makegamma** para realizar su cálculo.



## 18 Límites

### 18.1 Definiciones para Límites

#### lhospitallim

Variable optativa

Valor por defecto: 4

Es el número máximo de veces que la regla de L'Hopital es aplicada en la función `limit`, evitando bucles infinitos al iterar la regla en casos como `limit (cot(x)/csc(x), x, 0)`.

**limit** (*expr*, *x*, *val*, *dir*)

Función

**limit** (*expr*, *x*, *val*)

Función

**limit** (*expr*)

Función

Calcula el ímite de *expr* cuando la variable real *x* se aproxima al valor *val* desde la dirección *dir*. El argumento *dir* puede ser el valor `plus` para un límite por la derecha, `minus` para un límite por la izquierda o simplemente se omite para indicar un límite en ambos sentidos.

La función `limit` utiliza los símbolos especiales siguientes: `inf` (más infinito) y `minf` (menos infinito). En el resultado también puede hacer uso de `und` (indefinido), `ind` (indefinido pero acotado) y `infinity` (infinito complejo).

La variable `lhospitallim` guarda el número máximo de veces que la regla de L'Hopital es aplicada en la función `limit`, evitando bucles infinitos al iterar la regla en casos como `limit (cot(x)/csc(x), x, 0)`.

Si la variable `tlimswitch` vale `true` hará que el paquete sobre límites utilice desarrollos de Taylor siempre que sea posible.

La variable `limsubst` evita que la función `limit` realice sustituciones sobre formas desconocidas, a fin de evitar fallos tales como que `limit (f(n)/f(n+1), n, inf)` devuelva 1. Dándole a `limsubst` el valor `true` se permitirán tales sustituciones.

La función `limit` con un solo argumento se utiliza frecuentemente para simplificar expresiones constantes, como por ejemplo `limit (inf-1)`.

La instrucción `example (limit)` muestra algunos ejemplos.

Para información sobre el método utilizado véase Wang, P., "Evaluation of Definite Integrals by Symbolic Manipulation", Ph.D. thesis, MAC TR-92, October 1971.

#### limsubst

Variable optativa

Valor por defecto: `false`

La variable `limsubst` evita que la función `limit` realice sustituciones sobre formas desconocidas, a fin de evitar fallos tales como que `limit (f(n)/f(n+1), n, inf)` devuelva 1. Dándole a `limsubst` el valor `true` se permitirán tales sustituciones.

**tlimit** (*expr*, *x*, *val*, *dir*)

Función

**tlimit** (*expr*, *x*, *val*)

Función

**tlimit** (*expr*)

Función

Devuelve el límite solicitado dándole a `tlimswitch` el valor `true`.

**tlimswitch**

Variable optativa

Valor por defecto: `false`

Si la variable `tlimswitch` vale `true` hará que el paquete sobre límites utilice desarrollos de Taylor siempre que sea posible.

## 19 Diferenciación

### 19.1 Definiciones para Diferenciación

**antid** (*expr*, *x*, *u(x)*)

Función

Devuelve una lista con dos elementos, de manera que se pueda calcular la antiderivada de *expr* respecto de *x* a partir de la lista. La expresión *expr* puede contener una función no especificada *u* y sus derivadas.

Sea *L* la lista con dos elementos que devuelve la función **antid**. Entonces, *L*[1] + 'integrate (*L*[2], *x*) es una antiderivada de *expr* con respecto a *x*.

Si la ejecución de **antid** resulta exitosa, el segundo elemento de la lista retornada es cero. En caso contrario, el segundo elemento es distinto de cero y el primero puede ser nulo o no. Si **antid** no es capaz de hacer ningún progreso, el primer elemento es nulo y el segundo no nulo.

Es necesario ejecutar **load** ("antid") para cargar esta función. El paquete **antid** define también las funciones **nonzeroandfreeof** y **linear**.

La función **antid** está relacionada con **antidiff** como se indica a continuación. Sea *L* la lista devuelta por la función **antid**. Entonces, el resultado de **antidiff** es igual a *L*[1] + 'integrate (*L*[2], *x*), donde *x* es la variable de integración.

Ejemplos:

```
(%i1) load ("antid")$
(%i2) expr: exp (z(x)) * diff (z(x), x) * y(x);
 z(x) d
(%o2) y(x) %e (--- (z(x)))
 dx
(%i3) a1: antid (expr, x, z(x));
 z(x) z(x) d
(%o3) [y(x) %e , - %e (--- (y(x)))]
 dx
(%i4) a2: antidiff (expr, x, z(x));
 /
 z(x) [z(x) d
(%o4) y(x) %e - I %e (--- (y(x))) dx
] dx
 /
(%i5) a2 - (first (a1) + 'integrate (second (a1), x));
(%o5) 0
(%i6) antid (expr, x, y(x));
 z(x) d
(%o6) [0, y(x) %e (--- (z(x)))]
 dx
(%i7) antidiff (expr, x, y(x));
 /
 [z(x) d
(%o7) I y(x) %e (--- (z(x))) dx
] dx
 /
```

**antidiff** (*expr*, *x*, *u(x)*)

Función

Devuelve la antiderivada de *expr* respecto de *x*. La expresión *expr* puede contener una función no especificada *u* y sus derivadas.

Cuando **antidiff** se ejecuta con éxito, la expresión resultante no tiene símbolos integrales (esto es, no tiene referencias a la función **integrate**). En otro caso, **antidiff** devuelve una expresión que se encuentra total o parcialmente bajo el signo de integración. Si **antidiff** no puede realizar ningún progreso, el valor devuelto se encuentra completamente bajo la integral.

Es necesario ejecutar **load ("antid")** para cargar esta función. El paquete **antid** define también las funciones **nonzeroandfreeof** y **linear**.

La función **antidiff** está relacionada con **antid** como se indica a continuación. Sea *L* la lista de dos elementos que devuelve **antid**. Entonces, el valor retornado por **antidiff** es igual a *L*[1] + 'integrate (*L*[2], *x*), donde *x* es la variable de integración.

Ejemplos:

```
(%i1) load ("antid")$
(%i2) expr: exp (z(x)) * diff (z(x), x) * y(x);
(%o2) y(x) %ez(x) d
 dx
 (--- (z(x)))
(%i3) a1: antid (expr, x, z(x));
(%o3) [y(x) %ez(x), - %ez(x) d
 dx
 (--- (y(x)))]
(%i4) a2: antidiff (expr, x, z(x));
(%o4) y(x) %ez(x) - I %ez(x) d
 dx
 (--- (y(x))) dx
 /
(%i5) a2 - (first (a1) + 'integrate (second (a1), x));
(%o5) 0
(%i6) antid (expr, x, y(x));
(%o6) [0, y(x) %ez(x) d
 dx
 (--- (z(x)))]
(%i7) antidiff (expr, x, y(x));
(%o7) I y(x) %ez(x) d
 dx
 (--- (z(x))) dx
 /
```

**atomgrad**

Propiedad

La propiedad **atomgrad** es asignada por **gradef**.

**atvalue** (expr, [x<sub>1</sub> = a<sub>1</sub>, ..., x<sub>m</sub> = a<sub>m</sub>], c)

Función

**atvalue** (expr, x<sub>1</sub> = a<sub>1</sub>, c)

Función

Asigna el valor  $c$  a  $\text{expr}$  en el punto  $x = a$ .

La expresión  $\text{expr}$  es una función del tipo  $f(x_1, \dots, x_m)$ , o una derivada,  $\text{diff}(f(x_1, \dots, x_m), x_1, n_1, \dots, x_n, n_m)$  en la que aparecen los argumentos de la función de forma explícita. Los símbolos  $n_i$  se refieren al orden de diferenciación respecto de  $x_i$ .

El punto en el que **atvalue** establece el valor se especifica mediante la lista de ecuaciones  $[x_1 = a_1, \dots, x_m = a_m]$ . Si hay una única variable  $x_1$ , la ecuación puede escribirse sin formar parte de una lista.

La llamada **printprops** ([f<sub>1</sub>, f<sub>2</sub>, ...], **atvalue**) muestra los valores asignados por **atvalue** a las funciones  $f_1, f_2, \dots$ . La llamada **printprops** (f, **atvalue**) muestra los valores asignados por **atvalue** a la función  $f$ . La llamada **printprops** (all, **atvalue**) muestra los valores asignados por **atvalue** a todas las funciones.

Los símbolos @1, @2, ... representan las variables  $x_1, x_2, \dots$  cuando se muestran los valores asignados por **atvalue**.

La función **atvalue** evalúa sus argumentos y devuelve  $c$ , el valor asignado.

Ejemplos:

```
(%i1) atvalue (f(x,y), [x = 0, y = 1], a^2);
 2
(%o1) a
(%i2) atvalue ('diff (f(x,y), x), x = 0, 1 + y);
(%o2) @2 + 1
(%i3) printprops (all, atvalue);
 !
 d
 --- (f(@1, @2))! = @2 + 1
 d@1
 !@1 = 0

 2
 f(0, 1) = a

(%o3) done
(%i4) diff (4*f(x,y)^2 - u(x,y)^2, x);
 d
 d
(%o4) 8 f(x, y) (--- (f(x, y))) - 2 u(x, y) (--- (u(x, y)))
 dx
 dx
(%i5) at (% , [x = 0, y = 1]);
 !
 2
 d
 --- (u(x, y))!
 dx
(%o5) 16 a - 2 u(0, 1) (--- (u(x, y)))
 dx
 !
 !x = 0, y = 1
```



**cartan -**

Función

El cálculo exterior de formas diferenciales es una herramienta básica de la geometría diferencial desarrollada por Elie Cartan, teniendo importantes aplicaciones en la teoría de ecuaciones diferenciales en derivadas parciales. El paquete **cartan** dispone de las funciones **ext\_diff** y **lie\_diff**, así como de los operadores  $\sim$  (producto exterior) y  $|$  (contracción de una forma con un vector). La orden **demo (tensor)** permite ver una breve descripción de estas instrucciones, junto con ejemplos.

El paquete **cartan** fue escrito por F.B. Estabrook y H.D. Wahlquist.

**del (x)**

Función

La expresión **del (x)** representa el diferencial de la variable  $x$ .

La función **diff** devuelve una expresión que contiene a **del** si no se ha especificado una variable independiente. En este caso, el valor retornado es el llamado "diferencial total".

Ejemplos:

```
(%i1) diff (log (x));
(%o1)

$$\frac{\text{del}(x)}{x}$$

(%i2) diff (exp (x*y));
(%o2)

$$x \, e^{xy} \text{del}(y) + y \, e^{xy} \text{del}(x)$$

(%i3) diff (x*y*z);
(%o3)

$$x \, y \text{del}(z) + x \, z \text{del}(y) + y \, z \text{del}(x)$$

```

**delta (t)**

Función

Es la función delta de Dirac.

En el estado actual de desarrollo de Maxima, sólo **laplace** reconoce la función **delta**.

Ejemplo:

```
(%i1) laplace (delta (t - a) * sin(b*t), t, s);
Is a positive, negative, or zero?

p;
(%o1)

$$\sin(a \, b) \, e^{-a \, s}$$

```

**dependencies**

Variable del sistema

Valor por defecto: []

La variable **dependencies** es la lista de átomos que tienen algún tipo de dependencia funcional, asignada por **depends** o **gradef**. La lista **dependencies** es acumulativa: cada llamada a **depends** o **gradef** añade elementos adicionales.

Véanse **depends** y **gradef**.

**depends (f\_1, x\_1, ..., f\_n, x\_n)**

Función

Declara dependencias funcionales entre variables con el propósito de calcular derivadas. En ausencia de una dependencia declarada, **diff (f, x)** devuelve cero.

Si se declara `depends (f, x)`, `diff (f, x)` devuelve una derivada simbólica (esto es, una expresión con `diff`).

Cada argumento `f.i`, `x.i`, etc., puede ser el nombre de una variable, de un arreglo o una lista de nombres. Cada elemento de `f.i` (quizás un único elemento) se declara como dependiente de cada elemento de `x.i` (quizás también un único elemento). Si alguno de los `f.i` es el nombre de un arreglo o contiene el nombre de un arreglo, todos los elementos del arreglo dependen de `x.i`.

La función `diff` reconoce dependencias indirectas establecidas por `depends` y aplica la regla de la cadena en tales casos.

La instrucción `remove (f, dependency)` borra todas las dependencias declaradas para `f`.

La función `depends` devuelve una lista con las dependencias que han sido establecidas. Las dependencias se añaden a la variable global `dependencies`. La función `depends` evalúa sus argumentos.

La función `diff` es la única instrucción de Maxima que reconoce las dependencias establecidas por `depends`. Otras funciones (`integrate`, `laplace`, etc.) solamente reconocen dependencias explícitamente representadas por sus argumentos. Por ejemplo, `integrate` no reconoce la dependencia de `f` respecto de `x` a menos que se represente explícitamente como `integrate (f(x), x)`.

```
(%i1) depends ([f, g], x);
(%o1) [f(x), g(x)]
(%i2) depends ([r, s], [u, v, w]);
(%o2) [r(u, v, w), s(u, v, w)]
(%i3) depends (u, t);
(%o3) [u(t)]
(%i4) dependencies;
(%o4) [f(x), g(x), r(u, v, w), s(u, v, w), u(t)]
(%i5) diff (r.s, u);
(%o5)
 dr ds
 -- . s + r . --
 du du

(%i6) diff (r.s, t);
(%o6)
 dr du ds du
 -- -- . s + r . -- --
 du dt du dt

(%i7) remove (r, dependency);
(%o7) done
(%i8) diff (r.s, t);
(%o8)
 ds du
 r . -- --
 du dt
```

## derivabbrev

Variable optativa

Valor por defecto: `false`

Si `derivabbrev` vale `true`, las derivadas simbólicas (esto es, expresiones con `diff`) se muestran como subíndices. En otro caso, las derivadas se muestran en la notación de Leibniz,  $dy/dx$ .

**derivdegree** (*expr*, *y*, *x*)

Función

Devuelve el mayor grado de la derivada de la variable dependiente *y* respecto de la variable independiente *x* que aparece en *expr*.

Ejemplo:

```
(%i1) 'diff (y, x, 2) + 'diff (y, z, 3) + 'diff (y, x) * x^2;
 3 2
 d y d y 2 dy
(%o1) ----- + ----- + x ----
 3 2 dx
 dz dx
(%i2) derivdegree (%o1, y, x);
(%o2) 2
```

**derivlist** (*var\_1*, ..., *var\_k*)

Función

Hace que las derivadas calculadas por la instrucción **ev** se calculen respecto de las variables indicadas.

**derivsubst**

Variable optativa

Valor por defecto: **false**

Si **derivsubst** vale **true**, una sustitución no sintáctica del estilo **subst** (*x*, 'diff (*y*, *t*), 'diff (*y*, *t*, 2)) devuelve 'diff (*x*, *t*).

**diff** (*expr*, *x\_1*, *n\_1*, ..., *x\_m*, *n\_m*)

Función

**diff** (*expr*, *x*, *n*)

Función

**diff** (*expr*, *x*)

Función

**diff** (*expr*)

Función

Devuelve la derivada o diferencial de *expr* respecto de alguna o de todas las variables presentes en *expr*.

La llamada **diff** (*expr*, *x*, *n*) devuelve la *n*-ésima derivada de *expr* respecto de *x*.

La llamada **diff** (*expr*, *x\_1*, *n\_1*, ..., *x\_m*, *n\_m*) devuelve la derivada parcial de *expr* con respecto de *x\_1*, ..., *x\_m*. Equivale a **diff** (... (**diff** (*expr*, *x\_m*, *n\_m*) ...), *x\_1*, *n\_1*).

La llamada **diff** (*expr*, *x*) devuelve la primera derivada de *expr* respecto de la variable *x*.

La llamada **diff** (*expr*) devuelve el diferencial total de *expr*, esto es, la suma de las derivadas de *expr* respecto de cada una de sus variables, multiplicadas por el diferencial **del** de cada una de ellas.

La forma nominal de **diff** es necesaria en algunos contextos, como para definir ecuaciones diferenciales. En tales casos, **diff** puede ir precedida de un apóstrofo (como 'diff) para evitar el cálculo de la derivada.

Si **derivabbrev** vale **true**, las derivadas se muestran como subíndices. En otro caso, se muestran en la notación de Leibniz, *dy/dx*.

Ejemplos:

```

(%i1) diff (exp (f(x)), x, 2);
 2
 f(x) d f(x) d 2
(%o1) %e (--- (f(x))) + %e (--- (f(x)))
 2 dx
 dx
(%i2) derivabbrev: true$
(%i3) 'integrate (f(x, y), y, g(x), h(x));
 h(x)
 /
 [
(%o3) I f(x, y) dy
]
 /
 g(x)

(%i4) diff (% , x);
 h(x)
 /
 [
(%o4) I f(x, y) dy + f(x, h(x)) h(x) - f(x, g(x)) g(x)
] x x x
 /
 g(x)

```

Para el paquete sobre tensores se han introducido las siguientes modificaciones:

- (1) Las derivadas de los objetos indexados en *expr* tendrán las variables *x<sub>i</sub>* añadidas como argumentos adicionales. Entonces se ordenarán todos los índices de derivadas.
- (2) Las *x<sub>i</sub>* pueden ser enteros entre 1 hasta el valor de la variable **dimension** [valor por defecto: 4]. Esto hará que la diferenciación sea llevada a cabo con respecto al *x<sub>i</sub>*-ésimo número de la lista **coordinates**, la cual debería contener una lista con los nombres de las coordenadas, por ejemplo, [x, y, z, t]. Si **coordinates** es una variable atómica, entonces esa variable será utilizada como variable de diferenciación. Se permite la utilización de arreglos con los nombres de las coordenadas o nombres con subíndices, como **X[1]**, **X[2]**, ... to be used. Si a **coordinates** no se le ha asignado ningún valor, entonces las variables serán tratadas como se ha indicado en (1).

## diff

Símbolo especial

Si el nombre **diff** está presente en una llamada a la función **ev** en modo **evflag**, entonces se calculan todas las derivadas presentes en *expr*.

## express (expr)

Función

Transforma los nombres de operadores diferenciales en expresiones en términos de derivadas parciales. La función **express** reconoce los operadores **grad**, **div**, **curl** y **laplacian**, así como el producto cruzado  $\sim$ .

Las derivadas simbólicas (es decir, las que incluyen la forma nominal **diff**) que aparecen en la expresión devuelta por **express** se pueden calcular añadiéndole a **ev** el término **diff**. En este contexto, **diff** se tía como **evfun**.

Es necesario ejecutar **load ("vect")** para cargar esta función.

Ejemplos:

```
(%i1) load ("vect")$
(%i2) grad (x^2 + y^2 + z^2);

(%o2)

$$\text{grad} (z^2 + y^2 + x^2)$$

(%i3) express (%);
(%o3)

$$\left[\frac{d}{dx} (z^2 + y^2 + x^2), \frac{d}{dy} (z^2 + y^2 + x^2), \frac{d}{dz} (z^2 + y^2 + x^2) \right]$$

(%i4) ev (% , diff);
(%o4)

$$[2x, 2y, 2z]$$

(%i5) div ([x^2, y^2, z^2]);

(%o5)

$$\text{div} [x^2, y^2, z^2]$$

(%i6) express (%);
(%o6)

$$\frac{d}{dz} (z^2) + \frac{d}{dy} (y^2) + \frac{d}{dx} (x^2)$$

(%i7) ev (% , diff);
(%o7)

$$2z + 2y + 2x$$

(%i8) curl ([x^2, y^2, z^2]);

(%o8)

$$\text{curl} [x^2, y^2, z^2]$$

(%i9) express (%);
(%o9)

$$\left[\frac{d}{dy} (z^2) - \frac{d}{dz} (y^2), \frac{d}{dz} (x^2) - \frac{d}{dx} (z^2), \frac{d}{dx} (y^2) - \frac{d}{dy} (x^2) \right]$$

(%i10) ev (% , diff);
(%o10)

$$[0, 0, 0]$$

(%i11) laplacian (x^2 * y^2 * z^2);

(%o11)

$$\text{laplacian} (x^2 y^2 z^2)$$

(%i12) express (%);
(%o12)

$$\frac{d^2}{dz^2} (x^2 y^2 z^2) + \frac{d^2}{dy^2} (x^2 y^2 z^2) + \frac{d^2}{dx^2} (x^2 y^2 z^2)$$

(%i13) ev (% , diff);
(%o13)

$$2y^2 z^2 + 2x^2 z^2 + 2x^2 y^2$$

(%i14) [a, b, c] ~ [x, y, z];
(%o14)

$$[a, b, c] \sim [x, y, z]$$

(%i15) express (%);
(%o15)

$$[bz - cy, cx - az, ay - bx]$$

```

**gradef** ( $f(x_1, \dots, x_n), g_1, \dots, g_m$ )

Función

**gradef** ( $a, x, \text{expr}$ )

Función

Define las derivadas parciales, o componentes del gradiente, de la función  $f$  o variable  $a$ .

La llamada **gradef** ( $f(x_1, \dots, x_n), g_1, \dots, g_m$ ) define  $df/dx_i$  como  $g_i$ , donde  $g_i$  es una expresión;  $g_i$  puede ser una llamada a función, pero no el nombre de una función. El número de derivadas parciales  $m$  puede ser menor que el número de argumentos  $n$ , en cuyo caso las derivadas se definen solamente con respecto a  $x_1, \dots, x_m$ .

La llamada **gradef** ( $a, x, \text{expr}$ ) define la derivada de la variable  $a$  respecto de  $x$  en  $\text{expr}$ . Con esto se establece la dependencia de  $a$  respecto de  $x$  a través de **depends** ( $a, x$ ).

El primer argumento  $f(x_1, \dots, x_n)$  o  $a$  no se evalúa, pero sí lo hacen el resto de argumentos  $g_1, \dots, g_m$ . La llamada a **gradef** devuelve la función o variable para la que se define la derivada parcial.

La instrucción **gradef** puede redefinir las derivadas de las funciones propias de Maxima. Por ejemplo, **gradef** ( $\sin(x), \text{sqrt}(1 - \sin(x)^2)$ ) redefine la derivada de  $\sin$ .

La instrucción **gradef** no puede definir derivadas parciales de funciones subindicadas.

La llamada **printprops** ( $[f_1, \dots, f_n], \text{gradef}$ ) muestra las derivadas parciales de las funciones  $f_1, \dots, f_n$ , tal como las definió **gradef**.

La llamada **printprops** ( $[a_n, \dots, a_n], \text{atomgrad}$ ) muestra las derivadas parciales de las variables  $a_n, \dots, a_n$ , tal como las definió **gradef**.

La variable **gradefs** contiene la lista de las funciones para las que se han definido derivadas parciales con la instrucción **gradef**, pero no incluye las variables para las que se han definido las derivadas parciales.

Los gradientes son necesarios cuando una función no se conoce explícitamente pero sí sus primeras derivadas y es necesario calcular las derivadas de orden mayor.

**gradefs**

Variable del sistema

Valor por defecto: []

La variable **gradefs** contiene la lista de las funciones para las que se han definido derivadas parciales con la instrucción **gradef**, pero no incluye las variables para las que se han definido las derivadas parciales.

**laplace** ( $\text{expr}, t, s$ )

Función

Calcula la transformada de Laplace de  $\text{expr}$  con respecto de la variable  $t$  y parámetro de transformación  $s$ . Si **laplace** no puede encontrar la solución, se devuelve la forma nominal 'laplace.

La función **laplace** reconoce en  $\text{expr}$  las funciones **delta**, **exp**, **log**, **sin**, **cos**, **sinh**, **cosh** y **erf**, así como **derivative**, **integrate**, **sum** y **ilt**. Bajo la presencia de otras funciones, **laplace** puede que no sea capaz de calcular la transformada.

La función **laplace** reconoce integrales de convolución de la forma **integrate** ( $f(x) * g(t - x), x, 0, t$ ), no pudiendo reconocer otros tipos de convoluciones.

Las relaciones funcionales se deben representar explícitamente en *expr*; las relaciones implícitas establecidas por **depends** no son reconocidas. Así, si  $f$  depende de  $x$  y  $y$ ,  $f(x, y)$  debe aparecer en *expr*.

Véase también `ilt`, la transformada inversa de Laplace.

Ejemplos:

```
(%i1) laplace (exp (2*t + a) * sin(t) * t, t, s);
```

```
(%o1)
 a
 %e (2 s - 4)

 2 2
 (s - 4 s + 5)
```

```
(%i2) laplace ('diff (f (x), x), x, s);
```

```
(%o2) s laplace(f(x), x, s) - f(0)
```

```
(%i3) diff (diff (delta (t), t), t);
```

```
(%o3)
 2
 d
 --- (delta(t))
 2
 dt
```

```
(%i4) laplace (%, t, s);
```

```
(%o4)
 d ! 2
 -- (delta(t))! + s - delta(0) s
 dt !
 !t = 0
```

## 20 Integración

### 20.1 Introducción a la Integración

Maxima tiene varias rutinas para calcular integrales. La función `integrate` hace uso de la mayor parte de ellas. También está el paquete `antid`, que opera con funciones no especificadas y sus derivadas. Para usos numéricos se dispone de la función `romberg`, un integrador adaptativo que utiliza la regla de cuadratura de Newton-Cotes, llamada `quanc8` y una batería de integradores adaptativos de Quadpack, como `quad_qag`, `quad_qags`, etc. También se trabajan funciones hipergeométricas, véase `specint` para más detalles. En términos generales, Maxima sólo opera con funciones que son integrables en términos de funciones elementales, como las racionales, trigonométricas, logarítmicas, exponenciales, radicales, etc., y unas pocas extensiones de éstas, como la función de error o los dilogaritmos. No opera con integrales en términos de funciones desconocidas, como  $g(x)$  o  $h(x)$ .

### 20.2 Definiciones para Integración

**changevar** (*expr*, *f(x,y)*, *y*, *x*)

Función

Hace el cambio de variable dado por  $f(x,y) = 0$  en todas las integrales que aparecen en *expr* con la integración respecto de *x*. La nueva variable será *y*.

```
(%i1) assume(a > 0)$
(%i2) 'integrate (%e**sqrt(a*y), y, 0, 4);
 4
 /
 [sqrt(a) sqrt(y)
(%o2) I %e
]
 /
 0
(%i3) changevar (%, y-z^2/a, z, y);
 0
 /
 [
 2 I abs(z)
 z %e dz
]
 /
 - 2 sqrt(a)
(%o3) - -----
 a
```

Si una expresión contiene formas nominales, como aquella en la que aparece `'integrate` en el ejemplo, podrá ser evaluada por `ev` si se utiliza el término `nouns`. Por ejemplo, la expresión devuelta por `changevar` se puede evaluar haciendo `ev (%o3, nouns)`.

La función `changevar` también se puede utilizar para cambiar los índices de una suma o producto. Sin embargo, debe tenerse en cuenta que cuando se realiza un cambio en



una suma o producto, el mismo debe expresarse en términos de sumas, como  $i = j + \dots$ , no como una función de mayor grado.

Ejemplo:

```
(%i4) sum (a[i]*x^(i-2), i, 0, inf);
 inf
 ====
 \ i - 2
 > a x
 / i
 ====
 i = 0
(%i5) changevar (%, i-2-n, n, i);
 inf
 ====
 \ n
 > a x
 / n + 2
 ====
 n = - 2
```

### **dblnt** (*f*, *r*, *s*, *a*, *b*)

Función

Es una rutina para integrales dobles escrita en lenguaje Maxima y posteriormente traducida y compilada a código máquina. La instrucción `load (dblnt)` carga esta función. Utiliza el método de Simpson en las dos direcciones *x* e *y* para calcular

$$\int_a^b \int_{r(x)}^{s(x)} f(x,y) \, dy \, dx$$

La función *f* debe ser una función traducida o compilada de dos variables, a la vez que *r* y *s* deben ser cada una de ellas una función traducida o compilada de una variable, mientras que *a* y *b* deben ser números en coma flotante. La rutina tiene dos variables globales que determinan el número de divisiones de los intervalos *x* e *y*: `dblnt_x` y `dblnt_y`, ambos con un valor por defecto de 10, pero que pueden cambiarse de forma independiente a otros valores enteros (hay  $2 \cdot \text{dblnt\_x} + 1$  puntos a calcular en la dirección *x* y  $2 \cdot \text{dblnt\_y} + 1$  en la dirección *y*). La rutina subdivide el eje *X* y luego para cada valor de *X* calcula primero *r(x)* y *s(x)*; entonces se subdivide el eje *Y* entre *r(x)* y *s(x)*, evaluándose la integral a lo largo del eje *Y* aplicando la regla de Simpson; a continuación, se evalúa la integral a lo largo del eje *X* utilizando también la regla de Simpson tomando como valores de función las integrales sobre *Y*. Este procedimiento puede ser numéricamente inestable por múltiples motivos, pero es razonablemente rápido: evítese su uso con funciones con grandes oscilaciones o que tengan singularidades. Las integrales del eje *Y* dependen de la proximidad de los límites *r(x)* y *s(x)*, de manera que si la distancia  $s(x) - r(x)$  varía rápidamente con *X*, puede dar lugar errores importantes debido a truncamientos de diferente amplitud en las integrales de *Y*. Se puede aumentar `dblnt_x` y `dblnt_y` al objeto de mejorar el recubrimiento de la región de integración, pero a costa del tiempo de cómputo. Es

necesario que las funciones  $f$ ,  $r$  y  $s$  estén traducidas o compiladas antes de utilizar `dblint`, lo cual redundará en una mejora del tiempo de ejecución de varios órdenes de magnitud respecto de la ejecución de código interpretado.

**defint** (*expr*, *x*, *a*, *b*)

Función

Intenta calcular una integral definida. La función `defint` es invocada por `integrate` cuando se especifican los límites de integración, por ejemplo `integrate (expr, x, a, b)`. Así, desde el punto de vista del usuario, es suficiente con utilizar `integrate`.

La función `defint` devuelve una expresión simbólica, bien sea el resultado calculado o la forma nominal. Véase `quad_qag` y sus funciones relacionadas para aproximaciones numéricas de integrales definidas.

**erf** (*x*)

Función

Es la función de error, cuya derivada es  $2\exp(-x^2)/\sqrt{\pi}$ .

**erfflag**

Variable optativa

Valor por defecto: `true`

Si `erfflag` vale `false`, la función `risch` no introduce la función `erf` en el resultado si no había ninguna en el integrando.

**ilt** (*expr*, *t*, *s*)

Función

Calcula la transformada inversa de Laplace de *expr* con respecto de *t* y parámetro *s*. El argumento *expr* debe ser una fracción de polinomios cuyo denominador tenga sólo factores lineales y cuadráticos. Utilizando las funciones `laplace` y `ilt`, junto con las funciones `solve` o `linsolve`, el usuario podrá resolver ciertas ecuaciones integrales.

```
(%i1) 'integrate (sinh(a*x)*f(t-x), x, 0, t) + b*f(t) = t**2;
 t
 /
 [
(%o1) I f(t - x) sinh(a x) dx + b f(t) = t
]
 /
 0
(%i2) laplace (% , t, s);
(%o2) b laplace(f(t), t, s) + $\frac{a \operatorname{laplace}(f(t), t, s)}{s^2 - a^2} = \frac{2}{s^3}$
(%i3) linsolve ([%], ['laplace(f(t), t, s)]);
(%o3) [laplace(f(t), t, s) = $\frac{2 s^2 - 2 a^2}{b s^5 + (a - a^2 b) s^3}$]
(%i4) ilt (rhs (first (%)), s, t);
Is a b (a b - 1) positive, negative, or zero?
```

$$\begin{aligned}
 & \text{pos;} \\
 & \frac{\sqrt{a b (a b - 1)} t}{2 \cosh\left(\frac{\sqrt{a b (a b - 1)} t}{b}\right)} + \frac{a t^2}{a b - 1} \\
 & (\%04) - \frac{a^3 b^2 - 2 a^2 b + a}{a^3 b^2 - 2 a^2 b + a} + \frac{a t^2}{a b - 1} \\
 & + \frac{a^3 b^2 - 2 a^2 b + a}{a^3 b^2 - 2 a^2 b + a}
 \end{aligned}$$

**integrate** (*expr*, *x*)

Función

**integrate** (*expr*, *x*, *a*, *b*)

Función

Calcula simbólicamente la integral de *expr* respecto de *x*. La llamada **integrate** (*expr*, *x*) resuelve una integral indefinida, mientras que **integrate** (*expr*, *x*, *a*, *b*) resuelve una integral definida con límites de integración *a* y *b*. Los límites no pueden contener a *x*. El argumento *a* no necesita ser menor que *b*. Si *b* es igual a *a*, **integrate** devuelve cero.

Véase **quad\_qag** y funciones relacionadas para la aproximación numérica de integrales definidas. Véase **residue** para el cálculo de residuos (integración compleja). Véase **antid** para un método alternativo de resolución de integrales indefinidas.

Se obtendrá una integral (es decir, una expresión sin **integrate**) si **integrate** tiene éxito en el cálculo. En otro caso, la respuesta es la forma nominal de la integral (esto es, el operador '**integrate**' precedido de apóstrofo) o una expresión que contiene una o más formas nominales. La forma nominal de **integrate** se muestra con un símbolo integral.

En ciertos casos es útil proporcionar una forma nominal 'a mano', haciendo preceder **integrate** con una comilla simple o apóstrofo, como en '**integrate** (*expr*, *x*)'. Por ejemplo, la integral puede depender de algunos parámetros que todavía no han sido calculados. La forma nominal puede aplicarse después a sus argumentos haciendo **ev** (*i*, **nouns**) donde *i* es la forma nominal de interés.

La función **integrate** trata de manera diferente las integrales definidas de las indefinidas, empleando una batería de heurísticas especial para cada caso. Casos especiales de integrales definidas incluyen las que tienen límites de integración iguales a cero o a infinito (**inf** o **minf**), funciones trigonométricas con límites de integración igual a cero y **%pi** o **2 %pi**, funciones racionales, integrales relacionadas con las funciones **beta** y **psi** y algunas integrales logarítmicas y trigonométricas. El tratamiento de funciones racionales puede incluir el cálculo de residuos. Si no se reconoce ninguno de los casos especiales, se intenta resolver la integral indefinida y evaluarla en los límites de integración. Esto incluye tomar límites cuando alguno de los extremos del intervalo de integración se acerca a más infinito o a menos infinito; véase también **ldefint**.

Casos especiales de integrales indefinidas incluyen a las funciones trigonométricas, exponenciales, logarítmicas y racionales. La función **integrate** también hace uso de una pequeña tabla de integrales elementales.

La función `integrate` puede llevar a cabo cambios de variable si el integrando es de la forma  $f(g(x)) * \text{diff}(g(x), x)$ , entonces `integrate` trata de encontrar una subexpresión de  $g(x)$  tal que la derivada de  $g(x)$  divida el integrando. Esta búsqueda puede hacer uso de las derivadas establecidas con la función `gradef`. Véanse también `changevar` y `antid`.

Si ninguna de las heurísticas descritas resuelve la integral indefinida, se ejecuta el algoritmo de Risch. La variable `risch` puede utilizarse como una `evflag`, en una llamada a `ev` o en la línea de comandos por ejemplo, `ev (integrate (expr, x), risch)` o `integrate (expr, x), risch`. Si `risch` está presente, `integrate` llama a la función `risch` sin intentar primero las heurísticas. Véase también `risch`.

La función `integrate` opera únicamente con relaciones funcionales que se representen explícitamente con la notación  $f(x)$ , sin considerar las dependencias implícitas establecidas mediante la función `depends`.

Es posible que `integrate` necesite conocer alguna propiedad de alguno de los parámetros presentes en el integrando, en cuyo caso `integrate` consultará en primer lugar la base de datos creada con `assume`, y si la variable de interés no se encuentra ahí, `integrate` le preguntará al usuario. Dependiendo de la pregunta, posibles respuestas son: `yes;`, `no;`, `pos;`, `zero;` o `neg;`.

Por defecto, `integrate` no se considera lineal. Véanse `declare` y `linear`.

La función `integrate` intentará la integración por partes sólo en casos especiales.

Ejemplos:

- Integrales elementales indefinidas y definidas.

```
(%i1) integrate (sin(x)^3, x);
 3
 cos (x)
(%o1) ----- - cos(x)
 3
(%i2) integrate (x/ sqrt (b^2 - x^2), x);
 2 2
 - sqrt(b - x)
(%o2) -----
 %pi
(%i3) integrate (cos(x)^2 * exp(x), x, 0, %pi);
 3 %e 3
 ----- - -
 5 5
(%o3) -----
 sqrt(%pi)
(%i4) integrate (x^2 * exp(-x^2), x, minf, inf);
 sqrt(%pi)
(%o4) -----
 2
```

- Utilización de `assume` e interacción.

```
(%i1) assume (a > 1)$
(%i2) integrate (x**a/(x+1)**(5/2), x, 0, inf);
 2 a + 2
Is ----- an integer?
 5
```

```
no;
Is 2 a - 3 positive, negative, or zero?

neg;

(%o2) 3
 beta(a + 1, - - a)
 2
```

- Cambio de variable. En este ejemplo hay dos cambios de variable: uno utilizando una derivada establecida con `gradef` y otra utilizando la derivada `diff(r(x))` de una función no especificada `r(x)`.

```
(%i3) gradef (q(x), sin(x**2));
(%o3) q(x)
(%i4) diff (log (q (r (x))), x);
 d 2
 (-- (r(x))) sin(r (x))
 dx
(%o4) -----
 q(r(x))

(%i5) integrate (%, x);
(%o5) log(q(r(x)))
```

- El valor devuelto contiene la forma nominal `'integrate`. En este ejemplo, Maxima puede extraer un factor del denominador de una función racional, pero no puede factorizar el resto. La función `grind` muestra la forma nominal `'integrate` del resultado. Véase también `integrate_use_rootsof` para más información sobre integrales de funciones racionales.

```
(%i1) expand ((x-4) * (x^3+2*x+1));
 4 3 2
(%o1) x - 4 x + 2 x - 7 x - 4
(%i2) integrate (1/%, x);
 / 2
 [x + 4 x + 18
 I ----- dx
] 3
log(x - 4) / x + 2 x + 1
----- - -----
 73 73

(%o2)
log(x-4)/73-('integrate((x^2+4*x+18)/(x^3+2*x+1),x))/73$

(%i3) grind (%);
log(x-4)/73-('integrate((x^2+4*x+18)/(x^3+2*x+1),x))/73$
```

- Definición de una función mediante una integral. El cuerpo de una función no se evalúa cuando ésta se define, de manera que el cuerpo de `f_1` en este ejemplo contiene la forma nominal de `integrate`. El operador consistente en dos comillas simples `''` hace que se evalúe la integral y su resultado será el que defina a la función `f_2`.

```
(%i1) f_1 (a) := integrate (x^3, x, 1, a);
 3
(%o1) f_1(a) := integrate(x , x, 1, a)
(%i2) ev (f_1 (7), nouns);
```

```
(%o2) 600
(%i3) /* Note parentheses around integrate(...) here */
 f_2 (a) := ''(integrate (x^3, x, 1, a));
 4
 a 1
(%o3) f_2(a) := -- - -
 4 4

(%i4) f_2 (7);
(%o4) 600
```

**integration\_constant\_counter**

Variable del sistema

Valor por defecto: 0

La variable `integration_constant_counter` es un contador que se actualiza cada vez que se introduce una constante de integración en una expresión como resultado de una integración indefinida.

**integrate\_use\_rootsof**

Variable optativa

Valor por defecto: false

Si `integrate_use_rootsof` vale `true` y el denominador de una función racional no se puede factorizar, `integrate` devuelve la integral como una suma respecto de las raíces desconocidas del denominador.

Por ejemplo, dándole a `integrate_use_rootsof` el valor `false`, `integrate` devuelve la integral no resuelta de la función racional en forma nominal:

```
(%i1) integrate_use_rootsof: false$
(%i2) integrate (1/(1+x+x^5), x);
 / 2
 [x - 4 x + 5
 I ----- dx
] 3 2
 / x - x + 1
(%o2) ----- - ----- + -----
 7 14 7 sqrt(3)
```

Si ahora se le da a la variable el valor `true`, la parte no resuelta de la integral se expresa como una suma cuyos sumandos dependen de las raíces del denominador de la función racional:

```
(%i3) integrate_use_rootsof: true$
(%i4) integrate (1/(1+x+x^5), x);
=====
\ 2
 (%r4 - 4 %r4 + 5) log(x - %r4)
> -----
/
=====
 3 %r4 - 2 %r4
 3 2
 %r4 in rootsof(x - x + 1)
(%o4) -----
 7
```

$$\frac{\log(x^2 + x + 1)}{14} + \frac{5 \operatorname{atan}\left(\frac{2x + 1}{\sqrt{3}}\right)}{7 \sqrt{3}}$$

Alternativamente, el usuario puede calcular las raíces del denominador separadamente y luego representar el integrando en función de dichas raíces, como por ejemplo  $1/((x - a)(x - b)(x - c))$  o  $1/((x^2 - (a+b)x + a*b)(x - c))$  si el denominador es un polinomio de tercer grado. En algunos casos, esto ayudará a Maxima mejorar sus resultados.

### **ldefint** (*expr*, *x*, *a*, *b*)

Función

Calcula la integral definida de *expr* utilizando **limit** tras el cálculo de la integral indefinida de *expr* respecto a *x* en los extremos de integración *b* y *a*. Si no consigue calcular la integral definida, **ldefint** devuelve una expresión con los límites en forma nominal.

La función **integrate** no llama a **ldefint**, de modo que la ejecución de **ldefint** (*expr*, *x*, *a*, *b*) puede dar un resultado diferente que **integrate** (*expr*, *x*, *a*, *b*). La función **ldefint** siempre utiliza el mismo método para calcular la integral definida, mientras que **integrate** puede hacer uso de varias heurísticas y reconocer así casos especiales.

### **qq**

Función

El paquete **qq**, que puede cargarse haciendo **load** ("qq"), contiene la función **quanc8** que puede tomar 3 ó 4 argumentos. La versión para 3 argumentos calcula la integral de la función especificada en el primer argumento dentro del intervalo que va desde *lo* hasta *hi*, como en **quanc8** ('function', *lo*, *hi*). El nombre de la función debe ir precedido del apóstrofo. La versión para 4 argumentos calcula la integral de la función especificada como primer argumento con respecto a la variable, que es el segundo argumento, en el intervalo que va desde *lo* hasta *hi*, como en **quanc8**(<*f*(*x*) or expression in *x*>, *x*, *lo*, *hi*). El método utilizado es el de cuadratura polinomial de octavo orden de Newton-Cotes.

### **quanc8** (*expr*, *a*, *b*)

Función

Es un integrador adaptativo. Se incluyen ejemplos de uso. El método que sigue es el de cuadratura polinomial de octavo orden de Newton-Cotes, de ahí el nombre de la función **quanc8**, disponible para 3 y 4 argumentos. Se hace control de error absoluto y relativo. Se carga en memoria haciendo **load** ("qq"). Véase también **qq**.

### **residue** (*expr*, *z*, *z\_0*)

Función

Calcula el residuo en el plano complejo de la expresión *expr* cuando la variable *z* toma el valor *z\_0*. El residuo es el coeficiente de  $(z - z_0)^{-1}$  en el desarrollo de Laurent de *expr*.

```
(%i1) residue (s/(s**2+a**2), s, a%i);
1
(%o1) -
```

```
(%i2) residue (sin(a*x)/x**4, x, 0);
3
a
(%o2) - --
6
```

**risch** (*expr*, *x*)

Función

Integra *expr* respecto de *x* utilizando el caso trascendental del algoritmo de Risch. El caso algebraico del algoritmo de Risch no se ha implementado. Este método trata los casos de exponenciales y logaritmos anidados que no resuelve el procedimiento principal de **integrate**. La función **integrate** llamará automáticamente a **risch** si se presentan estos casos.

Si la variable **erfflag** vale **false**, evita que **risch** introduzca la función **erf** en la respuesta si ésta no estaba presente previamente en el integrando.

```
(%i1) risch (x^2*erf(x), x);
3 2 2
%pi x erf(x) + (sqrt(%pi) x + sqrt(%pi)) %e - x
(%o1) -----
3 %pi
(%i2) diff(% , x), ratsimp;
2
(%o2) x erf(x)
```

**romberg** (*expr*, *x*, *a*, *b*)

Función

**romberg** (*expr*, *a*, *b*)

Función

Integración de Romberg. Hay dos formas de utilizar esta función. La primera es ineficiente: **romberg** (<integrando>, <variable de integración>, <límite inferior>, <límite superior>).

Ejemplos:

```
(%i1) showtime: true$
(%i2) romberg (sin(y), y, 0, %pi);
Evaluation took 0.00 seconds (0.01 elapsed) using 25.293 KB.
(%o2) 2.000000016288042
(%i3) 1/((x-1)^2+1/100) + 1/((x-2)^2+1/1000) + 1/((x-3)^2+1/200)$
(%i4) f(x) := ''%$
(%i5) rombergtol: 1e-6$
(%i6) rombergit: 15$
(%i7) romberg (f(x), x, -5, 5);
Evaluation took 11.97 seconds (12.21 elapsed) using 12.423 MB.
(%o7) 173.6730736617464
```

La segunda forma es más eficiente y se utiliza como sigue:

```
romberg (<function name>, <lower limit>, <upper limit>);
```

Continuando con el ejemplo anterior, se tiene:



```
(%i8) f(x) := (mode_declare ([function(f), x], float), ''(%th(5)))$
(%i9) translate(f);
(%o9) [f]
(%i10) romberg (f, -5, 5);
Evaluation took 3.51 seconds (3.86 elapsed) using 6.641 MB.
(%o10) 173.6730736617464
```

El primer argumento debe ser una función traducida o compilada. (Si está compilada, debe declararse para que devuelva un `flonum`.) Si el primer argumento no está ya traducido, `romberg` no intentará traducirlo y emitirá un mensaje de error.

La precisión de la integración se controla con las variables globales `rombergtol` (valor por defecto 1.E-4) y `rombergit` (valor por defecto 11). La función `romberg` devolverá un resultado si la diferencia relativa de aproximaciones sucesivas es menor que `rombergtol`. Intentará disminuir el tamaño de los pasos `rombergit` veces antes de abandonar. El número de iteraciones y evaluaciones de las funciones que hará `romberg` se controla con `rombergabs` y `rombergmin`.

La función `romberg` puede llamarse recursivamente, de manera que puede calcular integrales dobles y triples.

Ejemplo:

```
(%i1) assume (x > 0)$
(%i2) integrate (integrate (x*y/(x+y), y, 0, x/2), x, 1, 3)$
(%i3) radcan (%);
(%o3) 26 log(3) - 26 log(2) - 13
 - -----
 3
(%i4) %,numer;
(%o4) .8193023963959073
(%i5) define_variable (x, 0.0, float, "Global variable in function F")$
(%i6) f(y) := (mode_declare (y, float), x*y/(x+y))$
(%i7) g(x) := romberg ('f, 0, x/2)$
(%i8) romberg (g, 1, 3);
(%o8) .8193022864324522
```

## rombergabs

Variable optativa

Valor por defecto: 0.0

Supuesto que las sucesivas estimaciones producidas por `romberg` son `y[0]`, `y[1]`, `y[2]`, etc., entonces `romberg` devolverá un resultado después de `n` iteraciones, si

$$(\text{abs}(y[n] - y[n-1]) \leq \text{rombergabs} \text{ or } \text{abs}(y[n] - y[n-1]) / (\text{if } y[n] = 0.0 \text{ then } 1.0 \text{ else } y[n]) \leq \text{rombergtol})$$

vale `true`. (La condición sobre el número de iteraciones dado por `rombergmin` también debe verificarse.) Así, si `rombergabs` es 0.0 (el valor por defecto) se realiza el test relativo.

## rombergit

Variable optativa

Valor por defecto: 11

La precisión de la función de integración `romberg` está controlada por las variables globales `rombergtol` y `rombergit`. La función `romberg` devolverá un resultado si la

diferencia relativa de aproximaciones sucesivas es menor que `rombergtol`. Intentará disminuir el tamaño de los pasos `rombergit` veces antes de abandonar.

**rombergmin**

Variable optativa

Valor por defecto: 0

La variable `rombergmin` controla el número mínimo de evaluaciones de función que realizará la función `romberg`, la cual calculará su primer argumento al menos  $2^{(\text{rombergmin}+2)+1}$  veces.

**rombergtol**

Variable optativa

Valor por defecto: 1e-4

La precisión de la función de integración `romberg` está controlada por las variables globales `rombergtol` y `rombergit`. La función `romberg` devolverá un resultado si la diferencia relativa de aproximaciones sucesivas es menor que `rombergtol`. Intentará disminuir el tamaño de los pasos `rombergit` veces antes de abandonar.

**tldefint** (*expr*, *x*, *a*, *b*)

Función

Equivale a `ldefint` cuando `tlimswitch` vale `true`.

**quad\_qag** (*f(x)*, *x*, *a*, *b*, *key*, *epsrel*, *limit*)

Función

Calcula numéricamente la integral

$$\int_a^b f(x)dx$$

utilizando un integrador adaptativo simple.

La función a integrar es  $f(x)$ , con variable independiente  $x$ , siendo el intervalo de integración el comprendido entre  $a$  y  $b$ . El argumento *key* indica el integrador a utilizar y debe ser un número entero entre 1 y 6, ambos inclusive. El valor de *key* selecciona el orden de la regla de integración de Gauss-Kronrod.

La integración numérica se hace de forma adaptativa particionando la región de integración en subintervalos hasta conseguir la precisión requerida.

Los argumentos opcionales *epsrel* y *limit* son el error relativo y el número máximo de subintervalos, respectivamente. La variable *epsrel* tiene por defecto el valor 1e-8 y *limit* el de 200.

La función `quad_qag` devuelve una lista de cuatro elementos:

- la aproximación a la integral,
- el error absoluto estimado de la aproximación,
- el número de evaluaciones del integrando,
- un código de error.

El código de error (el cuarto elemento del resultado) puede tener los siguientes valores:

- 0            si no ha habido problemas;
- 1            si se utilizaron demasiados intervalos;

- 2 si se encontró un número excesivo de errores de redondeo;
- 3 si el integrando ha tenido un comportamiento extraño frente a la integración;
- 6 si los argumentos de entrada no son válidos.

Ejemplos:

```
(%i1) quad_qag (x^(1/2)*log(1/x), x, 0, 1, 3);
(%o1) [.4444444444492108, 3.1700968502883E-9, 961, 0]
(%i2) integrate (x^(1/2)*log(1/x), x, 0, 1);
(%o2) 4
 -
 9
```

### **quad\_qags** ( $f(x)$ , $x$ , $a$ , $b$ , $epsrel$ , $limit$ )

Función

Integra numéricamente la función dada utilizando una cuadratura adaptativa con extrapolación. La función a integrar es  $f(x)$ , con variable independiente  $x$ , siendo el intervalo de integración el comprendido entre  $a$  y  $b$ .

Los argumentos opcionales *epsrel* y *limit* son el error relativo y el número máximo de subintervalos, respectivamente. La variable *epsrel* tiene por defecto el valor  $1e-8$  y *limit* el de 200.

La función **quad\_qags** devuelve una lista de cuatro elementos:

- la aproximación a la integral,
- el error absoluto estimado de la aproximación,
- el número de evaluaciones del integrando,
- un código de error.

El código de error (el cuarto elemento del resultado) puede tener los siguientes valores:

- 0 si no ha habido problemas;
- 1 si se utilizaron demasiados intervalos;
- 2 si se encontró un número excesivo de errores de redondeo;
- 3 si el integrando ha tenido un comportamiento extraño frente a la integración;
- 4 fallo de convergencia;
- 5 la integral es probablemente divergente o de convergencia lenta;
- 6 si los argumentos de entrada no son válidos.

Ejemplos:

```
(%i1) quad_qags (x^(1/2)*log(1/x), x, 0, 1);
(%o1) [.4444444444444448, 1.11022302462516E-15, 315, 0]
```

Nótese que **quad\_qags** es más precisa y eficiente que **quad\_qag** para este integrando.

**quad\_qagi** (*f(x)*, *x*, *a*, *inf**type*, *eps**rel*, *limit*)

Función

Calcula numéricamente cualquiera las siguientes integrales:

$$\int_a^\infty f(x)dx$$

$$\int_\infty^a f(x)dx$$

$$\int_{-\infty}^\infty f(x)dx$$

utilizando la rutina QAGI de Quadpack QAGI. La función a integrar es  $f(x)$ , con variable independiente  $x$ , siendo el intervalo de integración de rango infinito.

El argumento *inf**type* determina el intervalo de integración como sigue:

**inf** El intervalo es desde  $a$  hasta más infinito.  
**minf** El intervalo es desde menos infinito hasta  $a$ .  
**both** El intervalo es toda la recta real.

Los argumentos opcionales *eps**rel* y *limit* son el error relativo y el número máximo de subintervalos, respectivamente. La variable *eps**rel* tiene por defecto el valor  $1e-8$  y *limit* el de 200.

La función **quad\_qagi** devuelve una lista de cuatro elementos:

la aproximación a la integral,  
 el error absoluto estimado de la aproximación,  
 el número de evaluaciones del integrando,  
 un código de error.

El código de error (el cuarto elemento del resultado) puede tener los siguientes valores:

0 si no ha habido problemas;  
 1 si se utilizaron demasiados intervalos;  
 2 si se encontró un número excesivo de errores de redondeo;  
 3 si el integrando ha tenido un comportamiento extraño frente a la integración;  
 4 fallo de convergencia;  
 5 la integral es probablemente divergente o de convergencia lenta;  
 6 si los argumentos de entrada no son válidos.

Ejemplos:

```
(%i1) quad_qagi (x^2*exp(-4*x), x, 0, inf);
(%o1) [0.03125, 2.95916102995002E-11, 105, 0]
(%i2) integrate (x^2*exp(-4*x), x, 0, inf);
1
--
32
```

**quad\_qawc** ( $f(x)$ ,  $x$ ,  $c$ ,  $a$ ,  $b$ ,  $epsrel$ ,  $limit$ )

Función

Calcula numéricamente el valor principal de Cauchy de

$$\int_a^b \frac{f(x)}{x - c} dx$$

utilizando la rutina QAWC de Quadpack. La función a integrar es  $f(x)/(x - c)$ , con variable independiente  $x$ , siendo el intervalo de integración el comprendido entre  $a$  y  $b$ .

Los argumentos opcionales *epsrel* y *limit* son el error relativo y el número máximo de subintervalos, respectivamente. La variable *epsrel* tiene por defecto el valor 1e-8 y *limit* el de 200.

**quad\_qawc** returns a list of four elements:

- la aproximación a la integral,
- el error absoluto estimado de la aproximación,
- el número de evaluaciones del integrando,
- un código de error.

El código de error (el cuarto elemento del resultado) puede tener los siguientes valores:

- 0 si no ha habido problemas;
- 1 si se utilizaron demasiados intervalos;
- 2 si se encontró un número excesivo de errores de redondeo;
- 3 si el integrando ha tenido un comportamiento extraño frente a la integración;
- 6 si los argumentos de entrada no son válidos.

Ejemplos:

```
(%i1) quad_qawc (2^(-5)*((x-1)^2+4^(-5))^(-1), x, 2, 0, 5);
(%o1) [- 3.130120337415925, 1.306830140249558E-8, 495, 0]
(%i2) integrate (2^(-alpha)*(((x-1)^2 + 4^(-alpha))*(x-2))^(-1), x, 0, 5);
Principal Value
```

$$\begin{aligned}
 & \frac{\log\left(\frac{\alpha^9}{64^4} + \frac{\alpha^9}{64^4}\right)}{\alpha^2 + 2} \\
 & - \frac{\frac{3\alpha}{2^4} \operatorname{atan}\left(\frac{\alpha/2}{4^4}\right) - \frac{3\alpha}{2^4} \operatorname{atan}\left(\frac{\alpha/2}{4^4}\right) \alpha}{\alpha}
 \end{aligned}$$

```

 2 4 + 2 2 4 + 2
(%i3) ev (% , alpha=5, numer);
(%o3) - 3.130120337415917

```

**quad\_qawf** ( $f(x)$ ,  $x$ ,  $a$ ,  $\omega$ ,  $\text{trig}$ ,  $\text{epsabs}$ ,  $\text{limit}$ ,  $\text{maxp1}$ ,  $\text{limlst}$ ) Función  
 Calcula numéricamente integrales de tipo Fourier utilizando la rutina QAWF de Quadpack. La integral es

$$\int_a^\infty f(x)w(x)dx$$

La función peso  $w$  se selecciona mediante  $\text{trig}$ :

**cos**       $w(x) = \cos(\omega x)$

**sin**       $w(x) = \sin(\omega x)$

Los argumentos opcionales son:

**epsabs**      El error absoluto deseado para la aproximación. El valor por defecto es 1d-10.

**limit**      Tamaño del arreglo interno de trabajo.  $(\text{limit} - \text{limlst})/2$  es el número máximo de subintervalos para la partición. El valor por defecto es 200.

**maxp1**      Número máximo de momentos de Chebyshev. Debe ser mayor que 0. El valor por defecto es 100.

**limlst**      Cota superior del número de ciclos. Debe ser mayor o igual que 3. El valor por defecto es 10.

Los argumentos opcionales  $\text{epsrel}$  y  $\text{limit}$  son el error relativo y el número máximo de subintervalos, respectivamente. La variable  $\text{epsrel}$  tiene por defecto el valor 1e-8 y  $\text{limit}$  el de 200.

**quad\_qawf** returns a list of four elements:

- la aproximación a la integral,
- el error absoluto estimado de la aproximación,
- el número de evaluaciones del integrando,
- un código de error.

El código de error (el cuarto elemento del resultado) puede tener los siguientes valores:

- 0      si no ha habido problemas;
- 1      si se utilizaron demasiados intervalos;
- 2      si se encontró un número excesivo de errores de redondeo;
- 3      si el integrando ha tenido un comportamiento extraño frente a la integración;
- 6      si los argumentos de entrada no son válidos.

Ejemplos:

```
(%i1) quad_qawf (exp(-x^2), x, 0, 1, 'cos);
(%o1) [.6901942235215714, 2.84846300257552E-11, 215, 0]
(%i2) integrate (exp(-x^2)*cos(x), x, 0, inf);
 - 1/4
 %e sqrt(%pi)
(%o2) -----
 2
(%i3) ev (% , numer);
(%o3) .6901942235215714
```

**quad\_qawo** ( $f(x)$ ,  $x$ ,  $a$ ,  $b$ ,  $\omega$ ,  $\text{trig}$ ,  $\text{epsabs}$ ,  $\text{limit}$ ,  $\text{maxp1}$ ,  $\text{limlst}$ )

Función

Realiza la integración numérica utilizando la rutina QAWO de Quadpack:

$$\int_a^b f(x)w(x)dx$$

La función peso  $w$  se selecciona mediante  $\text{trig}$ :

**cos**       $w(x) = \cos(\omega x)$

**sin**       $w(x) = \sin(\omega x)$

Los argumentos opcionales son:

**epsabs**      El error absoluto deseado para la aproximación. El valor por defecto es  $1d-10$ .

**limit**      Tamaño del arreglo interno de trabajo.  $(\text{limit} - \text{limlst})/2$  es el número máximo de subintervalos para la partición. El valor por defecto es 200.

**maxp1**      Número máximo de momentos de Chebyshev. Debe ser mayor que 0. El valor por defecto es 100.

**limlst**      Cota superior del número de ciclos. Debe ser mayor o igual que 3. El valor por defecto es 10.

Los argumentos opcionales  $\text{epsrel}$  y  $\text{limit}$  son el error relativo y el número máximo de subintervalos, respectivamente. La variable  $\text{epsrel}$  tiene por defecto el valor  $1e-8$  y  $\text{limit}$  el de 200.

**quad\_qawo** returns a list of four elements:

- la aproximación a la integral,
- el error absoluto estimado de la aproximación,
- el número de evaluaciones del integrando,
- un código de error.

El código de error (el cuarto elemento del resultado) puede tener los siguientes valores:

- 0      si no ha habido problemas;
- 1      si se utilizaron demasiados intervalos;
- 2      si se encontró un número excesivo de errores de redondeo;

- 3 si el integrando ha tenido un comportamiento extraño frente a la integración;
- 6 si los argumentos de entrada no son válidos.

Ejemplos:

```
(%i1) quad_qawo (x^(-1/2)*exp(-2^(-2)*x), x, 1d-8, 20*2^2, 1, cos);
(%o1) [1.376043389877692, 4.72710759424899E-11, 765, 0]
(%i2) rectform (integrate (x^(-1/2)*exp(-2^(-alpha)*x) * cos(x), x, 0, inf));
 alpha/2 - 1/2 2 alpha
sqrt(%pi) 2 sqrt(sqrt(2 + 1) + 1)
(%o2) -----
 2 alpha
 sqrt(2 + 1)
(%i3) ev (% , alpha=2, numer);
(%o3) 1.376043390090716
```

**quad\_qaws** ( $f(x)$ ,  $x$ ,  $a$ ,  $b$ ,  $alfa$ ,  $beta$ ,  $wfun$ ,  $epsabs$ ,  $limit$ )

Función

Realiza la integración numérica utilizando la rutina QAWS de Quadpack:

$$\int_a^b f(x)w(x)dx$$

La función peso  $w$  se selecciona mediante  $wfun$ :

- 1  $w(x) = (x - a)^{alfa}(b - x)^beta$
- 2  $w(x) = (x - a)^{alfa}(b - x)^beta \log(x - a)$
- 3  $w(x) = (x - a)^{alfa}(b - x)^beta \log(b - x)$
- 2  $w(x) = (x - a)^{alfa}(b - x)^beta \log(x - a) \log(b - x)$

Los argumentos opcionales son:

- epsabs* El error absoluto deseado para la aproximación. El valor por defecto es 1d-10.
- limit* Tamaño del arreglo interno de trabajo.  $(limit - limlst)/2$  es el número máximo de subintervalos para la partición. El valor por defecto es 200.

Los argumentos opcionales *epsrel* y *limit* son el error relativo y el número máximo de subintervalos, respectivamente. La variable *epsrel* tiene por defecto el valor 1e-8 y *limit* el de 200.

**quad\_qaws** returns a list of four elements:

- la aproximación a la integral,
- el error absoluto estimado de la aproximación,
- el número de evaluaciones del integrando,
- un código de error.

El código de error (el cuarto elemento del resultado) puede tener los siguientes valores:

- 0 si no ha habido problemas;



- 1 si se utilizaron demasiados intervalos;
- 2 si se encontró un número excesivo de errores de redondeo;
- 3 si el integrando ha tenido un comportamiento extraño frente a la integración;
- 6 si los argumentos de entrada no son válidos.

Ejemplos:

```
(%i1) quad_qaws (1/(x+1+2^(-4)), x, -1, 1, -0.5, -0.5, 1);
(%o1) [8.750097361672832, 1.24321522715422E-10, 170, 0]
(%i2) integrate ((1-x*x)^(-1/2)/(x+1+2^(-alpha)), x, -1, 1);
 alpha
Is 4 2 - 1 positive, negative, or zero?
```

pos;

```

 alpha alpha
2 %pi 2 sqrt(2 2 + 1)
(%o2) -----
 alpha
 4 2 + 2
(%i3) ev (%, alpha=4, numer);
(%o3) 8.750097361672829
```

## 21 Ecuaciones

### 21.1 Definiciones para Ecuaciones

**%rnum\_list**

Variable del sistema

Valor por defecto: `[]`

La variable `%rnum_list` es la lista de variables introducidas en las soluciones por la función `algsys`. Las variables `%r` se añaden a `%rnum_list` en su orden de creación. Esto es útil para hacer sustituciones en la solución.

**algexact**

Variable opcional

Default value: `false`

El contenido de la variable `algexact` afecta al comportamiento de `algsys` de la siguiente forma:

Si `algexact` vale `true`, `algsys` llamará siempre a `solve` y luego utilizará `realroots`.

Si `algexact` vale `false`, `solve` será llamada sólo si la ecuación no es univariante, o si es cuadrática o bicuadrática.

Sin embargo, `algexact: true` no garantiza que únicamente se obtengan soluciones exactas, ya que aunque `algsys` intente siempre dar soluciones exactas, dará resultados aproximados si no encuentra una solución mejor.

**algsys** (`[expr_1, ..., expr_m], [x_1, ..., x_n]`)

Función

**algsys** (`[eqn_1, ..., eqn_m], [x_1, ..., x_n]`)

Función

Resuelve el sistema de ecuaciones polinómicas `expr_1, ..., expr_m` o las ecuaciones `eqn_1, ..., eqn_m` para las variables `x_1, ..., x_n`. La expresión `expr` equivale a la ecuación `expr = 0`. Puede haber más ecuaciones que variables o viceversa.

La función `algsys` devuelve una lista de soluciones, cada una de las cuales consistente a su vez en una lista de ecuaciones asociando valores a las variables `x_1, ..., x_n` que satisfacen el sistema de ecuaciones. Si `algsys` no puede encontrar soluciones devuelve la lista vacía `[]`.

Si es necesario se introducen en la solución los símbolos `%r1, %r2, ...`, para representar parámetros arbitrarios; estas variables también se añaden a la lista `%rnum_list`.

El proceso que se sigue es el siguiente:

- (1) Primero se factorizan las ecuaciones y se reparten en subsistemas.
- (2) Para cada subsistema  $S_i$ , se seleccionan una ecuación  $E$  y una variable  $x$ . Se elige la variable que tenga grado menor. Entonces se calcula el resultado de  $E$  y  $E_j$  respecto de  $x$ , siendo las  $E_j$  el resto de ecuaciones del subsistema  $S_i$ . De aquí se obtiene otro subsistema  $S_i'$  con una incógnita menos, ya que  $x$  ha sido eliminada. El proceso ahora vuelve al paso (1).
- (3) En ocasiones se obtiene un subsistema consistente en una única ecuación. Si la ecuación es multivariante y no se han introducido aproximaciones en formato decimal de coma flotante, entonces se llama a `solve` para tratar de encontrar una solución exacta.

En algunos casos, `solve` no puede encontrar la solución, o si lo consigue puede que el resultado tenga una expresión muy grande.

Si la ecuación tiene una sólo incógnita y es lineal, o cuadrática o bicuadrática, entonces se llama a la función `solve` si no se han introducido aproximaciones en formato decimal. Si se han introducido aproximaciones, o si hay más de una incógnita, o si no es lineal, ni cuadrática ni bicuadrática, y si la variable `realonly` vale `true`, entonces se llama a la función `realroots` para calcular las soluciones reales. Si `realonly` vale `false`, entonces se llama a `allroots` para obtener las soluciones reales y complejas.

Si `algsys` devuelve una solución que tiene menos dígitos significativos de los requeridos, el usuario puede cambiar a voluntad el valor de `algepsilon` para obtener mayor precisión.

Si `algexact` vale `true`, se llamará siempre a `solve`.

Cuando `algsys` encuentra una ecuación con múltiples incógnitas y que contiene aproximaciones en coma flotante (normalmente debido a la imposibilidad de encontrar soluciones exactas en pasos anteriores), entonces no intenta aplicar los métodos exactos a estas ecuaciones y presenta el mensaje: "`algsys cannot solve - system too complicated.`"

Las interacciones con `radcan` pueden dar lugar a expresiones grandes o complicadas. En tal caso, puede ser posible aislar partes del resultado con `pickapart` o `reveal`.

Ocasionalmente, `radcan` puede introducir la unidad imaginaria `%i` en una solución que de hecho es real.

Ejemplos:

```
(%i1) e1: 2*x*(1 - a1) - 2*(x - 1)*a2;
(%o1) 2 (1 - a1) x - 2 a2 (x - 1)
(%i2) e2: a2 - a1;
(%o2) a2 - a1
(%i3) e3: a1*(-y - x^2 + 1);
(%o3) a1 (- y - x^2 + 1)
(%i4) e4: a2*(y - (x - 1)^2);
(%o4) a2 (y - (x - 1)^2)
(%i5) algsys ([e1, e2, e3, e4], [x, y, a1, a2]);
(%o5) [[x = 0, y = %r1, a1 = 0, a2 = 0],
[x = 1, y = 0, a1 = 1, a2 = 1]]
(%i6) e1: x^2 - y^2;
(%o6) x^2 - y^2
(%i7) e2: -1 - y + 2*y^2 - x + x^2;
(%o7) 2 y^2 - y + x^2 - x - 1
(%i8) algsys ([e1, e2], [x, y]);
(%o8) [[x = - ----, y = ----],
 1 1
 sqrt(3) sqrt(3)]
```

$$[x = \frac{1}{\sqrt{3}}, y = -\frac{1}{\sqrt{3}}], [x = -\frac{1}{3}, y = -\frac{1}{3}], [x = 1, y = 1]]$$

**allroots** (*expr*)

### Función

**allroots** (*eqn*)

### Función

Calcula aproximaciones numéricas de las raíces reales y complejas del polinomio *expr* o ecuación polinómica *eqn* de una variable.

Si la variable `polyfactor` vale `true` hace que la función `allroots` factorice el polinomio para números reales si el polinomio es real, o para números complejos si el polinomio es complejo.

La función `allroots` puede dar resultados inexactos en caso de que haya raíces múltiples. Si el polinomio es real, `allroots (%i*p))` puede alcanzar mejores aproximaciones que `allroots (p)`, ya que `allroots` ejecuta entonces un algoritmo diferente.

La función `allroots` no opera sobre expresiones no polinómicas, pues requiere que el numerador sea reducible a un polinomio y el denominador sea, como mucho, un número complejo.

Para polinomios complejos se utiliza el algoritmo de Jenkins y Traub descrito en (Algorithm 419, *Comm. ACM*, vol. 15, (1972), p. 97). Para polinomios reales se utiliza el algoritmo de Jenkins descrito en (Algorithm 493, *ACM TOMS*, vol. 1, (1975), p.178).

Ejemplos:

```
(%i1) eqn: (1 + 2*x)^3 = 13.5*(1 + x^5);
 3 5
(%o1) (2 x + 1) = 13.5 (x + 1)
(%i2) soln: allroots (eqn);
(%o2) [x = .8296749902129361, x = - 1.015755543828121,
x = .9659625152196369 %i - .4069597231924075,
x = - .9659625152196369 %i - .4069597231924075, x = 1.0]
(%i3) for e in soln
 do (e2: subst (e, eqn), disp (expand (lhs(e2) - rhs(e2))));
 - 3.5527136788005E-15
 - 5.32907051820075E-15
 4.44089209850063E-15 %i - 4.88498130835069E-15
 - 4.44089209850063E-15 %i - 4.88498130835069E-15
 3.5527136788005E-15
(%o3) done
(%i4) polyfactor: true$
```

```
(%i5) allroots (eqn);
(%o5) - 13.5 (x - 1.0) (x - .8296749902129361)

(x + 1.015755543828121) (x + .8139194463848151 x
+ 1.098699797110288)
```

**breakup**

Variable opcional

Valor por defecto: **true**

Si **breakup** vale **true**, **solve** expresa sus soluciones a las ecuaciones cúbicas y cuárticas en términos de subexpresiones comunes, las cuales son asignadas a etiquetas del tipo **%t1**, **%t2**, etc. En otro caso, no se identifican subexpresiones comunes.

La asignación **breakup: true** sólo tiene efecto cuando **programmode** vale **false**.

Ejemplos:

```
(%i1) programmode: false$
(%i2) breakup: true$
(%i3) solve (x^3 + x^2 - 1);
```

```
(%t3) sqrt(23) 25 1/3
 (----- + --)
 6 sqrt(3) 54
```

Solution:

```
(%t4) sqrt(3) %i 1
 (- ---- - -) %t3 + ---- - -
 2 2 9 %t3 3
```

```
(%t5) sqrt(3) %i 1
 (- ---- - -) %t3 + ---- - -
 2 2 9 %t3 3
```

```
(%t6) 1 1
 x = %t3 + ---- - -
 9 %t3 3
```

```
(%o6) [%t4, %t5, %t6]
```

```
(%i6) breakup: false$
(%i7) solve (x^3 + x^2 - 1);
```

Solution:

```
(%t7) sqrt(3) %i 1
 (----- - -)
 2 2 sqrt(23) 25 1/3
 x = ----- + (----- + --)
```

```

 sqrt(23) 25 1/3 6 sqrt(3) 54
 9 (----- + --)
 6 sqrt(3) 54

 sqrt(3) %i 1 1
 (- ----- - -) - -
 2 2 3

(%t8) x = (sqrt(23) 25 1/3 sqrt(3) %i 1
 6 sqrt(3) 54 2 2)

 sqrt(3) %i 1
 - ----- - -
 2 2 1
 + ----- - -
 sqrt(23) 25 1/3 3
 9 (----- + --)
 6 sqrt(3) 54

(%t9) x = (sqrt(23) 25 1/3 1 1
 6 sqrt(3) 54 9 (----- + --) 3
 sqrt(23) 25 1/3
 6 sqrt(3) 54

(%o9) [%t7, %t8, %t9]
```

**dimension** (*eqn*)

Función

**dimension** (*eqn\_1*, ..., *eqn\_n*)

Función

El paquete **dimen** es para análisis dimensional. La instrucción **load** ("dimen") carga el paquete y **demo** ("dimen") presenta una pequeña demostración.

**dispflag**

Variable opcional

Valor por defecto: **true**

Si **dispflag** vale **false**, entonces se inhibirá que Maxima muestre resultados de las funciones que resuelven ecuaciones cuando éstas son llamadas desde dentro de un bloque (**block**). Cuando un bloque termina con el signo del dólar, \$, a la variable **dispflag** se le asigna **false**.

**funcsolve** (*eqn*, *g(t)*)

Función

Devuelve [*g(t)* = ...] o [], dependiendo de que exista o no una función racional *g(t)* que satisfaga *eqn*, la cual debe ser un polinomio de primer orden, lineal para *g(t)* y *g(t+1)*

```

(%i1) eqn: (n + 1)*f(n) - (n + 3)*f(n + 1)/(n + 1) = (n - 1)/(n + 2);
 (n + 3) f(n + 1) n - 1
(%o1) (n + 1) f(n) - ----- = -----
 n + 1 n + 2

(%i2) funcsolve (eqn, f(n));
```

Dependent equations eliminated: (4 3)

$$f(n) = \frac{n}{(n+1)(n+2)}$$

Aviso: esta es una implemetación rudimentaria, por lo que debe ser utilizada con cautela.

## globalsolve

Variable opcional

Valor por defecto: `false`

Si `globalsolve` vale `true`, a las incógnitas de las ecuaciones se les asignan las soluciones encontradas por `linsolve` y por `solve` cuando se resuelven sistemas de dos o más ecuaciones lineales.

Si `globalsolve` vale `false`, las soluciones encontradas por `linsolve` y por `solve` cuando se resuelven sistemas de dos o más ecuaciones lineales se expresan como ecuaciones y a las incógnitas no se le asignan valores.

Cuando se resuelven ecuaciones que no son sistemas de dos o más ecuaciones lineales, `solve` ignora el valor de `globalsolve`. Otras funciones que resuelven ecuaciones (como `algsys`) ignoran siempre el valor de `globalsolve`.

Ejemplos:

```
(%i1) globalsolve: true$
(%i2) solve ([x + 3*y = 2, 2*x - y = 5], [x, y]);
Solution

(%t2) 17
 x : --
 7

(%t3) 1
 y : - -
 7

(%o3) [[%t2, %t3]]
(%i3) x;

(%o3) 17
 --
 7

(%i4) y;

(%o4) 1
 - -
 7

(%i5) globalsolve: false$
(%i6) kill (x, y)$
(%i7) solve ([x + 3*y = 2, 2*x - y = 5], [x, y]);
Solution

(%t7) 17
 x = --
```

```

 7
 1
 7
(%t8) y = - -
(%o8) [%t7, %t8]]
(%i8) x;
(%o8) x
(%i9) y;
(%o9) y

```

**ieqn** (*ie, unk, tech, n, guess*)

Función

El paquete **inteqn** se dedica a la resolución de ecuaciones integrales. Para hacer uso de él, ejecutar la instrucción **load ("inteqn")**.

El argumento *ie* es la ecuación integral; *unk* es la función incógnita; *tech* es el método a aplicar para efectuar la resolución del problema (*tech* = **first** significa: aplica el primer método que encuentre una solución; *tech* = **all** significa: aplica todos los métodos posibles); *n* es el número máximo de términos que debe tomar **taylor**, **neumann**, **firstkindseries** o **fredseries** (también es el máximo nivel de recursión para el método de diferenciación); *guess* es la solución candidata inicial para **neumann** o **firstkindseries**.

Valores por defecto para los argumentos segundo a quinto son:

*unk*:  $p(x)$ , donde  $p$  es la primera función desconocida que Maxima encuentra en el integrando y  $x$  es la variable que actúa como argumento en la primera aparición de  $p$  encontrada fuera de una integral en el caso de ecuaciones de segunda especie (**secondkind**), o es la única variable aparte de la de integración en el caso de ecuaciones de primera especie (**firstkind**). Si el intento de encontrar  $x$  falla, el usuario será consultado para suministrar una variable independiente.

**ieqnprint**

Variable opcional

Valor por defecto: **true**

La variable **ieqnprint** controla el comportamiento del resultado retornado por la instrucción **ieqn**. Si **ieqnprint** vale **false**, la lista devuelta por la función **ieqn** tiene el formato

[*solución, método utilizado, nterms, variable*]

donde *variable* estará ausente si la solución es exacta; en otro caso, será la palabra **approximate** o **incomplete** según que la solución sea inexacta o que no tenga forma explícita, respectivamente. Si se ha utilizado un método basado en series, *nterms* es el número de términos utilizado, que puede ser menor que el **n** dado a **ieqn**.

**lhs** (*expr*)

Función

Devuelve el miembro izquierdo (es decir, el primer argumento) de la expresión *expr*, cuando el operador de *expr* es uno de los operadores de relación **< <= = # equal notequal >= >**, o un operadores de asignación **:= ::= : ::**, o un operador infijo binario definido por el usuario mediante **infix**.



Si *expr* es un átomo o si su operador es diferente de los citados más arriba, *lhs* devuelve *expr*.

Véase también *rhs*.

Ejemplo:

```
(%i1) e: aa + bb = cc;
(%o1) bb + aa = cc
(%i2) lhs (e);
(%o2) bb + aa
(%i3) rhs (e);
(%o3) cc
(%i4) [lhs (aa < bb), lhs (aa <= bb), lhs (aa >= bb), lhs (aa > bb)];
(%o4) [aa, aa, aa, aa]
(%i5) [lhs (aa = bb), lhs (aa # bb), lhs (equal (aa, bb)), lhs (notequal (aa, bb))];
(%o5) [aa, aa, aa, aa]
(%i6) e1: '(foo(x) := 2*x);
(%o6) foo(x) := 2 x
(%i7) e2: '(bar(y) ::= 3*y);
(%o7) bar(y) ::= 3 y
(%i8) e3: '(x : y);
(%o8) x : y
(%i9) e4: '(x :: y);
(%o9) x :: y
(%i10) [lhs (e1), lhs (e2), lhs (e3), lhs (e4)];
(%o10) [foo(x), bar(y), x, x]
(%i11) infix (")["];
(%o11)][
(%i12) lhs (aa][bb);
(%o12) aa
```

**linsolve** ([*expr\_1*, ..., *expr\_m*], [*x\_1*, ..., *x\_n*]) Función

Resuelve la lista de ecuaciones lineales simultáneas para la lista de variables. Las expresiones deben ser polinomios lineales respecto de las variables o ecuaciones.

Si *globalsolve* vale *true*, a cada incógnita se le asigna el valor de la solución encontrada.

Si *backsubst* vale *false*, *linsolve* no hace la sustitución tras la triangularización de las ecuaciones. Esto puede ser necesario en problemas muy grandes en los que la sustitución puede dar lugar a la generación de expresiones enormes.

Si *linsolve\_params* vale *true*, *linsolve* también genera símbolos *%r* para representar parámetros arbitrarios como los descritos para la función *algsys*. Si vale *false*, el resultado devuelto por *linsolve* expresará, si es el sistema es indeterminado, unas variables en función de otras.

Si *programmode* vale *false*, *linsolve* muestra la solución con etiquetas de expresiones intermedias (*%t*) y devuelve la lista de etiquetas.

```
(%i1) e1: x + z = y;
(%o1) z + x = y
(%i2) e2: 2*a*x - y = 2*a^2;
```

```

 2
(%o2) 2 a x - y = 2 a
(%i3) e3: y - 2*z = 2;
(%o3) y - 2 z = 2
(%i4) [globalsolve: false, programmode: true];
(%o4) [false, true]
(%i5) linsolve ([e1, e2, e3], [x, y, z]);
(%o5) [x = a + 1, y = 2 a, z = a - 1]
(%i6) [globalsolve: false, programmode: false];
(%o6) [false, false]
(%i7) linsolve ([e1, e2, e3], [x, y, z]);
Solution

(%t7) z = a - 1

(%t8) y = 2 a

(%t9) x = a + 1
(%o9) [%t7, %t8, %t9]
(%i9) ' ';
(%o9) [z = a - 1, y = 2 a, x = a + 1]
(%i10) [globalsolve: true, programmode: false];
(%o10) [true, false]
(%i11) linsolve ([e1, e2, e3], [x, y, z]);
Solution

(%t11) z : a - 1

(%t12) y : 2 a

(%t13) x : a + 1
(%o13) [%t11, %t12, %t13]
(%i13) ' ';
(%o13) [z : a - 1, y : 2 a, x : a + 1]
(%i14) [x, y, z];
(%o14) [a + 1, 2 a, a - 1]
(%i15) [globalsolve: true, programmode: true];
(%o15) [true, true]
(%i16) linsolve ([e1, e2, e3], '[x, y, z]);
(%o16) [x : a + 1, y : 2 a, z : a - 1]
(%i17) [x, y, z];
(%o17) [a + 1, 2 a, a - 1]

```

**linsolvewarn**

Variable opcional

Valor por defecto: true

Si linsolvewarn vale true, linsolve mostrará el mensaje: "Dependent equations eliminated".

**linsolve\_params**

Variable opcional

Valor por defecto: `true`

Si `linsolve_params` vale `true`, `linsolve` también genera símbolos `%r` para representar parámetros arbitrarios como los descritos para la función `algsys`. Si vale `false`, el resultado devuelto por `linsolve` expresará, si es el sistema es indeterminado, unas variables en función de otras.

**multiplicities**

System variable

Valor por defecto: `not_set_yet`

La variable `multiplicities` es una con las multiplicidades de las soluciones encontradas por `solve` o `realroots`.

**nroots** (*p*, *low*, *high*)

Función

Devuelve el número de raíces reales del polinomio real univariante *p* en el intervalo semiabierto (*low*, *high*]. Los extremos del intervalo pueden ser `minf` o `inf`, menos y más infinito.

La función `nroots` utiliza el método de las secuencias de Sturm.

```
(%i1) p: x^10 - 2*x^4 + 1/2$
(%i2) nroots (p, -6, 9.1);
(%o2) 4
```

**nthroot** (*p*, *n*)

Función

Siendo *p* un polinomio de coeficientes enteros y *n* un entero positivo, `nthroot` devuelve un polinomio *q*, también de coeficientes enteros, tal que  $q^n = p$ , o un mensaje de error indicando que *p* no es una *n*-potencia exacta. Esta función es bastante más rápida que `factor` y que `sqfr`.

**programmode**

Variable opcional

Valor por defecto: `true`

Si `programmode` vale `true`, `solve`, `realroots`, `allroots` y `linsolve` devuelve sus soluciones como elementos de una lista.

Si `programmode` vale `false`, `solve` y las demás crean expresiones intermedias etiquetadas `%t1`, `t2`, etc., y les asinan las soluciones.

```
(%i1) solve(x^2+x+1);
(%o1) [x = - \frac{\sqrt{3} \%i + 1}{2}, x = \frac{\sqrt{3} \%i - 1}{2}]

(%i2) programmode:false$
(%i3) solve(x^2+x+1);
Solution:

(%t3) x = - \frac{\sqrt{3} \%i + 1}{2}

\frac{\sqrt{3} \%i - 1}{2}
```

```
(%t4) x = -----
 2
(%o4) [%t4, %t5]
```

**realonly**

Variable opcional

Valor por defecto: `false`

Si `realonly` vale `true`, `algsys` sólo devuelve aquellas soluciones exentas de la constante `%i`.

**realroots** (*poly*, *bound*)

Función

Encuentra todas las raíces reales del polinomio real univariante con la tolerancia especificada por `bound`, que si es menor que 1, hace que se busquen las raíces enteras de forma exacta. El argumento `bound` puede ser arbitrariamente pequeño a fin de conseguir la precisión deseada. El primer argumento también puede ser una ecuación. La función `realroots` asigna un valor a `multiplicities`, útil en caso de raíces múltiples. La llamada `realroots(p)` equivale a `realroots(p, rootsepsilon)`. La variable `rootsepsilon` es un número real que establece los intervalos de confianza para las raíces. Hágase `example(realroots)` para ver un ejemplo.

**rhs** (*expr*)

Función

Devuelve el miembro derecho (es decir, el segundo argumento) de la expresión *expr*, cuando el operador de *expr* es uno de los operadores de relación `<` `<=` `#` `equal` `notequal` `>` `>`, o un operadores de asignación `:=` `::=` `:` `::`, o un operador infijo binario definido por el usuario mediante `infix`.

Si *expr* es un átomo o si su operador es diferente de los citados más arriba, `rhs` devuelve *expr*.

Véase también `lhs`.

Ejemplo:

```
(%i1) e: aa + bb = cc;
(%o1) bb + aa = cc
(%i2) lhs (e);
(%o2) bb + aa
(%i3) rhs (e);
(%o3) cc
(%i4) [rhs (aa < bb), rhs (aa <= bb), rhs (aa >= bb), rhs (aa > bb)];
(%o4) [bb, bb, bb, bb]
(%i5) [rhs (aa = bb), rhs (aa # bb), rhs (equal (aa, bb)), rhs (notequal (aa, bb))];
(%o5) [bb, bb, bb, bb]
(%i6) e1: '(foo(x) := 2*x);
(%o6) foo(x) := 2 x
(%i7) e2: '(bar(y) ::= 3*y);
(%o7) bar(y) ::= 3 y
(%i8) e3: '(x : y);
(%o8) x : y
(%i9) e4: '(x :: y);
(%o9) x :: y
```

```
(%i10) [rhs (e1), rhs (e2), rhs (e3), rhs (e4)];
(%o10) [2 x, 3 y, y, y]
(%i11) infix ("["");
(%o11)][
(%i12) rhs (aa) [bb);
(%o12) bb
```

**rootsconmode**

Variable opcional

Valor por defecto: `true`

La variable `rootsconmode` controla el comportamiento de la instrucción `rootscontract`. Véase `rootscontract` para más detalles.

**rootscontract** (*expr*)

Función

Convierte productos de raíces en raíces de productos. Por ejemplo, `rootscontract (sqrt(x)*y^(3/2))` devuelve `sqrt(x*y^3)`.

Si `radexpand` vale `true` y `domain` vale `real`, `rootscontract` convierte `abs` en `sqrt`, por ejemplo, `rootscontract (abs(x)*sqrt(y))` devuelve `sqrt(x^2*y)`.

La opción `rootsconmode` afecta el resultado de `rootscontract` como sigue:

| Problema             | Valor de<br>rootsconmode | Resultado de<br>rootscontract |
|----------------------|--------------------------|-------------------------------|
| $x^{(1/2)}y^{(3/2)}$ | false                    | $(x*y^3)^{(1/2)}$             |
| $x^{(1/2)}y^{(1/4)}$ | false                    | $x^{(1/2)}y^{(1/4)}$          |
| $x^{(1/2)}y^{(1/4)}$ | true                     | $(x*y^{(1/2)})^{(1/2)}$       |
| $x^{(1/2)}y^{(1/3)}$ | true                     | $x^{(1/2)}y^{(1/3)}$          |
| $x^{(1/2)}y^{(1/4)}$ | all                      | $(x^2*y)^{(1/4)}$             |
| $x^{(1/2)}y^{(1/3)}$ | all                      | $(x^3*y^2)^{(1/6)}$           |

Si `rootsconmode` vale `false`, `rootscontract` contrae sólomente respecto de exponentes racionales cuyos denominadores sean iguales. La clave para los ejemplos `rootsconmode: true` es simplemente que 2 divide a 4 pero no a 3. La asignación `rootsconmode: all` hace que se calcule el mínimo común múltiplo de los denominadores de los exponentes.

La función `rootscontract` utiliza `ratsimp` de forma similar a como lo hace `logcontract`.

Ejemplos:

```
(%i1) rootsconmode: false$
(%i2) rootscontract (x^(1/2)*y^(3/2));
 3
(%o2) sqrt(x y)
(%i3) rootscontract (x^(1/2)*y^(1/4));
 1/4
(%o3) sqrt(x) y
(%i4) rootsconmode: true$
(%i5) rootscontract (x^(1/2)*y^(1/4));
 1/4
(%o5) sqrt(x sqrt(y))
(%i6) rootscontract (x^(1/2)*y^(1/3));
```

```

 1/3
(%o6) sqrt(x) y
(%i7) rootsconmode: all$
(%i8) rootscontract (x^(1/2)*y^(1/4));
 2 1/4
(%o8) (x y)
(%i9) rootscontract (x^(1/2)*y^(1/3));
 3 2 1/6
(%o9) (x y)
(%i10) rootsconmode: false$
(%i11) rootscontract (sqrt(sqrt(x) + sqrt(1 + x))
 *sqrt(sqrt(1 + x) - sqrt(x)));
(%o11) 1
(%i12) rootsconmode: true$
(%i13) rootscontract (sqrt(5 + sqrt(5)) - 5^(1/4)*sqrt(1 + sqrt(5)));
(%o13) 0

```

**rootsepsilon**

Variable opcional

Valor por defecto: 1.0e-7

La variable **rootsepsilon** es la tolerancia que establece el intervalo de confianza para las raíces calculadas por la función **realroots**.

**solve** (*expr*, *x*)

Función

**solve** (*expr*)

Función

**solve** ([*eqn\_1*, ..., *eqn\_n*], [*x\_1*, ..., *x\_n*])

Función

Resuelve la ecuación algebraica *expr* de incógnita *x* y devuelve una lista de igualdades con la *x* despejada. Si *expr* no es una igualdad, se supone que se quiere resolver la ecuación *expr* = 0. El argumento *x* puede ser una función (por ejemplo, **f(x)**), u otra expresión no atómica, excepto una suma o producto. Puede omitirse *x* si *expr* contiene solamente una variable. El argumento *expr* puede ser una expresión racional y puede contener funciones trigonométricas, exponenciales, etc.

Se utiliza el siguiente método de resolución:

Sea *E* la expresión y *X* la incógnita. Si *E* es lineal respecto de *X* entonces *X* se resuelve de forma trivial. En caso contrario, si *E* es de la forma **A**\**X*<sup>**N**</sup> + **B** entonces el resultado es (-**B**/**A**)<sup>1/**N**</sup> multiplicado por las **N**-ésimas raíces de la unidad.

Si *E* no es lineal respecto de *X* entonces el máximo común divisor de los exponentes de *X* en *E* (supóngase que es **N**) se divide entre los exponentes y la multiplicidad de las raíces se multiplica por **N**. Entonces es llamado recursivamente **solve** para este resultado. Si *E* es factorizable entonces **solve** es invocado para cada uno de los factores. Finalmente, **solve** usará, según sea necesario, las fórmulas cuadrática, cúbica o cuártica.

En caso de que *E* sea un polinomio respecto de una función de la incógnita, por ejemplo **F(X)**, entonces se calcula primero para **F(X)** (sea **C** el resultado obtenido), entonces la ecuación **F(X)=C** se resuelve para *X* en el supuesto que se conozca la inversa de la función *F*.

Si la variable **breakup** vale **false** hará que **solve** muestre las soluciones de las ecuaciones cúbicas o cuárticas como expresiones únicas, en lugar de utilizar varias subexpresiones comunes, que es el formato por defecto.

A la variable **multiplicities** se le asignará una lista con las multiplicidades de las soluciones individuales devueltas por **solve**, **realroots** o **allroots**. La instrucción **apropos (solve)** hará que se muestren las variables optativas que de algún modo afectan al comportamiento de **solve**. Se podrá luego utilizar la función **describe** para aquellas variables cuyo objeto no esté claro.

La llamada **solve ([eqn\_1, ..., eqn\_n], [x\_1, ..., x\_n])** resuelve un sistema de ecuaciones polinómicas simultáneas (lineales o no) llamando a **linsolve** o **algsys** y devuelve una lista de listas con soluciones para las incógnitas. En caso de haberse llamado a **linsolve** esta lista contendrá una única lista de soluciones. La llamada a **solve** tiene dos listas como argumentos. La primera lista tiene las ecuaciones a resolver y la segunda son las incógnitas cuyos valores se quieren calcular. Si el número de variables en las ecuaciones es igual al número de incógnitas, el segundo argumento puede omitirse. En caso de sistemas lineales incompatibles se mostrará el mensaje **inconsistent** (véase la variable **solve\_inconsistent\_error**); si no existe una única solución, se mostrará el mensaje **singular**.

Si **programmode** vale **false**, **solve** muestra la solución con etiquetas de expresiones intermedias (%t) y devuelve la lista de etiquetas.

Si **globalsolve** vale **true** y el problema consiste en resolver un sistema de dos o más ecuaciones lineales, a cada incógnita se le asigna el valor encontrado en la resolución del sistema.

Ejemplos:

```
(%i1) solve (asin (cos (3*x))*(f(x) - 1), x);

SOLVE is using arc-trig functions to get a solution.
Some solutions will be lost.
 %pi
(%o1) [x = ---, f(x) = 1]
 6
(%i2) ev (solve (5^f(x) = 125, f(x)), solveradcan);
 log(125)
(%o2) [f(x) = -----]
 log(5)
(%i3) [4*x^2 - y^2 = 12, x*y - x = 2];
 2 2
(%o3) [4 x - y = 12, x y - x = 2]
(%i4) solve (%, [x, y]);
(%o4) [[x = 2, y = 2], [x = .5202594388652008 %i
- .1331240357358706, y = .0767837852378778
- 3.608003221870287 %i], [x = - .5202594388652008 %i
- .1331240357358706, y = 3.608003221870287 %i]
```

```

+ .0767837852378778], [x = - 1.733751846381093,
y = - .1535675710019696]]
(%i5) solve (1 + a*x + x^3, x);
(%o5) [x = (- (sqrt(3) %i - 1) (sqrt(4 a^3 + 27) - 1)^(1/3)
- (sqrt(3) %i - 1) a) / (sqrt(4 a^3 + 27) - 1)^(1/3), x =
(sqrt(3) %i - 1) a / (sqrt(4 a^3 + 27) - 1)^(1/3), x =
(sqrt(3) %i - 1) a / (sqrt(4 a^3 + 27) - 1)^(1/3)]
(%i6) solve (x^3 - 1);
(%o6) [x = (sqrt(3) %i - 1)^(1/3), x = (sqrt(3) %i + 1)^(1/3), x = 1]
(%i7) solve (x^6 - 1);
(%o7) [x = (sqrt(3) %i + 1)^(1/6), x = (sqrt(3) %i - 1)^(1/6), x = -1,
x = (sqrt(3) %i + 1)^(1/6), x = (sqrt(3) %i - 1)^(1/6)]

```



```

 x = - ----, x = - ----, x = 1]
 2 2
(%i8) ev (x^6 - 1, %[1]);
 6
 (sqrt(3) %i + 1)
(%o8) ----- - 1
 64
(%i9) expand (%);
(%o9) 0
(%i10) x^2 - 1;
 2
 x - 1
(%o10)
(%i11) solve (%, x);
(%o11) [x = - 1, x = 1]
(%i12) ev (%th(2), %[1]);
(%o12) 0

```

**solvedecomposes**

Variable opcional

Valor por defecto: `true`

Si `solvedecomposes` vale `true`, `solve` llama a `polydecomp` en caso de que se le pida resolver ecuaciones polinómicas.

**solveexplicit**

Variable opcional

Valor por defecto: `false`

Si `solveexplicit` vale `true`, le inhibe a `solve` devolver soluciones implícitas, esto es, soluciones de la forma  $F(x) = 0$ , donde  $F$  es cierta función.

**solvefactors**

Variable opcional

Valor por defecto: `true`

Si `solvefactors` vale `false`, `solve` no intenta factorizar la expresión. Este valor `false` puede ser útil en algunos casos en los que la factorización no es necesaria.

**solvenullwarn**

Variable opcional

Valor por defecto: `true`

Si `solvenullwarn` vale `true`, `solve` muestra un mensaje de aviso si es llamado con una lista de ecuaciones vacía o con una lista de incógnitas vacía. Por ejemplo, `solve ([], [])` imprimirá dos mensajes de aviso y devolverá `[]`.

**solveradcan**

Variable opcional

Valor por defecto: `false`

Si `solveradcan` vale `true`, `solve` llama a `radcan`, lo que hará que `solve` se ejecute de forma más lenta, pero permitirá que se resuelvan ciertas ecuaciones que contengan exponenciales y logaritmos.

**solvetrigwarn**

Variable opcional

Valor por defecto: `true`

Si `solvetrigwarn` vale `true`, `solve` puede presentar un mensaje diciendo que está utilizando funciones trigonométricas inversas para resolver la ecuación, y que por lo tanto puede estar ignorando algunas soluciones.

**solve\_inconsistent\_error**

Variable opcional

Valor por defecto: `true`

Si `solve_inconsistent_error` vale `true`, `solve` y `linsolve` devuelven un error si las ecuaciones a resolver son incompatibles. Si vale `false`, `solve` y `linsolve` devuelven la lista vacía `[]` en caso de incompatibilidad.

Ejemplo:

```
(%i1) solve_inconsistent_error: true$
(%i2) solve ([a + b = 1, a + b = 2], [a, b]);
Inconsistent equations: (2)
-- an error. Quitting. To debug this try debugmode(true);
(%i3) solve_inconsistent_error: false$
(%i4) solve ([a + b = 1, a + b = 2], [a, b]);
(%o4) []
```



## 22 Ecuaciones Diferenciales

### 22.1 Definiciones para Ecuaciones Diferenciales.

**bc2** (*soluc*, *xval1*, *yval1*, *xval2*, *yval2*)

Función

Resuelve el problema del valor en la frontera para ecuaciones diferenciales de segundo orden. Aquí, *soluc* es una solución general de la ecuación, como las que calcula `ode2`, *xval1* es una ecuación de la forma  $x = x_0$  para la variable independiente y *yval1* es una ecuación de la forma  $y = y_0$  para la variable dependiente. Los argumentos *xval2* y *yval2* son ecuaciones a verificar por estas mismas variables en otro punto. Véase `ode2` para un ejemplo sobre su utilización.

**dsolve** (*ecu*, *x*)

Función

**dsolve** (*eqn\_1*, ..., *eqn\_n*), [*x\_1*, ..., *x\_n*])

Función

La función **dsolve** resuelve sistemas de ecuaciones diferenciales ordinarias lineales utilizando la transformada de Laplace. Aquí las *eqi* ( $i=1,\dots,n$ ) son ecuaciones diferenciales con variables dependientes  $x_1, \dots, x_n$ . Las relaciones funcionales deben indicarse explícitamente tanto para las ecuaciones como para las variables. Por ejemplo,

```
'diff(f,x,2)=sin(x)+'diff(g,x);
'diff(f,x)+x^2-f=2*'diff(g,x,2);
```

no es el formato apropiado. El método correcto es

```
'diff(f(x),x,2)=sin(x)+'diff(g(x),x);
'diff(f(x),x)+x^2-f=2*'diff(g(x),x,2);
```

La llamada a la función debe ser entonces `dsolve(['%3,%4'],[f(x),g(x)]);` .

Si las condiciones iniciales en 0 son conocidas, deben ser suministradas antes de llamar a **dsolve** haciendo uso previo de la función **atvalue**,

```
(%i1) 'diff(f(x),x)='diff(g(x),x)+sin(x);
 d d
(%o1) -- (f(x)) = -- (g(x)) + sin(x)
 dx dx
(%i2) 'diff(g(x),x,2)='diff(f(x),x)-cos(x);
 2
 d d
(%o2) --- (g(x)) = -- (f(x)) - cos(x)
 2 dx
 dx
(%i3) atvalue('diff(g(x),x),x=0,a);
(%o3) a
(%i4) atvalue(f(x),x=0,1);
(%o4) 1
(%i5) dsolve(['%1,%2'],[f(x),g(x)]);
 x
(%o5) [f(x) = a %e - a + 1, g(x) =
```

x

```

 cos(x) + a %ex - a + g(0) - 1]
(%i6) [%o1,%o2],%o5,diff;
 x x x x
(%o6) [a %ex = a %ex , a %ex - cos(x) = a %ex - cos(x)]

```

Si `desolve` no encuentra una solución, entonces devuelve `false`.

**ic1** (*soluc, xval, yval*) Función

Resuelve el problema del valor inicial en ecuaciones diferenciales de primer orden. Aquí, *soluc* es una solución general de la ecuación, como las que calcula `ode2`, *xval* es una ecuación de la forma  $x = x0$  para la variable independiente y *yval* es una ecuación de la forma  $y = y0$  para la variable dependiente. Véase `ode2` para un ejemplo sobre su utilización.

**ic2** (*soluc, xval, yval, dval*) Función

Resuelve el problema del valor inicial en ecuaciones diferenciales de segundo orden. Aquí, *soluc* es una solución general de la ecuación, como las que calcula `ode2`, *xval* es una ecuación de la forma  $x = x0$  para la variable independiente y *yval* es una ecuación de la forma  $y = y0$  para la variable dependiente, siendo *dval* una ecuación para la derivada de la variable dependiente respecto de la independiente en el punto *xval*. Véase `ode2` para un ejemplo sobre su utilización.

**ode2** (*ecu, dvar, ivar*) Función

La función `ode2` resuelve ecuaciones diferenciales ordinarias de primer y segundo orden. Admite tres argumentos: una ecuación diferencial ordinaria *ecu*, la variable dependiente *dvar* y la variable independiente *ivar*. Si ha tenido éxito en la resolución de la ecuación, devuelve una solución, explícita o implícita, para la variable dependiente. El símbolo `%c` se utiliza para representar la constante en el caso de ecuaciones de primer orden y los símbolos `%k1` y `%k2` son las constantes de las ecuaciones de segundo orden. Si por cualquier razón `ode2` no puede calcular la solución, devolverá `false`, acompañado quizás de un mensaje de error. Los métodos utilizados para las ecuaciones de primer orden, en el orden en que se hace la tentativa de resolución son: lineal, separable, exacto (pudiendo solicitar en este caso un factor de integración), homogéneo, ecuación de Bernoulli y un método homogéneo generalizado. Para las ecuaciones de segundo orden: coeficiente constante, exacto, homogéneo lineal con coeficientes no constantes que pueden ser transformados en coeficientes constantes, ecuación equidimensional o de Euler, método de variación de parámetros y ecuaciones exentas de las variables dependientes o independientes de manera que se puedan reducir a dos ecuaciones lineales de primer a ser resueltas secuencialmente. Durante el proceso de resolución de ecuaciones diferenciales ordinarias, ciertas variables se utilizan con el único propósito de suministrar información al usuario: `method` almacena el método utilizado para encontrar la solución (como por ejemplo `linear`), `intfactor` para el factor de integración que se haya podido utilizar, `odeindex` para el índice del método de Bernoulli o el homogéneo generalizado y `yp` para la solución particular del método de variación de parámetros.

A fin de resolver problemas con valores iniciales y problemas con valores en la frontera, la función `ic1` está disponible para ecuaciones de primer orden y las funciones `ic2` y

bc2 para problemas de valores iniciales y de frontera, respectivamente, en el caso de las ecuaciones de segundo orden.

Ejemplo:

```
(%i1) x^2*'diff(y,x) + 3*y*x = sin(x)/x;
```

$$x^2 \frac{dy}{dx} + 3xy = \frac{\sin(x)}{x}$$

```
(%o1)
```

```
(%i2) ode2(%,y,x);
```

$$y = \frac{\%c - \cos(x)}{3x}$$

```
(%o2)
```

```
(%i3) ic1(%o2,x=%pi,y=0);
```

$$y = -\frac{\cos(x) + 1}{3x}$$

```
(%o3)
```

```
(%i4) 'diff(y,x,2) + y*'diff(y,x)^3 = 0;
```

$$\frac{d^2y}{dx^2} + y \left(\frac{dy}{dx}\right)^3 = 0$$

```
(%o4)
```

```
(%i5) ode2(%,y,x);
```

$$\frac{y^3 + 6\%k1 y}{6} = x + \%k2$$

```
(%o5)
```

```
(%i6) ratsimp(ic2(%o5,x=0,y=0,'diff(y,x)=2));
```

$$-\frac{2y^2 - 3y}{6} = x$$

```
(%o6)
```

```
(%i7) bc2(%o5,x=0,y=1,x=1,y=3);
```

$$\frac{y^3 - 10y}{6} = x - \frac{3}{2}$$

```
(%o7)
```



## 23 Métodos numéricos

### 23.1 Introducción a los métodos numéricos

### 23.2 Series de Fourier

El paquete `fft` contiene funciones para el cálculo numérico (no simbólico) de la transformada rápida de Fourier. La instrucción `load ("fft")` carga el paquete. Véase `fft`.

El paquete `fourie` contiene funciones para el cálculo simbólico de series de Fourier. La instrucción `load ("fourie")` carga el paquete. Hay funciones en el paquete `fourie` para calcular los coeficientes de Fourier y para la transformación de expresiones. Véase Definiciones para las series de Fourier.

### 23.3 Definiciones para los métodos numéricos

**polartorect** (*magnitude\_array*, *phase\_array*) Función

Transforma valores complejos de la forma  $r e^{i t}$  a la forma  $a + b i$ . La instrucción `load ("fft")` carga esta función en Maxima. Véase también `fft`.

El módulo y la fase,  $r$  y  $t$ , se toman de *magnitude\_array* y *phase\_array*, respectivamente. Los valores originales de los arreglos de entrada se reemplazan por las partes real e imaginaria,  $a$  y  $b$ . Los resultados se calculan como

```
a: r cos (t)
b: r sin (t)
```

Los arreglos de entrada deben ser de igual tamaño y de dimensión 1. El tamaño del arreglo no necesita ser potencia de 2.

La función `polartorect` es la inversa de `recttopolar`.

**recttopolar** (*real\_array*, *imaginary\_array*) Función

Transforma valores complejos de la forma  $a + b i$  a la forma  $r e^{i t}$ . La instrucción `load ("fft")` carga esta función en Maxima. Véase también `fft`.

Las partes real e imaginaria,  $a$  y  $b$ , se toman de *real\_array* y *imaginary\_array*, respectivamente. Los valores originales de los arreglos de entrada se reemplazan por el módulo y el argumento,  $r$  y  $t$ . Los resultados se calculan como

```
r: sqrt (a^2 + b^2)
t: atan2 (b, a)
```

El ángulo calculado está en el rango entre  $-\pi$  y  $\pi$ .

Los arreglos de entrada deben ser de igual tamaño y de dimensión 1. El tamaño del arreglo no necesita ser potencia de 2.

La función `recttopolar` es la inversa de `polartorect`.

**ift** (*real\_array*, *imaginary\_array*) Función

Transformada discreta inversa de Fourier. La instrucción `load ("fft")` carga esta función en Maxima.



La función **ift** calcula la transformada rápida inversa compleja de Fourier de arreglos unidimensionales de números decimales en coma flotante. La transformada inversa se define como

$$x[j]: \text{sum } (y[j] \exp (+2 \%i \%pi \ j \ k / n), k, 0, n-1)$$

Véase **fft** para más detalles.

|                                                                    |         |
|--------------------------------------------------------------------|---------|
| <b>fft</b> ( <i>real_array</i> , <i>imaginary_array</i> )          | Función |
| <b>ift</b> ( <i>real_array</i> , <i>imaginary_array</i> )          | Función |
| <b>recttopolar</b> ( <i>real_array</i> , <i>imaginary_array</i> )  | Función |
| <b>polartorect</b> ( <i>magnitude_array</i> , <i>phase_array</i> ) | Función |

Transformada rápida de Fourier y funciones asociadas. La instrucción **load** ("**fft**") carga estas funciones en Maxima.

Las funciones **fft** y **ift** calculan la transformada rápida compleja de Fourier y su inversa de arreglos unidimensionales de números decimales en coma flotante. El tamaño de *imaginary\_array* debe ser igual al de *real\_array*.

Los resultados devueltos por las funciones **fft** y **ift** sustituyen al contenido original de los arreglos de entrada. La función **fillarray** puede hacer una copia de un array en caso de ser necesario.

La transformada discreta de Fourier y la transformada inversa se definen como sigue. Sea **x** el arreglo con los datos originales, con

$$x[i]: \text{real\_array}[i] + \%i \text{ imaginary\_array}[i]$$

Sea **y** el arreglo con los datos transformados. Las transformadas directa e inversa son

$$y[k]: (1/n) \text{sum } (x[j] \exp (-2 \%i \%pi \ j \ k / n), j, 0, n-1)$$

$$x[j]: \text{sum } (y[j] \exp (+2 \%i \%pi \ j \ k / n), k, 0, n-1)$$

Arreglos apropiados se pueden preparar con la función **array**. Por ejemplo:

```
array (my_array, float, n-1)$
```

declara un arreglo unidimensional con **n** elementos, indexado de 0 a **n-1**, ambos inclusive. El número de elementos, **n**, debe ser par.

La función **fft** se puede aplicar a datos reales (con partes imaginarias nulas) para obtener coeficientes seno y coseno. Después de llamar a la función **fft**, los coeficientes del seno y coseno, llamémosles **a** y **b**, se pueden calcular como

```
a[0]: real_array[0]
b[0]: 0
```

y

```
a[j]: real_array[j] + real_array[n-j]
b[j]: imaginary_array[j] - imaginary_array[n-j]
```

para **j** tomando valores de 1 a **n/2-1**, y

```
a[n/2]: real_array[n/2]
b[n/2]: 0
```

La función **recttopolar** transforma valores complejos de la forma **a + b %i** a la forma **r %e^(%i t)**. Véase **recttopolar**.

La función **polartorect** transforma valores complejos de la forma **r %e^(%i t)** a la forma **a + b %i**. Véase **polartorect**.



```
(%i8) fortran (f);
 f
(%o8) done
```

**fortspaces**

Variable opcional

Valor por defecto: `false`Si `fortspaces` vale `true`, `fortran` rellena las líneas con espacios de 80 columnas.**horner** (*expr*, *x*)

Función

**horner** (*expr*)

Función

Cambia el formato de *expr* según la regla de Horner utilizando *x* como variable principal, si ésta se especifica. El argumento *x* se puede omitir, en cuyo caso se considerará como variable principal la de *expr* en su formato racional canónico (CRE).

La función `horner` puede mejorar las estabilidad si *expr* va a ser numéricamente evaluada. También es útil si Maxima se utiliza para generar programas que serán ejecutados en Fortran. Véase también `stringout`.

```
(%i1) expr: 1e-155*x^2 - 5.5*x + 5.2e155;
 2
(%o1) 1.0E-155 x - 5.5 x + 5.2E+155
(%i2) expr2: horner (% , x), keepfloat: true;
(%o2) (1.0E-155 x - 5.5) x + 5.2E+155
(%i3) ev (expr, x=1e155);
Maxima encountered a Lisp error:

floating point overflow

Automatically continuing.
To reenable the Lisp debugger set *debugger-hook* to nil.
(%i4) ev (expr2, x=1e155);
(%o4) 7.0E+154
```

**find\_root** (*f(x)*, *x*, *a*, *b*)

Función

**find\_root** (*f*, *a*, *b*)

Función

Calcula el cero de la función *f* cuando *x* varía en el rango [*a*, *b*]. La función debe tener signos diferentes en los extremos del intervalo. Si esta condición no se cumple, la acción de esta función la gobierna la variable `find_root_error`. Si `find_root_error` vale `true` entonces se emite un mensaje de error, en otro caso se devuelve el valor de `find_root_error`. En caso contrario, dado que Maxima puede evaluar el primer argumento en el rango especificado, el cual es continuo, está garantizado que `find_root` encontrará el cero buscado (o uno de ellos en caso de haber varios). La precisión de `find_root` está gobernada por `find_root_abs` y `find_root_rel`, que deben ser números no negativos en coma flotante. La función `find_root` se detendrá cuando el primer argumento tome un valor menor o igual que `find_root_abs`, o cuando sucesivas aproximaciones de la raíz se diferencien en menos de `find_root_rel * <one of the approximants>`. El valor por defecto de `find_root_abs` y `find_root_rel` es 0.0 en ambos casos, de forma que `find_root` obtiene la mejor aproximación posible

para la aritmética de simple precisión de que se dispone. El primer argumento puede ser una ecuación. La ordenación de los dos últimos argumentos es irrelevante. Así,

```
find_root (sin(x) = x/2, x, %pi, 0.1);
```

equivale a

```
find_root (sin(x) = x/2, x, 0.1, %pi);
```

El método que se aplica es una búsqueda binaria en el rango especificado por los dos últimos argumentos. Cuando el algoritmo considera que la función está lo suficientemente cerca de ser lineal, comienza a utilizar interpolación lineal.

Ejemplos:

```
(%i1) f(x) := sin(x) - x/2;
(%o1)
x
f(x) := sin(x) - -
2
(%i2) find_root (sin(x) - x/2, x, 0.1, %pi);
(%o2) 1.895494267033981
(%i3) find_root (sin(x) = x/2, x, 0.1, %pi);
(%o3) 1.895494267033981
(%i4) find_root (f(x), x, 0.1, %pi);
(%o4) 1.895494267033981
(%i5) find_root (f, 0.1, %pi);
(%o5) 1.895494267033981
```

### **find\_root\_abs**

Variable opcional

Valor por defecto: 0.0

La variable `find_root_abs` es la precisión de la instrucción `find_root`, cuyo comportamiento lo controlan `find_root_abs` y `find_root_rel`, variables que deben guardar dos números no negativos decimales en coma flotante. La función `find_root` parará cuando su primer argumento tome un valor menor o igual que `find_root_abs` o si sucesivas aproximaciones a la raíz difieren entre sí en no más de `find_root_rel * <one of the approximants>`. El valor por defecto de `find_root_abs` y `find_root_rel` es 0.0 en ambos casos, de forma que `find_root` obtiene la mejor aproximación posible para la aritmética de simple precisión de que se dispone.

### **find\_root\_error**

Variable opcional

Valor por defecto: `true`

La variable `find_root_error` controla el comportamiento de `find_root`. Cuando se llama a la función `find_root`, ésta determina si la función a ser resuelta satisface la condición de que sus valores en los extremos de los intervalos tengan signos diferentes. Si son de signo opuesto, se procede a la interpolación. Si son del mismo signo, y `find_root_error` vale `true`, entonces se señala un error. Si son del mismo signo y `find_root_error` no vale `true`, retornará el valor de `find_root_error`.

### **find\_root\_rel**

Variable opcional

Valor por defecto: 0.0

La variable `find_root_rel` es la precisión de la instrucción `find_root`, cuyo comportamiento lo controlan `find_root_abs` y `find_root_rel`, variables que deben guardar

dos números no negativos decimales en coma flotante. La función `find_root` parará cuando su primer argumento tome un valor menor o igual que `find_root_abs` o si sucesivas aproximaciones a la raíz difieren entre sí en no más de `find_root_rel * <one of the approximants>`. El valor por defecto de `find_root_abs` y `find_root_rel` es 0.0 en ambos casos, de forma que `find_root` obtiene la mejor aproximación posible para la aritmética de simple precisión de que se dispone.

## 23.4 Definiciones para las series de Fourier

**equalp** (*x*, *y*) Función

Devuelve `true` si `equal` (*x*, *y*), en otro caso devuelve `false`. No devuelve el mensaje de error que se obtiene de `equal` (*x*, *y*) en un caso como éste.

**remfun** (*f*, *expr*) Función

**remfun** (*f*, *expr*, *x*) Función

La llamada `remfun` (*f*, *expr*) reemplaza todas las subexpresiones *f* (*arg*) por *arg* en *expr*.

La llamada `remfun` (*f*, *expr*, *x*) reemplaza todas las subexpresiones *f* (*arg*) por *arg* en *expr* sólo si *arg* contiene a la variable *x*.

**funp** (*f*, *expr*) Función

**funp** (*f*, *expr*, *x*) Función

La llamada `funp` (*f*, *expr*) devuelve `true` si *expr* contiene la función *f*.

La llamada `funp` (*f*, *expr*, *x*) devuelve `true` si *expr* contiene la función *f* y la variable *x* está presente en el argumento de alguna de las presencias de *f*.

**absint** (*f*, *x*, *halfplane*) Función

**absint** (*f*, *x*) Función

**absint** (*f*, *x*, *a*, *b*) Función

La llamada `absint` (*f*, *x*, *halfplane*) devuelve la integral indefinida de *f* con respecto a *x* en el semiplano dado (*pos*, *neg* o *both*). La función *f* puede contener expresiones de la forma `abs (x)`, `abs (sin (x))`, `abs (a) * exp (-abs (b) * abs (x))`.

La llamada `absint` (*f*, *x*) equivale a `absint` (*f*, *x*, *pos*).

La llamada `absint` (*f*, *x*, *a*, *b*) devuelve la integral definida de *f* con respecto a *x* de *a* a *b*.

**fourier** (*f*, *x*, *p*) Función

Devuelve una lista con los coeficientes de Fourier de *f*(*x*) definida en el intervalo `[-%pi, %pi]`.

**foursimp** (*l*) Función

Simplifica `sin (n %pi)` a 0 si `sinnpiflag` vale `true` y `cos (n %pi)` a `(-1)^n` si `cosnpiflag` vale `true`.

|                                                                                                                                                                                                                                                                             |                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| <b>sinnpiflag</b>                                                                                                                                                                                                                                                           | Variable opcional |
| Valor por defecto: <code>true</code>                                                                                                                                                                                                                                        |                   |
| Véase <code>foursimp</code> .                                                                                                                                                                                                                                               |                   |
| <b>cosnpiflag</b>                                                                                                                                                                                                                                                           | Variable opcional |
| Valor por defecto: <code>true</code>                                                                                                                                                                                                                                        |                   |
| Véase <code>foursimp</code> .                                                                                                                                                                                                                                               |                   |
| <b>fourexpend</b> ( <i>l</i> , <i>x</i> , <i>p</i> , <i>limit</i> )                                                                                                                                                                                                         | Función           |
| Calcula y devuelve la serie de Fourier a partir de la lista de los coeficientes de Fourier <i>l</i> hasta el término <i>limit</i> ( <i>limit</i> puede ser <code>inf</code> ). Los argumentos <i>x</i> y <i>p</i> tienen el mismo significado que en <code>fourier</code> . |                   |
| <b>fourcos</b> ( <i>f</i> , <i>x</i> , <i>p</i> )                                                                                                                                                                                                                           | Función           |
| Devuelve los coeficientes de los cosenos de Fourier de $f(x)$ definida en $[0, \pi]$ .                                                                                                                                                                                      |                   |
| <b>foursin</b> ( <i>f</i> , <i>x</i> , <i>p</i> )                                                                                                                                                                                                                           | Función           |
| Devuelve los coeficientes de los senos de Fourier de $f(x)$ definida en $[0, \pi]$ .                                                                                                                                                                                        |                   |
| <b>totalfourier</b> ( <i>f</i> , <i>x</i> , <i>p</i> )                                                                                                                                                                                                                      | Función           |
| Devuelve <code>fourexpend(foursimp(fourier(f, x, p)), x, p, 'inf')</code> .                                                                                                                                                                                                 |                   |
| <b>fourint</b> ( <i>f</i> , <i>x</i> )                                                                                                                                                                                                                                      | Función           |
| Calcula y devuelve la lista de los coeficientes integrales de Fourier de $f(x)$ definida en $[\min, \max]$ .                                                                                                                                                                |                   |
| <b>fourintcos</b> ( <i>f</i> , <i>x</i> )                                                                                                                                                                                                                                   | Función           |
| Devuelve los coeficientes integrales de los cosenos $f(x)$ en $[0, \max]$ .                                                                                                                                                                                                 |                   |
| <b>fourintsin</b> ( <i>f</i> , <i>x</i> )                                                                                                                                                                                                                                   | Función           |
| Devuelve los coeficientes integrales de los senos $f(x)$ en $[0, \max]$ .                                                                                                                                                                                                   |                   |



## 24 Estadística

### 24.1 Definiciones para Estadística

**gauss** ( $n, s$ )

Función

Devuelve un número decimal pseudoaleatorio en coma flotante con distribución normal de media  $n$  y desviación típica  $s$ .





## 25 Tablas y Arreglos

### 25.1 Definiciones para Arreglos y Tablas

**array** (*nombre*, *dim\_1*, ..., *dim\_n*) Función  
**array** (*nombre*, *type*, *dim\_1*, ..., *dim\_n*) Función  
**array** (*[nombre\_1, ..., nombre\_m]*, *dim\_1*, ..., *dim\_n*) Función

Crea un arreglo de dimensión  $n$ , que debe ser menor o igual que 5. Los subíndices de la  $i$ -ésima dimensión son enteros que toman valores entre 0 y  $dim_i$ .

La llamada **array** (*nombre*, *dim\_1*, ..., *dim\_n*) crea un arreglo de tipo general.

La llamada **array** (*nombre*, *type*, *dim\_1*, ..., *dim\_n*) crea un arreglo con sus elementos del tipo especificado. El tipo *type* puede ser **fixnum** para enteros de tamaño limitado o **flonum** para números decimales en coma flotante.

La llamada **array** (*[nombre\_1, ..., nombre\_m]*, *dim\_1*, ..., *dim\_n*) crea  $m$  arreglos, todos ellos de igual dimensión.

Si el usuario asigna un valor a una variable subindicada antes de declarar el arreglo correspondiente, entonces se construye un arreglo no declarado. Los arreglos no declarados, también conocidos por el nombre de "arreglos de claves" (hashed arrays), son más generales que los arreglos declarados. El usuario no necesita declarar su tamaño máximo y pueden ir creciendo de forma dinámica. Los subíndices de los arreglos no declarados no necesitan ser necesariamente números. Sin embargo, a menos que un arreglo tenga sus elementos dispersos, probablemente sea más eficiente declararlo siempre que sea posible antes que dejarlo como no declarado. La función **array** puede utilizarse para transformar un arreglo no declarado a uno declarado.

**arrayapply** (*A*, [*i\_1*, ..., *i\_n*]) Función  
 Evalúa  $A[i_1, \dots, i_n]$ , donde  $A$  es un arreglo y  $i_1, \dots, i_n$  son enteros.

Esto es como **apply**, excepto por el hecho de que el primer argumento es un arreglo en lugar de una función.

**arrayinfo** (*A*) Función  
 Devuelve una lista con información sobre el arreglo  $A$ . Para arreglos de claves (hashed arrays) devuelve una lista **hashed**, el número de subíndices y los subíndices de aquellos elementos que tengan valor. Para arreglos declarados devuelve una lista **declared**, el número de subíndices y los valores dados a la función **array** cuando fue invocada para crear  $A$ . Hágase **example(arrayinfo)**; para ver un ejemplo.

**arraymake** (*name*, [*i\_1*, ..., *i\_n*]) Función  
 Devuelve la expresión  $name[i_1, \dots, i_n]$ .

Esta función es similar a **funmake**, excepto que el valor retornado es referencia a un arreglo no evaluado, en lugar de una llamada a una función no evaluada.

**arrays** Variable del sistema  
 Valor por defecto: []

La variable `arrays` es una lista con todos los arreglos que han sido alojados, hayan sido declarados o no.

Véanse también `array`, `arrayapply`, `arrayinfo`, `arraymake`, `fillarray`, `listarray` y `rearray`.

### **bashindices** (*expr*) Función

Transforma la expresión *expr* dándole a cada suma y producto un único índice. Esto le da a `changevar` mayor precisión cuando opera con sumas y productos. La forma del único índice es *jnumber*. La cantidad *number* se determina en función de `gensumnum`, valor que puede cambiar el usuario. Por ejemplo, haciendo `gensumnum:0$`.

### **fillarray** (*A*, *B*) Función

Rellena el arreglo *A* con los valores de *B*, que puede ser una lista o arreglo.

Si *A* es un array de números decimales en coma flotante (enteros) entonces *B* debe ser o bien una lista de números decimales en coma flotante (enteros), o bien otro arreglo de números en coma flotante (enteros).

Si las dimensiones de los arreglos son diferentes, *A* se rellena según el orden de las filas. Si no hay suficientes elementos en *B* el último elemento se utiliza para cubrir el resto de *A*. Si hay demasiados, los elementos sobrantes son ignorados.

La función `fillarray` devuelve su primer argumento.

### **listarray** (*A*) Función

Devuelve una lista con los elementos del arreglo declarado o tabla de claves *A*, en el orden de las filas. Los elementos que todavía no han sido declarados se representan mediante `#####`.

### **make\_array** (*tipo*, *dim\_1*, ..., *dim\_n*) Función

Construye y devuelve un arreglo de Lisp. El argumento *tipo* puede ser `any`, `flonum`, `fixnum`, `hashed` o `functional`. Hay *n* índices, y el índice *i*-ésimo va de 0 a *dim\_i* - 1.

La ventaja de `make_array` sobre `array` estriba en que el valor retornado no tiene nombre, y una vez que un puntero deja de referenciarlo, el valor desaparece. Por ejemplo, si `y: make_array (...)` entonces `y` apunta a un objeto que ocupa cierto espacio en la memoria, pero después de `y: false`, `y` ya no apunta al objeto, por lo que éste puede ser considerado basura y posteriormente eliminado.

En la instrucción `y: make_array ('functional', 'f', 'hashed', 1)` el segundo argumento de `make_array` es la función que se debe invocar para calcular los elementos del arreglo, pasándose recursivamente a `make_array` el resto de los argumentos para generar la "memoria" de la función arreglo.

### **rearray** (*A*, *dim\_1*, ..., *dim\_n*) Función

Cambia las dimensiones de un arreglo. El nuevo arreglo será rellenado con los elementos del viejo según el orden de las filas. Si el arreglo antiguo era demasiado pequeño, los elementos restantes se rellenan con `false`, `0.0` o `0`, dependiendo del tipo del arreglo. El tipo del arreglo no se puede cambiar.

**remarray** (*A\_1*, ..., *A\_n*)

Función

**remarray** (*all*)

Función

Borra los arreglos y las funciones relacionadas con ellos, liberando el espacio de memoria ocupado.

La llamada **remarray** (*all*) borra todos los elementos de la lista global **arrays**.

La función **remarray** devuelve la lista de los arreglos borrados.

**subvar** (*x*, *i*)

Función

Evalúa la expresión subindicada *x*[*i*].

La función **subvar** evalúa sus argumentos.

La instrucción **arraymake** (*x*, [*i*]) construye la expresión *x*[*i*], pero no la evalúa.

Ejemplos:

```
(%i1) x : foo $

(%i2) i : 3 $

(%i3) subvar (x, i);
(%o3) foo
 3

(%i4) foo : [aa, bb, cc, dd, ee]$

(%i5) subvar (x, i);
(%o5) cc

(%i6) arraymake (x, [i]);
(%o6) foo
 3

(%i7) ''%;
(%o7) cc
```

**use\_fast\_arrays**

Variable opcional

Valor por defecto: **false**

Si **use\_fast\_arrays** vale **true** entonces tan solo se reconocen dos tipos de arreglos.



## 26 Matrices y Álgebra Lineal

### 26.1 Introducción a las Matrices y al Álgebra Lineal

#### 26.1.1 Operador punto

El operador `.` realiza la multiplicación matricial y el producto escalar. Cuando los operandos son dos matrices columna o matrices fila `a` y `b`, la expresión `a.b` es equivalente a `sum(a[i]*b[i], i, 1, length(a))`. Si `a` y `b` no son complejos, estamos en el caso del producto escalar. En caso de ser `a` y `b` vectores en el campo complejo, el producto escalar se define como `conjugate(a).b`; la función `innerproduct` del paquete `eigen` realiza el producto escalar complejo.

Cuando los operandos son matrices de índole más general, el resultado que se obtiene es el producto matricial de `a` por `b`. El número de filas de `b` debe ser igual al número de columnas de `a`, y el resultado tiene un número de filas igual al de `a` y un número de columnas igual al de `b`.

Al objeto de distinguir `.` como operador aritmético del punto decimal de la notación en coma flotante, puede ser necesario dejar espacios a ambos lados. Por ejemplo, `5.e3` es 5000.0 pero `5 . e3` es 5 por `e3`.

Hay algunas variables globales que controlan la simplificación de expresiones que contengan al operador `.`, a saber, `dot`, `dot0nscsimp`, `dot0simp`, `dot1simp`, `dotassoc`, `dotconstrules`, `dotdistrib`, `dotexptsimp`, `dotident`, y `dotscrules`.

#### 26.1.2 Vectores

El paquete `vect` define funciones para análisis vectorial. Para cargar el paquete en memoria se debe hacer `load("vect")` y con `demo("vect")` se presenta una demostración sobre las funciones del paquete.

El paquete de análisis vectorial puede combinar y simplificar expresiones simbólicas que incluyan productos escalares y vectoriales, junto con los operadores de gradiente, divergencia, rotacional y laplaciano. La distribución de estos operadores sobre sumas o productos se gobierna por ciertas variables, al igual que otras transformaciones, incluida la expansión en componentes en cualquier sistema de coordenadas especificado. También hay funciones para obtener el potencial escalar o vectorial de un campo.

El paquete `vect` contiene las siguientes funciones: `vectorsimp`, `scalefactors`, `express`, `potential` y `vectorpotential`.

Aviso: el paquete `vect` declara el operador `.` como conmutativo.

#### 26.1.3 Paquete eigen

El paquete `eigen` contiene funciones para el cálculo simbólico de valores y vectores propios. Maxima carga el paquete automáticamente si se hace una llamada a cualquiera de las dos funciones `eigenvalues` o `eigenvectors`. El paquete se puede cargar de forma explícita mediante `load("eigen")`.

La instrucción `demo ("eigen")` hace una demostración de las funciones de este paquete; `batch ("eigen")` realiza la misma demostración pero sin pausas entre los sucesivos cálculos.

Las funciones del paquete `eigen` son `innerproduct`, `unitvector`, `columnvector`, `gramschmidt`, `eigenvalues`, `eigenvectors`, `uniteigenvectors` y `similaritytransform`.

## 26.2 Definiciones para las Matrices y el Álgebra Lineal

**addcol** (*M*, *lista\_1*, ..., *lista\_n*) Función

Añade la/s columna/s dada/s por la/s lista/s (o matrices) a la matriz *M*.

**addrow** (*M*, *lista\_1*, ..., *lista\_n*) Función

Añade la/s fila/s dada/s por la/s lista/s (o matrices) a la matriz *M*.

**adjoint** (*M*) Función

Devuelve el adjunto de la matriz *M*. La matriz adjunta es la transpuesta de la matriz de cofactores de *M*.

**augcoefmatrix** (*eqn\_1*, ..., *eqn\_m*), [*x\_1*, ..., *x\_n*] Función

Devuelve la matriz aumentada de coeficientes del sistema de ecuaciones lineales *eqn\_1*, ..., *eqn\_m* de variables *x\_1*, ..., *x\_n*. Se trata de la matriz de coeficientes con una columna adicional para los términos constantes de cada ecuación, es decir, aquellos términos que no dependen de las variables *x\_1*, ..., *x\_n*.

```
(%i1) m: [2*x - (a - 1)*y = 5*b, c + b*y + a*x = 0]$
(%i2) augcoefmatrix (m, [x, y]);
 [2 1 - a - 5 b]
(%o2) [
 [a b c]
```

**charpoly** (*M*, *x*) Función

Calcula el polinomio característico de la matriz *M* respecto de la variable *x*. Esto es, `determinant (M - diagsmatrix (length (M), x))`.

```
(%i1) a: matrix ([3, 1], [2, 4]);
 [3 1]
(%o1) [
 [2 4]
(%i2) expand (charpoly (a, lambda));
 2
(%o2) lambda - 7 lambda + 10
(%i3) (programmode: true, solve (%));
(%o3) [lambda = 5, lambda = 2]
(%i4) matrix ([x1], [x2]);
 [x1]
(%o4) [
 [x2]
(%i5) ev (a . % - lambda*%, %th(2)[1]);
 [x2 - 2 x1]
(%o5) [
```

```

 [2 x1 - x2]
(%i6) %[1, 1] = 0;
(%o6) x2 - 2 x1 = 0
(%i7) x2^2 + x1^2 = 1;
 2 2
(%o7) x2 + x1 = 1
(%i8) solve ([%th(2), %], [x1, x2]);
 1 2
(%o8) [[x1 = - ----, x2 = - ----],
 sqrt(5) sqrt(5)

 1 2
 [x1 = ----, x2 = ----]]
 sqrt(5) sqrt(5)

```

**coefmatrix** (*eqn\_1*, ..., *eqn\_m*), [*x\_1*, ..., *x\_n*] Función  
 Devuelve la matriz de coeficientes para las variables *x\_1*, ..., *x\_n* del sistema de ecuaciones lineales *eqn\_1*, ..., *eqn\_m*.

```

(%i1) coefmatrix([2*x-(a-1)*y+5*b = 0, b*y+a*x = 3], [x,y]);
 [2 1 - a]
(%o1) []
 [a b]

```

**col** (*M*, *i*) Función  
 Devuelve la *i*-ésima columna de la matriz *M*. El resultado es una matriz de una sola columna.

**columnvector** (*L*) Función  
**covect** (*L*) Función

Devuelve una matriz con una columna y **length** (*L*) filas, conteniendo los elementos de la lista *L*.

La llamada **covect** es un sinónimo de **columnvector**.

Es necesario cargar la función haciendo **load** ("eigen").

Ejemplo:

```

(%i1) load ("eigen")$
Warning - you are redefining the Macsyma function eigenvalues
Warning - you are redefining the Macsyma function eigenvectors
(%i2) columnvector ([aa, bb, cc, dd]);
 [aa]
 []
 [bb]
(%o2) []
 [cc]
 []
 [dd]

```

**conjugate** (*x*) Función  
 Devuelve el conjugado complejo de *x*.



```
(%i1) declare ([aa, bb], real, cc, complex, ii, imaginary);

(%o1) done
(%i2) conjugate (aa + bb*%i);

(%o2) aa - %i bb
(%i3) conjugate (cc);

(%o3) conjugate(cc)
(%i4) conjugate (ii);

(%o4) - ii
(%i5) conjugate (xx + yy);

(%o5) conjugate(yy) + conjugate(xx)
```

**copymatrix** (*M*)

Función

Devuelve una copia de la matriz *M*. Esta es la única manera de obtener una réplica de *M* además de la de copiar elemento a elemento.

Nótese que una asignación de una matriz a otra, como en `m2: m1`, no hace una copia de *m1*. Asignaciones del tipo `m2 [i,j]: x` o `setelmx (x, i, j, m2)` también modifica *m1* [i,j]. Si se crea una copia con `copymatrix` y luego se hacen asignaciones se tendrá una copia separada y modificada.

**determinant** (*M*)

Función

Calcula el determinante de *M* por un método similar al de eliminación de Gauss

La forma del resultado depende del valor asignado a `ratmx`.

Existe una rutina especial para calcular determinantes de matrices con elementos dispersas, la cual será invocada cuando las variables `ratmx` y `sparse` valgan ambas `true`.

**detout**

Variable opcional

Valor por defecto: `false`

Cuando `detout` vale `true`, el determinante de la matriz cuya inversa se calcula aparece como un factor fuera de la matriz.

Para que esta variable surta efecto, `doallmxops` y `doscmxops` deberían tener el valor `false` (véanse sus descripciones). Alternativamente, esta variable puede ser suministrada a `ev`.

Ejemplo:

```
(%i1) m: matrix ([a, b], [c, d]);
 [a b]
(%o1) []
 [c d]

(%i2) detout: true$
(%i3) doallmxops: false$
(%i4) doscmxops: false$
```

```
(%i5) invert (m);
```

$$\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$


---

```
(%o5)
```

$$a \ d - \ b \ c$$
**diagmatrix** (*n*, *x*)

Función

Devuelve una matriz diagonal de orden *n* con los elementos de la diagonal todos ellos iguales a *x*. La llamada **diagmatrix** (*n*, 1) devuelve una matriz identidad (igual que **ident** (*n*)).

La variable *n* debe ser un número entero, en caso contrario **diagmatrix** envía un mensaje de error.

*x* puede ser cualquier tipo de expresión, incluso otra matriz. Si *x* es una matriz, no se copia; todos los elementos de la diagonal son iguales a *x*.

**doallmxops**

Variable opcional

Valor por defecto: **true**

Cuando **doallmxops** vale **true**, todas las operaciones relacionadas con matrices son llevadas a cabo. Cuando es **false**, entonces las selecciones para **dot** controlan las operaciones a ejecutar.

**domxexpt**

Variable opcional

Valor por defecto: **true**

Cuando **domxexpt** vale **true**, un exponente matricial, como **exp** (*M*) donde *M* es una matriz, se interpreta como una matriz cuyo elemento [*i*, *j*] es igual a **exp** (*m*[*i*, *j*]). En otro caso, **exp** (*M*) se evalúa como **exp** (**ev**(*M*)).

La variable **domxexpt** afecta a todas las expresiones de la forma *base*<sup>*exponente*</sup> donde *base* es una expresión escalar o constante y *exponente* es una lista o matriz.

Ejemplo:

```
(%i1) m: matrix ([1, %i], [a+b, %pi]);
```

$$\begin{bmatrix} 1 & \%i \\ b + a & \%pi \end{bmatrix}$$

```
(%o1)
```

```
(%i2) domxexpt: false$
```

```
(%i3) (1 - c)^m;
```

$$\begin{bmatrix} 1 & \%i \\ b + a & \%pi \end{bmatrix}$$

```
(%o3) (1 - c)
```

```
(%i4) domxexpt: true$
```

```
(%i5) (1 - c)^m;
```

$$\begin{bmatrix} 1 - c & (1 - c) \%i \\ b + a & (1 - c) \%pi \end{bmatrix}$$

```
(%o5)
```

- dommxops** Variable opcional  
 Valor por defecto: `true`  
 Cuando `dommxops` vale `true`, se realizan todas las operaciones entre matrices o entre matrices y listas (pero no las operaciones entre matrices y escalares); si esta variable es `false` tales operaciones no se realizan.
- domxnctimes** Variable opcional  
 Valor por defecto: `false`  
 Cuando `domxnctimes` vale `true`, se calculan los productos no conmutativos entre matrices.
- dontfactor** Variable opcional  
 Valor por defecto: `[]`  
 En `dontfactor` puede guardarse una lista de variables respecto de las cuales no se realizarán factorizaciones. Inicialmente, la lista está vacía.
- doscmxops** Variable opcional  
 Valor por defecto: `false`  
 Cuando `doscmxops` vale `true`, se realizan las operaciones entre escalares y matrices.
- doscmxplus** Variable opcional  
 Valor por defecto: `false`  
 Cuando `doscmxplus` vale `true`, las operaciones entre escalares y matrices dan como resultado una matriz.
- dot0nscsimp** Variable opcional  
 Valor por defecto: `true`  
 (Esta descripción no está clara en la versión inglesa original.)
- dotassoc** Variable opcional  
 Valor por defecto: `true`  
 Cuando `dotassoc` vale `true`, una expresión como  $(A.B).C$  se transforma en  $A.(B.C)$ .
- dotconstrules** Variable opcional  
 Valor por defecto: `true`  
 Cuando `dotconstrules` vale `true`, un producto no conmutativo de una constante con otro término se transforma en un producto conmutativo.
- dotdistrib** Variable opcional  
 Valor por defecto: `false`  
 Cuando `dotdistrib` vale `true`, una expresión como  $A.(B + C)$  se transforma en  $A.B + A.C$ .

**dotexptsimp**

Variable opcional

Valor por defecto: `true`Cuando `dotexptsimp` vale `true`, una expresión como  $A.A$  se transforma en  $A^2$ .**dotident**

Variable opcional

Valor por defecto: 1

El valor de la variable `dotident` es el resultado devuelto por  $X^0$ .**dotscrules**

Variable opcional

Valor por defecto: `false`Cuando `dotscrules` vale `true`, una expresión como  $A.SC$  o  $SC.A$  se transforma en  $SC*A$  y  $A.(SC*B)$  en  $SC*(A.B)$ .**echelon** ( $M$ )

Función

Devuelve la forma escalonada de la matriz  $M$ , obtenida por eliminación gaussiana. La forma escalonada se calcula a partir de  $M$  mediante operaciones elementales con sus filas, de tal manera que el primer elemento no nulo de cada fila en la matriz resultado es la unidad y que cada elemento de la columna por debajo del primer uno de cada fila sean todos ceros.

La función `triangularize` también lleva a cabo la eliminación gaussiana, pero no normaliza el primer elemento no nulo de cada fila.

Otras funciones, como `lu_factor` y `cholesky`, también dan como resultados matrices triangularizadas.

```
(%i1) M: matrix ([3, 7, aa, bb], [-1, 8, 5, 2], [9, 2, 11, 4]);
 [3 7 aa bb]
 []
(%o1) [-1 8 5 2]
 []
 [9 2 11 4]

(%i2) echelon (M);
 [1 -8 -5 -2]
 []
 [28 11]
 [0 1 -- --]
(%o2) [37 37]
 []
 [37 bb - 119]
 [0 0 1 -----]
 [37 aa - 313]
```

**eigenvalues** ( $M$ )

Función

**eivals** ( $M$ )

Función

Devuelve una lista con dos sublistas. La primera sublista la forman los valores propios de la matriz  $M$  y la segunda sus multiplicidades correspondientes.

El nombre `eivals` es un sinónimo de `eigenvalues`.

La función `eigenvalues` llama a la función `solve` para calcular las raíces del polinomio característico de la matriz. En ocasiones, `solve` no podrá encontrar dichas raíces, en cuyo caso otras funciones de este paquete no trabajarán correctamente, a excepción de `innerproduct`, `unitvector`, `columnvector` y `gramschmidt`.

En algunos casos los valores propios encontrados por `solve` serán expresiones complicadas, las cuales se podrán simplificar haciendo uso de otras funciones.

El paquete `eigen.mac` se carga en memoria de forma automática cuando se invocan `eigenvalues` o `eigenvectors`. Si `eigen.mac` no está ya cargado, `load ("eigen")` lo carga. Tras la carga, todas las funciones y variables del paquete estarán activas.

### **eigenvectors** ( $M$ )

Función

### **eivects** ( $M$ )

Función

Tomando la matriz  $M$  como argumento, devuelve una lista de listas, la primera de las cuales es la salida de `eigenvalues` y las siguientes son los vectorios propios de la matriz asociados a los valores propios correspondientes. Los vectores propios calculados son los vectores propios por la derecha.

El nombre `eivects` es un sinónimo de `eigenvectors`.

El paquete `eigen.mac` se carga en memoria de forma automática cuando se invocan `eigenvalues` o `eigenvectors`. Si `eigen.mac` no está ya cargado, `load ("eigen")` lo carga. Tras la carga, todas las funciones y variables del paquete estarán activas.

Las variables que afectan a esta función son:

`nondiagonalizable` toma el valor `true` o `false` dependiendo de si la matriz no es diagonalizable o diagonalizable tras la ejecución de `eigenvectors`.

`hermitianmatrix`, si vale `true`, entonces los vectores propios degenerados de la matriz hermítica son ortogonalizados mediante el algoritmo de Gram-Schmidt.

`knoweigvals`, si vale `true`, entonces el paquete `eigen` da por sentado que los valores propios de la matriz son conocidos por el usuario y almacenados en la variable global `listeigvals`. `listeigvals` debería ser similar a la salida de `eigenvalues`.

La función `algsys` se utiliza aquí para calcular los vectores propios. A veces, `algsys` no podrá calcular una solución. En algunos casos, será posible simplificar los valores propios calculándolos en primer lugar con `eigenvalues` y luego utilizando otras funciones para simplificarlos. Tras la simplificación, `eigenvectors` podrá ser llamada otra vez con la variable `knoweigvals` ajustada al valor `true`.

### **ematrix** ( $m, n, x, i, j$ )

Función

Devuelve una matriz de orden  $m$  por  $n$ , con todos sus elementos nulos, excepto el que ocupa la posición  $[i, j]$ , que es igual a  $x$ .

### **entermatrix** ( $m, n$ )

Función

Devuelve una matriz de orden  $m$  por  $n$ , cuyos elementos son leídos de forma interactiva.

Si  $n$  es igual a  $m$ , Maxima pregunta por el tipo de matriz (diagonal, simétrica, antisimétrica o general) y luego por cada elemento. Cada respuesta introducida por el usuario debe terminar con un punto y coma ; o con un signo de dólar \$.

Si  $n$  y  $m$  no son iguales, Maxima pregunta por el valor de cada elemento.

Los elementos de la matriz pueden ser cualquier tipo de expresión, que en todo caso será evaluada. `entermatrix` evalúa sus argumentos.

```
(%i1) n: 3$
(%i2) m: entermatrix (n, n)$

Is the matrix 1. Diagonal 2. Symmetric 3. Antisymmetric 4. General
Answer 1, 2, 3 or 4 :
1$
Row 1 Column 1:
(a+b)^n$
Row 2 Column 2:
(a+b)^(n+1)$
Row 3 Column 3:
(a+b)^(n+2)$

Matrix entered.
(%i3) m;

 3
[(b + a) 0 0]
[4
[0 (b + a) 0]
[5
[0 0 (b + a)]
```

**genmatrix** ( $a, i\_2, j\_2, i\_1, j\_1$ )

Función

**genmatrix** ( $a, i\_2, j\_2, i\_1$ )

Función

**genmatrix** ( $a, i\_2, j\_2$ )

Función

Devuelve una matriz generada a partir de  $a$ , siendo  $a[i\_1, j\_1]$  el elemento superior izquierdo y  $a[i\_2, j\_2]$  el inferior derecho de la matriz. Aquí  $a$  es un arreglo (creado por `array`, no por `make_array`) o una función arreglo (una función arreglo se crea como cualquier otra función con `:=` o `define`, pero sus argumentos encerrados entre corchetes en lugar de paréntesis).

Si se omite  $j\_1$ , entonces se le asigna el valor  $i\_1$ . Si tanto  $j\_1$  como  $i\_1$  se omiten, a las dos variables se le asigna el valor .

Si un elemento  $i, j$  del arreglo no está definido, se le asignará el elemento simbólico  $a[i, j]$ .

```
(%i1) h[i,j] := 1/(i+j-1)$
(%i2) genmatrix (h, 3, 3);

[1 1]
[1 - -]
[2 3]
[- -]
[1 1 1]
(%o2) [- - -]
```

```

[2 3 4]
[]
[1 1 1]
[- - -]
[3 4 5]

(%i3) array (a, fixnum, 2, 2)$
(%i4) a[1,1]: %e$
(%i5) a[2,2]: %pi$
(%i6) kill (a[1,2], a[2,1])$
(%i7) genmatrix (a, 2, 2);

(%o7)
[%e a]
[1, 2]
[]
[a %pi]
[2, 1]

```

**gramschmidt (x)**

Función

**gschmit (x)**

Función

Ejecuta el algoritmo de ortogonalización de Gram-Schmidt sobre  $x$ , que puede ser una matriz o una lista de listas. La función **gramschmidt** no altera el valor de  $x$ .

Si  $x$  es una matriz, el algoritmo se aplica a las filas de  $x$ . Si  $x$  es una lista de listas, el algoritmo se aplica a las sublistas, las cuales deben tener el mismo número de miembros. En cualquier caso, el valor devuelto es una lista de listas, cuyas sublistas son ortogonales.

La función **factor** es invocada en cada paso del algoritmo para simplificar resultados intermedios. Como consecuencia, el valor retornado puede contener enteros factorizados.

El nombre **gschmit** es sinónimo de **gramschmidt**.

Es necesario cargar la función haciendo `load ("eigen")`.

Ejemplo:

```

(%i1) load ("eigen")$
Warning - you are redefining the Macsyma function eigenvalues
Warning - you are redefining the Macsyma function eigenvectors
(%i2) x: matrix ([1, 2, 3], [9, 18, 30], [12, 48, 60]);

(%o2)
[1 2 3]
[]
[9 18 30]
[]
[12 48 60]

(%i3) y: gschmidt (x);

(%o3)
 2 2 4 3
 3 3 3 5 2 3 2 3
[[1, 2, 3], [- ---, - --, ---], [- ----, ----, 0]]
 2 7 7 2 7 5 5

(%i4) i: innerproduct$
(%i5) [i (y[1], y[2]), i (y[2], y[3]), i (y[3], y[1])];
(%o5)
[0, 0, 0]

```

**hach** (*a*, *b*, *m*, *n*, *l*) Función

La función **hach** es una implementación del algoritmo Hacijan de programación lineal.

Es necesario cargar la función haciendo `load ("kach")`. La instrucción `demo ("kach")` ejecuta una demostración de esta función.

**ident** (*n*) Función

Devuelve la matriz identidad de orden *n*.

**innerproduct** (*x*, *y*) Función

**inprod** (*x*, *y*) Función

Devuelve el producto interior o escalar de *x* por *y*, que deben ser listas de igual longitud, o ambas matrices columna o fila de igual longitud. El valor devuelto es `conjugate (x) . y`, donde `.` es el operador de multiplicación no conmutativa.

Es necesario cargar la función haciendo `load ("eigen")`.

El nombre **inprod** es sinónimo de **innerproduct**.

**invert** (*M*) Función

Devuelve la inversa de la matriz *M*, calculada por el método del adjunto.

La implementación actual no es eficiente para matrices de orden grande.

Cuando `detout` vale `true`, el determinante queda fuera de la inversa a modo de factor escalar.

Los elementos de la matriz inversa no se expanden. Si *M* tiene elementos polinómicos, se puede mejorar el aspecto del resultado haciendo `expand (invert (m))`, `detout`.

Véase la descripción de `^^` (exponente no conmutativo) para información sobre otro método para invertir matrices.

**lmxchar** Variable opcional

Valor por defecto: `[`

La variable **lmxchar** guarda el carácter a mostrar como delimitador izquierdo de la matriz. Véase también **rmxchar**.

Ejemplo:

```
(%i1) lmxchar: "|"
(%i2) matrix ([a, b, c], [d, e, f], [g, h, i]);
 | a b c |
 | |
(%o2) | d e f |
 | |
 | g h i |
```

**matrix** (*fila\_1*, ..., *fila\_n*) Función

Devuelve una matriz rectangular con las filas *fila\_1*, ..., *fila\_n*. Cada fila es una lista de expresiones. Todas las filas deben tener el mismo número de miembros.

Las operaciones `+` (suma), `-` (resta), `*` (multiplicación) y `/` (división), se llevan a cabo elemento a elemento cuando los operandos son dos matrices, un escalar y una matriz



o una matriz con un escalar. La operación  $\wedge$  (exponenciación, equivalente a `**`) se lleva cabo también elemento a elemento si los operandos son un escalar y una matriz o una matriz y un escalar, pero no si los operandos son dos matrices.

El producto matricial se representa con el operador de multiplicación no conmutativa `..`. El correspondiente operador de exponenciación no conmutativa es `^^`. Dada la matriz  $A$ ,  $A.A = A^{..2}$  y  $A^{..-1}$  es la inversa de  $A$ , si existe.

Algunas variables controlan la simplificación de expresiones que incluyan estas operaciones: `doallmxops`, `domxexpt`, `domxmxops`, `doscmxops` y `doscmxplus`.

Hay otras opciones adicionales relacionadas con matrices: `lmxchar`, `rmxchar`, `ratmx`, `listarith`, `detout`, `scalarmatrix` y `sparse`.

Hay también algunas funciones que admiten matrices como argumentos o que devuelven resultados matriciales: `eigenvalues`, `eigenvectors`, `determinant`, `charpoly`, `genmatrix`, `addcol`, `addrow`, `copymatrix`, `transpose`, `echelon` y `rank`.

Ejemplos:

- Construcción de matrices a partir de listas.

```
(%i1) x: matrix ([17, 3], [-8, 11]);
 [17 3]
(%o1) []
 [- 8 11]
(%i2) y: matrix ([%pi, %e], [a, b]);
 [%pi %e]
(%o2) []
 [a b]
```

- Suma elemento a elemento.

```
(%i3) x + y;
 [%pi + 17 %e + 3]
(%o3) []
 [a - 8 b + 11]
```

- Resta elemento a elemento.

```
(%i4) x - y;
 [17 - %pi 3 - %e]
(%o4) []
 [- a - 8 11 - b]
```

- Multiplicación elemento a elemento.

```
(%i5) x * y;
 [17 %pi 3 %e]
(%o5) []
 [- 8 a 11 b]
```

- División elemento a elemento.

```
(%i6) x / y;
 [17 - 1]
 [--- 3 %e]
 [%pi]
(%o6) []
 [8 11]
```

$$\begin{bmatrix} - & - & -- & ] \\ [ & a & b & ] \end{bmatrix}$$

- Matriz elevada a un exponente escalar, operación elemento a elemento.

(%i7)  $x \wedge 3$ ;

(%o7) 
$$\begin{bmatrix} 4913 & 27 \\ [ & ] \\ - 512 & 1331 \end{bmatrix}$$

- Base escalar y exponente matricial, operación elemento a elemento.

(%i8)  $\exp(y)$ ;

(%o8) 
$$\begin{bmatrix} \%pi & \%e \\ [ \%e & \%e \\ [ & ] \\ [ a & b \\ [ \%e & \%e \end{bmatrix}$$

- Base y exponente matriciales. Esta operación no se realiza elemento a elemento.

(%i9)  $x \wedge y$ ;

(%o9) 
$$\begin{bmatrix} \%pi & \%e \\ [ & ] \\ [ a & b \\ [ 17 & 3 \\ [ & ] \\ - 8 & 11 \end{bmatrix}$$

- Multiplicación matricial no conmutativa.

(%i10)  $x \cdot y$ ;

(%o10) 
$$\begin{bmatrix} 3 a + 17 \%pi & 3 b + 17 \%e \\ [ & ] \\ 11 a - 8 \%pi & 11 b - 8 \%e \end{bmatrix}$$

(%i11)  $y \cdot x$ ;

(%o11) 
$$\begin{bmatrix} 17 \%pi - 8 \%e & 3 \%pi + 11 \%e \\ [ & ] \\ 17 a - 8 b & 11 b + 3 a \end{bmatrix}$$

- Exponenciación matricial no conmutativa. Una base escalar  $b$  elevada a un exponente matricial  $M$  se lleva a cabo elemento a elemento y por lo tanto  $b^{\wedge}m$  equivale a  $b^{\wedge}m$ .

(%i12)  $x^{\wedge\wedge} 3$ ;

(%o12) 
$$\begin{bmatrix} 3833 & 1719 \\ [ & ] \\ - 4584 & 395 \end{bmatrix}$$

(%i13)  $\%e^{\wedge\wedge} y$ ;

(%o13) 
$$\begin{bmatrix} \%pi & \%e \\ [ \%e & \%e \\ [ & ] \\ [ a & b \\ [ \%e & \%e \end{bmatrix}$$

- Una matriz elevada al exponente -1 con el operador de exponenciación no conmutativa equivale a la matriz inversa, si existe.

```
(%i14) x ^^ -1;
[11 3]
[--- - ---]
[211 211]
(%o14)
[]
[8 17]
[--- ---]
[211 211]

(%i15) x . (x ^^ -1);
[1 0]
(%o15) []
[0 1]
```

**matrixmap** (*f*, *M*)

Función

Devuelve una matriz con el elemento *i,j* igual a *f*(*M*[*i,j*]).

Véanse también `map`, `fullmap`, `fullmapl` y `apply`.

**matrixp** (*expr*)

Función

Devuelve `true` si *expr* es una matriz, en caso contrario `false`.

**matrix\_element\_add**

Variable opcional

Valor por defecto: `+`

La variable `matrix_element_add` guarda el símbolo del operador a ejecutar en lugar de la suma en el producto matricial; a `matrix_element_add` se le puede asignar cualquier operador n-ario (esto es, una función que admite cualquier número de argumentos). El valor asignado puede ser el nombre de un operador encerrado entre apóstrofes, el nombre de una función o una expresión lambda.

Véanse también `matrix_element_mult` y `matrix_element_transpose`.

Ejemplo:

```
(%i1) matrix_element_add: "*"
(%i2) matrix_element_mult: "^"
(%i3) aa: matrix ([a, b, c], [d, e, f]);
[a b c]
(%o3) []
[d e f]
(%i4) bb: matrix ([u, v, w], [x, y, z]);
[u v w]
(%o4) []
[x y z]
(%i5) aa . transpose (bb);
[u v w x y z]
[a b c a b c]
(%o5) []
[u v w x y z]
[d e f d e f]
```

**matrix\_element\_mult**

Variable opcional

Valor por defecto: \*

La variable `matrix_element_mult` guarda el símbolo del operador a ejecutar en lugar de la multiplicación en el producto matricial; a `matrix_element_mult` se le puede asignar cualquier operador binario. El valor asignado puede ser el nombre de un operador encerrado entre apóstrofos, el nombre de una función o una expresión lambda.

El operador `.` puede ser una opción útil en determinados contextos.

Véanse también `matrix_element_add` y `matrix_element_transpose`.

Ejemplo:

```
(%i1) matrix_element_add: lambda ([[x]], sqrt (apply ("+", x)))$
(%i2) matrix_element_mult: lambda ([x, y], (x - y)^2)$
(%i3) [a, b, c] . [x, y, z];
 2 2 2
(%o3) sqrt((c - z) + (b - y) + (a - x))
(%i4) aa: matrix ([a, b, c], [d, e, f]);
 [a b c]
(%o4) []
 [d e f]
(%i5) bb: matrix ([u, v, w], [x, y, z]);
 [u v w]
(%o5) []
 [x y z]
(%i6) aa . transpose (bb);
 [2 2 2]
 [sqrt((c - w) + (b - v) + (a - u))]
(%o6) Col 1 = []
 [2 2 2]
 [sqrt((f - w) + (e - v) + (d - u))]

 [2 2 2]
 [sqrt((c - z) + (b - y) + (a - x))]
Col 2 = []
 [2 2 2]
 [sqrt((f - z) + (e - y) + (d - x))]
```

**matrix\_element\_transpose**

Variable opcional

Valor por defecto: false

La variable `matrix_element_transpose` es una operación que se aplica a cada elemento de una matriz a la que se le calcula la transpuesta. A `matrix_element_mult` se le puede asignar cualquier operador unitario. El valor asignado puede ser el nombre de un operador encerrado entre apóstrofos, el nombre de una función o una expresión lambda.

Cuando `matrix_element_transpose` es igual a `transpose`, la función `transpose` se aplica a cada elemento. Cuando `matrix_element_transpose` es igual a `nonscalars`, la función `transpose` se aplica a todos los elementos no escalares. Si alguno de los elementos es un átomo, la opción `nonscalars` se aplica `transpose` sólo si el átomo se declara no escalar, mientras que la opción `transpose` siempre aplica `transpose`.

La opción por defecto, `false`, significa que no se aplica ninguna operación.

Véanse también `matrix_element_add` y `matrix_element_mult`.

Ejemplos:

```
(%i1) declare (a, nonscalar)$
(%i2) transpose ([a, b]);
 [transpose(a)]
(%o2) []
 [b]
(%i3) matrix_element_transpose: nonscalars$
(%i4) transpose ([a, b]);
 [transpose(a)]
(%o4) []
 [b]
(%i5) matrix_element_transpose: transpose$
(%i6) transpose ([a, b]);
 [transpose(a)]
(%o6) []
 [transpose(b)]
(%i7) matrix_element_transpose: lambda ([x], realpart(x) - %i*imagpart(x))$
(%i8) m: matrix ([1 + 5*%i, 3 - 2*%i], [7*%i, 11]);
 [5 %i + 1 3 - 2 %i]
(%o8) []
 [7 %i 11]
(%i9) transpose (m);
 [1 - 5 %i - 7 %i]
(%o9) []
 [2 %i + 3 11]
```

**mattrace** ( $M$ ) Función

Devuelve la traza (esto es, la suma de los elementos de la diagonal principal) de la matriz cuadrada  $M$ .

Para disponer de esta función es necesario cargar el paquete haciendo `load ("nchrpl")`.

**minor** ( $M, i, j$ ) Función

Devuelve el menor  $(i, j)$  de la matriz  $M$ . Esto es, la propia matriz  $M$ , una vez extraídas la fila  $i$  y la columna  $j$ .

**ncexpt** ( $a, b$ ) Función

Si una expresión exponencial no conmutativa es demasiado grande para mostrarse en la forma  $a^b$  entonces aparece como `ncexpt (a,b)`.

El nombre `ncexpt` no corresponde al de una función u operador, sino que tan solo aparece en la salida y no se reconoce como una entrada válida.

**ncharpoly** ( $M, x$ ) Función

Devuelve el polinomio característico de la matriz  $M$  respecto de la variable  $x$ . Es una alternativa a la función `charpoly` de Maxima.

La función `ncharpoly` opera calculando trazas de las potencias de la matriz dada, que son iguales a las sumas de las potencias de las raíces del polinomio característico. A partir de estas cantidades se pueden calcular las funciones simétricas de las raíces, que no son otra cosa sino los coeficientes del polinomio característico. La función `charpoly` opera calculando el determinante de  $x * \text{ident}[n] - a$ . La función `ncharpoly` es m'as eficiente en el caso de matrices grandes y densas.

Para disponer de esta función es necesario cargar el paquete haciendo `load("nchrpl")`.

**newdet** ( $M, n$ ) Función

Calcula el determinante de la matriz o arreglo  $M$  por el algoritmo del árbol menor de Johnson-Gentleman. El argumento  $n$  es el orden; es opcional si  $M$  es una matriz.

**nonscalar** Declaración

Hace que los átomos se comporten como hace una lista o matriz con respecto del operador `.` de la multiplicación no conmutativa.

**nonscalarp** ( $expr$ ) Función

Devuelve `true` si  $expr$  no es escalar, es decir, si contiene átomos declarados como no escalares, listas o matrices.

**permanent** ( $M, n$ ) Función

Calcula la permanente de la matriz  $M$ . La permanente es como un determinante pero sin cambios de signo.

**rank** ( $M$ ) Función

Calcula el rango de la matriz  $M$ . Esto es, el orden del mayor subdeterminante no singular de  $M$ .

La función *rango* puede retornar una respuesta errónea si no detecta que un elemento de la matriz equivalente a cero lo es.

**ratmx** Variable opcional

Valor por defecto: `false`

Si `ratmx` vale `false`, el determinante y la suma, resta y producto matriciales se calculan cuando las matrices se expresan en términos de sus elementos, pero no se calcula la inversión matricial en su representación general.

Si `ratmx` vale `true`, las cuatro operaciones citadas más arriba se calculan en el formato CRE y el resultado de la matriz inversa también se da en formato CRE. Esto puede hacer que se expandan los elementos de la matriz, dependiendo del valor de `ratfac`, lo que quizás no sea siempre deseable.

**row** ( $M, i$ ) Función

Devuelve la  $i$ -ésima fila de la matriz  $M$ . El valor que devuelve tiene formato de matriz.

**scalarmatrixp** Variable opcional

Valor por defecto: `true`

Si `scalarmatrixp` vale `true`, entonces siempre que una matriz 1 x 1 se produce como resultado del cálculo del producto no conmutativo de matrices se cambia al formato escalar.

Si `scalarmatrixp` vale `all`, entonces todas las matrices 1 x 1 se simplifican a escalares.

Si `scalarmatrixp` vale `false`, las matrices 1 x 1 no se convierten en escalares.

**setelm** $x, i, j, M$  Función

Asigna el valor  $x$  al  $(i, j)$ -ésimo elemento de la matriz  $M$  y devuelve la matriz actualizada.

La llamada  $M[i, j] : x$  hace lo mismo, pero devuelve  $x$  en lugar de  $M$ .

**similaritytransform**  $(M)$  Función

**simtran**  $(M)$  Función

La función `similaritytransform` calcula la transformada de similitud de la matriz  $M$ . Devuelve una lista que es la salida de la instrucción `uniteigenvectors`. Además, si la variable `nondiagonalizable` vale `false` entonces se calculan dos matrices globales `leftmatrix` y `rightmatrix`. Estas matrices tienen la propiedad de que `leftmatrix . M . rightmatrix` es una matriz diagonal con los valores propios de  $M$  en su diagonal. Si `nondiagonalizable` vale `true` entonces no se calculan estas matrices.

Si la variable `hermitianmatrix` vale `true` entonces `leftmatrix` es el conjugado complejo de la transpuesta de `rightmatrix`. En otro caso `leftmatrix` es la inversa de `rightmatrix`.

Las columnas de la matriz `rightmatrix` son los vectores propios de  $M$ . Las otras variables (véanse `eigenvalues` y `eigenvectors`) tienen el mismo efecto, puesto que `similaritytransform` llama a las otras funciones del paquete para poder formar `rightmatrix`.

Estas funciones se cargan con `load("eigen")`.

El nombre `simtran` es sinónimo de `similaritytransform`.

**sparse** Variable opcional

Valor por defecto: `false`

Si `sparse` vale `true` y si `ratmx` vale `true`, entonces `determinant` utilizará rutinas especiales para calcular determinantes dispersos.

**submatrix**  $(i_1, \dots, i_m, M, j_1, \dots, j_n)$  Función

**submatrix**  $(i_1, \dots, i_m, M)$  Función

**submatrix**  $(M, j_1, \dots, j_n)$  Función

Devuelve una nueva matriz formada a partir de la matriz  $M$  pero cuyas filas  $i_1, \dots, i_m$  y columnas  $j_1, \dots, j_n$  han sido eliminadas.

**transpose**  $(M)$  Función

Calcula la transpuesta de  $M$ .

Si  $M$  es una matriz, el valor devuelto es otra matriz  $N$  tal que  $N[i,j] = M[j,i]$ .

Si  $M$  es una lista, el valor devuelto es una matriz  $N$  de `length(m)` filas y 1 columna, tal que  $N[i,1] = M[i]$ .

### **triangularize** ( $M$ )

Función

Devuelve la forma triangular superior de la matriz  $M$ , obtenida por eliminación gaussiana. El resultado es el mismo que el devuelto por `echelon`, con la salvedad de que el primer elemento no nulo de cada fila no se normaliza a 1.

Las funciones `lu_factor` y `cholesky` también triangularizan matrices.

```
(%i1) M: matrix ([3, 7, aa, bb], [-1, 8, 5, 2], [9, 2, 11, 4]);
 [3 7 aa bb]
 []
(%o1) [- 1 8 5 2]
 []
 [9 2 11 4]
(%i2) triangularize (M);
 [- 1 8 5 2]
 []
(%o2) [0 - 74 - 56 - 22]
 []
 [0 0 626 - 74 aa 238 - 74 bb]
```

### **uniteigenvectors** ( $M$ )

Función

#### **ueivects** ( $M$ )

Función

Calcula los vectores propios unitarios de la matriz  $M$ . El valor que devuelve es una lista de listas, la primera de las cuales es la salida de la función `eigenvalues` y el resto de sublistas son los vectores propios unitarios de la matriz correspondiente a esos valores propios, respectivamente.

Las variables citadas en la descripción de la función `eigenvectors` tienen los mismos efectos en `uniteigenvectors`.

Si `knowneigvects` vale `true`, el paquete `eigen` da por supuesto que el usuario conoce los vectores propios de la matriz y que están guardados en la variable global `listeigvects`, en tal caso el contenido de `listeigvects` debe ser una lista de estructura similar a la que devuelve la función `eigenvectors`.

Si `knowneigvects` vale `true` y la lista de vectores propios está en la variable `listeigvects`, el valor de la variable `nondiagonalizable` puede que no sea el correcto. Si tal es el caso, debe asignarsele el valor correcto.

Para utilizar esta función es necesario cargarla haciendo `load ("eigen")`.

El nombre `ueivects` es sinónimo de `uniteigenvectors`.

### **unitvector** ( $x$ )

Función

#### **uvect** ( $x$ )

Función

Devuelve  $x/\text{norm}(x)$ , esto es, el vector unitario de igual dirección y sentido que  $x$ .

`load ("eigen")` loads this function.

Para utilizar esta función es necesario cargarla haciendo `load ("eigen")`.

El nombre `uvect` es sinónimo de `unitvector`.



**vectorsimp** (*expr*) Función

Realiza simplificaciones y expansiones de acuerdo con los valores de las siguientes variables globales:

`expandall`, `expanddot`, `expanddotplus`, `expandcross`, `expandcrossplus`, `expandcrosscross`, `expandgrad`, `expandgradplus`, `expandgradprod`, `expanddiv`, `expanddivplus`, `expanddivprod`, `expandcurl`, `expandcurlplus`, `expandcurlcurl`, `expandlaplacian`, `expandlaplacianplus` y `expandlaplacianprod`.

Todas estas variables tienen por defecto el valor `false`. El sufijo `plus` se refiere al uso de la suma o la distributividad. El sufijo `prod` se refiere a la expansión de operadores que realizan cualquier tipo de producto.

`expandcrosscross`

Simplifica  $p (q r)$  en  $(p.r) * q - (p.q) * r$ .

`expandcurlcurl`

Simplifica  $curlcurlp$  en  $graddivp + divgradp$ .

`expandlaplaciantodivgrad`

Simplifica  $laplacianp$  en  $divgradp$ .

`expandcross`

Activa `expandcrossplus` y `expandcrosscross`.

`expandplus`

Activa `expanddotplus`, `expandcrossplus`, `expandgradplus`, `expanddivplus`, `expandcurlplus` y `expandlaplacianplus`.

`expandprod`

Activa `expandgradprod`, `expanddivprod` y `expandlaplacianprod`.

Estas variables están declaradas como `evflag`.

**zeromatrix** (*m*, *n*) Función

Devuelve una matriz rectangular *m* por *n* con todos sus elementos iguales a cero.

**"["** Símbolo especial

Los símbolos `[` y `]` marcan el comienzo y final, respectivamente, de una lista.

Los símbolos `[` y `]` también se utilizan para indicar los subíndices de los elementos de una lista, arreglo o función arreglo.

Ejemplos:

```
(%i1) x: [a, b, c];
(%o1) [a, b, c]
(%i2) x[3];
(%o2) c
(%i3) array (y, fixnum, 3);
(%o3) y
(%i4) y[2]: %pi;
(%o4) %pi
(%i5) y[2];
(%o5) %pi
```

```
(%i6) z['foo]: 'bar;
(%o6)
(%i7) z['foo];
(%o7)
(%i8) g[k] := 1/(k^2+1);
(%o8)
 1
g_k := ----
 2
 k + 1

(%i9) g[10];
(%o9)
 1

 101
```



## 27 Afines

### 27.1 Definiciones para Afines

**fast\_linsolve** ( $[expr\_1, \dots, expr\_m], [x\_1, \dots, x\_n]$ ) Función

Resuelve las ecuaciones lineales simultáneas  $expr\_1, \dots, expr\_m$  para las variables  $x\_1, \dots, x\_n$ . Cada  $expr\_i$  puede ser una ecuación o una expresión general; en caso de tratarse de una expresión general, será tratada como una ecuación de la forma  $expr\_i = 0$ .

El valor que devuelve es una lista de ecuaciones de la forma  $[x\_1 = a\_1, \dots, x\_n = a\_n]$  donde todas las  $a\_1, \dots, a\_n$  están exentas de  $x\_1, \dots, x\_n$ .

La función **fast\_linsolve** es más rápida que **linsolve** para sistemas de ecuaciones con coeficientes dispersos.

**grobner\_basis** ( $[expr\_1, \dots, expr\_m]$ ) Función

Devuelve una base de Groebner para las ecuaciones  $expr\_1, \dots, expr\_m$ . La función **polysimp** puede ser entonces utilizada para simplificar otras funciones relativas a las ecuaciones.

```
grobner_basis ([3*x^2+1, y*x])$
```

```
polysimp (y^2*x + x^3*9 + 2) ==> -3*x + 2
```

**polysimp**( $f$ ) alcanza 0 si y sólo si  $f$  está en el ideal generado por  $expr\_1, \dots, expr\_m$ , es decir, si y sólo si  $f$  es una combinación polinómica de los elementos de  $expr\_1, \dots, expr\_m$ .

**set\_up\_dot\_simplifications** ( $eqns, check\_through\_degree$ ) Función

**set\_up\_dot\_simplifications** ( $eqns$ ) Función

Las  $eqns$  son ecuaciones polinómicas de variables no conmutativas. El valor de **current\_variables** es la lista de variables utilizadas para el cálculo de los grados. Las ecuaciones deben ser homogéneas, al objeto de completar el procedimiento.

El grado es el devuelto por **nc\_degree**. Éste a su vez depende de los pesos de las variables individuales.

**declare\_weight** ( $x\_1, w\_1, \dots, x\_n, w\_n$ ) Función

Asigna los pesos  $w\_1, \dots, w\_n$  a  $x\_1, \dots, x\_n$ , respectivamente. Estos pesos son los utilizados en el cálculo de **nc\_degree**.

**nc\_degree** ( $p$ ) Función

Devuelve el grado de un polinomio no conmutativo  $p$ . Véase **declare\_weights**.

**dotsimp** ( $f$ ) Función

Devuelve 0 si y sólo si  $f$  está en el ideal generado por las ecuaciones, esto es, si y sólo si  $f$  es una combinación lineal de los elementos de las ecuaciones.

**fast\_central\_elements** ( $[x_1, \dots, x_n], n$ ) Función

Si se ha ejecutado **set\_up\_dot\_simplifications** con antelación, obtiene los polinomios centrales de grado  $n$  de variables  $x_1, \dots, x_n$ .

Por ejemplo:

```
set_up_dot_simplifications ([y.x + x.y], 3);
fast_central_elements ([x, y], 2);
[y.y, x.x];
```

**check\_overlaps** ( $n, add\_to\_simps$ ) Función

Revisa la superposición hasta el grado  $n$ , asegurándose de que el usuario tiene suficientes reglas de simplificación en cada grado para que **dotsimp** trabaje correctamente. Este proceso puede acelerarse si se conoce de antemano cuál es la dimensión del espacio de monomios. Si éste es de dimensión global finita, entonces debería usarse **hilbert**. Si no se conoce la dimensiones de los monomios, no se debería especificar una **rank\_function**. Un tercer argumento opcional es **reset**.

**mono** ( $[x_1, \dots, x_n], n$ ) Función

Devuelve la lista de monomios independientes.

**monomial\_dimensions** ( $n$ ) Función

Calcula el desarrollo de Hilbert de grado  $n$  para el algebra actual.

**extract\_linear\_equations** ( $[p_1, \dots, p_n], [m_1, \dots, m_n]$ ) Función

Hace una lista de los coeficientes de los polinomios no conmutativos  $p_1, \dots, p_n$  de los monomios no conmutativos  $m_1, \dots, m_n$ . Los coeficientes deben escalares. Hágase uso de **list\_nc\_monomials** para construir la lista de monomios.

**list\_nc\_monomials** ( $[p_1, \dots, p_n]$ ) Función

**list\_nc\_monomials** ( $p$ ) Función

Devuelve una lista de los monomios no conmutativos que aparecen en el polinomio  $p$  o una lista de polinomios en  $p_1, \dots, p_n$ .

**all\_dotsimp\_denoms** Variable

Valor por defecto: **false**

Cuando **all\_dotsimp\_denoms** es una lista, los denominadores encontrados por **dotsimp** son añadidos a la lista. La variable **all\_dotsimp\_denoms** puede inicializarse como una lista vacía `[]` antes de llamar a **dotsimp**.

Por defecto, **dotsimp** no recolecta los denominadores.

## 28 itensor

### 28.1 Introducción a itensor

Maxima implementa dos tipos diferentes de manipulación simbólica de tensores: la manipulación de componentes (paquete `ctensor`) y la manipulación indexada (paquete `itensor`).

Véase más abajo la nota sobre 'notación tensorial'.

La manipulación de componentes significa que los objetos geométricos tensoriales se representan como arreglos (arrays) o matrices. Operaciones tensoriales como la contracción o la diferenciación covariante se llevan a cabo sumando índices mudos con la sentencia `do`. Esto es, se realizan operaciones directamente con las componentes del tensor almacenadas en un arreglo o matriz.

La manipulación indexada de tensores se lleva a cabo mediante la representación de los tensores como funciones de sus índices covariantes, contravariantes y de derivadas. Operaciones tensoriales como la contracción o la diferenciación covariante se realizan manipulando directamente los índices, en lugar de sus componentes asociadas.

Estas dos técnicas para el tratamiento de los procesos diferenciales, algebraicos y analíticos en el contexto de la geometría riemanniana tienen varias ventajas y desventajas que surgen según la naturaleza y dificultad del problema que está abordando el usuario. Sin embargo, se deben tener presentes las siguientes características de estas dos técnicas:

La representación de los tensores y sus operaciones en términos de sus componentes facilita el uso de paquete `ctensor`. La especificación de la métrica y el cálculo de los tensores inducidos e invariantes es inmediato. Aunque toda la potencia de simplificación de Maxima se encuentra siempre a mano, una métrica compleja con dependencias funcionales y de coordenadas intrincada, puede conducir a expresiones de gran tamaño en las que la estructura interna quede oculta. Además, muchos cálculos requieren de expresiones intermedias que pueden provocar la detención súbita de los programas antes de que se termine el cálculo. Con la experiencia, el usuario podrá evitar muchas de estas dificultades.

Debido a la forma en que los tensores y sus operaciones se representan en términos de operaciones simbólicas con sus índices, expresiones que serían intratables en su representación por componentes pueden en ocasiones simplificarse notablemente utilizando las rutinas especiales para objetos simétricos del paquete `itensor`. De esta manera, la estructura de expresiones grandes puede hacerse más transparente. Por otro lado, debido a la forma especial de la representación indexada de tensores en `itensor`, en algunos casos el usuario encontrará dificultades con la especificación de la métrica o la definición de funciones.

#### 28.1.1 Notación tensorial

Hasta ahora, el paquete `itensor` de Maxima utilizaba una notación que algunas veces llevaba a una ordenación incorrecta de los índices. Por ejemplo:

```
(%i2) imetric(g);
(%o2)
done
(%i3) ishow(g([], [j,k])*g([], [i,l])*a([i,j], []))$
 i l j k
```

```
(%t3)
 g g a
 i j

(%i4) ishow(contract(%))$
 k l
 a

(%t4)
```

Este resultado no es correcto a menos que **a** sea un tensor simétrico. La razón por la que esto ocurre es que aunque **itensor** mantenga correctamente el orden dentro del conjunto de índices covariantes y contravariantes, una vez un índice sea aumentado o disminuido, su posición relativa al otro conjunto de índices se pierde.

Para evitar este problema, se ha desarrollado una notación totalmente compatible con la anterior. En esta notación, los índices contravariantes se insertan en las posiciones correctas en la lista de índices covariantes, pero precedidos del signo negativo.

En esta notación, el ejemplo anterior da el resultado correcto:

```
(%i5) ishow(g([-j,-k],[])*g([-i,-l],[])*a([i,j],[]))$
 i l j k
 g a g
 i j

(%i6) ishow(contract(%))$
 l k
 a

(%t6)
```

El único código que hace uso de esta notación es la función **lc2kdt**.

Debido a que este código es nuevo, puede contener errores.

### 28.1.2 Manipulación indexada de tensores

El paquete **itensor** se carga haciendo **load(itensor)**. Para acceder a las demos se hará **demo(tensor)**.

En el paquete **itensor** un tensor se representa como un objeto indexado, esto es, como una función de tres grupos de índices: los covariantes, los contravariantes y los de derivadas. Los índices covariantes se especifican mediante una lista que será el primer argumento del objeto indexado, siendo los índices contravariantes otra lista que será el segundo argumento del mismo objeto indexado. Si al objeto indexado le falta cualquiera de estos grupos de índices, entonces se le asignará al argumento correspondiente la lista vacía **[]**. Así, **g([a,b],[c])** representa un objeto indexado llamado **g**, el cual tiene dos índices covariantes (**a,b**), un índice contravariante (**c**) y no tiene índices de derivadas.

Los índices de derivadas, si están presentes, se añaden como argumentos adicionales a la función simbólica que representa al tensor. Se pueden especificar explícitamente por el usuario o pueden crearse durante el proceso de diferenciación respecto de alguna coordenada. Puesto que la diferenciación ordinaria es conmutativa, los índices de derivadas se ordenan alfabéticamente, a menos que la variable **iframe\_flag** valga **true**, indicando que se está utilizando una métrica del sistema de referencia. Esta ordenación canónica hace posible que Maxima reconozca, por ejemplo, que **t([a],[b],i,j)** es lo mismo que **t([a],[b],j,i)**. La diferenciación de un objeto indexado con respecto de alguna coordenada cuyo índice no aparece como argumento de dicho objeto indexado, dará como resultado cero. Esto se debe a que Maxima no sabe si el tensor representado por el objeto indexado depende implícitamente de la coordenada correspondiente. Modificando la función **diff** de Maxima

en `itensor`, se da por hecho que todos los objetos indexados dependen de cualquier variable de diferenciación, a menos que se indique lo contrario. Esto hace posible que la convención sobre la sumación se extienda a los índices de derivadas. El paquete `itensor` trata a los índices de derivadas como covariantes.

Las siguientes funciones forman parte del paquete `itensor` para la manipulación indexada de vectores. En lo que respecta a las rutinas de simplificación, no se considera en general que los objetos indexados tengan propiedades simétricas. Esto puede cambiarse reasignando a la variable `allsym[false]` el valor `true`, con lo cual los objetos indexados se considerarán simétricos tanto respecto de sus índices covariantes como contravariantes.

En general, el paquete `itensor` trata a los tensores como objetos opacos. Las ecuaciones tensoriales se manipulan en base a reglas algebraicas, como la simetría y la contracción. Además, en el paquete `itensor` hay funciones para la diferenciación covariante, la curvatura y la torsión. Los cálculos se pueden realizar respecto de una métrica del sistema de referencia móvil, dependiendo de las asignaciones dadas a la variable `iframe_flag`.

La siguiente sesión de ejemplo demuestra cómo cargar el paquete `itensor`, especificar el nombre de la métrica y realizar algunos cálculos sencillos.

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) imetric(g);
(%o2) done
(%i3) components(g([i,j],[]),p([i,j],[])*e([],[]))$
(%i4) ishow(g([k,l],[]))$
(%t4)
 e p
 k l

(%i5) ishow(diff(v([i],[]),t))$
(%t5)
 0

(%i6) depends(v,t);
(%o6)
 [v(t)]

(%i7) ishow(diff(v([i],[]),t))$
(%t7)
 d
 -- (v)
 dt i

(%i8) ishow(idiff(v([i],[]),j))$
(%t8)
 v
 i,j

(%i9) ishow(extdiff(v([i],[]),j))$
(%t9)
 v - v
 j,i i,j

 2

(%i10) ishow(liediff(v,w([i],[])))$
(%t10)
 %3 %3
 v w + v w
 i,%3 ,i %3

(%i11) ishow(covdiff(v([i],[]),j))$
(%t11)
 %4
 v - v i chr2
 i,j %4 i j
```



```

(%i12) ishow(ev(%,ichr2))$
 %4 %5
(%t12) v - g v (e p + e p - e p - e p
 i,j %4 j %5,i ,i j %5 i j,%5 ,%5 i j
 + e p + e p)/2
 i %5,j ,j i %5

(%i13) iframe_flag:true;
(%o13)
(%i14) ishow(covdiff(v([i],[]),j))$
 true
 %6
(%t14) v - v icc2
 i,j %6 i j
(%i15) ishow(ev(%,icc2))$
 %6
(%t15) v - v ifc2
 i,j %6 i j
(%i16) ishow(radcan(ev(%,ifc2,ifc1)))$
 %6 %8 %6 %8
(%t16) - (ifg v ifb + ifg v ifb - 2 v
 %6 j %8 i %6 i j %8 i,j
 %6 %8
 - ifg v ifb)/2
 %6 %8 i j

(%i17) ishow(canform(s([i,j],[])-s([j,i])))$
(%t17) s - s
 i j j i
(%i18) decsym(s,2,0,[sym(all)],[]);
(%o18) done
(%i19) ishow(canform(s([i,j],[])-s([j,i])))$
(%t19) 0
(%i20) ishow(canform(a([i,j],[])+a([j,i])))$
(%t20) a + a
 j i i j
(%i21) decsym(a,2,0,[anti(all)],[]);
(%o21) done
(%i22) ishow(canform(a([i,j],[])+a([j,i])))$
(%t22) 0

```

## 28.2 Definiciones para itensor

### 28.2.1 Trabajando con objetos indexados

**entertensor** (*nombre*)

Función

Permite crear un objeto indexado llamado *nombre*, con cualquier número de índices tensoriales y de derivadas. Se admiten desde un único índice hasta una lista de índices. Véase el ejemplo en la descripción de `covdiff`.

**changename** (*anterior, nuevo, expr*)

Función

Cambia el nombre de todos los objetos indexados llamados *anterior* a *new* en *expr*. El argumento *anterior* puede ser un símbolo o una lista de la forma [*nombre, m, n*], en cuyo caso sólo los objetos indexados de llamados *nombre* con *m* índices covariantes y *n* contravariantes se renombrarán como *nuevo*.

**listoftens**

Función

Hace un listado de todos los tensores y sus índices en una expresión tensorial. Por ejemplo,

```
(%i6) ishow(a([i,j],[k])*b([u],[v])+c([x,y],[])*d([],[])*e)$
 k
(%t6) d e c + a b
 x y i j u,v
(%i7) ishow(listoftens(%))$
 k
(%t7) [a , b , c , d]
 i j u,v x y
```

**ishow** (*expr*)

Función

Muestra *expr* con todos los objetos indexados que contiene, junto con los correspondientes índices covariantes (como subíndices) y contravariantes (como superíndices). Los índices de derivadas se muestran como subíndices, separados de los covariantes por una coma; véanse los múltiples ejemplos de este documento.

**indices** (*expr*)

Función

Devuelve una lista con dos elementos. El primer elemento es una lista con los índices libres, aquellos que aparecen una sola vez. El segundo elemento es una lista con los índices mudos en *expr*, aquellos que aparecen exactamente dos veces. Por ejemplo,

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) ishow(a([i,j],[k,l],m,n)*b([k,o],[j,m,p],q,r))$
 k l j m p
(%t2) a b
 i j,m n k o,q r
(%i3) indices(%);
(%o3) [[l, p, i, n, o, q, r], [k, j, m]]
```

Un producto tensorial que contenga el mismo índice más de dos veces es sintácticamente incorrecto. La función **indices** intenta tratar estas expresiones de una forma razonable; sin embargo, cuando se la obliga a manipular una expresión incorrecta puede tener un comportamiento imprevisto.

**rename** (*expr*) Función  
**rename** (*expr*, *count*) Función

Devuelve una expresión equivalente a *expr* pero con los índices mudos de cada término elegidos del conjunto [%1, %2, ...] si el segundo argumento opcional se omite. En otro caso, los índices mudos son indexados empezando con el valor *count*. Cada índice mudo en un producto será diferente. En el caso de las sumas, la función **rename** operará sobre cada término de la suma reiniciando el contador con cada término. De esta manera **rename** puede servir como simplificador tensorial. Además, los índices se ordenarán alfanuméricamente, si la variable **allsym** vale **true**, respecto de los índices covariantes y contravariantes dependiendo del valor de **flipflag**. Si **flipflag** vale **false**, entonces los índices se renombrarán de acuerdo con el orden de los índices contravariantes. Si **flipflag** vale **true**, entonces los índices se renombrarán de acuerdo con el orden de los índices covariantes. Suele acontecer que el efecto combinado de los dos cambios de nombre reduzcan la expresión más de lo que que pueda reducir cualquiera de ellas por separado.

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) allsym:true;
(%o2) true
(%i3) g([], [%4,%5])*g([], [%6,%7])*ichr2([%1,%4], [%3])*
ichr2([%2,%3], [u])*ichr2([%5,%6], [%1])*ichr2([%7,r], [%2])-
g([], [%4,%5])*g([], [%6,%7])*ichr2([%1,%2], [u])*
ichr2([%3,%5], [%1])*ichr2([%4,%6], [%3])*ichr2([%7,r], [%2]),noeval$
(%i4) expr:ishow(%)$

 %4 %5 %6 %7 %3 u %1 %2
(%t4) g g ichr2 ichr2 ichr2 ichr2
 %1 %4 %2 %3 %5 %6 %7 r

 %4 %5 %6 %7 u %1 %3 %2
 - g g ichr2 ichr2 ichr2 ichr2
 %1 %2 %3 %5 %4 %6 %7 r ■

(%i5) flipflag:true;
(%o5) true
(%i6) ishow(rename(expr))$
 %2 %5 %6 %7 %4 u %1 %3
(%t6) g g ichr2 ichr2 ichr2 ichr2
 %1 %2 %3 %4 %5 %6 %7 r

 %4 %5 %6 %7 u %1 %3 %2
 - g g ichr2 ichr2 ichr2 ichr2
 %1 %2 %3 %4 %5 %6 %7 r ■

(%i7) flipflag:false;
(%o7) false
(%i8) rename(%th(2));
(%o8) 0
(%i9) ishow(rename(expr))$
```

$$\begin{array}{cccccccc}
& \%1 & \%2 & \%3 & \%4 & & \%5 & & \%6 & & \%7 & & u \\
(\%t9) & g & & g & & \text{ichr2} & & \text{ichr2} & & \text{ichr2} & & \text{ichr2} & \\
& & & & & \%1 & \%6 & & \%2 & \%3 & & \%4 & r & \%5 & \%7 \\
& & & & & & & & & & & & & & \\
& & \%1 & \%2 & \%3 & \%4 & & \%6 & & \%5 & & \%7 & & u \\
- & g & & g & & \text{ichr2} & & \text{ichr2} & & \text{ichr2} & & \text{ichr2} & & \\
& & & & & \%1 & \%3 & & \%2 & \%6 & & \%4 & r & \%5 & \%7
\end{array}$$

**flipflag**

Variable opcional

Valor por defecto: `false`

Si vale `false` los índices se renombrarán de acuerdo con el orden de los índices covariantes, si `true` se renombrarán de acuerdo con el orden de los índices contravariantes.

Si `flipflag` vale `false`, entonces `rename` construye una lista con los índices contravariantes según van apareciendo de izquierda a derecha; si vale `true`, entonces va formando la lista con los covariantes. Al primer índice mudo se le da el nombre `%1`, al siguiente `%2`, etc. Finalmente se hace la ordenación. Véase el ejemplo en la descripción de la función `rename`.

**defcon** (*tensor\_1*)

Función

**defcon** (*tensor\_1*, *tensor\_2*, *tensor\_3*)

Función

Le asigna a *gives tensor\_1* la propiedad de que la contracción de un producto de *tensor\_1* por *tensor\_2* da como resultado un *tensor\_3* con los índices apropiados. Si sólo se aporta un argumento, *tensor\_1*, entonces la contracción del producto de *tensor\_1* por cualquier otro objeto indexado que tenga los índices apropiados, por ejemplo `my_tensor`, dará como resultado un objeto indexado con ese nombre, `my_tensor`, y con un nuevo conjunto de índices que reflejen las contracciones realizadas. Por ejemplo, si `imetric:g`, entonces `defcon(g)` implementará el aumento o disminución de los índices a través de la contracción con el tensor métrico. Se puede dar más de un `defcon` para el mismo objeto indexado, aplicándose el último. La variable `contractions` es una lista con aquellos objetos indexados a los que se le han dado propiedades de contracción con `defcon`.

**remcon** (*tensor\_1*, ..., *tensor\_n*)

Función

**remcon** (*all*)

Función

Borra todas las propiedades de contracción de *tensor\_1*, ..., *tensor\_n*). La llamada `remcon(all)` borra todas las propiedades de contracción de todos los objetos indexados.

**contract** (*expr*)

Función

Lleva a cabo las contracciones tensoriales en *expr*, la cual puede ser cualquier combinación de sumas y productos. Esta función utiliza la información dada a la función `defcon`. Para obtener mejores resultados, *expr* debería estar completamente expandida. La función `ratexpand` es la forma más rápida de expandir productos y potencias de sumas si no hay variables en los denominadores de los términos.

**indexed\_tensor** (*tensor*)

Función

Debe ejecutarse antes de asignarle componentes a un *tensor* para el que ya existe un valor, como *ichr1*, *ichr2* o *icurvature*. Véase el ejemplo de la descripción de *icurvature*.

**components** (*tensor*, *expr*)

Función

Permite asignar un valor indexado a la expresión *expr* dando los valores de las componentes de *tensor*. El tensor debe ser de la forma  $t([\dots], [\dots])$ , donde cualquiera de las listas puede estar vacía. La expresión *expr* puede ser cualquier objeto indexado que tenga otros objetos con los mismos índices libres que *tensor*. Cuando se utiliza para asignar valores al tensor métrico en el que las componentes contengan índices mudos, se debe tener cuidado en no generar índices mudos múltiples. Se pueden borrar estas asignaciones con la función **remcomps**.

Es importante tener en cuenta que **components** controla la valencia del tensor, no el orden de los índices. Así, asignando componentes de la forma  $x([i, -j], [])$ ,  $x([-j, i], [])$  o  $x([i], [j])$  todos ellos producen el mismo resultado, la asignación de componentes a un tensor de nombre *x* con valencia (1,1).

Las componentes se pueden asignar a una expresión indexada de cuatro maneras, dos de las cuales implican el uso de la instrucción **components**:

1) Como una expresión indexada. Por ejemplo:

```
(%i2) components(g([], [i, j]), e([], [i]) * p([], [j]))$
(%i3) ishow(g([], [i, j]))$

(%t3) i j
 e p
```

2) Como una matriz:

```
(%i6) components(g([i, j], []), lg);
(%o6) done
(%i7) ishow(g([i, j], []))$
(%t7) g
 i j

(%i8) g([3, 3], []);
(%o8) 1

(%i9) g([4, 4], []);
(%o9) - 1
```

3) Como una función. Se puede utilizar una función de Maxima para especificar las componentes de un tensor en base a sus índices. Por ejemplo, el código siguiente asigna *kdelta* a *h* si *h* tiene el mismo número de índices covariantes y contravariantes y no tiene índices de derivadas, asignándole *g* en otro caso:

```
(%i4) h(l1, l2, [l3]) := if length(l1) = length(l2) and length(l3) = 0
then kdelta(l1, l2) else apply(g, append([l1, l2], l3))$
(%i5) ishow(h([i], [j]))$
```

```

(%t5)
 j
 kdelta
 i

(%i6) ishow(h([i,j],[k],1))$

 k
 g
 i j,1

```

4) Utilizando los patrones de Maxima, en particular las funciones `defrule` y `applyb1`:

```

(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) matchdeclare(l1,listp);
(%o2) done
(%i3) defrule(r1,m(l1,[]),(i1:idummy(),
 g([l1[1],l1[2]],[])*q([i1],[])*e([],[i1])))$

(%i4) defrule(r2,m([],l1),(i1:idummy(),
 w([],[l1[1],l1[2]])*e([i1],[])*q([],[i1])))$

(%i5) ishow(m([i,n],[])*m([],[i,m]))$
 i m
 m m
 i n

(%i6) ishow(rename(applyb1(% ,r1,r2)))$
 %1 %2 %3 m
 e q w q e g
 %1 %2 %3 n

```

### **remcomps** (*tensor*)

Función

Borra todos los valores de *tensor* que han sido asignados con la función `components`.

### **showcomps** (*tensor*)

Función

Muestra las componentes de un tensor definidas con la instrucción `components`. Por ejemplo:

```

(%i1) load(ctensor);
(%o1) /share/tensor/ctensor.mac
(%i2) load(itensor);
(%o2) /share/tensor/itensor.lisp
(%i3) lg:matrix([sqrt(r/(r-2*m)),0,0,0],[0,r,0,0],
 [0,0,sin(theta)*r,0],[0,0,0,sqrt((r-2*m)/r)]);
 r
 [sqrt(-----) 0 0 0]
 [r - 2 m]
 []
 []

```



En un abuso de la notación, `itensor` también permite a `kdelta` tener 2 índices covariantes y ninguno contravariante, o 2 contravariantes y ninguno covariante. Esto es una funcionalidad del paquete, lo que no implica que `kdelta([i,j],[ ])` sea un objeto tensorial de pleno derecho.

### **kdels** (*L1, L2*)

Función

Función delta de Kronecker simetrizada, utilizada en algunos cálculos. Por ejemplo:

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) kdelta([1,2],[2,1]);
(%o2)
- 1
(%i3) kdels([1,2],[2,1]);
(%o3)
1
(%i4) ishow(kdelta([a,b],[c,d]))$
 c d d c
(%t4) kdelta kdelta - kdelta kdelta
 a b a b
(%i4) ishow(kdels([a,b],[c,d]))$
 c d d c
(%t4) kdelta kdelta + kdelta kdelta
 a b a b
```

### **levi\_civita** (*L*)

Función

Es el tensor de permutación de Levi-Civita, el cual devuelve 1 si la lista *L* con una permutación par de enteros, -1 si es en una permutación impar y 0 si algunos de los índices de *L* están repetidos.

### **lc2kdt** (*expr*)

Función

Simplifica expresiones que contengan el símbolo de Levi-Civita, convirtiéndolas en expresiones con la delta de Kronecker siempre que sea posible. La diferencia principal entre esta función y la simple evaluación del símbolo de Levi-Civita consiste en que de esta última forma se obtienen expresiones de Kronecker con índices numéricos, lo que impide simplificaciones ulteriores. La función `lc2kdt` evita este problema, dando resultados con son más fáciles de simplificar con `rename` o `contract`.

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) expr:ishow('levi_civita([],[i,j])*levi_civita([k,l],[])*a([j],[k]))$
 i j k
(%t2) levi_civita a levi_civita
 j k l
(%i3) ishow(ev(expr,levi_civita))$
 i j k 1 2
(%t3) kdelta a kdelta
 1 2 j k l
(%i4) ishow(ev(%,kdelta))$
```



```

(%t4) (kdeltai1 kdeltaj2 - kdeltaj1 kdeltai2) akj

 1 2 2 1
 (kdelta1k kdelta21 - kdelta2k kdelta11)

(%i5) ishow(lc2kdt(expr))$
 k i j k j i
(%t5) a kdeltakj kdeltaik - a kdeltajk kdeltaij
(%i6) ishow(contract(expand(%)))$
 i i
(%t6) ai1 - ai1 kdelta11

```

La función `lc2kdt` en ocasiones hace uso del tensor métrico. Si el tensor métrico no fue previamente definido con `imetric`, se obtiene un mensaje de error.

```

(%i7) expr:ishow('levi_civita([], [i,j])*'levi_civita([], [k,l])*a([j,k], []))$
 i j k l
(%t7) levi_civita levi_civita aj k

(%i8) ishow(lc2kdt(expr))$
Maxima encountered a Lisp error:

Error in $IMETRIC [or a callee]:
$IMETRIC [or a callee] requires less than two arguments.

Automatically continuing.
To reenale the Lisp debugger set *debugger-hook* to nil.
(%i9) imetric(g);
(%o9) done
(%i10) ishow(lc2kdt(expr))$
 %3 i k %4 j 1 %3 i 1 %4 j k
(%t10) (g%3i kdeltakj g%4j kdelta11 - g%3i kdelta1k g%4j kdeltakj) akj
 %3 %4 %3 %4
(%i11) ishow(contract(expand(%)))$
 1 i 1 i
(%t11) a1 i1 - a1 i1 gi1

```

## lc\_1

## Función

Regla de simplificación utilizada en expresiones que contienen el símbolo de `levi_civita` sin evaluar. Junto con `lc_u`, puede utilizarse para simplificar muchas expresiones de forma más eficiente que la evaluación de `levi_civita`. Por ejemplo:

```
(%i1) load(itensor);
```

```
(%o1) /share/tensor/itensor.lisp
(%i2) el1:ishow('levi_civita([i,j,k],[i])*a([i])*a([j]))$
 i j
 a a levi_civita
 i j k
(%i3) el2:ishow('levi_civita([i,j,k])*a([i])*a([j]))$
 i j k
 levi_civita a a
 i j
(%i4) ishow(canform(contract(expand(applyb1(el1,lc_l,lc_u))))))$
(%t4) 0
(%i5) ishow(canform(contract(expand(applyb1(el2,lc_l,lc_u))))))$
(%t5) 0
```

**lc\_u**

Función

Regla de simplificación utilizada en expresiones que contienen el símbolo de `levi_civita` sin evaluar. Junto con `lc_l`, puede utilizarse para simplificar muchas expresiones de forma más eficiente que la evaluación de `levi_civita`. Véase `lc_l`.

**canten** (*expr*)

Función

Simplifica *expr* renombrando (véase `rename`) y permutando índices mudos. La función `rename` se restringe a sumas de productos de tensores en los cuales no hay derivadas, por lo que está limitada y sólo debería utilizarse si `canform` no es capaz de llevar a cabo la simplificación requerida.

La función `canten` devuelve un resultado matemáticamente correcto sólo si su argumento es una expresión completamente simétrica respecto de sus índices. Por esta razón, `canten` devuelve un error si `allsym` no vale `true`.

**concan** (*expr*)

Función

Similar a `canten` pero también realiza la contracción de los índices.

**28.2.2 Simetrías de tensores****allsym**

Variable opcional

Valor por defecto: `false`

Si vale `true` entonces todos los objetos indexados se consideran simétricos respecto de todos sus índices covariantes y contravariantes. Si vale `false` entonces no se tienen en cuenta ningún tipo de simetría para estos índices. Los índices de derivadas se consideran siempre simétricos, a menos que la variable `iframe_flag` valga `true`.

**decsym** (*tensor*, *m*, *n*, [*cov\_1*, *cov\_2*, ...], [*contr\_1*, *contr\_2*, ...])

Función

Declara propiedades de simetría para el *tensor* de *m* índices covariantes y *n* contravariantes. Los *cov\_i* y *contr\_i* son seudofunciones que expresan relaciones de simetría entre los índices covariantes y contravariantes, respectivamente. Éstos son de la forma `symoper(index_1, index_2, ...)` donde `symoper` es uno de `sym`, `anti` o `cyc` y los *index\_i* son enteros que indican la posición del índice en el *tensor*. Esto declarará a *tensor*

simétrico, antisimétrico o cíclico respecto de *index\_i*. La llamada `symoper(all)` indica que todos los índices cumplen la condición de simetría. Por ejemplo, dado un objeto `b` con 5 índices covariantes, `decsym(b,5,3,[sym(1,2),anti(3,4)],[cyc(all)])` declara `b` simétrico en el primer y segundo índices covariantes, antisimétrico en su tercer y cuarto índices también covariantes y cíclico en todos sus índices contravariantes. Cualquiera de las listas de declaración de simetrías puede ser nula. La función que realiza las simplificaciones es `canform`, como se ilustra en el siguiente ejemplo,

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) expr:contract(expand(a([i1,j1,k1],[])*kdels([i,j,k],[i1,j1,k1])))$
(%i3) ishow(expr)$
(%t3)
 a + a + a + a + a + a
 k j i k i j j k i j i k i k j i j k
(%i4) decsym(a,3,0,[sym(all)],[]);
(%o4) done
(%i5) ishow(canform(expr))$
(%t5)
 6 a
 i j k

(%i6) remsym(a,3,0);
(%o6) done
(%i7) decsym(a,3,0,[anti(all)],[]);
(%o7) done
(%i8) ishow(canform(expr))$
(%t8)
 0

(%i9) remsym(a,3,0);
(%o9) done
(%i10) decsym(a,3,0,[cyc(all)],[]);
(%o10) done
(%i11) ishow(canform(expr))$
(%t11)
 3 a + 3 a
 i k j i j k

(%i12) dispSYM(a,3,0);
(%o12) [[cyc, [[1, 2, 3]], []]]
```

### **remsym** (*tensor*, *m*, *n*)

Función

Borra todas las propiedades de simetría del *tensor* que tiene *m* índices covariantes y *n* contravariantes.

### **canform** (*expr*)

Función

Simplifica *expr* renombrando índices mudos y reordenando todos los índices según las condiciones de simetría que se le hayan impuesto. Si `allsym` vale `true` entonces todos los índices se consideran simétricos, en otro caso se utilizará la información sobre simetrías suministrada por `decsym`. Los índices mudos se renombran de la misma manera que en la función `rename`. Cuando `canform` se aplica a una expresión grande el cálculo puede llevar mucho tiempo. Este tiempo se puede acortar llamando primero a `rename`. Véase también el ejemplo de la descripción de `decsym`. La función `canform`

puede que no reduzca completamente una expresión a su forma más sencilla, pero en todo caso devolverá un resultado matemáticamente correcto.

### 28.2.3 Cálculo tensorial indexado

**diff** (*expr*, *v\_1*, [*n\_1*, [*v\_2*, *n\_2*] ...]) Función

Se trata de la función de Maxima para la diferenciación, ampliada para las necesidades del paquete **itensor**. Calcula la derivada de *expr* respecto de *v\_1* *n\_1* veces, respecto de *v\_2* *n\_2* veces, etc. Para el paquete de tensores, la función ha sido modificada de manera que *v\_i* puedan ser enteros desde 1 hasta el valor que tome la variable **dim**. Esto permite que la derivación se pueda realizar con respecto del *v\_i*-ésimo miembro de la lista **vect\_coords**. Si **vect\_coords** guarda una variable atómica, entonces esa variable será la que se utilice en la derivación. Con esto se hace posible la utilización de una lista con nombres de coordenadas subindicadas, como **x[1]**, **x[2]**, ...

**idiff** (*expr*, *v\_1*, [*n\_1*, [*v\_2*, *n\_2*] ...]) Función

Diferenciación inicial. Al contrario que **diff**, que deriva respecto de una variable independiente, **idiff** puede usarse para derivar respecto de una coordenada.

La función **idiff** también puede derivar el determinante del tensor métrico. Así, si **imetric** toma el valor **G** entonces **idiff(determinant(g),k)** devolverá **2\*determinant(g)\*ichr2([%i,k],[%i])** donde la índice mudo **%i** se escoge de forma apropiada.

**liediff** (*v*, *ten*) Función

Calcula la derivada de Lie de la expresión tensorial *ten* respecto de campo vectorial *v*. La expresión *ten* debe ser cualquier tensor indexado; *v* debe ser el nombre (sin índices) de un campo vectorial. Por ejemplo:

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) ishow(liediff(v,a([i,j],[])*b([],[k],1)))$
 k %2 %2 %2
(%t2) b (v a + v a + v a)
 ,1 i j,%2 ,j i %2 ,i %2 j

 %1 k %1 k %1 k
 + (v b - b v + v b) a
 ,%1 1 ,1 %1 ,1 %1 i j
```

**rediff** (*ten*) Función

Calcula todas las instrucciones **idiff** que aparezcan en la expresión tensorial *ten*.

**undiff** (*expr*) Función

Devuelve una expresión equivalente a *expr* pero con todas las derivadas de los objetos indexados reemplazadas por la forma nominal de la función **idiff**.

**evundiff** (*expr*)

Función

Equivale a **undiff** seguido de **ev** y **rediff**.

La razón de esta operación es evaluar de forma sencilla expresiones que no pueden ser directamente evaluadas en su forma derivada. Por ejemplo, lo siguiente provoca un error:

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) icurvature([i,j,k],[l],m);
Maxima encountered a Lisp error:
```

```
Error in $ICURVATURE [or a callee]:
$ICURVATURE [or a callee] requires less than three arguments.
```

Automatically continuing.

To reenale the Lisp debugger set *\*debugger-hook\** to nil.

Sin embargo, si **icurvature** se da en forma nominal, puede ser evaluada utilizando **evundiff**:

```
(%i3) ishow('icurvature([i,j,k],[l],m))$
 1
(%t3) icurvature
 i j k,m
(%i4) ishow(evundiff(%))$
 1 1 %1 1 %1
(%t4) - ichr2 - ichr2 ichr2 - ichr2 ichr2
 i k,j m %1 j i k,m %1 j,m i k
 1 1 %1 1 %1
 + ichr2 + ichr2 ichr2 + ichr2 ichr2
 i j,k m %1 k i j,m %1 k,m i j
```

**flush** (*expr*, *tensor\_1*, *tensor\_2*, ...)

Función

Iguala a cero en la expresión *expr* todas las apariciones de *tensor\_i* que no tengan índices de derivadas.

**flushd** (*expr*, *tensor\_1*, *tensor\_2*, ...)

Función

Iguala a cero en la expresión *expr* todas las apariciones de *tensor\_i* que tengan índices de derivadas

**flushnd** (*expr*, *tensor*, *n*)

Función

Iguala a cero en *expr* todas las apariciones del objeto diferenciado *tensor* que tenga *n* o más índices de derivadas, como demuestra el siguiente ejemplo:

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) ishow(a([i],[J,r],k,r)+a([i],[j,r,s],k,r,s))$
 J r j r s
(%t2) a + a
```

```
(%i3) ishow(flushnd(%,a,3))$
(%t3)
i,k r i,k r s
J r
a
i,k r
```

**coord** (*tensor\_1*, *tensor\_2*, ...) Función

Le da a *tensor\_i* la propiedad de diferenciación coordenada, que la derivada del vector contravariante cuyo nombre es uno de los *tensor\_i* es igual a la delta de Kronecker. Por ejemplo, si se ha hecho **coord(x)** entonces **idiff(x([],[i]),j)** da **kdelta([i],[j])**. La llamada **coord** devuelve una lista de todos los objetos indexados con esta propiedad.

**remcoord** (*tensor\_1*, *tensor\_2*, ...) Función

**remcoord** (*all*) Función

Borra todas las propiedades de diferenciación coordenada de **tensor\_i** que hayan sido establecidas por la función **coord**. La llamada **remcoord(all)** borra esta propiedad de todos los objetos indexados.

**makebox** (*expr*) Función

Muestra *expr* de la misma manera que lo hace **show**; sin embargo, cualquier tensor de d'Alembert que aparezca en *expr* estará indicado por []. Por ejemplo, **p([m],[n])** representa **g([],[i,j])\*p([m],[n],i,j)**.

**conmetderiv** (*expr*, *tensor*) Función

Simplifica expresiones que contengan derivadas ordinarias tanto de las formas covariantes como contravariantes del tensor métrico. Por ejemplo, **conmetderiv** puede relacionar la derivada del tensor métrico contravariante con los símbolos de Christoffel, como se ve en el ejemplo:

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) ishow(g([],[a,b],c))$
(%t2)
a b
g
,c

(%i3) ishow(conmetderiv(%,g))$
(%t3)
%1 b a %1 a b
- g ichr2 - g ichr2
%1 c %1 c
```

**simpmetderiv** (*expr*) Función

**simpmetderiv** (*expr*, *stop*) Función

Simplifica expresiones que contienen productos de las derivadas del tensor métrico. La función **simpmetderiv** reconoce dos identidades:

$$g_{,d}^{ab} g_{bc} + g_{bc,d}^{ab} = (g_{bc,d}^{ab}) = (\delta_{bc,d}^a) = 0$$

de donde

$$g_{,d}^{ab} g_{bc} = - g_{bc,d}^{ab}$$

y

$$g_{,j}^{ab} g_{ab,i} = g_{,i}^{ab} g_{ab,j}$$

que se deduce de las simetrías de los símbolos de Christoffel.

La función `simpmetderiv` tiene un argumento opcional, el cual detiene la función después de la primera sustitución exitosa en un expresión producto. La función `simpmetderiv` también hace uso de la variable global `flipflag` que determina cómo aplicar una ordenación “canónica” a los índices de los productos.

Todo esto se puede utilizar para conseguir buenas simplificaciones que serían difíciles o imposibles de conseguir, lo que se demuestra en el siguiente ejemplo, que utiliza explícitamente las simplificaciones parciales de `simpmetderiv`:

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) imetric(g);
(%o2) done
(%i3) ishow(g([],[a,b])*g([],[b,c])*g([a,b],[],d)*g([b,c],[],e))$
 a b b c
(%t3) g g g g
 a b,d b c,e
(%i4) ishow(canform(%))$

errexpl has improper indices
-- an error. Quitting. To debug this try debugmode(true);
(%i5) ishow(simpmetderiv(%))$
 a b b c
(%t5) g g g g
 a b,d b c,e
(%i6) flipflag:not flipflag;
(%o6) true
(%i7) ishow(simpmetderiv(%th(2)))$
 a b b c
(%t7) g g g g
 ,d ,e a b b c
```

```

(%i8) flipflag:not flipflag;
(%o8)
(%i9) ishow(simpmetderiv(%th(2),stop))$
 a b b c
(%t9) - g g g g
 ,e a b,d b c

(%i10) ishow(contract(%))$
 b c
(%t10) - g g
 ,e c b,d

```

Véase también `weyl.dem` para un ejemplo que utiliza `simpmetderiv` y `conmetderiv` para simplificar contracciones del tensor de Weyl.

**flush1deriv** (*expr*, *tensor*)

Función

Iguala a cero en *expr* todas las apariciones de *tensor* que tengan exactamente un índice derivado.

## 28.2.4 Tensores en espacios curvos

**imetric** (*g*)

Función

**imetric**

Variable de sistema

Especifica la métrica haciendo la asignación de la variable `imetric:g`, además las propiedades de contracción de la métrica *g* se fijan ejecutando las instrucciones `defcon(g)`, `defcon(g,g,kdelta)`. La variable `imetric`, a la que no se le asigna ningún valor por defecto, tiene el valor de la métrica que se le haya asignado con la instrucción `imetric(g)`.

**idim** (*n*)

Función

Establece las dimensiones de la métrica. También inicializa las propiedades de anti-simetría de los símbolos de Levi-Civita para la dimensión dada.

**ichr1** (*[i, j, k]*)

Función

Devuelve el símbolo de Christoffel de primera especie dado por la definición

$$(g_{ik,j} + g_{jk,i} - g_{ij,k})/2.$$

Para evaluar los símbolos de Christoffel de una métrica determinada, a la variable `imetric` hay que asignarle un nombre como en el ejemplo de la descripción de `chr2`.

**ichr2** (*[i, j], [k]*)

Función

Devuelve el símbolo de Christoffel de segunda especie dado por la definición

$$\text{ichr2}([i, j], [k]) = g^{ks} (g_{is,j} + g_{js,i} - g_{ij,s})/2$$



**icurvature** ( $[i, j, k], [h]$ )

Función

Devuelve el tensor de curvatura de Riemann en términos de los símbolos de Christoffel de segunda especie (*ichr2*). Se utiliza la siguiente notación:

$$\text{icurvature}_{i j k}^h = - \text{ichr2}_{i k, j}^h - \text{ichr2}_{i j, k}^h + \text{ichr2}_{i k}^{\%1} + \text{ichr2}_{i j, k}^h \\ + \text{ichr2}_{\%1 k}^h + \text{ichr2}_{i j}^{\%1}$$

**covdiff** (*expr*, *v\_1*, *v\_2*, ...)

Función

Devuelve la derivada covariante de *expr* respecto de las variables *v\_i* en términos de los símbolos de Christoffel de segunda especie (*ichr2*). Para evaluarlos debe hacerse *ev(expr, ichr2)*.

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) entertensor()$
Enter tensor name: a;
Enter a list of the covariant indices: [i,j];
Enter a list of the contravariant indices: [k];
Enter a list of the derivative indices: [];

(%t2)
 k
 a
 i j

(%i3) ishow(covdiff(%,s))$
 k %1 k %1 k k %1
(%t3) - a ichr2 - a ichr2 + a + ichr2 a
 i %1 j s %1 j i s i j, s %1 s i j

(%i4) imetric:g;
(%o4) g
(%i5) ishow(ev(%th(2),ichr2))$
%1 %4 k
g a (g - g + g)
i %1 s %4, j j s, %4 j %4, s
(%t5) -----
 2
 %1 %3 k
 g a (g - g + g)
 %1 j s %3, i i s, %3 i %3, s

 2
 k %2 %1
 g a (g - g + g)
 i j s %2, %1 %1 s, %2 %1 %2, s k
+ ----- + a
 2 i j, s
```

**lorentz\_gauge** (*expr*)

Función

Impone la condición de Lorentz sustituyendo por 0 todos los objetos indexados de *expr* que tengan un índice derivado idéntico a un índice contravariante.

**igeodesic\_coords** (*expr*, *nombre*)

Función

Elimina los símbolos no diferenciados de Christoffel y las primeras derivadas del tensor métrico de *expr*. El argumento *nombre* de la función **igeodesic\_coords** se refiere a la métrica *nombre* si aparece en *expr*, mientras que los coeficientes de conexión deben tener los nombres *ichr1* y/o *ichr2*. El siguiente ejemplo hace la verificación de la identidad cíclica satisfecha por el tensor de curvatura de Riemann haciendo uso de la función **igeodesic\_coords**.

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) ishow(icurvature([r,s,t],[u]))$
 u u %1 u u %1
(%t2) - ichr2 - ichr2 ichr2 + ichr2 + ichr2 ichr2
 r t,s %1 s r t r s,t %1 t r s
(%i3) ishow(igeodesic_coords(%,ichr2))$
 u u
(%t3) ichr2 - ichr2
 r s,t r t,s
(%i4) ishow(igeodesic_coords(icurvature([r,s,t],[u]),ichr2)+
 igeodesic_coords(icurvature([s,t,r],[u]),ichr2)+
 igeodesic_coords(icurvature([t,r,s],[u]),ichr2))$
 u u u u u
(%t4) - ichr2 + ichr2 + ichr2 - ichr2 - ichr2
 t s,r t r,s s t,r s r,t r t,s
 u
 + ichr2
 r s,t
(%i5) canform(%);
(%o5) 0
```

**28.2.5 Sistemas de referencia móviles**

Maxima puede hacer cálculos utilizando sistemas de referencia móviles, los cuales pueden ser ortonormales o cualesquiera otros.

Para utilizar sistemas de referencia, primero se debe asignar a la variable *iframe\_flag* el valor **true**. Con esto se hace que los símbolos de Christoffel, *ichr1* y *ichr2*, sean reemplazados por los coeficientes *icc1* y *icc2* en los cálculos, cambiando así el comportamiento de **covdiff** y **icurvature**.

El sistema de referencia se define con dos tensores: el campo del sistema de referencia inverso (*ifri*, la base dual tetrad) y la métrica del sistema de referencia *ifg*. La métrica del sistema de referencia es la matriz identidad en los sistemas de referencia ortonormales, o la métrica de Lorentz en sistemas de referencia ortonormales en el espacio-tiempo de

Minkowski. El campo del sistema de referencia inverso define la base del sistema de referencia con vectores unitarios. Las propiedades contractivas se definen para el campo y la métrica del sistema de referencia.

Si `iframe_flag` vale `true`, muchas expresiones de `itensor` utilizan la métrica `ifg` en lugar de la métrica definida por `imetric` para incrementar y reducir índices.

**IMPORTANTE:** Asignando a la variable `iframe_flag` el valor `true` NO deshace las propiedades contractivas de una métrica establecidas con una llamada a `defcon` o a `imetric`. Si se utiliza el campo del sistema de referencia, es mejor definir la métrica asignando su nombre a la variable `imetric` y NO hacer una llamada a la función `imetric`.

Maxima utiliza estos dos tensores para definir los coeficientes del sistema de referencia: `ifc1` y `ifc2`, los cuales forman parte de los coeficientes de conexión `icc1` y `icc2`, tal como demuestra el siguiente ejemplo:

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) iframe_flag:true;
(%o2) true
(%i3) ishow(covdiff(v([],[i]),j))$
 i i %1
 v + icc2 v
 ,j %1 j
(%t3)
(%i4) ishow(ev(%,icc2))$
 %1 i i i
 v (ifc2 + ichr2) + v
 %1 j %1 j ,j
(%t4)
(%i5) ishow(ev(%,ifc2))$
 %1 i %2
 v ifg (ifb - ifb + ifb)
 j %2 %1 %2 %1 j %1 j %2
(%t5) ----- + v
 2 ,j
(%i6) ishow(ifb([a,b,c]))$
 %5 %4
 ifr ifr (ifri - ifri)
 a b c %4,%5 c %5,%4
(%t6)
```

Se utiliza un método alternativo para calcular el sistema de referencia `ifb` si la variable `iframe_bracket_form` vale `false`:

```
(%i8) block([iframe_bracket_form:false],ishow(ifb([a,b,c])))$
 %7 %6 %6 %7
(%t8) (ifr ifr - ifr ifr) ifri
 a b,%7 a,%7 b c %6
```

**ifb**

Variable

Es el sistema de referencia soporte. La contribución de la métrica del campo a los coeficientes de conexión se expresa utilizando:

$$\text{ifc1}_{abc} = \frac{-\text{ifb}_{cab} + \text{ifb}_{bca} + \text{ifb}_{abc}}{2}$$

El sistema de referencia soporte se define en términos del campo y la métrica del sistema de referencia. Se utilizan dos métodos alternativos dependiendo del valor de `frame_bracket_form`. Si vale `true` (que es el valor por defecto) o si `itorsion_flag` vale `true`:

$$\text{ifb}_{abc} = \text{ifr}_{ab}^d \text{ifr}_{bc}^e (\text{ifri}_{ade} - \text{ifri}_{aed} - \text{ifri}_{af}^f \text{itr}_{de})$$

En otro caso:

$$\text{ifb}_{abc} = (\text{ifr}_{ab}^e \text{ifr}_{c,e}^d - \text{ifr}_{b,e}^d \text{ifr}_{ac}^e) \text{ifri}_{ad}$$

**icc1**

Variable

Coefficientes de conexión de primera especie. Se definen en `itensor` como

$$\text{icc1}_{abc} = \text{ichr1}_{abc} - \text{ikt1}_{abc} - \text{inmc1}_{abc}$$

En esta expresión, si `iframe_flag` vale `true`, el símbolo de Christoffel `ichr1` se reemplaza por el coeficiente de conexión del sistema de referencia `ifc1`. Si `itorsion_flag` vale `false`, `ikt1` será omitido. También se omite si se utiliza una base, ya que la torsión ya está calculada como parte del sistema de referencia.

**icc2**

Variable

Coefficientes de conexión de segunda especie. Se definen en `itensor` como

$$\text{icc2}_{ab}^c = \text{ichr2}_{ab}^c - \text{ikt2}_{ab}^c - \text{inmc2}_{ab}^c$$

En esta expresión, si la variable `iframe_flag` vale `true`, el símbolo de Christoffel `ichr2` se reemplaza por el coeficiente de conexión del sistema de referencia `ifc2`. Si

`itorsion_flag` vale `false`, `ikt2` se omite. También se omite si se utiliza una base de referencia. Por último, si `inonmet_flag` vale `false`, se omite `inmc2`.

**ifc1**

Variable

Coeficiente del sistema de referencia de primera especie, también conocido como coeficientes de rotación de Ricci. Este tensor representa la contribución de la métrica del sistema de referencia al coeficiente de conexión de primera especie, definido como

$$\text{ifc1}_{abc} = \frac{-\text{ifb}_{cab} + \text{ifb}_{bca} + \text{ifb}_{abc}}{2}$$

**ifc2**

Variable

Coeficiente del sistema de referencia de primera especie. Este tensor representa la contribución de la métrica del sistema de referencia al coeficiente de conexión de primera especie, definido como

$$\text{ifc2}_{ab}^c = \text{ifg}_{cd}^c \text{ifc1}_{abd}$$

**ifr**

Variable

El campo del sistema de referencia. Se contrae con el campo inverso `ifri` para formar la métrica del sistema de referencia, `ifg`.

**ifri**

Variable

Campo inverso del sistema de referencia. Especifica la base del sistema de referencia (vectores de la base dual).

**ifg**

Variable

La métrica del sistema de referencia. Su valor por defecto es `kdelta`, pero puede cambiarse utilizando `components`.

**ifgi**

Variable

La métrica inversa del sistema de referencia. Se contrae con la métrica `ifg` para dar `kdelta`.

**iframe\_bracket\_form**

Variable opcional

Valor por defecto: `true`

Especifica cómo se calcula `ifb`.

### 28.2.6 Torsión y no metricidad

Maxima trabaja con conceptos como la torsión y la no metricidad. Cuando la variable `itorsion_flag` vale `true`, la contribución de la torsión se añade a los coeficientes de conexión. También se añaden las componentes de no metricidad cuando `inonmet_flag` vale `true`.

#### inm

Variable

Vector de no metricidad. La no metricidad conforme se define a partir de la derivada covariante del tensor métrico. La derivada covariante del tensor métrico, que normalmente es nula, se calcula, cuando `inonmet_flag` vale `true`, como

$$g_{ij;k} = -g_{ij} \text{ inm}_k$$

#### inmc1

Variable

Permutación covariante de las componentes del vector de no metricidad. Se define como

$$\text{inmc1}_{abc} = \frac{g_{ab} \text{ inm}_c - \text{inm}_a g_{bc} - g_{ac} \text{ inm}_b}{2}$$

(Sustitúyase `g` por `ifg` si se utiliza una métrica para el sistema de referencia.)

#### inmc2

Variable

Permutación contravariante de las componentes del vector de no metricidad. Se utiliza en los coeficientes de conexión si `inonmet_flag` vale `true`. Se define como

$$\text{inmc2}_{ab} = \frac{c^c \text{ inm}_a \text{ kdelta}_{cb} - \text{kdelta}_{ab} \text{ inm}_c + g_{ab} \text{ inm}_c g_{cd}}{2}$$

(Sustitúyase `g` por `ifg` si se utiliza una métrica para el sistema de referencia.)

#### ikt1

Variable

Permutación covariante del tensor de permutación, también conocido como con-torsión. Se define como

$$\text{ikt1}_{abc} = \frac{-g_{ad} \text{ itr}_{cb} - g_{bd} \text{ itr}_{ca} - \text{itr}_{ab} g_{cd}}{2}$$

(Sustitúyase `g` por `ifg` si se utiliza una métrica para el sistema de referencia.)

**ikt2**

Variable

Permutación contravariante del tensor de permutación, también conocido como con-torsión. Se define como

$$\text{ikt2} = g \begin{matrix} c & cd \\ ab & abd \end{matrix}$$

(Sustitúyase *g* por *ifg* si se utiliza una métrica para el sistema de referencia.)

**itr**

Variable

Tensor de torsión. Para una métrica con torsión, la diferenciación covariante iterada de una función escalar no conmuta, tal como demuestra el siguiente ejemplo:

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) imetric:g;
(%o2) g
(%i3) covdiff(covdiff(f([],[]),i),j)-covdiff(covdiff(f([],[]),j),i)$
(%i4) ishow(%)$

(%t4) %4 %2
 f ichr2 - f ichr2
 ,%4 j i ,%2 i j

(%i5) canform(%)
(%o5) 0
(%i6) itorsion_flag:true;
(%o6) true
(%i7) covdiff(covdiff(f([],[]),i),j)-covdiff(covdiff(f([],[]),j),i)$
(%i8) ishow(%)$

(%t8) %8 %6
 f icc2 - f icc2 - f + f
 ,%8 j i ,%6 i j ,j i ,i j

(%i9) ishow(canform(%))$

(%t9) %1 %1
 f icc2 - f icc2
 ,%1 j i ,%1 i j

(%i10) ishow(canform(ev(%,icc2)))$

(%t10) %1 %1
 f ikt2 - f ikt2
 ,%1 i j ,%1 j i

(%i11) ishow(canform(ev(%,ikt2)))$

(%t11) %2 %1 %2 %1
 f g ikt1 - f g ikt1
 ,%2 i j %1 ,%2 j i %1

(%i12) ishow(factor(canform(rename(expand(ev(%,ikt1))))))$

 %3 %2 %1 %1
 f g g (itr - itr)
 ,%3 %2 %1 j i i j
```

```

(%t12) -----
 2
(%i13) decsym(itr,2,1,[anti(all)],[]);
(%o13) done
(%i14) defcon(g,g,kdelta);
(%o14) done
(%i15) subst(g,nounify(g),%th(3))$
(%i16) ishow(canform(contract(%)))$

(%t16) %1
 - f itr
 ,%1 i j

```

### 28.2.7 Álgebra exterior

Con el paquete `itensor` se pueden realizar operaciones en campos tensoriales covariantes antisimétricos. Un campo tensorial totalmente antisimétrico de rango  $(0,L)$  se corresponde con una  $L$ -forma diferencial. Sobre estos objetos se define una operación que se llama producto exterior.

Desafortunadamente no hay consenso entre los autores a la hora de definir el producto exterior. Algunos autores prefieren una definición que se corresponde con la noción de antisimetrización, con lo que el producto externo de dos campos vectoriales se definiría como

$$a_i \wedge a_j = \frac{a_i a_j - a_j a_i}{2}$$

De forma más general, el producto de una  $p$ -forma por una  $q$ -forma se definiría como

$$A_{i1..ip} \wedge B_{j1..jq} = \frac{1}{(p+q)!} D^{k1..kp \ l1..lq} A_{i1..ip} B_{j1..jq}$$

donde  $D$  es la delta de Kronecker.

Otros autores, sin embargo, prefieren una definición “geométrica” que se corresponde con la noción del elemento de volumen,

$$a_i \wedge a_j = a_i a_j - a_j a_i$$

y, en el caso general,

$$A_{i1..ip} \wedge B_{j1..jq} = \frac{1}{p! \ q!} D^{k1..kp \ l1..lq} A_{i1..ip} B_{j1..jq}$$

Puesto que `itensor` es un paquete de álgebra tensorial, la primera de estas dos definiciones parece la más natural. Sin embargo, muchas aplicaciones hacen uso de la segunda definición. Para resolver el dilema, se define una variable que controla el comportamiento del producto exterior: si `igeowedge_flag` vale `false` (el valor por defecto), se utiliza la primera definición, si vale `true`, la segunda.



" ~ "

Operador

El operador del producto exterior se representa por el símbolo  $\sim$ . Este es un operador binario. Sus argumentos deben ser expresiones que tengan escalares, tensores covariantes de rango uno o tensores covariantes de rango 1 que hayan sido declarados antisimétricos en todos los índices covariantes.

El comportamiento del operador del producto exterior se controla con la variable `igeowedge_flag`, como en el ejemplo siguiente:

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) ishow(a([i])~b([j]))$

(%t2)
 a b - b a
 i j i j

 2

(%i3) decsym(a,2,0,[anti(all)],[]);
(%o3) done
(%i4) ishow(a([i,j])~b([k]))$

(%t4)
 a b + b a - a b
 i j k i j k i k j

 3

(%i5) igeowedge_flag:true;
(%o5) true
(%i6) ishow(a([i])~b([j]))$
(%t6)
 a b - b a
 i j i j

(%i7) ishow(a([i,j])~b([k]))$
(%t7)
 a b + b a - a b
 i j k i j k i k j
```

" | "

Operador

La barra vertical `|` representa la operación "contracción con un vector". Cuando un tensor covariante totalmente antisimétrico se contrae con un vector contravariante, el resultado no depende del índice utilizado para la contracción. Así, es posible definir la operación de contracción de forma que no se haga referencia al índice.

En el paquete `itensor` la contracción con un vector se realiza siempre respecto del primer índice de la ordenación literal. Ejemplo:

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) decsym(a,2,0,[anti(all)],[]);
(%o2) done
(%i3) ishow(a([i,j],[])|v)$

(%t3)
 %1
 v a
 %1 j

(%i4) ishow(a([j,i],[])|v)$

(%t4)
 %1
 - v a
```

%1 j

Nótese que es primordial que los tensores utilizados junto con el operador `|` se declaren totalmente antisimétricos en sus índices covariantes. De no ser así, se pueden obtener resultados incorrectos.

### **extdiff** (*expr*, *i*)

Función

Calcula la derivada exterior de *expr* con respecto del índice *i*. La derivada exterior se define formalmente como el producto exterior del operador de la derivada parcial y una forma diferencial. Por lo tanto, esta operación también se ve afectada por el valor que tome la variable `igeowedge_flag`. Ejemplo:

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) ishow(extdiff(v([i]),j))$
```

$$\frac{v_{j,i} - v_{i,j}}{2}$$

```
(%t2)
```

```
(%i3) decsym(a,2,0,[anti(all)],[]);
(%o3) done
(%i4) ishow(extdiff(a([i,j]),k))$
```

$$\frac{a_{j,k,i} - a_{i,k,j} + a_{i,j,k}}{3}$$

```
(%t4)
```

```
(%i5) igeowedge_flag:true;
(%o5) true
(%i6) ishow(extdiff(v([i]),j))$
(%t6)
```

$$\frac{v_{j,i} - v_{i,j}}{2}$$

```
(%i7) ishow(extdiff(a([i,j]),k))$
(%t7)
```

$$\frac{a_{j,k,i} - a_{i,k,j} + a_{i,j,k}}{3}$$

### **hodge** (*expr*)

Función

Calcula el dual de Hodge *expr*. Por ejemplo:

```
(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) imetric(g);
(%o2) done
(%i3) idim(4);
(%o3) done
(%i4) icounter:100;
(%o4) 100
(%i5) decsym(A,3,0,[anti(all)],[])$

(%i6) ishow(A([i,j,k],[]))$
(%t6) A
```

```

 i j k
(%i7) ishow(canform(hodge(%)))$
 %1 %2 %3 %4
 levi_civita g A
 %1 %102 %2 %3 %4
(%t7) -----
 6
(%i8) ishow(canform(hodge(%)))$
 %1 %2 %3 %8 %4 %5 %6 %7
(%t8) levi_civita levi_civita g g
 %1 %106 %2 %107
 g g A /6
 %3 %108 %4 %8 %5 %6 %7

(%i9) lc2kdt(%)$

(%i10) %,kdelta$

(%i11) ishow(canform(contract(expand(%))))$
(%t11) - A
 %106 %107 %108

```

### igeowedge\_flag

Variable opcional

Valor por defecto: `false`

Controla el comportamiento del producto exterior y de la derivada exterior. Cuando vale `false`, la noción de formas diferenciales se corresponde con el de campo tensorial covariante totalmente antisimétrico. Cuando vale `true`, las formas diferenciales se corresponden con la idea de elemento de volumen.

### 28.2.8 Exportando expresiones en TeX

El paquete `itensor` dispone de soporte limitado para exportar expresiones con tensores a TeX. Puesto que las expresiones de `itensor` son llamadas a funciones, puede que la instrucción habitual en Maxima, `tex`, no devuelva los resultados esperados. Se puede utilizar el comando `tentex`, que tratará de traducir expresiones tensoriales a objetos de TeX correctamente indexados.

#### `tentex` (*expr*)

Función

Para utilizar la función `tentex`, primero se debe cargar `tentex`, tal como muestra el siguiente ejemplo:

```

(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) load(tentex);
(%o2) /share/tensor/tentex.lisp
(%i3) idummyx:m;
(%o3) m
(%i4) ishow(icurvature([j,k,l],[i]))$

```

```

 m1 i m1 i i i
(%t4) ichr2 ichr2 - ichr2 ichr2 - ichr2 + ichr2
 j k m1 l j l m1 k j l,k j k,l
(%i5) tentex(%)$
$$\Gamma_{j\backslash,k}^{\{m_1\}\backslash,\backslash\Gamma_{l\backslash,m_1}^{\{i\}}-\backslash\Gamma_{j\backslash,l}^{\{m_1\}\backslash,\backslash\Gamma_{k\backslash,m_1}^{\{i\}}-\backslash\Gamma_{j\backslash,l,k}^{\{i\}}+\backslash\Gamma_{j\backslash,k,l}^{\{i\}}}$

```

Nótese la asignación de la variable `idummyx` para evitar la aparición del símbolo del porcentaje en la expresión en TeX, que puede dar errores de compilación.

Téngase en cuenta que esta versión de la función `tentex` es experimental.

### 28.2.9 Interactuando con ctensor

El paquete `itensor` genera código Maxima que luego puede ser ejecutado en el contexto del paquete `ctensor`. La función que se encarga de esta tarea es `ic_convert`.

#### `ic_convert` (*eqn*)

Function

Convierte la ecuación *eqn* del entorno `itensor` a una sentencia de asignación de `ctensor`. Sumas implícitas sobre índices mudos se hacen explícitas mientras que objetos indexados se transforman en arreglos (los subíndices de los arreglos se ordenan poniendo primero los covariantes seguidos de los contravariantes. La derivada de un objeto indexado se reemplazará por la forma nominal de `diff` tomada con respecto a `ct_coords` con el subíndice correspondiente al índice derivado. Los símbolos de Christoffel `ichr1` `ichr2` se traducen a `lcs` y `mcs`, respectivamente. Además, se añaden bucles `do` para la sumación de todos los índices libres, de manera que la sentencia traducida pueda ser evaluada haciendo simplemente `ev`. Los siguientes ejemplos muestran las funcionalidades de esta función.

```

(%i1) load(itensor);
(%o1) /share/tensor/itensor.lisp
(%i2) eqn:ishow(t([i,j],[k])=f([],[])*g([l,m],[])*a([],[m],j)*b([i],[l,k]))$
 k m l k
(%t2) t = f a b g
 i j ,j i l m

(%i3) ic_convert(eqn);
(%o3) for i thru dim do (for j thru dim

do (for k thru dim do t : f sum(sum(diff(a , ct_coords) b
 i, j, k m j i, l, k

g , l, 1, dim), m, 1, dim)))
 1, m
(%i4) imetric(g);
(%o4) done
(%i5) metricconvert:true;
(%o5) true
(%i6) ic_convert(eqn);
(%o6) for i thru dim do (for j thru dim

```

```

do (for k thru dim do t : f sum(sum(diff(a , ct_coords) b
 i, j, k m j i, l, k
lg , l, 1, dim), m, 1, dim)))
 1, m

```

### 28.2.10 Palabras reservadas

Las siguientes palabras son utilizadas por el paquete `itensor` internamente, por lo que no deberían ser modificadas por el usuario:

| Palabra                 | Comentarios                               |
|-------------------------|-------------------------------------------|
| -----                   |                                           |
| <code>indices2()</code> | Versión interna de <code>indices()</code> |
| <code>conti</code>      | Lista los índices contravariantes         |
| <code>covi</code>       | Lista los índices covariantes             |
| <code>deri</code>       | Lista los índices de derivadas            |
| <code>name</code>       | Devuelve el nombre de un objeto indexado  |
| <code>concan</code>     |                                           |
| <code>irpmon</code>     |                                           |
| <code>lc0</code>        |                                           |
| <code>_lc2kdt0</code>   |                                           |
| <code>_lcprod</code>    |                                           |
| <code>_extlc</code>     |                                           |

## 29 ctensor

### 29.1 Introducción a ctensor

El paquete `ctensor` dispone de herramientas para manipular componentes de tensores. Para poder hacer uso de `ctensor` es necesario cargarlo previamente en memoria ejecutando `load(ctensor)`. Para comenzar una sesión interactiva con `ctensor`, ejecutar la función `csetup()`. Primero se le pregunta al usuario la dimensión de la variedad. Si la dimensión es 2, 3 o 4, entonces la lista de coordenadas será por defecto  $[x,y]$ ,  $[x,y,z]$  o  $[x,y,z,t]$ , respectivamente. Estos nombres pueden cambiarse asignando una nueva lista de coordenadas a la variable `ct_coords` (que se describe más abajo), siendo el usuario advertido sobre este particular. Se debe tener cuidado en evitar que los nombres de las coordenadas entren en conflicto con los nombres de otros objetos en Maxima.

A continuación, el usuario introduce la métrica, bien directamente, o desde un fichero especificando su posición ordinal. La métrica se almacena en la matriz `lg`. Por último, la métrica inversa se obtiene y almacena en la matriz `ug`. También se dispone de la opción de efectuar todos los cálculos en serie de potencias.

Se desarrolla a continuación un ejemplo para la métrica estática, esférica y simétrica, en coordenadas estándar, que se aplicará posteriormente al problema de derivar las ecuaciones de vacío de Einstein (de las que se obtiene la solución de Schwarzschild). Muchas de las funciones de `ctensor` se mostrarán en los ejemplos para la métrica estándar.

```
(%i1) load(ctensor);
(%o1) /share/tensor/ctensor.mac
(%i2) csetup();
Enter the dimension of the coordinate system:
4;
Do you wish to change the coordinate names?
n;
Do you want to
1. Enter a new metric?

2. Enter a metric from a file?

3. Approximate a metric with a Taylor series?
1;

Is the matrix 1. Diagonal 2. Symmetric 3. Antisymmetric 4. General
Answer 1, 2, 3 or 4
1;
Row 1 Column 1:
a;
Row 2 Column 2:
x^2;
Row 3 Column 3:
x^2*sin(y)^2;
Row 4 Column 4:
-d;
```

Matrix entered.

Enter functional dependencies with the DEPENDS function or 'N' if none  
depends([a,d],x);

Do you wish to see the metric?

y;

```

[a 0 0 0]
[
[2
[0 x 0 0]
[
[2 2
[0 0 x sin (y) 0]
[
[0 0 0 - d]

```

(%o2)

done

(%i3) christof(mcs);

```

a
x
mcs = ---
1, 1, 1 2 a

```

```

1
mcs = -
1, 2, 2 x

```

```

1
mcs = -
1, 3, 3 x

```

```

d
x
mcs = ---
1, 4, 4 2 d

```

```

x
mcs = - -
2, 2, 1 a

```

```

cos(y)
mcs = -----
2, 3, 3 sin(y)

```

```

2
x sin (y)
mcs = - -----
3, 3, 1 a

```

```

mcs = - cos(y) sin(y)

```

```

3, 3, 2
d
x
(%t11) mcs = ---
4, 4, 1 2 a
done
(%o11)

```

## 29.2 Definiciones para ctensor

### 29.2.1 Inicialización y preparación

|                                                                                                                                                                              |         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| <b>csetup</b> ()                                                                                                                                                             | Función |
| Es la función del paquete <b>ctensor</b> que inicializa el paquete y permite al usuario introducir una métrica de forma interactiva. Véase <b>ctensor</b> para más detalles. |         |

|                                                                                                                        |         |
|------------------------------------------------------------------------------------------------------------------------|---------|
| <b>metric</b> ( <i>dis</i> )                                                                                           | Función |
| <b>metric</b> ()                                                                                                       | Función |
| Es la función del paquete <b>ctensor</b> que calcula la métrica inversa y prepara el paquete para cálculos ulteriores. |         |

Si `cframe_flag` vale `false`, la función calcula la métrica inversa `ug` a partir de la matriz `lg` definida por el usuario. Se calcula también la métrica determinante y se almacena en la variable `gdet`. Además, el paquete determina si la métrica es diagonal y ajusta el valor de `diagmetric` de la forma apropiada. Si el argumento opcional `dis` está presente y no es igual a `false`, el usuario podrá ver la métrica inversa.

Si `cframe_flag` vale `true`, la función espera que los valores de `fri` (la matriz del sistema de referencia inverso) y `lfg` (la matriz del sistema de referencia) estén definidos. A partir de ellos, se calculan la matriz del sistema de referencia `fr` y su métrica `ufg`.

|                                                                      |         |
|----------------------------------------------------------------------|---------|
| <b>ct_coordsys</b> ( <i>sistema.coordenadas</i> , <i>extra_arg</i> ) | Función |
| <b>ct_coordsys</b> ( <i>sistema.coordenadas</i> )                    | Función |

Prepara un sistema de coordenadas predefinido y una métrica. El argumento *sistema.coordenadas* puede ser cualquiera de los siguientes símbolos:

| Símbolo             | Dim | Coordenadas | Descripción/comentarios            |
|---------------------|-----|-------------|------------------------------------|
| cartesian2d         | 2   | [x,y]       | Sistema de coordenadas cartesianas |
| polar               | 2   | [r,phi]     | Sistema de coordenadas polares     |
| elliptic            | 2   | [u,v]       | Sistema de coordenadas elípticas   |
| confocalelliptic    | 2   | [u,v]       | Coordenadas elípticas confocales   |
| bipolar             | 2   | [u,v]       | Sistema de coordenadas bipolares   |
| parabolic           | 2   | [u,v]       | Sistema de coordenadas parabólicas |
| cartesian3d         | 3   | [x,y,z]     | Sistema de coordenadas cartesianas |
| polarcylindrical    | 3   | [r,theta,z] | Polares en 2D con cilíndrica z     |
| ellipticcylindrical | 3   | [u,v,z]     | Elípticas en 2D con cilíndrica z   |



|                       |   |                   |                                    |
|-----------------------|---|-------------------|------------------------------------|
| confocalellipsoidal   | 3 | [u,v,w]           | Elipsoidales confocales            |
| bipolarcylindrical    | 3 | [u,v,z]           | Bipolares en 2D con cilíndrica z   |
| paraboliccylindrical  | 3 | [u,v,z]           | Parabólicas en 2D con cilíndrica z |
| paraboloidal          | 3 | [u,v,phi]         | Coordenadas paraboloidales         |
| conical               | 3 | [u,v,w]           | Coordenadas cónicas                |
| toroidal              | 3 | [u,v,phi]         | Coordenadas toroidales             |
| spherical             | 3 | [r,theta,phi]     | Sistema de coordenadas esféricas   |
| oblatespheroidal      | 3 | [u,v,phi]         | Coordenadas esferoidales obleadas  |
| oblatespheroidalsqrt  | 3 | [u,v,phi]         |                                    |
| prolatespheroidal     | 3 | [u,v,phi]         | Coordenadas esferoidales prolatas  |
| prolatespheroidalsqrt | 3 | [u,v,phi]         |                                    |
| ellipsoidal           | 3 | [r,theta,phi]     | Coordenadas elipsoidales           |
| cartesian4d           | 4 | [x,y,z,t]         | Sistema de coordenadas cartesianas |
| spherical4d           | 4 | [r,theta,eta,phi] | Sistema de coordenadas esféricas   |
| exteriorschwarzschild | 4 | [t,r,theta,phi]   | Métrica de Schwarzschild           |
| interiorschwarzschild | 4 | [t,z,u,v]         | Métrica interior de Schwarzschild  |
| kerr_newman           | 4 | [t,r,theta,phi]   | Métrica simétrica con carga axial  |

El argumento `sistema_coordenadas` puede ser también una lista de funciones de transformación, seguida de una lista que contenga los nombres de las coordenadas. Por ejemplo, se puede especificar una métrica esférica como se indica a continuación:

```
(%i1) load(ctensor);
(%o1) /share/tensor/ctensor.mac
(%i2) ct_coordsys([r*cos(theta)*cos(phi),r*cos(theta)*sin(phi),
r*sin(theta),[r,theta,phi]]);
(%o2) done
(%i3) lg:trigsimp(lg);
[1 0 0]
[0 0 0]
[0 2 0]
(%o3) [0 r 0]
[0 0 0]
[0 0 2 2]
[0 0 r cos(theta)]

(%i4) ct_coords;
(%o4) [r, theta, phi]
(%i5) dim;
(%o5) 3
```

Las funciones de transformación se pueden utilizar también si `cframe_flag` vale `true`:

```
(%i1) load(ctensor);
(%o1) /share/tensor/ctensor.mac
(%i2) cframe_flag:true;
(%o2) true
(%i3) ct_coordsys([r*cos(theta)*cos(phi),r*cos(theta)*sin(phi),
r*sin(theta),[r,theta,phi]]);
```



\
  
uriem

El paquete `ctensor` también puede trabajar con sistemas de referencia móviles. Si `cframe_flag` vale `true`, se pueden calcular los siguientes tensores:

```

lfg -- ufg
 \
fri -- fr -- lcs -- mcs -- lriem -- ric -- uric
 \ | \ \ \
 lg -- ug | weyl tracer - ein -- lein
 | \
 | riem
 |
 \uriem

```

### **christof** (*dis*)

Función

Es una función del paquete `ctensor`. Calcula los símbolos de Christoffel de ambos tipos. El argumento *dis* determina qué resultados se mostrarán de forma inmediata. Los símbolos de Christoffel de primer y segundo tipo se almacenan en los arreglos `lcs[i,j,k]` y `mcs[i,j,k]`, respectivamente, y se definen simétricos en sus dos primeros índices. Si el argumento de `christof` es `lcs` o `mcs` entonces serán mostrados únicamente los valores no nulos de `lcs[i,j,k]` o `mcs[i,j,k]`, respectivamente. Si el argumento es `all` entonces se mostrarán los valores no nulos de `lcs[i,j,k]` y `mcs[i,j,k]`. Si el argumento vale `false` entonces no se mostrarán los elementos. El arreglo `mcs[i,j,k]` está definido de tal modo que el último índice es contravariante.

### **ricci** (*dis*)

Función

Es una función del paquete `ctensor`. La función `ricci` calcula las componentes covariantes (simétricas) `ric[i,j]` del tensor de Ricci. Si el argumento *dis* vale `true`, entonces se muestran las componentes no nulas.

### **uricci** (*dis*)

Función

Esta función calcula en primer lugar las componentes covariantes `ric[i,j]` del tensor de Ricci. Después se calcula el tensor de Ricci utilizando la métrica contravariante. Si el valor del argumento *dis* vale `true`, entonces se mostrarán directamente las componentes `uric[i,j]` (el índice *i* es covariante y el *j* contravariante). En otro caso, `ricci(false)` simplemente calculará las entradas del arreglo `uric[i,j]` sin mostrar los resultados.

### **scurvature** ()

Función

Devuelve la curvatura escalar (obtenida por contracción del tensor de Ricci) de la variedad de Riemannian con la métrica dada.

**einstein** (*dis*)

Función

Es una función del paquete `ctensor`. La función `einstein` calcula el tensor de Einstein después de que los símbolos de Christoffel y el tensor de Ricci hayan sido calculados (con las funciones `christof` y `ricci`). Si el argumento *dis* vale `true`, entonces se mostrarán los valores no nulos del tensor de Einstein `ein[i,j]`, donde *j* es el índice contravariante. La variable `rateinstein` causará la simplificación racional de estas componentes. Si `ratfac` vale `true` entonces las componentes también se factorizarán.

**leinstein** (*dis*)

Función

Es el tensor covariante de Einstein. La función `leinstein` almacena los valores del tensor covariante de Einstein en el arreglo `lein`. El tensor covariante de Einstein se calcula a partir del tensor de Einstein `ein` multiplicándolo por el tensor métrico. Si el argumento *dis* vale `true`, entonces se mostrarán los valores no nulos del tensor covariante de Einstein.

**riemann** (*dis*)

Función

Es una función del paquete `ctensor`. La función `riemann` calcula el tensor de curvatura de Riemann a partir de la métrica dada y de los símbolos de Christoffel correspondientes. Se utiliza el siguiente convenio sobre los índices:

$$R[i,j,k,l] = R \begin{matrix} l \\ ijk \end{matrix} = \begin{matrix} l \\ ij,k \end{matrix} - \begin{matrix} l \\ ik,j \end{matrix} + \begin{matrix} l \\ mk \\ ij \end{matrix} - \begin{matrix} l \\ mj \\ ik \end{matrix}$$

Esta notación es consistente con la notación utilizada por el paquete `itensor` y su función `icurvature`. Si el argumento opcional *dis* vale `true`, se muestran las componentes no nulas de `riem[i,j,k,l]`. Como en el caso del tensor de Einstein, ciertas variables permiten controlar al usuario la simplificación de las componentes del tensor de Riemann. Si `ratriemann` vale `true`, entonces se hará la simplificación racional. Si `ratfac` vale `true`, entonces se factorizarán todas las componentes.

Si la variable `cframe_flag` vale `false`, el tensor de Riemann se calcula directamente a partir de los símbolos de Christoffel. Si `cframe_flag` vale `false`, el tensor covariante de Riemann se calcula a partir de los coeficientes del campo.

**lriemann** (*dis*)

Función

Es el tensor covariante de Riemann (`lriem[]`).

Calcula el tensor covariante de Riemann como un arreglo `lriem`. Si el argumento *dis* vale `true`, sólo se muestran los valores no nulos.

Si la variable `cframe_flag` vale `true`, el tensor covariante de Riemann se calcula directamente de los coeficientes del campo. En otro caso, el tensor de Riemann (3,1) se calcula en primer lugar.

Para más información sobre la ordenación de los índices, véase `riemann`.

**uriemann** (*dis*)

Función

Calcula las componentes contravariantes del tensor de curvatura de Riemann como un arreglo `uriem[i,j,k,l]`. Éstos se muestran si *dis* vale `true`.

**rinvariant** () Función

Calcula la invariante de Kretchmann (**kinvariant**) obtenida por contracción de los tensores.

```
lriem[i,j,k,l]*uriem[i,j,k,l].
```

Este objeto no se simplifica automáticamente al ser en ocasiones muy grande.

**weyl** (*dis*) Función

Calcula el tensor conforme de Weyl. Si el argumento *dis* vale **true**, se le mostrarán al usuario las componentes no nulas **weyl[i,j,k,l]**. En otro caso, estas componentes serán únicamente calculadas y almacenadas. Si la variable **ratweyl** vale **true**, entonces las componentes se simplifican racionalmente; si **ratfac** vale **true** los resultados también se simplificarán.

### 29.2.3 Desarrollo de Taylor

El paquete **ctensor** puede truncar resultados e interpretarlos como aproximaciones de Taylor. Este comportamiento se controla con la variable **ctayswitch**; cuando vale **true**, **ctensor** utiliza internamente la función **ctaylor** cuando simplifica resultados.

La función **ctaylor** es llamada desde las siguientes funciones del paquete **ctensor**:

| Función           | Comentarios   |
|-------------------|---------------|
| -----             | -----         |
| <b>christof()</b> | Sólo para mcs |
| <b>ricci()</b>    |               |
| <b>uricci()</b>   |               |
| <b>einstein()</b> |               |
| <b>riemann()</b>  |               |
| <b>weyl()</b>     |               |
| <b>checkdiv()</b> |               |

**ctaylor** () Función

La función **ctaylor** trunca su argumento convirtiéndolo en un desarrollo de Taylor por medio de la función **taylor** e invocando después a **ratdisrep**. Esto tiene el efecto de eliminar términos de orden alto en la variable de expansión **ctayvar**. El orden de los términos que deben ser eliminados se define **ctaypov**; el punto alrededor del cual se desarrolla la serie se especifica en **ctaypt**.

Como ejemplo, considérese una sencilla métrica que es una perturbación de la de Minkowski. Sin añadir restricciones, incluso una métrica diagonal produce expansiones del tensor de Einstein que pueden llegar a ser muy complejas:

```
(%i1) load(ctensor);
(%o1) /share/tensor/ctensor.mac
(%i2) ratfac:true;
(%o2) true
(%i3) derivabbrev:true;
(%o3) true
```

```

(%i4) ct_coords:[t,r,theta,phi];
(%o4) [t, r, theta, phi]
(%i5) lg:matrix([-1,0,0,0],[0,1,0,0],[0,0,r^2,0],[0,0,0,r^2*sin(theta)^2]);
 [- 1 0 0 0]
 []
 [0 1 0 0]
 []
 [2]
(%o5) [0 0 r 0]
 []
 [2 2]
 [0 0 0 r sin (theta)]
(%i6) h:matrix([h11,0,0,0],[0,h22,0,0],[0,0,h33,0],[0,0,0,h44]);
 [h11 0 0 0]
 []
 [0 h22 0 0]
(%o6) []
 [0 0 h33 0]
 []
 [0 0 0 h44]

(%i7) depends(l,r);
(%o7) [l(r)]
(%i8) lg:lg+l*h;
 [h11 l - 1 0 0 0]
 []
 [0 h22 l + 1 0 0]
 []
 [2]
(%o8) [0 0 r + h33 l 0]
 []
 [2 2]
 [0 0 0 r sin (theta) + h44 l]
(%i9) cmetric(false);
(%o9) done
(%i10) einstein(false);
(%o10) done
(%i11) ntermst(ein);
[[1, 1], 62]
[[1, 2], 0]
[[1, 3], 0]
[[1, 4], 0]
[[2, 1], 0]
[[2, 2], 24]
[[2, 3], 0]
[[2, 4], 0]
[[3, 1], 0]
[[3, 2], 0]
[[3, 3], 46]
[[3, 4], 0]

```



Esta capacidad del paquete `ctensor` puede ser muy útil; por ejemplo, cuando se trabaja en zonas del campo gravitatorio alejadas del origen de éste.

### 29.2.4 Campos del sistema de referencia

Cuando la variable `cframe_flag` vale `true`, el paquete `ctensor` realiza sus cálculos utilizando un sistema de referencia móvil.

**frame\_bracket** (*fr, fri, diagframe*)

Función

Es el sistema de referencia soporte (`fb[]`).

Calcula el soporte del sistema de referencia de acuerdo con la siguiente definición:

$$\begin{array}{ccccc} & c & & c & \\ \text{ifb} & = & ( & \text{ifri} & - & \text{ifri} & ) & \text{ifr} & \text{ifr} \\ & ab & & d,e & & e,d & & a & b \end{array}$$

### 29.2.5 Clasificación algebraica

Una nueva funcionalidad (Noviembre de 2004) de `ctensor` es su capacidad de obtener la clasificación de Petrov de una métrica espaciotemporal de dimensión 4. Para una demostración de esto véase el fichero `share/tensor/petrov.dem`.

**nptetrad** ()

Función

Calcula la cuaterna nula de Newman-Penrose (`np`). Véase `petrov` para un ejemplo.

La cuaterna nula se construye bajo la suposición de que se está utilizando una métrica tetradimensional ortonormal con signatura métrica  $(-,+,+,+)$ . Los componentes de la cuaterna nula se relacionan con la inversa de la matriz del sistema de referencia de la siguiente manera:

$$\begin{array}{l} \text{np}_1 = (\text{fri}_1 + \text{fri}_2) / \text{sqrt}(2) \\ \text{np}_2 = (\text{fri}_1 - \text{fri}_2) / \text{sqrt}(2) \\ \text{np}_3 = (\text{fri}_3 + \%i \text{fri}_4) / \text{sqrt}(2) \\ \text{np}_4 = (\text{fri}_3 - \%i \text{fri}_4) / \text{sqrt}(2) \end{array}$$

**psi** (*dis*)

Función

Calcula los cinco coeficientes de Newman-Penrose `psi[0]...psi[4]`. Si `psi` vale `true`, se muestran estos coeficientes. Véase `petrov` para un ejemplo.

Estos coeficientes se calculan a partir del tensor de Weyl.



**petrov ()**

Función

Calcula la clasificación de Petrov de la métrica caracterizada por `psi[0]...psi[4]`.

Por ejemplo, lo que sigue demuestra cómo obtener la clasificación de Petrov para la métrica de Kerr:

```
(%i1) load(ctensor);
(%o1) /share/tensor/ctensor.mac
(%i2) (cframe_flag:true,gcd:spmod,ctrgsimp:true,ratfac:true);
(%o2) true
(%i3) ct_coordsys(exteriorschwarzschild,all);
(%o3) done
(%i4) ug:invert(lg)$
(%i5) weyl(false);
(%o5) done
(%i6) nptetrad(true);
(%t6) np =
```

$$\begin{bmatrix} \frac{\sqrt{r-2m}}{\sqrt{2}} & \frac{\sqrt{r}}{\sqrt{2}} & 0 & 0 \\ \frac{\sqrt{2}\sqrt{r}}{\sqrt{2}} & \frac{\sqrt{2}\sqrt{r-2m}}{\sqrt{2}} & 0 & 0 \\ \frac{\sqrt{r-2m}}{\sqrt{2}} & -\frac{\sqrt{r}}{\sqrt{2}} & 0 & 0 \\ \frac{\sqrt{2}\sqrt{r}}{\sqrt{2}} & \frac{\sqrt{2}\sqrt{r-2m}}{\sqrt{2}} & 0 & 0 \\ 0 & 0 & \frac{r}{\sqrt{2}} & \frac{i r \sin(\theta)}{\sqrt{2}} \\ 0 & 0 & \frac{r}{\sqrt{2}} & -\frac{i r \sin(\theta)}{\sqrt{2}} \end{bmatrix}$$

```
(%t7) npi = matrix([- \frac{\sqrt{r}}{\sqrt{2}\sqrt{r-2m}}, \frac{\sqrt{r-2m}}{\sqrt{2}\sqrt{r}}, 0, 0],
```

$$\begin{bmatrix} -\frac{\sqrt{r}}{\sqrt{2}\sqrt{r-2m}} & \frac{\sqrt{r-2m}}{\sqrt{2}\sqrt{r}} & 0 & 0 \end{bmatrix},$$

```
0, 0, \frac{1}{\sqrt{2}r}, \frac{i}{\sqrt{2}r \sin(\theta)}],
```

$$\begin{bmatrix} 0 & 0 & \frac{1}{\sqrt{2}r} & -\frac{i}{\sqrt{2}r \sin(\theta)} \end{bmatrix})$$

```
(%o7) done
```

```

(%i7) psi(true);
(%t8) psi = 0
 0

(%t9) psi = 0
 1

(%t10) psi = --
 2 3
 m
 r

(%t11) psi = 0
 3

(%t12) psi = 0
 4
(%o12) done
(%i12) petrov();
(%o12) D

```

La función de clasificación de Petrov se basa en el algoritmo publicado en "Classifying geometries in general relativity: III Classification in practice" de Pollney, Skea, and d'Inverno, *Class. Quant. Grav.* 17 2885-2902 (2000). Excepto para algunos ejemplos sencillos, esta implementación no ha sido exhaustivamente probada, por lo que puede contener errores.

### 29.2.6 Torsión y no metricidad

El paquete `ctensor` es capaz de calcular e incluir coeficientes de torsión y no metricidad en los coeficientes de conexión.

Los coeficientes de torsión se calculan a partir de un tensor suministrado por el usuario, `tr`, el cual debe ser de rango (2,1). A partir de ahí, los coeficientes de torsión `kt` se calculan de acuerdo con las siguientes fórmulas:

$$kt_{ijk} = \frac{-g_{im}^{tr} \quad -g_{jm}^{tr} \quad -tr_{ij}^g \quad km}{2}$$

$$kt_{ij}^k = g_{ij}^{km} \quad kt_{ijm}$$

Los coeficientes de no metricidad se calculan a partir de un vector de no metricidad, **nm**, suministrado por el usuario. A partir de ahí, los coeficientes de no metricidad, **nmc**, se calculan como se indica a continuación:

$$nmc_{ij} = \frac{-nm_{ik} D_{kj} - D_{ik} nm_{kj} + g_{km} g_{mj}}{2}$$

donde D es la delta de Kronecker.

### **contortion** (*tr*)

Función

Calcula los coeficientes (2,1) de contorsión del tensor de torsión *tr*.

### **nonmetricity** (*nm*)

Función

Calcula los coeficientes (2,1) de no metricidad del vector de no metricidad *nm*.

## 29.2.7 Otras funcionalidades

### **ctransform** (*M*)

Función

Es una función del paquete **ctensor**. Realiza una transformación de coordenadas a partir de una matriz cuadrada simétrica *M* arbitraria. El usuario debe introducir las funciones que definen la transformación.

### **findde** (*A*, *n*)

Función

Devuelve la lista de las ecuaciones diferenciales que corresponden a los elementos del arreglo cuadrado *n*-dimensional. El argumento *n* puede ser 2 ó 3; **deindex** es una lista global que contiene los índices de *A* que corresponden a estas ecuaciones diferenciales. Para el tensor de Einstein (**ein**), que es un arreglo bidimensional, si se calcula para la métrica del ejemplo de más abajo, **findde** devuelve las siguientes ecuaciones diferenciales independientes:

```
(%i1) load(ctensor);
(%o1) /share/tensor/ctensor.mac
(%i2) derivabbrev:true;
(%o2) true
(%i3) dim:4;
(%o3) 4
(%i4) lg:matrix([a,0,0,0],[0,x^2,0,0],[0,0,x^2*sin(y)^2,0],[0,0,0,-d]);
(%o4)
[a 0 0 0]
[0 x^2 0 0]
[0 0 x^2 sin(y)^2 0]
[0 0 0 -d]
```

```

 [0 0 0 - d]
(%i5) depends([a,d],x);
(%o5) [a(x), d(x)]
(%i6) ct_coords:[x,y,z,t];
(%o6) [x, y, z, t]
(%i7) cmetric();
(%o7) done
(%i8) einstein(false);
(%o8) done
(%i9) findde(ein,2);

(%o9) [dx - a d + d, 2 a d dx - a (d)2 x - a d dx + 2 a d dx
 x x x x x x x x
 - 2 a dx, ax x + ax2 - a]x

(%i10) deindex;
(%o10) [[1, 1], [2, 2], [4, 4]]

```

**cograd ()**

Función

Calcula el gradiente covariante de una función escalar permitiendo al usuario elegir el nombre del vector correspondiente, como ilustra el ejemplo que acompaña a la definición de la función `contragrad`.

**contragrad ()**

Function

Calcula el gradiente contravariante de una función escalar permitiendo al usuario elegir el nombre del vector correspondiente, tal como muestra el siguiente ejemplo para la métrica de Schwarzschild:

```

(%i1) load(ctensor);
(%o1) /share/tensor/ctensor.mac
(%i2) derivabbrev:true;
(%o2) true
(%i3) ct_coordsys(exterior schwarzschild,all);
(%o3) done
(%i4) depends(f,r);
(%o4) [f(r)]
(%i5) cograd(f,g1);
(%o5) done
(%i6) listarray(g1);
(%o6) [0, f , 0, 0]
 r
(%i7) contragrad(f,g2);
(%o7) done
(%i8) listarray(g2);
 f r - 2 f m

```

```
(%o8) [0, $\frac{r}{r}$, 0, 0]
```

**dscalar ()**

Función

Calcula el tensor de d'Alembertian de la función escalar una vez se han declarado las dependencias. Por ejemplo:

```
(%i1) load(ctensor);
(%o1) /share/tensor/ctensor.mac
(%i2) derivabbrev:true;
(%o2) true
(%i3) ct_coordsys(exterior schwarzschild,all);
(%o3) done
(%i4) depends(p,r);
(%o4) [p(r)]
(%i5) factor(dscalar(p));
 2
 p r - 2 m p r + 2 p r - 2 m p
 r r r r r r

 2
 r
```

**checkdiv ()**

Función

Calcula la divergencia covariante del tensor de segundo rango (mixed second rank tensor), cuyo primer índice debe ser covariante, devolviendo las **n** componentes correspondientes del campo vectorial (la divergencia), siendo **n = dim**.

**cgeodesic (dis)**

Función

Es una función del paquete **ctensor** que calcula las ecuaciones geodésicas del movimiento para una métrica dada, las cuales se almacenan en el arreglo **geod[i]**. Si el argumento **dis** vale **true** entonces se muestran estas ecuaciones.

**bdvac (f)**

Función

Genera las componentes covariantes de las ecuaciones del campo vacío de la teoría gravitacional de Brans- Dicke gravitacional. El campo escalar se especifica con el argumento **f**, el cual debe ser el nombre de una función no evaluada (precedida de apóstrofo) con dependencias funcionales, por ejemplo, '**p(x)**'.

Las componentes del tensor covariante (second rank covariant field tensor) se almacenan en el arreglo **bd**.

**invariant1 ()**

Función

Genera el tensor de Euler-Lagrange (ecuaciones de campo) para la densidad invariante de  $R^2$ . Las ecuaciones de campo son las componentes del arreglo **inv1**.

### 29.2.8 Utilidades

 $\text{diagmatr}_{\text{xp}}(M)$ 

### Función

Devuelve **true** si  $M$  es una matriz diagonal o un arreglo bidimensional.

**symmetricp** ( $M$ )

Función

Devuelve **true** si  $M$  es una matriz simétrica o un arreglo bidimensional.

$$\text{ntermst} \ (f)$$

Función

Permite hacerse una idea del tamaño del tensor  $f$ .

$$\text{cdisplay } (ten)$$

### Función

Muestra todos los elementos del tensor *ten* como arreglo multidimensional. Tensors de rango 0 y 1, así como otros tipos de variables, se muestran como en `ldisplay`. Tensors de rango 2 se muestran como matrices bidimensionales, mientras que tensores de mayor rango se muestran como listas de matrices bidimensionales. Por ejemplo, el tensor de Riemann de la métrica de Schwarzschild se puede ver como:

```
(%i1) load(ctensor);
(%o1) /share/tensor/ctensor.mac
(%i2) ratfac:true;
(%o2) true
(%i3) ct_coordsys(exteriorschwarzschild,all);
(%o3) done
(%i4) riemann(false);
(%o4) done
(%i5) cdisplay(riem);
```

$$\text{riem}_{1,1} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ \left[ 0 - \frac{3 m (r - 2 m)}{4 r} + \frac{m}{3 r} - \frac{2 m}{4 r} \right] & 0 & 0 & 0 \\ \left[ 0 & 0 & \frac{m (r - 2 m)}{4 r} & 0 \right] \\ \left[ 0 & 0 & 0 & \frac{m (r - 2 m)}{4 r} \right] \end{bmatrix}$$

$$\begin{bmatrix} 2 m (r - 2 m) \\ 0 - \frac{4}{r} & 0 & 0 \\ 4 & & & \\ r & & & \end{bmatrix}$$

```

riem
1, 2 = [
[0 0 0 0]
[
[0 0 0 0]
[
[0 0 0 0]
[
[0 0 0 0]

[
m (r - 2 m)
[0 0 - ---- 0]
[
4
[
r
riem
1, 3 = [
[0 0 0 0]
[
[0 0 0 0]
[
[0 0 0 0]
[
[0 0 0 0]

[
m (r - 2 m)
[0 0 0 - ----]
[
4
[
r
riem
1, 4 = [
[0 0 0 0]
[
[0 0 0 0]
[
[0 0 0 0]
[
[0 0 0 0]

[
0 0 0 0]
[
[
2 m
[- ---- 0 0 0]
[
2
riem
2, 1 = [r (r - 2 m)
[
[
0 0 0 0]
[
[
0 0 0 0]

[
2 m
[---- 0 0 0]
[
2
[r (r - 2 m)
[
[
0 0 0 0]
[
[
0 0 0 0]
riem
2, 2 = [
[
0 0 - ---- 0]

```

$$\begin{bmatrix} r^2 (r - 2m) \\ 0 & 0 & 0 & -\frac{m}{r^2 (r - 2m)} \end{bmatrix}$$

$$\text{riem}_{2,3} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{m}{r^2 (r - 2m)} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\text{riem}_{2,4} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{m}{r^2 (r - 2m)} \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\text{riem}_{3,1} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ m & -0 & 0 & 0 \\ r & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\text{riem}_{3,2} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ m & 0 & 0 & 0 \\ 0 & -r & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$



$$\text{riem}_{3,3} = \begin{bmatrix} m & & & & \\ - & - & 0 & 0 & 0 \\ r & & & & \\ & & & & \\ & m & & & \\ 0 & - & - & 0 & 0 \\ & r & & & \\ 0 & 0 & 0 & 0 & 0 \\ & & & & \\ 0 & 0 & 0 & \frac{2m-r}{r} + 1 & \end{bmatrix}$$

$$\text{riem}_{3,4} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ & & & \\ 0 & 0 & 0 & 0 \\ & & & \\ & & 2m & \\ 0 & 0 & 0 & - \frac{2m}{r} \\ & & r & \\ & & & \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\text{riem}_{4,1} = \begin{bmatrix} & 0 & & 0 & 0 & 0 \\ & & & & & \\ & 0 & & 0 & 0 & 0 \\ & & & & & \\ & 0 & & 0 & 0 & 0 \\ & & & & & \\ & 2 & & & & \\ m \sin(\theta) & & & & & \\ - \frac{m \sin^2(\theta)}{r} & 0 & 0 & 0 & & \\ & r & & & & \end{bmatrix}$$

$$\text{riem}_{4,2} = \begin{bmatrix} 0 & & 0 & & 0 & 0 \\ & & & & & \\ 0 & & 0 & & 0 & 0 \\ & & & & & \\ 0 & & 0 & & 0 & 0 \\ & & & & & \\ & 2 & & & & \\ m \sin(\theta) & & & & & \\ 0 & - \frac{m \sin^2(\theta)}{r} & 0 & 0 & & \\ & r & & & & \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & & 0 & & 0 \\ & & & & & \end{bmatrix}$$



- ctrgrsimp** Variable opcional  
 Provoca que se realicen simplificaciones trigonométricas cuando se calculan tensores. La variable **ctrgrsimp** afecta únicamente a aquellos cálculos que utilicen un sistema de referencia móvil.
- cframe\_flag** Variable opcional  
 Provoca que los cálculos se realicen respecto de un sistema de referencia móvil.
- ctorsion\_flag** Variable opcional  
 Obliga a que se calcule también el tensor de contorsión junto con los coeficientes de conexión. El propio tensor de contorsión se calcula con la función **contortion** a partir del tensor **tr** suministrado por el usuario.
- cnonmet\_flag** Variable opcional  
 Obliga a que se calculen también los coeficientes de no metricidad junto con los coeficientes de conexión. Los coeficientes de no metricidad se calculan con la función **nonmetricity** a partir del vector de no metricidad **nm** suministrado por el usuario.
- ctayswitch** Variable opcional  
 Si vale **true**, obliga a que ciertos cálculos de **ctensor** se lleven a cabo utilizando desarrollos de series de Taylor. Estos cálculos hacen referencia a las funciones **christof**, **ricci**, **uricci**, **einstein** y **weyl**.
- ctayvar** Variable opcional  
 Variable utilizada para desarrollos de Taylor cuando la variable **ctayswitch** vale **true**.
- ctaypov** Variable opcional  
 Máximo exponente utilizado en los desarrollos de Taylor cuando **ctayswitch** vale **true**.
- ctaypt** Variable opcional  
 Punto alrededor del cual se realiza un desarrollo de Taylor cuando **ctayswitch** vale **true**.
- gdet** Variable opcional  
 Es el determinante del tensor métrico **lg**, calculado por **cmetric** cuando **cframe\_flag** vale **false**.
- ratchristof** Variable opcional  
 Obliga a que la función **christof** aplique la simplificación racional.
- rateinstein** Variable opcional  
 Valor por defecto: **true**  
 Si vale **true** entonces se hará la simplificación racional en los componentes no nulos de los tensores de Einstein; si **ratfac** vale **true** entonces las componentes también serán factorizadas.

|                                                                                                                                                                                                                                                                           |                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| <b>ratriemann</b>                                                                                                                                                                                                                                                         | Variable opcional |
| Valor por defecto: <code>true</code>                                                                                                                                                                                                                                      |                   |
| Es una de las variables que controlan la simplificación de los tensores de Riemann; si vale <code>true</code> , entonces se llevará a cabo la simplificación racional; si <code>ratfac</code> vale <code>true</code> entonces las componentes también serán factorizadas. |                   |
| <b>ratweyl</b>                                                                                                                                                                                                                                                            | Variable opcional |
| Valor por defecto: <code>true</code>                                                                                                                                                                                                                                      |                   |
| Si vale <code>true</code> , entonces la función <code>weyl</code> llevará a cabo la simplificación racional de los valores del tensor de Weyl. si <code>ratfac</code> vale <code>true</code> entonces las componentes también serán factorizadas.                         |                   |
| <b>lfg</b>                                                                                                                                                                                                                                                                | Variable          |
| Es la covariante de la métrica del sistema de referencia. Por defecto, está inicializada al sistema de referencia tetradimensional de Lorentz con signatura $(+,+,+,-)$ . Se utiliza cuando <code>cframe_flag</code> vale <code>true</code> .                             |                   |
| <b>ufg</b>                                                                                                                                                                                                                                                                | Variable          |
| Es la métrica del sistema de referencia inverso. La calcula <code>lfg</code> cuando <code>cmetric</code> es invocada tomando <code>cframe_flag</code> el valor <code>true</code> .                                                                                        |                   |
| <b>riem</b>                                                                                                                                                                                                                                                               | Variable          |
| Es el tensor (3,1) de Riemann. Se calcula cuando se invoca la función <code>riemann</code> . Para información sobre el indexado, véase la descripción de <code>riemann</code> .                                                                                           |                   |
| Si <code>cframe_flag</code> vale <code>true</code> , <code>riem</code> se calcula a partir del tensor covariante de Riemann <code>lriem</code> .                                                                                                                          |                   |
| <b>lriem</b>                                                                                                                                                                                                                                                              | Variable          |
| Es el tensor covariante de Riemann. Lo calcula la función <code>lriemann</code> .                                                                                                                                                                                         |                   |
| <b>uriem</b>                                                                                                                                                                                                                                                              | Variable          |
| Es el tensor contravariante de Riemann. Lo calcula la función <code>uriemann</code> .                                                                                                                                                                                     |                   |
| <b>ric</b>                                                                                                                                                                                                                                                                | Variable          |
| Es el tensor de Ricci. Lo calcula la función <code>ricci</code> .                                                                                                                                                                                                         |                   |
| <b>uric</b>                                                                                                                                                                                                                                                               | Variable          |
| Es el tensor contravariante de Ricci. Lo calcula la función <code>uricci</code> .                                                                                                                                                                                         |                   |
| <b>lg</b>                                                                                                                                                                                                                                                                 | Variable          |
| Es el tensor métrico. Este tensor se debe especificar (como matriz cuadrada de orden <code>dim</code> ) antes de que se hagan otros cálculos.                                                                                                                             |                   |
| <b>ug</b>                                                                                                                                                                                                                                                                 | Variable          |
| Es la inversa del tensor métrico. Lo calcula la función <code>cmetric</code> .                                                                                                                                                                                            |                   |

|                                                                                                                                                                                               |                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| <b>weyl</b>                                                                                                                                                                                   | Variable             |
| Es el tensor de Weyl. Lo calcula la función <code>weyl</code> .                                                                                                                               |                      |
| <b>fb</b>                                                                                                                                                                                     | Variable             |
| Son los coeficientes del sistema de referencia soporte, tal como los calcula <code>frame_bracket</code> .                                                                                     |                      |
| <b>kinvariant</b>                                                                                                                                                                             | Variable             |
| Es la invariante de Kretchmann, tal como la calcula la función <code>rinvariant</code> .                                                                                                      |                      |
| <b>np</b>                                                                                                                                                                                     | Variable             |
| Es la cuaterna nula de Newman-Penrose, tal como la calcula la función <code>nptetrad</code> .                                                                                                 |                      |
| <b>npi</b>                                                                                                                                                                                    | Variable             |
| Es la cuaterna nula "raised-index Newman-Penrose". Lo calcula la función <code>nptetrad</code> . Se define como <code>ug.np</code> . El producto <code>np.transpose(npi)</code> es constante: |                      |
| <pre>(%i39) trigsimp(np.transpose(npi));</pre> $\begin{bmatrix} 0 & -1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$ <pre>(%o39)</pre>                           |                      |
| <b>tr</b>                                                                                                                                                                                     | Variable             |
| Tensor de rango 3 suministrado por el usuario y que representa una torsión. Lo utiliza la función <code>contortion</code> .                                                                   |                      |
| <b>kt</b>                                                                                                                                                                                     | Variable             |
| Es el tensor de contorsión, calculado a partir de <code>tr</code> por la función <code>contortion</code> .                                                                                    |                      |
| <b>nm</b>                                                                                                                                                                                     | Variable             |
| Vector de no metricidad suministrado por el usuario. Lo utiliza la función <code>nonmetricity</code> .                                                                                        |                      |
| <b>nmc</b>                                                                                                                                                                                    | Variable             |
| Son los coeficientes de no metricidad, calculados a partir de <code>nm</code> por la función <code>nonmetricity</code> .                                                                      |                      |
| <b>tensorkill</b>                                                                                                                                                                             | Variable del sistema |
| Variable que indica si el paquete de tensores se ha inicializado. Utilizada por <code>csetup</code> y reinicializada por <code>init_ctensor</code> .                                          |                      |

**ct\_coords**

Variable opcional

Valor por defecto: []

La variable `ct_coords` contiene una lista de coordenadas. Aunque se define normalmente cuando se llama a la función `csetup`, también se pueden redefinir las coordenadas con la asignación `ct_coords: [j1, j2, ..., jn]` donde `j` es el nuevo nombre de las coordenadas. Véase también `csetup`.

**29.2.10 Nombres reservados**

Los siguientes nombres se utilizan internamente en el paquete `ctensor` y no deberían redefinirse:

| Nombre                   | Descripción                                                                                                  |
|--------------------------|--------------------------------------------------------------------------------------------------------------|
| -----                    |                                                                                                              |
| <code>_lg()</code>       | Toma el valor <code>lfg</code> si se utiliza métrica del sistema de referencia, <code>lg</code> en otro caso |
| <code>_ug()</code>       | Toma el valor <code>ufg</code> si se utiliza métrica del sistema de referencia, <code>ug</code> en otro caso |
| <code>cleanup()</code>   | Elimina elementos de la lista <code>deindex</code>                                                           |
| <code>contract4()</code> | Utilizada por <code>psi()</code>                                                                             |
| <code>filemet()</code>   | Utilizada por <code>csetup()</code> cuando se lee la métrica desde un fichero                                |
| <code>findde1()</code>   | Utilizada por <code>findde()</code>                                                                          |
| <code>findde2()</code>   | Utilizada por <code>findde()</code>                                                                          |
| <code>findde3()</code>   | Utilizada por <code>findde()</code>                                                                          |
| <code>kdelt()</code>     | Delta de Kronecker (no generalizada)                                                                         |
| <code>newmet()</code>    | Utilizada por <code>csetup()</code> para establecer una métrica interactivamente                             |
| <code>setflags()</code>  | Utilizada por <code>init_ctensor()</code>                                                                    |
| <code>readvalue()</code> |                                                                                                              |
| <code>resimp()</code>    |                                                                                                              |
| <code>sermet()</code>    | Utilizada por <code>csetup()</code> para definir una métrica como serie de Taylor                            |
| <code>txyzsum()</code>   |                                                                                                              |
| <code>tmetric()</code>   | Métrica del sistema de referencia, utilizada por <code>cmetric()</code> cuando <code>cframe_flag:true</code> |
| <code>triemann()</code>  | Tensor de Riemann en la base del sistema de referencia, se utiliza cuando <code>cframe_flag:true</code>      |
| <code>tricci()</code>    | Tensor de Ricci en la base del sistema de referencia, se utiliza cuando <code>cframe_flag:true</code>        |
| <code>trrc()</code>      | Coeficientes de rotación de Ricci, utilizada por <code>christof()</code>                                     |
| <code>yesp()</code>      |                                                                                                              |



## 30 atensor

### 30.1 Introducción a atensor

El paquete **atensor** contiene funciones para la manipulación algebraica de tensores. Para hacer uso de **atensor** es necesario cargarlo en memoria haciendo `load(atensor)`, seguido de una llamada a la función `init_atensor`.

La parte más importante de **atensor** es una batería de reglas de simplificación para el producto no conmutativo ("`.`"). El paquete **atensor** reconoce algunos tipos de álgebras; las correspondientes reglas de simplificación se activan tan pronto como se hace una llamada a la función `init_atensor`.

Las capacidades de **atensor** se pueden demostrar definiendo el álgebra de cuaterniones como un álgebra de Clifford  $Cl(0,2)$  con una base de dos vectores. Las tres unidades imaginarias son los dos vectores de la base junto con su producto:

$$\begin{array}{ccccc} i & = & v & & j & = & v & & k & = & v & & . & v \\ & & 1 & & 2 & & 1 & & 2 \end{array}$$

Aunque el paquete **atensor** incluye su propia definición para el álgebra de cuaterniones, no se utiliza en el siguiente ejemplo, en el cual se construye la tabla de multiplicación como una matriz:

```
(%i1) load(atensor);
(%o1) /share/tensor/atensor.mac
(%i2) init_atensor(clifford,0,0,2);
(%o2) done
(%i3) atensimp(v[1].v[1]);
(%o3) - 1
(%i4) atensimp((v[1].v[2]).(v[1].v[2]));
(%o4) - 1
(%i5) q:zeromatrix(4,4);
 [0 0 0 0]
 [
 [0 0 0 0]
(%o5) [
 [0 0 0 0]
 [
 [0 0 0 0]
(%i6) q[1,1]:1;
(%o6) 1
(%i7) for i thru adim do q[1,i+1]:q[i+1,1]:v[i];
(%o7) done
(%i8) q[1,4]:q[4,1]:v[1].v[2];
(%o8) v . v
 1 2
(%i9) for i from 2 thru 4 do for j from 2 thru 4 do
 q[i,j]:atensimp(q[i,1].q[1,j]);
(%o9) done
```



```
(%i10) q;
```

$$\begin{bmatrix} 1 & v & v & v & . & v \\ & 1 & 2 & 1 & 2 & \\ & & & & & \\ v & -1 & v & . & v & -v \\ 1 & & 1 & 2 & 2 & \\ & & & & & \\ v & -v & . & v & -1 & v \\ 2 & 1 & 2 & & 1 & \\ & & & & & \\ v & . & v & v & -v & -1 \\ 1 & 2 & 2 & 1 & & \end{bmatrix}$$

```
(%o10)
```

El paquete **atensor** reconoce como vectores de la base símbolos indexados, donde el símbolo es el almacenado en **asymbol** y el índice va desde 1 hasta **adim**. Para símbolos indexados, y sólo para ellos, se evalúan las formas bilineales **sf**, **af** y **av**. La evaluación sustituye el valor de **aform[i,j]** en lugar de **fun(v[i],v[j])**, donde **v** representa el valor de **asymbol** y **fun** es **af** o **sf**; o sustituye **v[aform[i,j]]** en lugar de **av(v[i],v[j])**.

Huelga decir que las funciones **sf**, **af** y **av** pueden volver a definirse.

Cuando se carga el paquete **atensor** se hacen las siguientes asignaciones de variables:

```
dotscrules:true;
dotdistrib:true;
dotexptsimp:false;
```

Si se quiere experimentar con una álgebra no asociativa, también se puede igualar la variable **dotassoc** a **false**. En tal caso, sin embargo, **atensimp** no será siempre capaz de realizar las simplificaciones deseadas.

## 30.2 Definiciones para atensor

**init\_atensor** (*alg\_type*, *opt\_dims*)

Función

**init\_atensor** (*alg\_type*)

Función

Inicializa el paquete **atensor** con el tipo de álgebra especificado, *alg\_type*, que puede ser una de las siguientes:

**universal**: El álgebra universal no tiene reglas de conmutación.

**grassmann**: El álgebra de Grassman se define mediante la relación de conmutación  $u.v+v.u=0$ .

**clifford**: El álgebra de Clifford se define mediante la regla de conmutación  $u.v+v.u=-2*sf(u,v)$  donde **sf** es una función escalar simétrica. Para esta álgebra, *opt\_dims* puede contener hasta tres enteros no negativos, que representan el número de dimensiones positivas, degeneradas y negativas, respectivamente, de esta álgebra. Si se suministran los valores de *opt\_dims*, **atensor** configurará los valores de **adim** y **aform** de forma apropiada. En otro caso, **adim** tomará por defecto el valor 0 y **aform** no se definirá.

**symmetric**: El álgebra simétrica se define mediante la regla de conmutación  $u.v-v.u=0$ .

**symplectic**: El álgebra simpléctica se define mediante la regla de conmutación  $u.v-v.u=2*af(u,v)$ , donde **af** es una función escalar antisimétrica. Para el álgebra

simplética, `opt_dims` puede contener hasta dos enteros no negativos, que representan las dimensiones no degeneradas y degeneradas, respectivamente. Si se suministran los valores de `opt_dims`, `atensor` configurará los valores de `adim` y `aform` de forma apropiada. En otro caso, `adim` tomará por defecto el valor 0 y `aform` no se definirá.

`lie_envelop`: El álgebra de la envolvente de Lie se define mediante la regla de conmutación  $u.v - v.u = 2*av(u,v)$ , donde `av` es una función antisimétrica.

La función `init_atensor` también reconoce algunos tipos de álgebras predefinidas:

`complex` implementa el álgebra de números complejos como un álgebra de Clifford  $Cl(0,1)$ . La llamada `init_atensor(complex)` equivale a `init_atensor(clifford,0,0,1)`.

`quaternion` implementa el álgebra de cuaterniones. La llamada `init_atensor(quaternion)` equivale a `init_atensor(clifford,0,0,2)`.

`pauli` implementa el álgebra de Pauli como un álgebra de Clifford  $Cl(3,0)$ . La llamada `init_atensor(pauli)` equivale a `init_atensor(clifford,3)`.

`dirac` implementa el álgebra de Dirac como un álgebra de Clifford  $Cl(3,1)$ . La llamada `init_atensor(dirac)` equivale a `init_atensor(clifford,3,0,1)`.

**atensimp** (*expr*) Función

Simplifica la expresión algebraica de un tensor *expr* de acuerdo con las reglas configuradas mediante una llamada a `init_atensor`. La simplificación incluye la aplicación recursiva de las reglas de conmutación y llamadas a `sf`, `af` y `av` siempre que sea posible. Se utiliza un algoritmo que asegure que la función termina siempre, incluso en el caso de expresiones complejas.

**alg\_type** Función

Tipo de álgebra. Valores válidos son `universal`, `grassmann`, `clifford`, `symmetric`, `symplectic` y `lie_envelop`.

**adim** Variable

Valor por defecto: 0

La dimensión del álgebra. El paquete `atensor` utiliza el valor de `adim` para determinar si un objeto indexado es un vector válido para la base. Véase `abasep`.

**aform** Variable

Valor por defecto: `ident(3)`

Valores por defecto para las formas bilineales `sf`, `af` y `av`. El valor por defecto es la matriz identidad `ident(3)`.

**asymbol** Variable

Valor por defecto: `v`

Símbolo para los vectores base.

**sf** (*u*, *v*) Función

Una función escalar simétrica que se utiliza en relaciones de conmutación. La implementación por defecto analiza si los dos argumentos son vectores base mediante `abasep` y en tal caso sustituye el valor correspondiente de la matriz `aform`.

**af** ( $u, v$ ) Función

Una función escalar antisimétrica que se utiliza en relaciones de conmutación. La implementación por defecto analiza si los dos argumentos son vectores base mediante **abasep** y en tal caso sustituye el valor correspondiente de la matriz **aform**.

**av** ( $u, v$ ) Función

Una función antisimétrica que se utiliza en relaciones de conmutación. La implementación por defecto analiza si los dos argumentos son vectores base mediante **abasep** y en tal caso sustituye el valor correspondiente de la matriz **aform**.

Ejemplo:

```
(%i1) load(atensor);
(%o1) /share/tensor/atensor.mac
(%i2) adim:3;
(%o2) 3
(%i3) aform:matrix([0,3,-2],[-3,0,1],[2,-1,0]);
 [0 3 -2]
 []
(%o3) [-3 0 1]
 []
 [2 -1 0]

(%i4) asymbol:x;
(%o4) x
(%i5) av(x[1],x[2]);
(%o5) x
 3
```

**abasep** ( $v$ ) Función

Analiza si su argumento es un vector base en **atensor**. Esto es, si se trata de un símbolo indexado, siendo el símbolo el mismo que el valor de **asymbol** y si el índice tiene un valor numérico entre 1 y **adim**.

## 31 Series

### 31.1 Introducción a las Series

Maxima dispone de las funciones `taylor` y `powerseries` para calcular las series de las funciones diferenciables. También tiene herramientas como `nusum` capaces de encontrar la expresión compacta de algunas series. Operaciones como la suma y la multiplicación operan de la forma habitual en el contexto de las series. Esta sección presenta las variables globales que controlan la expansión.

### 31.2 Definiciones para las Series

#### `cauchysum`

Variable opcional

Valor por defecto: `false`

Cuando se multiplican sumatorios infinitos, si `sumexpand` vale `true` y `cauchysum` vale `true`, entonces se utilizará el producto de Cauchy en lugar del usual. En el producto de Cauchy el índice de la suma interna es función del índice de la exterior en lugar de variar de forma independiente. Un ejemplo aclara esta idea:

```
(%i1) sumexpand: false$
(%i2) cauchysum: false$
(%i3) s: sum (f(i), i, 0, inf) * sum (g(j), j, 0, inf);

 inf inf
 ==== ====
 \ \
(%o3) (> f(i)) > g(j)
 / /
 ==== ====
 i = 0 j = 0

(%i4) sumexpand: true$
(%i5) cauchysum: true$
(%i6) ''s;

 inf i1
 ==== ====
 \ \
(%o6) > > g(i1 - i2) f(i2)
 / /
 ==== ====
 i1 = 0 i2 = 0
```

#### `deftaylor` ( $f_1(x_1)$ , $expr_1$ , ..., $f_n(x_n)$ , $expr_n$ )

Función

Para cada función  $f_i$  de variable  $x_i$ , `deftaylor` define  $expr_i$  como una serie de Taylor alrededor de cero. La expresión  $expr_i$  será un polinomio en  $x_i$  o una suma; `deftaylor` admite también expresiones más generales.

La llamada `powerseries` ( $f_i(x_i)$ ,  $x_i$ , 0) devuelve la serie definida por `deftaylor`.

La función `deftaylor` evalúa sus argumentos y devuelve la lista de las funciones  $f_1$ , ...,  $f_n$ .

Ejemplo:

```
(%i1) deftaylor (f(x), x^2 + sum(x^i/(2^i*i!^2), i, 4, inf));
(%o1) [f]
(%i2) powerseries (f(x), x, 0);
 inf
 ====
 \ i1
 > x 2
 / i1 2
 / 2 i1!
 / i1 = 4
(%i3) taylor (exp (sqrt (f(x))), x, 0, 4);
 2 3 4
 x 3073 x 12817 x
(%o3)/T/ 1 + x + -- + ----- + ----- + . . .
 2 18432 307200
```

### maxtayorder

Variable opcional

Valor por defecto: `true`

Si `maxtayorder` vale `true`, entonces durante la manipulación algebraica de series truncadas de Taylor, la función `taylor` trata de retener tantos términos correctos como sea posible.

### niceindices (expr)

Función

Cambia las etiquetas de los índices de sumas y productos de `expr`. La función `niceindices` trata de cambiar cada índice al valor de `niceindicespref[1]`, a menos que esa etiqueta aparezca ya en el sumando o factor, en cuyo caso `niceindices` realiza intentos con los siguientes elementos de `niceindicespref`, hasta que encuentre una variable que no esté en uso. Si todas las variables de la lista han sido ya revisadas, se formarán nuevos índices añadiendo números enteros al valor de `niceindicespref[1]`, como `i0`, `i1`, `i2`, ....

La función `niceindices` evalúa sus argumentos y devuelve una expresión.

Ejemplo:

```
(%i1) niceindicespref;
(%o1) [i, j, k, l, m, n]
(%i2) product (sum (f (foo + i*j*bar), foo, 1, inf), bar, 1, inf);
 inf inf
 /====\ =====
 ! ! \
 ! ! > f(bar i j + foo)
 ! ! /
 bar = 1 =====
 foo = 1
(%i3) niceindices (%);
 inf inf
 /====\ =====
 ! ! \
```

```
(%o3) ! ! > f(i j l + k)
 ! ! /
 1 = 1 =====
 k = 1
```

**niceindicespref**

Variable opcional

Valor por defecto: [i, j, k, l, m, n]

La variable **niceindicespref** es la lista de la que la función **niceindices** va tomando nombres de etiquetas para índices de sumatorios y productos.

En **niceindicespref** se guardan normalmente nombres de variables.

Ejemplo:

```
(%i1) niceindicespref: [p, q, r, s, t, u]$
(%i2) product (sum (f (foo + i*j*bar), foo, 1, inf), bar, 1, inf);
 inf inf
 /====\ =====
 ! ! \
(%o2) ! ! > f(bar i j + foo)
 ! ! /
 bar = 1 =====
 foo = 1
(%i3) niceindices (%);
 inf inf
 /====\ =====
 ! ! \
(%o3) ! ! > f(i j q + p)
 ! ! /
 q = 1 =====
 p = 1
```

**nusum** (*expr*, *x*, *i.0*, *i.1*)

Función

Calcula la suma hipergeométrica indefinida de *expr* con respecto a la variable *x* utilizando un procedimiento de decisión debido a R.W. Gosper. La expresión *expr* y el resultado deben poder ser escritos como productos de potencias enteras, factoriales, coeficientes binomiales y funciones racionales.

Los términos suma "definida" e "indefinida" se usan de forma análoga a integración "definida" e "indefinida". La suma indefinida significa dar un resultado simbólico.

Las funciones **nusum** y **unsum** disponen de cierta información sobre sumas y diferencias de productos finitos. Véase también **unsum**.

Ejemplos:

```
(%i1) nusum (n*n!, n, 0, n);

Dependent equations eliminated: (1)
(%o1) (n + 1)! - 1
(%i2) nusum (n^4*4^n/binomial(2*n,n), n, 0, n);
 4 3 2 n
2 (n + 1) (63 n + 112 n + 18 n - 22 n + 3) 4 2
```

```

(%o2) -----
 693 binomial(2 n, n)
 3 11 7
(%i3) unsum (% , n);
 4 n
 n 4
(%o3) -----
 binomial(2 n, n)
(%i4) unsum (prod (i^2, i, 1, n), n);
 n - 1
 /===\
 ! ! 2
(%o4) (! ! i) (n - 1) (n + 1)
 ! !
 i = 1
(%i5) nusum (% , n, 1, n);

Dependent equations eliminated: (2 3)
 n
 /===\
 ! ! 2
(%o5) (! ! i - 1)
 ! !
 i = 1

```

**pade** (*taylor\_series*, *numer\_deg\_bound*, *denom\_deg\_bound*) Función

Devuelve la lista de todas las funciones racionales que tienen el desarrollo de Taylor dado, en las que la suma de los grados del numerador y denominador es menor o igual que el nivel de truncamiento de la serie de potencias.

La expresión *taylor\_series* es una serie de Taylor univariante. Los argumentos *numer\_deg\_bound* y *denom\_deg\_bound* son enteros positivos que indican las cotas para numerador y denominador.

La expresión *taylor\_series* también puede ser una serie de Laurent, y las cotas de los grados pueden ser *inf*. El grado total se define como *numer\_deg\_bound* + *denom\_deg\_bound*. La longitud de una serie de potencias se define como "truncation level" + 1 - min(0, "order of series").

```

(%i1) taylor (1 + x + x^2 + x^3, x, 0, 3);
 2 3
(%o1) /T/ 1 + x + x + x + . . .
(%i2) pade (% , 1, 1);
 1
(%o2) [- ----]
 x - 1
(%i3) t: taylor(-(83787*x^10 - 45552*x^9 - 187296*x^8
 + 387072*x^7 + 86016*x^6 - 1507328*x^5
 + 1966080*x^4 + 4194304*x^3 - 25165824*x^2
 + 67108864*x - 134217728)
 /134217728, x, 0, 10);
 2 3 4 5 6 7

```

```
(%o3)/T/ 1 - $\frac{x}{2}$ + $\frac{3x}{16}$ - $\frac{x}{32}$ - $\frac{15x}{1024}$ + $\frac{23x}{2048}$ - $\frac{21x}{32768}$ - $\frac{189x}{65536}$
+ $\frac{5853x^8}{4194304}$ + $\frac{2847x^9}{8388608}$ - $\frac{83787x^{10}}{134217728}$ + . . .
(%i4) pade (t, 4, 4);
(%o4) []
```

No hay ninguna función racional de grado 4 en numerador y denominador con este desarrollo en serie de potencias. Es necesario dar un número de grados al numerador y denominador cuya suma sea al menos el grado del desarrollo de la serie, a fin de disponer de un número suficiente de coeficientes desconocidos para calcular.

```
(%i5) pade (t, 5, 5);
(%o5) [- (520256329 x5 - 96719020632 x4 - 489651410240 x3
- 1619100813312 x2 - 2176885157888 x - 2386516803584)
/(47041365435 x5 + 381702613848 x4 + 1360678489152 x3
+ 2856700692480 x2 + 3370143559680 x + 2386516803584)]
```

### powerdisp

Variable opcional

Valor por defecto: `false`

Si `powerdisp` vale `true`, se muestran las sumas con sus términos ordenados de menor a mayor potencia. Así, un polinomio se presenta como una serie de potencias truncada con el término constante al principio y el de mayor potencia al final.

Por defecto, los términos de una suma se muestran en el orden de las potencias decrecientes.

### powerseries (expr, x, a)

Función

Devuelve la forma general del desarrollo en serie de potencias de `expr` para la variable `x` alrededor del punto `a` (que puede ser `inf`, de infinito).

Si `powerseries` no es capaz de desarrollar `expr`, la función `taylor` puede calcular los primeros términos de la serie.

Si `verbose` vale `true`, `powerseries` va mostrando mensajes mientras progresa el cálculo.

```
(%i1) verbose: true$
(%i2) powerseries (log(sin(x)/x), x, 0);
can't expand
log(sin(x))
```



so we'll try again after applying the rule:

$$\log(\sin(x)) = \frac{\frac{d}{dx} (\sin(x))}{\sin(x)}$$

in the first simplification we have returned:

$$\frac{i \cot(x) dx - \log(x)}{\frac{(-1)^{i1} \frac{2^{i1}}{2} \text{bern}(2 i1) x}{i1 (2 i1)!}}$$

(%o2)

## psexpand

Variable opcional

Valor por defecto: **false**

Si **psexpand** vale **true**, toda expresión racional se muestra completamente expandida. La variable **ratexpand** tiene el mismo efecto.

Si **psexpand** vale **false**, las expresiones multivariantes se presentan tal como lo hace el paquete de funciones racionales.

Si **psexpand** vale **multi**, los términos de igual grado son agrupados.

## revert (expr, x)

Función

## revert2 (expr, x, n)

Función

Estas funciones devuelven el recíproco de *expr* en forma de desarrollo de Taylor alrededor de cero respecto de la variable *x*. La función **revert** devuelve un polinomio de grado igual a la mayor potencia en *expr*. La función **revert2** devuelve un polinomio de grado *n*, el cual puede ser mayor, igual o menor que el grado de *expr*.

Para utilizar estas funciones es necesario cargarlas en memoria mediante **load ("revert")**.

Ejemplos:

```
(%i1) load ("revert")$
(%i2) t: taylor (exp(x) - 1, x, 0, 6);

 2 3 4 5 6
 x x x x x
(%o2)/T/ x + -- + -- + -- + -- + -- + . . .
 2 6 24 120 720

(%i3) revert (t, x);
```

```

 6 5 4 3 2
 10 x - 12 x + 15 x - 20 x + 30 x - 60 x
(%o3)/R/ -----
 60
(%i4) ratexpand (%);
 6 5 4 3 2
 x x x x x
(%o4) - --- + --- - --- + --- - --- + x
 6 5 4 3 2
(%i5) taylor (log(x+1), x, 0, 6);
 2 3 4 5 6
 x x x x x
(%o5)/T/ x - --- + --- - --- + --- - --- + . . .
 2 3 4 5 6
(%i6) ratsimp (revert (t, x) - taylor (log(x+1), x, 0, 6));
(%o6)
 0
(%i7) revert2 (t, x, 4);
 4 3 2
 x x x
(%o7) - --- + --- - --- + x
 4 3 2

```

|                                                                                                                                                                                                       |         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| <b>taylor</b> ( <i>expr</i> , <i>x</i> , <i>a</i> , <i>n</i> )                                                                                                                                        | Función |
| <b>taylor</b> ( <i>expr</i> , [ <i>x</i> <sub>1</sub> , <i>x</i> <sub>2</sub> , ...], <i>a</i> , <i>n</i> )                                                                                           | Función |
| <b>taylor</b> ( <i>expr</i> , [ <i>x</i> , <i>a</i> , <i>n</i> , 'asympt'])                                                                                                                           | Función |
| <b>taylor</b> ( <i>expr</i> , [ <i>x</i> <sub>1</sub> , <i>x</i> <sub>2</sub> , ...], [ <i>a</i> <sub>1</sub> , <i>a</i> <sub>2</sub> , ...], [ <i>n</i> <sub>1</sub> , <i>n</i> <sub>2</sub> , ...]) | Función |

La llamada **taylor** (*expr*, *x*, *a*, *n*) expande la expresión *expr* en un desarrollo de Taylor o de Laurent respecto de la variable *x* alrededor del punto *a*, con términos hasta  $(x - a)^n$ .

Si *expr* es de la forma  $f(x)/g(x)$  y  $g(x)$  no tiene términos hasta de grado *n*, entonces **taylor** intenta expandir  $g(x)$  hasta el grado  $2n$ . Si aún así no hay términos no nulos, **taylor** dobla el grado de la expansión de  $g(x)$  hasta que el grado de la expansión sea menor o igual que  $n^2 \text{taylordepth}$ .

La llamada **taylor** (*expr*, [*x*<sub>1</sub>, *x*<sub>2</sub>, ...], *a*, *n*) devuelve la serie en potencias truncada de grado *n* en todas las variables *x*<sub>1</sub>, *x*<sub>2</sub>, ... alrededor del punto (*a*, *a*, ...).

La llamada **taylor** (*expr*, [*x*<sub>1</sub>, *a*<sub>1</sub>, *n*<sub>1</sub>], [*x*<sub>2</sub>, *a*<sub>2</sub>, *n*<sub>2</sub>], ...) devuelve la serie en potencias truncada en las variables *x*<sub>1</sub>, *x*<sub>2</sub>, ... alrededor del punto (*a*<sub>1</sub>, *a*<sub>2</sub>, ...); el truncamiento se realiza, respectivamente, en los grados *n*<sub>1</sub>, *n*<sub>2</sub>, ....

La llamada **taylor** (*expr*, [*x*<sub>1</sub>, *x*<sub>2</sub>, ...], [*a*<sub>1</sub>, *a*<sub>2</sub>, ...], [*n*<sub>1</sub>, *n*<sub>2</sub>, ...]) devuelve la serie en potencias truncada en las variables *x*<sub>1</sub>, *x*<sub>2</sub>, ... alrededor del punto (*a*<sub>1</sub>, *a*<sub>2</sub>, ...), el truncamiento se realiza, respectivamente, en los grados *n*<sub>1</sub>, *n*<sub>2</sub>, ....

La llamada **taylor** (*expr*, [*x*, *a*, *n*, 'asympt']) devuelve el desarrollo de *expr* en potencias negativas de  $x - a$ . El término de mayor orden es  $(x - a)^{-n}$ .

Si `maxtayorder` vale `true`, entonces durante la manipulación algebraica de las series (truncadas) de Taylor, la función `taylor` intenta mantener tantos términos correctos como sea posible.

Si `psexpand` vale `true`, una expresión racional desarrollada se muestra completamente expandida. La variable `ratexpand` tiene el mismo efecto. Si `psexpand` vale `false`, una expresión multivariante se mostrará tal como lo hace el paquete de funciones racionales. Si `psexpand` vale `multi`, los términos del mismo grado son agrupados.

Véase también la variable `taylor_logexpand` para el control del desarrollo.

Ejemplos:

```
(%i1) taylor (sqrt (sin(x) + a*x + 1), x, 0, 3);
```

```
(%o1)/T/ 1 +
$$\frac{(a+1)x^2}{2} - \frac{(a^2+2a+1)x^3}{8} + \frac{(3a^3+9a^2+9a-1)x^4}{48} + \dots$$

```

```
(%i2) %^2;
```

```
(%o2)/T/ 1 + (a+1) x -
$$\frac{x^3}{6} + \dots$$

```

```
(%i3) taylor (sqrt (x + 1), x, 0, 5);
```

```
(%o3)/T/ 1 +
$$\frac{x^2}{2} - \frac{x^3}{8} + \frac{x^4}{16} - \frac{5x^5}{128} + \frac{7x^6}{256} + \dots$$

```

```
(%i4) %^2;
```

```
(%o4)/T/ 1 + x + . . .
```

```
(%i5) product ((1 + x^i)^2.5, i, 1, inf)/(1 + x^2);
```

```
(%o5)
$$\frac{\prod_{i=1}^{\infty} (1+x^i)^{2.5}}{1+x^2}$$

```

```
(%i6) ev (taylor(%, x, 0, 3), keepfloat);
```

```
(%o6)/T/ 1 + 2.5 x + 3.375 x^2 + 6.5625 x^3 + . . .
```

```
(%i7) taylor (1/log (x + 1), x, 0, 3);
```

```
(%o7)/T/ 1 -
$$\frac{x}{2} + \frac{x^2}{4} - \frac{19x^3}{8} + \dots$$

```

```

(%i8) taylor (cos(x) - sec(x), x, 0, 5);

$$\frac{x^2}{4} - \frac{x^{12}}{6} + \frac{x^{24}}{6} - \frac{x^{720}}{6} + \dots$$

(%o8)/T/
(%i9) taylor ((cos(x) - sec(x))^3, x, 0, 5);
(%o9)/T/
(%i10) taylor (1/(cos(x) - sec(x))^3, x, 0, 5);

$$\frac{1}{x^6} + \frac{1}{2x^4} + \frac{11}{120x^2} - \frac{347}{15120} - \frac{6767x}{604800} - \frac{15377x^4}{7983360} + \dots$$

(%o10)/T/
(%i11) taylor (sqrt (1 - k^2*sin(x)^2), x, 0, 6);

$$\frac{k^2 x^2}{2} - \frac{(3k^4 - 4k^2)x^4}{24} - \frac{(45k^6 - 60k^4 + 16k^2)x^6}{720} + \dots$$

(%o11)/T/
(%i12) taylor ((x + 1)^n, x, 0, 4);

$$1 + nx + \frac{(n^2 - n)x^2}{2} + \frac{(n^3 - 3n^2 + 2n)x^3}{6} + \frac{(n^4 - 6n^3 + 11n^2 - 6n)x^4}{24} + \dots$$

(%o12)/T/
(%i13) taylor (sin (y + x), x, 0, 3, y, 0, 3);

$$y - \frac{y^3}{6} + \dots + (1 - \frac{y^2}{2} + \dots)x + (-\frac{y^3}{2} + \frac{y^2}{12} + \dots)x^2 + (-\frac{y^3}{6} + \frac{y^2}{12} + \dots)x^3 + \dots$$

(%o13)/T/
(%i14) taylor (sin (y + x), [x, y], 0, 3);

$$x^3 + 3yx^2 + 3y^2x + y^3$$


```

```
(%o14)/T/ y + x - ----- + . . .
 6
(%i15) taylor (1/sin (y + x), x, 0, 3, y, 0, 3);
(%o15)/T/ 1 y 1 1 1 2
 - + - + . . . + (- -- + - + . . .) x + (-- + . . .) x
 y 6 2 6 y 3
 y
 1 3
 + (- -- + . . .) x + . . .
 4
 y
(%i16) taylor (1/sin (y + x), [x, y], 0, 3);
(%o16)/T/ 1 x + y 3 2 2 3
 ----- + ----- + ----- + . . .
 x + y 6 360
```

**taylordepth**

Variable opcional

Valor por defecto: 3

Si todavía no hay términos no nulos, la función **taylor** dobla el grado del desarrollo de  $g(x)$  tantas veces como sea necesario para que el grado del desarrollo sea menor o igual que  $n 2^{\text{taylordepth}}$ .

**taylorinfo** (*expr*)

Función

Devuelve información sobre el desarrollo de Taylor *expr*. El valor devuelto por esta función es una lista de listas. Cada lista contiene el nombre de una variable, el punto de expansión y el grado del desarrollo.

La función **taylorinfo** devuelve **false** si *expr* no es un desarrollo de Taylor.

Ejemplo:

```
(%i1) taylor ((1 - y^2)/(1 - x), x, 0, 3, [y, a, inf]);
(%o1)/T/ - (y - a)2 - 2 a (y - a)2 + (1 - a)2
 2 2 2
 + (1 - a - 2 a (y - a) - (y - a)) x
 2 2 2
 + (1 - a - 2 a (y - a) - (y - a)) x
 2 2 3
 + (1 - a - 2 a (y - a) - (y - a)) x + . . .
(%i2) taylorinfo(%);
(%o2) [[y, a, inf], [x, 0, 3]]
```

**taylorp** (*expr*)

Función

Devuelve **true** si *expr* es un desarrollo de Taylor y **false** en caso contrario.

**taylor\_logexpand**

Variable opcional

Valor por defecto: `true`

La variable `taylor_logexpand` controla los desarrollos de logaritmos en la función `taylor`.

Si `taylor_logexpand` vale `true`, todos los logaritmos se expanden completamente de manera que algunos problemas que se plantean debido a ciertas identidades logarítmicas no interfieran con el proceso del cálculo del desarrollo de Taylor. Sin embargo, este proceder no es del todo correcto.

**taylor\_order\_coefficients**

Variable opcional

Valor por defecto: `true`

La variable `taylor_order_coefficients` controla la ordenación de los coeficientes en un desarrollo de Taylor.

Si `taylor_order_coefficients` vale `true`, los coeficientes del desarrollo de Taylor se ordenan de la forma canónica.

**taylor\_simplifier** (*expr*)

Función

Simplifica los coeficientes de la serie de potencias *expr*. Esta función es llamada desde la función `taylor`.

**taylor\_truncate\_polynomials**

Variable opcional

Valor por defecto: `true`

Si `taylor_truncate_polynomials` vale `true`, los polinomios quedan truncados en base a los niveles de truncamiento de entrada.

En otro caso, aquellos polinomios que se utilicen como entrada a la función `taylor` se consideran que tienen precisión infinita.

**taylorat** (*expr*)

Función

Convierte *expr* del formato de `taylor` al formato CRE (Canonical Rational Expression). El efecto es el mismo que haciendo `rat (ratdisrep (expr))`, pero más rápido.

**trunc** (*expr*)

Función

Devuelve la representación interna de la expresión *expr* de tal forma como si sus sumas fuesen una serie truncada de Taylor. La expresión *expr* no sufre ninguna otra modificación.

Ejemplo:

```
(%i1) expr: x^2 + x + 1;
(%o1) 2
 x + x + 1
(%i2) trunc (expr);
(%o2) 2
 1 + x + x + . . .
(%i3) is (expr = trunc (expr));
(%o3) true
```

**unsum** ( $f, n$ )

Función

Devuelve la diferencia  $f(n) - f(n - 1)$ . En cierto sentido **unsum** es la inversa de **sum**.

Véase también **nusum**.

Ejemplos:

```
(%i1) g(p) := p*4^n/binomial(2*n,n);
```

$$g(p) := \frac{p^4}{\text{binomial}(2n, n)}$$

```
(%o1)
```

```
(%i2) g(n^4);
```

$$\frac{n^4}{\text{binomial}(2n, n)}$$

```
(%o2)
```

```
(%i3) nusum (%, n, 0, n);
```

$$\frac{2(n+1)(63n^4 + 112n^3 + 18n^2 - 22n + 3)}{693 \text{binomial}(2n, n)} - \frac{2}{3}$$

```
(%o3)
```

```
(%i4) unsum (%, n);
```

$$\frac{n^4}{\text{binomial}(2n, n)}$$

```
(%o4)
```

**verbose**

Variable opcional

Valor por defecto: **false**

Si **verbose** vale **true**, la función **powerseries** va imprimiendo mensajes durante su ejecución.

## 32 Teoría de Números

### 32.1 Definiciones para Teoría de Números

**bern** ( $n$ ) Función

Devuelve el  $n$ -ésimo número de Bernoulli del entero  $n$ . Los números de Bernoulli iguales a cero son suprimidos si **zerobern** vale **false**.

Véase también **burn**.

```
(%i1) zerobern: true$
(%i2) map (bern, [0, 1, 2, 3, 4, 5, 6, 7, 8]);
 1 1 1 1 1
(%o2) [1, - -, -, 0, - --, 0, --, 0, - --]
 2 6 30 42 30

(%i3) zerobern: false$
(%i4) map (bern, [0, 1, 2, 3, 4, 5, 6, 7, 8]);
 1 1 1 5 691 7 3617 43867
(%o4) [1, - -, -, - --, --, - ----, -, - ----, -----]
 2 6 30 66 2730 6 510 798
```

**bernpoly** ( $x, n$ ) Función

Devuelve el  $n$ -ésimo polinomio de Bernoulli de variable  $x$ .

**bfzeta** ( $s, n$ ) Función

Devuelve la función zeta de Riemann para el argumento  $s$ . El valor que devuelve es del tipo "big float" (bfloat) y  $n$  es su número de dígitos.

Es necesario cargar en memoria esta función haciendo **load ("bffac")**.

**bfhzeta** ( $s, h, n$ ) Función

Devuelve la función zeta de Hurwitz para los argumentos  $s$  y  $h$ . El valor que devuelve es del tipo "big float" (bfloat) y  $n$  es su número de dígitos.

La función zeta de Hurwitz se define como

$$\sum ((k+h)^{-s}, k, 0, \text{inf})$$

Es necesario cargar en memoria esta función haciendo **load ("bffac")**.

**binomial** ( $x, y$ ) Función

Es el coeficiente binomial  $x!/(y! (x - y)!)$ . Si  $x$  y  $y$  son enteros, entonces se calcula el valor numérico del coeficiente binomial. Si  $y$  o  $x - y$  son enteros, el coeficiente binomial se expresa como un polinomio.

Ejemplos:

```
(%i1) binomial (11, 7);
(%o1) 330
(%i2) 11! / 7! / (11 - 7)!;
(%o2) 330
(%i3) binomial (x, 7);
```



```

(%o3) (x - 6) (x - 5) (x - 4) (x - 3) (x - 2) (x - 1) x

 5040
(%i4) binomial (x + 7, x);
 (x + 1) (x + 2) (x + 3) (x + 4) (x + 5) (x + 6) (x + 7)
(%o4) -----
 5040
(%i5) binomial (11, y);
(%o5) binomial(11, y)

```

**burn** (*n*)

Función

Devuelve el *n*-ésimo número de Bernoulli del entero *n*. La función **burn** puede ser más eficiente que **bern** para *n* grande (mayor que 105, por ejemplo), pues **bern** calcula todos los números de Bernoulli hasta *n* antes de devolver el resultado.

La función **burn** se beneficia del hecho de que los números racionales de Bernoulli pueden aproximarse por funciones zeta con una eficiencia aceptable.

Es necesario cargar en memoria esta función haciendo **load ("bffac")**.

**cf** (*expr*)

Función

Transforma *expr* a fracciones continuas. La expresión *expr* debe contener fracciones continuas y raíces cuadradas de números enteros. Los operandos de la expresión pueden combinarse con operadores aritméticos. Además de fracciones continuas y raíces cuadradas, los factores de la expresión deben ser enteros o números racionales. Maxima no tiene más conocimiento sobre operaciones con fracciones continuas que el que aporta la función **cf**.

La función **cf** evalúa sus argumentos después de asignar a la variable **listarith** el valor **false**, retornando una fracción continua en forma de lista.

Una fracción continua  $a + 1/(b + 1/(c + \dots))$  se representa como la lista **[a, b, c, ...]**, donde los elementos *a*, *b*, *c*, ... se evalúan como enteros. La expresión *expr* puede contener también **sqrt (n)** donde *n* es un entero; en tal caso, **cf** devolverá tantos términos de la fracción continua como indique el valor de la variable **cflength** multiplicado por el período.

Una fracción continua puede reducirse a un número evaluando la representación aritmética que devuelve **cfdisrep**. Véase también **cfexpand**, que es otra alternativa para evaluar fracciones continuas.

Véanse asimismo **cfdisrep**, **cfexpand** y **cflength**.

Ejemplos:

- La expresión *expr* contiene fracciones continuas y raíces cuadradas de enteros.

```

(%i1) cf ([5, 3, 1]*[11, 9, 7] + [3, 7]/[4, 3, 2]);
(%o1) [59, 17, 2, 1, 1, 1, 27]
(%i2) cf ((3/17)*[1, -2, 5]/sqrt(11) + (8/13));
(%o2) [0, 1, 1, 1, 3, 2, 1, 4, 1, 9, 1, 9, 2]

```

- La variable **cflength** controla cuantos períodos de la fracción continua se calculan para números irracionales algebraicos.

```
(%i1) cflength: 1$
(%i2) cf ((1 + sqrt(5))/2);
(%o2) [1, 1, 1, 1, 2]
(%i3) cflength: 2$
(%i4) cf ((1 + sqrt(5))/2);
(%o4) [1, 1, 1, 1, 1, 1, 2]
(%i5) cflength: 3$
(%i6) cf ((1 + sqrt(5))/2);
(%o6) [1, 1, 1, 1, 1, 1, 1, 1, 2]
```

- Una fracción continua puede calcularse evaluando la representación aritmética que devuelve `cfdisrep`.

```
(%i1) cflength: 3$
(%i2) cfdisrep (cf (sqrt (3)))$
(%i3) ev (% , numer);
(%o3) 1.731707317073171
```

- Maxima no sabe sobre operaciones con fracciones continuas más de lo que aporta la función `cf`.

```
(%i1) cf ([1,1,1,1,1,2] * 3);
(%o1) [4, 1, 5, 2]
(%i2) cf ([1,1,1,1,1,2]) * 3;
(%o2) [3, 3, 3, 3, 3, 6]
```

**cfdisrep** (*lista*)

Función

Construye y devuelve una expresión aritmética ordinaria de la forma  $a + 1/(b + 1/(c + \dots))$  a partir de la representación en formato lista de la fracción continua  $[a, b, c, \dots]$ .

```
(%i1) cf ([1, 2, -3] + [1, -2, 1]);
(%o1) [1, 1, 1, 2]
(%i2) cfdisrep (%);
(%o2) 1 + 1 / (1 + 1 / (1 + 1 / 2))
```

**cfexpand** (*x*)

Función

Devuelve la matriz con los numeradores y denominadores de la última (columna 1) y penúltima (columna 2) convergentes de la fracción continua  $x$ .

```
(%i1) cf (rat (ev (%pi, numer)));
'rat' replaced 3.141592653589793 by 103993//33102 = 3.141592653011902
(%o1) [3, 7, 15, 1, 292]
(%i2) cfexpand (%);
(%o2) [103993 355]
 []
```

```

 [33102 113]
(%i3) %[1,1]/ %[2,1], numer;
(%o3) 3.141592653011902

```

**cflength**

Variable opcional

Valor por defecto: 1

La variable **cflength** controla el número de términos de la fracción continua que devuelve la función **cf**, que será **cflength** multiplicado por el período. Así, el valor por defecto será el de un período.

```

(%i1) cflength: 1$
(%i2) cf ((1 + sqrt(5))/2);
(%o2) [1, 1, 1, 1, 2]
(%i3) cflength: 2$
(%i4) cf ((1 + sqrt(5))/2);
(%o4) [1, 1, 1, 1, 1, 1, 2]
(%i5) cflength: 3$
(%i6) cf ((1 + sqrt(5))/2);
(%o6) [1, 1, 1, 1, 1, 1, 1, 1, 2]

```

**divsum** (*n*, *k*)

Función

**divsum** (*n*)

Función

La llamada **divsum** (*n*, *k*) devuelve la suma de los divisores de *n* elevados a la *k*-ésima potencia.

La llamada **divsum** (*n*) devuelve la suma de los divisores de *n*.

```

(%i1) divsum (12);
(%o1) 28
(%i2) 1 + 2 + 3 + 4 + 6 + 12;
(%o2) 28
(%i3) divsum (12, 2);
(%o3) 210
(%i4) 1^2 + 2^2 + 3^2 + 4^2 + 6^2 + 12^2;
(%o4) 210

```

**euler** (*n*)

Función

Devuelve el *n*-ésimo número de Euler del entero no negativo *n*.

Para la constante de Euler-Mascheroni, véase **%gamma**.

```

(%i1) map (euler, [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]);
(%o1) [1, 0, - 1, 0, 5, 0, - 61, 0, 1385, 0, - 50521]

```

**%gamma**

Constante

Es la constante de Euler-Mascheroni, 0.5772156649015329 ....

**factorial** (*x*)

Función

Representa la función factorial. Maxima considera **factorial** (*x*) lo mismo que *x*!. Véase !.

**fib** (*n*) Función

Devuelve el *n*-ésimo número de Fibonacci. La llamada **fib**(0) es igual a 0, **fib**(1) devuelve 1 y **fib**(-*n*) es igual a  $(-1)^{(n+1)} * \text{fib}(n)$ .

Después de llamar a **fib**, la variable **prevfib** toma el valor **fib**(*x* - 1), que es el número de Fibonacci que precede al último calculado.

```
(%i1) map (fib, [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]);
(%o1) [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

**fibtophi** (*expr*) Función

Expresa los números de Fibonacci en términos de la razón áurea **%phi**, que es  $(1 + \sqrt{5})/2$ , aproximadamente 1.61803399.

Por defecto, Maxima desconoce **%phi**. Tras ejecutar **tellrat** (**%phi**<sup>2</sup> - **%phi** - 1) y **algebraic: true**, **ratsimp** puede simplificar algunas expresiones que contengan **%phi**.

```
(%i1) fibtophi (fib (n));
 n n
 %phi - (1 - %phi)
(%o1) -----
 2 %phi - 1
(%i2) fib (n-1) + fib (n) - fib (n+1);
(%o2) - fib(n + 1) + fib(n) + fib(n - 1)
(%i3) ratsimp (fibtophi (%));
(%o3) 0
```

**inrt** (*x*, *n*) Función

Devuelve la raíz entera *n*-ésima del valor absoluto de *x*.

```
(%i1) 1: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]$
(%i2) map (lambda ([a], inrt (10^a, 3)), 1);
(%o2) [2, 4, 10, 21, 46, 100, 215, 464, 1000, 2154, 4641, 10000]
```

**jacobi** (*p*, *q*) Función

Devuelve el símbolo de Jacobi para *p* y *q*.

```
(%i1) 1: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]$
(%i2) map (lambda ([a], jacobi (a, 9)), 1);
(%o2) [1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0]
```

**lcm** (*expr\_1*, ..., *expr\_n*) Función

Devuelve el mínimo común múltiplo de sus argumentos. Los argumentos pueden ser tanto expresiones en general como enteros.

Es necesario cargar en memoria esta función haciendo **load** ("functs").

**partfrac** (*expr*, *var*) Función

Expande la expresión *expr* en fracciones parciales respecto de la variable principal *var*. La función **partfrac** hace una descomposición completa en fracciones parciales. El algoritmo que se utiliza se basa en el hecho de que los denominadores de la expansión en fracciones parciales (los factores del denominador original) son primos

relativos. Los numeradores se pueden escribir como combinaciones lineales de los denominadores.

```
(%i1) 1/(1+x)^2 - 2/(1+x) + 2/(2+x);
(%o1)

$$\frac{1}{x^2 + 2x + 2} - \frac{2}{x + 1} + \frac{2}{(x + 1)^2}$$

(%i2) ratsimp (%);
(%o2)

$$-\frac{x}{x^3 + 4x^2 + 5x + 2}$$

(%i3) partfrac (% , x);
(%o3)

$$\frac{1}{x^2 + 2x + 2} - \frac{2}{x + 1} + \frac{1}{(x + 1)^2}$$

```

### **primep** (*n*)

Función

Devuelve **true** si *n* es un número primo, **false** en caso contrario.

### **qunit** (*n*)

Función

Devuelve la unidad principal de **sqrt** (*n*), siendo *n* un entero; consiste en la resolución de la ecuación de Pell  $a^2 - n b^2 = 1$ .

```
(%i1) qunit (17);
(%o1)

$$\sqrt{17} + 4$$

(%i2) expand (% * (sqrt(17) - 4));
(%o2)

$$1$$

```

### **totient** (*n*)

Función

Devuelve el número de enteros menores o iguales a *n* que son primos relativos con *n*.

### **zerobern**

Variable opcional

Valor por defecto: **true**

Si **zerobern** vale **false**, **bern** excluye los números de Bernoulli iguales a cero. See **bern**.

### **zeta** (*n*)

Función

Devuelve la función zeta de Riemann para *x* entero negativo, 0, 1 o número par positivo. No se evalúa **zeta** (*n*) para cualesquiera otros argumentos, incluyendo racionales no enteros, números en coma flotante o argumentos complejos.

Véanse también **bfzeta** y **zeta%pi**.

```
(%i1) map (zeta, [-4, -3, -2, -1, 0, 1, 2, 3, 4, 5]);
(%o1) [0, ---, 0, - ---, - ---, inf, ---, zeta(3), ---, zeta(5)]
 120 12 2 6 90
```

**zeta%pi**

Variable opcional

Valor por defecto: `true`

Si `zeta%pi` vale `true`, `zeta` devuelve una expresión proporcional a  $\pi^n$  si `n` es un número par positivo. En caso contrario, `zeta` no se evalúa y devuelve la forma nominal `zeta (n)`.

```
(%i1) zeta%pi: true$
```

```
(%i2) zeta (4);
```

```
(%o2)
```

$$\frac{\pi^4}{90}$$

```
(%i3) zeta%pi: false$
```

```
(%i4) zeta (4);
```

```
(%o4)
```

```
zeta(4)
```



## 33 Simetrías

### 33.1 Definiciones para Simetrías

#### **comp2pui** ( $n, l$ )

Function

réalise le passage des fonctions symétriques complètes, données dans la liste  $l$ , aux fonctions symétriques élémentaires de 0 à  $n$ . Si la liste  $l$  contient moins de  $n+1$  éléments les valeurs formelles viennent la compléter. Le premier élément de la liste  $l$  donne le cardinal de l'alphabet si il existe, sinon on le met égal à  $n$ .

```
(%i1) comp2pui (3, [4, g]);
(%o1) [4, g, 2 h2 - g^2, 3 h3 - g h2 + g (g^2 - 2 h2)]
```

#### **cont2part** ( $pc, lvar$ )

Function

rend le polynôme partitionné associé à la forme contractée  $pc$  dont les variables sont dans  $lvar$ .

```
(%i1) pc: 2*a^3*b*x^4*y + x^5;
(%o1) 3 4 5
 2 a b x y + x
(%i2) cont2part (pc, [x, y]);
(%o2) [[1, 5, 0], [2 a b, 4, 1]]
```

Autres fonctions de changements de représentations :

`contract`, `explose`, `part2cont`, `partpol`, `tcontract`, `tpartpol`.

#### **contract** ( $psym, lvar$ )

Function

rend une forme contractée (i.e. un monoôme par orbite sous l'action du groupe symétrique) du polynôme  $psym$  en les variables contenues dans la liste  $lvar$ . La fonction `explose` réalise l'opération inverse. La fonction `tcontract` teste en plus la symétrie du polynôme.

```
(%i1) psym: explose (2*a^3*b*x^4*y, [x, y, z]);
(%o1) 2 a^3 b y z^4 + 2 a^3 b x z^4 + 2 a^3 b y z^4 + 2 a^3 b x z^4
 + 2 a^3 b x y z^4 + 2 a^3 b x y z^4
(%i2) contract (psym, [x, y, z]);
(%o2) 2 a^3 b x^4 y
```

Autres fonctions de changements de représentations :

`cont2part`, `explose`, `part2cont`, `partpol`, `tcontract`, `tpartpol`.

#### **direct** ( $[p_1, \dots, p_n], y, f, [lvar_1, \dots, lvar_n]$ )

Function

calcul l'image directe (voir M. GIUSTI, D. LAZARD et A. VALIBOUZE, ISSAC 1988, Rome) associée à la fonction  $f$ , en les listes de variables  $lvar_1, \dots, lvar_n$ , et



aux polynômes  $p_1, \dots, p_n$  d'une variable  $y$ . l'arité de la fonction  $f$  est importante pour le calcul. Ainsi, si l'expression de  $f$  ne depend pas d'une variable, non seulement il est inutile de donner cette variable mais cela diminue considérablement les calculs si on ne le fait pas.

```
(%i1) direct ([z^2 - e1*z + e2, z^2 - f1*z + f2],
 z, b*v + a*u, [[u, v], [a, b]]);
 2
(%o1) y - e1 f1 y
 2 2 2 2
 - 4 e2 f2 - (e1 - 2 e2) (f1 - 2 f2) + e1 f1
 + -----
 2

(%i2) ratsimp (%);
 2 2 2
(%o2) y - e1 f1 y + (e1 - 4 e2) f2 + e2 f1
(%i3) ratsimp (direct ([z^3-e1*z^2+e2*z-e3,z^2 - f1*z + f2],
 z, b*v + a*u, [[u, v], [a, b]]));
 6 5 2 2 4
(%o3) y - 2 e1 f1 y + ((2 e1 - 6 e2) f2 + (2 e2 + e1) f1) y
 3 3 3 3
 + ((9 e3 + 5 e1 e2 - 2 e1) f1 f2 + (- 2 e3 - 2 e1 e2) f1) y
 2 2 4 2
 + ((9 e2 - 6 e1 e2 + e1) f2
 2 2 2 2 4
 + (- 9 e1 e3 - 6 e2 + 3 e1 e2) f1 f2 + (2 e1 e3 + e2) f1)
 2 2 2 3 2
 y + (((9 e1 - 27 e2) e3 + 3 e1 e2 - e1 e2) f1 f2
 2 2 3 5
 + ((15 e2 - 2 e1) e3 - e1 e2) f1 f2 - 2 e2 e3 f1) y
 2 3 3 2 2 3
 + (- 27 e3 + (18 e1 e2 - 4 e1) e3 - 4 e2 + e1 e2) f2
 2 3 3 2 2
 + (27 e3 + (e1 - 9 e1 e2) e3 + e2) f1 f2
 2 4 2 6
 + (e1 e2 e3 - 9 e3) f1 f2 + e3 f1
```

Recherche du polynôme dont les racines sont les somme  $a+u$  ou  $a$  est racine de  $z^2 - e1*z + e2$  et  $u$  est racine de  $z^2 - f1*z + f2$

```
(%i1) ratsimp (direct ([z^2 - e1*z + e2, z^2 - f1*z + f2],
 z, a + u, [[u], [a]]));
```

```

 4 3 2
(%o1) y + (- 2 f1 - 2 e1) y + (2 f2 + f1 + 3 e1 f1 + 2 e2
 2 2 2 2
+ e1) y + ((- 2 f1 - 2 e1) f2 - e1 f1 + (- 2 e2 - e1) f1
 2 2 2
- 2 e1 e2) y + f2 + (e1 f1 - 2 e2 + e1) f2 + e2 f1 + e1 e2 f1
 2
+ e2

```

`direct` peut prendre deux drapeaux possibles : `elementaires` et `puissances` (valeur par de'faut) qui permettent de de'composer les polyno^mes syme'triques apparaissant dans ce calcul par les fonctions syme'triques e'le'mentaires ou les fonctions puissances respectivement.

Fonctions de `sym` utilis'ees dans cette fonction :

`multi_orbit` (donc `orbit`), `pui_direct`, `multi_elem` (donc `elem`), `multi_pui` (donc `pui`), `pui2ele`, `ele2pui` (si le drapeau `direct` est a' `puissances`).

**ele2comp** ( $m, l$ ) Function

passé des fonctions syme'triques e'le'mentaires aux fonctions comple'tes. Similaire a' `comp2ele` et `comp2pui`.

Autres fonctions de changements de bases :

`comp2ele`, `comp2pui`, `ele2pui`, `elem`, `mon2schur`, `multi_elem`, `multi_pui`, `pui`, `pui2comp`, `pui2ele`, `puireduc`, `schur2comp`.

**ele2polynome** ( $l, z$ ) Function

donne le polyno^me en  $z$  dont les fonctions syme'triques e'le'mentaires des racines sont dans la liste  $l$ .  $l = [n, e_1, \dots, e_n]$  ou'  $n$  est le degre' du polyno^me et  $e_i$  la  $i$ -ie'me fonction syme'trique e'le'mentaire.

```

(%i1) ele2polynome ([2, e1, e2], z);
 2
(%o1) z - e1 z + e2
(%i2) polynome2ele (x^7 - 14*x^5 + 56*x^3 - 56*x + 22, x);
(%o2) [7, 0, - 14, 0, 56, 0, - 56, - 22]
(%i3) ele2polynome ([7, 0, -14, 0, 56, 0, -56, -22], x);
 7 5 3
(%o3) x - 14 x + 56 x - 56 x + 22

```

La re'ciproque: `polynome2ele` ( $P, z$ )

Autres fonctions a' voir :

`polynome2ele`, `pui2polynome`.

**ele2pui** ( $m, l$ ) Function

passé des fonctions syme'triques e'le'mentaires aux fonctions comple'tes. Similaire a' `comp2ele` et `comp2pui`.

Autres fonctions de changements de bases :

`comp2ele`, `comp2pui`, `ele2comp`, `elem`, `mon2schur`, `multi_elem`, `multi_pui`, `pui`,  
`pui2comp`, `pui2ele`, `puireduc`, `schur2comp`.

**elem** (*ele*, *sym*, *lvar*)

Function

decompose le polynôme symétrique *sym*, en les variables contenues de la liste *lvar*, par les fonctions symétriques élémentaires contenues dans la liste *ele*. Si le premier élément de *ele* est donné ce sera le cardinal de l'alphabet sinon on prendra le degré du polynôme *sym*. Si il manque des valeurs à la liste *ele* des valeurs formelles du type "ei" sont rajoutées. Le polynôme *sym* peut être donné sous 3 formes différentes : contractée (`elem` doit alors valoir 1 sa valeur par défaut), partitionnée (`elem` doit alors valoir 3) ou étendue (i.e. le polynôme en entier) (`elem` doit alors valoir 2). L'utilisation de la fonction `pui` se réalise sur le même modèle.

Sur un alphabet de cardinal 3 avec *e1*, la première fonction symétrique élémentaire, valant 7, le polynôme symétrique en 3 variables dont la forme contractée (ne dépendant ici que de deux de ses variables) est  $x^4 - 2x^3y$  se décompose ainsi en les fonctions symétriques élémentaires :

```
(%i1) elem ([3, 7], x^4 - 2*x*y, [x, y]);
(%o1) 7 (e3 - 7 e2 + 7 (49 - e2)) + 21 e3
 + (- 2 (49 - e2) - 2) e2
(%i2) ratsimp (%);
(%o2) 28 e3 + 2 e2^2 - 198 e2 + 2401
```

Autres fonctions de changements de bases :

`comp2ele`, `comp2pui`, `ele2comp`, `ele2pui`, `mon2schur`, `multi_elem`, `multi_pui`, `pui`,  
`pui2comp`, `pui2ele`, `puireduc`, `schur2comp`.

**explose** (*pc*, *lvar*)

Function

rend le polynôme symétrique associé à la forme contractée *pc*. La liste *lvar* contient les variables.

```
(%i1) explode (a*x + 1, [x, y, z]);
(%o1) a z + a y + a x + 1
```

Autres fonctions de changements de représentations :

`contract`, `cont2part`, `part2cont`, `partpol`, `tcontract`, `tpartpol`.

**kostka** (*part\_1*, *part\_2*)

Function

écrite par P. ESPERET, calcule le nombre de Kostka associé aux partitions *part\_1* et *part\_2*.

```
(%i1) kostka ([3, 3, 3], [2, 2, 2, 1, 1, 1]);
(%o1) 6
```

**lgtreillis** (*n*, *m*)

Function

rend la liste des partitions de poids *n* et de longueur *m*.

```
(%i1) lgtreillis (4, 2);
(%o1) [[3, 1], [2, 2]]
```

Voir également : `ltreillis`, `treillis` et `treinat`.

### **ltreillis** (*n*, *m*)

Function

rend la liste des partitions de poids *n* et de longueur inférieure ou égale à *m*.

```
(%i1) ltreillis (4, 2);
(%o1) [[4, 0], [3, 1], [2, 2]]
```

Voir également : `lgtreillis`, `treillis` et `treinat`.

### **mon2schur** (*l*)

Function

la liste *l* représente la fonction de Schur  $S_l$ : On a  $l = [i_1, i_2, \dots, i_q]$  avec  $i_1 \leq i_2 \leq \dots \leq i_q$ . La fonction de Schur est  $S_{[i_1, i_2, \dots, i_q]}$  est le mineur de la matrice infinie  $(h_{i-j})$   $i \geq 1, j \geq 1$  composé des *q* premières lignes et des colonnes  $i_1 + 1, i_2 + 2, \dots, i_q + q$ .

On écrit cette fonction de Schur en fonction des formes monomiales en utilisant les fonctions `treinat` et `kostka`. La forme rendue est un polynôme symétrique dans une de ses représentations contractées avec les variables  $x_1, x_2, \dots$

```
(%i1) mon2schur ([1, 1, 1]);
(%o1) x1 x2 x3
(%i2) mon2schur ([3]);
(%o2) x1 x2 x3 + x1^2 x2 + x1^3
(%i3) mon2schur ([1, 2]);
(%o3) 2 x1 x2 x3 + x1^2 x2
```

ce qui veut dire que pour 3 variables cela donne :

$$2 x_1 x_2 x_3 + x_1^2 x_2 + x_2^2 x_1 + x_1^2 x_3 + x_3^2 x_1 + x_2^2 x_3 + x_3^2 x_2$$

Autres fonctions de changements de bases :

`comp2ele`, `comp2pui`, `ele2comp`, `ele2pui`, `elem`, `multi_elem`, `multi_pui`, `pui`, `pui2comp`, `pui2ele`, `puireduc`, `schur2comp`.

### **multi\_elem** (*l\_elem*, *multi\_pc*, *Lvar*)

Function

décompose un polynôme multi-symétrique sous la forme multi-contractée *multi\_pc* en les groupes de variables contenue dans la liste de listes *Lvar* sur les groupes de fonctions symétriques élémentaires contenues dans *l\_elem*.

```
(%i1) multi_elem ([[2, e1, e2], [2, f1, f2]], a*x + a^2 + x^3, [[x, y], [a, b]]
(%o1) - 2 f2 + f1 (f1 + e1) - 3 e1 e2 + e1^3
(%i2) ratsimp (%);
(%o2) - 2 f2 + f1^2 + e1 f1 - 3 e1 e2 + e1^3
```

Autres fonctions de changements de bases :

`comp2ele`, `comp2pui`, `ele2comp`, `ele2pui`, `elem`, `mon2schur`, `multi_pui`, `pui`, `pui2comp`, `pui2ele`, `puireduc`, `schur2comp`.

**multi\_orbit** (*P*, [*lvar\_1*, *lvar\_2*, ..., *lvar\_p*])

Function

*P* est un polynôme en l'ensemble des variables contenues dans les listes *lvar\_1*, *lvar\_2*, ..., *lvar\_p*. Cette fonction ramène l'orbite du polynôme *P* sous l'action du produit des groupes symétriques des ensembles de variables représentés par ces *p* listes.

```
(%i1) multi_orbit (a*x + b*y, [[x, y], [a, b]]);
(%o1) [b y + a x, a y + b x]
(%i2) multi_orbit (x + y + 2*a, [[x, y], [a, b, c]]);
(%o2) [y + x + 2 c, y + x + 2 b, y + x + 2 a]
```

Voir également : **orbit** pour l'action d'un seul groupe symétrique.

**multi\_pui**

Function

est à la fonction **pui** ce que la fonction **multi\_elem** est à la fonction **elem**.

```
(%i1) multi_pui ([[2, p1, p2], [2, t1, t2]], a*x + a^2 + x^3, [[x, y], [a, b]])
3
3 p1 p2 p1
(%o1) t2 + p1 t1 + ----- - ----
2 2 2
```

**multinomial** (*r*, *part*)

Function

où *r* est le poids de la partition *part*. Cette fonction ramène le coefficient multinomial associé : si les parts de la partitions *part* sont *i\_1*, *i\_2*, ..., *i\_k*, le résultat de **multinomial** est  $r!/(i_1! i_2! \dots i_k!)$ .

**multsym** (*ppart\_1*, *ppart\_2*, *n*)

Function

réalise le produit de deux polynômes symétriques de *n* variables en ne travaillant que modulo l'action du groupe symétrique d'ordre *n*. Les polynômes sont dans leur représentation partitionnée.

Soient les 2 polynômes symétriques en *x*, *y*:  $3*(x + y) + 2*x*y$  et  $5*(x^2 + y^2)$  dont les formes partitionnées sont respectivement  $[[3, 1], [2, 1, 1]]$  et  $[[5, 2]]$ , alors leur produit sera donné par :

```
(%i1) multsym ([[3, 1], [2, 1, 1]], [[5, 2]], 2);
(%o1) [[10, 3, 1], [15, 3, 0], [15, 2, 1]]
```

soit  $10*(x^3*y + y^3*x) + 15*(x^2*y + y^2*x) + 15*(x^3 + y^3)$ .

Fonctions de changements de représentations d'un polynôme symétrique :

**contract**, **cont2part**, **explose**, **part2cont**, **partpol**, **tcontract**, **tpartpol**.

**orbit** (*P*, *lvar*)

Function

calcul l'orbite du polynôme *P* en les variables de la liste *lvar* sous l'action du groupe symétrique de l'ensemble des variables contenues dans la liste *lvar*.

```
(%i1) orbit (a*x + b*y, [x, y]);
(%o1) [a y + b x, b y + a x]
(%i2) orbit (2*x + x^2, [x, y]);
2 2
(%o2) [y + 2 y, x + 2 x]
```

Voir également : **multi\_orbit** pour l'action d'un produit de groupes symétriques sur un polynôme.

**part2cont** (*ppart*, *lvar*) Function  
 passe de la forme partitionnée à la forme contractée d'un polynôme symétrique.  
 La forme contractée est rendue avec les variables contenues dans *lvar*.

```
(%i1) part2cont ([[2*a^3*b, 4, 1]], [x, y]);
```

```
(%o1) 3 4
 2 a b x y
```

Autres fonctions de changements de représentations :

`contract`, `cont2part`, `explode`, `partpol`, `tcontract`, `tpartpol`.

**partpol** (*psym*, *lvar*) Function  
*psym* est un polynôme symétrique en les variables de *lvar*. Cette fonction ramène sa représentation partitionnée.

```
(%i1) partpol (-a*(x + y) + 3*x*y, [x, y]);
```

```
(%o1) [[3, 1, 1], [- a, 1, 0]]
```

Autres fonctions de changements de représentations :

`contract`, `cont2part`, `explode`, `part2cont`, `tcontract`, `tpartpol`.

**permut** (*l*) Function  
 ramène la liste des permutations de la liste *l*.

**polynome2ele** (*P*, *x*) Function  
 donne la liste  $l = [n, e_1, \dots, e_n]$  où  $n$  est le degré du polynôme  $P$  en la variable  $x$  et  $e_i$  la  $i$ -ième fonction symétrique élémentaire des racines de  $P$ .

```
(%i1) polynome2ele (x^7 - 14*x^5 + 56*x^3 - 56*x + 22, x);
```

```
(%o1) [7, 0, - 14, 0, 56, 0, - 56, - 22]
```

```
(%i2) ele2polynome ([7, 0, -14, 0, 56, 0, -56, -22], x);
```

```
(%o2) 7 5 3
 x - 14 x + 56 x - 56 x + 22
```

La réciproque : `ele2polynome` (*l*, *x*)

**prodrac** (*l*, *k*) Function  
*l* est une liste contenant les fonctions symétriques élémentaires sur un ensemble  $A$ .  
`prodrac` rend le polynôme dont les racines sont les produits  $k$  à  $k$  des éléments de  $A$ .

**pui** (*l*, *sym*, *lvar*) Function  
 décompose le polynôme symétrique *sym*, en les variables contenues de la liste *lvar*, par les fonctions puissances contenues dans la liste *l*. Si le premier élément de *l* est donné ce sera le cardinal de l'alphabet sinon on prendra le degré du polynôme *sym*. Si il manque des valeurs à la liste *l*, des valeurs formelles du type "pi" sont rajoutées. Le polynôme *sym* peut être donné sous 3 formes différentes : contractée (`pui` doit alors valoir 1 sa valeur par défaut), partitionnée (`pui` doit alors valoir 3) ou étendue (i.e. le polynôme en entier) (`pui` doit alors valoir 2). La fonction `elem` s'utilise de la même manière.

```
(%i1) pui;
(%o1)
1
(%i2) pui ([3, a, b], u*x*y*z, [x, y, z]);
2
a (a - b) u (a b - p3) u
(%o2) -----
6 3
(%i3) ratsimp (%);
3
(2 p3 - 3 a b + a) u
(%o3) -----
6
```

Autres fonctions de changements de bases :

comp2ele, comp2pui, ele2comp, ele2pui, elem, mon2schur, multi\_elem, multi\_pui, pui2comp, pui2ele, puireduc, schur2comp.

### pui2comp (*n*, *lpui*)

Function

rend la liste des *n* premières fonctions complètes (avec en tête le cardinal) en fonction des fonctions puissance données dans la liste *lpui*. Si la liste *lpui* est vide le cardinal est N sinon c'est son premier élément similaire à *comp2ele* et *comp2pui*.

```
(%i1) pui2comp (2, []);
2
p2 + p1
(%o1) [2, p1, -----]
2
(%i2) pui2comp (3, [2, a1]);
2
a1 (p2 + a1)
2 p3 + ----- + a1 p2
2
(%o2) [2, a1, -----, -----]
2 3
(%i3) ratsimp (%);
2 3
p2 + a1 2 p3 + 3 a1 p2 + a1
(%o3) [2, a1, -----, -----]
2 6
```

Autres fonctions de changements de bases :

comp2ele, comp2pui, ele2comp, ele2pui, elem, mon2schur, multi\_elem, multi\_pui, pui, pui2ele, puireduc, schur2comp.

### pui2ele (*n*, *lpui*)

Function

réalise le passage des fonctions puissances aux fonctions symétriques élémentaires. Si le drapeau *pui2ele* est *girard*, on récupère la liste des fonctions symétriques élémentaires de 1 à *n*, et s'il est égal à *close*, la *n*-ième fonction symétrique élémentaire.

Autres fonctions de changements de bases :

comp2ele, comp2pui, ele2comp, ele2pui, elem, mon2schur, multi\_elem, multi\_pui, pui, pui2comp, puireduc, schur2comp.

**pui2polynome** (*x*, *lpui*)

Function

calcul le polynôme en *x* dont les fonctions puissances des racines sont données dans la liste *lpui*.

```
(%i1) pui;
(%o1) 1
(%i2) kill(labels);
(%o0) done
(%i1) polynome2ele (x^3 - 4*x^2 + 5*x - 1, x);
(%o1) [3, 4, 5, 1]
(%i2) ele2pui (3, %);
(%o2) [3, 4, 6, 7]
(%i3) pui2polynome (x, %);
(%o3) x^3 - 4 x^2 + 5 x - 1
```

Autres fonctions à voir : polynome2ele, ele2polynome.

**pui\_direct** (*orbite*, [*lvar\_1*, ..., *lvar\_n*], [*d\_1*, *d\_2*, ..., *d\_n*])

Function

Soit *f* un polynôme en *n* blocs de variables *lvar\_1*, ..., *lvar\_n*. Soit *c\_i* le nombre de variables dans *lvar\_i*. Et *SC* le produit des *n* groupes symétriques de degré *c\_1*, ..., *c\_n*. Ce groupe agit naturellement sur *f*. La liste *orbite* est l'orbite, notée *SC(f)*, de la fonction *f* sous l'action de *SC*. (Cette liste peut être obtenue avec la fonction : multi\_orbit). Les *d\_i* sont des entiers tels que *c\_1* ≤ *d\_1*, *c\_2* ≤ *d\_2*, ..., *c\_n* ≤ *d\_n*. Soit *SD* le produit des groupes symétriques *S\_{d\_1}* × *S\_{d\_2}* × ... × *S\_{d\_n}*.

La fonction **pui\_direct** ramène les *n* premières fonctions puissances de *SD(f)* déduites des fonctions puissances de *SC(f)* où *n* est le cardinal de *SD(f)*.

Le résultat est rendu sous forme multi-contrainte par rapport à *SD*. i.e. on ne conserve qu'un élément par orbite sous l'action de *SD*).

```
(%i1) l: [[x, y], [a, b]];
(%o1) [[x, y], [a, b]]
(%i2) pui_direct (multi_orbit (a*x + b*y, 1), 1, [2, 2]);
(%o2) [a x, 4 a b x y + a^2 x^2]
(%i3) pui_direct (multi_orbit (a*x + b*y, 1), 1, [3, 2]);
(%o3) [2 a x, 4 a b x y + 2 a^2 x^2, 3 a^2 b x y + 2 a^3 x^3,
12 a^2 b x y + 4 a^3 b x y + 2 a^4 x^4,
10 a^3 b x y + 5 a^4 b x y + 2 a^5 x^5,
40 a^3 b x y + 15 a^4 b x y + 6 a^5 b x y + 2 a^6 x^6]
```



```
(%i4) pui_direct ([y + x + 2*c, y + x + 2*b, y + x + 2*a], [[x, y], [a, b, c]])
```

```
(%o4) [3 x + 2 a, 6 x y + 3 x2 + 4 a x + 4 a2,
```

```
9 x2 y + 12 a x y + 3 x3 + 6 a x2 + 12 a2 x + 8 a3]
```

### **puireduc** (*n*, *lpui*)

Function

*lpui* est une liste dont le premier e'le'ment est un entier *m*. **puireduc** donne les *n* premie'res fonctions puissances en fonction des *m* premie'res.

```
(%i1) puireduc (3, [2]);
```

```
(%o1) [2, p1, p2, p1 p2 - $\frac{p1^2 - p2^2}{2}$]
```

```
(%i2) ratsimp (%);
```

```
(%o2) [2, p1, p2, $\frac{3 p1 p2 - p1^3}{2}$]
```

### **resolvante** (*P*, *x*, *f*, [*x*<sub>1</sub>, ..., *x*<sub>*d*</sub>])

Function

calcule la re'solvante du polyno^me *P* de la variable *x* et de degre' *n* >= *d* par la fonction *f* exprime'e en les variables *x*<sub>1</sub>, ..., *x*<sub>*d*</sub>. Il est important pour l'efficacite' des calculs de ne pas mettre dans la liste [*x*<sub>1</sub>, ..., *x*<sub>*d*</sub>] les variables n'intervenant pas dans la fonction de transformation *f*.

Afin de rendre plus efficaces les calculs on peut mettre des drapeaux a' la variable **resolvante** afin que des algorithmes ade'quates soient utilise's :

Si la fonction *f* est unitaire :

- un polyno^me d'une variable,
- line'aire ,
- alterne'e,
- une somme de variables,
- syme'trique en les variables qui apparaissent dans son expression,
- un produit de variables,
- la fonction de la re'solvante de Cayley (utilisable qu'en degre' 5)

```
(x1*x2 + x2*x3 + x3*x4 + x4*x5 + x5*x1 -
(x1*x3 + x3*x5 + x5*x2 + x2*x4 + x4*x1))^2
```

generale,

le drapeau de **resolvante** pourra e^tre respectivement :

- unitaire,
- lineaire,
- alternee,
- somme,

- produit,
- cayley,
- generale.

```
(%i1) resolvante: unitaire$
(%i2) resolvante (x^7 - 14*x^5 + 56*x^3 - 56*x + 22, x, x^3 - 1, [x]);

" resolvante unitaire " [7, 0, 28, 0, 168, 0, 1120, - 154, 7840, - 2772, 56448,
413952, - 352352, 3076668, - 3363360, 23114112, - 30494464,
175230832, - 267412992, 1338886528, - 2292126760]
 3 6 3 9 6 3
[x - 1, x - 2 x + 1, x - 3 x + 3 x - 1,
 12 9 6 3 15 12 9 6 3
x - 4 x + 6 x - 4 x + 1, x - 5 x + 10 x - 10 x + 5 x
 18 15 12 9 6 3
- 1, x - 6 x + 15 x - 20 x + 15 x - 6 x + 1,
 21 18 15 12 9 6 3
x - 7 x + 21 x - 35 x + 35 x - 21 x + 7 x - 1]
[- 7, 1127, - 6139, 431767, - 5472047, 201692519, - 3603982011]
 7 6 5 4 3 2
(%o2) y + 7 y - 539 y - 1841 y + 51443 y + 315133 y
 + 376999 y + 125253

(%i3) resolvante: lineaire$
(%i4) resolvante (x^4 - 1, x, x1 + 2*x2 + 3*x3, [x1, x2, x3]);

" resolvante lineaire "
 24 20 16 12 8
(%o4) y + 80 y + 7520 y + 1107200 y + 49475840 y
 + 344489984 y + 655360000

(%i5) resolvante: general$
(%i6) resolvante (x^4 - 1, x, x1 + 2*x2 + 3*x3, [x1, x2, x3]);

" resolvante generale "
 24 20 16 12 8
(%o6) y + 80 y + 7520 y + 1107200 y + 49475840 y
 + 344489984 y + 655360000

(%i7) resolvante (x^4 - 1, x, x1 + 2*x2 + 3*x3, [x1, x2, x3, x4]);

" resolvante generale "
```

```

 24 20 16 12 8
(%o7) y + 80 y + 7520 y + 1107200 y + 49475840 y

 4
 + 344489984 y + 655360000
(%i8) direct ([x^4 - 1], x, x1 + 2*x2 + 3*x3, [[x1, x2, x3]]);
 24 20 16 12 8
(%o8) y + 80 y + 7520 y + 1107200 y + 49475840 y

 4
 + 344489984 y + 655360000
(%i9) resolvante :lineaire$
(%i10) resolvante (x^4 - 1, x, x1 + x2 + x3, [x1, x2, x3]);

" resolvante lineaire "

 4
(%o10) y - 1
(%i11) resolvante: symetrique$
(%i12) resolvante (x^4 - 1, x, x1 + x2 + x3, [x1, x2, x3]);

" resolvante symetrique "

 4
(%o12) y - 1
(%i13) resolvante (x^4 + x + 1, x, x1 - x2, [x1, x2]);

" resolvante symetrique "

 6 2
(%o13) y - 4 y - 1
(%i14) resolvante: alternee$
(%i15) resolvante (x^4 + x + 1, x, x1 - x2, [x1, x2]);

" resolvante alternee "

 12 8 6 4 2
(%o15) y + 8 y + 26 y - 112 y + 216 y + 229
(%i16) resolvante: produit$
(%i17) resolvante (x^7 - 7*x + 3, x, x1*x2*x3, [x1, x2, x3]);

" resolvante produit "

 35 33 29 28 27 26
(%o17) y - 7 y - 1029 y + 135 y + 7203 y - 756 y

 24 23 22 21 20
+ 1323 y + 352947 y - 46305 y - 2463339 y + 324135 y

 19 18 17 15
- 30618 y - 453789 y - 40246444 y + 282225202 y

 14 12 11 10
- 44274492 y + 155098503 y + 12252303 y + 2893401 y

```

```

 9 8 7 6
- 171532242 y + 6751269 y + 2657205 y - 94517766 y

 5 3
- 3720087 y + 26040609 y + 14348907
(%i18) resolvante: symetrique$
(%i19) resolvante (x^7 - 7*x + 3, x, x1*x2*x3, [x1, x2, x3]);

" resolvante symetrique "
 35 33 29 28 27 26
(%o19) y - 7 y - 1029 y + 135 y + 7203 y - 756 y

 24 23 22 21 20
+ 1323 y + 352947 y - 46305 y - 2463339 y + 324135 y

 19 18 17 15
- 30618 y - 453789 y - 40246444 y + 282225202 y

 14 12 11 10
- 44274492 y + 155098503 y + 12252303 y + 2893401 y

 9 8 7 6
- 171532242 y + 6751269 y + 2657205 y - 94517766 y

 5 3
- 3720087 y + 26040609 y + 14348907
(%i20) resolvante: cayley$
(%i21) resolvante (x^5 - 4*x^2 + x + 1, x, a, []);

" resolvante de Cayley "
 6 5 4 3 2
(%o21) x - 40 x + 4080 x - 92928 x + 3772160 x + 37880832 x

+ 93392896

```

Pour la re'solvante de Cayley, les 2 derniers arguments sont neutres et le polyno~me donne' en entre'e doit ne'cessairement e~tre de degre' 5.

Voir e'galement :

resolvante\_bipartite, resolvante\_produit\_sym, resolvante\_unitaire,  
 resolvante\_alternee1, resolvante\_klein, resolvante\_klein3, resolvante\_  
 vierer, resolvante\_diedrale.

**resolvante\_alternee1** ( $P, x$ )

Function

calcule la transformation de  $P(x)$  de degre  $n$  par la fonction  $\prod_{1 \leq i < j \leq n-1} (x_i - x_j)$ .

Voir e'galement :

resolvante\_produit\_sym, resolvante\_unitaire, resolvante , resolvante\_klein, resolvante\_klein3, resolvante\_vierer, resolvante\_diedrale, resolvante\_bipartite.

### resolvante\_bipartite ( $P, x$ )

Function

calcule la transformation de  $P(x)$  de degre  $n$  ( $n$  pair) par la fonction  $x_1 x_2 \dots x_{n/2} + x_{n/2+1} \dots x_n$

Voir e'galement :

resolvante\_produit\_sym, resolvante\_unitaire, resolvante , resolvante\_klein, resolvante\_klein3, resolvante\_vierer, resolvante\_diedrale, resolvante\_alternee1.

```
(%i1) resolvante_bipartite (x^6 + 108, x);
 10 8 6 4
(%o1) y - 972 y + 314928 y - 34012224 y
```

Voir e'galement :

resolvante\_produit\_sym, resolvante\_unitaire, resolvante, resolvante\_klein, resolvante\_klein3, resolvante\_vierer, resolvante\_diedrale, resolvante\_alternee1.

### resolvante\_diedrale ( $P, x$ )

Function

calcule la transformation de  $P(x)$  par la fonction  $x_1 x_2 + x_3 x_4$ .

```
(%i1) resolvante_diedrale (x^5 - 3*x^4 + 1, x);
 15 12 11 10 9 8 7
(%o1) x - 21 x - 81 x - 21 x + 207 x + 1134 x + 2331 x
 6 5 4 3 2
 - 945 x - 4970 x - 18333 x - 29079 x - 20745 x - 25326 x
 - 697
```

Voir e'galement :

resolvante\_produit\_sym, resolvante\_unitaire, resolvante\_alternee1, resolvante\_klein, resolvante\_klein3, resolvante\_vierer, resolvante.

### resolvante\_klein ( $P, x$ )

Function

calcule la transformation de  $P(x)$  par la fonction  $x_1 x_2 x_4 + x_4$ .

Voir e'galement :

resolvante\_produit\_sym, resolvante\_unitaire, resolvante\_alternee1, resolvante, resolvante\_klein3, resolvante\_vierer, resolvante\_diedrale.

### resolvante\_klein3 ( $P, x$ )

Function

calcule la transformation de  $P(x)$  par la fonction  $x_1 x_2 x_4 + x_4$ .

Voir e'galement :

resolvante\_produit\_sym, resolvante\_unitaire, resolvante\_alternee1, resolvante\_klein, resolvante, resolvante\_vierer, resolvante\_diedrale.

**resolvante\_produit\_sym** ( $P, x$ )

Function

 calcule la liste toutes les r\'esolvantes produit du polynôme  $P(x)$ .

```
(%i1) resolvante_produit_sym (x^5 + 3*x^4 + 2*x - 1, x);
 5 4 10 8 7 6 5
(%o1) [y + 3 y + 2 y - 1, y - 2 y - 21 y - 31 y - 14 y

 4 3 2 10 8 7 6 5 4
 - y + 14 y + 3 y + 1, y + 3 y + 14 y - y - 14 y - 31 y

 3 2 5 4
 - 21 y - 2 y + 1, y - 2 y - 3 y - 1, y - 1]
(%i2) resolvante: produit$
(%i3) resolvante (x^5 + 3*x^4 + 2*x - 1, x, a*b*c, [a, b, c]);

" resolvante produit "
 10 8 7 6 5 4 3 2
(%o3) y + 3 y + 14 y - y - 14 y - 31 y - 21 y - 2 y + 1
```

Voir également :

resolvante, resolvante\_unitaire, resolvante\_alternee1, resolvante\_klein, resolvante\_klein3, resolvante\_vierer, resolvante\_diedrale.

**resolvante\_unitaire** ( $P, Q, x$ )

Function

 calcule la r\'esolvante du polynôme  $P(x)$  par le polynôme  $Q(x)$ .

Voir également :

resolvante\_produit\_sym, resolvante, resolvante\_alternee1, resolvante\_klein, resolvante\_klein3, resolvante\_vierer, resolvante\_diedrale.

**resolvante\_vierer** ( $P, x$ )

Function

 calcule la transformation de  $P(x)$  par la fonction  $x_1 x_2 - x_3 x_4$ .

Voir également :

resolvante\_produit\_sym, resolvante\_unitaire, resolvante\_alternee1, resolvante\_klein, resolvante\_klein3, resolvante, resolvante\_diedrale.

**schur2comp** ( $P, Lvar$ )

Function

$P$  est un polynôme en les variables contenues dans la liste  $Lvar$ . Chacune des variables de  $Lvar$  représente une fonction symétrique complète. On représente dans  $Lvar$  la  $i$ ème fonction symétrique complète comme la concaténation de la lettre  $h$  avec l'entier  $i$  :  $h_i$ . Cette fonction donne l'expression de  $P$  en fonction des fonctions de Schur.

```
(%i1) schur2comp (h1*h2 - h3, [h1, h2, h3]);
(%o1)
 s
 1, 2

(%i2) schur2comp (a*h3, [h3]);
(%o2)
 s a
 3
```

**somrac** (*l*, *k*) Function  
 la liste *l* contient les fonctions syme'triques e'le'mentaires d'un polyno^me *P* . On  
 calcul le polyno^mes dont les racines sont les sommes *K* a' *K* distinctes des racines  
 de *P*.

Voir e'galement **prodrac**.

**tcontract** (*pol*, *lvar*) Function  
 teste si le polyno^me *pol* est syme'trique en les variables contenues dans la liste *lvar*.  
 Si oui il rend une forme contracte'e comme la fonction **contract**.

Autres fonctions de changements de repre'sentations :

**contract**, **cont2part**, **explode**, **part2cont**, **partpol**, **tpartpol**.

**tpartpol** (*pol*, *lvar*) Function  
 teste si le polyno^me *pol* est syme'trique en les variables contenues dans la liste *lvar*.  
 Si oui il rend sa forme partionne'e comme la fonction **partpol**.

Autres fonctions de changements de repre'sentations :

**contract**, **cont2part**, **explode**, **part2cont**, **partpol**, **tcontract**.

**treillis** (*n*) Function  
 rame'ne toutes les partitions de poids *n*.

```
(%i1) treillis (4);
```

```
(%o1) [[4], [3, 1], [2, 2], [2, 1, 1], [1, 1, 1, 1]]
```

Voir e'galement : **lgtreillis**, **ltreillis** et **treinat**.

**treinat** (*part*) Function  
 rame'ne la liste des partitions infe'rieures a' la partition *part* pour l'ordre naturel.

```
(%i1) treinat ([5]);
```

```
(%o1) [[5]]
```

```
(%i2) treinat ([1, 1, 1, 1, 1]);
```

```
(%o2) [[5], [4, 1], [3, 2], [3, 1, 1], [2, 2, 1], [2, 1, 1, 1],
```

```
[1, 1, 1, 1, 1]]
```

```
(%i3) treinat ([3, 2]);
```

```
(%o3) [[5], [4, 1], [3, 2]]
```

Voir e'galement : **lgtreillis**, **ltreillis** et **treillis**.

## 34 Grupos

### 34.1 Definiciones para Grupos

**todd\_coxeter** (*relaciones*, *subgrupo*)

Función

**todd\_coxeter** (*relaciones*)

Función

Busca el orden de  $G/H$  donde  $G$  es el módulo del Grupo Libre de *relations*, y  $H$  es el subgrupo de  $G$  generado por *subgrupo*. *subgrupo* es un argumento opcional, cuyo valor por defecto es [].

En este proceso se obtiene una tabla de multiplicación para la acción correcta de  $G$  sobre  $G/H$ , donde los co-cojuntos son enumerados  $[H, Hg_2, Hg_3, \dots]$ . Esto puede ser observado internamente en el `$todd_coxeter_state`.

Las tablas de multiplicación para las variables están en la `table:todd_coxeter_state[2]`. Entonces la `table[i]` retorna la tabla para la  $i$ -ésima variable.  
`mulcoset(coset,i) := tabla[varnum][coset];`

Ejemplo:

```
(%i1) symet(n):=create_list(
 if (j - i) = 1 then (p(i,j))<3> else
 if (not i = j) then (p(i,j))<2> else
 p(i,i) , j, 1, n-1, i, 1, j);

(%o1) symet(n) := create_list(if j - i = 1 then p(i, j)
 else (if not i = j then p(i, j)
 else p(i, i)), j, 1, n - 1,
 i, 1, j)
(%i2) p(i,j) := concat(x,i).concat(x,j);
(%o2) p(i, j) := concat(x, i) . concat(x, j)
(%i3) symet(5);
 <2> <3> <2> <2> <3>
(%o3) [x1 , (x1 . x2) , x2 , (x1 . x3) , (x2 . x3) ,
 x3 , (x1 . x4) , (x2 . x4) , (x3 . x4) , x4]
(%i4) todd_coxeter(%o3);

Rows tried 426
(%o4) 120
(%i5) todd_coxeter(%o3,[x1]);

Rows tried 213
(%o5) 60
(%i6) todd_coxeter(%o3,[x1,x2]);

Rows tried 71
(%o6) 20
```



```
(%i7) table:todd_coxeter_state[2]$
(%i8) table[1];
(%o8) {Array: (SIGNED-BYTE 30) #(0 2 1 3 7 6 5 4 8 11 17 9 12 14 #
13 20 16 10 18 19 15 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0)}
```

Nótese que sólo los elementos 1 hasta el 20 de este arreglo %o8 son significativos.  
`table[1][4] = 7` indica `coset4.var1 = coset7`

## 35 Entorno de Ejecución

### 35.1 Introducción al Entorno de Ejecución

El fichero `maxima-init.mac` se carga automáticamente cada vez que se empieza a ejecutar Maxima. Se puede utilizar `maxima-init.mac` para personalizar el entorno de Maxima. Si existe, `maxima-init.mac` se almacena normalmente en el directorio indicado por `:lisp (default-userdir)`, aunque puede estar alojado en cualquier otro directorio que esté al alcance de la función `file_search`.

He aquí un ejemplo de fichero `maxima-init.mac`:

```
setup_autoload ("specfun.mac", ultraspherical, assoc_legendre_p);
showtime:all;
```

En este ejemplo, `setup_autoload` le dice a Maxima que cargue en memoria el fichero `specfun.mac` si cualquiera de las funciones `ultraspherical` o `assoc_legendre_p` es invocada pero todavía no está definida. De esta manera, no es necesario recordar cargar el fichero antes de llamar a las funciones.

La sentencia `showtime: all` le dice a Maxima que haga una asignación a la variable `showtime`. El fichero `maxima-init.mac` puede contener cualesquiera otras asignaciones o sentencias de Maxima.

### 35.2 Interrupciones

El usuario puede detener un cómputo que esté consumiendo recursos excesivos con el carácter `^C` (control-C). La acción que se sigue por defecto es la detención del cómputo y la impresión de otro prompt. En este caso, no será posible reiniciar la tarea interrumpida.

Si a la variable `*debugger-hook*` se le asigna `nil` haciendo

```
:lisp (setq *debugger-hook* nil)
```

entonces tras recibir `^C`, Maxima entra en el depurador de Lisp y el usuario podrá utilizar el depurador para inspeccionar el entorno Lisp. La tarea que haya sido interrumpida podrá reiniciarse escribiendo `continue` en el depurado de Lisp. La forma de volver a Maxima desde el depurador de Lisp, que no sea la de permitir la computación hasta la terminación de la tarea, dependerá de la versión de Lisp.

En sistemas Unix el carácter `^Z` (control-Z) hace que Maxima deje de ejecutarse devolviendo el control al terminal del sistema. El comando `fg` hace que la ejecución de Maxima se reanude en el punto que lo dejó.

### 35.3 Definiciones para el Entorno de Ejecución

#### feature

Declaración

Maxima trata con dos tipos diferentes de características, las del sistema y las correspondientes a expresiones matemáticas. Véase también `status` para información sobre características del sistema. Véanse asimismo `features` y `featurep` para información sobre características matemáticas.

`feature` no es el nombre de una función ni de una variable.

**featurep** (*a*, *f*) Función

Intenta determinar si el objeto *a* tiene la característica *f* en base a la información en la base de datos actual. De ser así, devuelve **true**, en caso contrario **false**.

Nótese que **featurep** devuelve **false** cuando ni *f* ni la negación *f* puedan determinarse.

La función **featurep** evalúa sus argumentos.

Véanse también **declare** y **features**.

```
(%i1) declare (j, even)$
(%i2) featurep (j, integer);
(%o2) true
```

**room** () Función

**room** (*true*) Función

**room** (*false*) Función

Presenta una descripción del estado de almacenamiento y gestión de la pila en Maxima.

La llamada **room** invoca a la función Lisp homónima.

- **room** () prints out a moderate description.
- **room** (*true*) prints out a verbose description.
- **room** (*false*) prints out a terse description.

**status** (*feature*) Función

**status** (*feature*, *putative\_feature*) Función

**status** (*status*) Función

Devuelve información sobre la presencia o ausencia de ciertas características dependientes del sistema.

- **status** (*feature*) devuelve una lista con características del sistema. Éstas incluyen la versión de Lisp, tipo de sistema operativo, etc. La lista puede variar de un Lisp a otro.
- **status** (*feature*, *putative\_feature*) devuelve **true** si *putative\_feature* está en la lista de elementos retornados por **status** (*feature*) y **false** en otro caso. La función **status** no evalúa el argumento *putative\_feature*. El operador de dobles comillas simples, `'`, permite la evaluación. Una característica cuyo nombre contenga un carácter especial debe ser suministrada como un argumento del tipo cadena alfanumérica. Por ejemplo, **status** (*feature*, "ansi-cl").
- La llamada **status** (*status*) devuelve una lista con dos elementos [*feature*, *status*]. Los elementos *feature* y *status* son los dos argumentos que acepta la función **status**; no está claro si esta lista tiene algún interés adicional.

La variable **features** contiene una lista de características que se aplican a expresiones matemáticas. Véanse **features** y **featurep** para más información.

**time** (%o1, %o2, %o3, ...) Función

Devuelve una lista de los tiempos, en segundos, que fueron necesarios para calcular los resultados de las salidas %o1, %o2, %o3, .... Los tiempos devueltos son estimaciones hechas por Maxima del tiempo interno de computación. La función **time** sólo puede

utilizarse para variables correspondientes a líneas de salida; para cualquier otro tipo de variables, `time` devuelve `unknown`.

Hágase `showtime: true` para que Maxima devuelva el tiempo de ejecución de cada línea de salida.

### **timedate ()**

Función

Devuelve una cadena alfanumérica con la hora y fecha actuales. La cadena tiene el formato `HH:MM:SS Day, mm/dd/yyyy (GMT-n)`, donde los campos son: las horas, minutos, segundos, día de de la semana, mes, día del mes, año y número de horas de diferencia con respecto a la hora GMT.

El valor que devuelve es una cadena Lisp.

Ejemplo

```
(%i1) d: timedate ();
(%o1) 08:05:09 Wed, 11/02/2005 (GMT-7)
(%i2) print ("timedate reports current time", d)$
timedate reports current time 08:05:09 Wed, 11/02/2005 (GMT-7)
```



## 36 Opciones Misceláneas

### 36.1 Introducción a Opciones Misceláneas::

En esta sección se comentan varias opciones que tienen un efecto global sobre el comportamiento de Maxima. También se comentan varias listas, como la de las funciones definidas por el usuario.

### 36.2 Share

El directorio "share" de Maxima contiene programas y ficheros de interés para los usuarios de Maxima, pero no forman parte del núcleo de Maxima. Estos programas se cargan en memoria con llamadas a las funciones `load` o `setup_autoload`.

La llamada `:lisp *maxima-sharedir*` muestra la localización del directorio "share" dentro del sistema de ficheros del usuario.

La llamada `printfile ("share.usg")` muestra una lista actualizada de paquetes en "share". Los usuarios pueden encontrar más información accediendo directamente a los contenidos del directorio "share".

### 36.3 Definiciones para Opciones Misceláneas

#### alias

Variable del sistema

Valor por defecto: []

La variable `alias` es la lista de átomos que tienen un "alias" definido por el usuario (establecido mediante las funciones `alias`, `ordergreat` o `orderless` o declarando el átomo como un `noun` (nombre) con `declare`.

#### alphabetic

Declaración

La llamada `declare (char, alphabetic)` añade `char` al alfabeto de Maxima, el cual contiene inicialmente las letras de la A a la Z, de la a a la z, % y \_ . La variable `char` se especifica como una cadena alfanumérica de longitud 1, como "~".

```
(%i1) declare ("~", alphabetic);
(%o1) done
(%i2) foo~bar;
(%o2) foo~bar
(%i3) atom (foo~bar);
(%o3) true
```

#### apropos (cadena)

Función

Busca nombres en Maxima en los cuales aparezca *cadena* en cualquier lugar dentro de ellos. Así, `apropos (exp)` devuelve una lista con todas las variables y funciones que tengan `exp` formando parte de sus nombres, como `expand`, `exp` y `exponentialize`. De esta forma, si el usuario tan solo recuerda parte del nombre de algo, puede utilizar este comando para encontrar el resto del nombre. De manera semejante, también se puede hacer `apropos (tr_)` para encontrar una lista de muchas de las variables relacionadas con el traductor, buena parte de las cuales comienzan con `tr_`.

- args** (*expr*) Función  
 Devuelve la lista de argumentos de **expr**, que puede ser cualquier tipo de expresión a excepción de un átomo. Tan solo se muestran los argumentos del operador principal; subexpresiones de **expr** aparecen como elementos o subexpresiones de elementos de la lista de argumentos.  
 El orden de los miembros de la lista puede depender de la variable global **inflag**.  
 La llamada **args** (*expr*) es equivalente a **substpart** ("[" , *expr*, 0). Véase también **substpart**.  
 Véase asimismo **op**.
- genindex** Variable optativa  
 Valor por defecto: **i**  
 La variable **genindex** es el prefijo alfabético utilizado para generar la siguiente variable de sumación en caso de necesidad.
- gensumnum** Variable optativa  
 Valor por defecto: **0**  
 La variable **gensumnum** es el sufijo numérico utilizado para generar la siguiente variable de sumación. Si vale **false** entonces el índice consistirá solamente de **genindex**, sin sufijo numérico.
- inf** Constante  
 Símbolo que identifica al infinito positivo dentro de un contexto de números reales.
- infinity** Constante  
 Símbolo que identifica al infinito complejo, una magnitud infinita con ángulo de fase arbitrario. Véanse también **inf** y **minf**.
- infolists** Variable del sistema  
 Valor por defecto: **[]**  
 La variable **infolists** es una lista con los nombres de todas las listas que guardan información sobre Maxima. Estas son:
- labels**      Todas las etiquetas **%i**, **%o** y **%t** con valores asignados.
  - values**      Todos los átomos que son variables de usuario, no opciones de Maxima creadas con **:** o **::**.
  - functions**      Todas las funciones de usuario creadas con **:=** o **define**.
  - arrays**      Arreglos declarados y no declarados, creados por **:**, **::** o **:=**.
  - macros**      Cualquier macro definida por el usuario.
  - myoptions**      Todas las opciones inicializadas por el usuario, independientemente de que posteriormente hayan sido devueltas a sus valores por defecto.

|                          |                                                                                                                                                                                                                                                                                            |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>rules</b>             | Reglas de patrones y simplificación definidas por el usuario, creadas con <code>tellsimp</code> , <code>tellsimpafter</code> , <code>defmatch</code> o <code>defrule</code> .                                                                                                              |
| <b>alias</b>             | Átomos que tienen un "alias" definido por el usuario, creado por las funciones <code>alias</code> , <code>ordergreat</code> o <code>orderless</code> o por haber declarado el átomo como <code>noun</code> (nombre) con <code>declare</code> .                                             |
| <b>dependencias</b>      | Átomos que tienen dependencias funcionales, creados por las funciones <code>depends</code> o <code>grader</code> .                                                                                                                                                                         |
| <b>grader</b>            | Funciones que tienen derivadas definidas por el usuario, creadas por la función <code>grader</code> .                                                                                                                                                                                      |
| <b>props</b>             | Todos los átomos que tengan cualquier propiedad que no sea de las mencionadas hasta ahora, como las establecidas por <code>atvalue</code> , <code>matchdeclare</code> , etc., así como propiedades especificadas en la función <code>declare</code> .                                      |
| <b>let_rule_packages</b> | Todos los paquetes de reglas <code>let</code> definidos por el usuario, junto con el paquete especial <code>default_let_rule_package</code> ; <code>default_let_rule_package</code> es el nombre del paquete de reglas utilizado cuando no se use ningún otro especificado por el usuario. |

**integerp** (*expr*)

Función

Devuelve `true` si *expr* es un número entero y `false` en cualquier otro caso.

La función `integerp` devuelve `false` si su argumento es un símbolo, incluso cuando éste ha sido declarado como entero.

Ejemplos:

```
(%i1) integerp (0);
(%o1) true
(%i2) integerp (1);
(%o2) true
(%i3) integerp (-17);
(%o3) true
(%i4) integerp (0.0);
(%o4) false
(%i5) integerp (1.0);
(%o5) false
(%i6) integerp (%pi);
(%o6) false
(%i7) integerp (n);
(%o7) false
(%i8) declare (n, integer);
(%o8) done
(%i9) integerp (n);
(%o9) false
```

**m1pbranch**

Variable optativa

Valor por defecto: `false`



La variable `m1pbranch` es la rama principal de  $-1$  elevado a una potencia. Cantidades como  $(-1)^{(1/3)}$  (esto es, un exponente racional impar) y  $(-1)^{(1/4)}$  (esto es, un exponente racional par) son tratados como sigue:

dominio real

```
(-1)^(1/3): -1
(-1)^(1/4): (-1)^(1/4)
```

dominio complejo

```
m1pbranch:false m1pbranch:true
(-1)^(1/3) 1/2+%i*sqrt(3)/2
(-1)^(1/4) sqrt(2)/2+%i*sqrt(2)/2
```

### **numberp** (*expr*)

Función

Devuelve **true** si *expr* es un número entero, racional, de coma flotante o "bigfloat", en caso contrario devuelve **false**.

La función **numberp** devuelve **false** si su argumento es un símbolo, incluso cuando el argumento es un número simbólico como `%pi` o `%i`, o aunque haya sido declarado como "even" (par), "odd" (impar), "integer" (entero), "rational" (racional), "irrational" (irracional), "real" (real), "imaginary" (imaginario) o "complex" (complejo).

Ejemplos:

```
(%i1) numberp (42);
(%o1) true
(%i2) numberp (-13/19);
(%o2) true
(%i3) numberp (3.14159);
(%o3) true
(%i4) numberp (-1729b-4);
(%o4) true
(%i5) map (numberp, [%e, %pi, %i, %phi, inf, minf]);
(%o5) [false, false, false, false, false, false]
(%i6) declare (a, even, b, odd, c, integer, d, rational,
 e, irrational, f, real, g, imaginary, h, complex);
(%o6) done
(%i7) map (numberp, [a, b, c, d, e, f, g, h]);
(%o7) [false, false, false, false, false, false, false, false]
```

### **properties** (*a*)

Función

Devuelve una lista con los nombres de propiedades asociadas con el átomo *a*.

### **props**

Símbolo especial

**props** son átomos que tienen cualquier propiedad diferente de las mencionadas explícitamente en **infolists**, tales como **atvalues**, **matchdeclares**, etc., así como las propiedades especificadas en la función **declare**.

### **propvars** (*prop*)

Función

Devuelve una lista con aquellos átomos de la lista **props** que tienen la propiedad indicada por *prop*.

**put** (*átomo, valor, indicador*) Función

Asigna el *valor* a la propiedad (especificada por *indicador*) de *átomo*; *indicador* puede ser el nombre de cualquier propiedad y no solamente de aquellas definidas por el sistema.

La función **put** evalúa sus argumentos y devuelve *valor*.

Ejemplos:

```
(%i1) put (foo, (a+b)^5, expr);
 5
(%o1) (b + a)
(%i2) put (foo, "Hello", str);
(%o2) Hello
(%i3) properties (foo);
(%o3) [[user properties, str, expr]]
(%i4) get (foo, expr);
 5
(%o4) (b + a)
(%i5) get (foo, str);
(%o5) Hello
```

**qput** (*átomo, valor, indicador*) Función

Asigna *valor* a la propiedad de *átomo* que especifique *indicador*. Actúa del mismo modo que **put**, excepto que sus argumentos no son evaluados.

Ejemplo:

```
(%i1) foo: aa$
(%i2) bar: bb$
(%i3) baz: cc$
(%i4) put (foo, bar, baz);
(%o4) bb
(%i5) properties (aa);
(%o5) [[user properties, cc]]
(%i6) get (aa, cc);
(%o6) bb
(%i7) qput (foo, bar, baz);
(%o7) bar
(%i8) properties (foo);
(%o8) [value, [user properties, baz]]
(%i9) get ('foo, 'baz);
(%o9) bar
```

**rem** (*átomo, indicador*) Función

Elimina del *átomo* la propiedad indicada por *indicador*.

**remove** (*a\_1, p\_1, ..., a\_n, p\_n*) Función

**remove** (*[a\_1, ..., a\_m], [p\_1, ..., p\_n], ...*) Función

**remove** (*"a", operator*) Función

**remove** (*a, transfun*) Función

**remove** (*all, p*) Función

Elimina propiedades asociadas con átomos.

La llamada `remove (a_1, p_1, ..., a_n, p_n)` elimina la propiedad `p_k` del átomo `a_k`.

La llamada `remove ([a_1, ..., a_m], [p_1, ..., p_n], ...)` elimina las propiedades `p_1, ..., p_n` de los átomos `a_1, ..., a_m`. Puede tener más de un par de listas.

La llamada `remove (all, p)` elimina la propiedad `p` de todos los átomos que la tengan.

Las propiedades eliminadas pueden ser de las que define el sistema, como `function`, `macro` o `mode_declare`, o de las que define el usuario.

La llamada `remove ("a", operator)` o su equivalente `remove ("a", op)` elimina de `a` las propiedades de operador declaradas por `prefix`, `infix`, `nary`, `postfix`, `matchfix` o `nofix`. Nótese que el nombre del operador debe escribirse como cadena precedida de apóstrofo.

La llamada `remove` devuelve siempre `done` independientemente que haya algún átomo con la propiedad especificada.

**remvalue** (*nombre\_1, ..., nombre\_n*)

Función

**remvalue** (*all*)

Función

Elimina del sistema los valores de las variable de usuario *nombre\_1, ..., nombre\_n* (incluso las que tienen subíndices).

La llamada `remvalue (all)` elimina los valores de todas las variables en `values`, la lista de todas las variables a las que el usuario a dado algún nombre, pero no de aquéllas a las que Maxima asigna automáticamente un valor.

Véase también `values`.

**rncombine** (*expr*)

Función

Transforma *expr* combinando todos los términos de *expr* que tengan denominadores idénticos o que difieran unos de otros por factores numéricos. Su comportamiento es diferente al de la función `combine`, que combina términos con iguales denominadores.

Haciendo `pformat: true` y utilizando `combine` se consiguen resultados similares a aquéllos que se pueden obtener con `rncombine`, pero `rncombine` realiza el paso adicional de multiplicar denominadores numéricos. Esto da como resultado expresiones en las que se pueden reconocer algunas cancelaciones.

**scalarp** (*expr*)

Función

Devuelve `true` si *expr* es un número, constante o variable declarada como `scalar` con `declare`, o compuesta completamente de tales números, constantes o variables, pero que no contengan matrices ni listas.

**setup\_autoload** (*nombre\_fichero, función\_1, ..., función\_n*)

Función

Especifica que si alguna de las funciones *función\_1, ..., función\_n* es referenciada pero todavía no ha sido definida, se cargará *nombre\_fichero* mediante una llamada a `load`. El *nombre\_fichero* normalmente contendrá las definiciones de las funciones especificadas, aunque esto no es imperativo.

La función `setup_autoload` no opera con arreglos de funciones.

La función `setup_autoload` no evalúa sus argumentos.

Ejemplo:

```
(%i1) legendre_p (1, %pi);
(%o1) legendre_p(1, %pi)
(%i2) setup_autoload ("specfun.mac", legendre_p, ultraspherical);
(%o2) done
(%i3) ultraspherical (2, 1/2, %pi);
Warning - you are redefining the Macsyma function ultraspherical
Warning - you are redefining the Macsyma function legendre_p
 2
 3 (%pi - 1)
(%o3) ----- + 3 (%pi - 1) + 1
 2
(%i4) legendre_p (1, %pi);
(%o4) %pi
(%i5) legendre_q (1, %pi);
 %pi + 1
 %pi log(-----)
 1 - %pi
(%o5) ----- - 1
 2
```



## 37 Reglas y patrones

### 37.1 Introducción a reglas y patrones

Esta sección describe las reglas de simplificación y los patrones de comparación definidos por el usuario. Hay dos grupos de funciones que implementan diferentes esquemas de comparación de patrones. En un grupo están `tellsimp`, `tellsimpafter`, `defmatch`, `defrule`, `apply1`, `applyb1` y `apply2`. En el otro, se encuentran `let` y `letsimp`. Ambos esquemas definen patrones en términos de variables de patrones declaradas mediante `matchdeclare`.

Las reglas de comparación de patrones definidas por `tellsimp` y `tellsimpafter` se aplican automáticamente por el simplificador de Maxima. Las reglas definidas por `defmatch`, `defrule` y `let` se aplican previa llamada a una función.

Hay otros mecanismos para las reglas; las relativas a polinomios se controlan mediante `tellrat` y las del álgebra conmutativa y no conmutativa se definen en el paquete `affine`.

### 37.2 Definiciones sobre reglas y patrones

**apply1** (*expr*, *regla\_1*, ..., *regla\_n*)

Función

Aplica de forma repetida la *regla\_1* a *expr* hasta que falla, a continuación aplica repetidamente la misma regla a todas las subexpresiones de *expr*, de izquierda a derecha, hasta que la *regla\_1* haya fallado en todas las subexpresiones. Llámese *expr\_2* al resultado de transformar *expr* de esta forma. Entonces la *regla\_2* se aplica de la misma manera comenzando en el nivel superior de *expr\_2*. Cuando la *regla\_n* falla en la última expresión, se devuelve el resultado.

`maxapplydepth` es el nivel de las subexpresiones más internas procesadas por `apply1` y `apply2`.

Véase también `applyb1`, `apply2` y `let`.

**apply2** (*expr*, *regla\_1*, ..., *regla\_n*)

Función

Si la *regla\_1* falla en una subexpresión dada, entonces se aplica la *regla\_2* repetidamente, etc. Sólo si todas las reglas fallan en una subexpresión serán aplicadas todas las reglas de forma repetida a la siguiente subexpresión. Si alguna de las reglas tiene éxito entonces la misma subexpresión es reprocesada, comenzando por la primera regla.

`maxapplydepth` es el nivel de las subexpresiones más internas procesadas por `apply1` y `apply2`.

Véase también `applyb1` y `let`.

**applyb1** (*expr*, *regla\_1*, ..., *regla\_n*)

Función

Aplica la *regla\_1* reiteradamente hasta la subexpresión más interna de *expr* hasta que falle, a continuación pasa a aplicar la misma regla en un nivel superior (esto es, en subexpresiones más grandes), hasta que la *regla\_1* falle en la expresión de nivel más alto. Después se aplica la *regla\_2* de la misma manera al resultado obtenido de *regla\_1*. Tras la aplicación de la *regla\_n* a la expresión de mayor nivel, se devuelve el resultado.

La función `applyb1` es similar a `apply1` pero opera de abajo-arriba, en lugar de arriba-abajo.

`maxapplyheight` es la máxima altura a la que llega `applyb1` antes de terminar su cometido.

Véase también `apply1`, `apply2` y `let`.

### **current\_let\_rule\_package**

Variable opcional

Valor por defecto: `default_let_rule_package`

La variable `current_let_rule_package` es el nombre del paquete de reglas que están utilizando las funciones del paquete `let` (`letsimp`, etc.), a menos que se especifique otro paquete de reglas. A esta variable se le puede asignar el nombre de cualquier paquete de reglas definido por medio de la instrucción `let`.

Si se hace la llamada `letsimp (expr, rule_pkg_name)`, el paquete de reglas `rule_pkg_name` será utilizado únicamente para esa llamada y el valor de `current_let_rule_package` no cambia.

### **default\_let\_rule\_package**

Variable opcional

Valor por defecto: `default_let_rule_package`

La variable `default_let_rule_package` es el nombre del paquete de reglas utilizado cuando el usuario no especifica otro explícitamente con `let` o cambiando el valor de `current_let_rule_package`.

### **defmatch** (*nombre\_prog*, *patrón*, *x\_1*, ..., *x\_n*)

Función

Crea una función *nombre\_prog* (*expr*, *y\_1*, ..., *y\_n*) que analiza si *expr* coincide con el *patrón*.

El *patrón* es una expresión que contiene las variables de patrón *x\_1*, ..., *x\_n* así como parámetros de patrón, si los hubiere. Las variables de patrón se dan de forma explícita como argumentos a `defmatch` mientras que los parámetros de patrón son declarados por la función `matchdeclare`.

El primer argumento de la función creada *nombre\_prog* es una expresión a ser comparada con el patrón y los demás argumentos son las variables *y\_1*, ..., *y\_n* en la expresión que se corresponden con las variables ficticias *x\_1*, ..., *x\_n* del patrón.

Si el resultado de la comparación es positivo, *nombre\_prog* devuelve una lista de ecuaciones cuyos miembros izquierdos son las variables patrones y los parámetros de patrón, y cuyos miembros derechos son las expresiones en las que se han producido las coincidencias con patrones y parámetros. A los parámetros de patrón, no a las variables, se les asignan las subexpresiones con las que coinciden. Si la comparación falla, *nombre\_prog* devuelve `false`.

Cualquier variable no declarada como parámetro de patrón en `matchdeclare` o como variable en `defmatch` solamente coinciden con ellas mismas.

Un patrón que no contenga variables o parámetros de patrón devuelve `true` en caso de coincidencia.

Véase también `matchdeclare`, `defrule`, `tellsimp` y `tellsimpafter`.

Ejemplos:

Este `defmatch` define la función `linearp (expr, y)`, la cual analiza `expr` para comprobar si es de la forma  $a*y + b$  y que ni `a` ni `b` contengan a `y`.

```
(%i1) matchdeclare (a, freeof(x), b, freeof(x))$
(%i2) defmatch (linearp, a*x + b, x)$
(%i3) linearp (3*z + (y+1)*z + y^2, z);

 2
(%o3) [b = y , a = y + 4, x = z]
(%i4) a;
(%o4) y + 4
(%i5) b;
(%o5) y
```

Si el tercer argumento de `defmatch` en la entrada (%i2) hubiese sido omitido, entonces `linear` solamente trataría de hacer coincidir expresiones lineales en `x`, pero no respecto de cualquier otra variable.

```
(%i1) matchdeclare ([a, f], true)$
(%i2) constinterval (l, h) := constantp (h - l)$
(%i3) matchdeclare (b, constinterval (a))$
(%i4) matchdeclare (x, atom)$
(%i5) (remove (integrate, outative),
 defmatch (checklimits, 'integrate (f, x, a, b)),
 declare (integrate, outative))$
(%i6) 'integrate (sin(t), t, %pi + x, 2*%pi + x);
 x + 2 %pi
 /
 [
(%o6) I sin(t) dt
]
 /
 x + %pi
(%i7) checklimits (%);
(%o7) [b = x + 2 %pi, a = x + %pi, x = t, f = sin(t)]
(%i8) a;
(%o8) x + %pi
(%i9) b;
(%o9) x + 2 %pi
(%i10) f;
(%o10) sin(t)
(%i11) x;
(%o11) t
```

### **defrule** (*nombre\_regla*, *patrón*, *reemplazamiento*)

Función

Define y da nombre a una regla de reemplazamiento para el patrón dado. Si la regla *nombre\_regla* es aplicada a una expresión (por `apply1`, `applyb1` o `apply2`), cada subexpresión que coincida con el patrón será reemplazada por el contenido de *reemplazamiento*.

Las propias reglas pueden ser tratadas como funciones que transforman una expresión mediante una operación consistente en la búsqueda de una coincidencia y posterior



aplicación de un reemplazamiento. Si la comparación falla se devuelve la expresión original.

**disprule** (*nombre\_regla\_1, ..., nombre\_regla\_n*)

Función

**disprule** (*all*)

Función

Muestra las reglas de *nombre\_regla\_1, ..., nombre\_regla\_n*, tal como son devueltas por **defrule**, **tellsimp** o **tellsimpafter**, o un patrón definido por **defmatch**. Cada regla se muestra con una etiqueta de expresión intermedia (%t).

La llamada **disprule** (*all*) muestra todas las reglas.

La función **disprule** no evalúa sus argumentos y devuelve la lista de etiquetas de expresiones intermedias correspondientes a las reglas mostradas.

Véase también **letrules**, que muestra las reglas definidas por **let**.

Ejemplos:

```
(%i1) tellsimpafter (foo (x, y), bar (x) + baz (y));
(%o1) [foorule1, false]
(%i2) tellsimpafter (x + y, special_add (x, y));
(%o2) [+rule1, simplus]
(%i3) defmatch (quux, mumble (x));
(%o3) quux
(%i4) disprule (foorule1, "+rule1", quux);
(%t4) foorule1 : foo(x, y) -> baz(y) + bar(x)

(%t5) +rule1 : y + x -> special_add(x, y)

(%t6) quux : mumble(x) -> []

(%o6) [%t4, %t5, %t6]
(%i6) ' ';
(%o6) [foorule1 : foo(x, y) -> baz(y) + bar(x),
+rule1 : y + x -> special_add(x, y), quux : mumble(x) -> []]
```

**let** (*prod, repl, predname, arg\_1, ..., arg\_n*)

Función

**let** ([*prod, repl, predname, arg\_1, ..., arg\_n*], *nombre\_paquete*)

Función

Define una regla de sustitución para **letsimp** tal que *prod* es sustituido por *repl*, donde *prod* es un producto de potencias positivas o negativas de los términos siguientes:

- Átomos que **letsimp** buscará a menos que antes de llamar a **letsimp** se utilice la función **matchdeclare** para asociar un predicado con el átomo. En este caso **letsimp** hará coincidir el átomo con cualquier término del producto que satisfaga el predicado.
- Expresiones básicas como **sin(x)**, **n!**, **f(x,y)**, etc. Como en el caso anterior, **letsimp** buscará coincidencias exactas, a menos que se utilice **matchdeclare** para asociar un predicado con el argumento de la expresión básica (**sin(x)**, **n!**, **f(x,y)**, ...).

Si se incluye un predicado en la función **let** seguido de una lista de argumentos, una coincidencia aceptable (es decir, una que fuese aceptada si se hubiese omitido el predicado) se aceptará sólo si **predname** (*arg\_1', ..., arg\_n'*) vale **true**, donde

$arg.i'$  es el valor coincidente con  $arg.i$ . El argumento  $arg.i$  puede ser el nombre de cualquier átomo o el argumento de cualquier expresión básica que aparezca en  $prod$ .  $repl$  puede ser cualquier expresión racional. Si cualquiera de los átomos o argumentos de  $prod$  aparece en  $repl$  se llevan a cabo las sustituciones correspondientes.

La variable global `letrat` controla la simplificación de los cocientes por `letsimp`. Cuando `letrat` vale `false`, `letsimp` simplifica separadamente el numerador y denominador de  $expr$  y no simplifica el cociente. Sustituciones como que  $n!/n$  se reduzca a  $(n-1)!$  ya no se realizarán. Cuando `letrat` vale `true`, entonces se simplifican el numerador, el denominador y el cociente, en este orden.

Estas funciones de sustitución permiten al usuario trabajar con varios paquetes de reglas al mismo tiempo. Cada paquete de reglas puede contener cierto número de reglas `let` que son referenciadas por un nombre dado por el usuario. `let ([prod, repl, predname, arg_1, ..., arg_n], nombre_paquete)` añade la regla `predname` al paquete de reglas `nombre_paquete`. `letsimp (expr, package_name)` aplica las reglas de `nombre_paquete`. La llamada `letsimp (expr, nombre_paquete1, nombre_paquete2, ...)` es equivalente a `letsimp (expr, nombre_paquete1)` seguida de `letsimp (%, nombre_paquete2), ...`

`current_let_rule_package` es el nombre del paquete de reglas que se está utilizando. A esta variable se le puede asignar el nombre de cualquier paquete de reglas definido mediante el comando `let`. Siempre que una de las funciones incluidas en el paquete `let` sean invocadas sin nombre de paquete, se utilizará el paquete cuyo nombre se guarde en `current_let_rule_package`. Si se hace una llamada tal como `letsimp (expr, rule_pkg_name)`, el paquete de reglas `rule_pkg_name` es utilizado solamente para ese comando `letsimp`, sin efectuarse cambios en `current_let_rule_package`. A menos que se indique otra cosa, `current_let_rule_package` toma por defecto el valor de `default_let_rule_package`.

```
(%i1) matchdeclare ([a, a1, a2], true)$
(%i2) oneless (x, y) := is (x = y-1)$
(%i3) let (a1*a2!, a1!, oneless, a2, a1);
(%o3) a1 a2! --> a1! where oneless(a2, a1)
(%i4) lettrat: true$
(%i5) let (a1!/a1, (a1-1)!);
 a1!
(%o5) --- --> (a1 - 1)!
 a1
(%i6) letsimp (n*m!*(n-1)!/m);
(%o6) (m - 1)! n!
(%i7) let (sin(a)^2, 1 - cos(a)^2);
 2 2
(%o7) sin (a) --> 1 - cos (a)
(%i8) letsimp (sin(x)^4);
 4 2
(%o8) cos (x) - 2 cos (x) + 1
```

## letrat

Valor por defecto: `false`

Variable opcional

Cuando `letrat` vale `false`, `letsimp` simplifica separadamente el numerador y denominador de una fracción sin simplificar luego el cociente.

Cuando `letrat` vale `true`, se simplifican el numerador, denominador y cociente, por este orden.

```
(%i1) matchdeclare (n, true)$
(%i2) let (n!/n, (n-1)!);

 n!
(%o2) -- --> (n - 1)!
 n

(%i3) letrat: false$
(%i4) letsimp (a!/a);

 a!
(%o4) --
 a

(%i5) letrat: true$
(%i6) letsimp (a!/a);
(%o6) (a - 1)!
```

**letrules** () Función  
**letrules** (*nombre\_paquete*) Función

Muestra las reglas de un paquete de reglas. La llamada `letrules ()` muestra las reglas del paquete de reglas actual. La llamada `letrules (nombre_paquete)` muestra las reglas de `nombre_paquete`.

El paquete de reglas actual tiene su nombre almacenado en `current_let_rule_package`. A menos que se indique de otra manera, `current_let_rule_package` toma por defecto el valor de `default_let_rule_package`.

Véase también `disprule`, que muestra las reglas definidas por `tellsimp` y `tellsimpafter`.

**letsimp** (*expr*) Función  
**letsimp** (*expr*, *nombre\_paquete*) Función  
**letsimp** (*expr*, *nombre\_paquete\_1*, ..., *nombre\_paquete\_n*) Función

Aplica repetidamente las reglas definidas por `let` hasta que no se puedan hacer más cambios en *expr*.

La llamada `letsimp (expr)` utiliza las reglas de `current_let_rule_package`.

La llamada `letsimp (expr, nombre_paquete)` utiliza las reglas de *nombre\_paquete* sin efectuar cambios en `current_let_rule_package`.

La llamada `letsimp (expr, nombre_paquete_1, ..., nombre_paquete_n)` es equivalente a `letsimp (expr, nombre_paquete_1, seguida de letsimp (% , nombre_paquete_2) y así sucesivamente.`

**let\_rule\_packages** Variable opcional

Valor por defecto: `[default_let_rule_package]`

La variable `let_rule_packages` guarda una lista con todos los paquetes de reglas definidos por el usuario, junto con el paquete por defecto `default_let_rule_package`.

**matchdeclare** (*a\_1, pred\_1, ..., a\_n, pred\_n*) Función

Asocia un predicado *pred\_k* con una variable o lista de variables *a\_k*, de forma que *a\_k* se comparará con expresiones para las cuales el predicado devuelva algo que no sea **false**.

El predicado es el nombre de una función, una llamada de función sin su último argumento, **true** o **all**. Cualquier expresión se hace coincidir con **true** o **all**. Si el predicado se especifica como una llamada de función, la expresión a ser analizada es añadida a su lista de argumentos; siendo los argumentos evaluados en el momento de ser evaluada la comparación. En cambio, si el predicado se especifica como un nombre de función, la expresión a ser analizada será su único argumento. No es necesario definir una función de predicado cuando se hace una llamada a **matchdeclare**; el predicado no se evalúa hasta que se ensaya una comparación.

Un predicado de **matchdeclare** debe ser necesariamente un nombre de función o una llamada a función. En particular, un predicado no puede ser función **lambda** o un **block**.

Si una expresión satisface un predicado, se asigna a la variable de comparación la expresión, excepto cuando las variables de comparación son operandos de sumas **+** o multiplicaciones **\***. Solamente las sumas y multiplicaciones son tratadas de forma especial; los demás operadores n-arios (tanto los del sistema como los definidos por el usuario) son tratados como funciones ordinarias.

En el caso de sumas y multiplicaciones, a la variable de comparación se le puede asignar una expresión simple que satisfaga el predicado de comparación, o una suma o producto, respectivamente, de tales expresiones. Los predicados son evaluados en el orden en el que sus variables asociadas aparecen en el patrón de comparación, y un término que satisfaga más de un predicado es tomado por el primer predicado que satisfaga. Cada predicado se compara con todos los operandos de la suma o producto antes de ser evaluado el siguiente predicado. Además, si 0 o 1, respectivamente, satisface un predicado de comparación, y no hay otros términos que lo satisfagan, se asignará el 0 o 1 a la variable de comparación asociada al predicado.

El algoritmo para procesar patrones de suma y multiplicación hace que los resultados de algunas comparaciones dependan del orden de los términos en el patrón de comparación y en la expresión a ser comparada. Sin embargo, si todos los predicados de comparación son mutuamente excluyentes, el resultado de la comparación no depende para nada de la ordenación, puesto que un predicado de comparación no puede aceptar términos aceptados por otros predicados.

Invocando **matchdeclare** con una variable *a* como argumento cambia la propiedad de **matchdeclare** para *a*, si ya había una declarada; solamente el **matchdeclare** más reciente está activo cuando se define una regla. Cambios posteriores en la propiedad de **matchdeclare** (via **matchdeclare** o **remove**) no afectan a las reglas existentes.

**propvars** (**matchdeclare**) devuelve la lista de todas las variables para las cuales hay una propiedad de **matchdeclare**. La llamada **printprops** (*a*, **matchdeclare**) devuelve el predicado para la variable *a*. La llamada **printprops** (**all**, **matchdeclare**) devuelve la lista de predicados de todas las variables de **matchdeclare**. La llamada **remove** (*a*, **matchdeclare**) borra la propiedad **matchdeclare** de *a*.



```
(%i5) matchfix ("foo", "oof");
(%o5)
"foo"
(%i6) foo a, b, c oof;
(%o6)
fooa, b, coof
(%i7) >> w + foo x, y oof + z << / @ p, q ~;
 >>z + foox, yoof + w<<
(%o7)

 @p, q~
```

- Los operadores "matchfix" son funciones definidas por el usuario.

```
(%i1) matchfix ("!-", "-!");
(%o1)
"!-"
(%i2) !- x, y -! := x/y - y/x;
(%o2)
 x y
 !-x, y-! := - - -
 y x
(%i3) define (!-x, y-!, x/y - y/x);
(%o3)
 x y
 !-x, y-! := - - -
 y x
(%i4) define ("!-" (x, y), x/y - y/x);
(%o4)
 x y
 !-x, y-! := - - -
 y x
(%i5) dispfun ("!-");
(%t5)
 x y
 !-x, y-! := - - -
 y x

(%o5)
done
(%i6) !-3, 5-!;
 16
(%o6)
- --
 15
(%i7) "!-" (3, 5);
 16
(%o7)
- --
 15
```

|                                               |         |
|-----------------------------------------------|---------|
| <b>remlet</b> ( <i>prod</i> , <i>nombre</i> ) | Función |
| <b>remlet</b> ()                              | Función |
| <b>remlet</b> ( <i>all</i> )                  | Función |
| <b>remlet</b> ( <i>all</i> , <i>nombre</i> )  | Función |

Elimina la última regla de sustitución *prod* → *repl* que haya sido definida por la función **let**. Si se suministrar el nombre la regla será borrada del paquete con ese mismo nombre.

Las llamadas **remlet()** y **remlet(all)** eliminan todas las reglas de sustitución del paquete de reglas actual. Si se suministra el nombre de un paquete de reglas, como en **remlet (all, nombre)**, el paquete de reglas con ese *nombre* es también eliminado.

Si es necesario cambiar una sustitución haciendo uso de la misma producción, no es necesario llamar a `remlet`, simplemente redefinase la sustitución utilizando la misma producción con la función `let` junto con el nuevo reemplazamiento y/o nombre de predicado. De ser llamado nuevamente `remlet` (*prod*) la sustitución original sería recuperada.

Véase también `remrule`, que elimina una regla definida por `tellsimp` o `tellsimpafter`.

**remrule** (*op*, *nombre\_regla*)

Función

**remrule** (*op*, *all*)

Función

Elimina las reglas previamente definidas por `tellsimp` o `tellsimpafter`.

La llamada `remrule` (*op*, *nombre\_regla*) elimina la regla de nombre *nombre\_regla* del operador *op*.

Independientemente de que *op* sea un operador propio de Maxima o haya sido definido por el usuario (como los establecidos por `infix`, `prefix`, etc.), tanto *op* como *rulename* deben ir encerrados entre comillas dobles.

La llamada `remrule` (*function*, *all*) borra todas las reglas para el operador *op*.

Véase también `remlet`, que elimina una regla definida mediante `let`.

Ejemplos:

```
(%i1) tellsimp (foo (aa, bb), bb - aa);
(%o1) [foorule1, false]
(%i2) tellsimpafter (aa + bb, special_add (aa, bb));
(%o2) [+rule1, simplus]
(%i3) infix ("@");
(%o3) @
(%i4) tellsimp (aa @ bb, bb/aa);
(%o4) [@rule1, false]
(%i5) tellsimpafter (quux (%pi, %e), %pi - %e);
(%o5) [quuxrule1, false]
(%i6) tellsimpafter (quux (%e, %pi), %pi + %e);
(%o6) [quuxrule2, quuxrule1, false]
(%i7) [foo (aa, bb), aa + bb, aa @ bb, quux (%pi, %e), quux (%e, %pi)];
(%o7) [bb - aa, special_add(aa, bb), --, %pi - %e, %pi + %e]
 aa
(%i8) remrule (foo, forule1);
(%o8) foo
(%i9) remrule ("+", "+rule1");
(%o9) +
(%i10) remrule ("@", "@rule1");
(%o10) @
(%i11) remrule (quux, all);
(%o11) quux
(%i12) [foo (aa, bb), aa + bb, aa @ bb, quux (%pi, %e), quux (%e, %pi)];
(%o12) [foo(aa, bb), bb + aa, aa @ bb, quux(%pi, %e),
 quux(%e, %pi)]
```

**tellsimp** (*patrón, reemplazamiento*)

Función

La función **tellsimp** es similar a **tellsimpafter** pero coloca nueva información antes que la antigua, de manera que se aplica antes que las reglas de simplificación de Maxima.

La función **tellsimp** se utiliza cuando es importante utilizar la expresión antes de que el simplificador opere sobre ella; por ejemplo, cuando el simplificador ya "sabe" algo sobre una expresión, pero lo que devuelve no es lo que quiere el usuario. En cambio, cuando el simplificador ya "sabe" algo sobre una expresión pero lo que devuelve no es lo suficiente para el usuario, entonces éste podrá estar interesado en utilizar **tellsimpafter**.

El patrón no puede ser una suma, ni un producto, ni una variable ni un número.

**rules** es la lista de reglas definidas por **defrule**, **defmatch**, **tellsimp** y **tellsimpafter**.

Ejemplos:

```
(%i1) matchdeclare (x, freeof (%i));
(%o1) done
(%i2) %iargs: false$
(%i3) tellsimp (sin(%i*x), %i*sinh(x));
(%o3) [sinrule1, simp-%sin]
(%i4) trigexpand (sin (%i*y + x));
(%o4) sin(x) cos(%i y) + %i cos(x) sinh(y)
(%i5) %iargs:true$
(%i6) errcatch(0^0);
0
0 has been generated
(%o6) []
(%i7) ev (tellsimp (0^0, 1), simp: false);
(%o7) [^rule1, simpexpt]
(%i8) 0^0;
(%o8) 1
(%i9) remrule ("^", %th(2)[1]);
(%o9) ^
(%i10) tellsimp (sin(x)^2, 1 - cos(x)^2);
(%o10) [^rule2, simpexpt]
(%i11) (1 + sin(x))^2;
(%o11) (sin(x) + 1)^2
(%i12) expand (%);
(%o12) 2 sin(x) - cos^2(x) + 2
(%i13) sin(x)^2;
(%o13) 1 - cos^2(x)
(%i14) kill (rules);
(%o14) done
(%i15) matchdeclare (a, true);
(%o15) done
```



```
(%i16) tellsimp (sin(a)^2, 1 - cos(a)^2);
(%o16) [^rule3, simpexpt]
(%i17) sin(y)^2;
(%o17) 2
 1 - cos (y)
```

### **tellsimpafter** (*patrón*, *reemplazamiento*)

Función

Define una regla de simplificación que el simplificador aplicará después de las reglas de simplificación propias de de Maxima. El *patrón* es una expresión que contiene variables de patrón (declaradas por `matchdeclare`) junto con otros átomos y operadores. El contenido de *reemplazamiento* sustituye una expresión que coincida con el patrón; a las variables de patrón en *reemplazamiento* se les asignan los valores coincidentes en la expresión.

El *patrón* puede ser una expresión no atómica en la que el operador principal no sea una variable de patrón; la regla de simplificación se asocia con el operador principal. Los nombres de las funciones (con una excepción que se indica más abajo), listas y arrays pueden aparecer en el *patrón* como operador principal sólo como literales (no variables de patrones); esto excluye expresiones como `aa(x)` y `bb[y]`, si tanto `aa` como `bb` son patrones de variables. Nombres de funciones, listas y arrays que sean variables de patrón pueden aparecer como operadores que no sean el operador principal de *patrón*.

Hay una excepción a la regla indicada más arriba concerniente a los nombres de funciones. El nombre de una función subindicada en una expresión tal como `aa[x]` (y) puede ser una variable de patrón porque el operador principal no es `aa` sino el átomo de Lisp `mqapply`. Esta es una consecuencia de la representación de expresiones que contienen funciones subindicadas.

Las reglas de simplificación se aplican tras las evaluaciones (a menos que se supriman con el apóstrofo o la variable `noeval`). Las reglas establecidas por `tellsimpafter` se aplican en el orden en que han sido definidas y después de las reglas propias de Maxima. Las reglas se aplican de abajo arriba, esto es, se aplican primero a las subexpresiones antes que a toda la expresión. Puede ser necesario simplificar repetidamente un resultado (por ejemplo, mediante el operador de doble comilla simple `''` o la variable `infeval`) para asegurar que se aplican todas las reglas.

Las variables de patrón se tratan como variables locales en las reglas de simplificación. Una vez definida una regla, el valor de una variable de patrón no afecta a la regla, ni se ve influenciada por ésta. Una asignación a una variable de patrón que resulta de la aplicación exitosa de una regla no afecta a la asignación actual de la variable de patrón. Sin embargo, como cualquier otro átomo de Maxima, las propiedades de las variables de patrón (tal como se definen con `put` y sus funciones relacionadas) son globales.

La regla construida por `tellsimpafter` es nombrada detrás del operador principal de *patrón*. Reglas para operadores de Maxima y operadores definidos por el usuario con `infix`, `prefix`, `postfix`, `matchfix` y `nofix`, tienen nombres que son cadenas alfanuméricas de Maxima. Reglas para otras funciones tienen nombres que son identificadores ordinarios de Maxima.

El tratamiento de formas nominales y verbales es hasta cierto punto confuso. Si se define una regla para una forma nominal (o verbal) y ya existe una regla para la correspondiente forma verbal (o nominal), la regla recién definida se aplica a ambas formas (nominal y verbal). Si no existe regla para una forma verbal (o nominal) la regla recién definida se aplica únicamente a la forma nominal (o verbal).

La regla construida por `tellsimpafter` es una típica función de Lisp. Si el nombre de la regla es `$foorule1`, la sentencia `:lisp (trace $foorule1)` hace una traza de la función y `:lisp (symbol-function '$foorule1)` muestra su definición.

La función `tellsimpafter` no evalúa sus argumentos y devuelve la lista de reglas para el operador principal de *patrón*, incluida la regla recién establecida.

Véanse también `matchdeclare`, `defmatch`, `defrule`, `tellsimp`, `let`, `kill`, `remrule` y `clear_rules`.

Ejemplos:

*pattern* puede ser cualquier expresión no atómica en la que el operador principal no sea una variable de patrón.

```
(%i1) matchdeclare (aa, atom, [ll, mm], listp, xx, true)$
(%i2) tellsimpafter (sin (ll), map (sin, ll));
(%o2) [sinrule1, simp-%sin]
(%i3) sin ([1/6, 1/4, 1/3, 1/2, 1]*%pi);
(%o3) [-, -----, -----, 1, 0]
 2 2 2
(%i4) tellsimpafter (ll^mm, map ("^", ll, mm));
(%o4) [^rule1, simpexpt]
(%i5) [a, b, c]^[1, 2, 3];
(%o5) [a, b , c]
 2 3
(%i6) tellsimpafter (foo (aa (xx)), aa (foo (xx)));
(%o6) [foorule1, false]
(%i7) foo (bar (u - v));
(%o7) bar(foo(u - v))
```

Las reglas se aplican en el orden en que se definen. Si dos reglas coinciden con una expresión, se aplica aquélla que haya sido definida en primer lugar.

```
(%i1) matchdeclare (aa, integerp);
(%o1) done
(%i2) tellsimpafter (foo (aa), bar_1 (aa));
(%o2) [foorule1, false]
(%i3) tellsimpafter (foo (aa), bar_2 (aa));
(%o3) [foorule2, foorule1, false]
(%i4) foo (42);
(%o4) bar_1(42)
```

Las variables de patrón se tratan como variables locales en las reglas de simplificación. (Compárese con `defmatch`, que trata las variables de patrón como globales.)

```
(%i1) matchdeclare (aa, integerp, bb, atom);
(%o1) done
(%i2) tellsimpafter (foo(aa, bb), bar('aa=aa, 'bb=bb));
```

```
(%o2) [foorule1, false]
(%i3) bb: 12345;
(%o3) 12345
(%i4) foo (42, %e);
(%o4) bar(aa = 42, bb = %e)
(%i5) bb;
(%o5) 12345
```

Como cualquier otro átomo, las propiedades de las variables de patrón son globales, incluso cuando sus valores sean locales. En este ejemplo se declara una propiedad de asignación a través de `define_variable`. Esta es una propiedad del átomo `bb` en todo Maxima.

```
(%i1) matchdeclare (aa, integerp, bb, atom);
(%o1) done
(%i2) tellsimpafter (foo(aa, bb), bar('aa=aa, 'bb=bb));
(%o2) [foorule1, false]
(%i3) foo (42, %e);
(%o3) bar(aa = 42, bb = %e)
(%i4) define_variable (bb, true, boolean);
(%o4) true
(%i5) foo (42, %e);
Error: bb was declared mode boolean, has value: %e
-- an error. Quitting. To debug this try debugmode(true);
```

Las reglas se nombran después de los operadores principales. Los nombres de reglas tanto para las funciones de Maxima como para las definidas por el usuario son cadenas alfanuméricas, mientras que los nombres de las otras funciones son identificadores típicos.

```
(%i1) tellsimpafter (foo (%pi + %e), 3*%pi);
(%o1) [foorule1, false]
(%i2) tellsimpafter (foo (%pi * %e), 17*%e);
(%o2) [foorule2, foorule1, false]
(%i3) tellsimpafter (foo (%i ^ %e), -42*%i);
(%o3) [foorule3, foorule2, foorule1, false]
(%i4) tellsimpafter (foo (9) + foo (13), quux (22));
(%o4) [+rule1, simplus]
(%i5) tellsimpafter (foo (9) * foo (13), blurf (22));
(%o5) [*rule1, simptimes]
(%i6) tellsimpafter (foo (9) ^ foo (13), mumble (22));
(%o6) [^rule1, simpexpt]
(%i7) rules;
(%o7) [trigrule0, trigrule1, trigrule2, trigrule3, trigrule4,
htrigrule1, htrigrule2, htrigrule3, htrigrule4, foorule1,
foorule2, foorule3, +rule1, *rule1, ^rule1]
(%i8) foorule_name: first (%o1);
(%o8) foorule1
(%i9) plusrule_name: first (%o4);
(%o9) +rule1
(%i10) [?mstringp (foorule_name), symbolp (foorule_name)];
(%o10) [false, true]
```

```

(%i11) [?mstringp (plusrule_name), symbolp (plusrule_name)];
(%o11) [true, true]
(%i12) remrule (foo, foorule1);
(%o12) foo
(%i13) remrule ("^", "^rule1");
(%o13) ^

```

**clear\_rules ()**

Función

Ejecuta `kill (rules)` y después inicializa el siguiente número de regla a 1 para la adición `+`, multiplicación `*` y exponenciación `^`.



## 38 Listas

### 38.1 Introducción a las listas

Las listas son bloques de construcción básica para Maxima y Lisp. Todos los tipos de datos diferentes a los arreglos, tablas mixtas o números son representados como listas Lisp, estas listas Lisp tienen la forma

```
((MPLUS) $A 2)
```

para indicar la expresión  $a+2$ . Al nivel de Maxima se observará la notación infija  $a+2$ . Maxima también tiene listas con el formato

```
[1, 2, 7, x+y]
```

para una lista de 4 elementos. Internamente esto se corresponde con una lista Lisp de la forma

```
((MLIST) 1 2 7 ((MPLUS) $X $Y))
```

El elemento que denota el tipo de expresión en Maxima es también una lista, la cual tras ser analizada y simplificada tomará la forma

```
((MLIST SIMP) 1 2 7 ((MPLUS SIMP) $X $Y))
```

### 38.2 Definiciones para listas

**append** (*lista\_1*, ..., *lista\_n*)

Función

Devuelve una lista cuyos elementos son los de la lista *lista\_1* seguidos de los de *lista\_2*, ... La función **append** también opera con expresiones generales, como la llamada **append** ( $f(a,b)$ ,  $f(c,d,e)$ );, de la que se obtiene  $f(a,b,c,d,e)$ .

Tecléese **example(append)**; para ver un ejemplo.

**assoc** (*clave*, *lista*, *valor\_por\_defecto*)

Función

**assoc** (*clave*, *lista*)

Function

Esta función busca la *clave* en el lado derecho de la *lista*, la cual es de la forma  $[x,y,z,\dots]$ , donde cada elemento es una expresión formada por un operador binario y dos elementos. Por ejemplo,  $x=1$ ,  $2^3$ ,  $[a,b]$  etc. La *clave* se compara con el primer operando. La función **assoc** devuelve el segundo operando si se encuentra con que la *clave* coincide. Si la *clave* no coincide entonces devuelve el valor *valor\_por\_defecto*. El argumento *valor\_por\_defecto* es opcional; en caso de no estar presente, se devolverá **false**.

**atom** (*expr*)

Función

Devuelve **true** si *expr* es un átomo (número, nombre o cadena alfanumérica) y **false** en caso contrario. Así, **atom**(5) devolverá **true**, mientras que **atom**( $a[1]$ ) y **atom**( $\sin(x)$ ) darán como resultado **false** (dando por hecho que tanto  $a[1]$  como  $x$  no tienen valores asignados).

**cons** (*expr*, *lista*)

Función

Devuelve una nueva lista en la que el elemento *expr* ocupa la primera posición, seguido de los elementos de *lista*. La función **cons** también opera con otro tipo de expresiones, como **cons**( $x$ ,  $f(a,b,c)$ );  $\rightarrow f(x,a,b,c)$ .

**copylist** (*lista*)

Función

Devuelve una copia de la *lista*.**create\_list** (*form*, *x\_1*, *list\_1*, ..., *x\_n*, *list\_n*)

Función

Crea una lista mediante la evaluación de *form* con *x\_1* tomando cada uno de los valores de *list\_1*, para cada uno de estos valores liga *x\_2* con cada elemento de *list\_2*, .... El número de elementos en el resultado será el producto del número de elementos en cada lista. Cada variable *x\_i* debe ser un símbolo y no será evaluado. La lista de argumentos será evaluada una vez al comienzo de la iteración.

Por ejemplo:

```
(%i1) create_list(x^i,i,[1,3,7]);
 3 7
(%o1) [x, x , x]
```

Con una doble iteración:

```
(%i1) create_list([i,j],i,[a,b],j,[e,f,h]);
(%o1) [[a, e], [a, f], [a, h], [b, e], [b, f], [b, h]]
```

En lugar de *list\_i* se pueden suministrar dos argumentos cada uno de los cuales debería poder evaluarse a un número, los cuales serán los límites inferior y superior, ambos inclusive, para cada iteración.

Por ejemplo:

```
(%i1) create_list([i,j],i,[1,2,3],j,1,i);
(%o1) [[1, 1], [2, 1], [2, 2], [3, 1], [3, 2], [3, 3]]
```

Nótese que los límites o lista para la variable *j* pueden depender del valor actual de *i*.

**delete** (*expr\_1*, *expr\_2*)

Función

**delete** (*expr\_1*, *expr\_2*, *n*)

Función

Elimina las expresiones *expr\_1* de *expr\_2*. La expresión *expr\_1* a eliminar puede ser un sumando de *expr\_2* (en caso de que ésta sea una suma) o un factor de *expr\_2* (si se trata de un producto).

```
(%i1) delete(sin(x), x+sin(x)+y);
(%o1) y + x
```

La llamada **delete**(*expr\_1*, *expr\_2*, *n*) elimina las primeras *n* apariciones de *expr\_1* en *expr\_2*. En caso de que *expr\_1* aparezca menos de *n* veces dentro de la expresión de referencia *expr\_2* entonces se eliminan todas ellas.

```
(%i1) delete(a, f(a,b,c,d,a));
(%o1) f(b, c, d)
(%i2) delete(a, f(a,b,a,c,d,a), 2);
(%o2) f(b, c, d, a)
```

**eighth** (*expr*)

Función

Devuelve el octavo elemento de la lista o expresión *expr*. Véase **first** para más detalles.

**endcons** (*expr*, *lista*) Función  
 Devuelve una nueva lista formada por los elementos de *lista* seguidos de los de *expr*.  
 La función **endcons** también opera con expresiones generales, por ejemplo **endcons**(*x*, *f*(*a*,*b*,*c*)); -> *f*(*a*,*b*,*c*,*x*).

**fifth** (*expr*) Función  
 Devuelve el quinto elemento de la lista o expresión *expr*. Véase **first** para más detalles.

**first** (*expr*) Función  
 Devuelve la primera parte de *expr*, que puede consistir en el primer elemento de una lista, la primera fila de una matriz, el primer término de una suma, etc. Nótese que tanto **first** como sus funciones relacionadas, **rest** y **last**, operan sobre la forma en la que *expr* es mostrada por Maxima, no sobre la forma en la que es introducida la expresión. Sin embargo, cuando la variable **inflag** toma el valor **true** estas funciones tendrán en cuenta el formato interno de *expr*. Téngase en cuenta que el simplificador reordena las expresiones. Así, **first**(*x*+*y*) devolverá *x* si **inflag** vale **true** y *y* cuando **inflag** tome el valor **false** (**first**(*y*+*x*) devuelve el mismo resultado). Las funciones **second** ... **tenth** devuelven desde el segundo hasta el décimo elemento del argumento de entrada.

**fourth** (*expr*) Función  
 Devuelve el cuarto elemento de la lista o expresión *expr*. Véase **first** para más detalles.

**get** (*a*, *i*) Función  
 Recupera la propiedad de usuario indicada por *i* asociada al átomo *a* o devuelve **false** si *a* no tiene la propiedad *i*.

La función **get** evalúa sus argumentos.

```
(%i1) put (%e, 'transcendental, 'type);
(%o1) transcendental
(%i2) put (%pi, 'transcendental, 'type)$
(%i3) put (%i, 'algebraic, 'type)$
(%i4) typeof (expr) := block ([q],
 if numberp (expr)
 then return ('algebraic),
 if not atom (expr)
 then return (maplist ('typeof, expr)),
 q: get (expr, 'type),
 if q=false
 then errcatch (error(expr,"is not numeric.)) else q)$
(%i5) typeof (2*%e + x*%pi);
x is not numeric.
(%o5) [[transcendental, []], [algebraic, transcendental]]
(%i6) typeof (2*%e + %pi);
(%o6) [transcendental, [algebraic, transcendental]]
```



**join** (*l*, *m*) Función

Crea una nueva lista con los elementos de las listas *l* y *m* alternados. El resultado tiene como elementos  $[l[1], m[1], l[2], m[2], \dots]$ . Las listas *l* y *m* pueden contener cualquier tipo de elementos.

Si las listas son de diferente longitud, **join** ignora los elementos sobrantes de la lista más larga.

Maxima da error si o bien *l* o *m* no son listas.

Ejemplos:

```
(%i1) L1: [a, sin(b), c!, d - 1];
(%o1) [a, sin(b), c!, d - 1]
(%i2) join (L1, [1, 2, 3, 4]);
(%o2) [a, 1, sin(b), 2, c!, 3, d - 1, 4]
(%i3) join (L1, [aa, bb, cc, dd, ee, ff]);
(%o3) [a, aa, sin(b), bb, c!, cc, d - 1, dd]
```

**last** (*expr*) Función

Devuelve la última parte (término, fila, elemento, etc.) de *expr*.

**length** (*expr*) Función

Devuelve (por defecto) el número de partes de que consta *expr* en la versión correspondiente a la que muestra. En el caso de listas, se devuelve el número de elementos, si se trata de matrices el número de filas y se se trata de sumas el número de términos o sumandos (véase **dispform**).

La función **length** se ve afectada por el valor de la variable **inflag**. Así, **length(a/(b\*c))**; devuelve 2 si **inflag** vale **false** (dando por hecho que **exptdispflag** vale **true**), pero devuelve 3 si **inflag** vale **true** (ya que la representación interna es  $a*b^{-1}*c^{-1}$ ).

**listarith** Variable opcional

Valor por defecto: **true**

Cuando vale **false** provoca que no se realicen operaciones aritméticas con listas; cuando vale **true**, las operaciones con listas y matrices son contagiosas, en el sentido de que las listas se transforman en matrices, retornando resultados de este último tipo. Sin embargo, operaciones que involucren listas con listas devolverán también listas.

**listp** (*expr*) Función

Devuelve el valor **true** si *expr* es una lista, y **false** en caso contrario.

**makelist** (*expr*, *i*, *i.0*, *i.1*) Función

**makelist** (*expr*, *x*, *list*) Función

Construye y devuelve una lista, siendo cada uno de sus elementos generados por *expr*.

La llamada **makelist** (*expr*, *i*, *i.0*, *i.1*) devuelve una lista cuyo *j*-ésimo elemento es igual a **ev** (*expr*, *i=j*), tomando *j* los valores enteros entre *i.0* y *i.1*.

La llamada `makelist (expr, x, list)` devuelve una lista cuyo  $j$ -ésimo elemento es igual a `ev (expr, x=list[j])`, tomando  $j$  los valores enteros entre 1 through `length (list)`.

Ejemplos:

```
(%i1) makelist(concat(x,i),i,1,6);
(%o1) [x1, x2, x3, x4, x5, x6]
(%i2) makelist(x=y,y,[a,b,c]);
(%o2) [x = a, x = b, x = c]
```

**member** (*expr*, *lista*) Función

Devuelve el valor `true` si *expr* resulta ser un miembro de la *lista* (no que forme parte de uno de sus miembros). En caso contrario se devuelve `false`. La función `member` también opera sobre expresiones que no son necesariamente listas, como por ejemplo `member(b,f(a,b,c)); -> true`.

**ninth** (*expr*) Función

Devuelve el noveno elemento de la lista o expresión *expr*. Véase `first` para más detalles.

**rest** (*expr*, *n*) Función

**rest** (*expr*) Función

Devuelve *expr* sin sus primeros  $n$  elementos si  $n$  es positivo, o sus últimos  $-n$  elementos si  $n$  es negativo. En caso de que  $n$  tome el valor 1 puede ser omitido. La expresión *expr* puede ser una lista, una matriz o cualquier otra expresión.

**reverse** (*lista*) Función

Invierte el orden de los elementos de la *lista* (no los propios elementos). La función `reverse` también opera sobre expresiones generales, como en `reverse(a=b); gives b=a`.

**second** (*expr*) Función

Devuelve el segundo elemento de la lista o expresión *expr*. Véase `first` para más detalles.

**seventh** (*expr*) Función

Devuelve el séptimo elemento de la lista o expresión *expr*. Véase `first` para más detalles.

**sixth** (*expr*) Función

Devuelve el sexto elemento de la lista o expresión *expr*. Véase `first` para más detalles.

**tenth** (*expr*) Función

Devuelve el décimo elemento de la lista o expresión *expr*. Véase `first` para más detalles.

**third** (*expr*) Función  
Devuelve el tercer elemento de la lista o expresión *expr*. Véase **first** para más detalles.

## 39 Conjuntos

### 39.1 Introducción a los conjuntos

Maxima dispone de funciones para realizar operaciones con conjuntos, como la intersección o la unión. Los conjuntos deben ser finitos y definidos por enumeración. Maxima trata a los conjuntos y a las listas como objetos de distinta naturaleza, lo que permite trabajar con conjuntos cuyos elementos puedan ser también conjuntos o listas.

Además de funciones para operar con conjuntos finitos, Maxima dispone también de algunas funciones sobre combinatoria, como los números de Stirling de primera y segunda especie, números de Bell, coeficientes multinomiales, particiones de enteros no negativos y algunos otros. Maxima también define la función delta de Kronecker.

#### 39.1.1 Utilización

Para construir un conjunto cuyos elementos sean  $a_1, \dots, a_n$ , se utiliza la instrucción `set(a_1, ..., a_n)` o `{a_1, ..., a_n}`; para formar un conjunto vacío, basta con hacer `set()` o `{}`. Para introducir conjuntos en Maxima, `set(...)` y `{...}` son equivalentes. Los conjuntos se muestran siempre con llave.

Si un elemento se indica más de una vez, el proceso de simplificación elimina los elementos redundantes.

```
(%i1) set();
(%o1) {}
(%i2) set(a, b, a);
(%o2) {a, b}
(%i3) set(a, set(b));
(%o3) {a, {b}}
(%i4) set(a, [b]);
(%o4) {a, [b]}
(%i5) {};
(%o5) {}
(%i6) {a, b, a};
(%o6) {a, b}
(%i7) {a, {b}};
(%o7) {a, {b}}
(%i8) {a, [b]};
(%o8) {a, [b]}
```

Dos elementos candidatos a formar parte de un conjunto,  $x$  e  $y$ , son redundantes, esto es, se consideran el mismo elemento a efectos de construir el conjunto, si y sólo si `is(x = y)` devuelve el valor `true`. Nótese que `is(equal(x, y))` puede devolver `true` y `is(x = y)` retornar `false`; en cuyo caso los elementos  $x$  e  $y$  se considerarían distintos.

```
(%i1) x: a/c + b/c;
(%o1)
$$\frac{b}{c} + \frac{a}{c}$$

(%i2) y: a/c + b/c;
```

```

(%o2)

$$\frac{b}{c} + \frac{a}{c}$$

(%i3) z: (a + b)/c;
(%o3)

$$\frac{b + a}{c}$$

(%i4) is (x = y);
(%o4) true
(%i5) is (y = z);
(%o5) false
(%i6) is (equal (y, z));
(%o6) true
(%i7) y - z;
(%o7)

$$-\frac{b + a}{c} + \frac{b}{c} + \frac{a}{c}$$

(%i8) ratsimp (%);
(%o8) 0
(%i9) set (x, y, z);
(%o9)

$$\left\{ \frac{b + a}{c}, \frac{b}{c}, \frac{a}{c} \right\}$$


```

Para formar un conjunto a partir de los miembros de una lista úsese `setify`.

```

(%i1) setify([b, a]);
(%o1) {a, b}

```

Los elementos `x` e `y` de un conjunto se consideran iguales si `is(x = y)` devuelve el valor `true`. Así, `rat(x)` y `x` se consideran el mismo elemento de un conjunto; consecuentemente,

```

(%i1) set(x, rat(x));
(%o1) {x}

```

Además, puesto que `is((x-1)*(x+1) = x^2 - 1)` devuelve `false`, `(x-1)*(x+1)` y `x^2-1` se consideran elementos diferentes; así

```

(%i1) set((x - 1)*(x + 1), x^2 - 1);
(%o1) {(x - 1) (x + 1), x^2 - 1}

```

Para reducir este conjunto a otro unitario, aplicar `rat` a cada elemento del conjunto:

```

(%i1) set((x - 1)*(x + 1), x^2 - 1);
(%o1) {(x - 1) (x + 1), x^2 - 1}
(%i2) map(rat, %);
(%o2)/R/ {x^2 - 1}

```

Para eliminar redundancias con otros conjuntos, será necesario utilizar otras funciones de simplificación. He aquí un ejemplo que utiliza `trigsimp`:

```

(%i1) set(1, cos(x)^2 + sin(x)^2);
(%o1) {1, sin^2(x) + cos^2(x)}

```

```
(%i2) map(trigsimp, %);
(%o2) {1}
```

Se entiende que un conjunto está simplificado cuando entre sus elementos no hay redundancias y se hayan ordenados. La versión actual de las funciones para conjuntos utiliza la función `orderlessp` de Maxima para ordenar sus elementos; sin embargo, *futuras versiones de las funciones para operar con conjuntos podrán utilizar otras funciones de ordenación.*

Algunas operaciones con conjuntos, tales como la sustitución, fuerzan automáticamente una re-simplificación; por ejemplo,

```
(%i1) s: set (a, b, c)$
(%i2) subst (c=a, s);
(%o2) {a, b}
(%i3) subst ([a=x, b=x, c=x], s);
(%o3) {x}
(%i4) map (lambda ([x], x^2), set (-1, 0, 1));
(%o4) {0, 1}
```

Maxima considera a las listas y conjuntos como objetos diferentes; funciones tales como `union` y `intersection` provocarán un error si alguno de sus argumentos es una lista. Si se necesita aplicar una función de conjunto a una lista, se deberá utilizar la función `setify` para convertirla previamente en conjunto. Así,

```
(%i1) union ([1, 2], set (a, b));
Function union expects a set, instead found [1,2]
-- an error. Quitting. To debug this try debugmode(true);
(%i2) union (setify ([1, 2]), set (a, b));
(%o2) {1, 2, a, b}
```

Para extraer todos los elementos de un conjunto `s` que satisfagan un predicado `f`, úsese `subset(s,f)`. (Un *predicado* es una función booleana.) Por ejemplo, para encontrar las ecuaciones en un conjunto dado que no dependan de la variable `z`, se hará

```
(%i1) subset (set (x + y + z, x - y + 4, x + y - 5), lambda ([e], freeof (z, e)));
(%o1) {- y + x + 4, y + x - 5}
```

La sección **Definiciones para los conjuntos** incluye una lista completa de funciones para operar con conjuntos en Maxima.

### 39.1.2 Iteraciones con elementos

Hay dos formas para operar iterativamente sobre los elementos de un conjunto. Una es utilizar `map`; por ejemplo:

```
(%i1) map (f, set (a, b, c));
(%o1) {f(a), f(b), f(c)}
```

La otra forma consiste en hacer uso de la construcción `for x in s do`

```
(%i1) s: set (a, b, c);
(%o1) {a, b, c}
(%i2) for si in s do print (concat (si, 1));
a1
b1
c1
(%o2) done
```

Las funciones de Maxima `first` y `rest` funcionan también con conjuntos. En este caso, `first` devuelve el primer elemento que se muestra del conjunto, el cual puede depender de la implementación del sistema. Si `s` es un conjunto, entonces `rest(s)` equivale a `disjoin (first(s), s)`. Hay otras funciones que trabajan correctamente con conjuntos. En próximas versiones de las funciones para operar con conjuntos es posible que `first` y `rest` trabajen de modo diferente o que ya no lo hagan en absoluto.

### 39.1.3 Fallos

Las funciones para operar con conjuntos utilizan la función `orderlessp` de Maxima para ordenar los elementos de los conjuntos, así como la función `like` de Lisp para decidir sobre la igualdad de dichos elementos. Ambas funciones tienen fallos que son conocidos y que pueden aflorar si se trabaja con conjuntos que tengan elementos en formato de listas o matrices y que contengan expresiones CRE. Un ejemplo es

```
(%i1) set ([x], [rat (x)]);
Maxima encountered a Lisp error:
```

```
CAR: #:X13129 is not a LIST
```

```
Automatically continuing.
```

```
To reenale the Lisp debugger set *debugger-hook* to nil.
```

Esta instrucción provoca una parada de Maxima junto con la emisión de un mensaje de error, el cual dependerá de la versión de Lisp que utilice Maxima. Otro ejemplo es

```
(%i1) setify ([[rat(a)], [rat(b)]]);
Maxima encountered a Lisp error:
```

```
CAR: #:A13129 is not a LIST
```

```
Automatically continuing.
```

```
To reenale the Lisp debugger set *debugger-hook* to nil.
```

Estos fallos son causados por fallos en `orderlessp` y `like`, no por fallos cuyo origen se encuentre en las funciones para conjuntos. Para ilustrarlo, se pueden ejecutar las siguientes instrucciones

```
(%i1) orderlessp ([rat(a)], [rat(b)]);
Maxima encountered a Lisp error:
```

```
CAR: #:B13130 is not a LIST
```

```
Automatically continuing.
```

```
To reenale the Lisp debugger set *debugger-hook* to nil.
```

```
(%i2) is ([rat(a)] = [rat(a)]);
(%o2) false
```

Hasta que estos errores no se corrijan, no es aconsejable construir conjuntos que tengan por elementos listas o matrices que contengan expresiones en forma CRE; sin embargo, un conjunto con elementos de la forma CRE no deberían dar problemas:

```
(%i1) set (x, rat (x));
(%o1) {x}
```

La función `orderlessp` de Maxima tiene otro fallo que puede causar problemas con las funciones para conjuntos, en concreto, que el predicado de ordenación `orderlessp` no es transitivo. El ejemplo más simple que ilustra este punto es

```
(%i1) q: x^2$
(%i2) r: (x + 1)^2$
(%i3) s: x*(x + 2)$
(%i4) orderlessp (q, r);
(%o4) true
(%i5) orderlessp (r, s);
(%o5) true
(%i6) orderlessp (q, s);
(%o6) false
```

El fallo puede causar problemas con todas las funciones para conjuntos, así como también con otras funciones de Maxima. Es probable, pero no seguro, que si todos los elementos del conjunto están en la forma CRE o han sido simplificados con `ratsimp`, entonces el fallo no se manifiesta.

Los mecanismos `orderless` y `ordergreat` de Maxima son incompatibles con las funciones para conjuntos. Si se necesitan utilizar `orderless` o `ordergreat`, hágase antes de construir los conjuntos y no se utilice la instrucción `unorder`.

La función `sign` de Maxima tiene un fallo que puede causar que la función delta de Kronecker trabaje incorrectamente; por ejemplo:

```
(%i1) kron_delta (1/sqrt(2), sqrt(2)/2);
(%o1) 0
```

El valor correcto es 1. El fallo está causado por el de la función `sign`

```
(%i1) sign (1/sqrt(2) - sqrt(2)/2);
(%o1) pos
```

Se ruega a todo usuario que crea haber encontrado un fallo en las funciones para conjuntos que lo comunique en la base de datos de Maxima. Véase `bug_report`.

### 39.1.4 Autores

Stavros Macrakis de Cambridge, Massachusetts y Barton Willis de la University of Nebraska at Kearney (UNK).

## 39.2 Definiciones para los conjuntos

### `adjoin` ( $x, a$ )

Función

Incorpora  $x$  al conjunto  $a$  y devuelve el nuevo conjunto. Así, `adjoin(x, a)` y `union(set(x), a)` son equivalentes; sin embargo, haciendo `adjoin` puede ser algo más rápido que utilizando `union`. Si  $a$  no es un conjunto, se emite un error.

```
(%i1) adjoin (c, set (a, b));
(%o1) {a, b, c}
(%i2) adjoin (a, set (a, b));
(%o2) {a, b}
```

Véase también `disjoin`.



**belln** (*n*) Función

Para el entero no negativo *n*, devuelve el *n*-ésimo número de Bell. Si *s* es un conjunto con *n* elementos, **belln**(*n*) es el número de particiones de *s*. Por ejemplo:

```
(%i1) makelist (belln (i), i, 0, 6);
(%o1) [1, 1, 2, 5, 15, 52, 203]
(%i2) is (cardinality (set_partitions (set ())) = belln (0));
(%o2) true
(%i3) is (cardinality (set_partitions (set (1, 2, 3, 4, 5, 6))) = belln (6));
(%o3) true
```

Si *n* no es un entero no negativo, la función **belln**(*n*) no hace cálculo alguno.

```
(%i1) [belln (x), belln (sqrt(3)), belln (-9)];
(%o1) [belln(x), belln(sqrt(3)), belln(- 9)]
```

**cardinality** (*a*) Función

Devuelve el número de elementos del conjunto *a*.

```
(%i1) cardinality (set ());
(%o1) 0
(%i2) cardinality (set (a, a, b, c));
(%o2) 3
(%i3) cardinality (set (a, a, b, c)), simp: false;
(%o3) 3
```

En la salida (%o3) se observa que la cardinalidad trabaja correctamente incluso cuando la simplificación se ha desactivado.

**cartesian\_product** (*b\_1*, ... , *b\_n*) Función

Devuelve un conjunto formado por listas de la forma [*x\_1*, ..., *x\_n*], donde *x\_1* in *b\_1*, ..., *x\_n* in *b\_n*. Envía un error si cualquiera de los *b\_k* no es un conjunto.

```
(%i1) cartesian_product (set (0, 1));
(%o1) {[0], [1]}
(%i2) cartesian_product (set (0, 1), set (0, 1));
(%o2) {[0, 0], [0, 1], [1, 0], [1, 1]}
(%i3) cartesian_product (set (x), set (y), set (z));
(%o3) {[x, y, z]}
(%i4) cartesian_product (set (x), set (-1, 0, 1));
(%o4) {[x, - 1], [x, 0], [x, 1]}
```

**disjoin** (*x*, *a*) Función

Elimina *x* del conjunto *a* y devuelve un conjunto. Si *x* no es elemento de *a*, devuelve *a*. Cada una de las siguientes llamadas hace lo mismo: **disjoin**(*x*, *a*), **delete**(*x*, *a*) y **setdifference**(*a*,**set**(*x*)); sin embargo, **disjoin** es generalmente el método más rápido para eliminar un elemento de un conjunto. Devuelve un error en caso de que *a* no sea un conjunto.

**disjointp** (*a*, *b*) Función

Devuelve **true** si los conjuntos *a* y *b* son disjuntos. Envía un mensaje de error si bien *a* o bien *b* no son conjuntos.

**divisors** (*n*)

Función

Si *n* es un entero no nulo, devuelve el conjunto de sus divisores. El conjunto de divisores incluye los elementos 1 y *n*. Los divisores de un entero negativo son los divisores de su valor absoluto.

Se puede comprobar que 28 es un número perfecto.

```
(%i1) s: divisors(28);
(%o1) {1, 2, 4, 7, 14, 28}
(%i2) lreduce ("+", args(s)) - 28;
(%o2) 28
```

La función **divisors** opera por simplificación; no es necesario volver a evaluar tras una sustitución. Por ejemplo:

```
(%i1) divisors (a);
(%o1) divisors(a)
(%i2) subst (8, a, %);
(%o2) {1, 2, 4, 8}
```

La función **divisors** se distribuye respecto de igualdades, listas, matrices y conjuntos. He aquí un ejemplo de distribución sobre una lista y una igualdad.

```
(%i1) divisors ([a, b, c=d]);
(%o1) [divisors(a), divisors(b), divisors(c) = divisors(d)]
```

**elementp** (*x*, *a*)

Función

Devuelve **true** si y sólo si *x* es un elemento del conjunto *a*. Envía un error si *a* no es un conjunto.

**empty** (*a*)

Función

Devuelve **true** si y sólo si *a* es el conjunto vacío o la lista vacía.

```
(%i1) map (empty, [set (), []]);
(%o1) [true, true]
(%i2) map (empty, [a + b, set (set ()), %pi]);
(%o2) [false, false, false]
```

**equiv\_classes** (*s*, *f*)

Función

Devuelve el conjunto de las clases de equivalencia de *s* respecto de la relación de equivalencia *f*. La función *f* debe ser booleana y definida sobre el producto cartesiano de *s* por *s*. Además, la función *f* debe ser una relación de equivalencia, debiéndose tener en cuenta que **equiv\_classes** no chequea esta propiedad.

```
(%i1) equiv_classes (set (a, b, c), lambda ([x, y], is (x=y)));
(%o1) {{a}, {b}, {c}}
```

En este momento, **equiv\_classes** (*s*, *f*) aplica automáticamente la función **is** de Maxima después de aplicar la función *f*; por lo tanto, se puede volver a hacer el ejemplo anterior de forma más breve.

```
(%i1) equiv_classes (set (a, b, c), "=");
(%o1) {{a}, {b}, {c}}
```

Aquí otro ejemplo.

```
(%i1) equiv_classes (set (1, 2, 3, 4, 5, 6, 7), lambda ([x, y], remainder (x -
(%o1) {{1, 4, 7}, {2, 5}, {3, 6}}
```

**every** (*f*, *a*) Función  
**every** (*f*, *L\_1*, ..., *L\_n*) Función

El primer argumento *f* debe ser un predicado (es decir, una función que toma cualquiera de los valores **true**, **false**, or **unknown**).

Dado un conjunto como segundo argumento, **every** (*f*, *a*) devuelve **true** si *f*(*a\_i*) devuelve también **true** para todo *a\_i* de *a*. Puesto que los conjuntos no están ordenados, **every** puede evaluar *f*(*a\_i*) en cualquier orden. La función **every** puede que no evalúe *f* para todo *a\_i* de *a*. Debido a que el orden de evaluación no está especificado, el predicado *f* no debe tener efectos secundarios ni emitir señales de error para ninguna entrada.

Dada una o más listas como argumentos, **every** (*f*, *L\_1*, ..., *L\_n*) devuelve **true** si *f*(*x\_1*, ..., *x\_n*) devuelve **true** para cada *x\_1*, ..., *x\_n* de *L\_1*, ..., *L\_n*, respectivamente. La función **every** puede que no evalúe *f* para todas las combinaciones *x\_1*, ..., *x\_n*. Puesto que las listas están ordenadas, **every** evalúa en el orden del índice creciente.

Para usar **every** con múltiples conjuntos como argumentos, deberían convertirse primero en una secuencia ordenada, de manera que su alineación relativa quede bien definida.

Si la variable global **maperror** vale **true** (su valor por defecto), todas las listas *L\_1*, ..., *L\_n* deben tener igual número de miembros; en otro caso, **every** envía un mensaje de error. Cuando **maperror** vale **false**, las listas son truncadas cada una a la longitud de la lista más corta.

La función **is** se aplica automáticamente tras cada evaluación del predicado *f*.

```
(%i1) every ("=", [a, b], [a, b]);
(%o1) true
(%i2) every ("#", [a, b], [a, b]);
(%o2) false
```

**extremal\_subset** (*s*, *f*, *max*) Función  
**extremal\_subset** (*s*, *f*, *min*) Función

Cuando el tercer argumento es **max**, devuelve el subconjunto del conjunto o lista *s* para cuyos elementos la función real *f* toma su valor mayor; cuando el tercer argumento es **min**, devuelve el subconjunto para el cual *f* toma su valor menor.

```
(%i1) extremal_subset (set (-2, -1, 0, 1, 2), abs, max);
(%o1) {- 2, 2}
(%i2) extremal_subset (set (sqrt(2), 1.57, %pi/2), sin, min);
(%o2) {sqrt(2)}
```

**flatten** (*e*) Función

La función **flatten** evalúa una expresión como si su operador principal hubiese sido declarado n-ario; hay sin embargo una diferencia, **flatten** no es recursiva para otros argumentos de la función. Por ejemplo:

```
(%i1) expr: flatten (f (g (f (f (x)))))
(%o1) f(g(f(f(x))))
(%i2) declare (f, nary);
```

```
(%o2) done
(%i3) ev (expr);
(%o3) f(g(f(x)))
```

Aplicada a un conjunto, **flatten** reúne todos los elementos de los conjuntos que son a su vez elementos; por ejemplo:

```
(%i1) flatten (set (a, set (b), set (set (c))));
(%o1) {a, b, c}
(%i2) flatten (set (a, set ([a], set (a))));
(%o2) {a, [a]}
```

La función **flatten** trabaja correctamente cuando el operador principal es una función con subíndices

```
(%i1) flatten (f[5] (f[5] (x)));
(%o1) f (x)
 5
```

Para alisar (**flatten**) una expresión, el operador principal debe estar definido para cero o más argumentos; si no es éste el caso, Maxima se detendrá con un error. Expresiones con representaciones especiales, como expresiones CRE, no pueden ser alisadas; en este caso, **flatten** devuelve su argumento sin cambios.

### **full\_listify** (*a*) Función

Si *a* es un conjunto, convierte *a* en una lista y aplica **full\_listify** a cada miembro de la lista.

Para convertir sólo el operador de mayor nivel de un conjunto a una lista, véase **listify**.

### **fullsetify** (*a*) Función

Si *a* es una lista, convierte *a* en un conjunto y aplica **fullsetify** a cada elemento del conjunto.

```
(%i1) fullsetify ([a, [a]]);
(%o1) {a, {a}}
(%i2) fullsetify ([a, f([b])]);
(%o2) {a, f([b])}
```

En la salida (%o2) el argumento de **f** no se convierte en conjunto porque el operador principal de **f([b])** no es una lista.

Para convertir el operador de mayor nivel de una lista a un conjunto véase **setify**.

### **identity** (*x*) Función

La función **identity** devuelve su argumento cualquiera que sea éste. Para determinar si todos los elementos de un conjunto son **true**, se puede hacer lo siguiente

```
(%i1) every (identity, [true, true]);
(%o1) true
```

### **integer\_partitions** (*n*) Función

### **integer\_partitions** (*n*, *len*) Función

Si el segundo argumento opcional *len* no se especifica, devuelve el conjunto de todas las particiones del entero *n*. Cuando se especifica *len*, devuelve todas las particiones

de longitud *len* o menor; en este caso, se añaden ceros a cada partición con menos de *len* términos, a fin de que cada partición tenga exactamente los *len* términos. En cualquier caso las particiones son listas ordenadas de mayor a menor.

Se dice que la lista  $[a_1, \dots, a_m]$  es una partición del entero no negativo  $n$  si (1) cada  $a_i$  es un entero no nulo y (2)  $a_1 + \dots + a_m = n$ . De donde se deduce que 0 carece de particiones.

```
(%i1) integer_partitions (3);
(%o1) {[1, 1, 1], [2, 1], [3]}
(%i2) s: integer_partitions (25)$
(%i3) cardinality (s);
(%o3) 1958
(%i4) map (lambda ([x], apply ("+", x)), s);
(%o4) {25}
(%i5) integer_partitions (5, 3);
(%o5) {[2, 2, 1], [3, 1, 1], [3, 2, 0], [4, 1, 0], [5, 0, 0]}
(%i6) integer_partitions (5, 2);
(%o6) {[3, 2], [4, 1], [5, 0]}
```

Para encontrar todas las particiones que satisfagan cierta condición, utilícese la función **subset**; he aquí un ejemplo que encuentra todas las particiones de 10 formadas por números primos.

```
(%i1) s: integer_partitions (10)$
(%i2) xprimep(x) := integerp(x) and (x > 1) and primep(x)$
(%i3) subset (s, lambda ([x], every (xprimep, x)));
(%o3) {[2, 2, 2, 2, 2], [3, 3, 2, 2], [5, 3, 2], [5, 5], [7, 3]}
```

**intersect** (*a\_1*, ..., *a\_n*)

Función

Es una forma abreviada de la función **intersection**.

**intersection** (*a\_1*, ..., *a\_n*)

Función

Devuelve el conjunto de todos los elementos que son comunes a los conjuntos *a\_1* a *a\_n*. La función **intersect** debe recibir uno o más argumentos. Muestra un mensaje de error en caso de que cualquiera de los *a\_i* no sea un conjunto. Véase también **intersection**.

**kron\_delta** (*x*, *y*)

Función

Es la función delta de Kronecker; la llamada **kron\_delta** (*x*, *y*) devuelve la unidad cuando **is**(*x* = *y*) es verdadero y cero cuando **sign** (*|x - y|*) vale **pos** (es decir, cuando es positivo). Si **sign** (*|x - y|*) es nulo y *x - y* no es un número en coma flotante (ni de doble precisión ni del tipo **bfloat**), devuelve 0. En otro caso devuelve una forma nominal.

La función **kron\_delta** es simétrica; así por ejemplo, **kron\_delta**(*x*, *y*) - **kron\_delta**(*y*, *x*) devuelve cero.

A continuación algunos ejemplos.

```
(%i1) [kron_delta (a, a), kron_delta (a + 1, a)];
(%o1) [1, 0]
(%i2) kron_delta (a, b);
```

```
(%o2) kron_delta(a, b)
```

Bajo la hipótesis  $a > b$  se obtiene que `sign (|a - b|)` sea `pos` (positivo),

```
(%i1) assume (a > b)$
(%i2) kron_delta (a, b);
(%o2) 0
```

Si se da por válida la hipótesis  $x \geq y$ , entonces `sign (|x - y|)` se vuelve `pz` (es decir, cero o positivo); en este caso, `kron_delta (x, y)` no devuelve resultado alguno,

```
(%i1) assume(x >= y)$
(%i2) kron_delta (x, y);
(%o2) kron_delta(x, y)
```

Por último, puesto que  $1/10 - 0.1$  devuelve un número en coma flotante, se tiene

```
(%i1) kron_delta (1/10, 0.1);
 1
(%o1) kron_delta(--, 0.1)
 10
```

Si se quiere que `kron_delta (1/10, 0.1)` devuelva la unidad, hágase uso de `float`.

```
(%i1) float (kron_delta (1/10, 0.1));
(%o1) 1
```

### **listify** (*a*)

Función

Si *a* es un conjunto, devuelve una lista con los elementos de *a*; si *a* no es un conjunto, devuelve *a*. Para convertir un conjunto y todos sus elementos a listas, véase `full_listify`.

### **lreduce** (*f*, *s*)

Función

### **lreduce** (*f*, *s*, *init*)

Función

La función `lreduce` (left reduce, o reducir por la izquierda) convierte una función binaria en *n*-aria por composición; un ejemplo hará esto más claro. Cuando no se define el argumento opcional *init* se tiene

```
(%i1) lreduce (f, [1, 2, 3]);
(%o1) f(f(1, 2), 3)
(%i2) lreduce (f, [1, 2, 3, 4]);
(%o2) f(f(f(1, 2), 3), 4)
```

Nótese que la función *f* se aplica en primer lugar a los elementos que están más a la izquierda (de ahí el nombre `lreduce`). Cuando se define *init*, se tiene

```
(%i1) lreduce (f, [1, 2, 3], 4);
(%o1) f(f(f(4, 1), 2), 3)
```

La función `lreduce` facilita calcular el producto o suma de los miembros de una lista.

```
(%i1) lreduce ("+", args (set (a, b)));
(%o1) b + a
(%i2) lreduce ("*", args (set (1, 2, 3, 4, 5)));
(%o2) 120
```

Véase también `rreduce`, `xreduce` y `tree_reduce`.

**makeset** (*e*, *v*, *s*)

Función

Esta función es similar a **makelist**, con la salvedad de que **makeset** permite sustituciones múltiples. El primer argumento *e* es una expresión, el segundo argumento *v* es una lista de variables y *s* es una lista o conjunto de valores para las variables *v*. Cada miembro de *s* debe tener la misma longitud que *v*. Así se tiene que **makeset** (*e*, *v*, *s*) es el conjunto  $\{z \mid z = \text{substitute}(v \rightarrow s\_i) \text{ and } s\_i \text{ in } s\}$ .

```
(%i1) makeset (i/j, [i, j], [[a, b], [c, d]]);
```

```
(%o1) a c
 {-, -}
 b d
```

```
(%i2) ind: set (0, 1, 2, 3)$
```

```
(%i3) makeset (i^2 + j^2 + k^2, [i, j, k], cartesian_product (ind, ind, ind));
```

```
(%o3) {0, 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12, 13, 14, 17, 18,
 19, 22, 27}
```

**moebius** (*n*)

Función

Es la función de Moebius; si *n* es el producto de *k* primos diferentes, **moebius**(*n*) devuelve  $(-1)^k$ , retornando 1 si *n* = 1 0 para cualesquiera otros enteros positivos. La función de Moebius se distribuye respecto de igualdades, listas, matrices y conjuntos.

**multinomial\_coeff** (*a\_1*, ..., *a\_n*)

Función

**multinomial\_coeff** ()

Función

Devuelve el coeficiente multinomial. Si todos los *a<sub>k</sub>* son enteros no negativos, el coeficiente multinomial es el número de formas de colocar *a<sub>1</sub>* + ... + *a<sub>n</sub>* objetos diferentes en *n* cajas con *a<sub>k</sub>* elementos en la *k*-ésima caja. En general, **multinomial** (*a<sub>1</sub>*, ..., *a<sub>n</sub>*) calcula  $(a_1 + \dots + a_n)! / (a_1! \dots a_n!)$ . Si no se dan argumentos, **multinomial**() devuelve 1. Se puede usar **minfactorial** para simplificar el valor devuelto por **multinomial\_coeff**; por ejemplo,

```
(%i1) multinomial_coeff (1, 2, x);
```

```
(%o1) (x + 3)!

 2 x!
```

```
(%i2) minfactorial (%);
```

```
(%o2) (x + 1) (x + 2) (x + 3)

 2
```

```
(%i3) multinomial_coeff (-6, 2);
```

```
(%o3) (- 4)!

 2 (- 6)!
```

```
(%i4) minfactorial (%);
```

```
(%o4) 10
```

**num\_distinct\_partitions** (*n*)

Función

**num\_distinct\_partitions** (*n*, *a*)

Función

Si *n* es un entero no negativo, devuelve el número de particiones enteras diferentes de *n*.

Si el argumento opcional *a* es `list`, devuelve a su vez una lista del número de particiones de 1,2,3, ... , *n*. Si *n* no es un entero no negativo, devuelve una forma nominal.

Definición: si  $n = k_1 + \dots + k_m$ , siendo los  $k_1$  a  $k_m$  enteros positivos distintos, se llama a  $k_1 + \dots + k_m$  una partición de *n*.

**num\_partitions** (*n*)

Función

**num\_partitions** (*n*, *a*)

Función

Si *n* es un entero no negativo, devuelve el número de particiones de *n*. Si el argumento opcional *a* es `list`, devuelve a su vez otra lista con los números de particiones de 1,2,3, ... , *n*. Si *n* no es un entero no negativo, devuelve una forma nominal.

```
(%i1) num_partitions (5) = cardinality (integer_partitions (5));
(%o1) 7 = 7
(%i2) num_partitions (8, list);
(%o2) [1, 1, 2, 3, 5, 7, 11, 15, 22]
(%i3) num_partitions (n);
(%o3) num_partitions(n)
```

Para un entero no negativo *n*, `num_partitions(n)` equivale a `cardinality(integer_partitions(n))`; sin embargo, `num_partitions` es más rápido.

**partition\_set** (*a*, *f*)

Función

Devuelve una lista con dos conjuntos; el primer conjunto es el subconjunto de *a* para el cual el predicado *f* devuelve `false` y el segundo es el subconjunto de *a* para el cual *f* devuelve `true`. Si *a* no es un conjunto, se envía un mensaje de error. Véase también `subset`.

```
(%i1) partition_set (set (2, 7, 1, 8, 2, 8), evenp);
(%o1) [{1, 7}, {2, 8}]
(%i2) partition_set (set (x, rat(y), rat(y) + z, 1), lambda ([x], ratp(x)));
(%o2)/R/ [{1, x}, {y, y + z}]
```

**permutations** (*a*)

Función

Devuelve un conjunto con todas las permutaciones distintas de los miembros de la lista o conjunto *a*. (Cada permutación es una lista, no un conjunto.) Si *a* es una lista, sus miembros duplicados no son eliminados antes de buscar sus permutaciones. Así,

```
(%i1) permutations ([a, a]);
(%o1) {[a, a]}
(%i2) permutations ([a, a, b]);
(%o2) {[a, a, b], [a, b, a], [b, a, a]}
```

Si *a* no es una lista o conjunto, se muestra un mensaje de error.

**powerset** (*a*)

Función

**powerset** (*a*, *n*)

Función

Si no se define el segundo argumento opcional *n*, devuelve el conjunto de todos los subconjuntos del conjunto *a*. La llamada `powerset(a)` tiene  $2^{\text{cardinality}(a)}$  miembros. Dado un segundo argumento, `powerset(a,n)` devuelve el conjunto de todos los subconjuntos de *a* que tienen cardinalidad *n*. Muestra un error si *a* no es un conjunto; muestra otro error si *n* no es un positivo entero.



**rreduce** (*f*, *s*)

Función

**rreduce** (*f*, *s*, *init*)

Función

La función **rreduce** (right reduce, reducir por la derecha) extiende una función binaria a *n*-aria por composición; un ejemplo hará esto más claro. Si el argumento opcional *init* no está definido, se tiene

```
(%i1) rreduce (f, [1, 2, 3]);
(%o1) f(1, f(2, 3))
(%i2) rreduce (f, [1, 2, 3, 4]);
(%o2) f(1, f(2, f(3, 4)))
```

Nótese que la función *f* se aplica en primer lugar al elemento que está más a la derecha en la lista, de ahí el nombre **rreduce**. Si *init* está definido,

```
(%i1) rreduce (f, [1, 2, 3], 4);
(%o1) f(1, f(2, f(3, 4)))
```

La función **rreduce** permite calcular fácilmente el producto o suma de los miembros de una lista.

```
(%i1) rreduce ("+", args (set (a, b)));
(%o1) b + a
(%i2) rreduce ("*", args (set (1, 2, 3, 4, 5)));
(%o2) 120
```

Véase también **lreduce**, **tree\_reduce** y **xreduce**.

**setdifference** (*a*, *b*)

Función

Devuelve in conjunto con los elementos del conjunto *a* que no pertenecen al conjunto *b*. Envía un mensaje de error si *a* o *b* no son conjuntos.

**setequalp** (*a*, *b*)

Función

Devuelve **true** si los conjuntos *a* y *b* tienen el mismo número de elementos y **is** (*x* = *y*) vale **true** para *x* perteneciente a *a* e *y* perteneciente a *b*, considerados en el orden que determina la función **listify**. En caso contrario, **setequalp** devuelve **false**.

La función **setequalp** muestra un mensaje de error si cualquiera de las expresiones *a* o *b* no es conjunto.

**setify** (*a*)

Función

Construye un conjunto con los miembros de la lista *a*. Los elementos duplicados de la lista *a* son borrados y lordenados de acuerdo con el predicado **orderlessp**. Envía un mensaje de error si *a* no es una lista.

**setp** (*a*)

Función

Devuelve **true** si y sólo si *a* es un conjunto de Maxima. La función **setp** chequea si el operador de su argumento es el de los conjuntos; no chequea si el argumento es un conjunto simplificado. Así,

```
(%i1) setp (set (a, a)), simp: false;
(%o1) true
```

La función **setp** podría haberse programado en Maxima como **setp(a) := is (inpart (a, 0) = set)**.

**set\_partitions** (*a*)

Función

**set\_partitions** (*a*, *n*)

Función

Si está definido el argumento opcional *n*, devuelve el conjunto de todas las descomposiciones de *a* en *n* subconjuntos disjuntos no vacíos. Si *n* no está definido, devuelve el conjunto de todas las particiones.

Se dice que un conjunto *P* es una partición del conjunto *S* si verifica

1. cada elemento de *P* es un conjunto no vacío,
2. los elementos de *P* son disjuntos,
3. la unión de elementos de *P* es igual a *S*.

El conjunto vacío forma una partición de sí mismo, así,

```
(%i1) set_partitions (set ());
(%o1) {{}}
```

El cardinal del conjunto de particiones de un conjunto puede calcularse con `stirling2`; así

```
(%i1) s: set (0, 1, 2, 3, 4, 5)$
(%i2) p: set_partitions (s, 3)$
(%o3) 90 = 90
(%i4) cardinality(p) = stirling2 (6, 3);
```

Cada elemento de *p* debería tener 3 miembros:

```
(%i1) s: set (0, 1, 2, 3, 4, 5)$
(%i2) p: set_partitions (s, 3)$
(%o3) {3}
(%i4) map (cardinality, p);
```

Por último, para cada elemento de *p*, la unión de sus miembros debe ser igual a *s*:

```
(%i1) s: set (0, 1, 2, 3, 4, 5)$
(%i2) p: set_partitions (s, 3)$
(%o3) {{0, 1, 2, 3, 4, 5}}
(%i4) map (lambda ([x], apply (union, listify (x))), p);
```

**some** (*f*, *a*)

Función

**some** (*f*, *L\_1*, ..., *L\_n*)

Función

El primer argumento de *f* debe ser un predicado (esto es, una función que devuelve `true`, `false`, or `unknown`).

Si el segundo argumento es un conjunto, **some** (*f*, *a*) devuelve `true` si *f*(*a\_i*) devuelve también `true` para alguno de los *a\_i* en *a*. Puesto que los conjuntos no están ordenados, **some** puede evaluar *f*(*a\_i*) en cualquier orden. La función **some** puede que no evalúe *f* para todos los *a\_i* de *a*. Debido a que el orden de evaluación no está especificado, el predicado *f* no debería tener efectos secundarios ni mostrar mensajes de error.

Dada una o más listas como argumentos, **some** (*f*, *L\_1*, ..., *L\_n*) devuelve `true` si *f*(*x\_1*, ..., *x\_n*) devuelve también `true` para al menos un *x\_1*, ..., *x\_n* de *L\_1*, ..., *L\_n*, respectivamente. La función **some** puede que no evalúe *f* para todas las combinaciones *x\_1*, ..., *x\_n*.

Si la variable global `maperror` vale `true` (su valor por defecto), todas las listas  $L_1$ , ...,  $L_n$  deben tener igual número de elementos, en caso contrario, `some` muestra un error. Si `maperror` vale `false`, los argumentos se truncan para tener todos el número de elementos de la lista más corta.

La función `is` de Maxima se aplica automáticamente tras la evaluación del predicado  $f$ .

```
(%i1) some("<", [a, b, 5], [1, 2, 8]);
(%o1) true
(%i2) some("=", [2, 3], [2, 7]);
(%o2) true
```

### **stirling1** ( $n, m$ )

Función

Es el número de Stirling de primera especie. Si tanto  $n$  como  $m$  son enteros no negativos, el valor que toma `stirling1` ( $n, m$ ) es el número de permutaciones de un conjunto de  $n$  elementos con  $m$  ciclos. Para más detalles, véase Graham, Knuth and Patashnik *Concrete Mathematics*. Se utiliza una relación recursiva para definir `stirling1` ( $n, m$ ) para  $m$  menor que 0; no está definida para  $n$  menor que 0 ni para argumentos no enteros.

La función `stirling1` opera por simplificación, utilizando valores básicos especiales (véase Donald Knuth, *The Art of Computer Programming*, tercera edición, Volumen 1, Sección 1.2.6, ecuaciones 48, 49 y 50). Para que Maxima aplique estas reglas, los argumentos deben declararse como enteros y el primer argumento debe ser no negativo. Por ejemplo:

```
(%i1) declare (n, integer)$
(%i2) assume (n >= 0)$
(%i3) stirling1 (n, n);
(%o3) 1
```

La función `stirling1` no se evalúa para argumentos no enteros.

```
(%i1) stirling1 (sqrt(2), sqrt(2));
(%o1) stirling1(sqrt(2), sqrt(2))
```

Maxima conoce algunos valores especiales,

```
(%i1) declare (n, integer)$
(%i2) assume (n >= 0)$
(%i3) stirling1 (n + 1, n);
(%o3)
$$\frac{n (n + 1)}{2}$$

(%i4) stirling1 (n + 1, 1);
(%o4)
$$n!$$

```

### **stirling2** ( $n, m$ )

Función

Es el número de Stirling de segunda especie. Si  $n$  y  $m$  son enteros no negativos, `stirling2` ( $n, m$ ) es el número de formas en las que se puede particionar un conjunto de cardinal  $n$  en  $m$  subconjuntos disjuntos. Se utiliza una relación recursiva para definir `stirling2` ( $n, m$ ) con  $m$  menor que 0; la función no está definida para  $n$  menor que 0 ni para argumentos no enteros.

La función `stirling1` opera por simplificación, utilizando valores básicos especiales (véase Donald Knuth, *The Art of Computer Programming*, tercera edición, Volumen 1, Sección 1.2.6, ecuaciones 48, 49 y 50). Para que Maxima aplique estas reglas, los argumentos deben declararse como enteros y el primer argumento debe ser no negativo. Por ejemplo:

```
(%i1) declare (n, integer)$
(%i2) assume (n >= 0)$
(%i3) stirling2 (n, n);
(%o3) 1
```

La función `stirling2` no se evalúa para argumentos no enteros.

```
(%i1) stirling2 (%pi, %pi);
(%o1) stirling2(%pi, %pi)
```

Maxima conoce algunos valores especiales,

```
(%i1) declare (n, integer)$
(%i2) assume (n >= 0)$
(%i3) stirling2 (n + 9, n + 8);
(%o3) (n + 8) (n + 9)

2

(%i4) stirling2 (n + 1, 2);
(%o4) n
2 - 1
```

**subset** (*a*, *f*) Función

Devuelve el subconjunto del conjunto *a* que satisface el predicado *f*. Por ejemplo:

```
(%i1) subset (set (1, 2, x, x + y, z, x + y + z), atom);
(%o1) {1, 2, x, z}
(%i2) subset (set (1, 2, 7, 8, 9, 14), evenp);
(%o2) {2, 8, 14}
```

El segundo argumento de `subset` debe ser un predicado (una función booleana con un argumento); si el primer argumento de `subset` no es un conjunto, envía un mensaje de error. Véase también `partition_set`.

**subsetp** (*a*, *b*) Función

Devuelve `true` si y sólo si el conjunto *a* es un subconjunto de *b*. Muestra un mensaje de error si *a* o *b* no son conjuntos.

**symmdifference** (*a*<sub>1</sub>, ..., *a*<sub>*n*</sub>) Función

Devuelve el conjunto de elementos que pertenecen a un único conjunto de los *a*<sub>*k*</sub>. Muestra un mensaje de error si cualquiera de los argumentos *a*<sub>*k*</sub> no es un conjunto. Con dos argumentos, `symmdifference` (*a*, *b*) equivale a `union (setdifference (a, b), setdifference (b, a))`.

**tree\_reduce** ( $f, s$ ) Función  
**tree\_reduce** ( $f, s, init$ ) Función

La función **tree\_reduce** amplía un operador binario asociativo  $f : S \times S \rightarrow S$  a otro de cualquier número de argumentos utilizando un árbol de mínima profundidad. Un ejemplo aclarará esto,

```
(%i1) tree_reduce (f, [a, b, c, d]);
(%o1) f(f(a, b), f(c, d))
```

Dado un número impar de argumentos, **tree\_reduce** favorecerá la parte izquierda del árbol; por ejemplo,

```
(%i1) tree_reduce (f, [a, b, c, d, e]);
(%o1) f(f(f(a, b), f(c, d)), e)
```

Para la suma de números en coma flotante, la utilización de **tree\_reduce** puede redundar en una disminución del error de redondeo frente a funciones como **rreduce** o **lreduce**.

**union** ( $a_1, \dots, a_n$ ) Function  
 Return the union of the sets  $a_1$  through  $a_n$ . When **union** receives no arguments, it returns the empty set. Signal an error when one or more arguments to **union** is not a set.

**xreduce** ( $f, s$ ) Función  
**xreduce** ( $f, s, init$ ) Función

Esta función es similar a **lreduce** y a **rreduce**, excepto que **xreduce** puede utilizar tanto la asociatividad por la izquierda como por la derecha; en particular, si  $f$  es una función asociativa y Maxima la tiene implementada, **xreduce** puede utilizar la función n-aria, como la suma **+**, la multiplicación **\***, **and**, **or**, **max**, **min** y **append**. Para estos operadores, se espera que en general que **xreduce** sea mas rápida que **rreduce** o **lreduce**. Si  $f$  no es n-aria, **xreduce** utiliza la asociatividad por la izquierda.

La suma de números en coma flotante no es asociativa; sin embargo, **xreduce** utiliza la suma n-aria de Maxima cuando el conjunto o lista  $s$  contiene números en coma flotante.

## 40 Definición de Funciones

### 40.1 Introducción a la definición de funciones

### 40.2 Funciones

Para definir una función en Maxima es necesario utilizar el operador `':='`.

Por ejemplo,

```
f(x) := sin(x)
```

define una función `f`. También se pueden definir funciones anónimas utilizando `lambda`; por ejemplo,

```
lambda ([i, j], ...)
```

puede utilizarse en lugar de `f` donde

```
f(i,j) := block ([], ...);
map (lambda ([i], i+1), 1)
```

devolvería una lista con todos sus elementos aumentados en una unidad.

También se puede definir una función con un número variable de argumentos, sin más que añadir un argumento final al que se le asigna una lista con todos los argumentos adicionales.:

```
(%i1) f ([u]) := u;
(%o1) f([u]) := u
(%i2) f (1, 2, 3, 4);
(%o2) [1, 2, 3, 4]
(%i3) f (a, b, [u]) := [a, b, u];
(%o3) f(a, b, [u]) := [a, b, u]
(%i4) f (1, 2, 3, 4, 5, 6);
(%o4) [1, 2, [3, 4, 5, 6]]
```

El miembro derecho de una función debe ser una expresión. Así, si se quiere una secuencia de expresiones, se debe hacer

```
f(x) := (expr1, expr2, ..., exprn);
```

siendo el valor que alcance `exprn` el devuelto por la función.

Si se quiere hacer un `return` desde alguna de las expresiones de la función, se debe utilizar la estructura `block` junto con `return`. Por ejemplo,

```
block ([], expr1, ..., if (a > 10) then return(a), ..., exprn)
```

es una expresión de pleno derecho, por lo que puede ocupar el lado derecho de la definición de una función. Aquí puede ocurrir que el retorno se produzca antes que se alcance la última expresión.

Los primeros corchetes del bloque `([])` pueden contener una lista de variables junto con posibles asignaciones, tal como `[a: 3, b, c: []]`, lo que provocará que las tres variables `a`, `b` y `c` se consideren locales y sean independientes de otras globales con el mismo nombre; las variables locales sólo estarán activas mientras se ejecute el código que está dentro de la estructura `block`, o dentro de funciones que son llamadas desde dentro de `block`. A esto se le llama asignación dinámica, pues las variables sobreviven desde el inicio del bloque hasta

que éste deje de estar operativo. Una vez se salga del bloque los valores originales de las variables, si es que los había, quedan restaurados. Es recomendable proteger las variables de esta forma. Se tendrá en cuenta que las asignaciones a las variables del bloque se hacen en paralelo, lo que significa que si como en el ejemplo anterior se hace `c: a` en el momento de entrar en el bloque, el valor de `c` será el que tenía `a` antes de entrar en el bloque, es decir, antes de la asignación `a: 3`. Así, haciendo lo siguiente

```
block ([a: a], expr1, ... a: a+3, ..., exprn)
```

se prevendría de que el valor externo de `a` fuese alterado, pero permitiría acceder a él desde dentro del bloque. La parte derecha de las asignaciones se evalúa dentro de su contexto antes de hacer efectiva la asignación. Utilizando únicamente `block([x], ..` haría que `x` se tuviese a sí misma como valor, justo como si se acabase de iniciar una nueva sesión de Maxima.

Los valores de los argumentos de una función se tratan exactamente de la misma forma que las variables de un bloque. Así, con

```
f(x) := (expr1, ..., exprn);
```

y

```
f(1);
```

se estaría en un contexto similar para la evaluación de las expresiones como si se hubiera hecho

```
block ([x: 1], expr1, ..., exprn)
```

Dentro de las funciones, cuando el lado derecho de la definición deba ser evaluado será útil hacer uso de `define` y posiblemente de `buildq`.

## 40.3 Macros

### **buildq** (*L*, *expr*)

Función

Sustituye en paralelo las variables nombradas en la lista *L* en la expresión *expr*, sin evaluar ésta. La expresión resultante se simplifica pero no se evalúa hasta que `buildq` termine de hacer las sustituciones.

Los elementos de *L* son símbolos o expresiones de asignación del tipo *symbol: value*, evaluadas en paralelo. Esto es, el valor de una variable en la parte derecha de una asignación es el valor que toma dicha variable en el contexto desde el que se invoca a `buildq`. En caso de que a una variable de *L* no se le haga una asignación explícita, su valor en `buildq` es el mismo que tiene en el contexto desde el que se llama a `buildq`.

Las variables referenciadas en *L* se sustituyen en *expr* en paralelo. Esto es, la sustitución para cada variable se determina antes de que se hagan las sustituciones, de forma que la sustitución de una variable no tiene efecto alguno sobre las otras.

Si alguna variable *x* aparece como `splice (x)` en *expr*, entonces a *x* se le debe asignar una lista, la cual será interpolada en *expr* en lugar de hacer una simple sustitución; ver ejemplo más abajo.

Cualesquiera otras variables de *expr* que no aparezcan en *L* se traspasan al resultado tal cual, incluso cuando tienen asignados valores en el contexto desde el que se llama a `buildq`.

Ejemplos:

**a** queda asociada explícitamente a **x**, mientras que **b** tiene la misma asociación (29) que en el contexto de llamada y **c** es traspasado al resultado sin ser sustituido. La expresión resultante no se evalúa hasta que no se le obligue a ello mediante la evaluación explícita `'%.`

```
(%i1) (a: 17, b: 29, c: 1729)$
(%i2) buildq ([a: x, b], a + b + c);
(%o2) x + c + 29
(%i3) '%;
(%o3) x + 1758
```

En este ejemplo, **e** se asocia a una lista, la cual aparece como tal en los argumentos de **foo** e interpolada en los argumentos de **bar**.

```
(%i1) buildq ([e: [a, b, c]], foo (x, e, y));
(%o1) foo(x, [a, b, c], y)
(%i2) buildq ([e: [a, b, c]], bar (x, splice (e), y));
(%o2) bar(x, a, b, c, y)
```

Como se ve a continuación, el resultado se simplifica tras las sustituciones. Si la simplificación se realizase antes que las sustituciones, ambos resultados serían iguales.

```
(%i1) buildq ([e: [a, b, c]], splice (e) + splice (e));
(%o1) 2 c + 2 b + 2 a
(%i2) buildq ([e: [a, b, c]], 2 * splice (e));
(%o2) 2 a b c
```

Las variables de *L* se asocian en paralelo; si se hiciese secuencialmente, el primer resultado sería **foo (b, b)**. Las sustituciones se llevan a cabo en paralelo. Compárese el segundo resultado con el resultado de **subst**, que hace las sustituciones de forma secuencial.

```
(%i1) buildq ([a: b, b: a], foo (a, b));
(%o1) foo(b, a)
(%i2) buildq ([u: v, v: w, w: x, x: y, y: z, z: u], bar (u, v, w, x, y, z));
(%o2) bar(v, w, x, y, z, u)
(%i3) subst ([u=v, v=w, w=x, x=y, y=z, z=u], bar (u, v, w, x, y, z));
(%o3) bar(u, u, u, u, u, u)
```

Se construye a continuación un sistema de ecuaciones con algunas variables o expresiones en el lado izquierdo y sus valores en el derecho; **macroexpand** muestra la expresión devuelta por **show\_values**.

```
(%i1) show_values ([L]) := buildq ([L], map ("=", 'L, L));
(%o1) show_values([L]) := buildq([L], map("=", 'L, L))
(%i2) (a: 17, b: 29, c: 1729)$
(%i3) show_values (a, b, c - a - b);
(%o3) [a = 17, b = 29, c = 1729]
```

### **macroexpand** (*expr*)

Función

Devuelve la macroexpansión de *expr*, sin evaluarla, cuando *expr* es una llamada a una función macro; en caso contrario, **macroexpand** devuelve *expr*.

Si la expansión de *expr* devuelve otra llamada a una función macro, esta llamada también se expande.



La función `macroexpand` no evalúa su argumento. Sin embargo, si la expansión de una llamada a función macro tiene efectos laterales, éstos se ejecutan.

Véanse también `::=`, `macros` y `macroexpand1`.

Ejemplos:

```
(%i1) g (x) ::= x / 99;
(%o1) g(x) ::=
$$\frac{x}{99}$$

(%i2) h (x) ::= buildq ([x], g (x - a));
(%o2) h(x) ::= buildq([x], g(x - a))
(%i3) a: 1234;
(%o3) 1234
(%i4) macroexpand (h (y));
(%o4)
$$\frac{y - a}{99}$$

(%i5) h (y);
(%o5)
$$\frac{y - 1234}{99}$$

```

### **macroexpand1** (expr)

Función

Devuelve la macroexpansión de `expr`, sin evaluarla, cuando `expr` es una llamada a una función macro; en caso contrario, `macroexpand1` devuelve `expr`.

La función `macroexpand1` no evalúa su argumento. Sin embargo, si la expansión de una llamada a función macro tiene efectos laterales, éstos se ejecutan.

Si la expansión de `expr` devuelve otra llamada a una función macro, esta llamada no se expande.

Véanse también `::=`, `macros` y `macroexpand`.

Ejemplos:

```
(%i1) g (x) ::= x / 99;
(%o1) g(x) ::=
$$\frac{x}{99}$$

(%i2) h (x) ::= buildq ([x], g (x - a));
(%o2) h(x) ::= buildq([x], g(x - a))
(%i3) a: 1234;
(%o3) 1234
(%i4) macroexpand1 (h (y));
(%o4) g(y - a)
(%i5) h (y);
(%o5)
$$\frac{y - 1234}{99}$$

```

### **macros**

Variable global

Valor por defecto: []

La variable `macros` es la lista de las funciones macro definidas por el usuario. El operador de definición de funciones macro `::=` coloca la nueva función macro en esta lista, mientras que `kill`, `remove` y `remfunction` eliminan las funciones macro de la lista.

Véase también `infolists`.

### `splice (a)`

Función

Interpola la lista nombrada por el átomo `a` dentro de una expresión, pero sólo si `splice` aparece dentro de `buildq`; en otro caso, `splice` se considera una función no definida. Si `a` aparece dentro de `buildq` sin `splice`, entonces queda sustituida por una lista dentro del resultado. El argumento de `splice` debe ser un átomo, no pudiendo ser una lista literal ni una expresión que devuelva una lista.

Normalmente `splice` suministra los argumentos para una función u operador. Para una función `f`, la expresión `f (splice (a))` dentro de `buildq` se convierte en `f (a[1], a[2], a[3], ...)`. Dado un operador `o`, la expresión `"o" (splice (a))` dentro de `buildq` se convierte en `"o" (a[1], a[2], a[3], ...)`, donde `o` puede ser cualquier tipo de operador, normalmente uno que admita varios argumentos. Nótese que el operador debe ir encerrado entre dobles comillas `"`.

Ejemplos:

```
(%i1) buildq ([x: [1, %pi, z - y]], foo (splice (x)) / length (x));
 foo(1, %pi, z - y)
(%o1) -----
 length([1, %pi, z - y])
(%i2) buildq ([x: [1, %pi]], "/" (splice (x)));
 1
(%o2) -----
 %pi
(%i3) matchfix ("<>", "<>");
(%o3) <>
(%i4) buildq ([x: [1, %pi, z - y]], "<>" (splice (x)));
(%o4) <>1, %pi, z - y<>
```

## 40.4 Definiciones para la definición de funciones

### `apply (f, [x_1, ..., x_n])`

Función

Devuelve el resultado de aplicar la función `f` a la lista de argumentos `x_1, ..., x_n`. El símbolo `f` debe ser el nombre de una función o de una expresión lambda.

Es útil en aquellos casos en los que se desea calcular los argumentos de una función antes de aplicárselos a ésta. Por ejemplo, si `l` es la lista `[1, 5, -10.2, 4, 3]`, entonces `apply (min, l)` devuelve `-10.2`. La función `apply` es útil también cuando se llama a funciones que aún no tienen sus argumentos evaluados y se requiere que sí lo estén. Por ejemplo, si `filespec` es una variable cuyo valor es la lista `[test, case]` entonces `apply (closefile, filespec)` es equivalente a `closefile (test, case)`. En general, el primer argumento de `apply` debería ir precedido de un apóstrofo (`'`) para que se evalúe a sí mismo. Puesto que algunas variables atómicas tienen el mismo nombre que ciertas funciones, el valor de la variable sería el utilizado antes que el de la función porque `apply` tiene su dos argumentos evaluados.

**block** (*[v<sub>1</sub>, ..., v<sub>m</sub>], expr<sub>1</sub>, ..., expr<sub>n</sub>*) Función  
**block** (*expr<sub>1</sub>, ..., expr<sub>n</sub>*) Función

La función **block** evalúa *expr<sub>1</sub>, ..., expr<sub>n</sub>* secuencialmente y devuelve el valor de la última expresión evaluada. La secuencia puede alterarse con las funciones **go**, **throw** y **return**. La última expresión es *expr<sub>n</sub>* a menos que **return** o una expresión que contenga un **throw** sea evaluada. Las variables *v<sub>1</sub>, ..., v<sub>m</sub>* son locales en el bloque; éstas se distinguen de las globales que tengan el mismo nombre. Si no se declaran variables locales entonces se puede omitir la lista. Dentro del bloque, cualquier otra variable distinta de *v<sub>1</sub>, ..., v<sub>m</sub>* se considera global.

La función **block** guarda los valores actuales de las variables *v<sub>1</sub>, ..., v<sub>m</sub>*, si los tienen, a la entrada del bloque y luego las evalúa a sí mismas, es decir les saca el valor temporalmente. A las variables locales se les puede asignar cualquier valor dentro del bloque, pero al salir de éste, los valores inicialmente almacenados quedan restaurados, al tiempo que los asignados dentro del bloque se pierden.

Un **block** puede aparecer dentro de otro **block**. Las variables locales se inicializan cada vez que se entra dentro de un nuevo bloque. Las variables locales de un bloque se consideran globales dentro de otro anidado dentro del primero. Si una variable es no local dentro de un bloque, su valor es el que le corresponde en el bloque superior. Este criterio se conoce con el nombre de "alcance dinámico".

Si se quieren guardar y restaurar otras propiedades locales además del valor de las variables, por ejemplo **arreglo** (excepto arreglos completos), **function**, **dependencies**, **atvalue**, **matchdeclare**, **atomgrad**, **constant** y **nonscalar** entonces debería usarse la función **local** dentro del bloque, siendo sus argumentos los nombres de las variables.

El valor del bloque es el de la última sentencia o el argumento de la función **return**, que puede utilizarse para salir del bloque. La función **go** puede usarse para transferir el control a la sentencia del bloque que esté etiquetada con el argumento de **go**. Para etiquetar una sentencia basta que vaya precedida de un argumento atómico como cualquier otra sentencia dentro del bloque. Por ejemplo, **block ([x], x:1, tururu, x: x+1, ..., go(tururu), ...)**. El argumento de **go** debe ser el nombre de una etiqueta colocada dentro del bloque. No se puede utilizar **go** para trasladarse a una etiqueta de un bloque que no sea el que contenga a **go**.

Normalmente los bloques aparecerán al lado derecho de las definiciones de funciones, pero también pueden utilizarse en otros contextos.

**break** (*expr<sub>1</sub>, ..., expr<sub>n</sub>*) Función

Calcula e imprime *expr<sub>1</sub>, ..., expr<sub>n</sub>* para luego provocar la detención de Maxima, de modo que el usuario pueda examinar y cambiar el entorno de ejecución. Pulsando posteriormente **exit**; el cálculo se reanuda.

**catch** (*expr<sub>1</sub>, ..., expr<sub>n</sub>*) Función

Evalúa *expr<sub>1</sub>, ..., expr<sub>n</sub>* una a una; si alguna de ellas conlleva la evaluación de una expresión de la forma **throw** (*arg*), entonces el valor de **catch** es el de **throw** (*arg*) y ya no se evalúan más expresiones. Esta respuesta pasa todos los niveles de anidamiento hasta el **catch** más próximo. Si no hay ningún **catch** que contenga un **throw** se emite un mensaje de error.

Si la evaluación de los argumentos no conlleva la evaluación de ningún `throw`, entonces el valor de `catch` es el devuelto por `expr_n`.

```
(%i1) lambda ([x], if x < 0 then throw(x) else f(x))$
(%i2) g(1) := catch (map ('%, 1))$
(%i3) g ([1, 2, 3, 7]);
(%o3) [f(1), f(2), f(3), f(7)]
(%i4) g ([1, 2, -3, 7]);
(%o4) - 3
```

La función `g` devuelve las imágenes por `f` de todos los elementos de la lista `l` si ésta contiene únicamente números no negativos; si no es este el caso, entonces `g` captura el primer negativo que encuentra y lo devuelve por medio del `throw`.

**compile** (*nombre\_fichero*, *f\_1*, ..., *f\_n*) Function

Traduce las funciones de Maxima *f\_1*, ..., *f\_n* a código Lisp y lo guarda en el fichero *nombre\_fichero*.

El código Lisp traducido no se evalúa, ni el fichero de salida es procesado por el compilador de Lisp. `translate` creates and evaluates Lisp translations. `compile_file` translates Maxima into Lisp, and then executes the Lisp compiler.

Véanse también `translate`, `translate_file` y `compile_file`.

**compile** (*f\_1*, ..., *f\_n*) Función

**compile** (*functions*) Función

**compile** (*all*) Función

Traduce las funciones de Maxima *f\_1*, ..., *f\_n* a Lisp, evaluando el código resultante, y llama a la función Lisp `COMPILE` para cada función traducida. La función `compile` devuelve una lista con los nombres de las funciones compiladas.

Las llamadas `compile (all)` o `compile (functions)` compilan todas las funciones definidas por el usuario.

La función `compile` no evalúa sus argumentos, pero con la colocación de dos comillas simple (``) sí lo hace.

**define** (*f(x\_1, ..., x\_n)*, *expr*) Función

Define una función de nombre *f* con argumentos *x\_1*, ..., *x\_n* y cuerpo *expr*.

`define` no evalúa su primer argumento en la mayor parte de los casos y evalúa su segundo argumento, a menos que vaya precedido de apóstrofo ('). Sin embargo, si el primer argumento es una expresión de la forma `ev (expr)`, `funmake (expr)` o `arraymake (expr)`, el primer argumento es evaluado, lo que permite calcular el nombre de la función junto con su cuerpo.

`define` es similar al operador de definición de funciones `:=`, pero cuando `define` aparece dentro de una función, la definición se crea usando el valor de `expr` en el momento de su ejecución, en lugar de la que tenía en el momento de la definición de la función que la contiene.

Ejemplos:

```

(%i1) foo: 2^bar;

(%o1)
2
(%i2) g(x) := (f_1 (y) := foo*x*y,
 f_2 (y) := 'foo*x*y,
 define (f_3 (y), foo*x*y),
 define (f_4 (y), 'foo*x*y));

(%o2) g(x) := (f_1(y) := foo x y, f_2(y) := 2^bar x y,
 define(f_3(y), foo x y), define(f_4(y), 2^bar x y))

(%i3) functions;
(%o3) [g(x)]
(%i4) g(a);

(%o4) f_4(y) := a 2^bar y
(%i5) functions;
(%o5) [g(x), f_1(y), f_2(y), f_3(y), f_4(y)]
(%i6) dispfun (f_1, f_2, f_3, f_4);
(%t6) f_1(y) := foo x y

(%t7) f_2(y) := 2^bar x y

(%t8) f_3(y) := a 2^bar y

(%t9) f_4(y) := a 2^bar y

(%o9) done

```

**define\_variable** (*nombre*, *valor\_por\_defecto*, *modo*)

Función

Introduce una variable global en el entorno de Maxima.

La función `define_variable` sigue los siguientes pasos:

1. `mode_declare (nombre, modo)` declara el modo de *nombre* al traductor. Véase en `mode_declare` un listado de los posibles modos.
2. Si la variable no tiene asignado un valor, *valor\_por\_defecto* se le asigna a *nombre*.
3. `declare (nombre, special)` la declara como especial.
4. Asocia *nombre* con una función test para asegurar que a *name* sólo se le asignan valores del modo declarado.

**dispfun** (*f\_1*, ..., *f\_n*)

Función

**dispfun** (*all*)

Función

Muestra la definición de las funciones de usuario *f\_1*, ..., *f\_n*. Cada argumento puede ser el nombre de una macro (definida mediante `:=`), una función ordinaria (definida mediante `:=` o `define`), una función arreglo (definida mediante `:=` o `define`, pero encerrando los argumentos dentro de corchetes `[ ]`), una función de subíndice (definida

mediante `:=` o `define`, pero encerrando algunos argumentos entre corchetes y otros entre paréntesis ( `)`), una función de subíndice seleccionada por un subíndice variable, o una función de subíndice definida con un subíndice constante.

La llamada `dispfun (all)` muestra todas las funciones de usuario tal como las dan las listas `functions`, `arrays` y `macros`, omitiendo las funciones con subíndices definidas con subíndices constantes.

La función `dispfun` crea una etiqueta (`%t1`, `%t2`, etc.) para cada función mostrada, y asigna la definición de la función a la etiqueta. En contraste, `fundef` devuelve las definiciones de las funciones.

La función `dispfun` no evalúa sus argumentos; el operador de doble comilla simple `' '` permite la evaluación.

La función `dispfun` devuelve la lista de etiquetas de expresiones intermedias correspondientes a las funciones mostradas.

Ejemplos:

```
(%i1) m(x, y) ::= x^(-y);

(%o1) m(x, y) ::= x- y
(%i2) f(x, y) := x^(-y);

(%o2) f(x, y) := x- y
(%i3) g[x, y] := x^(-y);

(%o3) gx, y := x- y
(%i4) h[x](y) := x^(-y);

(%o4) hx(y) := x- y
(%i5) i[8](y) := 8^(-y);

(%o5) i8(y) := 8- y
(%i6) dispfun (m, f, g, h, h[5], h[10], i[8]);

(%t6) m(x, y) ::= x- y

(%t7) f(x, y) := x- y

(%t8) gx, y := x- y

(%t9) hx(y) := x- y
```

```

(%t10) h (y) := --
 5 y
 5

(%t11) h (y) := ---
 10 y
 10

(%t12) i (y) := 8
 8

(%o12) [%t6, %t7, %t8, %t9, %t10, %t11, %t12]
(%i12) ',,%;

(%o12) [m(x, y) ::= x-y, f(x, y) := x-y, gx, y := x-y,
 h (y) := x-y, h (y) := --, h (y) := ---, i (y) := 8-y]
 5 5 10 y 8
 5 5 10 y 8

```

## functions

Variable del sistema

Valor por defecto: []

La variable **functions** es una lista que contiene las funciones definidas por el usuario en la sesión actual. Una función definida por el usuario es aquella que ha sido construida mediante cualquiera de los métodos **define** o **:=**. Una función puede definirse durante una sesión de Maxima o en un fichero que posteriormente será cargado en memoria por **load** o **batch**. Téngase en cuenta que las funciones Lisp no se añaden a la lista **functions**.

## fundef (f)

Función

Devuelve la definición de la función *f*.

Cada argumento puede ser el nombre de una macro (definida mediante **:=**), una función ordinaria (definida mediante **:=** o **define**), una función arreglo (definida mediante **:=** o **define**, pero encerrando los argumentos dentro de corchetes **[ ]**), una función de subíndice (definida mediante **:=** o **define**, pero encerrando algunos argumentos entre corchetes y otros entre paréntesis **( )**), una función de subíndice seleccionada por un subíndice variable, o una función de subíndice definida con un subíndice constante.

La función **fundef** no evalúa sus argumentos; el operador de doble comilla simple **'** permite la evaluación.

La llamada de función **fundef (f)** devuelve la definición de *f*. Por el contrario, **dispfun (f)** crea una etiqueta intermedia y le asigna la definición a la etiqueta.

**funmake** (*nombre*, [*arg\_1*, ..., *arg\_n*])

Función

Devuelve una expresión *name* (*arg\_1*, ..., *arg\_n*). El valor así retornado es simplificado pero no evaluado, de forma que la función no es invocada.

La función **funmake** evalúa sus argumentos.

Ejemplos:

- **funmake** evalúa sus argumentos, pero no el valor retornado.

```
(%i1) det(a,b,c) := b^2 -4*a*c$
(%i2) x: 8$
(%i3) y: 10$
(%i4) z: 12$
(%i5) f: det$
(%i6) funmake (f, [x, y, z]);
(%o6) det(8, 10, 12)
(%i7) ' '%;
(%o7) - 284
```

- Maxima simplifica el valor retornado por **funmake**.

```
(%i1) funmake (sin, [%pi/2]);
(%o1) 1
```

**lambda** (*[x\_1*, ..., *x\_m*], *expr\_1*, ..., *expr\_n*)

Función

**lambda** (*[[L]]*, *expr\_1*, ..., *expr\_n*)

Function

**lambda** (*[x\_1*, ..., *x\_m*, *[L]]*, *expr\_1*, ..., *expr\_n*)

Function

Define y devuelve una expresión **lambda** (es decir, una función anónima). La función puede tener argumentos *x\_1*, ..., *x\_m* y/o argumentos opcionales *L*, que aparecerán dentro del cuerpo de la función como una lista. El valor que devuelve la función es *expr\_n*. Una expresión **lambda** puede asignarse a una variable y ser evaluada como si fuese una función ordinaria. Además, puede aparecer en algunos contextos en los que sea necesario un nombre de función.

Cuando se evalúa la función, se crean las variables *x\_1*, ..., *x\_m* sin asignación de valores. Una función **lambda** puede aparecer dentro de un **block** o de otra **lambda**. Las variables locales se inicializan cada vez que se entra dentro de un nuevo bloque o de otra función **lambda**. Las variables locales se consideran globales dentro de un bloque o función **lambda** anidado dentro del primero. Si una variable es no local dentro de un bloque o función **lambda**, su valor es el que le corresponde en el bloque o función **lambda** superior. Este criterio se conoce con el nombre de "alcance dinámico".

Una vez establecidas las variables locales *expr\_1* a *expr\_n* son secuencialmente evaluadas. La variable especial **%%** representa el valor de la expresión inmediata anterior. Las sentencias **throw** y **catch** pueden aparecer también en la lista de expresiones.

La función **return** no puede aparecer en una expresión **lambda** a menos que se encuentre acotada dentro de un bloque (**block**), en cuyo caso **return** establece el valor de retorno del bloque, pero no de la expresión **lambda**, a menos que el bloque resulte ser precisamente *expr\_n*. De igual manera, **go** no puede aparecer en una expresión **lambda** si no es dentro de un **block**.

Las funciones **lambda** no evalúan sus argumentos; el operador de doble comilla simple **' '** permite su evaluación.



Ejemplo:

- Una función lambda puede asignarse a una variable y ser evaluada como si fuese una función ordinaria.

```
(%i1) f: lambda ([x], x^2);
(%o1) 2
 lambda([x], x)
(%i2) f(a);
(%o2) 2
 a
```

- Una expresión lambda puede aparecer en algunos contextos en los que sea necesario un nombre de función.

```
(%i3) lambda ([x], x^2) (a);
(%o3) 2
 a
(%i4) apply (lambda ([x], x^2), [a]);
(%o4) 2
 a
(%i5) map (lambda ([x], x^2), [a, b, c, d, e]);
(%o5) 2 2 2 2 2
 [a , b , c , d , e]
```

- Los argumentos son variables locales. Otras variables se consideran globales. Las variables globales son evaluadas en el momento que lo es la expresión, a menos que la evaluación de las mismas sea forzada, como cuando se hace uso de ''.

```
(%i6) a: %pi$
(%i7) b: %e$
(%i8) g: lambda ([a], a*b);
(%o8) lambda([a], a b)
(%i9) b: %gamma$
(%i10) g(1/2);
(%o10) %gamma

 2
(%i11) g2: lambda ([a], a*''b);
(%o11) lambda([a], a %gamma)
(%i12) b: %e$
(%i13) g2(1/2);
(%o13) %gamma

 2
```

- Las expresiones lambda pueden anidarse. Las variables locales de expresiones lambda exteriores se consideran globales en expresiones internas, a menos que se enmascaren con variables locales de igual nombre.

```
(%i14) h: lambda ([a, b], h2: lambda ([a], a*b), h2(1/2));
(%o14) 1
 lambda([a, b], h2 : lambda([a], a b), h2(-))
 2
(%i15) h(%pi, %gamma);
```

```
(%o15) %gamma

 2
```

- Puesto que `lambda` no evalúa sus argumentos, la expresión `lambda i` de más abajo no define una función del tipo "multiplicar por `a`". Tal tipo de función se puede definir a través de `buildq`, como en la expresión `lambda i2` de más abajo.

```
(%i16) i: lambda ([a], lambda ([x], a*x));
(%o16) lambda([a], lambda([x], a x))
(%i17) i(1/2);
(%o17) lambda([x], a x)
(%i18) i2: lambda([a], buildq([a: a], lambda([x], a*x)));
(%o18) lambda([a], buildq([a : a], lambda([x], a x)))
(%i19) i2(1/2);
```

```
(%o19) x
 lambda([x], -)
 2
```

```
(%i20) i2(1/2)(%pi);
```

```
(%o20) %pi

 2
```

- Una expresión `lambda` puede tener un número variable de argumentos, los cuales se indican mediante `[L]`, bien sea solo o como un último argumento. Estos argumentos aparecerán dentro del cuerpo de la función en forma de lista.

```
(%i1) f : lambda ([aa, bb, [cc]], aa * cc + bb);
(%o1) lambda([aa, bb, [cc]], aa cc + bb)
(%i2) f (foo, %i, 17, 29, 256);
(%o2) [17 foo + %i, 29 foo + %i, 256 foo + %i]
(%i3) g : lambda ([[aa]], apply ("+", aa));
(%o3) lambda([[aa]], apply(+, aa))
(%i4) g (17, 29, x, y, z, %e);
(%o4) z + y + x + %e + 46
```

### **local** (*v<sub>1</sub>*, ..., *v<sub>n</sub>*)

Función

Declara las variables *v<sub>1</sub>*, ..., *v<sub>n</sub>* locales respecto de todas las propiedades en la sentencia en la que aparece esta función.

La función `local` no evalúa sus argumentos y devuelve `done`.

La función `local` sólo puede usarse dentro de un `block`, en el cuerpo de definición de funciones o de expresiones `lambda` o en la función `ev`, siéndole permitido aparecer una sólo vez en cada una de ellas.

La función `local` es independiente de `context`.

### **macroexpansion**

Variable opcional

Valor por defecto: `false`

La variable `macroexpansion` controla si la expansión (esto es, el valor de retorno) de una función macro se sustituye por la llamada a la función macro. Una sustitución puede acelerar futuras evaluaciones de la expresión, bajo el coste que implica tener que almacenar la expansión.

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>false</b>    | La expansión de una función macro no se sustituye por la llamada a la función macro.                                                                                                                                                                                                                                                                                                                                                                  |
| <b>expand</b>   | La primera vez que se evalúa una llamada a función macro se almacena la expansión. De esta manera la expansión no se recalcula en llamadas posteriores; cualesquiera efectos laterales (como <b>print</b> o asignaciones a variables globales) tan solo tienen lugar la primera vez que la función macro es evaluada. La expansión en una expresión no afecta a otras expresiones que llamen a la misma función macro.                                |
| <b>displace</b> | La primera vez que se evalúa una llamada a una función macro, la expansión se sustituye por la llamada, modificando así la expresión desde la que se hizo la llamada a la función macro. La expansión no se recalcula en llamadas posteriores; cualesquiera efectos laterales tan solo tienen lugar la primera vez que la función macro es evaluada. La expansión en una expresión no afecta a otras expresiones que llamen a la misma función macro. |

Ejemplos:

Si **macroexpansion** vale **false**, una función macro es llamada cada vez que la expresión de llamada es evaluada.

```
(%i1) f (x) := h (x) / g (x);
(%o1)
 h(x)
f(x) := ----
 g(x)
(%i2) g (x) ::= block (print ("x + 99 is equal to", x), return (x + 99));
(%o2) g(x) ::= block(print("x + 99 is equal to", x),
 return(x + 99))
(%i3) h (x) ::= block (print ("x - 99 is equal to", x), return (x - 99));
(%o3) h(x) ::= block(print("x - 99 is equal to", x),
 return(x - 99))
(%i4) macroexpansion: false;
(%o4)
false
(%i5) f (a * b);
x - 99 is equal to x
x + 99 is equal to x
(%o5)
a b - 99

a b + 99
(%i6) dispfun (f);
(%t6)
 h(x)
f(x) := ----
 g(x)
(%o6)
done
(%i7) f (a * b);
x - 99 is equal to x
x + 99 is equal to x
(%o7)
a b - 99

```

$$a \ b + 99$$

Si `macroexpansion` vale `expand`, una función macro tan solo es llamada una vez.

```
(%i1) f (x) := h (x) / g (x);
(%o1)
 h(x)
f(x) := ----
 g(x)
(%i2) g (x) ::= block (print ("x + 99 is equal to", x), return (x + 99));
(%o2) g(x) ::= block(print("x + 99 is equal to", x),
 return(x + 99))
(%i3) h (x) ::= block (print ("x - 99 is equal to", x), return (x - 99));
(%o3) h(x) ::= block(print("x - 99 is equal to", x),
 return(x - 99))

(%i4) macroexpansion: expand;
(%o4)
expand
(%i5) f (a * b);
x - 99 is equal to x
x + 99 is equal to x
 a b - 99
(%o5)

 a b + 99
(%i6) dispfun (f);
 h(x)
(%t6)
f(x) := ----
 g(x)

(%o6)
done
(%i7) f (a * b);
 a b - 99
(%o7)

 a b + 99
```

Si `macroexpansion` vale `expand`, una función macro es llamada una vez y la expresión de llamada se modifica.

```
(%i1) f (x) := h (x) / g (x);
(%o1)
 h(x)
f(x) := ----
 g(x)
(%i2) g (x) ::= block (print ("x + 99 is equal to", x), return (x + 99));
(%o2) g(x) ::= block(print("x + 99 is equal to", x),
 return(x + 99))
(%i3) h (x) ::= block (print ("x - 99 is equal to", x), return (x - 99));
(%o3) h(x) ::= block(print("x - 99 is equal to", x),
 return(x - 99))

(%i4) macroexpansion: displace;
(%o4)
displace
(%i5) f (a * b);
x - 99 is equal to x
x + 99 is equal to x
 a b - 99
```

```

(%o5) -----
 a b + 99

(%i6) dispfun (f);
 x - 99
(%t6) f(x) := -----
 x + 99

(%o6) done
(%i7) f (a * b);
 a b - 99
(%o7) -----
 a b + 99

```

**mode\_checkp**

Variable opcional

Valor por defecto: `true`

Cuando `mode_checkp` vale `true`, `mode_declare` chequea los modos de las variables con valores asignados.

**mode\_check\_errorp**

Variable opcional

Valor por defecto: `false`

Cuando `mode_check_errorp` vale `true`, `mode_declare` llama a error.

**mode\_check\_warnp**

Variable opcional

Valor por defecto: `true`

Cuando `mode_check_warnp` vale `true`, se detallan los errores de modo.

**mode\_declare** (*y<sub>1</sub>, modo<sub>1</sub>, ..., y<sub>n</sub>, modo<sub>n</sub>*)

Función

La función `mode_declare` se utiliza para declarar los modos de variables y funciones para la ulterior traducción a Lisp o compilación de funciones. Se coloca habitualmente al comienzo de la definición de una función, de un script en Maxima o se ejecuta en tiempo real.

Los argumentos de `mode_declare` son pares formados por una variable y un modo, el cual debe ser `boolean`, `fixnum`, `number`, `rational` o `float`. Cada variable puede ser sustituida por una lista de variables, en cuyo caso todas ellas tendrán el mismo modo.

Código numérico que utilice arreglos puede ejecutarse más rápido declarando el tamaño que va a ocupar el arreglo, como en:

```
mode_declare (array (a [10, 10]), float)
```

para un arreglo de números en coma flotante de dimensiones 10 x 10.

Se puede declarar el modo del resultado de una función poniendo `function (f1, f2, ...)` como argumento; aquí `f1`, `f2`, ... son los nombres de las funciones. Por ejemplo, la expresión

```
mode_declare ([function (f1, f2, ...)], fixnum)
```

declara que el valor a devolver por `f1`, `f2`, ... son enteros de modo "single-word".

El nombre `modeddeclare` es sinónimo de `mode_declare`.

**mode\_identity** (*arg\_1*, *arg\_2*)

Función

Es una forma especial usada con `mode_declare` y `macros` para declarar, por ejemplo, una lista de listas de números.

**transcompile**

Variable opcional

Valor por defecto: `true`

Si `transcompile` vale `true`, `translate` y `translate_file` generan declaraciones para hacer el código traducido más apto para la compilación.

La función `compile` hace la asignación `transcompile: true`.

**translate** (*f\_1*, ..., *f\_n*)

Función

**translate** (*functions*)

Función

**translate** (*all*)

Función

Traduce las funciones definidas por el usuario *f\_1*, ..., *f\_n* del lenguaje de Maxima a Lisp y evalúa las traducciones Lisp. Normalmente las funciones traducidas se ejecutan más rápidamente que las originales.

Las llamadas `translate (all)` o `translate (functions)` traducen todas las funciones de usuario.

Las funciones a ser traducidas deberían incluir una llamada a `mode_declare` al comienzo siempre que sea posible, a fin de producir código más eficiente. Por ejemplo:

```
f (x_1, x_2, ...) := block ([v_1, v_2, ...],
 mode_declare (v_1, modo_1, v_2, modo_2, ...), ...)
```

donde *x\_1*, *x\_2*, ... son los parámetros que se pasan a la función y *v\_1*, *v\_2*, ... son las variables locales.

Los nombres de las funciones traducidas son eliminados de la lista `functions` si `savedef` vale `false` (ver más abajo) y son añadidos a las listas `props`.

Las funciones no deberían ser traducidas hasta no estar completamente depuradas.

Se supone que las expresiones están simplificadas; en caso de no estarlo, se generará código correcto pero ineficiente. Así, el usuario no debería asignar a `simp` el valor `false`, el cual inhibe la simplificación de la expresión a ser traducida.

Cuando la variable `translate` vale `true`, se traducen automáticamente las funciones de usuario a Lisp.

Nótese que las funciones traducidas puede que no se ejecuten exactamente igual a como lo hacían antes de la traducción, debido a posibles incompatibilidades entre las versiones de Maxima y Lisp. En general, la función `rat` con más de un argumento y la función `ratvars` no deberían utilizarse si algunas de las variables son declaradas como expresiones racionales canónicas (CRE) mediante `mode_declare`. Además, la asignación `prederror: false` no traducirá.

Si `savedef` vale `true`, entonces la versión de Maxima de una función de usuario permanecerá cuando la función sea traducida por `translate`. Con esto se hace posible que se muestre la definición llamando a `dispfun` y que la función sea editada.

Si `transrun` vale `false` entonces las versiones interpretadas de todas las funciones serán ejecutadas en lugar de las versiones traducidas.

El resultado devuelto por `translate` es una lista con los nombres de las funciones traducidas.

|                                                                                     |         |
|-------------------------------------------------------------------------------------|---------|
| <b>translate_file</b> ( <i>nombre_fichero_maxima</i> )                              | Función |
| <b>translate_file</b> ( <i>nombre_fichero_maxima</i> , <i>nombre_fichero_lisp</i> ) | Función |

Traduce un fichero en código Maxima a un fichero en código Lisp. La función **translate\_file** devuelve una lista con los nombres de tres ficheros: el nombre del fichero en Maxima, el nombre del fichero en Lisp y el nombre del fichero que contiene información adicional sobre la traducción. La función **translate\_file** evalúa sus argumentos.

La llamada **translate\_file** ("foo.mac"); **load**("foo.LISP") es lo mismo que **batch** ("foo.mac"), excepto por la presencia de ciertas restricciones, como el uso de ' ' y %, por ejemplo.

La llamada **translate\_file** (*nombre\_fichero\_maxima*) traduce un fichero en Maxima, *nombre\_fichero\_maxima*, a otro en Lisp de nombre similar. Por ejemplo, **foo.mac** se traduce en **foo.LISP**. El nombre del fichero en Maxima puede incluir el nombre de un directorio, en cuyo caso el fichero de salida Lisp se guardará en el mismo directorio desde el que se leyó la fuente Maxima.

La llamada **translate\_file** (*nombre\_fichero\_maxima*, *nombre\_fichero\_lisp*) traduce el fichero Maxima *nombre\_fichero\_maxima* en el fichero Lisp *nombre\_fichero\_lisp*. La función **translate\_file** ignora la extensión del fichero, en caso de que exista, de *nombre\_fichero\_lisp*; la extensión del fichero de salida Lisp será invariablemente LISP. El nombre del fichero Lisp puede incluir la ruta del directorio, en cuyo caso se almacenará en el directorio especificado.

La función **translate\_file** también escribe un fichero de mensajes de avisos del traductor con diversos niveles de gravedad. La extensión de este fichero es UNLISP. Este fichero puede contener información valiosa, aunque de difícil interpretación, para detectar fallos en el código traducido. El fichero UNLISP se guarda siempre en el mismo directorio desde el que se leyó la fuente de Maxima.

La función **translate\_file** emite código Lisp que incluye algunas declaraciones y definiciones que entran en efecto tan pronto como el código Lisp es compilado. Véase **compile\_file** para más información sobre este particular.

Véanse también **tr\_array\_as\_ref**, **tr\_bound\_function\_apply**, **tr\_exponent**, **tr\_file\_tty\_messagesp**, **tr\_float\_can\_branch\_complex**, **tr\_function\_call\_default**, **tr\_numer**, **tr\_optimize\_max\_loop**, **tr\_semicompile**, **tr\_state\_vars**, **tr\_warnings\_get**, **tr\_warn\_bad\_function\_calls**, **tr\_warn\_fexpr**, **tr\_warn\_meval**, **tr\_warn\_mode**, **tr\_warn\_undeclared**, **tr\_warn\_undefined\_variable**, y **tr\_windy**.

|                 |                   |
|-----------------|-------------------|
| <b>transrun</b> | Variable opcional |
|-----------------|-------------------|

Valor por defecto: **true**

Si **transrun** vale **false** entonces se ejecutarán las versiones interpretadas de todas las funciones, en lugar de las versiones traducidas.

|                        |                   |
|------------------------|-------------------|
| <b>tr_array_as_ref</b> | Variable opcional |
|------------------------|-------------------|

Valor por defecto: **true**

Si **translate\_fast\_arrays** vale **false**, referencias de arreglos en el código Lisp creadas por **translate\_file** se ven afectadas por **tr\_array\_as\_ref**.

El valor de la variable `tr_array_as_ref` no tiene ningún efecto cuando `translate_fast_arrays` vale `true`.

### **tr\_bound\_function\_applyp**

Variable opcional

Valor por defecto: `true`

Si `tr_bound_function_applyp` vale `true`, Maxima envía un aviso si encuentra una variable con valor asignado que está siendo utilizada como una función. `tr_bound_function_applyp` no influye en el código generado bajo estas circunstancias.

Por ejemplo, una expresión como `g (f, x) := f (x+1)` provocará un mensaje de esta naturaleza.

### **tr\_file\_tty\_messagesp**

Variable opcional

Valor por defecto: `false`

Si `tr_file_tty_messagesp` vale `true`, los mensajes generados por `translate_file` durante la traducción de un fichero se muestran en la consola y se insertan en el fichero UNLISP. Si vale `false`, los mensajes sobre la traducción del fichero sólo se incorporan al fichero UNLISP.

### **tr\_float\_can\_branch\_complex**

Variable opcional

Valor por defecto: `true`

Le dice al traductor de Maxima a Lisp que las funciones `acos`, `asin`, `asec` y `acsc` pueden devolver valores complejos.

### **tr\_function\_call\_default**

Variable opcional

Valor por defecto: `general`

El valor `false` significa llama a `meval`, `expr` significa que Lisp asignó los argumentos de la función, `general`, el valor por defecto, devuelve código apropiado para `mexprs` y `mlexprs` pero no para `macros`. La opción `general` asegura que las asignaciones de las variables son correctas en el código compilado. En modo `general`, cuando se traduce `F(X)`, si `F` es una variable con valor, entonces se entiende que se quiere calcular `apply (f, [x])`, y como tal se traduce, con el apropiado aviso. No es necesario desactivar esto. Con los valores por defecto la falta de mensajes de aviso implica compatibilidad completa entre el código traducido y compilado con el interpretado por Maxima.

### **tr\_numer**

Variable opcional

Valor por defecto: `false`

Si `tr_numer` vale `true` se utilizan las propiedades numéricas en aquellos átomos que las posean, como en `%pi`.

### **tr\_optimize\_max\_loop**

Variable opcional

Valor por defecto: 100

El valor de `tr_optimize_max_loop` es el número máximo de veces que el traductor repetirá la macro-expansión y la optimización en el tratamiento de una expresión.



- tr\_semicompile** Variable opcional  
 Valor por defecto: `false`  
 Si `tr_semicompile` vale `true`, las salidas de `translate_file` y `compile` serán macro-expandidas pero no compiladas a código máquina por el compilador de Lisp.
- tr\_state\_vars** Variable del sistema  
 Valor por defecto:  
`[transcompile, tr_semicompile, tr_warn_undeclared, tr_warn_meval, tr_warn_fexpr, tr_warn_mode, tr_warn_undefined_variable, tr_function_call_default, tr_array_as_ref, tr_numer]`  
 Es la lista de variables que afectan la forma en que se obtiene la salida del código traducido. Esta información es útil para desarrolladores que pretendan corregir posibles fallos del traductor. Comparando el código traducido con el que se debería obtener bajo unas ciertas condiciones, es posible hacer el seguimiento de los fallos.
- tr\_warnings\_get ()** Función  
 Devuelve una lista con los avisos dados por el traductor.
- tr\_warn\_bad\_function\_calls** Variable opcional  
 Valor por defecto: `true`  
 Devuelve un aviso cuando se hacen llamadas a funciones que quizás no sean correctas debido a declaraciones inapropiadas realizadas durante la traducción.
- tr\_warn\_fexpr** Variable opcional  
 Valor por defecto: `compile`  
 Devuelve un aviso si se encuentra con alguna FEXPR. Las FEXPR no deberían aparecer en el código traducido.
- tr\_warn\_meval** Variable opcional  
 Valor por defecto: `compile`  
 Devuelve un aviso si la función `meval` es llamada. Si `meval` es invocada, es señal de la presencia de problemas en la traducción.
- tr\_warn\_mode** Variable opcional  
 Valor por defecto: `all`  
 Devuelve un aviso cuando a las variables se les asignan valores incompatibles con su modo.
- tr\_warn\_undeclared** Variable opcional  
 Valor por defecto: `compile`  
 Determina cuando enviar mensajes sobre variables no declaradas.
- tr\_warn\_undefined\_variable** Variable opcional  
 Valor por defecto: `all`  
 Devuelve un aviso cuando se detectan variables globales no definidas.

**tr\_windy**

Variable opcional

Valor por defecto: `true`

Genera comentarios de ayuda y consejos sobre programación.

**compile\_file** (*nombre\_fich*)

Función

**compile\_file** (*nombre\_fich*, *nombre\_fich\_compilado*)

Función

**compile\_file** (*nombre\_fich*, *nombre\_fich\_compilado*, *nombre\_fich\_lisp*)

Función

Traduce el fichero Maxima *nombre\_fich* a Lisp, ejecuta el compilador de Lisp y, en caso de ser exitosa la compilación, carga el código compilado en Maxima.

La función `compile_file` devuelve una lista con los nombres de tres ficheros: el fichero original en Maxima, la traducción Lisp, notas sobre la traducción y el código compilado. Si la compilación falla, el cuarto elemento es `false`.

Algunas declaraciones y definiciones entran en efecto tan pronto como el código Lisp es compilado (sin cargar el código compilado). Éstas incluyen funciones definidas con el operador `:=`, macros definidas con el operador `::=`, `alias`, `declare`, `define_variable`, `mode_declare` y `infix`, `matchfix`, `nofix`, `postfix`, `prefix` y `compile`.

Asignaciones y llamadas a funciones no se evalúan hasta que el código compilado es cargado. En particular, dentro del fichero Maxima, asignaciones a los controles ("flags") de traducción (`tr_numer`, etc.) no tienen efecto durante la traducción.

El *nombre\_fich* no puede contener sentencias del tipo `:lisp`.

La función `compile_file` evalúa sus argumentos.

**declare\_translated** (*f1*, *f2*, ...)

Función

Cuando se traduce un fichero de código Maxima a Lisp, es importante para el traductor saber qué funciones de las que están en el fichero van a ser llamadas como traducidas o compiladas, y cuáles son simplemente funciones Maxima o que no están definidas. Se genera el código (MFUNCTION-CALL *fn* *arg1* *arg2* ...) cuando el traductor no sabe si *fn* va a ser una función lisp.



## 41 Programación

### 41.1 Introducción a la Programación

Maxima dispone de los bucles `do` para hacer iteraciones, así como estructuras más primitivas del estilo de `go`.

### 41.2 Definiciones para la Programación

**backtrace** ()

Función

**backtrace** (*n*)

Función

Devuelve la pila de llamadas, esto es, la lista de funciones que han llamado a la función actualmente activa.

La llamada a `backtrace()` devuelve la pila completa de llamadas.

Ejemplos:

```
(%i1) h(x) := g(x/7)$
(%i2) g(x) := f(x-11)$
(%i3) f(x) := e(x^2)$
(%i4) e(x) := (backtrace(), 2*x + 13)$
(%i5) h(10);
#0: e(x=4489/49)
#1: f(x=-67/7)
#2: g(x=10/7)
#3: h(x=10)

(%o5)
9615

49
```

La llamada `backtrace (n)` devuelve las *n* funciones más recientes, incluyendo a la función actualmente activa.

Ejemplos:

```
(%i1) h(x) := (backtrace(1), g(x/7))$
(%i2) g(x) := (backtrace(1), f(x-11))$
(%i3) f(x) := (backtrace(1), e(x^2))$
(%i4) e(x) := (backtrace(1), 2*x + 13)$
(%i5) h(10);
#0: h(x=10)
#0: g(x=10/7)
#0: f(x=-67/7)
#0: e(x=4489/49)

(%o5)
9615

49
```

**do**

Operador especial

La sentencia `do` se utiliza para realizar iteraciones. Debido a su generalidad la sentencia `do` se describirá en dos partes. En primer lugar se mostrará su forma más usual,

análoga a la de otros lenguajes de programación (Fortran, Algol, PL/I, etc.); después se mencionarán otras formas de uso.

Hay tres variantes de esta sentencia que se diferencian entre sí únicamente por las condiciones de fin de bucle. Son las siguientes:

- *for variable: valor\_inicial step incremento thru límite do cuerpo*
- *for variable: valor\_inicial step incremento while condición do cuerpo*
- *for variable: valor\_inicial step incremento unless condición do cuerpo*

El *valor\_inicial*, el *incremento*, el *límite* y el *cuerpo* pueden ser cualquier tipo de expresión válida de Maxima. Si el incremento es igual a la unidad (1) entonces "*step 1*" puede omitirse.

La ejecución de la sentencia **do** se realiza asignando el *valor\_inicial* a la variable (llamada de aquí en adelante *variable-control*). A continuación: (1) si la *variable-control* ha excedido el límite de la especificación dada por un **thru**, o si la condición impuesta por **unless** es verdadera (**true**), o si la condición dada por **while** es falsa (**false**) entonces la iteración **do** termina. (2) El cuerpo se evalúa. (3) El incremento es sumado a la *variable-control*. El proceso de (1) a (3) se repite hasta que la condición de fin de iteración se satisfaga. También es posible especificar varias condiciones de terminación del bucle, en cuyo caso **do** terminará cuando se satisfaga alguna de ellas.

En general la condición **thru** se satisfará cuando la *variable-control* sea mayor que el límite si el incremento es no negativo, o cuando la *variable-control* sea menor que el límite cuando el incremento es negativo. El incremento y el límite pueden ser expresiones no numéricas, tanto en cuanto esta desigualdad pueda quedar determinada. Sin embargo, a menos que el incremento sea un número negativo en el momento de comenzar el cómputo de **do**, Maxima supondrá que se evaluará a una cantidad positiva. En caso de no ser efectivamente positivo, la sentencia **do** puede dar un resultado inesperado.

Nótese que el límite, el incremento y la condición de terminación se evalúan en cada iteración del bucle. Así, si alguna de expresiones necesitan de muchos cálculos y devuelven un resultado que no va a cambiar durante toda la ejecución del cuerpo, será más eficiente dar este valor a una variable antes de comenzar la sentencia **do** y utilizarla luego durante su ejecución.

El valor que habitualmente devuelva la sentencia **do** será el átomo **done**. Sin embargo, la función **return** puede usarse dentro del cuerpo para salir de **do** de forma prematura retornando un valor determinado. Nótese no obstante que un **return** dentro de un **do** que está dentro de un bloque (**block**) provocará una salida de **do** pero no de **block**. Repárese también en que la función **go** no puede usarse para salir de **do** e ir a algún lugar de **block**.

La *variable-control* es siempre local respecto de **do**, por lo que se puede utilizar cualquier nombre de variable sin afectar el valor de cualquier otra variable externa a **do** y que tenga el mismo nombre. La *variable-control* no tendrá asignado ningún valor una vez se haya concluido el **do**.

```
(%i1) for a:-3 thru 26 step 7 do display(a)$
 a = - 3
```

```

a = 4

a = 11

a = 18

a = 25

(%i1) s: 0$
(%i2) for i: 1 while i <= 10 do s: s+i;
(%o2) done
(%i3) s;
(%o3) 55

```

Nótese que la condición `while i <= 10` es equivalente a `unless i > 10` y a `thru 10`.

```

(%i1) series: 1$
(%i2) term: exp (sin (x))$
(%i3) for p: 1 unless p > 7 do
 (term: diff (term, x)/p,
 series: series + subst (x=0, term)*x^p)$
(%i4) series;

```

$$\frac{x^7}{90} - \frac{x^6}{240} - \frac{x^5}{15} - \frac{x^4}{8} + \frac{x^2}{2} + x + 1$$

lo que da ocho términos del desarrollo de Taylor de la función  $e^{\sin(x)}$ .

```

(%i1) poly: 0$
(%i2) for i: 1 thru 5 do
 for j: i step -1 thru 1 do
 poly: poly + i*x^j$
(%i3) poly;

```

$$5x^5 + 9x^4 + 12x^3 + 14x^2 + 15x$$

```

(%o3) 5 x + 9 x + 12 x + 14 x + 15 x
(%i4) guess: -3.0$
(%i5) for i: 1 thru 10 do
 (guess: subst (guess, x, 0.5*(x + 10/x)),
 if abs (guess^2 - 10) < 0.00005 then return (guess));
(%o5) - 3.162280701754386

```

Este ejemplo calcula la raíz cuadrada negativa de 10 haciendo 10 iteraciones del método de Newton-Raphson. De no haberse alcanzado el criterio de convergencia el valor devuelto hubiese sido `done`.

En lugar de añadir siempre una cantidad a la variable-control a veces se puede querer que cambie en cada iteración siguiendo algún otro criterio. En tal caso se puede hacer uso de `next expresión` en lugar de `step incremento`. Esto hará que a la variable-control se le asigne el resultado de evaluar la expresión en cada iteración del bucle.

```

(%i6) for count: 2 next 3*count thru 20 do display (count)$
count = 2

count = 6

```

```
count = 18
```

En ocasiones puede interesar realizar una iteración en la que la variable-control no se utilice nunca. Se podrá entonces dar únicamente las condiciones de terminación del bucle omitiendo la inicialización y actualizando la información, tal como se hace en el siguiente ejemplo para calcular la raíz cuadrada de 5 utilizando un valor inicial alejado de la solución.

```
(%i1) x: 1000$
(%i2) thru 20 do x: 0.5*(x + 5.0/x)$
(%i3) x;
(%o3) 2.23606797749979
(%i4) sqrt(5), numer;
(%o4) 2.23606797749979
```

Si así se quiere, incluso es posible omitir las condiciones de terminación completamente y escribir únicamente `do body`, lo que provocará entrar en un bucle infinito. En tal caso, debería usarse la función `return` a fin de terminar con la ejecución de `do`.

```
(%i1) newton (f, x):= ([y, df, dfx], df: diff (f ('x), 'x),
do (y: ev(df), x: x - f(x)/y,
if abs (f (x)) < 5e-6 then return (x)))$
(%i2) sqr (x) := x^2 - 5.0$
(%i3) newton (sqr, 1000);
(%o3) 2.236068027062195
```

(En este ejemplo, cuando se ejecuta `return` obliga a que sea `x` el valor devuelto por `do`. Al salirse del bloque, `x` es también el valor que devuelve `block` por ser `do` la última sentencia del bloque.)

Hay todavía otra forma de `do` en Maxima. Su sintaxis es:

```
for variable in lista test_de_parada do cuerpo
```

Los elementos de *list* son cualesquiera expresiones que se irán asignando sucesivamente a la variable en cada repetición del cuerpo. El test de parada *end\_tests* (que es opcional) puede usarse para terminar la ejecución de `do`; de otro modo las iteraciones se pararán cuando la lista se haya agotado o cuando se ejecute un `return` dentro del cuerpo. (De hecho, la lista puede ser cualquier expresión no atómica, de la cual se irán extrayendo de forma sucesiva sus diferentes partes.)

```
(%i1) for f in [log, rho, atan] do ldisp(f(1))$
(%t1) 0
(%t2) rho(1)
(%t3) %pi
(%t3) ---
(%t3) 4
(%i4) ev(%t3,numer);
(%o4) 0.78539816
```

**errcatch** (*expr\_1*, ..., *expr\_n*)

Función

Evalúa las expresiones *expr\_1*, ..., *expr\_n* una a una y devuelve [*expr\_n*] (una lista) en caso de que no ocurra ningún error. En caso de aparecer algún error durante

el cálculo de alguno de los argumentos, `errcatch` evita que el error se propague y devuelve la lista vacía `[]` sin evaluar más argumentos.

La función `errcatch` es útil en ficheros `batch` donde se sospeche que pueda aparecer algún error, el cual provocaría la terminación de la ejecución del `batch` de no ser previamente detectado.

**error** (*expr\_1*, ..., *expr\_n*)

Función

**error**

Variable del sistema

Calcula y devuelve *expr\_1*, ..., *expr\_n*, enviando posteriormente una señal de error a Maxima o al `errcatch` más cercano.

A la variable `error` se le asigna una lista con la descripción del error. El primer elemento de `error` es una cadena de formato, la cual une todas las cadenas de los argumentos *expr\_1*, ..., *expr\_n*, siendo los demás elementos de la lista los valores de los argumentos que no son cadenas.

La llamada a `errormsg()` formatea e imprime `error`. Se reimprime así el mensaje de error más reciente.

**errormsg** ()

Función

Reimprime el mensaje de error más reciente. La variable `error` guarda el mensaje y `errormsg` lo formatea e imprime.

**for**

Operador especial

Utilizado en las iteraciones. Véase `do` para una descripción de las técnicas de iteración en Maxima.

**go** (*etiqueta*)

Función

Se utiliza dentro de un bloque (`block`) para transferir el control a la sentencia del bloque que esté etiquetada con el argumento de `go`. Una sentencia queda etiquetada cuando está precedida por un argumento de tipo átomo como cualquier otra sentencia de `block`. Por ejemplo:

```
block ([x], x:1, tururu, x+1, ..., go(tururu), ...)
```

El argumento de `go` debe ser el nombre de una etiqueta que aparezca en el mismo bloque (`block`). No se puede utilizar `go` para transferir el control a un bloque que no sea aquel que contenga la sentencia `go`.

**if**

Operador especial

La sentencia `if` se utiliza para la ejecución condicionada. Su sintaxis es:

```
if <condición> then <expr_1> else <expr_2>
```

El resultado de una sentencia `if` es *expr\_1* si la condición es verdadera (`true`) y *expr\_2* si es falsa. Tanto *expr\_1* como *expr\_2* son expresiones correctas en Maxima (incluyendo sentencias `if` anidadas) y *condition* es una expresión que puede ser verdadera (`true`) o falsa (`false`), formada con los operadores relacionales y lógicos siguientes:



| Operación             | Símbolo  | Tipo                       |
|-----------------------|----------|----------------------------|
| menor que             | <        | operador relacional infijo |
| menor o igual que     | <=       | operador relacional infijo |
| igualdad (sintáctica) | =        | operador relacional infijo |
| negación de =         | #        | operador relacional infijo |
| igualdad (por valor)  | equal    | operador relacional infijo |
| negación de equal     | notequal | operador relacional infijo |
| mayor o igual que     | >=       | operador relacional infijo |
| mayor que             | >        | operador relacional infijo |
| y                     | and      | operador lógico infijo     |
| o                     | or       | operador lógico infijo     |
| no                    | not      | operador lógico prefijo    |

**map** (*f*, *expr\_1*, ..., *expr\_n*) Función

Devuelve una expresión cuyo operador principal es el mismo que aparece en las expresiones *expr\_1*, ..., *expr\_n* pero cuyas subpartes son los resultados de aplicar *f* a cada una de las subpartes de las expresiones; *f* puede ser tanto el nombre de una función de *n* argumentos o una forma **lambda** de *n* argumentos.

Uno de los usos que tiene **map** es la de aplicar (o mapear) una función (por ejemplo, **partfrac**) sobre cada término de una expresión extensa en la que normalmente no se podría utilizar la función debido a insuficiencias en el espacio de almacenamiento durante el curso de un cálculo.

```
(%i1) map(f,x+a*y+b*z);
(%o1) f(b z) + f(a y) + f(x)
(%i2) map(lambda([u],partfrac(u,x)),x+1/(x^3+4*x^2+5*x+2));
(%o2) 1 1 1
 ----- - ----- + ----- + x
 x + 2 x + 1 2
 (x + 1)
(%i3) map(ratsimp, x/(x^2+x)+(y^2+y)/y);
(%o3) 1
 y + ----- + 1
 x + 1
(%i4) map("=", [a,b], [-0.5,3]);
(%o4) [a = - 0.5, b = 3]
```

**mapatom** (*expr*) Función

Devuelve **true** si y sólo *expr* es tratado por las rutinas de mapeo como un átomo.

**maperror** Variable optativa

Valor por defecto: **true**

Cuando **maperror** toma el valor **false**, hace que todas las funciones de mapeo, como por ejemplo

`map (f, expr_1, expr_2, ...)`

(1) paren cuando hayan terminado de procesar la expi más corta, a menos que todas ellas sean del mismo tamaño y (2) apliquen `f` a `[expr_1, expr_2, ...]` si es el caso que las `expr_i` no son todas del mismo tipo de objeto.

Cuando `maperror` toma el valor `true` entonces se emite un mensaje de error cuando se presenta cualquiera de los dos casos anteriores.

**maplist** (*f*, *expr\_1*, ..., *expr\_n*)

Función

Devuelve una lista con las aplicaciones de *f* a las partes de las expresiones *expr\_1*, ..., *expr\_n*; *f* es el nombre de una función ou una expresión lambda.

La función `maplist` difiere de `map (f, expr_1, ..., expr_n)`, la cual devuelve una expresión con el mismo operador principal que tenga *expr\_i*, excepto en simplificaciones y en el caso en el que `map` hace un `apply`.

**prederror**

Variable optativa

Valor por defecto: `true`

Cuando `prederror` toma el valor `true`, se emite un mensaje de error siempre que el predicado de una sentencia `if` o de una función `is` no se pueda evaluar ni a verdadero (`true`) ni a falso (`false`).

Si toma el valor `false`, se devuelve bajo las mismas circunstancias anteriores el valor `unknown`. El modo `prederror: false` no está soportado en el código traducido; sin embargo, `maybe` está soportado en código traducido.

Véanse también `is` y `maybe`.

**return** (*valor*)

Función

Puede utilizarse para salir de un bloque, devolviendo su. Véase `block` para más información.

**scanmap** (*f*, *expr*)

Función

**scanmap** (*f*, *expr*, *bottomup*)

Función

Aplica recursivamente *f* sobre *expr*, de arriba hacia abajo. Esto es más útil cuando se busca una factorización completa, por ejemplo:

```
(%i1) exp: (a^2+2*a+1)*y + x^2$
```

```
(%i2) scanmap(factor,exp);
```

```
(%o2) (a + 1)^2 y + x^2
```

Nótese que cómo `scanmap` aplica la función dada `factor` a las subexpresiones que forman a *expr*; si se presenta otra forma de *expr* a `scanmap` entonces el resultado puede ser diferente. Así, `%o2` no se restaura cuando `scanmap` se aplica a la forma expandida de *exp*:

```
(%i3) scanmap(factor,expand(exp));
```

```
(%o3) a^2 y + 2 a y + y^2 + x^2
```

Aquí hay otro ejemplo de la forma en que `scanmap` aplica recursivamente una función dada a todas las subexpresiones, incluyendo exponentes:

```
(%i4) expr : u*v^(a*x+b) + c$
(%i5) scanmap('f, expr);
 f(f(f(a) f(x)) + f(b))
(%o5) f(f(f(u) f(f(v))) + f(c))
```

`scanmap (f, expr, bottomup)` aplica  $f$  a  $expr$  de abajo hacia arriba. Por ejemplo, para  $f$  no definida,

```
scanmap(f,a*x+b) ->
 f(a*x+b) -> f(f(a*x)+f(b)) -> f(f(f(a)*f(x))+f(b))
scanmap(f,a*x+b,bottomup) -> f(a)*f(x)+f(b)
 -> f(f(a)*f(x))+f(b) ->
 f(f(f(a)*f(x))+f(b))
```

En este caso se obtiene la misma respuesta por cualquiera de los dos métodos.

**throw** (*expr*) Función

Evalúa *expr* y devuelve el valor del `catch` más reciente. La función `throw` se utiliza junto con `catch` como un mecanismo de retorno no local.

**outermap** (*f, a\_1, ..., a\_n*) Función

Aplica la función  $f$  a cada uno de los elementos del producto vectorial  $a_1$  por  $a_2$  ... por  $a_n$ .

La función  $f$  tiene  $n$  argumentos o también puede tratarse de una función lambda de  $n$  argumentos. Los argumentos  $a_1, \dots, a_n$  pueden ser listas o no. Argumentos en forma de listas pueden tener longitudes diferentes. Argumentos que no son listas son tratados como listas de longitud 1 con el propósito de poder realizar el producto vectorial.

El resultado de aplicar  $f$  al producto vectorial se organiza como una lista anidada. La profundidad del anidamiento es igual al número de argumentos en forma de lista (los argumentos que no son listas no contribuyen a sumar niveles). Una lista con nivel de anidamiento  $k$  tiene la misma longitud que el  $k$ -ésimo argumento con forma de lista.

La función `outermap` evalúa sus argumentos.

Véanse también `map`, `maplist` y `apply`.

Ejemplos:

```
(%i1) f (x, y) := x - y$
(%i2) outermap (f, [2, 3, 5], [a, b, c, d]);
(%o2) [[2 - a, 2 - b, 2 - c, 2 - d],
 [3 - a, 3 - b, 3 - c, 3 - d], [5 - a, 5 - b, 5 - c, 5 - d]]
(%i3) outermap (lambda ([x, y], y/x), [55, 99], [Z, W]);
 Z W Z W
(%o3) [[--, --], [--, --]]
 55 55 99 99
(%i4) g: lambda ([x, y, z], x + y*z)$
(%i5) outermap (g, [a, b, c], %pi, [11, 17]);
(%o5) [[a + 11 %pi, a + 17 %pi], [b + 11 %pi, b + 17 %pi],
 [c + 11 %pi, c + 17 %pi]]
(%i6) flatten (%);
(%o6) [a + 11 %pi, a + 17 %pi, b + 11 %pi, b + 17 %pi,
 c + 11 %pi, c + 17 %pi]
```

## 42 Depurado

### 42.1 Depuración del código fuente

Maxima es capaz de dar asistencia en la depuración del código fuente. Un usuario puede establecer un punto de referencia dentro del código de una función a partir del cual se siga la ejecución línea a línea. La compliación puede ser posteriormente examinada, conjuntamente con los valores que se han ido asignando a las variables.

La instrucción `:help`, o `:h`, muestra la lista de comandos para la depuración. (En general, los comandos pueden abreviarse; en algunos casos la lista de alternativas podrá ser listada.) Dentro del depurador, el usuario podrá examinar también cualquier función propia de Maxima, definirla y manipular variables y expresiones.

El punto de referencia se establecerá con la instrucción `:br`. Ya dentro del depurador, el usuario podrá avanzar una línea de cada vez utilizando la instrucción `:n` (de “next”, en inglés). La orden `:bt` (de “backtrace”) muestra la lista de la pila. Finalmente, con el comando `:r` (“resume”) se abandona el depurador continuando con la ejecución. El uso de estas instrucciones se muestra en el siguiente ejemplo.

```
(%i1) load ("/tmp/foobar.mac");

(%o1) /tmp/foobar.mac

(%i2) :br foo
Turning on debugging debugmode(true)
Bkpt 0 for foo (in /tmp/foobar.mac line 1)

(%i2) bar (2,3);
Bkpt 0:(foobar.mac 1)
/tmp/foobar.mac:1::

(dbm:1) :bt <-- pulsando :bt se retrocede
#0: foo(y=5)(foobar.mac line 1)
#1: bar(x=2,y=3)(foobar.mac line 9)

(dbm:1) :n <-- pulsando :n se avanza una línea
(foobar.mac 2)
/tmp/foobar.mac:2::

(dbm:1) :n <-- pulsando :n se avanza otra línea
(foobar.mac 3)
/tmp/foobar.mac:3::

(dbm:1) u; <-- se pide el valor de u
28

(dbm:1) u: 33; <-- se cambia el valor de u a 33
33
```

```
(dbm:1) :r <-- pulsando :r se termina la depuración
```

```
(%o2) 1094
```

El fichero `/tmp/foobar.mac` contiene lo siguiente:

```
foo(y) := block ([u:y^2],
 u: u+3,
 u: u^2,
 u);
```

```
bar(x,y) := (
 x: x+2,
 y: y+2,
 x: foo(y),
 x+y);
```

## USO DEL DEPURADOR EN EMACS

Si el usuario está corriendo el código bajo GNU emacs en un entorno de texto (`dbl shell`), o está ejecutando el entorno gráfico `xmaxima`, entonces cuando una función pare en el punto de referencia, podrá observar su posición actual en el archivo fuente, el cual será mostrado en la otra mitad de la ventana, bien resaltada en rojo, o con una pequeña flecha apuntando a la línea correcta. El usuario puede avanzar líneas simples tecleando `M-n` (`Alt-n`).

Bajo Emacs se debe ejecutar el programa en una ventana de texto `dbl`, la cual requiere el archivo `dbl.el` que está en el directorio `elisp`. El usuario debe instalar los archivos `elisp` o agregar el directorio `elisp` de Maxima a la ruta de búsqueda: por ejemplo, se puede añadir lo siguiente al archivo `‘.emacs’` o al `site-init.el`

```
(setq load-path (cons "/usr/share/maxima/5.9.1/emacs" load-path))
(autoload 'dbl "dbl")
```

entonces en emacs

```
M-x dbl
```

debería abrir una ventana del sistema en la cual se pueden ejecutar programas, por ejemplo Maxima, `gcl`, `gdb`, etc. En esta ventana también se puede ejecutar el depurador, mostrando el código fuente en la otra ventana.

El usuario puede colocar un punto de referencia en una línea determinada sin más que teclear `C-x space`. Con esto se le hace saber al depurador en qué función está el cursor y en qué línea del mismo. Si el cursor está en la línea 2 de `foo`, entonces insertará en la otra ventana la instrucción `“:br foo 2”`, a fin de detener `foo` justo en la segunda línea. Para tener esto operativo, el usuario debe tener activo `maxima-mode.el` (`modo-maxima.el`) en la ventana en la que está `foobar.mac`. Hay otros comandos disponibles en la ventana, como evaluar la función dentro de Maxima tecleando `Alt-Control-x`.

## 42.2 Claves de depuración

Las claves de depuración son palabras que no son interpretadas como expresiones de Maxima. Una clave de depuración puede introducirse dentro de Maxima o del depurador. Las claves de depuración comienzan con dos puntos, `‘:’`. Por ejemplo, para evaluar una expresión Lisp, se puede teclear `:lisp` seguido de la expresión a ser evaluada.

```
(%i1) :lisp (+ 2 3)
5
```

El número de argumentos depende del comando en particular. Además, tampoco es necesario teclear el nombre completo de la instrucción, tan solo lo justo para diferenciarla de las otras instrucciones. Así, `:br` sería suficiente para `:break`.

Las claves de depuración se listan a continuación.

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>:break F n</code>            | Establece un punto de referencia en la función <code>F</code> en la línea <code>n</code> contando a partir del comienzo de la función. Si <code>F</code> es una cadena, entonces se entiende que se trata de un fichero, siendo entonces <code>n</code> el número de línea a partir del comienzo del fichero. El valor <code>n</code> es opcional; en caso de no ser suministrado, se entenderá que vale cero (primera línea de la función o fichero). |
| <code>:bt</code>                   | Retrocede en la pila.                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <code>:continue</code>             | Continúa el cómputo de la función.                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <code>:delete</code>               | Borra los puntos de referencia especificados, o todos si no se especifica ninguno.                                                                                                                                                                                                                                                                                                                                                                     |
| <code>:disable</code>              | Deshabilita los puntos de referencia especificados, o todos si no se especifica ninguno.                                                                                                                                                                                                                                                                                                                                                               |
| <code>:enable</code>               | Habilita los puntos de referencia especificados, o todos si no se especifica ninguno.                                                                                                                                                                                                                                                                                                                                                                  |
| <code>:frame n</code>              | Imprime el elemento <code>n</code> de la pila, o el actualmente activo si no se especifica ninguno.                                                                                                                                                                                                                                                                                                                                                    |
| <code>:help</code>                 | Imprime la ayuda sobre un comando del depurador, o de todos los comandos si no se especifica ninguno.                                                                                                                                                                                                                                                                                                                                                  |
| <code>:info</code>                 | Imprime información sobre un elemento.                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>:lisp expresión</code>       | Evalúa la <code>expresión</code> Lisp.                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>:lisp-quiet expresión</code> | Evalúa la <code>expresión</code> Lisp sin devolver el resultado.                                                                                                                                                                                                                                                                                                                                                                                       |
| <code>:next</code>                 | Como <code>:step</code> , excepto que <code>:next</code> se salta las llamadas a funciones.                                                                                                                                                                                                                                                                                                                                                            |
| <code>:quit</code>                 | Sale del nivel actual del depurador sin completar el cómputo.                                                                                                                                                                                                                                                                                                                                                                                          |
| <code>:resume</code>               | Continúa con el cómputo.                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <code>:step</code>                 | Sigue con el cómputo de la función o fichero hasta que alcance una nueva línea fuente.                                                                                                                                                                                                                                                                                                                                                                 |
| <code>:top</code>                  | Retorna a Maxima desde cualquier nivel del depurador sin completar el cómputo.                                                                                                                                                                                                                                                                                                                                                                         |

## 42.3 Definiciones para Depurado

### **refcheck**

Variable opcional

Valor por defecto: **false**

Cuando **refcheck** vale **true**, Maxima imprime un mensaje cada vez que una variable es utilizada por vez primera en un cálculo.

### **setcheck**

Variable opcional

Valor por defecto: **false**

Cuando el valor de **setcheck** es una lista de variables (se admite que tengan subíndices) Maxima devuelve un mensaje indicando si los valores que han sido asignados a las variables lo han sido con el operador ordinario **:**, o con el operador de asignación **::** o como resultado de haberse realizado una llamada de función, pero en ningún caso cuando la asignación haya sido hecha mediante los operadores **:=** o **::=**. El mensaje contiene el nombre de la variable y su valor.

La variable **setcheck** admite también los valores **all** o **true** con lo que el informe incluirá todas las variables.

Cada nueva asignación de **setcheck** establece una nueva lista de variables a ser monitorizada, de forma que cualquier otra variable previamente asignada a **setcheck** es olvidada.

Los nombres asignados a **setcheck** deben estar precedidos del apóstrofo **'** a fin de evitar que las variables sean evaluadas antes de ser almacenadas en **setcheck**. Por ejemplo, si **x**, **y** y **z** ya guardan algún valor entonces se hará

```
setcheck: ['x, 'y, 'z]$
```

para colocarlas en la lista de variables a monitorizar.

No se generará ninguna salida cuando una variable de la lista **setcheck** sea asignada a ella misma, como en **X: 'X**.

### **setcheckbreak**

Variable opcional

Valor por defecto: **false**

Si **setcheckbreak** es igual **true**, Maxima se detendrá siempre que a una variable de la lista **setcheck** se le asigne un nuevo valor. La detención tendrá lugar justo antes de hacerse la asignación. En ese momento **setval** guarda el valor que se le va a dar a la variable. Entonces el usuario podrá darle un valor diferente pasándoselo a la variable **setval**.

Véanse también **setcheck** y **setval**.

### **setval**

Variable del sistema

Guarda el valor que va a ser asignado a una variable cuando **setcheckbreak** realiza una detención. Entonces se podrá asignarle otro valor pasándoselo previamente a **setval**.

Véanse también **setcheck** y **setcheckbreak**.

**timer** (*f\_1*, ..., *f\_n*) Función  
**timer** () Función

Dadas las funciones *f\_1*, ..., *f\_n*, **timer** coloca cada una de ellas en la lista de funciones para las cuales se generarán estadísticas relativas al tiempo de cómputo. Así, **timer(f)\$timer(g)\$** coloca a *f* y luego a *g* en dicha lista de forma acumulativa.

Si no se le pasan argumentos a **timer** se obtendrá la lista de funciones cuyos tiempos de ejecución se quieren monitorizar.

Maxima almacena la duración del cómputo de cada función de la lista, de forma que **timer\_info** devolverá las estadísticas correspondientes, incluyendo el tiempo medio de cada llamada a la función, el número de llamadas realizadas y el tiempo total transcurrido. La instrucción **untimer** borra las funciones de la lista.

La función **timer** no evalúa sus argumentos, de forma que *f(x) := x^2\$g:f\$timer(g)\$* no coloca a *f* en la lista.

Si **trace(f)** está activada, entonces **timer(f)** está desactivada; **trace** y **timer** no pueden estar operativas al mismo tiempo.

Véase también **timer\_devalue**.

**untimer** (*f\_1*, ..., *f\_n*) Función  
**untimer** () Función

Dadas las funciones *f\_1*, ..., *f\_n*, **untimer** las elimina de la lista de funciones cuyos tiempos de ejecución se quiere monitorizar.

Si no se le suministran argumentos, **untimer** borra completamente la lista.

Tras la ejecución de **untimer(f)**, **timer\_info(f)** aún devuelve las estadísticas de tiempo previamente registradas, pero **timer\_info()** (sin argumentos) no devuelve información sobre aquellas funciones que ya no están en la lista. La ejecución de **timer(f)** inicializa todas las estadísticas a cero y coloca *f* nuevamente en la lista.

**timer\_devalue** Variable opcional

Valor por defecto: **false**

Si **timer\_devalue** es igual a **true**, Maxima le resta a cada función cuyos tiempos de ejecución se quiere monitorizar el tiempo gastado en llamadas a otras funciones presentes también en la lista de monitorización. En caso contrario, los tiempos que se obtienen para cada función incluyen también los consumidos en otras funciones. Nótese que el tiempo consumido en llamadas a otras funciones que no están en la lista de monitorización no se resta del tiempo total.

Véanse también **timer** y **timer\_info**.

**timer\_info** (*f\_1*, ..., *f\_n*) Función  
**timer\_info** () Función

Dadas las funciones *f\_1*, ..., *f\_n*, **timer\_info** devuelve una matriz con información relativa a los tiempos de ejecución de cada una de estas funciones. Sin argumentos, **timer\_info** devuelve la información asociada a todas las funciones cuyos tiempos de ejecución se quiere monitorizar.

La matriz devuelta por **timer\_info** incluye los nombres de las funciones, tiempo de ejecución en cada llamada, número de veces que ha sido llamada, tiempo total



de ejecución y tiempo consumido en la recolección de basura, `gctime` (del inglés, "garbage collection time") en la versión original de Macsyma, aunque ahora toma el valor constante cero.

Los datos con los que `timer_info` construye su respuesta pueden obtenerse también con la función `get`:

```
get(f, 'calls); get(f, 'runtime); get(f, 'gctime);
```

Véase también `timer`.

**trace** (*f*<sub>1</sub>, ..., *f*<sub>*n*</sub>)

Función

**trace** ()

Función

Dadas las funciones *f*<sub>1</sub>, ..., *f*<sub>*n*</sub>, `trace` imprime información sobre depuración cada vez que estas funciones son llamadas; `trace(f)$trace(g)$` coloca de forma acumulativa a *f* y luego a *g* en la lista de funciones a ser rastreadas.

Si no se suministran argumentos, `trace` devuelve una lista con todas las funciones a ser rastreadas.

La función `untrace` desactiva el rastreo. Véase también `trace_options`.

La función `trace` no evalúa sus argumentos, de forma que `f(x) := x^2$ g:f$ trace(g)$` no coloca a *f* en la lista de rastreo.

Cuando una función se redefine es eliminada de la lista de rastreo. Así, tras `timer(f)$ f(x) := x^2$`, la función *f* dejará de estar en dicha lista.

Si `timer(f)` está activado, entonces `trace(f)` está desactivado, ya que `trace` y `timer` no pueden estar ambos activos para la misma función.

**trace\_options** (*f*, *option*<sub>1</sub>, ..., *option*<sub>*n*</sub>)

Función

**trace\_options** (*f*)

Función

Establece las opciones de rastreo para la función *f*. Cualquier otra opción previamente especificada queda reemplazada por las nuevas. La ejecución de `trace_options(f, ...)` no tiene ningún efecto, a menos que se haya invocado previamente a `trace(f)` (es indiferente que esta invocación sea anterior o posterior a `trace_options`).

`trace_options(f)` inicializa todas las opciones a sus valores por defecto.

Las claves de opciones son:

- **noprint**: No se imprime mensaje alguno ni a la entrada ni a la salida de la función.
- **break**: Coloca un punto de referencia antes de que la función comience a ejecutarse y otro después de que termine su ejecución. Véase **break**.
- **lisp\_print**: Muestra los argumentos y valores retornados como objetos de Lisp.
- **info**: Imprime `-> true` tanto a la entrada como a la salida de la función.
- **errorcatch**: Detecta errores, otorgando la posibilidad de marcar un error, reintentar la llamada a la función o especificar un valor de retorno.

Las opciones de rastreo se especifican de dos formas. La única presencia de la clave de opción ya activa la opción. (Nótese que la opción *foo* no se activa mediante `foo: true` u otra forma similar; se tendrá en cuenta también que las claves no necesitan ir

precedidas del apóstrofo.) Especificando la clave de opción junto con una función de predicado se hace que la opción quede condicionada al predicado.

La lista de argumentos para las funciones de predicado es siempre `[level, direction, function, item]` donde `level` es el nivel de recursión para la función, `direction` puede ser tanto `enter` como `exit`, `function` es el nombre de la función y `item` es la lista de argumentos (a la entrada) o el valor de retorno (a la salida).

A continuación un ejemplo de opciones de rastreo no condicionales:

```
(%i1) ff(n) := if equal(n, 0) then 1 else n * ff(n - 1)$
```

```
(%i2) trace (ff)$
```

```
(%i3) trace_options (ff, lisp_print, break)$
```

```
(%i4) ff(3);
```

Para la misma función, con la opción `break` condicionada a un predicado:

```
(%i5) trace_options (ff, break(pp))$
```

```
(%i6) pp (level, direction, function, item) := block (print (item),
 return (function = 'ff and level = 3 and direction = exit))$
```

```
(%i7) ff(6);
```

**untrace** (*f<sub>1</sub>*, ..., *f<sub>n</sub>*)

Función

**untrace** ()

Función

Dadas las funciones *f<sub>1</sub>*, ..., *f<sub>n</sub>*, **untrace** desactiva el rastreo previamente activado por la función **trace**. Si no se aportan argumentos, **untrace** desactiva el rastreo de todas las funciones.

La llamada a **untrace** devuelve una lista con las funciones para las que el rastreo se ha desactivado.



## 43 augmented\_lagrangian

### 43.1 Definiciones para augmented\_lagrangian

**niter**

Option variable

Default value: 10

Number of iterations for `augmented_lagrangian_method`.

**augmented\_lagrangian\_method** (*FOM*, *xx*, *constraints*, *yy*)

Function

Augmented Lagrangian method for constrained optimization. *FOM* is the figure of merit expression, *xx* is a list of variables over which to minimize, *constraints* is a list of expressions to be held equal to zero, and *yy* is a list of initial guesses for *xx*.

At present this code minimizes the augmented Lagrangian by solving for a stationary point of its gradient. That's pretty weak, and the code could be improved by plugging in a conjugate gradient or quasi-Newton minimization algorithm.

For references see

<http://www-fp.mcs.anl.gov/otc/Guide/OptWeb/continuous/constrained/nonlinearcon>

and

<http://www.cs.ubc.ca/spider/ascher/542/chap10.pdf>

Package `mnewton` (to solve  $\text{grad } L = 0$ ) has to be loaded before calling `augmented_lagrangian_method`.

Example:

```
(%i1) load (mnewton)$

(%i2) load("augmented_lagrangian")$

(%i3) FOM: x^2 + 2*y^2;
 2 2
(%o3) 2 y + x
(%i4) xx: [x, y];
(%o4) [x, y]
(%i5) C: [x + y - 1];
(%o5) [y + x - 1]
(%i6) yy: [1, 1];
(%o6) [1, 1]
(%i7) augmented_lagrangian_method (FOM, xx, C, yy);
(%o7) [0.6478349834, 0.3239174917]
```

To use this function write first `load("mnewton")` and `load("augmented_lagrangian")`. See also `niter`.



## 44 bode

### 44.1 Definiciones para bode

**bode\_gain** (*H*, *range*, ...*plot\_opts*...)

Function

Function to draw Bode gain plots.

Examples (1 through 7 from

<http://www.swarthmore.edu/NatSci/echeeve1/Ref/Bode/BodeHow.html>,

8 from Ron Crummett):

```
(%i1) load("bode")$

(%i2) H1 (s) := 100 * (1 + s) / ((s + 10) * (s + 100))$

(%i3) bode_gain (H1 (s), [w, 1/1000, 1000])$

(%i4) H2 (s) := 1 / (1 + s/omega0)$

(%i5) bode_gain (H2 (s), [w, 1/1000, 1000]), omega0 = 10$

(%i6) H3 (s) := 1 / (1 + s/omega0)^2$

(%i7) bode_gain (H3 (s), [w, 1/1000, 1000]), omega0 = 10$

(%i8) H4 (s) := 1 + s/omega0$

(%i9) bode_gain (H4 (s), [w, 1/1000, 1000]), omega0 = 10$

(%i10) H5 (s) := 1/s$

(%i11) bode_gain (H5 (s), [w, 1/1000, 1000])$

(%i12) H6 (s) := 1/((s/omega0)^2 + 2 * zeta * (s/omega0) + 1)$

(%i13) bode_gain (H6 (s), [w, 1/1000, 1000]),
 omega0 = 10, zeta = 1/10$

(%i14) H7 (s) := (s/omega0)^2 + 2 * zeta * (s/omega0) + 1$

(%i15) bode_gain (H7 (s), [w, 1/1000, 1000]),
 omega0 = 10, zeta = 1/10$

(%i16) H8 (s) := 0.5 / (0.0001 * s^3 + 0.002 * s^2 + 0.01 * s)$

(%i17) bode_gain (H8 (s), [w, 1/1000, 1000])$
```

To use this function write first `load("bode")`. See also `bode_phase`

**bode\_phase** (*H, range, ...plot\_opts...*)

Function

Function to draw Bode phase plots.

Examples (1 through 7 from

<http://www.swarthmore.edu/NatSci/echeeve1/Ref/Bode/BodeHow.html>,

8 from Ron Crummett):

```
(%i1) load("bode")$

(%i2) H1 (s) := 100 * (1 + s) / ((s + 10) * (s + 100))$

(%i3) bode_phase (H1 (s), [w, 1/1000, 1000])$

(%i4) H2 (s) := 1 / (1 + s/omega0)$

(%i5) bode_phase (H2 (s), [w, 1/1000, 1000]), omega0 = 10$

(%i6) H3 (s) := 1 / (1 + s/omega0)^2$

(%i7) bode_phase (H3 (s), [w, 1/1000, 1000]), omega0 = 10$

(%i8) H4 (s) := 1 + s/omega0$

(%i9) bode_phase (H4 (s), [w, 1/1000, 1000]), omega0 = 10$

(%i10) H5 (s) := 1/s$

(%i11) bode_phase (H5 (s), [w, 1/1000, 1000])$

(%i12) H6 (s) := 1/((s/omega0)^2 + 2 * zeta * (s/omega0) + 1)$

(%i13) bode_phase (H6 (s), [w, 1/1000, 1000]),
 omega0 = 10, zeta = 1/10$

(%i14) H7 (s) := (s/omega0)^2 + 2 * zeta * (s/omega0) + 1$

(%i15) bode_phase (H7 (s), [w, 1/1000, 1000]),
 omega0 = 10, zeta = 1/10$

(%i16) H8 (s) := 0.5 / (0.0001 * s^3 + 0.002 * s^2 + 0.01 * s)$

(%i17) bode_phase (H8 (s), [w, 1/1000, 1000])$

(%i18) block ([bode_phase_unwrap : false],
 bode_phase (H8 (s), [w, 1/1000, 1000]));

(%i19) block ([bode_phase_unwrap : true],
 bode_phase (H8 (s), [w, 1/1000, 1000]));
```

To use this function write first `load("bode")`. See also `bode_gain`

## 45 cholesky

### 45.1 Definiciones para cholesky

#### cholesky (A)

Function

Compute Cholesky decomposition of  $A$ , a lower-triangular matrix  $L$  such that  $L \cdot \text{transpose}(L) = A$ .

Some examples follow.

Example 1:

```
(%i1) load("cholesky")$

(%i2) A : matrix ([a, b, c], [d, e, f], [g, h, i]);
 [a b c]
 []
(%o2) [d e f]
 []
 [g h i]

(%i3) A2 : transpose (A) . A;
 [2 2 2]
 [g + d + a g h + d e + a b g i + d f + a c]
 []
(%o3) [2 2 2]
 [g h + d e + a b h + e + b h i + e f + b c]
 []
 []
 [g i + d f + a c h i + e f + b c i + f + c]
(%i4) B : cholesky (A2)$

(%i5) B . transpose (B) - A2;
 [0 0 0]
 []
(%o5) [0 0 0]
 []
 [0 0 0]
```

Example 2:

```
(%i6) A : matrix ([2, 3, 4], [-2, 2, -3], [11, -2, 3]);
 [2 3 4]
 []
(%o6) [-2 2 -3]
 []
 [11 -2 3]

(%i7) A2 : transpose (A) . A;
 [129 -20 47]
 []
(%o7) [-20 17 0]
 []
 [47 0 34]
```



```

(%i8) B : cholesky (A2);
 [sqrt(129) 0 0]
 []
 [20 sqrt(1793)]
 [- ---- - ---- 0]
 [sqrt(129) sqrt(129)]
(%o8) []
 []
 [47 940 sqrt(129) 153]
 [---- - ---- - ----]
 [sqrt(129) 129 sqrt(1793) sqrt(1793)]
(%i9) B . transpose (B) - A2;
 [0 0 0]
 []
(%o9) [0 0 0]
 []
 [0 0 0]

```

To use this function write first `load("cholesky")`.

## 46 descriptive

### 46.1 Introducción a descriptive

El paquete `descriptive` contiene funciones para realizar cálculos y gráficos estadísticos descriptivos. Junto con el código fuente se distribuyen tres conjuntos de datos: `pidigits.data`, `wind.data` y `biomed.data`, los cuales también se pueden descargar desde el sitio web [www.biomates.net](http://www.biomates.net).

Cualquier manual de estadística se puede utilizar como referencia al paquete `descriptive`.

Para comentarios, fallos y sugerencias, por favor contactar con '`mario AT edu DOT xunta DOT es`'.

Aquí un sencillo ejemplo sobre cómo operan las funciones de `descriptive`, dependiendo de la naturaleza de sus argumentos, listas o matrices,

```
(%i1) load(descriptive)$
(%i2) /* univariate sample */
 mean([a,b,c]);

 c + b + a

 3

(%i3) /* multivariate sample */
 matrix([a,b],[c,d],[e,f]);
 [a b]
 []
(%o3) [c d]
 []
 [e f]

(%i4) mean(%);

 e + c + a f + d + b
 [-----, -----]
 3 3
```

Nótese que en las muestras multivariantes la media se calcula para cada columna.

En caso de varias muestras de diferente tamaño, la función `map` de Maxima puede utilizarse para obtener los resultados deseados para cada muestra,

```
(%i5) map(mean,[[a,b,c],[d,e]]);
 c + b + a e + d
 [-----, -----]
 3 2

(%o5)
```

En este caso, dos muestras de tamaños 3 y 2 han sido almacenadas en una lista.

Muestras univariantes deben guardarse en listas como en

```
(%i6) s1:[3,1,4,1,5,9,2,6,5,3,5];
(%o6) [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
```

y muestras multivariantes en matrices como las del siguiente ejemplo

```
(%i7) s2:matrix([13.17, 9.29],[14.71, 16.88],[18.50, 16.88],
 [10.58, 6.63],[13.33, 13.25],[13.21, 8.12]);
 [13.17 9.29]
 []
 [14.71 16.88]
 []
 [18.5 16.88]
(%o7) []
 [10.58 6.63]
 []
 [13.33 13.25]
 []
 [13.21 8.12]
```

En este caso, el número de columnas es igual al de la dimensión de la variable aleatoria y el número de filas coincide con el tamaño muestral.

Los datos pueden suministrarse manualmente, pero las muestras grandes se suelen almacenar en ficheros de texto. Por ejemplo, el fichero `pidigits.data` contiene los 100 primeros dígitos del número  $\pi$ :

```
3
1
4
1
5
9
2
6
5
3 ...
```

A fin de leer estos dígitos desde Maxima,

```
(%i8) load(numericalio)$
(%i9) s1:read_list("path_to/pidigits.data")$
(%i10) length(s1);
(%o10) 100
```

Por otro lado, el archivo `wind.data` contiene los promedios diarios de la velocidad del viento en cinco estaciones meteorológicas en Irlanda (esta muestra es parte de un conjunto de datos correspondientes a 12 estaciones meteorológicas. El fichero original se puede descargar libremente del 'StatLib Data Repository' y se analiza en Haslett, J., Raftery, A. E. (1989) *Space-time Modelling with Long-memory Dependence: Assessing Ireland's Wind Power Resource, with Discussion*. Applied Statistics 38, 1-50). Así se leen los datos:

```
(%i11) s2:read_matrix("path_to/wind.data")$
(%i12) length(s2);
(%o12) 100
(%i13) s2[%]; /* last record */
(%o13) [3.58, 6.0, 4.58, 7.62, 11.25]
```

Algunas muestras contienen datos no numéricos. Como ejemplo, el archivo `biomed.data` (el cual es parte de otro mayor descargado también del 'StatLib Data Repository') contiene cuatro mediciones sanguíneas tomadas a dos grupos de pacientes, A y B, de diferentes edades,

```
(%i14) s3:read_matrix("path_to/biomed.data")$
(%i15) length(s3);
(%o15) 100
(%i16) s3[1]; /* first record */
(%o16) [A, 30, 167.0, 89.0, 25.6, 364]
```

El primer individuo pertenece al grupo A, tiene 30 años de edad y sus medidas sanguíneas fueron 167.0, 89.0, 25.6 y 364.

Debe tenerse cuidado cuando se trabaje con datos categóricos. En el siguiente ejemplo, se asigna al símbolo **a** cierto valor en algún momento previo y luego se toma una muestra con el valor categórico **a**,

```
(%i17) a:1$
(%i18) matrix([a,3],[b,5]);
(%o18) [1 3]
 []
 [b 5]
```

## 46.2 Definiciones para el tratamiento de datos

**cfrec** (*list*)

Función

**cfrec** (*list, m*)

Función

El argumento de **cfrec** debe ser una lista de números, los cuales serán luego agrupados en intervalos y hecho el recuento de cuántos hay en cada grupo. Opcionalmente, la función **cfrec** admite un segundo argumento para indicar el número de clases a considerar, siendo 10 su valor por defecto.

```
(%i1) load("numericalio.lisp")$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) cfrec(s1,5);
(%o3) [[0, 1.8, 3.6, 5.4, 7.2, 9.0], [16, 24, 18, 17, 25]]
```

La primera lista contiene los extremos de los intervalos y la segunda los resultados de los recuentos: hay 16 dígitos dentro del intervalo [0, 1.8], esto es ceros y unos, 24 dígitos en (1.8, 3.6], es decir doses y treses, y así sucesivamente.

**dfrec** (*list*)

Función

Calcula las frecuencias absolutas en muestras discretas, tanto numéricas como categóricas. Su único argumento debe ser una lista.

```
(%i1) load("numericalio.lisp")$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) dfrec(s1);
(%o3) [[0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
 [8, 8, 12, 12, 10, 8, 9, 8, 12, 13]]
(%i4) dfrec(transpose(col(s3,1))[1]);
(%o4) [[A, B], [35, 65]]
```

La primera lista son los valores de la muestra y la segunda sus frecuencias absolutas. Las instrucciones **? col** y **? transpose** pueden ayudar a comprender la última entrada.

**subsample** (*data\_matrix*, *logical\_expression*)

Función

**subsample** (*data\_matrix*, *logical\_expression*, *col\_num*, *col\_num*, ...)

Función

Esta es una variante de la función `submatrix` de Maxima. El primer argumento es una matriz de datos, el segundo es una expresión lógica no evaluada y el resto de argumentos opcionales son los números de las columnas a tomar en consideración,

```
(%i1) load("numericalio.lisp")$
(%i2) s2:read_matrix("path_to/wind.data")$
(%i3) subsample(s2,'(%c[1]>18));
 [19.38 15.37 15.12 23.09 25.25]
 []
 [18.29 18.66 19.08 26.08 27.63]
(%o3) []
 [20.25 21.46 19.95 27.71 23.38]
 []
 [18.79 18.96 14.46 26.38 21.84]
```

Estos son los registros multivariantes en los que la velocidad del viento en la primera estación meteorológica fue menor que 18. Véase cómo en la expresión lógica no evaluada a la *i*-ésima componente se la referencia como `%c[i]`. El símbolo `%c[i]` es utilizado dentro de la función `subsample`, por lo que si se utiliza también como variable categórica, Maxima puede dar resultados imprevistos. En el siguiente ejemplo, se solicitan únicamente la primera, segunda y quinta componentes de aquellos registros con velocidades del viento mayores o iguales que 16 nudos en la estación número 1 y menores que 25 nudos en la estación número 4,

```
(%i4) subsample(s2,'(%c[1]>=16 and %c[4]<25),1,2,5);
 [19.38 15.37 25.25]
 []
 [17.33 14.67 19.58]
(%o4) []
 [16.92 13.21 21.21]
 []
 [17.25 18.46 23.87]
```

He aquí un ejemplo con las variables categóricas de `biomed.data`. Se piden los registros correspondientes a aquellos pacientes del grupo B mayores de 38 años,

```
(%i5) subsample(s3,'(%c[1]=B and %c[2]>38));
 [B 39 28.0 102.3 17.1 146]
 []
 [B 39 21.0 92.4 10.3 197]
 []
 [B 39 23.0 111.5 10.0 133]
 []
 [B 39 26.0 92.6 12.3 196]
(%o5) []
 [B 39 25.0 98.7 10.0 174]
 []
 [B 39 21.0 93.2 5.9 181]
 []
 [B 39 18.0 95.0 11.3 66]
```

```
[
[B 39 39.0 88.5 7.6 168]
```

Es probable que el análisis estadístico requiera únicamente de las medidas sanguíneas,

```
(%i6) subsample(s3, '(%c[1]=B and %c[2]>38),3,4,5,6);
[28.0 102.3 17.1 146]
[
[21.0 92.4 10.3 197]
[
[23.0 111.5 10.0 133]
[
[26.0 92.6 12.3 196]
(%o6) [
[25.0 98.7 10.0 174]
[
[21.0 93.2 5.9 181]
[
[18.0 95.0 11.3 66]
[
[39.0 88.5 7.6 168]
```

Esta es la media multivariante de s3,

```
(%i7) s3:read_matrix("path_to/biomed.data")$
(%i8) mean(s3);
65 B + 35 A 317 6 NA + 8145.0
(%o8) [-----, ---, 87.178, -----, 18.123,
100 10 100
3 NA + 19587
-----]
100
```

Aquí la primera componente carece de significado, ya que tanto A como B son categóricas, la segunda componente es la edad media de los individuos en forma racional, al tiempo que los valores cuarto y quinto muestran cierto comportamiento extraño; lo cual se debe a que el símbolo NA se utiliza para indicar datos *no disponibles*, por lo que ambas medias no tienen sentido. Una posible solución puede ser extraer de la matriz aquellas filas con símbolos NA, lo que acarrearía cierta pérdida de información,

```
(%i9) mean(subsample(s3, '(%c[4]#NA and %c[6]#NA),3,4,5,6));
(%o9) [79.4923076923077, 86.2032967032967,
2514
16.93186813186813, ----]
13
```

### 46.3 Definiciones de valores descriptivos

**mean** (*list*)

Función

**mean** (*matrix*)

Función

Es la media muestral, definida como

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Ejemplo:

```
(%i1) load(numericalio)$
(%i1) s1:read_list("path_to/pidigits.data")$
(%i2) mean(s1);

 471
(%o2) ---
 100

(%i3) %, numer;
(%o3) 4.71
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) mean(s2);
(%o5) [9.9485, 10.1607, 10.8685, 15.7166, 14.8441]
```

**var** (*list*)

Función

**var** (*matrix*)

Función

Es la varianza muestral, definida como

$$\frac{s^2}{n} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) var(s1), numer;
(%o3) 8.425899999999999
```

Véase también **var1**.

**var1** (*list*)

Función

**var1** (*matrix*)

Función

Es la cuasivarianza muestral, definida como

$$\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) var1(s1), numer;
(%o3) 8.5110101010101
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) var(s2);
(%o5) [17.22190675000001, 14.98773651, 15.47572875,
 32.17651044000001, 24.42307619000001]
```

Véase también `var`.

**std** (*list*)

Función

**std** (*matrix*)

Función

Es la desviación típica muestral, raíz cuadrada de `var`.

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) std(s1), numer;
(%o3) 2.902740084816414
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) std(s2);
(%o5) [4.149928523480858, 3.871399812729241,
 3.933920277534866, 5.672434260526957, 4.941970881136392]
```

Véanse también `var` y `std1`.

**std1** (*list*)

Función

**std1** (*matrix*)

Función

Es la cuasidesviación típica muestral, raíz cuadrada de `var1`.

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) std1(s1), numer;
(%o3) 2.917363553109228
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) std1(s2);
(%o5) [4.17083509672109, 3.89090320978032,
 3.953738641137555, 5.701010936401517, 4.966867617451963]
```

Véanse también `var1` y `std`.

**ncmoment** (*list*, *k*)

Función

**ncmoment** (*matrix*, *k*)

Función

Es el momento no central de orden *k*, definido como

$$\frac{1}{n} \sum_{i=1}^n x_i^k$$

Ejemplo:



```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) ncmoment(s1,1), numer; /* the mean */
(%o3) 4.71
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) ncmoment(s2,5);
(%o5) [319793.8724761506, 320532.1923892463,
 391249.5621381556, 2502278.205988911, 1691881.797742255]
```

Véase también `cmoment`.

**cmoment** (*list*, *k*)

Función

**cmoment** (*matrix*, *k*)

Función

Es el momento central de orden *k*, definido como

$$\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^k$$

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) cmoment(s1,2), numer; /* the variance */
(%o3) 8.425899999999999
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) cmoment(s2,3);
(%o5) [11.29584771375004, 16.97988248298583,
 5.626661952750102, 37.5986572057918, 25.85981904394192]
```

Véanse también `cmoment` y `mean`.

**vc** (*list*)

Función

**vc** (*matrix*)

Función

Es el coeficiente de variación, o cociente entre la desviación típica muestral (`std`) y la media (`mean`),

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) vc(s1), numer;
(%o3) .6193977819764815
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) vc(s2);
(%o5) [.4192426091090204, .3829365309260502,
 0.363779605385983, .3627381836021478, .3346021393989506]
```

Véanse también `std` y `mean`.

**mini** (*list*)

Función

**mini** (*matrix*)

Función

Es el valor mínimo de la muestra *list*,

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) mini(s1);
(%o3)
0
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) mini(s2);
(%o5)
[0.58, 0.5, 2.67, 5.25, 5.17]
```

Véase también `maxi`.

**maxi** (*list*)

Función

**maxi** (*matrix*)

Función

Es el valor máximo de la muestra *list*,

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) maxi(s1);
(%o3)
9
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) maxi(s2);
(%o5)
[20.25, 21.46, 20.04, 29.63, 27.63]
```

Véase también `mini`.

**range** (*list*)

Función

**range** (*matrix*)

Función

Es la diferencia entre los valores extremos.

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) range(s1);
(%o3)
9
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) range(s2);
(%o5)
[19.67, 20.96, 17.37, 24.38, 22.46]
```

**quantile** (*list*, *p*)

Función

**quantile** (*matrix*, *p*)

Función

Es el  $p$ -cuantil, siendo  $p$  un número del intervalo  $[0, 1]$ , de la muestra *list*. Aunque existen varias definiciones para el cuantil muestral (Hyndman, R. J., Fan, Y. (1996) *Sample quantiles in statistical packages*. American Statistician, 50, 361-365), la programada en el paquete **descriptive** es la basada en la interpolación lineal.

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) /* 1st and 3rd quartiles */
[quantile(s1,1/4),quantile(s1,3/4)], numer;
(%o3)
[2.0, 7.25]
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) quantile(s2,1/4);
(%o5)
[7.2575, 7.477500000000001, 7.82, 11.28, 11.48]
```



```

 20
(%i4) s2:read_matrix("path_to/wind.data")$
(%o4) [3.287959999999999, 3.075342, 3.23907,
 4.715664000000001, 4.028546000000002]

```

Véase también `mean`.

**mediandev** (*list*)

Función

**mediandev** (*matrix*)

Función

Es la desviación mediana, definida como

$$\frac{1}{n} \sum_{i=1}^n |x_i - med|$$

siendo `med` la mediana de *list*.

Ejemplo:

```

(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) mediandev(s1);

 5
(%o3) -
 2

(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) mediandev(s2);
(%o5) [2.75, 2.755, 3.08, 4.315, 3.31]

```

Véase también `mean`.

**harmean** (*list*)

Función

**harmean** (*list*)

Función

Es la media armónica, definida como

$$\frac{n}{\sum_{i=1}^n \frac{1}{x_i}}$$

Ejemplo:

```

(%i1) y:[5,7,2,5,9,5,6,4,9,2,4,2,5]$
(%i2) harmean(y), numer;
(%o2) 3.901858027632205
(%i3) s2:read_matrix("path_to/wind.data")$
(%i4) harmean(s2);
(%o4) [6.948015590052786, 7.391967752360356,
 9.055658197151745, 13.44199028193692, 13.01439145898509]

```

Véanse también `mean` y `geomean`.

**geomean** (*list*)

Función

**geomean** (*matrix*)

Función

Es la media geométrica, definida como

$$\left( \prod_{i=1}^n x_i \right)^{\frac{1}{n}}$$

Ejemplo:

```
(%i1) y:[5,7,2,5,9,5,6,4,9,2,4,2,5]$
(%i2) geomean(y), numer;
(%o2) 4.454845412337012
(%i3) s2:read_matrix("path_to/wind.data")$
(%i4) geomean(s2);
(%o4) [8.82476274347979, 9.22652604739361,
10.0442675714889, 14.61274126349021, 13.96184163444275]
```

Véanse también `mean` y `harmean`.

**kurtosis** (*list*)

Función

**kurtosis** (*matrix*)

Función

Es el coeficiente de curtosis, definido como

$$\frac{1}{ns^4} \sum_{i=1}^n (x_i - \bar{x})^4 - 3$$

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) kurtosis(s1), numer;
(%o3) - 1.273247946514421
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) kurtosis(s2);
(%o5) [- .2715445622195385, 0.119998784429451,
- .4275233490482866, - .6405361979019522,
- .4952382132352935]
```

Véanse también `mean`, `var` y `skewness`.

**skewness** (*list*)

Función

**skewness** (*matrix*)

Función

Es el coeficiente de asimetría, definido como

$$\frac{1}{ns^3} \sum_{i=1}^n (x_i - \bar{x})^3$$

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) skewness(s1), numer;
(%o3) - 1.273247946514421
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) skewness(s2);
(%o5) [.1580509020000979, .2926379232061854,
.0924217441610772, .2059984348148687, .2142520248890832]
```

Véanse también `mean`, `var` y `kurtosis`.

**pearskewness** (*list*)

Función

**pearskewness** (*matrix*)

Función

Es el coeficiente de asimetría de Pearson, definido como

$$\frac{3 (\bar{x} - med)}{s}$$

siendo *med* la mediana de *list*.

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) pearskewness(s1);

(%o3)
 189 sqrt(11)

 10 sqrt(84259)

(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) pearskewness(s2);
(%o5) [- .0801997662921189, .2357036272952649,
 .1050904062491204, .1245042340592368, .4464181795804519]
```

Véanse también *mean*, *var* y *median*.**quarskewness** (*list*)

Función

**quarskewness** (*matrix*)

Función

Es el coeficiente de asimetría cuartílico, definido como

$$\frac{c_{\frac{3}{4}} - 2c_{\frac{1}{2}} + c_{\frac{1}{4}}}{c_{\frac{3}{4}} - c_{\frac{1}{4}}}$$

siendo  $c_p$  el  $p$ -cuantil de la muestra *list*.

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) quarskewness(s1);

(%o3)
 1
 --
 21

(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) quarskewness(s2);
(%o5) [- .0408542246982353, .1467025572005382,
 .0336239103362392, 0.037800687285223, 0.210526315789474]
```

Véase también *quantile*.

## 46.4 Definiciones de valores descriptivos multivariantes

**cov** (*matrix*)

Función

Es la matriz de covarianzas de una muestra multivariante, definida como

$$S = \frac{1}{n} \sum_{j=1}^n (X_j - \bar{X}) (X_j - \bar{X})'$$

siendo  $X_j$  la  $j$ -ésima fila de la matriz muestral.

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s2:read_matrix("path_to/wind.data")$
(%i3) fpprec:7$ /* change precision for pretty output */
(%i4) cov(s2);
[17.22191 13.61811 14.37217 19.39624 15.42162]
[
[13.61811 14.98774 13.30448 15.15834 14.9711]
[
(%o4) [14.37217 13.30448 15.47573 17.32544 16.18171]
[
[19.39624 15.15834 17.32544 32.17651 20.44685]
[
[15.42162 14.9711 16.18171 20.44685 24.42308]
```

Véase también `cov1`.

**cov1** (*matrix*)

Función

Es la matriz de cuasivarianzas de una muestra multivariante, definida como

$$\frac{1}{n-1} \sum_{j=1}^n (X_j - \bar{X}) (X_j - \bar{X})'$$

siendo  $X_j$  la  $j$ -ésima fila de la matriz muestral.

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s2:read_matrix("path_to/wind.data")$
(%i3) fpprec:7$ /* change precision for pretty output */
(%i4) cov1(s2);
[17.39587 13.75567 14.51734 19.59216 15.5774]
[
[13.75567 15.13913 13.43887 15.31145 15.12232]
[
(%o4) [14.51734 13.43887 15.63205 17.50044 16.34516]
[
[19.59216 15.31145 17.50044 32.50153 20.65338]
[
[15.5774 15.12232 16.34516 20.65338 24.66977]
```

Véase también `cov`.

**listvar** (*matrix*)

Función

**listvar** (*matrix*, *logical\_value*)

Función

La función `listvar` devuelve una lista de medidas globales de variabilidad:

- *varianza total*: `trace(S_1)`,
- *varianza media*: `trace(S_1)/p`,
- *varianza generalizada*: `determinant(S_1)`,

- *desviación típica generalizada*: `sqrt(determinant(S_1))`,
- *varianza efectiva* `determinant(S_1)^(1/p)`, (definida en: Peña, D. (2002) *Análisis de datos multivariantes*; McGraw-Hill, Madrid.)
- *desviación típica efectiva*: `determinant(S_1)^(1/(2*p))`.

donde  $p$  es la dimensión de la variable aleatoria multivariante y  $S_1$  la matriz de covarianzas devuelta por la función `cov1`.

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s2:read_matrix("path_to/wind.data")$
(%i3) listvar(s2);
(%o3) [105.3383, 21.06767, 12874.35, 113.4652, 6.636591,
 2.576158]
```

La función `listvar` tiene un argumento lógico opcional: `listvar(x,true)` indica a Maxima que  $x$  es la matriz de datos, calculando entonces lo mismo que `listvar(x)`. Por otro lado, `listvar(x,false)` significa que  $x$  no es la matriz de datos, sino la de covarianzas, evitando así recalcularla,

```
(%i4) s:cov1(s2)$
(%i5) listvar(s,false);
(%o5) [105.3383, 21.06767, 12874.35, 113.4652, 6.636591,
 2.576158]
```

Véanse también `cov` y `cov1`.

**cor** (*matrix*)

Función

**cor** (*matrix, logical\_value*)

Función

Es la matriz de correlaciones de la muestra multivariante.

Ejemplo:

```
(%i1) load(numericalio)$
(%i2) s2:read_matrix("path_to/wind.data")$
(%i3) cor(s2);
[1.0 .8476339 .8803515 .8239624 .7519506]
[
[.8476339 1.0 .8735834 .6902622 0.782502]
[
(%o3) [.8803515 .8735834 1.0 .7764065 .8323358]
[
[.8239624 .6902622 .7764065 1.0 .7293848]
[
[.7519506 0.782502 .8323358 .7293848 1.0]
```

La función `cor` tiene un argumento lógico opcional: `cor(x,true)` indica a Maxima que  $x$  es la matriz de datos, calculando entonces lo mismo que `cor(x)`. Por otro lado, `cor(x,false)` significa que  $x$  no es la matriz de datos, sino la de covarianzas, evitando así recalcularla,

```
(%i4) s:cov1(s2)$
(%i5) cor(s,false); /* this is faster */
[1.0 .8476339 .8803515 .8239624 .7519506]
```



```

[
[.8476339 1.0 .8735834 .6902622 0.782502]
[
(%o5) [.8803515 .8735834 1.0 .7764065 .8323358]
[
[.8239624 .6902622 .7764065 1.0 .7293848]
[
[.7519506 0.782502 .8323358 .7293848 1.0]

```

Véanse también `cov` y `cov1`.

**listdep** (*matrix*)

Función

**listdep** (*matrix*, *logical\_value*)

Función

La función `listdep` devuelve una lista con medidas de correlación:

- *matriz de precisión*: es la inversa de la matriz de covarianzas  $S_1$ ,

$$S_1^{-1} = (s^{ij})_{i,j=1,2,\dots,p}$$

- *vector de correlación múltiple*:  $(R_1^2, R_2^2, \dots, R_p^2)$ , donde

$$R_i^2 = 1 - \frac{1}{s^{ii}s_{ii}}$$

es un indicador de la bondad de ajuste del modelo de regresión lineal multivariante de  $X_i$  cuando el resto de variables se utilizan como regresores.

- *matriz de correlaciones parciales*: en la que el elemento  $(i, j)$  es

$$r_{ij.rest} = -\frac{s^{ij}}{\sqrt{s^{ii}s^{jj}}}$$

Ejemplo:

```

(%i1) load(numericalio)$
(%i2) s2:read_matrix("path_to/wind.data")$
(%i3) z:listdep(s2)$
(%i4) fpprec:5$ /* for pretty output */
(%i5) z[1]; /* precision matrix */
[.38486 - .13856 - .15626 - .10239 .03118]
[
[- .13856 .34107 - .15233 .03845 - .05284]
[
(%o5) [- .15626 - .15233 .47296 - .02482 - .10054]
[
[- .10239 .03845 - .02482 .10937 - .03403]
[
[.03118 - .05284 - .10054 - .03403 .14834]
(%i6) z[2]; /* multiple correlation vector */
[.85063, .80634, .86474, .71867, .72675]
(%o6) [
(%i7) z[3]; /* partial correlation matrix */
[- 1.0 .38244 .36627 .49908 - .13049]

```

```

[
[.38244 - 1.0 .37927 - .19907 .23492]
[
(%o7) [.36627 .37927 - 1.0 .10911 .37956]
[
[.49908 - .19907 .10911 - 1.0 .26719]
[
[- .13049 .23492 .37956 .26719 - 1.0]

```

La función `listdep` tiene un argumento lógico opcional: `listdep(x,true)` indica a Maxima que `x` es la matriz de datos, calculando entonces lo mismo que `listdep(x)`. Por otro lado, `listdep(x,false)` significa que `x` no es la matriz de datos, sino la de covarianzas, evitando así recalcularla.

Véanse también `cov` y `cov1`.

## 46.5 Definiciones para gráficos estadísticos

|                                                                            |         |
|----------------------------------------------------------------------------|---------|
| <b>dataplot</b> ( <i>list</i> )                                            | Función |
| <b>dataplot</b> ( <i>list</i> , <i>option_1</i> , <i>option_2</i> , ...)   | Función |
| <b>dataplot</b> ( <i>matrix</i> )                                          | Función |
| <b>dataplot</b> ( <i>matrix</i> , <i>option_1</i> , <i>option_2</i> , ...) | Función |

La función `dataplot` permite la visualización directa de datos muestrales, tanto univariantes (*list*) como multivariantes (*matrix*). Asignando valores a las siguientes opciones (*options*) se pueden controlar ciertos aspectos del gráfico:

- `'outpudev`, "x" por defecto, indica el dispositivo de salida; valores aceptados son "x", "eps" y "png", para la pantalla, y los formatos de ficheros postscript y png, respectivamente.
- `'maintitle`, "" por defecto, es el título principal entre comillas dobles.
- `'axisnames`, ["x", "y", "z"] por defecto, es una lista con nombres para los ejes x, y y z.
- `'joined`, false por defecto, un valor lógico que indica si los puntos en 2D deben unirse por segmentos o no.
- `'picturescales`, [1.0, 1.0] por defecto, factores de escala para el tamaño del gráfico.
- `'threedim`, true por defecto, indica a Maxima si debe dibujar una matriz de tres columnas como un diagrama tridimensional o como un diagrama de dispersión multivariante. Véanse ejemplos más abajo.
- `'axisrot`, [60, 30] por defecto, cambia el punto de vista cuando `'threedim` toma el valor true y los datos se suministran en una matriz de tres columnas. El primer número es el ángulo de rotación del eje x, siendo el segundo el ángulo de rotación del eje z, ambos medidos en grados sexagesimales.
- `'nclases`, 10 por defecto, es el número de clases a considerar en los histogramas de la diagonal de los diagramas de dispersión multivariantes.
- `'pointstyle`, 0 por defecto, es un entero que indica cómo dibujar los puntos muestrales.

Por ejemplo, con el siguiente código se realiza un gráfico con los veinte primeros dígitos del número  $\pi$  y el resultado se guarda en un fichero en formato eps.

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) dataplot(makelist(s1[k],k,1,20),
 'outputdev="eps",'pointstyle=3)$
```

Nótese que las muestras univariantes se representan como series temporales. En el siguiente ejemplo se utilizan más datos y con diferentes opciones,

```
(%i4) dataplot(makelist(s1[k],k,1,50),
 'maintitle="First pi digits",
 'axisnames=["digit order","digit value"],
 'pointstyle=1,
 'joined=true,
 'outputdev="eps")$
```

La función `dataplot` puede utilizarse para representar puntos del plano. El siguiente ejemplo es un diagrama de dispersión de los pares formados con las velocidades del viento correspondientes a las estaciones meteorológicas uno y cinco,

```
(%i5) s2:read_matrix("path_to/wind.data")$
(%i6) dataplot(submatrix(s2,2,3,4),
 'outputdev="eps", 'pointstyle=1,
 'maintitle="Pairs of wind speeds measured in knots",
 'axisnames=["Wind speed in A","Wind speed in E"])$
```

Si los puntos están almacenados en una matriz de dos columnas, `dataplot` los puede dibujar directamente, pero si están guardados como una lista de pares deben transformarse previamente a una matriz, tal como se hace en el siguiente ejemplo.

```
(%i7) x: [[-1,2],[5,7],[5,-3],[-6,-9],[-4,6]]$
(%i8) dataplot(apply('matrix,x),
 'outputdev="eps",
 'maintitle="Points", 'joined=true,
 'axisnames=["","",""],
 'picturescales=[0.5, 1.0])$
```

Puntos del espacio tridimensional pueden verse como proyecciones sobre un plano. En este ejemplo se piden gráficos de las velocidades del viento correspondientes a tres estaciones meteorológicas, primero una representación tridimensional y luego como diagrama de dispersión multivariante.

```
(%i9) /* 3D plot */
 dataplot(submatrix(s2,4,5), 'pointstyle=2,
 'outputdev="eps", 'picturescales=[0.6, 0.6],
 'maintitle="Pairs of wind speeds measured in knots",
 'axisnames=["Station A","Station B","Station C"])$
(%i10) /* Multivariate scatterplot */
 dataplot(submatrix(s2,4,5),
 'outputdev="eps", 'nclases=6, 'threedim=false)$
```

Nótese que en este último ejemplo, el número de clases de los histogramas de la diagonal se ajusta a 6, y que a la opción `'threedim` se le da el valor `false`.

Para más de tres dimensiones sólo son posibles los diagramas de dispersión multivariantes, como en

```
(%i11) dataplot(s2,'outputdev="eps")$
```

|                                                                                        |         |
|----------------------------------------------------------------------------------------|---------|
| <b>histogram</b> ( <i>list</i> )                                                       | Función |
| <b>histogram</b> ( <i>list</i> , <i>option_1</i> , <i>option_2</i> , ...)              | Función |
| <b>histogram</b> ( <i>one_column_matrix</i> )                                          | Función |
| <b>histogram</b> ( <i>one_column_matrix</i> , <i>option_1</i> , <i>option_2</i> , ...) | Función |

Dibuja un histograma. Los datos muestrales deben suministrarse como una lista de números o como una matriz columna. Asignando valores a las siguientes opciones (*options*) se pueden controlar ciertos aspectos del gráfico:

- `'outpudev`, "x" por defecto, indica el dispositivo de salida; valores aceptados son "x", "eps" y "png", para la pantalla, y los formatos de ficheros postscript y png, respectivamente.
- `'maintitle`, "" por defecto, es el título principal entre comillas dobles.
- `'axisnames`, ["x", "Fr."] por defecto, es una lista con nombres para los ejes x e y.
- `'picturescales`, [1.0, 1.0] por defecto, factores de escala para el tamaño del gráfico.
- `'nclases`, 10 por defecto, es el número de intervalos de clase.
- `'relbarwidth`, 0.9 por defecto, es un número decimal entre 0 y 1 para controlar el ancho de las barras.
- `'barcolor`, 1 por defecto, es un entero que indica el color de las barras.
- `'colorintensity`, 1 por defecto, es un número decimal entre 0 y 1 para establecer la intensidad del color.

En los siguientes dos ejemplos se solicitan sendos histogramas para los 100 primeros dígitos del número  $\pi$  y para las velocidades del viento en la estación meteorológica número 3.

```
(%i1) load(numericalio)$
(%i2) s1:read_list("path_to/pidigits.data")$
(%i3) histogram(s1,'maintitle="pi digits",
 'axisnames=["","Absolute frequency"],
 'relbarwidth=0.2,'barcolor=3,
 'colorintensity=0.6,'outputdev="eps")$
(%i4) s2:read_matrix("path_to/wind.data")$
(%i5) histogram(col(s2,3),
 'outputdev="eps", 'colorintensity=0.3)$
```

Nótese que en el primer caso, `s1` es una lista y que en el segundo `col(s2,3)` es una matriz.

Véase también `barsplot`.

|                                                                                       |         |
|---------------------------------------------------------------------------------------|---------|
| <b>barsplot</b> ( <i>list</i> )                                                       | Función |
| <b>barsplot</b> ( <i>list</i> , <i>option_1</i> , <i>option_2</i> , ...)              | Función |
| <b>barsplot</b> ( <i>one_column_matrix</i> )                                          | Función |
| <b>barsplot</b> ( <i>one_column_matrix</i> , <i>option_1</i> , <i>option_2</i> , ...) | Función |

Similar a **histogram** pero para muestras discretas, numéricas o categóricas. Estas son las variables,

- **'outputdev**, "x" por defecto, indica el dispositivo de salida; valores aceptados son "x", "eps" y "png", para la pantalla, y los formatos de ficheros postscript y png, respectivamente.
- **'maintitle**, "" por defecto, es el título principal entre comillas dobles.
- **'axisnames**, ["x", "Fr."] por defecto, es una lista con nombres para los ejes x e y.
- **'picturyscale**, [1.0, 1.0] por defecto, factores de escala para el tamaño del gráfico.
- **'relbarwidth**, 0.9 por defecto, es un número decimal entre 0 y 1 para controlar el ancho de las barras.
- **'barcolor**, 1 por defecto, es un entero que indica el color de las barras.
- **'colorintensity**, 1 por defecto, es un número decimal entre 0 y 1 para establecer la intensidad del color.

En este ejemplo se representa el gráfico de barras correspondiente a los grupos A y B de los pacientes de la muestra **s3**,

```
(%i1) load(numericalio)$
(%i2) s3:read_matrix("biomed.data")$
(%i3) barsplot(col(s3,1),
 'outputdev="eps",
 'maintitle="Groups of patients",
 'axisnames=["Group", "# of individuals"],
 'colorintensity=0.2)$
```

La primera columna de la muestra **s3** almacena los valores categóricos A y B, llamados también factores. Por otro lado, los números enteros positivos en la segunda columna son las edades en años, que es una variable discreta, por lo que podemos representar las frecuencias absolutas de estos valores,

```
(%i4) barsplot(col(s3,2),
 'outputdev="eps",
 'maintitle="Ages",
 'axisnames=["Years", "# of individuals"],
 'colorintensity=0.2,
 'relbarwidth=0.6)$
```

Véase también **histogram**.

|                                                                         |         |
|-------------------------------------------------------------------------|---------|
| <b>boxplot</b> ( <i>data</i> )                                          | Función |
| <b>boxplot</b> ( <i>data</i> , <i>option_1</i> , <i>option_2</i> , ...) | Función |

Dibuja diagramas de cajas. El argumento *data* puede ser una lista, lo cual no es de gran interés puesto que estos gráficos se utilizan fundamentalmente para comparar

varias muestras, o una matriz, de manera que es posible comparar dos o más componentes de una variable estadística multivariante. También se permite que *data* sea una lista de muestras con tamaños muestrales diferentes; de hecho esta es la única función del paquete **descriptive** que admite este tipo de estructura. Véase ejemplo más abajo. Estas son las opciones válidas:

- `'outputdev`, "x" por defecto, indica el dispositivo de salida; valores aceptados son "x", "eps" y "png", para la pantalla, y los formatos de ficheros postscript y png, respectivamente.
- `'maintitle`, "" por defecto, es el título principal entre comillas dobles.
- `'axisnames`, ["sample", "y"] por defecto, es una lista con nombres para los ejes x e y.
- `'picturescales`, [1.0, 1.0] por defecto, factores de escala para el tamaño del gráfico.

Ejemplos:

```
(%i1) load(numericalio)$
(%i2) s2:read_matrix("path_to/wind.data")$
(%i3) boxplot(s2,'outputdev="eps",
 'maintitle="Velocidad del viento en nudos",
 'axisnames=["Estaciones",""])$
(%i4) boxplot([[6,4,6,2,4,8,6,4,6,4,3,2],
 [8,10,7,9,12,8,10],
 [16,13,17,12,11,18,13,18,14,12]],
 'outputdev="eps")$
```



## 47 diag

### 47.1 Definiciones para diag

**diag** (*lm*) Function  
 Constructs a square matrix with the matrices of *lm* in the diagonal. *lm* is a list of matrices or scalars.

Example:

```
(%i1) load("diag")$

(%i2) a1:matrix([1,2,3],[0,4,5],[0,0,6])$

(%i3) a2:matrix([1,1],[1,0])$

(%i4) diag([a1,x,a2]);
 [1 2 3 0 0 0]
 []
 [0 4 5 0 0 0]
 []
 [0 0 6 0 0 0]
(%o4) []
 [0 0 0 x 0 0]
 []
 [0 0 0 0 1 1]
 []
 [0 0 0 0 1 0]
```

To use this function write first `load("diag")`.

**JF** (*lambda,n*) Function  
 Returns the Jordan cell of order *n* with eigenvalue *lambda*.

Example:

```
(%i1) load("diag")$

(%i2) JF(2,5);
 [2 1 0 0 0]
 []
 [0 2 1 0 0]
 []
(%o2) [0 0 2 1 0]
 []
 [0 0 0 2 1]
 []
 [0 0 0 0 2]

(%i3) JF(3,2);
 [3 1]
(%o3) []
```



$$\begin{bmatrix} 0 & 3 \end{bmatrix}$$

To use this function write first `load("diag")`.

### **jordan** (*mat*)

Function

Returns the Jordan form of matrix *mat*, but codified in a Maxima list. To get the corresponding matrix, call function `dispJordan` using as argument the output of `JF`.

Example:

```
(%i1) load("diag")$

(%i3) a:matrix([2,0,0,0,0,0,0,0],
 [1,2,0,0,0,0,0,0],
 [-4,1,2,0,0,0,0,0],
 [2,0,0,2,0,0,0,0],
 [-7,2,0,0,2,0,0,0],
 [9,0,-2,0,1,2,0,0],
 [-34,7,1,-2,-1,1,2,0],
 [145,-17,-16,3,9,-2,0,3])$

(%i34) jordan(a);
(%o4) [[2, 3, 3, 1], [3, 1]]
(%i5) dispJordan(%);
 [2 1 0 0 0 0 0 0]
 []
 [0 2 1 0 0 0 0 0]
 []
 [0 0 2 0 0 0 0 0]
 []
 [0 0 0 2 1 0 0 0]
(%o5) []
 [0 0 0 0 2 1 0 0]
 []
 [0 0 0 0 0 2 0 0]
 []
 [0 0 0 0 0 0 2 0]
 []
 [0 0 0 0 0 0 0 3]
```

To use this function write first `load("diag")`. See also `dispJordan` and `minimalPoly`.

### **dispJordan** (*l*)

Function

Returns the Jordan matrix associated to the codification given by the Maxima list *l*, which is the output given by function `jordan`.

Example:

```
(%i1) load("diag")$

(%i2) b1:matrix([0,0,1,1,1],
 [0,0,0,1,1],
 [0,0,0,0,1],
```

```

[0,0,0,0,0],
[0,0,0,0,0])$

(%i3) jordan(b1);
(%o3) [[0, 3, 2]]
(%i4) dispJordan(%);
 [0 1 0 0 0]
 []
 [0 0 1 0 0]
 []
(%o4) [0 0 0 0 0]
 []
 [0 0 0 0 1]
 []
 [0 0 0 0 0]

```

To use this function write first `load("diag")`. See also `jordan` and `minimalPoly`.

### **minimalPoly** (*l*)

Function

Returns the minimal polynomial associated to the codification given by the Maxima list *l*, which is the output given by function `jordan`.

Example:

```

(%i1) load("diag")$

(%i2) a:matrix([2,1,2,0],
 [-2,2,1,2],
 [-2,-1,-1,1],
 [3,1,2,-1])$

(%i3) jordan(a);
(%o3) [[- 1, 1], [1, 3]]
(%i4) minimalPoly(%);
 3
(%o4) (x - 1) (x + 1)

```

To use this function write first `load("diag")`. See also `jordan` and `dispJordan`.

### **ModeMatrix** (*A,l*)

Function

Returns the matrix *M* such that  $(Mm1).A.M = J$ , where *J* is the Jordan form of *A*. The Maxima list *l* is the codified form of the Jordan form as returned by function `jordan`.

Example:

```

(%i1) load("diag")$

(%i2) a:matrix([2,1,2,0],
 [-2,2,1,2],
 [-2,-1,-1,1],
 [3,1,2,-1])$

```

```
(%i3) jordan(a);
(%o3) [[- 1, 1], [1, 3]]
(%i4) M: ModeMatrix(a,%);
 [1 - 1 1 1]
 []
 [1]
 [- - - 1 0 0]
 [9]
 []
(%o4) [13]
 [- -- 1 - 1 0]
 [9]
 []
 [17]
 [-- - 1 1 1]
 [9]
(%i5) is((M^^-1).a.M = dispJordan(%o3));
(%o5) true
```

Note that `dispJordan(%o3)` is the Jordan form of matrix `a`.

To use this function write first `load("diag")`. See also `jordan` and `dispJordan`.

### **mat\_function** (*f,mat*)

Function

Returns  $f(mat)$ , where  $f$  is an analytic function and  $mat$  a matrix. This computation is based on Cauchy's integral formula, which states that if  $f(x)$  is analytic and

$mat=diag([JF(m1,n1), \dots, JF(mk,nk)])$ ,

then

$f(mat)=ModeMatrix*diag([f(JF(m1,n1)), \dots, f(JF(mk,nk))])*ModeMatrix^{(-1)}$  ■

Note that there are about 6 or 8 other methods for this calculation.

Some examples follow.

Example 1:

```
(%i1) load("diag")$

(%i2) b2:matrix([0,1,0], [0,0,1], [-1,-3,-3])$

(%i3) mat_function(exp,t*b2);
 2 - t
 t %e - t - t
(%o3) matrix([----- + t %e + %e ,
 2
 - t - t - t
 2 %e %e - t - t %e
t (- ---- - ---- + %e) + t (2 %e - ----)
 t 2 t

 - t - t - t
 + 2 %e , t (%e - ----) + t (---- - ----)
 - t - t - t
```

```

+ %e-t], [- $\frac{t^2 e^{-t}}{2}$, - t2 (- $\frac{e^{-t}}{t}$ - $\frac{e^{-t}}{2}$ + %e-t),
- t2 ($\frac{e^{-t}}{2}$ - $\frac{e^{-t}}{t}$)], [- $\frac{t^2 e^{-t}}{2}$ - t %e-t ,
t2 (- $\frac{e^{-t}}{t}$ - $\frac{e^{-t}}{2}$ + %e-t) - t (2 %e-t - $\frac{e^{-t}}{t}$),
t2 ($\frac{e^{-t}}{2}$ - $\frac{e^{-t}}{t}$) - t (%e-t - $\frac{e^{-t}}{t}$)]]
(%i4) ratsimp(%);
[2 - t]
[(t + 2 t + 2) %e]
[-----]
[2]
[]
[]
[2 - t]
(%o4) Col 1 = [t %e]
[- -----]
[2]
[]
[2 - t]
[(t - 2 t) %e]
[-----]
[2]
[]
[2 - t]
[(t + t) %e]
[]
[]
Col 2 = [2 - t]
[- (t - t - 1) %e]
[]
[2 - t]
[(t - 3 t) %e]
[2 - t]
[t %e]
[-----]
[2]
[]
[2 - t]
Col 3 = [(t - 2 t) %e]

```

$$\begin{bmatrix} -\frac{1}{2} \\ \frac{1}{2} \\ \frac{1}{2} \\ \frac{1}{2} \\ \frac{1}{2} \\ \frac{1}{2} \end{bmatrix}$$

Example 2:

```
(%i5) b1:matrix([0,0,1,1,1],
 [0,0,0,1,1],
 [0,0,0,0,1],
 [0,0,0,0,0],
 [0,0,0,0,0])$

(%i6) mat_function(exp,t*b1);
 [2]
 [t]
 [1 0 t t -- + t]
 [2]
 []
(%o6) [0 1 0 t t]
 []
 [0 0 1 0 t]
 []
 [0 0 0 1 0]
 []
 [0 0 0 0 1]

(%i7) minimalPoly(jordan(b1));
 3
 x
(%o7) ident(5)+t*b1+1/2*(t^2)*b1^2;
 [2]
 [t]
 [1 0 t t -- + t]
 [2]
 []
(%o8) [0 1 0 t t]
 []
 [0 0 1 0 t]
 []
 [0 0 0 1 0]
 []
 [0 0 0 0 1]

(%i9) mat_function(exp,%i*t*b1);
 [2]
 [t]
 [1 0 %i t %i t %i t - --]
 [2]
```

```

(%o9) [
 [0 1 0 %i t %i t]
 [
 [0 0 1 0 %i t]
 [
 [0 0 0 1 0]
 [
 [0 0 0 0 1]
(%i10) mat_function(cos,t*b1)+%i*mat_function(sin,t*b1);
 [
 [2]
 [t]
 [1 0 %i t %i t %i t - --]
 [2]
 [
(%o10) [0 1 0 %i t %i t]
 [
 [0 0 1 0 %i t]
 [
 [0 0 0 1 0]
 [
 [0 0 0 0 1]

```

Example 3:

```

(%i11) a1:matrix([2,1,0,0,0,0],
 [-1,4,0,0,0,0],
 [-1,1,2,1,0,0],
 [-1,1,-1,4,0,0],
 [-1,1,-1,1,3,0],
 [-1,1,-1,1,1,2])$

(%i12) fpow(x):=block([k],declare(k,integer),x^k)$

(%i13) mat_function(fpow,a1);
 [k k - 1] [k - 1]
 [3 - k 3] [k 3]
 [] []
 [k - 1] [k k - 1]
 [- k 3] [3 + k 3]
 [] []
 [k - 1] [k - 1]
 [- k 3] [k 3]
(%o13) Col 1 = [] Col 2 = []
 [k - 1] [k - 1]
 [- k 3] [k 3]
 [] []
 [k - 1] [k - 1]
 [- k 3] [k 3]
 [] []
 [k - 1] [k - 1]
 [- k 3] [k 3]

```

$$\begin{array}{lcl}
 \begin{array}{c}
 \begin{bmatrix} 0 \\ 0 \\ k^3 - k^3 \end{bmatrix} \\
 \text{Col 3} = \begin{bmatrix} k^3 - k^3 \\ -k^3 \\ k^3 - k^3 \\ -k^3 \\ k^3 - k^3 \\ -k^3 \end{bmatrix}
 \end{array}
 &
 \begin{array}{c}
 \begin{bmatrix} 0 \\ 0 \\ k^3 - k^3 \end{bmatrix} \\
 \text{Col 4} = \begin{bmatrix} k^3 - k^3 \\ k^3 \\ k^3 - k^3 \\ 3 + k^3 \\ k^3 - k^3 \\ k^3 \\ k^3 - k^3 \\ k^3 \end{bmatrix}
 \end{array}
 \\
 \begin{array}{c}
 \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ k^3 \\ 3 \end{bmatrix} \\
 \text{Col 5} = \begin{bmatrix} 0 \\ k^3 \\ 3 \\ k^3 - k^3 \\ 3 - 2 \end{bmatrix}
 \end{array}
 &
 \begin{array}{c}
 \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ k^3 \\ 2 \end{bmatrix} \\
 \text{Col 6} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ k^3 \\ 2 \end{bmatrix}
 \end{array}
 \end{array}$$

To use this function write first `load("diag")`.

## 48 distrib

### 48.1 Introducción a distrib

El paquete **distrib** contiene un conjunto de funciones para la realización de cálculos probabilísticos con modelos univariantes, tanto discretos como continuos.

A continuación un breve recordatorio de las definiciones básicas sobre distribuciones de probabilidad.

Sea  $f(x)$  la *función de densidad* de una variable aleatoria  $X$  absolutamente continua. La *función de distribución* se define como

$$F(x) = \int_{-\infty}^x f(u) \, du$$

que es igual a la probabilidad  $\Pr(X \leq x)$ .

La *media* es un parámetro de localización y se define como

$$E[X] = \int_{-\infty}^{\infty} x f(x) \, dx$$

La *varianza* es una medida de dispersión,

$$V[X] = \int_{-\infty}^{\infty} f(x) (x - E[X])^2 \, dx$$

que es un número real positivo. La raíz cuadrada de la varianza es la *desviación típica*,  $D[X] = \sqrt{V[X]}$ , siendo otra medida de dispersión.

El *coeficiente de asimetría* es una medida de forma,

$$SK[X] = \frac{\int_{-\infty}^{\infty} f(x) (x - E[X])^3 \, dx}{D[X]^3}$$

Y el *coeficiente de curtosis* mide el apuntamiento de la densidad,

$$KU[X] = \frac{\int_{-\infty}^{\infty} f(x) (x - E[X])^4 \, dx}{D[X]^4} - 3$$

Si  $X$  es normal,  $KU[X] = 0$ . De hecho, tanto la asimetría como la curtosis son parámetros de forma para medir la no normalidad de una distribución.

Si la variable aleatoria  $X$  es discreta, su función de densidad, o de *probabilidad*,  $f(x)$  toma valores positivos dentro de un conjunto numerable de valores  $x_i$ , y cero en cualquier otro lugar. En este caso, la función de distribución es

$$F(x) = \sum_{x_i \leq x} f(x_i)$$



La media, varianza, desviación típica y los coeficientes de asimetría y curtosis adquieren las formas

$$E[X] = \sum_{x_i} x_i f(x_i),$$

$$V[X] = \sum_{x_i} f(x_i) (x_i - E[X])^2,$$

$$D[X] = \sqrt{V[X]},$$

$$SK[X] = \frac{\sum_{x_i} f(x) (x - E[X])^3 dx}{D[X]^3}$$

y

$$KU[X] = \frac{\sum_{x_i} f(x) (x - E[X])^4 dx}{D[X]^4} - 3,$$

respectivamente.

El paquete **distrib** incluye funciones para la simulación de variables aleatorias. Algunas de estas funciones hacen uso de variables opcionales que indican el algoritmo a ejecutar. El método general de inversión (basado en el hecho de que si  $u$  es un número aleatorio generado uniformemente en  $(0, 1)$ , entonces  $F^{-1}(u)$  es una variable aleatoria con distribución  $F$ ) está implementado en la mayoría de los casos, aunque se trate de una técnica subóptima en términos de tiempo, pero útil para compararla con otros algoritmos. En el siguiente ejemplo se compara mediante histogramas el comportamiento de los algoritmos **ahrens\_cheng** y **inverse** para la simulación de variables chi-cuadrada:

```
(%i1) load(descriptive)$
(%i2) showtime:true$
Evaluation took 0.00 seconds (0.00 elapsed) using 80 bytes.
(%i3) rchi2_algorithm: 'ahrens_cheng$ histogram(rchi2(10,500))$
Evaluation took 0.00 seconds (0.00 elapsed) using 80 bytes.
Evaluation took 0.70 seconds (0.77 elapsed) using 5.517 MB.
(%i5) rchi2_algorithm: 'inverse$ histogram(rchi2(10,500))$
Evaluation took 0.00 seconds (0.00 elapsed) using 80 bytes.
Evaluation took 10.37 seconds (10.45 elapsed) using 321.278 MB.
```

A fin de hacer comparaciones visuales entre algoritmos para variables discretas, se debería utilizar la función **barsplot** del paquete **descriptive**.

Nótese que aún queda trabajo por hacer, puesto que estas funciones de simulación todavía no se han puesto a prueba con tests de bondad de ajuste más rigurosos.

Por favor, consúltese cualquier manual introductorio de probabilidad y estadística para más información sobre toda esta parafernalia matemática.

Se sigue cierta convención a la hora de nombrar las funciones del paquete **distrib**. Cada nombre tiene dos partes, el primero hace referencia a la función o parámetro que se quiere calcular,

## Funciones:

|                           |         |
|---------------------------|---------|
| Función de densidad       | (den*)  |
| Función de distribución   | (dis*)  |
| Cuantil                   | (q*)    |
| Media                     | (mean*) |
| Varianza                  | (var*)  |
| Desviación típica         | (std*)  |
| Coefficiente de asimetría | (skw*)  |
| Coefficiente de curtosis  | (kur*)  |
| Valor aleatorio           | (r*)    |

La segunda parte hace referencia explícita al modelo probabilístico,

## Distribuciones continuas:

|                   |             |
|-------------------|-------------|
| Normal            | (*normal)   |
| Student           | (*student)  |
| Chi <sup>2</sup>  | (*chi2)     |
| F                 | (*f)        |
| Exponencial       | (*exp)      |
| Lognormal         | (*logn)     |
| Gamma             | (*gamma)    |
| Beta              | (*beta)     |
| Continua uniforme | (*contu)    |
| Logística         | (*log)      |
| Pareto            | (*pareto)   |
| Weibull           | (*weibull)  |
| Rayleigh          | (*rayleigh) |
| Laplace           | (*laplace)  |
| Cauchy            | (*cauchy)   |
| Gumbel            | (*gumbel)   |

## Distribuciones discretas:

|                   |              |
|-------------------|--------------|
| Binomial          | (*binomial)  |
| Poisson           | (*poisson)   |
| Bernoulli         | (*bernoulli) |
| Geométrica        | (*geo)       |
| Uniforme discreta | (*discu)     |
| Hipergeométrica   | (*hypergeo)  |
| Binomial negativa | (*negbinom)  |

Por ejemplo, `denstudent(x,n)` es la función de densidad de la distribución de Student con  $n$  grados de libertad, `stdpareto(a,b)` es la desviación típica de la distribución de Pareto de parámetros  $a$  y  $b$ , y `kurpoisson(m)` es el coeficiente de curtosis de la distribución de Poisson de media  $m$ .

Para poder hacer uso del paquete `distrib` es necesario cargarlo primero tecleando

```
(%i1) load(distrib)$
```

Para comentarios, errores o sugerencias, por favor contactar conmigo en '*mario AT edu DOT xunta DOT es*'.

## 48.2 Definiciones para distribuciones continuas

**dennormal** ( $x, m, s$ ) Función

Devuelve el valor correspondiente a  $x$  de la función de densidad de la variable aleatoria normal  $N(m, s)$ , con  $s > 0$ .

**disnormal** ( $x, m, s$ ) Función

Devuelve el valor correspondiente a  $x$  de la función de distribución de la variable aleatoria normal  $N(m, s)$ , con  $s > 0$ . Esta función se define en términos de la función de error, **erf**, de Maxima.

```
(%i1) assume(s>0)$ disnormal(x,m,s);
 x - m
 erf(-----)
 sqrt(2) s
(%o2) ----- + -
 2 2
```

Véase también **erf**.

**qnormal** ( $q, m, s$ ) Función

Devuelve el  $q$ -cuantil de una variable aleatoria normal  $N(m, s)$ , con  $s > 0$ ; en otras palabras, es la inversa de **disnormal**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

**meannormal** ( $m, s$ ) Función

Devuelve la media de una variable aleatoria normal  $N(m, s)$ , con  $s > 0$ , es decir  $m$ .

**varnormal** ( $m, s$ ) Función

Devuelve la varianza de una variable aleatoria normal  $N(m, s)$ , con  $s > 0$ , es decir  $s^2$ .

**stdnormal** ( $m, s$ ) Función

Devuelve la desviación típica de una variable aleatoria normal  $N(m, s)$ , con  $s > 0$ , es decir  $s$ .

**skwnormal** ( $m, s$ ) Función

Devuelve el coeficiente de asimetría de una variable aleatoria normal  $N(m, s)$ , con  $s > 0$ , que es siempre igual a 0.

**kurnormal** ( $m, s$ ) Función

Devuelve el coeficiente de curtosis de una variable aleatoria normal  $N(m, s)$ , con  $s > 0$ , que es siempre igual a 0.

**rnormal\_algorithm** Variable opcional

Valor por defecto: **box\_mueller**

Este es el algoritmo seleccionado para simular variables aleatorias normales. Los algoritmos implementados son **box\_mueller** y **inverse**:

- **box\_mueller**, basado en el algoritmo descrito en Knuth, D.E. (1981) *Seminumerical Algorithms. The Art of Computer Programming.* Addison-Wesley.
- **inverse**, basado en el método inverso.

Véase también **rnormal**.

**rnormal** ( $m,s$ ) Función  
**rnormal** ( $m,s,n$ ) Función

Devuelve un valor aleatorio  $N(m,s)$ , con  $s > 0$ . Llamando a **rnormal** con un tercer argumento  $n$ , se simula una muestra aleatoria de tamaño  $n$ .

Hay dos algoritmos programados para esta función, el que se vaya a utilizar tendrá que ser seleccionado dándole un valor a la variable global **rnormal\_algorithm**, cuyo valor por defecto es **box\_mueller**.

También existe una función en Maxima para simular variables normales, la cual utiliza el llamado método Ziggurat de Marsaglia.

Véanse también **rnormal\_algorithm** y **gauss**.

**denstudent** ( $x,n$ ) Función

Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria de Student  $t(n)$ , con  $n > 0$ .

**disstudent** ( $x,n$ ) Función

Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria de Student  $t(n)$ , con  $n > 0$ . Esta función no tiene expresión compacta y se calcula numéricamente si la variable global **numer** vale **true**, en otro caso devuelve una expresión nominal.

```
(%i1) disstudent(1/2, 7/3);
 1 7
(%o1) disstudent(-, -)
 2 3
(%i2) %,numer;
(%o2) .6698450596140417
```

**qstudent** ( $q,n$ ) Función

Devuelve el  $q$ -cuantil de una variable aleatoria de Student  $t(n)$ , con  $n > 0$ ; en otras palabras, se trata de la inversa de **disstudent**. El argumento  $q$  debe ser un número de  $[0,1]$ .

**meanstudent** ( $n$ ) Función

Devuelve la media de una variable aleatoria de Student  $t(n)$ , con  $n > 0$ , que vale siempre 0.

**varstudent** ( $n$ ) Función

Devuelve la varianza de una variable aleatoria de Student  $t(n)$ , con  $n > 2$ .

```
(%i1) assume(n>2)$ varstudent(n);
 n
(%o2) -----
 n - 2
```

**stdstudent** (*n*) Función  
Devuelve la desviación típica de una variable aleatoria de Student  $t(n)$ , con  $n > 2$ .

**skwstudent** (*n*) Función  
Devuelve el coeficiente de asimetría de una variable aleatoria de Student  $t(n)$ , con  $n > 3$ , que vale siempre 0.

**kurstudent** (*n*) Función  
Devuelve el coeficiente de curtosis una variable aleatoria de Student  $t(n)$ , con  $n > 4$ .

**rstudent\_algorithm** Variable opcional  
Valor por defecto: **ratio**  
Es el algoritmo seleccionado para simular variables aleatorias de Student. Los métodos implementados son **inverse** y **ratio**:

- **inverse**, basado en el método inverso.
- **ratio**, basado en el hecho de que si  $Z$  es una variable aleatoria normal  $N(0,1)$  y  $S^2$  es una variable aleatoria chi cuadrada con  $n$  grados de libertad,  $Chi^2(n)$ , entonces

$$X = \frac{Z}{\sqrt{\frac{S^2}{n}}}$$

es una variable aleatoria de Student con  $n$  grados de libertad,  $t(n)$ .

Véase también **rstudent**.

**rstudent** (*n*) Función  
**rstudent** (*n,m*) Función

Devuelve un valor aleatorio  $t(n)$ , con  $n > 0$ . Llamando a **rstudent** con un segundo argumento  $m$ , se obtiene una muestra aleatoria simulada de tamaño  $m$ .

Hay dos algoritmos programados para esta función, el que se vaya a utilizar tendrá que ser seleccionado dándole un valor a la variable global **rstudent\_algorithm**, cuyo valor por defecto es **ratio**.

Véase también **rstudent\_algorithm**.

**denchi2** (*x,n*) Función

Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria chi-cuadrado  $Chi^2(n)$ , con  $n > 0$ .

La variable aleatoria  $Chi^2(n)$  equivale a una  $Gamma(n/2,2)$ , por lo que cuando Maxima no tiene suficiente información para obtener el resultado, devuelve una forma nominal basada en la función de densidad de la gamma.

```
(%i1) denchi2(x,n);
 n
(%o1) dengamma(x, -, 2)
 2
(%i2) assume(x>0, n>0)$ denchi2(x,n);
 n/2 - 1 - x/2
```

|       |       |          |
|-------|-------|----------|
|       | x     | %e       |
| (%o2) | ----- |          |
|       | n/2   | n        |
|       | 2     | gamma(-) |
|       |       | 2        |

**dischi2** (*x,n*)

Función

Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria chi-cuadrado  $Chi^2(n)$ , con  $n > 0$ .

Esta función no tiene expresión compacta y se calcula numéricamente si la variable global **numer** vale **true**, en otro caso devuelve una expresión nominal basada en la distribución gamma, puesto que la variable aleatoria  $Chi^2(n)$  equivale a una  $Gamma(n/2, 2)$ .

```
(%i1) dischi2(3,4);
(%o1) disgamma(3, 2, 2)
(%i2) dischi2(3,4),numer;
(%o2) .4421745996289249
```

**qchi2** (*q,n*)

Función

Devuelve el  $q$ -cuantil de una variable aleatoria  $Chi^2(n)$ , con  $n > 0$ ; en otras palabras, se trata de la inversa de **dischi2**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

Esta función no tiene expresión compacta y se calcula numéricamente si la variable global **numer** vale **true**, en otro caso devuelve una expresión nominal basada en la función cuantil de la gamma, puesto que la variable aleatoria  $Chi^2(n)$  equivale a una  $Gamma(n/2, 2)$ .

```
(%i1) qchi2(0.99,9);
(%o1) 21.66599433346194
(%i2) qchi2(0.99,n);

(%o2) qgamma(0.99, - , 2)
 n
 2
```

**meanchi2** (*n*)

Función

Devuelve la media de una variable aleatoria  $Chi^2(n)$ , con  $n > 0$ .

La variable aleatoria  $Chi^2(n)$  equivale a una  $Gamma(n/2, 2)$ , por lo que cuando Maxima no tiene suficiente información para obtener el resultado, devuelve una forma nominal basada en la media de la gamma.

```
(%i1) meanchi2(n);

(%o1) meangamma(- , 2)
 n
 2

(%i2) assume(n>0)$ meanchi2(n);
(%o3) n
```

**varchi2** (*n*)

Función

Devuelve la varianza de una variable aleatoria  $Chi^2(n)$ , con  $n > 0$ .

La variable aleatoria  $Chi^2(n)$  equivale a una  $Gamma(n/2, 2)$ , por lo que cuando Maxima no tiene suficiente información para obtener el resultado, devuelve una forma nominal basada en la varianza de la gamma.

```
(%i1) varchi2(n);
(%o1) $\frac{n}{2}$
 vargamma(-, 2)
(%i2) assume(n>0)$ varchi2(n);
(%o3) $2 n$
```

### stdchi2 (n)

Función

Devuelve la desviación típica de una variable aleatoria  $Chi^2(n)$ , con  $n > 0$ .

La variable aleatoria  $Chi^2(n)$  equivale a una  $Gamma(n/2, 2)$ , por lo que cuando Maxima no tiene suficiente información para obtener el resultado, devuelve una forma nominal basada en la desviación típica de la gamma.

```
(%i1) stdchi2(n);
(%o1) $\frac{n}{2}$
 stdgamma(-, 2)
(%i2) assume(n>0)$ stdchi2(n);
(%o3) $\sqrt{2} \sqrt{n}$
```

### skwchi2 (n)

Función

Devuelve el coeficiente de asimetría de una variable aleatoria  $Chi^2(n)$ , con  $n > 0$ .

La variable aleatoria  $Chi^2(n)$  equivale a una  $Gamma(n/2, 2)$ , por lo que cuando Maxima no tiene suficiente información para obtener el resultado, devuelve una forma nominal basada en el coeficiente de asimetría de la gamma.

```
(%i1) skwchi2(n);
(%o1) $\frac{n}{2}$
 skwgamma(-, 2)
(%i2) assume(n>0)$ skwchi2(n);
(%o3) $\frac{2 \sqrt{2}}{\sqrt{n}}$
```

### kurchi2 (n)

Función

Devuelve el coeficiente de curtosis una variable aleatoria  $Chi^2(n)$ , con  $n > 0$ .

La variable aleatoria  $Chi^2(n)$  equivale a una  $Gamma(n/2, 2)$ , por lo que cuando Maxima no tiene suficiente información para obtener el resultado, devuelve una forma nominal basada en el coeficiente de curtosis de la gamma.

```
(%i1) kurchi2(n);
(%o1) $\frac{n}{2}$
 kurgamma(-, 2)
(%i2) assume(n>0)$ kurchi2(n);
```

|       |    |
|-------|----|
|       | 12 |
| (%03) | -- |
|       | n  |

## rchi2\_algorithm

Variable opcional

Valor por defecto: ahrens\_cheng

Es el algoritmo seleccionado para simular variables aleatorias chi-cuadrado. Los métodos implementados son `ahrens_cheng` y `inverse`:

- `ahrens_cheng`, basado en la simulación aleatoria de variables gamma. Véase `rgamma_algorithm` para más detalles.
- `inverse`, basado en el método inverso.

Véase también `rchi2`.

$$\mathbf{rchi2} \ (n)$$

### Función

$$\mathbf{rchi2} \quad (n,m)$$

## Función

Devuelve un valor aleatorio  $Chi^2(n)$ , con  $n > 0$ . Llamando a `rchi2` con un segundo argumento  $m$ , se simulará una muestra aleatoria de tamaño  $m$ .

Hay dos algoritmos programados para esta función, el que se vaya a utilizar tendrá que ser seleccionado dándole un valor a la variable global `rchi2_algorithm`, cuyo valor por defecto es `ahrens_cheng`.

Véase también `rchi2_algorithm`.

$$\text{denf } (x, m, n)$$

### Función

Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria  $F(m, n)$ , con  $m, n > 0$ .

$$\mathbf{disf} \ (x,m,n)$$

### Función

Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria  $F(m, n)$ , con  $m, n > 0$ .

Esta función no tiene expresión compacta, por lo que es evaluada numéricamente si la variable global `numer` vale `true`, en caso contrario devuelve una forma nominal.

```
(%i1) disf(2,3,9/4);
 9
(%o1) disf(2, 3, -)
 4
(%i2) %,numer;
(%o2) 0.66756728179008
```

 $\mathbf{qf} \ (q,m,n)$ 

### Función

Devuelve el  $q$ -cuantil de una variable aleatoria  $F(m, n)$ , con  $m, n > 0$ ; en otras palabras, se trata de la inversa de **disf**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

Esta función no tiene expresión compacta, por lo que es evaluada numéricamente si la variable global `numer` vale `true`, en caso contrario devuelve una forma nominal.



```
(%i1) qf(2/5,sqrt(3),5);
 2
(%o1) qf(-, sqrt(3), 5)
 5
(%i2) %,numer;
(%o2) 0.518947838573693
```

**meanf** ( $m,n$ ) Función  
Devuelve la media de una variable aleatoria  $F(m,n)$ , con  $m > 0, n > 2$ .

**varf** ( $m,n$ ) Función  
Devuelve la varianza de una variable aleatoria  $F(m,n)$ , con  $m > 0, n > 4$ .

**stdf** ( $m,n$ ) Función  
Devuelve la desviación típica de una variable aleatoria  $F(m,n)$ , con  $m > 0, n > 4$ .

**skwf** ( $m,n$ ) Función  
Devuelve el coeficiente de asimetría de una variable aleatoria  $F(m,n)$ , con  $m > 0, n > 6$ .

**kurf** ( $m,n$ ) Función  
Devuelve el coeficiente de curtosis una variable aleatoria  $F(m,n)$ , con  $m > 0, n > 8$ .

**rf\_algorithm** Variable opcional  
Valor por defecto: **inverse**

Es el algoritmo seleccionado para simular variables aleatorias  $F$ . Los algoritmos programados son **ratio** y **inverse**:

- **ratio**, basado en el hecho de que si  $X$  es una variable aleatoria  $Chi^2(m)$  y  $Y$  es una  $Chi^2(n)$ , entonces

$$F = \frac{nX}{mY}$$

es una variable aleatoria  $F$  con  $m$  y  $n$  grados de libertad,  $F(m,n)$ .

- **inverse**, basado en el método inverso.

Véase también **rf**.

**rf** ( $m,n$ ) Función  
**rf** ( $m,n,k$ ) Función

Devuelve un valor aleatorio  $F(m,n)$ , con  $m,n > 0$ . Llamando a **rf** con un tercer argumento  $k$ , se simulará una muestra aleatoria de tamaño  $k$ .

Hay dos algoritmos programados para esta función, el que se vaya a utilizar tendrá que ser seleccionado dándole un valor a la variable global **rf\_algorithm**, cuyo valor por defecto es **inverse**.

Véase también **rf\_algorithm**.

**denexp** (*x,m*) Función

Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria exponencial  $Exp(m)$ , con  $m > 0$ .

La variable aleatoria  $Exp(m)$  equivale a una Weibull  $Wei(1,1/m)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la densidad de Weibull.

```
(%i1) denexp(x,m);
(%o1) denweibull(x, 1, -)
 m
(%i2) assume(x>0,m>0)$ denexp(x,m);
 - m x
(%o3) m %e
```

**disexp** (*x,m*) Función

Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria exponencial  $Exp(m)$ , con  $m > 0$ .

La variable aleatoria  $Exp(m)$  equivale a una Weibull  $Wei(1,1/m)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la distribución de Weibull.

```
(%i1) disexp(x,m);
(%o1) disweibull(x, 1, -)
 m
(%i2) assume(x>0,m>0)$ disexp(x,m);
 - m x
(%o3) 1 - %e
```

**qexp** (*q,m*) Función

Devuelve el  $q$ -cuantil de una variable aleatoria exponencial  $Exp(m)$ , con  $m > 0$ ; en otras palabras, se trata de la inversa de **disexp**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

La variable aleatoria  $Exp(m)$  equivale a una Weibull  $Wei(1,1/m)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en el cuantil de la Weibull.

```
(%i51) qexp(0.56,5);
(%o1) .1641961104139661
(%i2) qexp(0.56,m);
(%o2) qweibull(0.56, 1, -)
 m
```

**meanexp** (*m*) Función

Devuelve la media de una variable aleatoria exponencial  $Exp(m)$ , con  $m > 0$ .

La variable aleatoria  $Exp(m)$  equivale a una Weibull  $Wei(1,1/m)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la media de la Weibull.

```
(%i1) meanexp(m);
(%o1) meanweibull(1, -)
 1
 m
(%i2) assume(m>0)$ meanexp(m);
(%o3) 1
 -
 m
```

**varexp** (*m*)

Función

Devuelve la varianza de una variable aleatoria exponencial  $Exp(m)$ , con  $m > 0$ .

La variable aleatoria  $Exp(m)$  equivale a una Weibull  $Wei(1, 1/m)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la varianza de la Weibull.

```
(%i1) varexp(m);
(%o2) varweibull(1, -)
 1
 m
(%i3) assume(m>0)$ varexp(m);
(%o4) 1
 --
 2
 m
```

**stdexp** (*m*)

Función

Devuelve la desviación típica de una variable aleatoria exponencial  $Exp(m)$ , con  $m > 0$ .

La variable aleatoria  $Exp(m)$  equivale a una Weibull  $Wei(1, 1/m)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la desviación típica de la Weibull.

```
(%i1) stdexp(m);
(%o1) stdweibull(1, -)
 1
 m
(%i2) assume(m>0)$ stdexp(m);
(%o3) 1
 -
 m
```

**skwexp** (*m*)

Función

Devuelve el coeficiente de asimetría de una variable aleatoria exponencial  $Exp(m)$ , con  $m > 0$ .

La variable aleatoria  $Exp(m)$  equivale a una Weibull  $Wei(1, 1/m)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en el coeficiente de asimetría de la Weibull.

```
(%i1) skwexp(m);
1
```

```
(%o1) skwweibull(1, -)
 m
(%i2) assume(m>0)$ skwexp(m);
(%o3) 2
```

**kurexp** (*m*)

Función

Devuelve el coeficiente de curtosis una variable aleatoria exponencial  $Exp(m)$ , con  $m > 0$ .

La variable aleatoria  $Exp(m)$  equivale a una Weibull  $Wei(1, 1/m)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en el coeficiente de curtosis de la Weibull.

```
(%i1) kurexp(m);
 1
(%o1) kurweibull(1, -)
 m
(%i2) assume(m>0)$ kurexp(m);
(%o3) 6
```

**rexp\_algorithm**

Variable opcional

Valor por defecto: **inverse**

Es el algoritmo seleccionado para simular variables aleatoria exponenciales. Los algoritmos programados son **inverse**, **ahrens\_cheng** y **ahrens\_dieter**

- **inverse**, basado en el método inverso.
- **ahrens\_cheng**, basado en el hecho de que la variable aleatoria  $Exp(m)$  equivale a una  $Gamma(1, 1/m)$ . Véase **rgamma\_algorithm** para más detalles.
- **ahrens\_dieter**, basada en el algoritmo descrito en Ahrens, J.H. y Dieter, U. (1972) *Computer methods for sampling from the exponential and normal distributions..* Comm, ACM, 15, Oct., 873-882.

Véase también **rexp**.

**rexp** (*m*)

Función

**rexp** (*m,k*)

Función

Devuelve un valor aleatorio  $Exp(m)$ , con  $m > 0$ . Llamando a **rexp2** con un segundo argumento *k*, se simulará una muestra aleatoria de tamaño *k*.

Hay tres algoritmos programados para esta función, el que se vaya a utilizar tendrá que ser seleccionado dándole un valor a la variable global **rexp\_algorithm**, cuyo valor por defecto es **inverse**.

Véase también **rexp\_algorithm**.

**denlogn** (*x,m,s*)

Función

Devuelve el valor correspondiente a *x* de la función de densidad de una variable aleatoria lognormal  $log - N(m, s)$ , con  $s > 0$ .

**dislogn** ( $x, m, s$ ) Función

Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria lognormal  $\log - N(m, s)$ , con  $s > 0$ . Esta función se define en términos de la función de error, **erf**, de Maxima.

```
(%i1) assume(s>0)$ dislogn(x,m,s);
 log(x) - m
 erf(-----)
 sqrt(2) s 1
(%o2) ----- + -
 2 2
```

Véase también **erf**.

**qlogn** ( $q, m, s$ ) Función

Devuelve el  $q$ -cuantil de una variable aleatoria lognormal  $\log - N(m, s)$ , con  $s > 0$ ; en otras palabras, se trata de la inversa de **dislogn**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

**meanlogn** ( $m, s$ ) Función

Devuelve la media de una variable aleatoria lognormal  $\log - N(m, s)$ , con  $s > 0$ .

**varlogn** ( $m, s$ ) Función

Devuelve la varianza de una variable aleatoria lognormal  $\log - N(m, s)$ , con  $s > 0$ .

**stdlogn** ( $m, s$ ) Función

Devuelve la desviación típica de una variable aleatoria lognormal  $\log - N(m, s)$ , con  $s > 0$ .

**skwlogn** ( $m, s$ ) Función

Devuelve el coeficiente de asimetría de una variable aleatoria lognormal  $\log - N(m, s)$ , con  $s > 0$ .

**kurlogn** ( $m, s$ ) Función

Devuelve el coeficiente de curtosis una variable aleatoria lognormal  $\log - N(m, s)$ , con  $s > 0$ .

**rlogn** ( $m, s$ ) Función

**rlogn** ( $m, s, n$ ) Función

Devuelve un valor aleatorio  $\log - N(m, s)$ , con  $s > 0$ . Llamando a **rlogn** con un tercer argumento  $n$ , se simulará una muestra aleatoria de tamaño  $n$ .

Las variables lognormales se simulan mediante variables normales. Hay dos algoritmos programados para esta función, el que se vaya a utilizar tendrá que ser seleccionado dándole un valor a la variable global **rnormal\_algorithm**, cuyo valor por defecto es **box\_mueller**.

Véase también **rnormal\_algorithm**.

**dengamma** ( $x, a, b$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria  $\text{Gamma}(a, b)$ , con  $a, b > 0$ .

**disgamma** ( $x, a, b$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria  $\text{Gamma}(a, b)$ , con  $a, b > 0$ .

Esta función no tiene expresión compacta, por lo que es evaluada numéricamente si la variable global `numer` vale `true`, en caso contrario devuelve una forma nominal.

```
(%i1) disgamma(3,5,21);
(%o1) disgamma(3, 5, 21)
(%i2) %,numer;
(%o2) 4.402663157135039E-7
```

**qgamma** ( $q, a, b$ ) Función  
 Devuelve el  $q$ -cuantil de una variable aleatoria  $\text{Gamma}(a, b)$ , con  $a, b > 0$ ; en otras palabras, se trata de la inversa de **disgamma**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

**meangamma** ( $a, b$ ) Función  
 Devuelve la media de una variable aleatoria  $\text{Gamma}(a, b)$ , con  $a, b > 0$ .

**vargamma** ( $a, b$ ) Función  
 Devuelve la varianza de una variable aleatoria  $\text{Gamma}(a, b)$ , con  $a, b > 0$ .

**stdgamma** ( $a, b$ ) Función  
 Devuelve la desviación típica de una variable aleatoria  $\text{Gamma}(a, b)$ , con  $a, b > 0$ .

**skwgamma** ( $a, b$ ) Función  
 Devuelve el coeficiente de asimetría de una variable aleatoria  $\text{Gamma}(a, b)$ , con  $a, b > 0$ .

**kurgamma** ( $a, b$ ) Función  
 Devuelve el coeficiente de curtosis una variable aleatoria  $\text{Gamma}(a, b)$ , con  $a, b > 0$ .

**rgamma\_algorithm** Variable opcional

Valor por defecto: `ahrens_cheng`

Es el algoritmo seleccionado para simular random gamma variates. Algoritmos programados son `ahrens_cheng` y `inverse`

- **ahrens\_cheng**, se trata de una combinación de dos procedimientos, según sea el valor del parámetro  $a$ :

Para  $a \geq 1$ , Cheng, R.C.H. y Feast, G.M. (1979). *Some simple gamma variate generators*. Appl. Stat., 28, 3, 290-295.

Para  $0 < a < 1$ , Ahrens, J.H. y Dieter, U. (1974). *Computer methods for sampling from gamma, beta, poisson and binomial distributions*. Computing, 12, 223-246.

- **inverse**, basado en el método inverso.

Véase también **rgamma**.

**rgamma** ( $a, b$ ) Función  
**rgamma** ( $a, b, n$ ) Función

Devuelve un valor aleatorio  $\text{Gamma}(a, b)$ , con  $a, b > 0$ . Llamando a **rgamma** con un tercer argumento  $n$ , se simulará una muestra aleatoria de tamaño  $n$ .

Hay dos algoritmos programados para esta función, el que se vaya a utilizar tendrá que ser seleccionado dándole un valor a la variable global **rgamma\_algorithm**, cuyo valor por defecto es **ahrens\_cheng**.

Véase también **rgamma\_algorithm**.

**denbeta** ( $x, a, b$ ) Función

Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria  $\text{Beta}(a, b)$ , con  $a, b > 0$ .

**disbeta** ( $x, a, b$ ) Función

Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria  $\text{Beta}(a, b)$ , con  $a, b > 0$ .

Esta función no tiene expresión compacta, por lo que es evaluada numéricamente si la variable global **numer** vale **true**, en caso contrario devuelve una forma nominal.

```
(%i1) disgamma(1/3,15,2);
 1
(%o1) disgamma(-, 15, 2)
 3
(%i2) %,numer;
(%o2) 1.391214268475648E-24
```

**qbeta** ( $q, a, b$ ) Función

Devuelve el  $q$ -cuantil de una variable aleatoria  $\text{Beta}(a, b)$ , con  $a, b > 0$ ; en otras palabras, se trata de la inversa de **disbeta**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

**meanbeta** ( $a, b$ ) Función

Devuelve la media de una variable aleatoria  $\text{Beta}(a, b)$ , con  $a, b > 0$ .

**varbeta** ( $a, b$ ) Función

Devuelve la varianza de una variable aleatoria  $\text{Beta}(a, b)$ , con  $a, b > 0$ .

**stdbeta** ( $a, b$ ) Función

Devuelve la desviación típica de una variable aleatoria  $\text{Beta}(a, b)$ , con  $a, b > 0$ .

**skwbeta** ( $a, b$ ) Función

Devuelve el coeficiente de asimetría de una variable aleatoria  $\text{Beta}(a, b)$ , con  $a, b > 0$ .

**kurbeta** ( $a, b$ ) Función  
 Devuelve el coeficiente de curtosis de una variable aleatoria  $Beta(a, b)$ , con  $a, b > 0$ .

**rbeta\_algorithm** Variable opcional  
 Valor por defecto: **cheng**

Es el algoritmo seleccionado para simular random beta variates. Algoritmos programados son **cheng**, **inverse** y **ratio**

- **cheng**, es el algoritmo definido en Cheng, R.C.H. (1978). *Generating Beta Variates with Nonintegral Shape Parameters*. Communications of the ACM, 21:317-322
- **inverse**, basado en el método inverso.
- **ratio**, basado en el hecho de que si  $X$  es una variable aleatoria  $Gamma(a, 1)$  y  $Y$  es una  $Gamma(b, 1)$ , entonces el cociente  $X/(X + Y)$  se distribuye como una  $Beta(a, b)$ .

Véase también **rbeta**.

**rbeta** ( $a, b$ ) Función  
**rbeta** ( $a, b, n$ ) Función

Devuelve un valor aleatorio  $Beta(a, b)$ , con  $a, b > 0$ . Llamando a **rbeta** con un tercer argumento  $n$ , se simulará una muestra aleatoria de tamaño  $n$ .

Hay tres algoritmos programados para esta función, el que se vaya a utilizar tendrá que ser seleccionado dándole un valor a la variable global **rbeta\_algorithm**, cuyo valor por defecto es **cheng**.

Véase también **rbeta\_algorithm**.

**dencontu** ( $x, a, b$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria uniforme continua  $cUnif(a, b)$ , con  $a < b$ .

**discontu** ( $x, a, b$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria uniforme continua  $cUnif(a, b)$ , con  $a < b$ .

**qcontu** ( $q, a, b$ ) Función  
 Devuelve el  $q$ -cuantil de una variable aleatoria uniforme continua  $cUnif(a, b)$ , con  $a < b$ ; en otras palabras, se trata de la inversa de **discontu**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

**meancontu** ( $a, b$ ) Función  
 Devuelve la media de una variable aleatoria uniforme continua  $cUnif(a, b)$ , con  $a < b$ .

**varcontu** ( $a, b$ ) Función  
 Devuelve la varianza de una variable aleatoria uniforme continua  $cUnif(a, b)$ , con  $a < b$ .



- stdcontu** ( $a, b$ ) Función  
Devuelve la desviación típica de una variable aleatoria uniforme continua  $cUnif(a, b)$ , con  $a < b$ .
- skwcontu** ( $a, b$ ) Función  
Devuelve el coeficiente de asimetría de una variable aleatoria uniforme continua  $cUnif(a, b)$ , con  $a < b$ .
- kurcontu** ( $a, b$ ) Función  
Devuelve el coeficiente de curtosis una variable aleatoria uniforme continua  $cUnif(a, b)$ , con  $a < b$ .
- rcontu** ( $a, b$ ) Función  
**rcontu** ( $a, b, n$ ) Función  
Devuelve un valor aleatorio  $cUnif(a, b)$ , con  $a < b$ . Llamando a **rcontu** con un tercer argumento  $n$ , se simulará una muestra aleatoria de tamaño  $n$ .  
Esta función es una aplicación directa de la función **random** de Maxima.  
Véase también **random**.
- denlog** ( $x, a, b$ ) Función  
Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria logística  $log(a, b)$ , con  $b > 0$ .
- dislog** ( $x, a, b$ ) Función  
Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria logística  $log(a, b)$ , con  $b > 0$ .
- qlog** ( $q, a, b$ ) Función  
Devuelve el  $q$ -cuantil de una variable aleatoria logística  $log(a, b)$ , con  $b > 0$ ; en otras palabras, se trata de la inversa de **dislog**. El argumento  $q$  debe ser un número de  $[0, 1]$ .
- meanlog** ( $a, b$ ) Función  
Devuelve la media de una variable aleatoria logística  $log(a, b)$ , con  $b > 0$ .
- varlog** ( $a, b$ ) Función  
Devuelve la varianza de una variable aleatoria logística  $log(a, b)$ , con  $b > 0$ .
- stdlog** ( $a, b$ ) Función  
Devuelve la desviación típica de una variable aleatoria logística  $log(a, b)$ , con  $b > 0$ .
- skwlog** ( $a, b$ ) Función  
Devuelve el coeficiente de asimetría de una variable aleatoria logística  $log(a, b)$ , con  $b > 0$ .

- kurlog** ( $a, b$ ) Función  
Devuelve el coeficiente de curtosis una variable aleatoria logística  $\log(a, b)$ , con  $b > 0$ .
- rlog** ( $a, b$ ) Función  
**rlog** ( $a, b, n$ ) Función  
Devuelve un valor aleatorio  $\log(a, b)$ , con  $b > 0$ . Llamando a **rlog** con un tercer argumento  $n$ , se simulará una muestra aleatoria de tamaño  $n$ .  
Sólo se ha programado el método inverso.
- denpareto** ( $x, a, b$ ) Función  
Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria de Pareto  $Par(a, b)$ , con  $a, b > 0$ .
- dispareto** ( $x, a, b$ ) Función  
Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria de Pareto  $Par(a, b)$ , con  $a, b > 0$ .
- qpareto** ( $q, a, b$ ) Función  
Devuelve el  $q$ -cuantil de una variable aleatoria de Pareto  $Par(a, b)$ , con  $a, b > 0$ ; en otras palabras, se trata de la inversa de **dispareto**. El argumento  $q$  debe ser un número de  $[0, 1]$ .
- meanpareto** ( $a, b$ ) Función  
Devuelve la media de una variable aleatoria de Pareto  $Par(a, b)$ , con  $a > 1, b > 0$ .
- varpareto** ( $a, b$ ) Función  
Devuelve la varianza de una variable aleatoria de Pareto  $Par(a, b)$ , con  $a > 2, b > 0$ .
- stdpareto** ( $a, b$ ) Función  
Devuelve la desviación típica de una variable aleatoria de Pareto  $Par(a, b)$ , con  $a > 2, b > 0$ .
- skwpareto** ( $a, b$ ) Función  
Devuelve el coeficiente de asimetría de una variable aleatoria de Pareto  $Par(a, b)$ , con  $a > 3, b > 0$ .
- kurpareto** ( $a, b$ ) Función  
Devuelve el coeficiente de curtosis de una variable aleatoria de Pareto  $Par(a, b)$ , con  $a > 4, b > 0$ .
- rpareto** ( $a, b$ ) Función  
**rpareto** ( $a, b, n$ ) Función  
Devuelve un valor aleatorio  $Par(a, b)$ , con  $a > 0, b > 0$ . Llamando a **rpareto** con un tercer argumento  $n$ , se simulará una muestra aleatoria de tamaño  $n$ .  
Sólo se ha programado el método inverso.

- denweibull** ( $x, a, b$ ) Función  
Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria de Weibull  $Wei(a, b)$ , con  $a, b > 0$ .
- disweibull** ( $x, a, b$ ) Función  
Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria de Weibull  $Wei(a, b)$ , con  $a, b > 0$ .
- qweibull** ( $q, a, b$ ) Función  
Devuelve el  $q$ -cuantil de una variable aleatoria de Weibull  $Wei(a, b)$ , con  $a, b > 0$ ; en otras palabras, se trata de la inversa de **disweibull**. El argumento  $q$  debe ser un número de  $[0, 1]$ .
- meanweibull** ( $a, b$ ) Función  
Devuelve la media de una variable aleatoria de Weibull  $Wei(a, b)$ , con  $a, b > 0$ .
- varweibull** ( $a, b$ ) Función  
Devuelve la varianza de una variable aleatoria de Weibull  $Wei(a, b)$ , con  $a, b > 0$ .
- stdweibull** ( $a, b$ ) Función  
Devuelve la desviación típica de una variable aleatoria de Weibull  $Wei(a, b)$ , con  $a, b > 0$ .
- skwweibull** ( $a, b$ ) Función  
Devuelve el coeficiente de asimetría de una variable aleatoria de Weibull  $Wei(a, b)$ , con  $a, b > 0$ .
- kurweibull** ( $a, b$ ) Función  
Devuelve el coeficiente de curtosis una variable aleatoria de Weibull  $Wei(a, b)$ , con  $a, b > 0$ .
- rweibull** ( $a, b$ ) Función  
**rweibull** ( $a, b, n$ ) Función  
Devuelve un valor aleatorio  $Wei(a, b)$ , con  $a, b > 0$ . Llamando a **rweibull** con un tercer argumento  $n$ , se simulará una muestra aleatoria de tamaño  $n$ .  
Sólo se ha programado el método inverso.
- denrayleigh** ( $x, b$ ) Función  
Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria de Rayleigh  $Ray(b)$ , con  $b > 0$ .  
La variable aleatoria  $Ray(b)$  equivale a una Weibull  $Wei(2, 1/b)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la densidad de la Weibull.

```
(%i1) denrayleigh(x,b);
(%o1) denweibull(x, 2, -)
 b
(%i2) assume(x>0,b>0)$ denrayleigh(x,b);
 2 2
 - b x
(%o3) 2 b x %e
```

**disrayleigh** (*x,b*)

Función

Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria de Rayleigh  $Ray(b)$ , con  $b > 0$ .

La variable aleatoria  $Ray(b)$  equivale a una Weibull  $Wei(2, 1/b)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la distribución de la Weibull.

```
(%i1) disrayleigh(x,b);
(%o1) disweibull(x, 2, -)
 b
(%i2) assume(x>0,b>0)$ disrayleigh(x,b);
 2 2
 - b x
(%o3) 1 - %e
```

**qrayleigh** (*q,b*)

Función

Devuelve el  $q$ -cuantil de una variable aleatoria de Rayleigh  $Ray(b)$ , con  $b > 0$ ; en otras palabras, se trata de la inversa de **disrayleigh**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

La variable aleatoria  $Ray(b)$  equivale a una Weibull  $Wei(2, 1/b)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en los cuantiles de la Weibull.

```
(%i1) qrayleigh(0.99,b);
(%o1) qweibull(0.99, 2, -)
 b
(%i2) assume(x>0,b>0)$ qrayleigh(0.99,b);
 2.145966026289347
(%o3) -----
 b
```

**meanrayleigh** (*b*)

Función

Devuelve la media de una variable aleatoria de Rayleigh  $Ray(b)$ , con  $b > 0$ .

La variable aleatoria  $Ray(b)$  equivale a una Weibull  $Wei(2, 1/b)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la media de la Weibull.

```
(%i1) meanrayleigh(b);
```

$$\frac{1}{b}$$

```
(%o1) meanweibull(2, -)
(%i2) assume(b>0)$ meanrayleigh(b);
```

$$\frac{\sqrt{\pi}}{2 b}$$

```
(%o3)
```

**varrayleigh (b)**

Función

Devuelve la varianza de una variable aleatoria de Rayleigh  $Ray(b)$ , con  $b > 0$ .

La variable aleatoria  $Ray(b)$  equivale a una Weibull  $Wei(2, 1/b)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la varianza de la Weibull.

```
(%i1) varrayleigh(b);
```

$$\frac{1}{b}$$

```
(%o1) varweibull(2, -)
(%i2) assume(b>0)$ varrayleigh(b);
```

$$\frac{\pi}{4 b^2}$$

```
(%o3)
```

**stdrayleigh (b)**

Función

Devuelve la desviación típica de una variable aleatoria de Rayleigh  $Ray(b)$ , con  $b > 0$ .

La variable aleatoria  $Ray(b)$  equivale a una Weibull  $Wei(2, 1/b)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la desviación típica de la Weibull.

```
(%i1) stdrayleigh(b);
```

$$\frac{1}{b}$$

```
(%o1) stdweibull(2, -)
(%i2) assume(b>0)$ stdrayleigh(b);
```

$$\frac{\sqrt{\pi}}{2 b}$$

```
(%o3)
```

**skwrayleigh (b)**

Función

Devuelve el coeficiente de asimetría de una variable aleatoria de Rayleigh  $Ray(b)$ , con  $b > 0$ .

La variable aleatoria  $Ray(b)$  equivale a una Weibull  $Wei(2, 1/b)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en el coeficiente de asimetría de la Weibull.

```
(%i1) skwrayleigh(b);
```

$$\frac{1}{b}$$

```
(%o1) skwweibull(2, -)
```

```
(%i2) assume(b>0)$ skwrayleigh(b);
```

$$\frac{\frac{3}{2} \sqrt{\pi}}{4} - \frac{3 \sqrt{\pi}}{4}$$

```
(%o3)
```

$$\frac{\pi^{3/2}}{(1 - \frac{\pi}{4})}$$
**kurrayleigh** (*b*)

Función

Devuelve el coeficiente de curtosis una variable aleatoria de Rayleigh  $Ray(b)$ , con  $b > 0$ .

La variable aleatoria  $Ray(b)$  equivale a una Weibull  $Wei(2, 1/b)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en el coeficiente de curtosis de la Weibull.

```
(%i1) kurrayleigh(b);
```

$$\frac{1}{b}$$

```
(%o1) kurweibull(2, -)
```

```
(%i2) assume(b>0)$ kurrayleigh(b);
```

$$\frac{2}{3} - \frac{\pi}{16}$$

```
(%o3)
```

$$\frac{\pi^2}{(1 - \frac{\pi}{4})} - 3$$
**rrayleigh** (*b*)

Función

**rrayleigh** (*b,n*)

Función

Devuelve un valor aleatorio  $Ray(b)$ , con  $b > 0$ . Llamando a **rrayleigh** con un segundo argumento  $n$ , se simulará una muestra aleatoria de tamaño  $n$ .

Sólo se ha programado el método inverso.

**denlaplace** (*x,a,b*)

Función

Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria de Laplace  $Lap(a, b)$ , con  $b > 0$ .

**dislaplace** (*x,a,b*)

Función

Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria de Laplace  $Lap(a, b)$ , con  $b > 0$ .

- qlaplace** ( $q,a,b$ ) Función  
 Devuelve el  $q$ -cuantil de una variable aleatoria de Laplace  $Lap(a,b)$ , con  $b > 0$ ; en otras palabras, se trata de la inversa de **dislaplace**. El argumento  $q$  debe ser un número de  $[0, 1]$ .
- meanlaplace** ( $a,b$ ) Función  
 Devuelve la media de una variable aleatoria de Laplace  $Lap(a,b)$ , con  $b > 0$ .
- varlaplace** ( $a,b$ ) Función  
 Devuelve la varianza de una variable aleatoria de Laplace  $Lap(a,b)$ , con  $b > 0$ .
- stdlaplace** ( $a,b$ ) Función  
 Devuelve la desviación típica de una variable aleatoria de Laplace  $Lap(a,b)$ , con  $b > 0$ .
- skwlaplace** ( $a,b$ ) Función  
 Devuelve el coeficiente de asimetría de una variable aleatoria de Laplace  $Lap(a,b)$ , con  $b > 0$ .
- kurlaplace** ( $a,b$ ) Función  
 Devuelve el coeficiente de curtosis una variable aleatoria de Laplace  $Lap(a,b)$ , con  $b > 0$ .
- rlaplace** ( $a,b$ ) Función  
**rlaplace** ( $a,b,n$ ) Función  
 Devuelve un valor aleatorio  $Lap(a,b)$ , con  $b > 0$ . Llamando a **rlaplace** con un tercer argumento  $n$ , se simulará una muestra aleatoria de tamaño  $n$ .  
 Sólo se ha programado el método inverso.
- dencauchy** ( $x,a,b$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria de Cauchy  $Cau(a,b)$ , con  $b > 0$ .
- discauchy** ( $x,a,b$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria de Cauchy  $Cau(a,b)$ , con  $b > 0$ .
- qcauchy** ( $q,a,b$ ) Función  
 Devuelve el  $q$ -cuantil de una variable aleatoria de Cauchy  $Cau(a,b)$ , con  $b > 0$ ; en otras palabras, se trata de la inversa de **discauchy**. El argumento  $q$  debe ser un número de  $[0, 1]$ .
- rcauchy** ( $a,b$ ) Función  
**rcauchy** ( $a,b,n$ ) Función  
 Devuelve un valor aleatorio  $Cau(a,b)$ , con  $b > 0$ . Llamando a **rcauchy** con un tercer argumento  $n$ , se simulará una muestra aleatoria de tamaño  $n$ .  
 Sólo se ha programado el método inverso.

**dengumbel** ( $x, a, b$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de densidad de una variable aleatoria de Gumbel  $Gum(a, b)$ , con  $b > 0$ .

**disgumbel** ( $x, a, b$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria de Gumbel  $Gum(a, b)$ , con  $b > 0$ .

**qgumbel** ( $q, a, b$ ) Función  
 Devuelve el  $q$ -cuantil de una variable aleatoria de Gumbel  $Gum(a, b)$ , con  $b > 0$ ; en otras palabras, se trata de la inversa de **disgumbel**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

**meangumbel** ( $a, b$ ) Función  
 Devuelve la media de una variable aleatoria de Gumbel  $Gum(a, b)$ , con  $b > 0$ .  

```
(%i1) assume(b>0)$ meangumbel(a,b);
```

```
(%o2) %gamma b + a
```

 donde el símbolo **%gamma** representa la constante de Euler-Mascheroni. Véase también **%gamma**.

**vargumbel** ( $a, b$ ) Función  
 Devuelve la varianza de una variable aleatoria de Gumbel  $Gum(a, b)$ , con  $b > 0$ .

**stdgumbel** ( $a, b$ ) Función  
 Devuelve la desviación típica de una variable aleatoria de Gumbel  $Gum(a, b)$ , con  $b > 0$ .

**skwgumbel** ( $a, b$ ) Función  
 Devuelve el coeficiente de asimetría de una variable aleatoria de Gumbel  $Gum(a, b)$ , con  $b > 0$ .  

```
(%i1) assume(b>0)$ skwgumbel(a,b);
```

```
(%o2) 12 sqrt(6) zeta(3)
```

```

```

```
3
```

```
%pi
```

```
(%i3) numer:true$ skwgumbel(a,b);
```

```
(%o4) 1.139547099404649
```

 donde **zeta** representa la función zeta de Riemann.

**kurgumbel** ( $a, b$ ) Función  
 Devuelve el coeficiente de curtosis de una variable aleatoria de Gumbel  $Gum(a, b)$ , con  $b > 0$ .

**rgumbel** ( $a, b$ ) Función  
**rgumbel** ( $a, b, n$ ) Función  
 Devuelve un valor aleatorio  $Gum(a, b)$ , con  $b > 0$ . Llamando a **rgumbel** con un tercer argumento  $n$ , se simulará una muestra aleatoria de tamaño  $n$ .  
 Sólo se ha programado el método inverso.



### 48.3 Definiciones para distribuciones discretas

**denbinomial** ( $x, n, p$ ) Función

Devuelve el valor correspondiente a  $x$  de la función de probabilidad de una variable aleatoria binomial  $B(n, p)$ , con  $0 < p < 1$  y  $n$  entero positivo.

**disbinomial** ( $x, n, p$ ) Función

Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria binomial  $B(n, p)$ , con  $0 < p < 1$  y  $n$  entero positivo.

Esta función no tiene expresión compacta y se calcula numéricamente si la variable global **numer** vale **true**, en otro caso devuelve una expresión nominal.

```
(%i1) disbinomial(5,7,1/6);
 1
(%o1) disbinomial(5, 7, -)
 6
(%i2) disbinomial(5,7,1/6),numer;
(%o2) .9998713991769548
```

**qbinomial** ( $q, n, p$ ) Función

Devuelve el  $q$ -cuantil de una variable aleatoria binomial  $B(n, p)$ , con  $0 < p < 1$  y  $n$  entero positivo; en otras palabras, se trata de la inversa de **disbinomial**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

**meanbinomial** ( $n, p$ ) Función

Devuelve la media de una variable aleatoria binomial  $B(n, p)$ , con  $0 < p < 1$  y  $n$  entero positivo.

**varbinomial** ( $n, p$ ) Función

Devuelve la varianza de una variable aleatoria binomial  $B(n, p)$ , con  $0 < p < 1$  y  $n$  entero positivo.

**stdbinomial** ( $n, p$ ) Función

Devuelve la desviación típica de una variable aleatoria binomial  $B(n, p)$ , con  $0 < p < 1$  y  $n$  entero positivo.

**skwbinomial** ( $n, p$ ) Función

Devuelve el coeficiente de asimetría de una variable aleatoria binomial  $B(n, p)$ , con  $0 < p < 1$  y  $n$  entero positivo.

**kurbinomial** ( $n, p$ ) Función

Devuelve el coeficiente de curtosis de una variable aleatoria binomial  $B(n, p)$ , con  $0 < p < 1$  y  $n$  entero positivo.

**rbinomial\_algorithm** Variable opcional

Valor por defecto: **kachit**

Es el algoritmo seleccionado para simular variables binomiales. Los algoritmos programados son **kachit**, **bernoulli** y **inverse**:

- **kachit**, basado en el algoritmo descrito en Kachitvichyanukul, V. y Schmeiser, B.W. (1988) *Binomial Random Variate Generation*. Communications of the ACM, 31, Feb., 216.
- **bernoulli**, basado en la simulación de ensayos de Bernoulli.
- **inverse**, basado en el método inverso.

Véase también **rbinomial**.

**rbinomial** (*n,p*) Función  
**rbinomial** (*n,p,m*) Función

Devuelve un valor aleatorio  $B(n, p)$ , con  $0 < p < 1$  y  $n$  entero positivo. Llamando a **rbinomial** con un tercer argumento  $m$ , se simulará una muestra aleatoria de tamaño  $m$ .

Hay tres algoritmos programados para esta función, el que se vaya a utilizar tendrá que ser seleccionado dándole un valor a la variable global **rbinomial\_algorithm**, cuyo valor por defecto es **kachit**.

Véase también **rbinomial\_algorithm**.

**denpoisson** (*x,m*) Función

Devuelve el valor correspondiente a  $x$  de la función de probabilidad de una variable aleatoria de Poisson  $Poi(m)$ , con  $m > 0$ .

**dispoisson** (*x,m*) Función

Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria de Poisson  $Poi(m)$ , con  $m > 0$ .

Esta función no tiene expresión compacta y se calcula numéricamente si la variable global **numer** vale **true**, en otro caso devuelve una expresión nominal.

```
(%i1) dispoisson(3,5);
(%o1) dispoisson(3, 5)
(%i2) dispoisson(3,5),numer;
(%o2) .2650259152973617
```

**qpoisson** (*q,m*) Función

Devuelve el  $q$ -cuantil de una variable aleatoria de Poisson  $Poi(m)$ , con  $m > 0$ ; en otras palabras, se trata de la inversa de **dispoisson**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

**meanpoisson** (*m*) Función

Devuelve la media de una variable aleatoria de Poisson  $Poi(m)$ , con  $m > 0$ .

**varpoisson** (*m*) Función

Devuelve la varianza de una variable aleatoria de Poisson  $Poi(m)$ , con  $m > 0$ .

**stdpoisson** (*m*) Función

Devuelve la desviación típica de una variable aleatoria de Poisson  $Poi(m)$ , con  $m > 0$ .

**skwpoisson** ( $m$ ) Función  
 Devuelve el coeficiente de asimetría de una variable aleatoria de Poisson  $Poi(m)$ , con  $m > 0$ .

**kurpoisson** ( $m$ ) Función  
 Devuelve el coeficiente de curtosis de una variable aleatoria de Poisson  $Poi(m)$ , con  $m > 0$ .

**rpoisson\_algorithm** Variable opcional  
 Valor por defecto: **kachit**  
 Es el algoritmo seleccionado para simular variables de Poisson. Los algoritmos programados son **ahrens\_dieter** y **inverse**:  

- **ahrens\_dieter**, basado en el algoritmo descrito en Ahrens, J.H. and Dieter, U. (1982) *Computer Generation of Poisson Deviates From Modified Normal Distributions*. ACM Trans. Math. Software, 8, 2, June, 163-179.
- **inverse**, basado en el método inverso.

 Véase también **rpoisson**.

**rpoisson** ( $m$ ) Función  
**rpoisson** ( $m, n$ ) Función  
 Devuelve un valor aleatorio  $Poi(m)$ , con  $m > 0$ . Llamando a **rpoisson** con un segundo argumento  $n$ , se simulará una muestra aleatoria de tamaño  $n$ .  
 Hay dos algoritmos programados para esta función, el que se vaya a utilizar tendrá que ser seleccionado dándole un valor a la variable global **rpoisson\_algorithm**, cuyo valor por defecto es **ahrens\_dieter**.  
 Véase también **rpoisson\_algorithm**.

**denbernoulli** ( $x, p$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de probabilidad de una variable aleatoria de Bernoulli  $Ber(p)$ , con  $0 < p < 1$ .  
 La variable aleatoria  $Ber(p)$  equivale a una binomial  $B(1, p)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la función de probabilidad de la binomial.

```
(%i1) denbernoulli(1,p);
(%o1) denbinomial(1, 1, p)
(%i2) assume(0<p,p<1)$ denbernoulli(1,p);
(%o3) p
```

**disbernoulli** ( $x, p$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria de Bernoulli  $Ber(p)$ , con  $0 < p < 1$ .

**qbernoulli** ( $q, p$ ) Función  
 Devuelve el  $q$ -cuantil de una variable aleatoria de Bernoulli  $Ber(p)$ , con  $0 < p < 1$ ; en otras palabras, se trata de la inversa de **disbernoulli**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

**meanbernoulli** (*p*) Función

Devuelve la media de una variable aleatoria de Bernoulli  $Ber(p)$ , con  $0 < p < 1$ .

La variable aleatoria  $Ber(p)$  equivale a una binomial  $B(1, p)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la media de la binomial.

```
(%i1) meanbernoulli(p);
(%o1) meanbinomial(1, p)
(%i2) assume(0<p,p<1)$ meanbernoulli(p);
(%o3) p
```

**varbernoulli** (*p*) Función

Devuelve la varianza de una variable aleatoria de Bernoulli  $Ber(p)$ , con  $0 < p < 1$ .

La variable aleatoria  $Ber(p)$  equivale a una binomial  $B(1, p)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la varianza de la binomial.

```
(%i1) varbernoulli(p);
(%o1) varbinomial(1, p)
(%i2) assume(0<p,p<1)$ varbernoulli(p);
(%o3) (1 - p) p
```

**stdbernoulli** (*p*) Función

Devuelve la desviación típica de una variable aleatoria de Bernoulli  $Ber(p)$ , con  $0 < p < 1$ .

La variable aleatoria  $Ber(p)$  equivale a una binomial  $B(1, p)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en la desviación típica de la binomial.

```
(%i1) stdbernoulli(p);
(%o1) stdbinomial(1, p)
(%i2) assume(0<p,p<1)$ stdbernoulli(p);
(%o3) sqrt(1 - p) sqrt(p)
```

**skwbernoulli** (*p*) Función

Devuelve el coeficiente de asimetría de una variable aleatoria de Bernoulli  $Ber(p)$ , con  $0 < p < 1$ .

La variable aleatoria  $Ber(p)$  equivale a una binomial  $B(1, p)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en el coeficiente de asimetría de la binomial.

```
(%i1) skwbernoulli(p);
(%o1) skwbinomial(1, p)
(%i2) assume(0<p,p<1)$ skwbernoulli(p);
 1 - 2 p
(%o3) -----
 sqrt(1 - p) sqrt(p)
```

**kurbernoulli** (*p*) Función

Devuelve el coeficiente de curtosis una variable aleatoria de Bernoulli  $Ber(p)$ , con  $0 < p < 1$ .

La variable aleatoria  $Ber(p)$  equivale a una binomial  $B(1, p)$ , por lo que si Maxima no tiene suficiente información para calcular el resultado, devolverá una forma nominal basada en el coeficiente de curtosis de la binomial.

```
(%i1) kurbernoulli(p);
(%o1) kurbinomial(1, p)
(%i2) assume(0<p,p<1)$ kurbernoulli(p);
 1 - 6 (1 - p) p
(%o3) -----
 (1 - p) p
```

**rbernoulli** (*p*) Función**rbernoulli** (*p,n*) Función

Devuelve un valor aleatorio  $Ber(p)$ , con  $0 < p < 1$ . Llamando a **rbernoulli** con un segundo argumento *n*, se simulará una muestra aleatoria de tamaño *n*.

Es aplicación directa de la función **random** de Maxima.

Véase también **random**.

**dengeo** (*x,p*) Función

Devuelve el valor correspondiente a *x* de la función de probabilidad de una variable aleatoria geométrica  $Geo(p)$ , con  $0 < p < 1$ .

**disgeo** (*x,p*) Función

Devuelve el valor correspondiente a *x* de la función de distribución de una variable aleatoria geométrica  $Geo(p)$ , con  $0 < p < 1$ .

**qgeo** (*q,p*) Función

Devuelve el *q*-cuantil de una variable aleatoria geométrica  $Geo(p)$ , con  $0 < p < 1$ ; en otras palabras, se trata de la inversa de **disgeo**. El argumento *q* debe ser un número de  $[0, 1]$ .

**meangeo** (*p*) Función

Devuelve la media de una variable aleatoria geométrica  $Geo(p)$ , con  $0 < p < 1$ .

**vargo** (*p*) Función

Devuelve la varianza de una variable aleatoria geométrica  $Geo(p)$ , con  $0 < p < 1$ .

**stdgeo** (*p*) Función

Devuelve la desviación típica de una variable aleatoria geométrica  $Geo(p)$ , con  $0 < p < 1$ .

**skwgeo** (*p*) Función

Devuelve el coeficiente de asimetría de una variable aleatoria geométrica  $Geo(p)$ , con  $0 < p < 1$ .

**kurgeo** ( $p$ ) Función  
 Devuelve el coeficiente de curtosis de una variable aleatoria geométrica  $Geo(p)$ , con  $0 < p < 1$ .

**rgeo\_algorithm** Variable opcional  
 Valor por defecto: **bernoulli**  
 Es el algoritmo seleccionado para simular variables geométricas. Los algoritmos programados son **bernoulli**, **devroye** y **inverse**:

- **bernoulli**, basado en la simulación de ensayos de Bernoulli.
- **devroye**, basado en el algoritmo descrito en Devroye, L. (1986) *Non-Uniform Random Variate Generation*. Springer Verlag, p. 480.
- **inverse**, basado en el método inverso.

Véase también **rgeo**.

**rgeo** ( $p$ ) Función  
**rgeo** ( $p, n$ ) Función  
 Devuelve un valor aleatorio  $Geo(p)$ , con  $0 < p < 1$ . Llamando a **rgeo** con un segundo argumento  $n$ , se simulará una muestra aleatoria de tamaño  $n$ .  
 Hay tres algoritmos programados para esta función, el que se vaya a utilizar tendrá que ser seleccionado dándole un valor a la variable global **rgeo\_algorithm**, cuyo valor por defecto es **bernoulli**.  
 Véase también **rgeo\_algorithm**.

**dendiscu** ( $x, n$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de probabilidad de una variable aleatoria uniforme discreta  $dUnif(n)$ , con  $n$  entero positivo.

**disdiscu** ( $x, n$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria uniforme discreta  $dUnif(n)$ , con  $n$  entero positivo.

**qdiscu** ( $q, n$ ) Función  
 Devuelve el  $q$ -cuantil de una variable aleatoria uniforme discreta  $dUnif(n)$ , con  $n$  entero positivo; en otras palabras, se trata de la inversa de **disdiscu**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

**meandiscu** ( $n$ ) Función  
 Devuelve la media de una variable aleatoria uniforme discreta  $dUnif(n)$ , con  $n$  entero positivo.

**vardiscu** ( $n$ ) Función  
 Devuelve la varianza de una variable aleatoria uniforme discreta  $dUnif(n)$ , con  $n$  entero positivo.

- stddiscu** ( $n$ ) Función  
Devuelve la desviación típica de una variable aleatoria uniforme discreta  $dUnif(n)$ , con  $n$  entero positivo.
- skwdiscu** ( $n$ ) Función  
Devuelve el coeficiente de asimetría de una variable aleatoria uniforme discreta  $dUnif(n)$ , con  $n$  entero positivo.
- kurdiscu** ( $n$ ) Función  
Devuelve el coeficiente de curtosis de una variable aleatoria uniforme discreta  $dUnif(n)$ , con  $n$  entero positivo.
- rdiscu** ( $n$ ) Función  
**rdiscu** ( $n, m$ ) Función  
Devuelve un valor aleatorio  $dUnif(n)$ , con  $n$  entero positivo. Llamando a **rdiscu** con un segundo argumento  $m$ , se simulará una muestra aleatoria de tamaño  $m$ .  
Se trata de una aplicación directa de la función **random** de Maxima.  
Véase también **random**.
- denhypergeo** ( $x, n1, n2, n$ ) Función  
Devuelve el valor correspondiente a  $x$  de la función de probabilidad de una variable aleatoria hipergeométrica  $Hyp(n1, n2, n)$ , con  $n1$ ,  $n2$  y  $n$  enteros positivos y  $n \leq n1 + n2$ .
- dishypergeo** ( $x, n1, n2, n$ ) Función  
Devuelve el valor correspondiente a  $x$  de la función de distribución of una variable aleatoria hipergeométrica  $Hyp(n1, n2, n)$ , con  $n1$ ,  $n2$  y  $n$  enteros positivos y  $n \leq n1 + n2$ .
- qhypergeo** ( $q, n1, n2, n$ ) Función  
Devuelve el  $q$ -cuantil de una variable aleatoria hipergeométrica  $Hyp(n1, n2, n)$ , con  $n1$ ,  $n2$  y  $n$  enteros positivos y  $n \leq n1 + n2$ ; en otras palabras, se trata de la inversa de **dishypergeo**. El argumento  $q$  debe ser un número de  $[0, 1]$ .
- meanhypergeo** ( $n1, n2, n$ ) Función  
Devuelve la media de una variable aleatoria uniforme discreta  $Hyp(n1, n2, n)$ , con  $n1$ ,  $n2$  y  $n$  enteros positivos y  $n \leq n1 + n2$ .
- varhypergeo** ( $n1, n2, n$ ) Función  
Devuelve la varianza de una variable aleatoria hipergeométrica  $Hyp(n1, n2, n)$ , con  $n1$ ,  $n2$  y  $n$  enteros positivos y  $n \leq n1 + n2$ .
- stdhypergeo** ( $n1, n2, n$ ) Función  
Devuelve la desviación típica de una variable aleatoria hipergeométrica  $Hyp(n1, n2, n)$ , con  $n1$ ,  $n2$  y  $n$  enteros positivos y  $n \leq n1 + n2$ .

**skwhypergeo** ( $n1, n2, n$ ) Función  
 Devuelve el coeficiente de asimetría de una variable aleatoria hipergeométrica  $Hyp(n1, n2, n)$ , con  $n1$ ,  $n2$  y  $n$  enteros positivos y  $n \leq n1 + n2$ .

**kurhypergeo** ( $n1, n2, n$ ) Función  
 Devuelve el coeficiente de curtosis de una variable aleatoria hipergeométrica  $Hyp(n1, n2, n)$ , con  $n1$ ,  $n2$  y  $n$  enteros positivos y  $n \leq n1 + n2$ .

**rhypergeo\_algorithm** Variable opcional  
 Valor por defecto: **kachit**  
 Es el algoritmo seleccionado para simular variables hipergeométricas. Los algoritmos programados son **kachit** and **inverse**:

- **kachit**, basado en el algoritmo descrito en Kachitvichyanukul, V., Schmeiser, B.W. (1985) *Computer generation of hypergeometric random variates*. Journal of Statistical Computation and Simulation 22, 127-145.
- **inverse**, basado en el método inverso.

Véase también **rhypergeo**.

**rhypergeo** ( $n1, n2, n$ ) Función  
**rhypergeo** ( $n1, n2, n, m$ ) Función  
 Devuelve un valor aleatorio  $Hyp(n1, n2, n)$ , con  $n1$ ,  $n2$  y  $n$  enteros positivos y  $n \leq n1 + n2$ . Llamando a **rhypergeo** con un cuarto argumento  $m$ , se simulará una muestra aleatoria de tamaño  $m$ .  
 Hay dos algoritmos programados para esta función, el que se vaya a utilizar tendrá que ser seleccionado dándole un valor a la variable global **rhypergeo\_algorithm**, cuyo valor por defecto es **kachit**.  
 Véase también **rhypergeo\_algorithm**.

**dennegbinom** ( $x, n, p$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de probabilidad de una variable aleatoria binomial negativa  $NB(n, p)$ , con  $0 < p < 1$  y  $n$  entero positivo.

**disnegbinom** ( $x, n, p$ ) Función  
 Devuelve el valor correspondiente a  $x$  de la función de distribución de una variable aleatoria binomial negativa  $NB(n, p)$ , con  $0 < p < 1$  y  $n$  entero positivo.  
 Esta función no tiene expresión compacta y se calcula numéricamente si la variable global **numer** vale **true**, en otro caso devuelve una expresión nominal.

```
(%i1) disnegbinom(3,4,1/8);
 1
(%o1) disnegbinom(3, 4, -)
 8
(%i2) disnegbinom(3,4,1/8),numer;
(%o2) .006238937377929698
```



**qnegbinom** ( $q, n, p$ ) Función  
 Devuelve el  $q$ -cuantil de una variable aleatoria binomial negativa  $NB(n, p)$ , con  $0 < p < 1$  y  $n$  entero positivo; en otras palabras, se trata de la inversa de **disnegbinom**. El argumento  $q$  debe ser un número de  $[0, 1]$ .

**meannegbinom** ( $n, p$ ) Función  
 Devuelve la media de una variable aleatoria binomial negativa  $NB(n, p)$ , con  $0 < p < 1$  and  $n$  entero positivo.

**varnegbinom** ( $n, p$ ) Función  
 Devuelve la varianza de una variable aleatoria binomial negativa  $NB(n, p)$ , con  $0 < p < 1$  and  $n$  entero positivo.

**stdnegbinom** ( $n, p$ ) Función  
 Devuelve la desviación típica de una variable aleatoria binomial negativa  $NB(n, p)$ , con  $0 < p < 1$  and  $n$  entero positivo.

**skwnegbinom** ( $n, p$ ) Función  
 Devuelve el coeficiente de asimetría de una variable aleatoria binomial negativa  $NB(n, p)$ , con  $0 < p < 1$  and  $n$  entero positivo.

**kurnegbinom** ( $n, p$ ) Función  
 Devuelve el coeficiente de curtosis una variable aleatoria binomial negativa  $NB(n, p)$ , con  $0 < p < 1$  and  $n$  entero positivo.

**rnegbinom\_algorithm** Variable opcional  
 Valor por defecto: **bernoulli**  
 Es el algoritmo seleccionado para simular variable binomiales negativas. Los algoritmos programados son **devroye**, **bernoulli** y **inverse**:

- **devroye**, basado en el algoritmo descrito en Devroye, L. (1986) *Non-Uniform Random Variate Generation*. Springer Verlag, p. 480.
- **bernoulli**, basado en la simulación de ensayos de Bernoulli.
- **inverse**, basado en el método inverso.

Véase también **rnegbinom**.

**rnegbinoml** ( $n, p$ ) Función  
**rnegbinom** ( $n, p, m$ ) Función  
 Devuelve un valor aleatorio  $NB(n, p)$ , con  $0 < p < 1$  y  $n$  entero positivo. Llamando a **rnegbinoml** con un tercer argumento  $m$ , se simulará una muestra aleatoria de tamaño  $m$ .

Hay tres algoritmos programados para esta función, el que se vaya a utilizar tendrá que ser seleccionado dándole un valor a la variable global **rnegbinom\_algorithm**, cuyo valor por defecto es **bernoulli**.

Véase también **rnegbinom\_algorithm**.

## 49 eval\_string

### 49.1 Definiciones para eval\_string

**eval\_string** (*str*) Function

Parse the Maxima string *str* as a Maxima expression and evaluate it. *str* is a Maxima string. It may or may not have a terminator (dollar sign \$ or semicolon ;). Only the first expression is parsed and evaluated, if there is more than one.

Complain if *str* is not a Maxima string.

Examples:

```
(%i1) load("eval_string")$

(%i2) eval_string ("foo: 42; bar: foo^2 + baz");
(%o2) 42
(%i3) eval_string ("(foo: 42, bar: foo^2 + baz)");
(%o3) baz + 1764
```

To use this function write first load("eval\_string"). See also parse\_string.

**parse\_string** (*str*) Function

Parse the Maxima string *str* as a Maxima expression (do not evaluate it). *str* is a Maxima string. It may or may not have a terminator (dollar sign \$ or semicolon ;). Only the first expression is parsed, if there is more than one.

Complain if *str* is not a Maxima string.

Examples:

```
(%i1) load("eval_string")$

(%i2) parse_string ("foo: 42; bar: foo^2 + baz");
(%o2) foo : 42
(%i3) parse_string ("(foo: 42, bar: foo^2 + baz)");
(%o3) 2
 (foo : 42, bar : foo + baz)
```

To use this function write first load("eval\_string"). See also eval\_string.



## 50 f90

### 50.1 Definiciones para f90

**f90** (*expr*)

Function

The f90 command is an update to the original maxima fortran command. The primary difference is the way long lines are broken.

In the next example, notice how the fortran command breaks lines within symbols. The f90 command never breaks within a symbol.

```
(%i1) load("f90")$

(%i2) expr:expand((xxx+yyy+7)^4);
 4 3 3 2 2
(%o2) yyy + 4 xxx yyy + 28 yyy + 6 xxx yyy
 2 2 3 2
+ 84 xxx yyy + 294 yyy + 4 xxx yyy + 84 xxx yyy
 4 3 2
+ 588 xxx yyy + 1372 yyy + xxx + 28 xxx + 294 xxx
+ 1372 xxx + 2401
(%i3) fortran(expr);
 yyy**4+4*xxx*yyy**3+28*yyy**3+6*xxx**2*yyy**2+84*xxx*yyy**2+294*yy
1 y**2+4*xxx**3*yyy+84*xxx**2*yyy+588*xxx*yyy+1372*yyy+xxx**4+28*
2 xxx**3+294*xxx**2+1372*xxx+2401
(%o3)
 done
(%i4) f90(expr);
yyy**4+4*xxx*yyy**3+28*yyy**3+6*xxx**2*yyy**2+84*xxx*yyy**2+294* &
yyy**2+4*xxx**3*yyy+84*xxx**2*yyy+588*xxx*yyy+1372*yyy+xxx** &
4+28*xxx**3+294*xxx**2+1372*xxx+2401
(%o4)
 done
```

The f90 implementation was done as a quick hack. It is not a necessarily a good example upon which to base other language translations.

To use this function write first `load("f90")`.



## 51 ggf

### 51.1 Definiciones para ggf

#### GGFINFINITY

Option variable

Default variable: 3

This is an option variable for function `ggf`.

When computing the continued fraction of the generating function, a partial quotient having a degree (strictly) greater than *GGFINFINITY* will be discarded and the current convergent will be considered as the exact value of the generating function; most often the degree of all partial quotients will be 0 or 1; if you use a greater value, then you should give enough terms in order to make the computation accurate enough.

See also `ggf`.

#### GGFCFMAX

Option variable

Default variable: 3

This is an option variable for function `ggf`.

When computing the continued fraction of the generating function, if no good result has been found (see the *GGFINFINITY* flag) after having computed *GGFCFMAX* partial quotients, the generating function will be considered as not being a fraction of two polynomials and the function will exit. Put freely a greater value for more complicated generating functions.

See also `ggf`.

#### `ggf` (*l*)

Function

Compute the generating function (if it is a fraction of two polynomials) of a sequence, its first terms being given. *l* is a list of numbers.

The solution is returned as a fraction of two polynomials. If no solution has been found, it returns with `done`.

This function is controlled by global variables *GGFINFINITY* and *GGFCFMAX*. See also *GGFINFINITY* and *GGFCFMAX*.

To use this function write first `load("ggf")`.



## 52 ifactor

### 52.1 Definiciones para ifactor

|                                                  |                 |
|--------------------------------------------------|-----------------|
| <b>save_primes</b>                               | Option variable |
| Default value: nil                               |                 |
| Saves primes found.                              |                 |
| <b>primep_number_of_tests</b>                    | Option variable |
| Default value: 25                                |                 |
| Number of primep-test runs.                      |                 |
| <b>pollard_rho_limit</b>                         | Option variable |
| Default value: 10000                             |                 |
| Limit for pollard-rho factorization depth.       |                 |
| <b>pollard_rho_tests</b>                         | Option variable |
| Default value: 5                                 |                 |
| Number of pollard-rho rounds between ecm rounds. |                 |
| <b>pollard_rho_limit_step</b>                    | Option variable |
| Default value: 1000                              |                 |
| Step for pollard-rho factorization limit.        |                 |
| <b>ecm_number_of_curves</b>                      | Option variable |
| Default value: 50                                |                 |
| Number of curves tried in one round.             |                 |
| <b>ecm_limit</b>                                 | Option variable |
| Default value: 200                               |                 |
| Starting smootheness limit for ecm method.       |                 |
| <b>ecm_max_limit</b>                             | Option variable |
| Default value: 51199                             |                 |
| Maximum smootheness for ecm method.              |                 |
| <b>ecm_limit_delta</b>                           | Option variable |
| Default value: 200                               |                 |
| Increase of smoothness limit for ecm method.     |                 |
| <b>ifactor_verbose</b>                           | Option variable |
| Default value: nil                               |                 |
| Display factorization steps.                     |                 |



**ifactors** (*n*) Function

Returns a list of prime power factors of positive integer *n*.

```
(%i1) load("ifactor")$

(%i2) ifactors(23456345);
(%o2) [5, 11, 17, 25087]
```

To use this function write first `load("ifactor")`.

**ifactor** (*n*) Function

Factors positive integer *n*. **ifactor** uses Pollard's rho and elliptic curve factorization methods to factor integers.

```
(%i1) load("ifactor")$

(%i2) ifactor(23456345200);
 4 2
(%o2) 2 5 43 313 4357
```

To use this function write first `load("ifactor")`.

**primep\_pr** (*n*) Function

Probabilistic primality test. **primep\_pr** uses **primep\_pr\_number\_of\_tests** Miller-Rabin's pseudo-primality tests and one Lucas pseudo-primality test.

```
(%i1) load("ifactor")$

(%i2) primep_pr(23456345);
(%o2) false
```

To use this function write first `load("ifactor")`.

**next\_prime** (*n*) Function

Get smallest prime bigger than *n*.

```
(%i1) load("ifactor")$

(%i2) next_prime(27);
(%o2) 29
```

To use this function write first `load("ifactor")`.

**prev\_prime** (*n*) Function

Get greatest prime smaller than *n*.

```
(%i1) load("ifactor")$

(%i2) prev_prime(27);
(%o2) 23
```

To use this function write first `load("ifactor")`.

## 53 impdiff

### 53.1 Definiciones para impdiff

**implicit\_derivative** (*f,indvarlist,orderlist,depvar*)

Function

This subroutine computes implicit derivatives of multivariable functions. *f* is an array function, the indexes are the derivative degree in the *indvarlist* order; *indvarlist* is the independent variable list; *orderlist* is the order desired; and *depvar* is the dependent variable.

To use this function write first `load("impdiff")`.



## 54 interpol

### 54.1 Introducción a interpol

El paquete `interpol` desarrolla los métodos de interpolación polinómica de Lagrange, lineal y de *splines* cúbicos.

Para comentarios, fallos o sugerencias, contactar con '`mario ARROBA edu PUNTO xunta PUNTO es`'.

### 54.2 Definiciones para interpol

**lagrange** (*points*)

Función

**lagrange** (*points, option*)

Función

Calcula el polinomio de interpolación por el método de Lagrange. El argumento *points* debe ser:

- una matriz de dos columnas, `p:matrix([2,4],[5,6],[9,3])`,
- una lista de pares de números, `p: [[2,4],[5,6],[9,3]]`,
- una lista de números, `p: [4,6,3]`, en cuyo caso las abscisas se asignarán automáticamente a 1, 2, 3, etc.

En los dos primeros casos los pares se ordenan con respecto a la primera coordenada antes de proceder a los cálculos.

Mediante el argumento *option* es posible seleccionar el nombre de la variable independiente, que por defecto es 'x'; para definir otra, escríbase algo como `varname='z`.

Ejemplos:

```
(%i1) load("interpol")$
(%i2) p:[[7,2],[8,2],[1,5],[3,2],[6,7]]$
(%i3) lagrange(p);
 4 3 2
 73 x - 1402 x + 8957 x - 21152 x + 15624
(%o3) -----
 420

(%i4) f(x):='';
 4 3 2
 73 x - 1402 x + 8957 x - 21152 x + 15624
(%o4) f(x) := -----
 420

(%i5) /* Evaluate the polynomial at some points */
 map(f,[2.3,5/7,%pi]);
 919062
(%o5) [- 1.5675350000000005, -----,
 84035
 4 3 2
 73 %pi - 1402 %pi + 8957 %pi - 21152 %pi + 15624
 -----]
 420
```

```
(%i6) %,numer;
(%o6) [- 1.5675350000000005, 10.9366573451538,
2.89319655125692]
(%i7) /* Plot the polynomial together with points */
plot2d([f(x),[discrete,p]], [x,0,10],
[gnuplot_curve_styles,
["with lines","with points pointsize 3"]])$
(%i8) /* Change variable name */
lagrange(p, varname=w);
4 3 2
73 w - 1402 w + 8957 w - 21152 w + 15624
(%o8) -----
420
```

**charfun2** (*x*, *a*, *b*)

Función

Devuelve **true** si el número *x* pertenece al intervalo  $[a, b)$ , y **false** en caso contrario.

**linearinterpol** (*points*)

Función

**linearinterpol** (*points*, *option*)

Función

Calcula rectas de interpolación. El argumento *points* debe ser:

- una matriz de dos columnas, *p*: `matrix([2,4],[5,6],[9,3])`,
- una lista de pares de números, *p*: `[[2,4],[5,6],[9,3]]`,
- una lista de números, *p*: `[4,6,3]`, en cuyo caso las abscisas se asignarán automáticamente a 1, 2, 3, etc.

En los dos primeros casos los pares se ordenan con respecto a la primera coordenada antes de proceder a los cálculos.

Mediante el argumento *option* es posible seleccionar el nombre de la variable independiente, que por defecto es 'x'; para definir otra, escríbase algo como `varname='z`.

Ejemplos:

```
(%i1) load("interpol")$
(%i2) p: matrix([7,2],[8,3],[1,5],[3,2],[6,7])$
(%i3) linearinterpol(p);
(%o3) - ((9 x - 39) charfun2(x, minf, 3)
+ (30 - 6 x) charfun2(x, 7, inf)
+ (30 x - 222) charfun2(x, 6, 7)
+ (18 - 10 x) charfun2(x, 3, 6))/6
(%i4) f(x):='';
(%o4) f(x) := - ((9 x - 39) charfun2(x, minf, 3)
+ (30 - 6 x) charfun2(x, 7, inf)
+ (30 x - 222) charfun2(x, 6, 7)
+ (18 - 10 x) charfun2(x, 3, 6))/6
(%i5) /* Evaluate the polynomial at some points */
map(f,[7.3,25/7,%pi]);
(%o5) 62 18 - 10 %pi
[2.3, --, - ----]
 21 6
```

```
(%i6) %,numer;
(%o6) [2.3, 2.952380952380953, 2.235987755982988]
(%i7) /* Plot the polynomial together with points */
 plot2d(['(f(x))',[discrete,args(p)]],[x,-5,20],
 [gnuplot_curve_styles,
 ["with lines","with points pointsize 3"]])$
(%i8) /* Change variable name */
 linearinterp(p, varname='s');
(%o8) - ((9 s - 39) charfun2(s, minf, 3)
 + (30 - 6 s) charfun2(s, 7, inf)
 + (30 s - 222) charfun2(s, 6, 7)
 + (18 - 10 s) charfun2(s, 3, 6))/6
```

**cspline** (*points*)

Función

**cspline** (*points, option1, option2, ...*)

Función

Calcula el polinomio de interpolación por el método de los *splines* cúbicos. El argumento *points* debe ser:

- una matriz de dos columnas, *p*: `matrix([2,4],[5,6],[9,3])`,
- una lista de pares de números, *p*: `[[2,4],[5,6],[9,3]]`,
- una lista de números, *p*: `[4,6,3]`, en cuyo caso las abscisas se asignarán automáticamente a 1, 2, 3, etc.

En los dos primeros casos los pares se ordenan con respecto a la primera coordenada antes de proceder a los cálculos.

Esta función dispone de tres opciones para acomodarse a necesidades concretas:

- '*d1*', por defecto '*unknown*', es la primera derivada en  $x_1$ ; si toma el valor '*unknown*', la segunda derivada en  $x_1$  se iguala a 0 (*spline* cúbico natural); en caso de tomar un valor numérico, la segunda derivada se calcula en base a este número.
- '*dn*', por defecto '*unknown*', es la primera derivada en  $x_n$ ; si toma el valor '*unknown*', la segunda derivada en  $x_n$  se iguala a 0 (*spline* cúbico natural); en caso de tomar un valor numérico, la segunda derivada se calcula en base a este número.
- '*varname*', por defecto '*x*', es el nombre de la variable independiente.

Ejemplos:

```
(%i1) load("interp1")$
(%i2) p:[[7,2],[8,2],[1,5],[3,2],[6,7]]$
(%i3) /* Unknown first derivatives at the extremes
 is equivalent to natural cubic splines */
 cspline(p);
 3 2
(%o3) ((3477 x - 10431 x - 18273 x + 74547)
 3 2
 charfun2(x, minf, 3) + (- 15522 x + 372528 x - 2964702 x
 + 7842816) charfun2(x, 7, inf)
 3 2
```

```

+ (28290 x3 - 547524 x2 + 3475662 x - 7184700)
charfun2(x, 6, 7) + (- 6574 x3 + 80028 x2 - 289650 x
+ 345924) charfun2(x, 3, 6))/9864
(%i4) f(x):=''%$
(%i5) /* Some evaluations */
 map(f,[2.3,5/7,%pi]), numer;
(%o5) [1.991460766423358, 5.823200187269904,
 2.227405312429501]
(%i6) /* Plotting interpolating function */
 plot2d(['(f(x))',[discrete,p]], [x,0,10],
 [gnuplot_curve_styles,
 ["with lines","with points pointsize 3"]])$
(%i7) /* New call, but giving values at the derivatives */
 cspline(p,d1=0,dn=0);
 3 2
(%o7) ((17541 x3 - 102933 x2 + 153243 x + 33669)
 3 2
charfun2(x, minf, 3) + (- 55692 x3 + 1280916 x2 - 9801792 x
+ 24990624) charfun2(x, 7, inf)
 3 2
+ (65556 x3 - 1265292 x2 + 8021664 x - 16597440)
 3 2
charfun2(x, 6, 7) + (- 15580 x3 + 195156 x2 - 741024 x
+ 927936) charfun2(x, 3, 6))/20304
(%i8) /* Defining new interpolating function */
 g(x):=''%$
(%i9) /* Plotting both functions together */
 plot2d(['(f(x))','(g(x))',[discrete,p]], [x,0,10],
 [gnuplot_curve_styles,
 ["with lines","with lines","with points pointsize 3"]])$

```

## 55 lindstedt

### 55.1 Definiciones para lindstedt

**Lindstedt** (*eq,pvar,torder,ic*)

Function

This is a first pass at a Lindstedt code. It can solve problems with initial conditions entered, which can be arbitrary constants, (just not %k1 and %k2) where the initial conditions on the perturbation equations are  $z[i] = 0, z'[i] = 0$  for  $i > 0$ . *ic* is the list of initial conditions.

Problems occur when initial conditions are not given, as the constants in the perturbation equations are the same as the zero order equation solution. Also, problems occur when the initial conditions for the perturbation equations are not  $z[i] = 0, z'[i] = 0$  for  $i > 0$ , such as the Van der Pol equation.

Example:

```
(%i1) load("makeOrders")$

(%i2) load("lindstedt")$

(%i3) Lindstedt('diff(x,t,2)+x-(e*x^3)/6,e,2,[1,0]);
 2
 e (cos(5 T) - 24 cos(3 T) + 23 cos(T))
(%o3) [[-----
 36864
 e (cos(3 T) - cos(T))
 - ----- + cos(T)],
 192
 2
 7 e e
T = (- ---- - -- + 1) t]]
 3072 16
```

To use this function write first `load("makeOrders")` and `load("lindstedt")`.





## 56 linearalgebra

### 56.1 Introducción a linearalgebra

linearalgebra is a collection of functions for linear algebra.

Example:

```
(%i1) load (linearalgebra);
Warning - you are redefining the Maxima function require_list
Warning - you are redefining the Maxima function matrix_size
Warning - you are redefining the Maxima function rank
(%o1) /usr/local/share/maxima/5.9.2/share/linearalgebra/linearalgebra.mac
(%i2) M : matrix ([1, 2], [1, 2]);
 [1 2]
(%o2) []
 [1 2]

(%i3) nullspace (M);
 [1]
 []
(%o3) span([1])
 [- -]
 [2]

(%i4) columnspace (M);
 [1]
(%o4) span([])
 [1]

(%i5) ptriangularize (M - z*ident(2), z);
 [1 2 - z]
(%o5) []
 [2]
 [0 3 z - z]

(%i6) M : matrix ([1, 2, 3], [4, 5, 6], [7, 8, 9]) - z*ident(3);
 [1 - z 2 3]
 []
(%o6) [4 5 - z 6]
 []
 [7 8 9 - z]

(%i7) MM : ptriangularize (M, z);
 [4 5 - z 6]
 []
 [2]
 [66 z 102 z 132]
 [0 -- - -- + ----- + ---]
(%o7) [49 7 49 49]
 []
 [3 2]
 [49 z 245 z 147 z]
 [0 0 ----- - ----- - -----]
 [264 88 44]
```

```

(%i8) algebraic : true;
(%o8) true
(%i9) tellrat (MM [3, 3]);
(%o9)
$$[z^3 - 15z^2 - 18z]$$

(%i10) MM : ratsimp (MM);
(%o10)
$$\begin{bmatrix} 4 & 5 - z & 6 \\ 0 & 66 & 7z^2 - 102z - 132 \\ 0 & 49 & 49 \end{bmatrix}$$

(%i11) nullspace (MM);
(%o11)
$$\text{span}\left(\begin{bmatrix} 1 \\ z^2 - 14z - 16 \\ 8 \end{bmatrix}, \begin{bmatrix} 2 \\ z^2 - 18z - 12 \\ 12 \end{bmatrix}\right)$$

(%i12) M : matrix ([1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]);
(%o12)
$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

(%i13) columnspace (M);
(%o13)
$$\text{span}\left(\begin{bmatrix} 1 \\ 5 \\ 9 \\ 13 \end{bmatrix}, \begin{bmatrix} 2 \\ 6 \\ 10 \\ 14 \end{bmatrix}\right)$$

(%i14) apply ('orthogonal_complement, args (nullspace (transpose (M))));
(%o14)
$$\text{span}\left(\begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}\right)$$


```

$$\begin{bmatrix} 3 \\ -2 \end{bmatrix}$$

## 56.2 Definiciones para linearalgebra

### **addmatrices** ( $f, M_1, \dots, M_n$ )

Function

Using the function  $f$  as the addition function, return the sum of the matrices  $M_1, \dots, M_n$ . The function  $f$  must accept any number of arguments (a Maxima nary function).

Examples:

```
(%i1) m1 : matrix([1,2],[3,4])$
(%i2) m2 : matrix([7,8],[9,10])$
(%i3) addmatrices('max,m1,m2);
(%o3) matrix([7,8],[9,10])
(%i4) addmatrices('max,m1,m2,5*m1);
(%o4) matrix([7,10],[15,20])
```

### **blockmatrixp** ( $M$ )

Function

Return true if and only if  $M$  is a matrix and every entry of  $M$  is a matrix.

### **columnop** ( $M, i, j, \theta$ )

Function

If  $M$  is a matrix, return the matrix that results from doing the column operation  $C_i \leftarrow C_i - \theta * C_j$ . If  $M$  doesn't have a row  $i$  or  $j$ , signal an error.

### **columnswap** ( $M, i, j$ )

Function

If  $M$  is a matrix, swap columns  $i$  and  $j$ . If  $M$  doesn't have a column  $i$  or  $j$ , signal an error.

### **columnspace** ( $M$ )

Function

If  $M$  is a matrix, return **span** ( $v_1, \dots, v_n$ ), where the set  $\{v_1, \dots, v_n\}$  is a basis for the column space of  $M$ . The span of the empty set is  $\{0\}$ . Thus, when the column space has only one member, return **span** ().

### **copy** ( $e$ )

Function

Return a copy of the Maxima expression  $e$ . Although  $e$  can be any Maxima expression, the copy function is the most useful when  $e$  is either a list or a matrix; consider: load (linearalgebra);  $m : [1, [2, 3]]$   $mm : m$   $mm[2][1] : x$   $m; mm;$

```
(%i1) load("linearalgebra")$
(%i2) m : [1,[2,3]]$
(%i3) mm : m$
(%i4) mm[2][1] : x$
(%i5) m;
(%o5) [1,[x,3]]
(%i6) mm;
(%o6) [1,[x,3]]
```

Let's try the same experiment, but this time let  $mm$  be a copy of  $m$   $m : [1, [2, 3]]$   $mm : copy(m)$   $mm[2][1] : x$   $m; mm;$

```
(%i7) m : [1,[2,3]]$
(%i8) mm : copy(m)$
(%i9) mm[2][1] : x$
(%i10) m;
(%o10) [1,[2,3]]
(%i11) mm;
(%o11) [1,[x,3]]
```

This time, the assignment to *mm* does not change the value of *m*.

**cholesky** (*M*) Function

**cholesky** (*M*, *field*) Function

Return the Cholesky factorization of the matrix selfadjoint (or hermitian) matrix *M*. The second argument defaults to 'generalring.' For a description of the possible values for *field*, see **lu\_factor**.

**ctranspose** (*M*) Function

Return the complex conjugate transpose of the matrix *M*. The function **ctranspose** uses **matrix\_element\_transpose** to transpose each matrix element.

**diag\_matrix** (*d\_1, d\_2,...,d\_n*) Function

Return a diagonal matrix with diagonal entries *d\_1, d\_2,...,d\_n*. When the diagonal entries are matrices, the zero entries of the returned matrix are zero matrices of the appropriate size; for example:

```
(%i1) load(linearalgebra)$

(%i2) diag_matrix(diag_matrix(1,2),diag_matrix(3,4));

 [[1 0] [0 0]]
 [[] []]
 [[0 2] [0 0]]
(%o2) [[]
 [[0 0] [3 0]]
 [[] []]
 [[0 0] [0 4]]]
(%i3) diag_matrix(p,q);

 [p 0]
(%o3) []
 [0 q]
```

**dotproduct** (*u*, *v*) Function

Return the dotproduct of vectors *u* and *v*. This is the same as **conjugate (transpose (u)) . v**. The arguments *u* and *v* must be column vectors.

**get\_lu\_factors** (*x*) Function

When *x* = **lu\_factor** (*A*), then **get\_lu\_factors** returns a list of the form [*P*, *L*, *U*], where *P* is a permutation matrix, *L* is lower triangular with ones on the diagonal, and *U* is upper triangular, and *A* = *P L U*.

- hankel** (*col*) Function  
**hankel** (*col*, *row*) Function  
 Return a Hankel matrix  $H$ . The first column of  $H$  is *col*; except for the first entry, the last row of  $H$  is *row*. The default for *row* is the zero vector with the same length as *col*.
- hessian** (*f*, *vars*) Function  
 Return the hessian matrix of  $f$  with respect to the variables in the list *vars*. The  $i,j$  entry of the hessian matrix is  $\text{diff}(f \text{ vars}[i], 1, \text{vars}[j], 1)$ .
- hilbert\_matrix** (*n*) Function  
 Return the  $n$  by  $n$  Hilbert matrix. When  $n$  isn't a positive integer, signal an error.
- identfor** (*M*) Function  
**identfor** (*M*, *fld*) Function  
 Return an identity matrix that has the same shape as the matrix  $M$ . The diagonal entries of the identity matrix are the multiplicative identity of the field *fld*; the default for *fld* is *generalring*.  
 The first argument  $M$  should be a square matrix or a non-matrix. When  $M$  is a matrix, each entry of  $M$  can be a square matrix – thus  $M$  can be a blocked Maxima matrix. The matrix can be blocked to any (finite) depth.  
 See also **zerofor**
- invert\_by\_lu** (*M*, (*rng* *generalring*)) Function  
 Invert a matrix  $M$  by using the LU factorization. The LU factorization is done using the ring *rng*.
- kronecker\_product** (*A*, *B*) Function  
 Return the Kronecker product of the matrices  $A$  and  $B$ .
- listp** (*e*, *p*) Function  
**listp** (*e*) Function  
 Given an optional argument  $p$ , return **true** if  $e$  is a Maxima list and  $p$  evaluates to **true** for every list element. When **listp** is not given the optional argument, return **true** if  $e$  is a Maxima list. In all other cases, return **false**.
- locate\_matrix\_entry** (*M*, *r-1*, *c-1*, *r-2*, *c-2*, *f*, *rel*) Function  
 The first argument must be a matrix; the arguments *r-1* through *c-2* determine a sub-matrix of  $M$  that consists of rows *r-1* through *r-2* and columns *c-1* through *c-2*. Find a entry in the sub-matrix  $M$  that satisfies some property. Three cases:  
 (1) *rel* = 'bool and  $f$  a predicate:  
 Scan the sub-matrix from left to right then top to bottom, and return the index of the first entry that satisfies the predicate  $f$ . If no matrix entry satisfies  $f$ , return false.  
 (2) *rel* = 'max and  $f$  real-valued:

Scan the sub-matrix looking for an entry that maximizes  $f$ . Return the index of a maximizing entry.

(3)  $rel = 'min$  and  $f$  real-valued:

Scan the sub-matrix looking for an entry that minimizes  $f$ . Return the index of a minimizing entry.

### **lu\_backsub** ( $M, b$ )

Function

When  $M = lu\_factor(A, field)$ , then  $lu\_backsub(M, b)$  solves the linear system  $Ax = b$ .

### **lu\_factor** ( $M, field$ )

Function

Return a list of the form  $[LU, perm, fld]$ , or  $[LU, perm, fld, lower-cnd, upper-cnd]$ , where

(1) The matrix  $LU$  contains the factorization of  $M$  in a packed form. Packed form means three things: First, the rows of  $LU$  are permuted according to the list  $perm$ . If, for example,  $perm$  is the list  $[3, 2, 1]$ , the actual first row of the  $LU$  factorization is the third row of the matrix  $LU$ . Second, the lower triangular factor of  $m$  is the lower triangular part of  $LU$  with the diagonal entries replaced by all ones. Third, the upper triangular factor of  $M$  is the upper triangular part of  $LU$ .

(2) When the field is either `floatfield` or `complexfield`, the numbers *lower-cnd* and *upper-cnd* are lower and upper bounds for the infinity norm condition number of  $M$ . For all fields, the condition number might not be estimated; for such fields, `lu_factor` returns a two item list. Both the lower and upper bounds can differ from their true values by arbitrarily large factors. (See also `mat_cond`.)

The argument  $M$  must be a square matrix.

The optional argument  $fld$  must be a symbol that determines a ring or field. The pre-defined fields and rings are:

(a) `generalring` – the ring of Maxima expressions, (b) `floatfield` – the field of floating point numbers of the type double, (c) `complexfield` – the field of complex floating point numbers of the type double, (d) `crering` – the ring of Maxima CRE expressions, (e) `rationalfield` – the field of rational numbers, (f) `runningerror` – track the all floating point rounding errors, (g) `noncommutingring` – the ring of Maxima expressions where multiplication is the non-commutative dot operator.

When the field is `floatfield`, `complexfield`, or `runningerror`, the algorithm uses partial pivoting; for all other fields, rows are switched only when needed to avoid a zero pivot.

Floating point addition arithmetic isn't associative, so the meaning of 'field' differs from the mathematical definition.

A member of the field `runningerror` is a two member Maxima list of the form  $[x, n]$ , where  $x$  is a floating point number and  $n$  is an integer. The relative difference between the 'true' value of  $x$  and  $x$  is approximately bounded by the machine epsilon times  $n$ . The running error bound drops some terms that of the order the square of the machine epsilon.

There is no user-interface for defining a new field. A user that is familiar with Common Lisp should be able to define a new field. To do this, a user must define functions for

the arithmetic operations and functions for converting from the field representation to Maxima and back. Additionally, for ordered fields (where partial pivoting will be used), a user must define functions for the magnitude and for comparing field members. After that all that remains is to define a Common Lisp structure `mring`. The file `mring` has many examples.

To compute the factorization, the first task is to convert each matrix entry to a member of the indicated field. When conversion isn't possible, the factorization halts with an error message. Members of the field needn't be Maxima expressions. Members of the `complexfield`, for example, are Common Lisp complex numbers. Thus after computing the factorization, the matrix entries must be converted to Maxima expressions.

See also `get_lu_factors`.

Examples:

```
(%i1) load (linearalgebra);
Warning - you are redefining the Maxima function require_list
Warning - you are redefining the Maxima function matrix_size
Warning - you are redefining the Maxima function rank
(%o1) /usr/local/share/maxima/5.9.2/share/linearalgebra/linearalgebra.mac
(%i2) w[i,j] := random (1.0) + %i * random (1.0);
(%o2) w := random(1.) + %i random(1.)
 i, j
(%i3) showtime : true$
Evaluation took 0.00 seconds (0.00 elapsed)
(%i4) M : genmatrix (w, 100, 100)$
Evaluation took 7.40 seconds (8.23 elapsed)
(%i5) lu_factor (M, complexfield)$
Evaluation took 28.71 seconds (35.00 elapsed)
(%i6) lu_factor (M, generalring)$
Evaluation took 109.24 seconds (152.10 elapsed)
(%i7) showtime : false$

(%i8) M : matrix ([1 - z, 3], [3, 8 - z]);
 [1 - z 3]
(%o8) []
 [3 8 - z]
(%i9) lu_factor (M, generalring);
 [1 - z 3]
 []
(%o9) [[3 9], [1, 2]]
 [----- - z - ----- + 8]
 [1 - z 1 - z]
(%i10) get_lu_factors (%);
 [1 0] [1 - z 3]
 [1 0] [] []
(%o10) [[], [3], [9]]
 [0 1] [----- 1] [0 - z - ----- + 8]
 [1 - z] [1 - z]
(%i11) %[1] . %[2] . %[3];
```



```
(%o11) [1 - z 3]
 []
 [3 8 - z]
```

**mat\_cond** (*M*, 1) Function

**mat\_cond** (*M*, *inf*) Function

Return the *p*-norm matrix condition number of the matrix *m*. The allowed values for *p* are 1 and *inf*. This function uses the LU factorization to invert the matrix *m*. Thus the running time for **mat\_cond** is proportional to the cube of the matrix size; **lu\_factor** determines lower and upper bounds for the infinity norm condition number in time proportional to the square of the matrix size.

**mat\_norm** (*M*, 1) Function

**mat\_norm** (*M*, *inf*) Function

**mat\_norm** (*M*, *frobenius*) Function

Return the matrix *p*-norm of the matrix *M*. The allowed values for *p* are 1, *inf*, and *frobenius* (the Frobenius matrix norm). The matrix *M* should be an unblocked matrix.

**matrixp** (*e*, *p*) Function

**matrixp** (*e*) Function

Given an optional argument *p*, return **true** if *e* is a matrix and *p* evaluates to **true** for every matrix element. When **matrixp** is not given an optional argument, return **true** if *e* is a matrix. In all other cases, return **false**.

See also **blockmatrixp**

**matrix\_size** (*M*) Function

Return a two member list that gives the number of rows and columns, respectively of the matrix *M*.

**mat\_fullunblocker** (*M*) Function

If *M* is a block matrix, unblock the matrix to all levels. If *M* is a matrix, return *M*; otherwise, signal an error.

**mat\_trace** (*M*) Function

Return the trace of the matrix *M*. If *M* isn't a matrix, return a noun form. When *M* is a block matrix, **mat\_trace**(*M*) returns the same value as does **mat\_trace**(**mat\_unblocker**(*m*)).

**mat\_unblocker** (*M*) Function

If *M* is a block matrix, unblock *M* one level. If *M* is a matrix, **mat\_unblocker** (*M*) returns *M*; otherwise, signal an error.

Thus if each entry of *M* is matrix, **mat\_unblocker** (*M*) returns an unblocked matrix, but if each entry of *M* is a block matrix, **mat\_unblocker** (*M*) returns a block matrix with on less level of blocking.

If you use block matrices, most likely you'll want to set `matrix_element_mult` to `"."` and `matrix_element_transpose` to `'transpose'`. See also `mat_fullunblocker`.

Example:

```
(%i1) load (linearalgebra);
Warning - you are redefining the Maxima function require_list
Warning - you are redefining the Maxima function matrix_size
Warning - you are redefining the Maxima function rank
(%o1) /usr/local/share/maxima/5.9.2/share/linearalgebra/linearalgebra.mac
(%i2) A : matrix ([1, 2], [3, 4]);
 [1 2]
(%o2) []
 [3 4]
(%i3) B : matrix ([7, 8], [9, 10]);
 [7 8]
(%o3) []
 [9 10]
(%i4) matrix ([A, B]);
 [[1 2] [7 8]]
(%o4) [[] []]
 [[3 4] [9 10]]
(%i5) mat_unblocker (%);
 [1 2 7 8]
(%o5) []
 [3 4 9 10]
```

### **nonnegintegerp** ( $n$ )

Function

Return `true` if and only if  $n \geq 0$  and  $n$  is an integer.

### **nullspace** ( $M$ )

Function

If  $M$  is a matrix, return `span` ( $v_1, \dots, v_n$ ), where the set  $\{v_1, \dots, v_n\}$  is a basis for the nullspace of  $M$ . The span of the empty set is  $\{0\}$ . Thus, when the nullspace has only one member, return `span` ().

### **nullity** ( $M$ )

Function

If  $M$  is a matrix, return the dimension of the nullspace of  $M$ .

### **orthogonal\_complement** ( $v_1, \dots, v_n$ )

Function

Return `span` ( $u_1, \dots, u_m$ ), where the set  $\{u_1, \dots, u_m\}$  is a basis for the orthogonal complement of the set  $(v_1, \dots, v_n)$ .

Each vector  $v_1$  through  $v_n$  must be a column vector.

### **polynomialp** ( $p, L, coeffp, exponp$ )

Function

### **polynomialp** ( $p, L, coeffp$ )

Function

### **polynomialp** ( $p, L$ )

Function

Return `true` if  $p$  is a polynomial in the variables in the list  $L$ , The predicate `coeffp` must evaluate to `true` for each coefficient, and the predicate `exponp` must evaluate to `true` for all exponents of the variables in  $L$ . If you want to use a non-default value

for *exponp*, you must supply *coeffp* with a value even if you want to use the default for *coeffp*.

`polynomialp (p, L, coeffp)` is equivalent to `polynomialp (p, L, coeffp, 'nonnegintegerp)`.

`polynomialp (p, L)` is equivalent to `polynomialp (p, L, 'constantp, 'nonnegintegerp)`.

The polynomial needn't be expanded:

```
(%i1) load (linearalgebra);
Warning - you are redefining the Maxima function require_list
Warning - you are redefining the Maxima function matrix_size
Warning - you are redefining the Maxima function rank
(%o1) /usr/local/share/maxima/5.9.2/share/linearalgebra/linearalgebra.mac
(%i2) polynomialp ((x + 1)*(x + 2), [x]);
(%o2) true
(%i3) polynomialp ((x + 1)*(x + 2)^a, [x]);
(%o3) false
```

An example using non-default values for *coeffp* and *exponp*:

```
(%i1) load (linearalgebra);
Warning - you are redefining the Maxima function require_list
Warning - you are redefining the Maxima function matrix_size
Warning - you are redefining the Maxima function rank
(%o1) /usr/local/share/maxima/5.9.2/share/linearalgebra/linearalgebra.mac
(%i2) polynomialp ((x + 1)*(x + 2)^(3/2), [x], numberp, numberp);
(%o2) true
(%i3) polynomialp ((x^(1/2) + 1)*(x + 2)^(3/2), [x], numberp, numberp);
(%o3) true
```

Polynomials with two variables:

```
(%i1) load (linearalgebra);
Warning - you are redefining the Maxima function require_list
Warning - you are redefining the Maxima function matrix_size
Warning - you are redefining the Maxima function rank
(%o1) /usr/local/share/maxima/5.9.2/share/linearalgebra/linearalgebra.mac
(%i2) polynomialp (x^2 + 5*x*y + y^2, [x]);
(%o2) false
(%i3) polynomialp (x^2 + 5*x*y + y^2, [x, y]);
(%o3) true
```

### **polytocompanion** (*p*, *x*)

Function

If *p* is a polynomial in *x*, return the companion matrix of *p*. For a monic polynomial *p* of degree *n*, we have  $p = (-1)^n \text{charpoly}(\text{polytocompanion}(p, x))$ .

When *p* isn't a polynomial in *x*, signal an error.

### **ptriangularize** (*M*, *v*)

Function

If *M* is a matrix with each entry a polynomial in *v*, return a matrix *M2* such that

(1) *M2* is upper triangular,

(2)  $M2 = E_n \dots E_1 M$ , where  $E_1$  through  $E_n$  are elementary matrices whose entries are polynomials in  $v$ ,

(3)  $|\det(M)| = |\det(M2)|$ ,

Note: This function doesn't check that every entry is a polynomial in  $v$ .

**rowop** ( $M, i, j, \theta$ )

Function

If  $M$  is a matrix, return the matrix that results from doing the row operation  $R_i \leftarrow R_i - \theta R_j$ . If  $M$  doesn't have a row  $i$  or  $j$ , signal an error.

**rank** ( $M$ )

Function

Return the rank of that matrix  $M$ . The rank is the dimension of the column space.

Example:

```
(%i1) load(linearalgebra)$
(%i2) rank(matrix([1,2],[2,4]));
(%o2) 1
(%i3) rank(matrix([1,b],[c,d]));
Proviso: {d-b*c # 0}
(%o3) 2
```

**rowswap** ( $M, i, j$ )

Function

If  $M$  is a matrix, swap rows  $i$  and  $j$ . If  $M$  doesn't have a row  $i$  or  $j$ , signal an error.

**toeplitz** ( $col$ )

Function

**toeplitz** ( $col, row$ )

Function

Return a Toeplitz matrix  $T$ . The first column of  $T$  is  $col$ ; except for the first entry, the first row of  $T$  is  $row$ . The default for  $row$  is complex conjugate of  $col$ .

Example:

```
(%i1) load(linearalgebra)$

(%i2) toeplitz([1,2,3],[x,y,z]);

[1 y z]
[]
(%o2) [2 1 y]
[]
[3 2 1]
(%i3) toeplitz([1,1+%i]);

[1 1 - %I]
(%o3) []
[%I + 1 1]
```

**vandermonde\_matrix** ( $[x_1, \dots, x_n]$ )

Function

Return a  $n$  by  $n$  matrix whose  $i$ -th row is  $[1, x_i, x_i^2, \dots, x_i^{(n-1)}]$ .

**zerofor** ( $M$ )

Function

**zerofor** ( $M$ ,  $fld$ )

Function

Return a zero matrix that has the same shape as the matrix  $M$ . Every entry of the zero matrix is the additive identity of the field  $fld$ ; the default for  $fld$  is *generalring*.

The first argument  $M$  should be a square matrix or a non-matrix. When  $M$  is a matrix, each entry of  $M$  can be a square matrix – thus  $M$  can be a blocked Maxima matrix. The matrix can be blocked to any (finite) depth.

See also **identfor**

**zeromatrixp** ( $M$ )

Function

If  $M$  is not a block matrix, return **true** if **is (equal (e, 0))** is true for each element  $e$  of the matrix  $M$ . If  $M$  is a block matrix, return **true** if **zeromatrixp** evaluates to true for each element of  $e$ .

## 57 lsquares

### 57.1 Definiciones para lsquares

#### DETCOEf

Global variable

This variable is used by functions `lsquares` and `plsquares` to store the Coefficient of Determination which measures the goodness of fit. It ranges from 0 (no correlation) to 1 (exact correlation).

When `plsquares` is called with a list of dependent variables, *DETCOEf* is set to a list of Coefficients of Determination. See `plsquares` for details.

See also `lsquares`.

#### lsquares (Mat, VarList, equation, ParamList)

Function

#### lsquares (Mat, VarList, equation, ParamList, GuessList)

Function

Multiple nonlinear equation adjustment of a data table by the "least squares" method. *Mat* is a matrix containing the data, *VarList* is a list of variable names (one for each *Mat* column), *equation* is the equation to adjust (it must be in the form: `depvar=f(indepvari,..., paramj,...)`, `g(depvar)=f(indepvari,..., paramj,...)` or `g(depvar, paramk,...)=f(indepvari,..., paramj,...)`), *ParamList* is the list of the parameters to obtain, and *GuessList* is an optional list of initial approximations to the parameters; when this last argument is present, `mnewton` is used instead of `solve` in order to get the parameters.

The equation may be fully nonlinear with respect to the independent variables and to the dependent variable. In order to use `solve()`, the equations must be linear or polynomial with respect to the parameters. Equations like  $y = a \cdot b^x + c$  may be adjusted for `[a,b,c]` with `solve` if the *x* values are little positive integers and there are few data (see the example in `lsquares.dem`). `mnewton` allows to adjust a nonlinear equation with respect to the parameters, but a good set of initial approximations must be provided.

If possible, the adjusted equation is returned. If there exists more than one solution, a list of equations is returned. The Coefficient of Determination is displayed in order to inform about the goodness of fit, from 0 (no correlation) to 1 (exact correlation). This value is also stored in the global variable *DETCOEf*.

Examples using `solve`:

```
(%i1) load("lsquares")$

(%i2) lsquares(matrix([1,2,0],[3,5,4],[4,7,9],[5,8,10]),
 [x,y,z], z=a*x*y+b*x+c*y+d, [a,b,c,d]);
Determination Coefficient = 1.0
 x y + 23 y - 29 x - 19
(%o2) z = -----
 6

(%i3) lsquares(matrix([0,0],[1,0],[2,0],[3,8],[4,44]),
 [n,p], p=a4*n^4+a3*n^3+a2*n^2+a1*n+a0,
 [a0,a1,a2,a3,a4]);
```

```

Determination Coefficient = 1.0
 4 3 2
 3 n - 10 n + 9 n - 2 n
(%o3) p = -----
 6
(%i4) lsquares(matrix([1,7],[2,13],[3,25]),
 [x,y], (y+c)^2=a*x+b, [a,b,c]);
Determination Coefficient = 1.0
(%o4) [y = 28 - sqrt(657 - 216 x),
 y = sqrt(657 - 216 x) + 28]
(%i5) lsquares(matrix([1,7],[2,13],[3,25],[4,49]),
 [x,y], y=a*b^x+c, [a,b,c]);
Determination Coefficient = 1.0
 x
(%o5) y = 3 2 + 1

```

Examples using `mnewton`:

```

(%i6) load("lsquares")$
(%i7) lsquares(matrix([1.1,7.1],[2.1,13.1],[3.1,25.1],[4.1,49.1]),
 [x,y], y=a*b^x+c, [a,b,c], [5,5,5]);
 x
(%o7) y = 2.799098974610482 1.999999999999991
 + 1.09999999999999874
(%i8) lsquares(matrix([1.1,4.1],[4.1,7.1],[9.1,10.1],[16.1,13.1]),
 [x,y], y=a*x^b+c, [a,b,c], [4,1,2]);
 .4878659755898127
(%o8) y = 3.177315891123101 x
 + .7723843491402264
(%i9) lsquares(matrix([0,2,4],[3,3,5],[8,6,6]),
 [m,n,y], y=(A*m+B*n)^(1/3)+C, [A,B,C], [3,3,3]);
 1/3
(%o9) y = (3.9999999999999862 n + 4.999999999999359 m)
 + 2.000000000000012

```

To use this function write first `load("lsquares")`. See also `DETCOE` and `mnewton`.

|                                                                                                        |          |
|--------------------------------------------------------------------------------------------------------|----------|
| <b>plsquares</b> ( <i>Mat</i> , <i>VarList</i> , <i>depvars</i> )                                      | Function |
| <b>plsquares</b> ( <i>Mat</i> , <i>VarList</i> , <i>depvars</i> , <i>maxexpon</i> )                    | Function |
| <b>plsquares</b> ( <i>Mat</i> , <i>VarList</i> , <i>depvars</i> , <i>maxexpon</i> , <i>maxdegree</i> ) | Function |

Multivariable polynomial adjustment of a data table by the "least squares" method. *Mat* is a matrix containing the data, *VarList* is a list of variable names (one for each *Mat* column, but use "-" instead of varnames to ignore *Mat* columns), *depvars* is the name of a dependent variable or a list with one or more names of dependent variables (which names should be in *VarList*), *maxexpon* is the optional maximum exponent for each independent variable (1 by default), and *maxdegree* is the optional maximum polynomial degree (*maxexpon* by default); note that the sum of exponents of each term must be equal or smaller than *maxdegree*, and if *maxdegree* = 0 then no limit is applied.

If *depvars* is the name of a dependent variable (not in a list), `plsquares` returns the adjusted polynomial. If *depvars* is a list of one or more dependent variables, `plsquares` returns a list with the adjusted polynomial(s). The Coefficients of Determination are displayed in order to inform about the goodness of fit, which ranges from 0 (no correlation) to 1 (exact correlation). These values are also stored in the global variable *DET**COEF* (a list if *depvars* is a list).

A simple example of multivariable linear adjustment:

```
(%i1) load("plsquares")$

(%i2) plsquares(matrix([1,2,0],[3,5,4],[4,7,9],[5,8,10]),
 [x,y,z],z);
Determination Coefficient for z = .9897039897039897
 11 y - 9 x - 14
(%o2) z = -----
 3
```

The same example without degree restrictions:

```
(%i3) plsquares(matrix([1,2,0],[3,5,4],[4,7,9],[5,8,10]),
 [x,y,z],z,1,0);
Determination Coefficient for z = 1.0
 x y + 23 y - 29 x - 19
(%o3) z = -----
 6
```

How many diagonals does a N-sides polygon have? What polynomial degree should be used?

```
(%i4) plsquares(matrix([3,0],[4,2],[5,5],[6,9],[7,14],[8,20]),
 [N,diagonals],diagonals,5);
Determination Coefficient for diagonals = 1.0
 2
 N - 3 N
(%o4) diagonals = -----
 2

(%i5) ev(%, N=9); /* Testing for a 9 sides polygon */
(%o5) diagonals = 27
```

How many ways do we have to put two queens without they are threatened into a n x n chessboard?

```
(%i6) plsquares(matrix([0,0],[1,0],[2,0],[3,8],[4,44]),
 [n,positions],[positions],4);
Determination Coefficient for [positions] = [1.0]
 4 3 2
 3 n - 10 n + 9 n - 2 n
(%o6) [positions] = -----
 6

(%i7) ev(%[1], n=8); /* Testing for a (8 x 8) chessboard */
(%o7) positions = 1288
```

An example with six dependent variables:

```
(%i8) mtrx:matrix([0,0,0,0,0,1,1,1],[0,1,0,1,1,1,0,0],
```



```

[1,0,0,1,1,1,0,0],[1,1,1,1,0,0,0,1]))$
(%i8) plsquares(mtrx,[a,b,_And,_Or,_Xor,_Nand,_Nor,_Nxor],
[_And,_Or,_Xor,_Nand,_Nor,_Nxor],1,0);
Determination Coefficient for
[_And, _Or, _Xor, _Nand, _Nor, _Nxor] =
[1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
(%o2) [_And = a b, _Or = - a b + b + a,
_Xor = - 2 a b + b + a, _Nand = 1 - a b,
_Nor = a b - b - a + 1, _Nxor = 2 a b - b - a + 1]

```

To use this function write first `load("lsquares")`.

## 58 makeOrders

### 58.1 Definiciones para makeOrders

**makeOrders** (*indvarlist, orderlist*)

Function

Returns a list of all powers for a polynomial up to and including the arguments.

```
(%i1) load("makeOrders")$
```

```
(%i2) makeOrders([a,b],[2,3]);
```

```
(%o2) [[0, 0], [0, 1], [0, 2], [0, 3], [1, 0], [1, 1],
 [1, 2], [1, 3], [2, 0], [2, 1], [2, 2], [2, 3]]
```

```
(%i3) expand((1+a+a^2)*(1+b+b^2+b^3));
```

```
(%o3) a2 b3 + a3 b3 + b3 + a2 b2 + a b2 + b2 + a2 b + a b2
 + b + a2 + a + 1
```

where  $[0, 1]$  is associated with the term  $b$  and  $[2, 3]$  with  $a^2b^3$ .

To use this function write first `load("makeOrders")`.



## 59 mnewton

### 59.1 Definiciones para mnewton

#### NEWONEPSILON

Option variable

Default value:  $10.0^{-(\text{fpprec}/2)}$ Precision to determine when the `mnewton` function has converged towards the solution.See also `mnewton`.

#### NEWTONMAXITER

Option variable

Default value: 50

Maximum number of iterations to stop the `mnewton` function if it does not converge or if it converges too slowly.See also `mnewton`.

#### mnewton (*FuncList*, *VarList*, *GuessList*)

Function

Multiple nonlinear functions solution using the Newton method. *FuncList* is the list of functions to solve, *VarList* is the list of variable names, and *GuessList* is the list of initial approximations.

The solution is returned in the same format that `solve()` returns. If the solution isn't found, `[]` is returned.

This function is controlled by global variables `NEWONEPSILON` and `NEWTONMAXITER`.

```
(%i1) load("mnewton")$
```

```
(%i2) mnewton([x1+3*log(x1)-x2^2, 2*x1^2-x1*x2-5*x1+1],
 [x1, x2], [5, 5]);
```

```
(%o2) [[x1 = 3.756834008012769, x2 = 2.779849592817897]]
```

```
(%i3) mnewton([2*a^a-5], [a], [1]);
```

```
(%o3) [[a = 1.70927556786144]]
```

```
(%i4) mnewton([2*3^u-v/u-5, u+2^v-4], [u, v], [2, 2]);
```

```
(%o4) [[u = 1.066618389595407, v = 1.552564766841786]]
```

To use this function write first `load("mnewton")`. See also `NEWONEPSILON` and `NEWTONMAXITER`.



## 60 numericalio

### 60.1 Introducción a numericalio

`numericalio` is a collection of functions to read and write data files. The entire file is read to construct one object; partial reads are not supported.

It is assumed that each item to read or write is atomic: an integer, float, bigfloat, string, or symbol, and not a rational or complex number or any other kind of nonatomic expression. These functions may attempt to do something sensible faced with nonatomic expressions, but the results are not specified here and subject to change.

Atoms in both input and output files have the same format as in Maxima batch files or the interactive console. In particular, strings are enclosed in double quotes, backslash `\` prevents any special interpretation of the next character, and the question mark `?` is recognized at the beginning of a symbol to mean a Lisp symbol (as opposed to a Maxima symbol). No continuation character (to join broken lines) is recognized.

`separator_flag` tells which character separates elements. It is an optional argument for all read and write functions.

For input, these values of `separator_flag` are recognized: `comma` for comma separated values, `pipe` for values separated by the vertical bar character `|`, `semicolon` for values separated by semicolon `;`, and `space` for values separated by space or tab characters. If the file name ends in `.csv` and `separator_flag` is not specified, `comma` is assumed. If the file name ends in something other than `.csv` and `separator_flag` is not specified, `space` is assumed.

For output, the same four flags are recognized as for input, and also `tab`, for values separated by the tab character.

In input, multiple successive space and tab characters count as a single separator. However, multiple comma, pipe, or semicolon characters are significant. Successive comma, pipe, or semicolon characters (with or without intervening spaces or tabs) are considered to have `false` between the separators. For example, `1234,,Foo` is treated the same as `1234,false,Foo`. In output, `false` atoms are written as such; a list `[1234, false, Foo]` is written `1234,false,Foo`, and there is no attempt to collapse the output to `1234,,Foo`.

### 60.2 Definiciones para numericalio

|                                                                                                                                                                                                |          |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>read_matrix</b> ( <i>file_name</i> )                                                                                                                                                        | Function |
| <b>read_matrix</b> ( <i>file_name</i> , <i>separator_flag</i> )                                                                                                                                | Function |
| Reads the file <i>file_name</i> and returns its entire content as a matrix. If <i>separator_flag</i> is not specified, the file is assumed space-delimited.                                    |          |
| <b>read_matrix</b> infers the size of the matrix from the input data. Each line of the file becomes one row of the matrix. If some lines have different lengths, <b>read_matrix</b> complains. |          |

- read\_lisp\_array** (*file\_name*, *A*) Function  
**read\_lisp\_array** (*file\_name*, *A*, *separator\_flag*) Function  
 read\_lisp\_array requires that the array be declared by **make\_array** before calling the read function. (This obviates the need to infer the array dimensions, which could be a hassle for arrays with multiple dimensions.)  
 read\_lisp\_array does not check to see that the input file conforms in some way to the array dimensions; the input is read as a flat list, then the array is filled using **fillarray**.
- read\_maxima\_array** (*file\_name*, *A*) Function  
**read\_maxima\_array** (*file\_name*, *A*, *separator\_flag*) Function  
 read\_maxima\_array requires that the array be declared by **array** before calling the read function. (This obviates the need to infer the array dimensions, which could be a hassle for arrays with multiple dimensions.)  
 read\_maxima\_array does not check to see that the input file conforms in some way to the array dimensions; the input is read as a flat list, then the array is filled using **fillarray**.
- read\_hashed\_array** (*file\_name*, *A*) Function  
**read\_hashed\_array** (*file\_name*, *A*, *separator\_flag*) Function  
 read\_hashed\_array treats the first item on a line as a hash key, and associates the remainder of the line (as a list) with the key. For example, the line 567 12 17 32 55 is equivalent to **A[567]: [12, 17, 32, 55]**\$. Lines need not have the same numbers of elements.
- read\_nested\_list** (*file\_name*) Function  
**read\_nested\_list** (*file\_name*, *separator\_flag*) Function  
 read\_nested\_list returns a list which has a sublist for each line of input. Lines need not have the same numbers of elements. Empty lines are *not* ignored: an empty line yields an empty sublist.
- read\_list** (*file\_name*) Function  
**read\_list** (*file\_name*, *separator\_flag*) Function  
 read\_list reads all input into a flat list. read\_list ignores end-of-line characters.
- write\_data** (*X*, *file\_name*) Function  
**write\_data** (*object*, *file\_name*, *separator\_flag*) Function  
 write\_data writes the object *X* to the file *file\_name*.  
 write\_data writes matrices in row-major form, with one line per row.  
 write\_data writes Lisp and Maxima declared arrays in row-major form, with a new line at the end of every slab. Higher-dimensional slabs are separated by additional new lines.  
 write\_data writes hashed arrays with a key followed by the associated list on each line.  
 write\_data writes a nested list with each sublist on one line.

`write_data` writes a flat list all on one line.

Whether `write_data` appends or truncates its output file is governed by the global variable `file_output_append`.





## 61 opsubst

### 61.1 Definiciones para opsubst

|                                                   |          |
|---------------------------------------------------|----------|
| <b>opsubst</b> ( $f,g,e$ )                        | Function |
| <b>opsubst</b> ( $g=f,e$ )                        | Function |
| <b>opsubst</b> ( $[g1=f1,g2=f2,\dots, gn=fn],e$ ) | Function |

The function **opsubst** is similar to the function **subst**, except that **opsubst** only makes substitutions for the operators in an expression. In general, When  $f$  is an operator in the expression  $e$ , substitute  $g$  for  $f$  in the expression  $e$ .

To determine the operator, **opsubst** sets **inflag** to true. This means **opsubst** substitutes for the internal, not the displayed, operator in the expression.

Examples:

```
(%i1) load (opsubst)$

(%i2) opsubst(f,g,g(g(x)));
(%o2) f(f(x))
(%i3) opsubst(f,g,g(g));
(%o3) f(g)
(%i4) opsubst(f,g[x],g[x](z));
(%o4) f(z)
(%i5) opsubst(g[x],f, f(z));
(%o5) g (z)
 x
(%i6) opsubst(tan, sin, sin(sin));
(%o6) tan(sin)
(%i7) opsubst([f=g,g=h],f(x));
(%o7) h(x)
```

Internally, Maxima does not use the unary negation, division, or the subtraction operators; thus:

```
(%i8) opsubst("+","-",a-b);
(%o8) a - b
(%i9) opsubst("f","-", -a);
(%o9) - a
(%i10) opsubst("^^","/",a/b);
(%o10) a
 -
 b
```

The internal representation of  $-a*b$  is  $*(-1,a,b)$ ; thus

```
(%i11) opsubst("[","*", -a*b);
(%o11) [- 1, a, b]
```

When either operator isn't a Maxima symbol, generally some other function will signal an error:

```
(%i12) opsubst(a+b,f, f(x));
```

```
Improper name or value in functional position:
```

```
b + a
```

```
-- an error. Quitting. To debug this try debugmode(true);
```

However, subscripted operators are allowed:

```
(%i13) opsubst(g[5],f, f(x));
```

```
(%o13) g (x)
 5
```

To use this function write first `load("opsubst")`.

## 62 orthopoly

### 62.1 Introducción a polinomios ortogonales

**orthopoly** is a package for symbolic and numerical evaluation of several kinds of orthogonal polynomials, including Chebyshev, Laguerre, Hermite, Jacobi, Legendre, and ultraspherical (Gegenbauer) polynomials. Additionally, **orthopoly** includes support for the spherical Bessel, spherical Hankel, and spherical harmonic functions.

For the most part, **orthopoly** follows the conventions of Abramowitz and Stegun *Handbook of Mathematical Functions*, Chapter 22 (10th printing, December 1972); additionally, we use Gradshteyn and Ryzhik, *Table of Integrals, Series, and Products* (1980 corrected and enlarged edition), and Eugen Merzbacher *Quantum Mechanics* (2nd edition, 1970).

Barton Willis of the University of Nebraska at Kearney (UNK) wrote the **orthopoly** package and its documentation. The package is released under the GNU General Public License (GPL).

#### 62.1.0.1 Getting Started with orthopoly

`load (orthopoly)` loads the **orthopoly** package.

To find the third-order Legendre polynomial,

```
(%i1) legendre_p (3, x);
```

$$(\%o1) \quad -\frac{5(1-x)^3}{2} + \frac{15(1-x)^2}{2} - 6(1-x) + 1$$

To express this as a sum of powers of  $x$ , apply `ratsimp` or `rat` to the result.

```
(%i2) [ratsimp (%), rat (%)];
```

$$(\%o2)/R/ \quad \left[ \frac{5x^3 - 3x}{2}, \frac{5x^3 - 3x}{2} \right]$$

Alternatively, make the second argument to `legendre_p` (its “main” variable) a canonical rational expression (CRE).

```
(%i1) legendre_p (3, rat (x));
```

$$(\%o1)/R/ \quad \frac{5x^3 - 3x}{2}$$

For floating point evaluation, **orthopoly** uses a running error analysis to estimate an upper bound for the error. For example,

```
(%i1) jacobi_p (150, 2, 3, 0.2);
```

```
(%o1) interval(- 0.062017037936715, 1.533267919277521E-11)
```

Intervals have the form `interval (c, r)`, where  $c$  is the center and  $r$  is the radius of the interval. Since Maxima does not support arithmetic on intervals, in some situations, such

as graphics, you want to suppress the error and output only the center of the interval. To do this, set the option variable `orthopoly_returns_intervals` to `false`.

```
(%i1) orthopoly_returns_intervals : false;
(%o1) false
(%i2) jacobi_p (150, 2, 3, 0.2);
(%o2) - 0.062017037936715
```

Refer to the section véase [\[Floating point Evaluation\]](#), página [\[undefined\]](#) for more information.

Most functions in `orthopoly` have a `gradef` property; thus

```
(%i1) diff (hermite (n, x), x);
(%o1) 2 n H (x)
 n - 1
(%i2) diff (gen_laguerre (n, a, x), x);
 (a) (a)
 n L (x) - (n + a) L (x) unit_step(n)
 n n - 1
(%o2) -----
 x
```

The unit step function in the second example prevents an error that would otherwise arise by evaluating with  $n$  equal to 0.

```
(%i3) ev (%, n = 0);
(%o3) 0
```

The `gradef` property only applies to the “main” variable; derivatives with respect other arguments usually result in an error message; for example

```
(%i1) diff (hermite (n, x), x);
(%o1) 2 n H (x)
 n - 1
(%i2) diff (hermite (n, x), n);
```

Maxima doesn't know the derivative of `hermite` with respect the first argument -- an error. Quitting. To debug this try `debugmode(true)`;

Generally, functions in `orthopoly` map over lists and matrices. For the mapping to fully evaluate, the option variables `doallmxops` and `listarith` must both be `true` (the defaults). To illustrate the mapping over matrices, consider

```
(%i1) hermite (2, x);
(%o1) - 2 (1 - 2 x)
 2
(%i2) m : matrix ([0, x], [y, 0]);
 [0 x]
(%o2) []
 [y 0]
(%i3) hermite (2, m);
 [2]
 [- 2 - 2 (1 - 2 x)]
(%o3) []
 [2]
```

$$\begin{bmatrix} -2(1-2y) & -2 \\ & \end{bmatrix}$$

In the second example, the  $i, j$  element of the value is `hermite (2, m[i,j])`; this is not the same as computing  $-2 + 4m \cdot m$ , as seen in the next example.

```
(%i4) -2 * matrix ([1, 0], [0, 1]) + 4 * m . m;
 [4 x y - 2 0]
(%o4) [
 [0 4 x y - 2]
```

If you evaluate a function at a point outside its domain, generally `orthopoly` returns the function unevaluated. For example,

```
(%i1) legendre_p (2/3, x);
(%o1) P (x)
 2/3
```

`orthopoly` supports translation into TeX; it also does two-dimensional output on a terminal.

```
(%i1) spherical_harmonic (1, m, theta, phi);
 m
(%o1) Y (theta, phi)
 1

(%i2) tex (%);
$$$Y_{1}^{\{m\}}\left(\vartheta,\varphi\right)$$$
(%o2) false
(%i3) jacobi_p (n, a, a - b, x/2);
 (a, a - b) x
(%o3) P (-)
 n 2

(%i4) tex (%);
$$$P_{n}^{\{\left(a,a-b\right)\}}\left(\frac{x}{2}\right)$$$
(%o4) false
```

### 62.1.0.2 Limitations

When an expression involves several orthogonal polynomials with symbolic orders, it's possible that the expression actually vanishes, yet Maxima is unable to simplify it to zero. If you divide by such a quantity, you'll be in trouble. For example, the following expression vanishes for integers  $n$  greater than 1, yet Maxima is unable to simplify it to zero.

```
(%i1) (2*n - 1) * legendre_p (n - 1, x) * x - n * legendre_p (n, x) + (1 - n) * leg
(%o1) (2 n - 1) P (x) x - n P (x) + (1 - n) P (x)
 n - 1 n n - 2
```

For a specific  $n$ , we can reduce the expression to zero.

```
(%i2) ev (% ,n = 10, ratsimp);
(%o2) 0
```

Generally, the polynomial form of an orthogonal polynomial is ill-suited for floating point evaluation. Here's an example.

```
(%i1) p : jacobi_p (100, 2, 3, x)$
(%i2) subst (0.2, x, p);
(%o2) 3.444276702383359*10^+35
```

```
(%i3) jacobi_p (100, 2, 3, 0.2);
(%o3) INTERVAL(0.1841360913516871,6.899030092581599*10^-12)
(%i4) float(jacobi_p (100, 2, 3, 2/10));
(%o4) 0.1841360913516856
```

The true value is about 0.184; this calculation suffers from extreme subtractive cancellation error. Expanding the polynomial and then evaluating, gives a better result.

```
(%i5) p : expand(p)$
(%i6) subst (0.2, x, p);
(%o6) 0.18413609766122982
```

This isn't a general rule; expanding the polynomial does not always result in an expression that is better suited for numerical evaluation. By far, the best way to do numerical evaluation is to make one or more of the function arguments floating point numbers. By doing that, specialized floating point algorithms are used for evaluation.

Maxima's `float` function is somewhat indiscriminant; if you apply `float` to an expression involving an orthogonal polynomial with a symbolic degree or order parameter, these parameters may be converted into floats; after that, the expression will not evaluate fully. Consider

```
(%i1) assoc_legendre_p (n, 1, x);
(%o1)
$$P_n^1(x)$$

(%i2) float (%);
(%o2)
$$P_n^{1.0}(x)$$

(%i3) ev (% , n=2, x=0.9);
(%o3)
$$P_2^{1.0}(0.9)$$

```

The expression in (%o3) will not evaluate to a float; orthopoly doesn't recognize floating point values where it requires an integer. Similarly, numerical evaluation of the `pochhammer` function for orders that exceed `pochhammer_max_index` can be troublesome; consider

```
(%i1) x : pochhammer (1, 10), pochhammer_max_index : 5;
(%o1)
$$(1)_{10}$$

```

Applying `float` doesn't evaluate `x` to a float

```
(%i2) float (x);
(%o2)
$$(1.0)_{10.0}$$

```

To evaluate `x` to a float, you'll need to bind `pochhammer_max_index` to 11 or greater and apply `float` to `x`.

```
(%i3) float (x), pochhammer_max_index : 11;
(%o3) 3628800.0
```

The default value of `pochhammer_max_index` is 100; change its value after loading `orthopoly`.

Finally, be aware that reference books vary on the definitions of the orthogonal polynomials; we've generally used the conventions of Abramowitz and Stegun.

Before you suspect a bug in orthopoly, check some special cases to determine if your definitions match those used by orthonormal. Definitions often differ by a normalization; occasionally, authors use “shifted” versions of the functions that makes the family orthogonal on an interval other than  $(-1, 1)$ . To define, for example, a Legendre polynomial that is orthogonal on  $(0, 1)$ , define

```
(%i1) shifted_legendre_p (n, x) := legendre_p (n, 2*x - 1)$

(%i2) shifted_legendre_p (2, rat (x));
 2
 6 x - 6 x + 1
(%o2)/R/
(%i3) legendre_p (2, rat (x));
 2
 3 x - 1
(%o3)/R/ -----
 2
```

### 62.1.0.3 Floating point Evaluation

Most functions in orthopoly use a running error analysis to estimate the error in floating point evaluation; the exceptions are the spherical Bessel functions and the associated Legendre polynomials of the second kind. For numerical evaluation, the spherical Bessel functions call SLATEC functions. No specialized method is used for numerical evaluation of the associated Legendre polynomials of the second kind.

The running error analysis ignores errors that are second or higher order in the machine epsilon (also known as unit roundoff). It also ignores a few other errors. It's possible (although unlikely) that the actual error exceeds the estimate.

Intervals have the form `interval (c, r)`, where  $c$  is the center of the interval and  $r$  is its radius. The center of an interval can be a complex number, and the radius is always a positive real number.

Here is an example.

```
=
(%i1) fpprec : 50$
(%i2) y0 : jacobi_p (100, 2, 3, 0.2);
(%o2) INTERVAL(0.1841360913516871,6.8990300925815987*10^-12)
(%i3) y1 : bfloat (jacobi_p (100, 2, 3, 1/5));
(%o3) 1.8413609135168563091370224958913493690868904463668B-1
```

Let's test that the actual error is smaller than the error estimate

```
(%i4) is (abs (part (y0, 1) - y1) < part (y0, 2));
(%o4) true
```

Indeed, for this example the error estimate is an upper bound for the true error.

Maxima does not support arithmetic on intervals.

```
(%i1) legendre_p (7, 0.1) + legendre_p (8, 0.1);
(%o1) interval(0.18032072148437508, 3.1477135311021797E-15)
```



```
+ interval(- 0.19949294375000004, 3.3769353084291579E-15)
```

A user could define arithmetic operators that do interval math. To define interval addition, we can define

```
(%i1) infix ("@+")$
```

```
(%i2) "@+(x,y) := interval (part (x, 1) + part (y, 1), part (x, 2) + part (y, 2))$
```

```
(%i3) legendre_p (7, 0.1) @+ legendre_p (8, 0.1);
```

```
(%o3) interval(- 0.019172222265624955, 6.5246488395313372E-15)
```

The special floating point routines get called when the arguments are complex. For example,

```
(%i1) legendre_p (10, 2 + 3.0*i);
```

```
(%o1) interval(- 3.876378825E+7 %i - 6.0787748E+7,
 1.2089173052721777E-6)
```

Let's compare this to the true value.

```
(%i1) float (expand (legendre_p (10, 2 + 3*i)));
```

```
(%o1) - 3.876378825E+7 %i - 6.0787748E+7
```

Additionally, when the arguments are big floats, the special floating point routines get called; however, the big floats are converted into double floats and the final result is a double.

```
(%i1) ultraspherical (150, 0.5b0, 0.9b0);
```

```
(%o1) interval(- 0.043009481257265, 3.3750051301228864E-14)
```

#### 62.1.0.4 Graphics and orthopoly

To plot expressions that involve the orthogonal polynomials, you must do two things:

1. Set the option variable `orthopoly_returns_intervals` to `false`,
2. Quote any calls to `orthopoly` functions.

If function calls aren't quoted, Maxima evaluates them to polynomials before plotting; consequently, the specialized floating point code doesn't get called. Here is an example of how to plot an expression that involves a Legendre polynomial.

```
(%i1) plot2d ('(legendre_p (5, x)), [x, 0, 1]), orthopoly_returns_intervals : false
```

```
(%o1)
```

The *entire* expression `legendre_p (5, x)` is quoted; this is different than just quoting the function name using `'legendre_p (5, x)`.

#### 62.1.0.5 Miscellaneous Functions

The `orthopoly` package defines the Pochhammer symbol and a unit step function. `orthopoly` uses the Kronecker delta function and the unit step function in `gradef` statements.

To convert Pochhammer symbols into quotients of gamma functions, use `makegamma`.

```
(%i1) makegamma (pochhammer (x, n));
```

```
gamma(x + n)
```

```
(%o1)
```

```

```

```

 gamma(x)
(%i2) makegamma (pochhammer (1/2, 1/2));
 1
(%o2) -----
 sqrt(%pi)

```

Derivatives of the pochhammer symbol are given in terms of the `psi` function.

```

(%i1) diff (pochhammer (x, n), x);
(%o1) (x) (psi (x + n) - psi (x))
 n 0 0
(%i2) diff (pochhammer (x, n), n);
(%o2) (x) psi (x + n)
 n 0

```

You need to be careful with the expression in (%o1); the difference of the `psi` functions has poles when  $x = -1, -2, \dots, -n$ . These poles cancel with factors in `pochhammer (x, n)` making the derivative a degree  $n - 1$  polynomial when  $n$  is a positive integer.

The Pochhammer symbol is defined for negative orders through its representation as a quotient of gamma functions. Consider

```

(%i1) q : makegamma (pochhammer (x, n));
 gamma(x + n)
(%o1) -----
 gamma(x)
(%i2) sublis ([x=11/3, n= -6], q);
 729
(%o2) - ----
 2240

```

Alternatively, we can get this result directly.

```

(%i1) pochhammer (11/3, -6);
 729
(%o1) - ----
 2240

```

The unit step function is left-continuous; thus

```

(%i1) [unit_step (-1/10), unit_step (0), unit_step (1/10)];
(%o1) [0, 0, 1]

```

If you need a unit step function that is neither left or right continuous at zero, define your own using `signum`; for example,

```

(%i1) xunit_step (x) := (1 + signum (x))/2$
(%i2) [xunit_step (-1/10), xunit_step (0), xunit_step (1/10)];
 1
(%o2) [0, -, 1]
 2

```

Do not redefine `unit_step` itself; some code in `orthopoly` requires that the unit step function be left-continuous.

### 62.1.0.6 Algorithms

Generally, `orthopoly` does symbolic evaluation by using a hypergeometric representation of the various orthogonal polynomials. The hypergeometric functions are evaluated using the (undocumented) functions `hypergeo11` and `hypergeo21`. The exceptions are the half-integer Bessel functions and the associated Legendre function of the second kind. The Bessel functions are evaluated using an explicit representation, while the associated Legendre function of the second kind is evaluated using recursion.

For floating point evaluation, we again convert most functions into a hypergeometric form; we evaluate the hypergeometric functions using forward recursion. Again, the exceptions are the half-integer Bessel functions and the associated Legendre function of the second kind. Numerically, the half-integer Bessel functions are evaluated using the SLATEC code, and the associated Legendre functions of the second kind is numerically evaluated using the same algorithm as its symbolic evaluation uses.

## 62.2 Definiciones para polinomios ortogonales

|                                                                                                                        |          |
|------------------------------------------------------------------------------------------------------------------------|----------|
| <b>assoc_legendre_p</b> ( $n, m, x$ )                                                                                  | Function |
| The associated Legendre function of the first kind.                                                                    |          |
| Reference: Abramowitz and Stegun, equations 22.5.37, page 779, 8.6.6 (second equation), page 334, and 8.2.5, page 333. |          |
| <b>assoc_legendre_q</b> ( $n, m, x$ )                                                                                  | Function |
| The associated Legendre function of the second kind.                                                                   |          |
| Reference: Abramowitz and Stegun, equation 8.5.3 and 8.1.8.                                                            |          |
| <b>chebyshev_t</b> ( $n, x$ )                                                                                          | Function |
| The Chebyshev function of the first kind.                                                                              |          |
| Reference: Abramowitz and Stegun, equation 22.5.47, page 779.                                                          |          |
| <b>chebyshev_u</b> ( $n, x$ )                                                                                          | Function |
| The Chebyshev function of the second kind.                                                                             |          |
| Reference: Abramowitz and Stegun, equation 22.5.48, page 779.                                                          |          |
| <b>gen_laguerre</b> ( $n, a, x$ )                                                                                      | Function |
| The generalized Laguerre polynomial.                                                                                   |          |
| Reference: Abramowitz and Stegun, equation 22.5.54, page 780.                                                          |          |
| <b>hermite</b> ( $n, x$ )                                                                                              | Function |
| The Hermite polynomial.                                                                                                |          |
| Reference: Abramowitz and Stegun, equation 22.5.55, page 780.                                                          |          |
| <b>intervalp</b> ( $e$ )                                                                                               | Function |
| Return true if the input is an interval and return false if it isn't.                                                  |          |

The Jacobi polynomial.

Reference: Abramowitz and Stegun, equation 22.5.42, page 779.

The Laguerre polynomial.

The Legendre polynomial of the first kind,

The Legendre polynomial of the first kind.

Returns a recursion relation for the orthogonal function family  $f$  with arguments  $args$ .

```
(%i1) orthopoly_recur(legendre_p, [n, x]);
```

$$(2n-1)P_n(x)x + (1-n)P_{n-2}(x)$$

```
(%o1) P(x) = -----
```

$$\frac{(2n-1)P_n(x)x + (1-n)P_{n-2}(x)}{n}$$

```
(%i1) orthopoly_recur (jacobi_p, [n, x]);
```

Additionally, when  $f$  isn't the name of one of the families of orthogonal polynomials, an error is signalled.

```
(%i1) orthopoly_recur (foo, [n, x]);
```

```
A recursion relation for foo isn't known to Maxima
-- an error. Quitting. To debug this try debugmode(true);
```

Default value: `true`

When `orthopoly_returns_intervals` is true, floating point results are returned in the form `interval (c, r)`, where  $c$  is the center of an interval and  $r$  is its radius. The center can be a complex number; in that case, the interval is a disk in the complex plane.

**orthopoly\_weight** (*f*, *args*)

Function

Returns a three element list; the first element is the formula of the weight for the orthogonal polynomial family *f* with arguments given by the list *args*; the second and third elements give the lower and upper endpoints of the interval of orthogonality. For example,

```
(%i1) w : orthopoly_weight (hermite, [n, x]);
 2
 - x
(%o1) [%e , - inf, inf]
(%i2) integrate (w[1] * hermite (3, x) * hermite (2, x), x, w[2], w[3]);
(%o2) 0
```

The main variable of *f* must be a symbol; if it isn't, Maxima signals an error.

**pochhammer** (*n*, *x*)

Function

The Pochhammer symbol. For nonnegative integers *n* with *n* ≤ pochhammer\_max\_index, the expression pochhammer (*x*, *n*) evaluates to the product *x* (*x* + 1) (*x* + 2) ... (*x* + *n* - 1) when *n* > 0 and to 1 when *n* = 0. For negative *n*, pochhammer (*x*, *n*) is defined as (-1)<sup>*n*</sup> / pochhammer (1 - *x*, -*n*). Thus

```
(%i1) pochhammer (x, 3);
(%o1) x (x + 1) (x + 2)
(%i2) pochhammer (x, -3);
(%o2) 1
 - -----
 (1 - x) (2 - x) (3 - x)
```

To convert a Pochhammer symbol into a quotient of gamma functions, (see Abramowitz and Stegun, equation 6.1.22) use makegamma; for example

```
(%i1) makegamma (pochhammer (x, n));
 gamma(x + n)
(%o1) -----
 gamma(x)
```

When *n* exceeds pochhammer\_max\_index or when *n* is symbolic, pochhammer returns a noun form.

```
(%i1) pochhammer (x, n);
(%o1) (x)
 n
```

**pochhammer\_max\_index**

Variable

Default value: 100

pochhammer (*n*, *x*) expands to a product if and only if *n* ≤ pochhammer\_max\_index.

Examples:

```
(%i1) pochhammer (x, 3), pochhammer_max_index : 3;
(%o1) x (x + 1) (x + 2)
(%i2) pochhammer (x, 4), pochhammer_max_index : 3;
(%o2) (x)
 4
```

Reference: Abramowitz and Stegun, equation 6.1.16, page 256.

- spherical\_bessel\_j** ( $n, x$ ) Function  
 The spherical Bessel function of the first kind.  
 Reference: Abramowitz and Stegun, equations 10.1.8, page 437 and 10.1.15, page 439.
- spherical\_bessel\_y** ( $n, x$ ) Function  
 The spherical Bessel function of the second kind.  
 Reference: Abramowitz and Stegun, equations 10.1.9, page 437 and 10.1.15, page 439.
- spherical\_hankel1** ( $n, x$ ) Function  
 The spherical hankel function of the first kind.  
 Reference: Abramowitz and Stegun, equation 10.1.36, page 439.
- spherical\_hankel2** ( $n, x$ ) Function  
 The spherical hankel function of the second kind.  
 Reference: Abramowitz and Stegun, equation 10.1.17, page 439.
- spherical\_harmonic** ( $n, m, x, y$ ) Function  
 The spherical harmonic function.  
 Reference: Merzbacher 9.64.
- unit\_step** ( $x$ ) Function  
 The left-continuous unit step function; thus **unit\_step** ( $x$ ) vanishes for  $x \leq 0$  and equals 1 for  $x > 0$ .  
 If you want a unit step function that takes on the value 1/2 at zero, use  $(1 + \text{signum}(x))/2$ .
- ultraspherical** ( $n, a, x$ ) Function  
 The ultraspherical polynomial (also known the Gegenbauer polynomial).  
 Reference: Abramowitz and Stegun, equation 22.5.46, page 779.



## 63 plotdf

### 63.1 Definiciones para plotdf

**plotdf** (*expr*,...*options*...)

Function

**plotdf** ([*expr1*,*expr1*],...*options*...)

Function

Plots the direction field of a first-order ordinary equation (ODE) or a system of two autonomous ODE's.

In Maxima 5.9.0, plotdf will only work from xmaxima and you cannot use the option "trajectory\_at". In Maxima 5.9.1, in addition to those two limitations there will also be two options missing from the plot menu: "Integrate" and "Plot vs t". To solve those problems see <http://fisica.fe.up.pt/maxima/plotdf/download.html>

In recent CVS versions plotdf should work fine both from xmaxima and maxima.

For one ODE use **plotdf** (*expr*,...,*options*,...), where *expr* is an expression depending on *x* and *y*, which represents the right-hand side of the ODE:

$$\frac{dy}{dx} = \text{expr}$$

*expr* can also depend on a set of parameters that must be given numerical values with the "parameters" option and those parameters can be changed interactively with the "sliders" option (see "PLOTDF OPTIONS" below).

For numerical values of *x*, *y*, and the parameters, **float**(**ev**(*expr*, *numer*)) must return a number.

For a system of two autonomous, first-order ODE's, use **plotdf** ([*expr1*,*expr2*],...,*options*,...), Where *expr1* and *expr2* are two expressions that depend on *x* and *y* (but not on *t*), and represent the right-hand side of the ODE's:

$$\begin{array}{ll} \frac{dx}{dt} = \text{expr1} & \frac{dy}{dt} = \text{expr2} \end{array}$$

as in the first case, options "parameters" and "sliders" can be used.

The menu in the plot window has the following options: "Zoom", will change the behavior of the mouse so that it will allow you to zoom in on a region of the plot by clicking with the left button. Each click near a point magnifies the plot, keeping the center at the point where you clicked. Holding the SHIFT key while clicking, zooms out to the previous magnification. To resume computing trajectories when you click on a point, select "Integrate" from the menu.

To change the ODE(s) in use, or change other settings, select "Config" in the menu, enter new values in the dialog window, and then click on "Replot" in the main menu bar. If you enter a pair of coordinates in the field "Trajectory at" in the Config dialog menu, and press the "enter" key, a new integral curve will be shown, in addition to the ones already shown. You can change the color before you enter the point coordinates. When you select "Replot" all integral curves, except the last one, will be erased.



Holding the right mouse button down while you move the cursor allows you to drag (translate) the plot sideways or up and down. Additional parameters such as the number of steps (nsteps), the initial t value (tinitial), and the x and y centers and radii, may be set in the Config menu.

You may print to a Postscript printer, or save the plot as a postscript file, with the menu option "Save". To switch between printing and saving to a Postscript file, select the "Print Options" in the dialog window of "Config", change the settings, go back to the main menu ("OK" twice) and select "Save".

Options for `plotdf`:

- `[xradius,12.0]`  
Determines the width in x direction of the x values shown by `plotdf`.
- `[yradius,10.5]`  
Defines the height in y direction of the y values shown by `plotdf`.
- `[xcenter,0.0],[ycenter,0.0]`  
(`xcenter,ycenter`) is the origin of the window shown by `plotdf`.
- `[width,500]`  
Width of `plotdf`'s canvas in pixels.
- `[height,500]`  
Height of `plotdf`'s canvas in pixels.
- `[bbox, -2, -1.2, 3.5, 6]`  
Bounding box (`xmin ymin xmax ymax`) of the region shown by `plotdf`. It will override the values of `xcenter`, `ycenter`, `xradius`, `yradius`.
- `[tinitial,0.0]`  
The initial value of the t variable used by `plotdf` to compute integral curves.
- `[nsteps,100]`  
Number of steps to do in one pass by the integrator of `plotdf`.
- `[tstep,0.01]`  
t step size used by `plotdf`'s integrator.
- `[xfun,"x^2;sin(x);exp(x)"]`  
A semicolon separated list of functions that `plotdf` will plot on top of the direction field.
- `[direction,forward]`  
The direction (in time) that the integral curves will follow in a direction field produced by `plotdf`. It may be "forward", "backward" or "both".
- `[versus_t,1]`  
Any value different from zero will make `plotdf` open a window showing the time dependence of the independent variables in the last integral curve computed.
- `[trajectory_at,0.1,3.2]`  
(`x,y`) coordinates of a point through which an integral curve should be shown by `plotdf`.

- `[parameters,"k=1.1,m=2.5"]`  
List of parameters, and their numerical values, used in the differential equation(s) given to `plotdf`.
- `[sliders,"k=0:4,m=1:3"]`  
Ranges of the parameters that `plotdf` will use to put sliders to accept interactive change of the equations parameters.

Examples:

- To show the direction field of the differential equation  $y' = \exp(-x) + y$  and the solution that goes through  $(2, -0.1)$ :

```
(%i1) load("plotdf")$
```

```
(%i2) plotdf(exp(-x)+y,[trajectory_at,2,-0.1])$
```

- To obtain the direction field for the equation  $\text{diff}(y, x) = x - y^2$  and the solution with initial condition  $y(-1) = 3$ , we can use the command:

```
(%i3) plotdf(x-y^2,[xfun,"sqrt(x);-sqrt(x)"],
 [trajectory_at,-1,3],
 [direction,forward],
 [yradius,5],[xcenter,6])$
```

- The following example shows the direction field of a harmonic oscillator, and the integral curve through  $(x, y) = (6, 0)$ , with a slider that will allow you to change the value of  $m$  interactively ( $k$  is fixed at 2):

```
(%i4) plotdf([y,-k*x/m],[parameters,"m=2,k=2"],
 [sliders,"m=1:5"],
 [trajectory_at,6,0])$
```

- The direction field of a Duffing equation:

```
(%i5) plotdf([y,-(k*x + c*y + l*x^3)/m],
 [parameters,"k=-1,m=1.0,c=0,l=1"],
 [sliders,"k=-2:2,m=-1:1"],
 [bbox,-3,-3,3,3],[tstep,0.1])$
```

- The direction field for a damped pendulum, including the solution for the given initial conditions, and with a slider that can be used to change the value of the mass  $m$ :

```
(%i6) plotdf([y,-g*sin(x)/l - b*y/m/l],
 [parameters,"g=9.8,l=0.5,m=0.3,b=0.05"],
 [trajectory_at,1.05,-9],[tstep,0.01],
 [xradius,6],[yradius,14],
 [xcenter,-4],[direction,forward],[nsteps,300],
 [sliders,"m=0.1:1"])$
```

To use this function write first `load("plotdf")`.



## 64 simplex

### 64.1 Introducción a simplex

`simplex` is a package for linear optimization using the simplex algorithm.

Example:

```
(%i1) load("simplex")$
(%i2) minimize_sx(x+y, [3*x+2*y>2, x+4*y>3]);
 9 7 1
(%o2) [--, [y = --, x = -]]
 10 10 5
```

### 64.2 Definiciones para simplex

#### `epsilon_sx`

Option variable

Default value:  $10^{-8}$

Epsilon used for numerical computations in `linear_program`.

See also: `linear_program`.

#### `linear_program (A, b, c)`

Function

`linear_program` is an implementation of the simplex algorithm. `linear_program(A, b, c)` computes a vector  $x$  for which  $c \cdot x$  is minimum possible among vectors for which  $A \cdot x = b$  and  $x \geq 0$ . Argument  $A$  is a matrix and arguments  $b$  and  $c$  are lists.

`linear_program` returns a list which contains the minimizing vector  $x$  and the minimum value  $c \cdot x$ . If the problem is not bounded, it returns "Problem not bounded!" and if the problem is not feasible, it returns "Problem not feasible!".

To use this function first load the `simplex` package with `load(simplex);`.

Example:

```
(%i2) A: matrix([1,1,-1,0], [2,-3,0,-1], [4,-5,0,0])$
(%i3) b: [1,1,6]$
(%i4) c: [1,-2,0,0]$
(%i5) linear_program(A, b, c);
 13 19 3
(%o5) [--, 4, --, 0], - -]
 2 2 2
```

See also: `minimize_sx`, `scale_sx`, and `epsilon_sx`.

#### `maximize_sx (obj, cond, [pos])`

Function

Maximizes linear objective function  $obj$  subject to some linear constraints  $cond$ . See `minimize_sx` for detailed description of arguments and return value.

See also: `minimize_sx`.

**minimize\_sx** (*obj*, *cond*, [*pos*])

Function

Minimizes a linear objective function *obj* subject to some linear constraints *cond*. *cond* a list of linear equations or inequalities. In strict inequalities  $>$  is replaced by  $\geq$  and  $<$  by  $\leq$ . The optional argument *pos* is a list of decision variables which are assumed to be positive.

If the minimum exists, **minimize\_sx** returns a list which contains the minimum value of the objective function and a list of decision variable values for which the minimum is attained. If the problem is not bounded, **minimize\_sx** returns "Problem not bounded!" and if the problem is not feasible, it returns "Problem not feasible!".

The decision variables are not assumed to be nonnegative by default. If all decision variables are nonnegative, set **nonnegative\_sx** to **true**. If only some of decision variables are positive, list them in the optional argument *pos* (note that this is more efficient than adding constraints).

**minimize\_sx** uses the simplex algorithm which is implemented in maxima **linear\_program** function.

To use this function first load the **simplex** package with **load(simplex);**.

Examples:

```
(%i1) minimize_sx(x+y, [3*x+y=0, x+2*y>2]);
 4 6 2
(%o1) [-, [y = -, x = - -]]
 5 5 5

(%i2) minimize_sx(x+y, [3*x+y>0, x+2*y>2]), nonnegative_sx=true;
(%o2) [1, [y = 1, x = 0]]

(%i3) minimize_sx(x+y, [3*x+y=0, x+2*y>2]), nonnegative_sx=true;
(%o3) Problem not feasible!

(%i4) minimize_sx(x+y, [3*x+y>0]);
(%o4) Problem not bounded!
```

See also: **maximize\_sx**, **nonnegative\_sx**, **epsilon\_sx**.

**nonnegative\_sx**

Option variable

If **nonnegative\_sx** is true all decision variables to **minimize\_sx** and **maximize\_sx** are assumed to be positive. The default value is **false**.

See also: **minimize\_sx**.

## 65 simplification

### 65.1 Introducción a simplification

The directory `maxima/share/simplification` contains several scripts which implement simplification rules and functions, and also some functions not related to simplification.

### 65.2 Definiciones para simplification

#### absimp

Add on package

The **absimp** package contains pattern-matching rules that extend the built-in simplification rules for the **abs** and **signum** functions. **absimp** respects relations established with the built-in **assume** function and by declarations such as **modeddeclare** (**m**, **even**, **n**, **odd**) for even or odd integers.

**absimp** defines **unitramp** and **unitstep** functions in terms of **abs** and **signum**.

**load (absimp)** loads this package. **demo (absimp)** shows a demonstration of this package.

Examples:

```
(%i1) load (absimp);
(%o1) /usr/share/maxima/5.9.2/share/simplification/absimp.mac
(%i2) (abs (x))^2;
(%o2) x^2
(%i3) diff (abs (x), x);
(%o3) x

 abs(x)
(%i4) cosh (abs (x));
(%o4) cosh(x)
```

#### facexp

Add on package

The **facexp** package contains several related functions that provide the user with the ability to structure expressions by controlled expansion. This capability is especially useful when the expression contains variables that have physical meaning, because it is often true that the most economical form of such an expression can be obtained by fully expanding the expression with respect to those variables, and then factoring their coefficients. While it is true that this procedure is not difficult to carry out using standard Maxima functions, additional fine-tuning may also be desirable, and these finishing touches can be more difficult to apply.

The function **facsum** and its related forms provide a convenient means for controlling the structure of expressions in this way. Another function, **collectterms**, can be used to add two or more expressions that have already been simplified to this form, without resimplifying the whole expression again. This function may be useful when the expressions are very large.

**load (facexp)** loads this package. **demo (facexp)** shows a demonstration of this package.

**facsum** (*expr*, *arg\_1*, ..., *arg\_n*)

Function

Returns a form of *expr* which depends on the arguments *arg\_1*, ..., *arg\_n*. The arguments can be any form suitable for **ratvars**, or they can be lists of such forms. If the arguments are not lists, then the form returned is fully expanded with respect to the arguments, and the coefficients of the arguments are factored. These coefficients are free of the arguments, except perhaps in a non-rational sense.

If any of the arguments are lists, then all such lists are combined into a single list, and instead of calling **factor** on the coefficients of the arguments, **facsum** calls itself on these coefficients, using this newly constructed single list as the new argument list for this recursive call. This process can be repeated to arbitrary depth by nesting the desired elements in lists.

It is possible that one may wish to **facsum** with respect to more complicated subexpressions, such as  $\log(x + y)$ . Such arguments are also permissible. With no variable specification, for example **facsum** (*expr*), the result returned is the same as that returned by **ratsimp** (*expr*).

Occasionally the user may wish to obtain any of the above forms for expressions which are specified only by their leading operators. For example, one may wish to **facsum** with respect to all **log**'s. In this situation, one may include among the arguments either the specific **log**'s which are to be treated in this way, or alternatively, either the expression **operator(log)** or **'operator(log)**. If one wished to **facsum** the expression *expr* with respect to the operators *op\_1*, ..., *op\_n*, one would evaluate **facsum** (*expr*, **operator** (*op\_1*, ..., *op\_n*)). The **operator** form may also appear inside list arguments.

In addition, the setting of the switches **facsum\_combine** and **nextlayerfactor** may affect the result of **facsum**.

**nextlayerfactor**

Global variable

Default value: **false**

When **nextlayerfactor** is **true**, recursive calls of **facsum** are applied to the factors of the factored form of the coefficients of the arguments.

When **false**, **facsum** is applied to each coefficient as a whole whenever recursive calls to **facsum** occur.

Inclusion of the atom **nextlayerfactor** in the argument list of **facsum** has the effect of **nextlayerfactor: true**, but for the next level of the expression *only*. Since **nextlayerfactor** is always bound to either **true** or **false**, it must be presented single-quoted whenever it appears in the argument list of **facsum**.

**facsum\_combine**

Global variable

Default value: **true**

**facsum\_combine** controls the form of the final result returned by **facsum** when its argument is a quotient of polynomials. If **facsum\_combine** is **false** then the form will be returned as a fully expanded sum as described above, but if **true**, then the expression returned is a ratio of polynomials, with each polynomial in the form described above.

The **true** setting of this switch is useful when one wants to **facsum** both the numerator and denominator of a rational expression, but does not want the denominator to be multiplied through the terms of the numerator.

**factorfacsum** (*expr*, *arg\_1*, ... *arg\_n*) Function

Returns a form of *expr* which is obtained by calling **facsum** on the factors of *expr* with *arg\_1*, ... *arg\_n* as arguments. If any of the factors of *expr* is raised to a power, both the factor and the exponent will be processed in this way.

**collectterms** (*arg\_1*, ..., *arg\_n*) Function

If several expressions have been simplified with **facsum**, **factorfacsum**, **factenexpand**, **facexptn** or **factorfacexptn**, and they are to be added together, it may be desirable to combine them using the function **collectterms**. **collectterms** can take as arguments all of the arguments that can be given to these other associated functions with the exception of **nextlayerfactor**, which has no effect on **collectterms**. The advantage of **collectterms** is that it returns a form similar to **facsum**, but since it is adding forms that have already been processed by **facsum**, it does not need to repeat that effort. This capability is especially useful when the expressions to be summed are very large.

**rempart** (*expr*, *n*) Function

Removes part *n* from the expression *expr*.

If *n* is a list of the form [*l*, *m*] then parts *l* thru *m* are removed.

**wronskian** ([*f\_1*, ..., *f\_n*], *x*) Function

Returns the Wronskian matrix of the functions *f\_1*, ..., *f\_n* in the variable *x*.

*f\_1*, ..., *f\_n* may be the names of user-defined functions, or expressions in the variable *x*.

The determinant of the Wronskian matrix is the Wronskian determinant of the set of functions. The functions are linearly dependent if this determinant is zero.

**tracematrix** (*M*) Function

Returns the trace (sum of the diagonal elements) of matrix *M*.

**rational** (*z*) Function

Multiplies numerator and denominator of *z* by the complex conjugate of denominator, thus rationalizing the denominator. Returns canonical rational expression (CRE) form if given one, else returns general form.

**logand** (*x,y*) Function

Returns logical (bit-wise) "and" of arguments *x* and *y*.

**logor** (*x,y*) Function

Returns logical (bit-wise) "or" of arguments *x* and *y*.



- logxor** (*x*,*y*) Function  
 Returns logical (bit-wise) exclusive-or of arguments *x* and *y*.
- nonzeroandfreeof** (*x*, *expr*) Function  
 Returns **true** if *expr* is nonzero and **freeof** (*x*, *expr*) returns **true**. Returns **false** otherwise.
- linear** (*expr*, *x*) Function  
 When *expr* is an expression linear in variable *x*, **linear** returns  $a*x + b$  where *a* is nonzero, and *a* and *b* are free of *x*. Otherwise, **linear** returns *expr*.
- quadratic** (*expr*, *x*) Function  
 When *expr* is an expression quadratic in variable *x*, **quadratic** returns  $a*x^2 + b*x + c$  where *a* is nonzero, and *a*, *b*, and *c* are free of *x*. Otherwise, **quadratic** returns *expr*.
- gcddivide** (*p*, *q*) Function  
 When **takegcd** is **true**, **gcddivide** divides the polynomials *p* and *q* by their greatest common divisor and returns the ratio of the results.  
 When **takegcd** is **false**, **gcddivide** returns the ratio  $p/q$ .
- arithmetic** (*a*, *d*, *n*) Function  
 Returns the *n*-th term of the arithmetic series *a*,  $a + d$ ,  $a + 2*d$ , ...,  $a + (n - 1)*d$ .
- geometric** (*a*, *r*, *n*) Function  
 Returns the *n*-th term of the geometric series *a*,  $a*r$ ,  $a*r^2$ , ...,  $a*r^{(n - 1)}$ .
- harmonic** (*a*, *b*, *c*, *n*) Function  
 Returns the *n*-th term of the harmonic series  $a/b$ ,  $a/(b + c)$ ,  $a/(b + 2*c)$ , ...,  $a/(b + (n - 1)*c)$ .
- arithsum** (*a*, *d*, *n*) Function  
 Returns the sum of the arithmetic series from 1 to *n*.
- geosum** (*a*, *r*, *n*) Function  
 Returns the sum of the geometric series from 1 to *n*. If *n* is infinity (**inf**) then a sum is finite only if the value of *r* is not equal to 1.
- gaussprob** (*x*) Function  
 Returns the Gaussian probability function  $\%e^{-(x^2/2)} / \text{sqrt}(2*\%pi)$ .
- gd** (*x*) Function  
 Returns the Gudermannian function  $2 * \text{atan}(\%e^x - \%pi/2)$ .

**agd** ( $x$ ) Function  
Returns the inverse Gudermannian function  $\log (\tan (\pi/4 + x/2))$ .

**vers** ( $x$ ) Function  
Returns the versed sine  $1 - \cos (x)$ .

**covers** ( $x$ ) Function  
Returns the covered sine  $1 - \sin (x)$ .

**exsec** ( $x$ ) Function  
Returns the exsecant  $\sec (x) - 1$ .

**hav** ( $x$ ) Function  
Returns the haversine  $(1 - \cos(x))/2$ .

**combination** ( $n, r$ ) Function  
Returns the number of combinations of  $n$  objects taken  $r$  at a time.

**permutation** ( $n, r$ ) Function  
Returns the number of permutations of  $r$  objects selected from a set of  $n$  objects.

**ineq** Add on package

The **ineq** package contains simplification rules for inequalities.

Be careful about using parentheses around the inequalities: when the user types in  $(A > B) + (C = 5)$  the result is  $A + C > B + 5$ , but  $A > B + C = 5$  is a syntax error, and  $(A > B + C) = 5$  is something else entirely.

Do **disrule (all)** to see a complete listing of the rule definitions.

The user will be queried if Maxima is unable to decide the sign of a quantity multiplying an inequality.

The most common mis-feature is illustrated by:

```
eq: a > b;
2*eq;
% - eq;
```

Another problem is 0 times an inequality; the default to have this turn into 0 has been left alone. However, if you type **X\*some\_inequality** and Maxima asks about the sign of **X** and you respond **zero** (or **z**), the program returns **X\*some\_inequality** and not use the information that **X** is 0. You should do **ev (% , x: 0)** in such a case, as the database will only be used for comparison purposes in decisions, and not for the purpose of evaluating **X**.

The user may note a slower response when this package is loaded, as the simplifier is forced to examine more rules than without the package, so you might wish to remove the rules after making use of them. Do **kill (rules)** to eliminate all of the rules (including any that you might have defined); or you may be more selective by killing only some of them; or use **remrule** on a specific rule.

Note that if you load this package after defining your own rules you will clobber your rules that have the same name. The rules in this package are: `*rule1`, ..., `*rule8`, `+rule1`, ..., `+rule18`, and you must enclose the rulename in quotes to refer to it, as in `remrule ("+", "+rule1")` to specifically remove the first rule on "+" or `disprule ("*rule2")` to display the definition of the second multiplicative rule.

### **reduce\_consts** (*expr*)

Function

Replaces constant subexpressions of *expr* with constructed constant atoms, saving the definition of all these constructed constants in the list of equations `const_eqns`, and returning the modified *expr*. Those parts of *expr* are constant which return `true` when operated on by the function `constantp`. Hence, before invoking `reduce_consts`, one should do

```
declare ([objects to be given the constant property], constant)$
```

to set up a database of the constant quantities occurring in your expressions.

If you are planning to generate Fortran output after these symbolic calculations, one of the first code sections should be the calculation of all constants. To generate this code segment, do

```
map ('fortran, const_eqns)$
```

Variables besides `const_eqns` which affect `reduce_consts` are:

`const_prefix` (default value: `xx`) is the string of characters used to prefix all symbols generated by `reduce_consts` to represent constant subexpressions.

`const_counter` (default value: 1) is the integer index used to generate unique symbols to represent each constant subexpression found by `reduce_consts`.

`load (rducon)` loads this function. `demo (rducon)` shows a demonstration of this function.

### **gcfac** (*expr*)

Function

`gcfac` is a factoring function that attempts to apply the same heuristics which scientists apply in trying to make expressions simpler. `gcfac` is limited to monomial-type factoring. For a sum, `gcfac` does the following:

1. Factors over the integers.
2. Factors out the largest powers of terms occurring as coefficients, regardless of the complexity of the terms.
3. Uses (1) and (2) in factoring adjacent pairs of terms.
4. Repeatedly and recursively applies these techniques until the expression no longer changes.

Item (3) does not necessarily do an optimal job of pairwise factoring because of the combinatorially-difficult nature of finding which of all possible rearrangements of the pairs yields the most compact pair-factored result.

`load (scifac)` loads this function. `demo (scifac)` shows a demonstration of this function.

### **sqrtdenest** (*expr*)

Function

Denests `sqrtdenest` of simple, numerical, binomial surds, where possible. E.g.

```
(c1) sqrt(sqrt(3)/2+1)/sqrt(11*sqrt(2)-12);
```

$$(d1) \quad \frac{\sqrt{\frac{\sqrt{3}}{2} + 1}}{\sqrt{11\sqrt{2} - 12}}$$

```
(c2) sqrtdenest(%);
```

$$(d2) \quad \frac{\sqrt{\frac{\sqrt{3}}{2} + \frac{1}{2}}}{\sqrt[3]{\frac{1}{4} - \frac{3}{4}}}$$

Sometimes it helps to apply `sqrtdenest` more than once, on such as (19601-13860  $\sqrt{2}$ )<sup>7/4</sup>.

`load (sqdnst)` loads this function.



## 66 solve\_rec

### 66.1 Introducción a solve\_rec

`solve_rec` is a package for solving linear recurrences with polynomial coefficients.

A demo is available with `demo(solve_rec);`.

Example:

```
(%i1) load("solve_rec")$
(%i2) solve_rec((n+4)*s[n+2] + s[n+1] - (n+1)*s[n], s[n]);
```

$$s_n = \frac{{}_2k_1 (2n+3)(-1)^n}{(n+1)(n+2)} + \frac{{}_2k_2}{(n+1)(n+2)}$$

```
(%o2)
```

### 66.2 Definiciones para solve\_rec

**reduce\_order** (*rec, sol, var*)

Function

Reduces the order of linear recurrence *rec* when a particular solution *sol* is known.

The reduced recurrence can be used to get other solutions.

Example:

```
(%i3) rec: x[n+2] = x[n+1] + x[n]/n;
```

$$x_{n+2} = x_{n+1} + \frac{x_n}{n}$$

```
(%o3)
```

```
(%i4) solve_rec(rec, x[n]);
```

WARNING: found some hypergeometrical solutions!

```
(%o4)
```

$$x_n = {}_1k_n$$

```
(%i5) reduce_order(rec, n, x[n]);
```

```
(%t5)
```

$$\frac{x_n}{n} = n \frac{{}_z}{n}$$

$$\frac{n-1}{n} = \frac{{}_u}{n} \frac{{}_j}{j}$$

$$\frac{n-1}{n} = \frac{{}_u}{n} \frac{{}_j}{j}$$

```
(%o6)
```

```
(%i6) solve_rec((n+2)*%u[n+1] + %u[n], %u[n]);
```

$$(n+2)u_{n+1} + u_n = 0$$

$$(\%o6) \quad \frac{{}^k n (-1)^n}{(n+1)!}$$

So the general solution is

$$\begin{aligned} & \frac{{}^k n}{2} > \frac{(-1)^n}{(n+1)!} + \frac{{}^k n}{1} \\ & n = 0 \end{aligned}$$

### simplify\_products

Option variable

If `simplify_products` is `true`, `solve_rec` will try to simplify products in result. The default value is `true`.

See also: `solve_rec`.

### solve\_rec (eqn, var, [init])

Function

Solves for hypergeometrical solutions to linear recurrence `eqn` with polynomials coefficient in variable `var`. Optional arguments `init` are initial conditions.

`solve_rec` can solve linear recurrences with constant coefficients, finds hypergeometrical solutions to homogeneous linear recurrences with polynomial coefficients, rational solutions to linear recurrences with polynomial coefficients and can solve Ricatti type recurrences.

Note that the running time of the algorithm used to find hypergeometrical solutions is exponential in the degree of the leading and trailing coefficient.

To use this function first load the `solve_rec` package with `load(solve_rec);`.

Example of linear recurrence with constant coefficients:

```
(%i2) solve_rec(a[n]=a[n-1]+a[n-2]+n/2^n, a[n]);
```

$$(\%o2) \quad a_n = \frac{(\sqrt{5}-1)^n {}^k n (-1)^n}{2^n} - \frac{n}{5 \cdot 2^n} + \frac{(\sqrt{5}+1)^n {}^k n}{2^n} - \frac{n}{5 \cdot 2^n}$$

Example of linear recurrence with polynomial coefficients:

```
(%i7) 2*x*(x+1)*y[x] - (x^2+3*x-2)*y[x+1] + (x-1)*y[x+2];
(%o7) (x - 1) y 2 (x + 3 x - 2) y + 2 x (x + 1) y
 x + 2 x + 1 x
(%i8) solve_rec(%, y[x], y[1]=1, y[3]=3);
(%o9)
 x
 3 2 x!
y = ---- - --
x 4 2
```

Example of Ricatti type recurrence:

```
(%i2) x*y[x+1]*y[x] - y[x+1]/(x+2) + y[x]/(x-1) = 0;
(%o2)
 y y
 x + 1 x
x y y - ---- + ---- = 0
x x + 1 x + 2 x - 1
(%i3) solve_rec(%, y[x], y[3]=5)$
(%i4) ratsimp(minfactorial(factcomb(%)));
(%o4) y = - ----
 x
 6 5 4 3 2
 5 x - 3 x - 25 x + 15 x + 20 x - 12 x - 1584
```

See also: `solve_rec_rat`, `simplify_products`, and `product_use_gamma`.

### **solve\_rec\_rat** (*eqn*, *var*, [*init*])

Function

Solves for rational solutions to linear recurrences. See `solve_rec` for description of arguments.

To use this function first load the `solve_rec` package with `load(solve_rec);`.

Example:

```
(%i1) (x+4)*a[x+3] + (x+3)*a[x+2] - x*a[x+1] + (x^2-1)*a[x];
(%o1) (x + 4) a + (x + 3) a - x a
 x + 3 x + 2 x + 1
 2
 + (x - 1) a
 x
(%i2) solve_rec_rat(% = (x+2)/(x+1), a[x]);
(%o2)
 1
a = ----
x (x - 1) (x + 1)
```

See also: `solve_rec`.

### **product\_use\_gamma**

Option variable

When simplifying products, `solve_rec` introduces gamma function into the expression if `product_use_gamma` is `true`. The default value is `true`.

See also: `simplify_products`, `solve_rec`.



**summand\_to\_rec** (*summand*, *k*, *n*)

Function

Returns the recurrence satisfied by the sum

```

inf
====
\
> summand
/
====
k = minf

```

where *summand* is hypergeometrical in *k* and *n*. Zeilberger package must be loaded before using this function.

To use this function first load the `solve_rec` package with `load(solve_rec)` and the `zeilberger` package with `load(zeilberger)`.

```

(%i17) load("zeilberger")$
(%i18) summand: binom(3*k+1,k)*binom(3*(n-k),n-k)/(3*k+1)$
(%i19) summand_to_rec(summand, k, n);
Dependent equations eliminated: (3 2)
(%o19) - 4 (n + 2) (2 n + 3) (2 n + 5) sm
 n + 2
+ 12 (2 n + 3) (9 n + 27 n + 22) sm
 n + 1
- 81 (n + 1) (3 n + 2) (3 n + 4) sm
 n
(%i21) sum(''summand, k, 0, n), n=0;
(%o21) 1
(%i22) sum(''summand, k, 0, n), n=1;
(%o22) 4
(%i23) product_use_gamma: false$
(%i24) solve_rec(%o19, sm[n], sm[0]=1, sm[1]=4);
 n - 1 n - 1
 /===\ /===\
 ! ! ! !
 (! ! (3 %j + 2)) (! ! (3 %j + 4)) 3
 ! !
 %j = 0 %j = 0
(%o24) sm = -----
 n
 n - 1
 /===\
 ! !
 (! ! (2 %j + 3)) 2 n!
 ! !
 %j = 0

```

## 67 stirling

### 67.1 Definiciones para stirling

**stirling** ( $z, n$ )

Function

Replace `gamma(x)` with the  $O(1/x^{2n-1})$  Stirling formula. when  $n$  isn't a nonnegative integer, signal an error.

Reference: Abramowitz & Stegun, "Handbook of mathematical functions", 6.1.40.

Examples:

```
(%i1) load (stirling)$

(%i2) stirling(gamma(%alpha+x)/gamma(x),1);
 1/2 - x x + %alpha - 1/2
(%o2) x (x + %alpha)
 1 1
 ----- - ---- - %alpha
 12 (x + %alpha) 12 x
 %e

(%i3) taylor(%,x,inf,1);
 %alpha 2 %alpha
 %alpha x %alpha - x %alpha
(%o3)/T/ x + ----- + . . .
 2 x

(%i4) map('factor,%);
 %alpha (%alpha - 1) %alpha x
(%o4) x + -----
 2
```

The function `stirling` knows the difference between the variable 'gamma' and the function gamma:

```
(%i5) stirling(gamma + gamma(x),0);
 x - 1/2 - x
(%o5) gamma + sqrt(2) sqrt(%pi) x %e
(%i6) stirling(gamma(y) + gamma(x),0);
 y - 1/2 - y
(%o6) sqrt(2) sqrt(%pi) y %e
 x - 1/2 - x
 + sqrt(2) sqrt(%pi) x %e
```

To use this function write first `load("stirling")`.



## 68 stringproc

### 68.1 Introducción al procesamiento de cadenas

stringproc.lisp enlarges Maximas capabilities of working with strings. If you find bugs or if you need more functions, feel free to mail. If you use it for education, please let me know.

Load stringproc.lisp by typing `load("stringproc");`.

Remark: stringproc.lisp is written for Maxima 5.9.2. Version 5.9.1 should use stringproc1.lisp.

In Maxima a string is easily constructed by typing `"text"`. Note that Maxima-strings are no Lisp-strings and vice versa. Tests can be done with `stringp` respectively `lstringp`. If for some reasons you have a value, that is a Lisp-string, maybe when using Maxima-function `sconcat`, you can convert via `sunlisp`.

```
(%i1) m: "text";
(%o1) text
(%i2) [stringp(m),lstringp(m)];
(%o2) [true, false]
(%i3) l: sconcat("text");
(%o3) text
(%i4) [stringp(l),lstringp(l)];
(%o4) [false, true]
(%i5) stringp(sunlisp(l));
(%o5) true
```

All functions in stringproc.lisp, that return strings, return Maxima-strings.

Characters are introduced as Maxima-strings of length 1. Of course, these are no Lisp-characters. Tests can be done with `charp` (respectively `lcharp` and conversion from Lisp to Maxima with `cunlisp`).

```
(%i1) c: "e";
(%o1) e
(%i2) [charp(c),lcharp(c)];
(%o2) [true, false]
(%i3) supcase(c);
(%o3) E
(%i4) charp(%);
(%o4) true
```

Again, all functions in stringproc.lisp, that return characters, return Maxima-characters. Due to the fact, that the introduced characters are strings of length 1, you can use a lot of string functions also for characters. As seen, `supcase` is one example.

It is important to know, that the first character in a Maxima-string is at position 1. This is designed due to the fact that the first element in a Maxima-list is at position 1 too. See definitions of `charat` and `charlist` for examples.

In applications string-functions are often used when working with files. You will find some useful stream- and print-functions in stringproc.lisp. The following example shows some of the here introduced functions at work.

Example:

Let file contain Maxima console I/O, saved with 'Save Console to File' or with copy and paste. `extracti` then extracts the values of all input labels to a batchable file, which path is the return value. The batch process can directly be started with `batch(%)`. Note that `extracti` fails if at least one label is damaged, maybe due to erasing the `)`. Or if there are input lines from a batch process. In this case terminators are missing. It fails too, if there are some characters behind the terminators, maybe due to comment.

```
extracti(file):= block(
 [s1: openr(file), ifile: sconc(file, ".in"), line, nl: false],
 s2: openw(ifile),

 while (stringp(line: readline(s1))) do (
 if ssearch(sconc("(", inchar), line) = 1 then (
 line: strim(" ", substring(line, ssearch(")", line)+1)),
 printf(s2, "~a~%", line),
 checklast(line))
 else if nl then (
 line: strimr(" ", line),
 printf(s2, "~a~%", line),
 checklast(line))),

 close(s1), close(s2),
 ifile)$

checklast(line):= block(
 [last: charat(line, slength(line))],
 if cequal(last, ";") or cequal(last, "$") then
 nl:false else nl:true)$
```

File 'C:\home\maxima\test.out':

```
(%i1) f(x):= sin(x)$
(%i2) diff(f(x), x);
(%o2) cos(x)
(%i3) df(x):= ' ';
(%o3) df(x) := cos(x)
(%i4) df(0);
(%o4) 1
```

Maxima:

```
(%i11) extracti("C:\\home\\maxima\\test.out");
(%o11) C:\home\maxima\test.out.in
(%i12) batch(%);

batching #pC:/home/maxima/test.out.in
(%i13) f(x) := sin(x)
(%i14) diff(f(x), x)
(%o14) cos(x)
(%i15) df(x) := cos(x)
(%o15) df(x) := cos(x)
(%i16) df(0)
```

```
(%o16) 1
```

## 68.2 Definiciones para entrada y salida

Example:

```
(%i1) s: openw("C:\\home\\file.txt");
(%o1) #<output stream C:\home\file.txt>
(%i2) control: "~2tAn atom: ~20t~a~%2tand a list: ~20t~{~r ~}~%2tand an integer:
(%i3) printf(s,control, 'true',[1,2,3],42)$
(%o3) false
(%i4) close(s);
(%o4) true
(%i5) s: openr("C:\\home\\file.txt");
(%o5) #<input stream C:\home\file.txt>
(%i6) while stringp(tmp:readline(s)) do print(tmp)$
 An atom: true
 and a list: one two three
 and an integer: 42
(%i7) close(s)$
```

**close** (*stream*) Function  
 Closes *stream* and returns **true** if *stream* had been open.

**flength** (*stream*) Function  
 Returns the number of elements in *stream*.

**fposition** (*stream*) Function  
**fposition** (*stream*, *pos*) Function  
 Returns the current position in *stream*, if *pos* is not used. If *pos* is used, **fposition** sets the position in *stream*. *pos* has to be a positive number, the first element in *stream* is in position 1.

**freshline** () Function  
**freshline** (*stream*) Function  
 Writes a new line to *stream*, if the position is not at the beginning of a line. **freshline** does not work properly with the streams **true** and **false**.

**newline** () Function  
**newline** (*stream*) Function  
 Writes a new line to *stream*. **newline** does not work properly with the streams **true** and **false**. See **sprint** for an example of using **newline**.

**opena** (*file*) Function  
 Returns an output stream to *file*. If an existing file is opened, **opena** appends elements at the end of file.

**openr** (*file*) Function  
 Returns an input stream to *file*. If *file* does not exist, it will be created.

**openw** (*file*) Function  
 Returns an output stream to *file*. If *file* does not exist, it will be created. If an existing file is opened, **openw** destructively modifies *file*.

**printf** (*dest, string*) Function  
**printf** (*dest, string, expr\_1, ..., expr\_n*) Function

**printf** is like **FORMAT** in Common Lisp. (From gcl.info: "format produces formatted output by outputting the characters of control-string *string* and observing that a tilde introduces a directive. The character after the tilde, possibly preceded by prefix parameters and modifiers, specifies what kind of formatting is desired. Most directives use one or more elements of *args* to create their output.")

The following description and the examples may give an idea of using **printf**. See Lisp reference for more information. Note that there are some directives, which do not work in Maxima. For example, `~:[` fails. **printf** is designed with the intention, that `~s` is read as `~a`. Also note that the selection directive `~[` is zero-indexed.

|                     |                                                               |
|---------------------|---------------------------------------------------------------|
| <code>~%</code>     | new line                                                      |
| <code>~&amp;</code> | fresh line                                                    |
| <code>~t</code>     | tab                                                           |
| <code>~\$</code>    | monetary                                                      |
| <code>~d</code>     | decimal integer                                               |
| <code>~b</code>     | binary integer                                                |
| <code>~o</code>     | octal integer                                                 |
| <code>~x</code>     | hexadecimal integer                                           |
| <code>~br</code>    | base-b integer                                                |
| <code>~r</code>     | spell an integer                                              |
| <code>~p</code>     | plural                                                        |
| <code>~f</code>     | floating point                                                |
| <code>~e</code>     | scientific notation                                           |
| <code>~g</code>     | <code>~f</code> or <code>~e</code> , depending upon magnitude |
| <code>~a</code>     | as printed by Maxima function <b>print</b>                    |
| <code>~s</code>     | like <code>~a</code>                                          |
| <code>~~</code>     | <code>~</code>                                                |
| <code>~&lt;</code>  | justification, <code>~&gt;</code> terminates                  |
| <code>~(</code>     | case conversion, <code>~)</code> terminates                   |
| <code>~[</code>     | selection, <code>~]</code> terminates                         |
| <code>~{</code>     | iteration, <code>~}</code> terminates                         |

```
(%i1) printf(false, "~s ~a ~4f ~a ~@r",
"String",sym,bound,sqrt(8),144), bound = 1.234;
(%o1) String sym 1.23 2*sqrt(2) CXLIV
(%i2) printf(false,"~{~a ~}",["one",2,"THREE"]);
(%o2) one 2 THREE
(%i3) printf(true,"~{~{~9,1f ~}~%~}",mat),
mat = args(matrix([1.1,2,3.33],[4,5,6],[7,8.88,9]))$
 1.1 2.0 3.3
 4.0 5.0 6.0
```

```

 7.0 8.9 9.0
(%i4) control: "~:(~r~) bird~p ~[is~;are~] singing."$
(%i5) printf(false,control, n,n,if n=1 then 0 else 1), n=2;
(%o5) Two birds are singing.

```

If *dest* is a stream or `true`, then `printf` returns `false`. Otherwise, `printf` returns a string containing the output.

**readline** (*stream*) Function

Returns a string containing the characters from the current position in *stream* up to the end of the line or `false` if the end of the file is encountered.

**sprint** (*expr\_1*, ..., *expr\_n*) Function

(From maxima.info (5.9.1): "Evaluates and displays its arguments one after the other 'on a line' starting at the leftmost position. The numbers are printed with the '-' right next to the number, and it disregards line length.")

```

(%i1) for n:0 thru 16 do sprint(fib(n))$0 1 1 2 3 5 8 13
21 34 55 89 144 233 377 610 987

```

If you prefer a line break before printing, add `,newline()`. See `ascii` for an example.

## 68.3 Definiciones para caracteres

**alphacharp** (*char*) Function

Returns `true` if *char* is an alphabetic character.

**alphanumericp** (*char*) Function

Returns `true` if *char* is an alphabetic character or a digit.

**ascii** (*int*) Function

Returns the character corresponding to the ASCII number *int*. (  $-1 < \text{int} < 256$  )

```

(%i1) for n from 0 thru 255 do (tmp: ascii(n),
if alphacharp(tmp) then sprint(tmp)), newline()$
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z a b c d e f g h i j k l m n o p q r s t u v w x y z

```

**cequal** (*char\_1*, *char\_2*) Function

Returns `true` if *char\_1* and *char\_2* are the same.

**cequalignore** (*char\_1*, *char\_2*) Function

Like `cequal` but ignores case.

**cgreaterp** (*char\_1*, *char\_2*) Function

Returns `true` if the ASCII number of *char\_1* is greater than the number of *char\_2*.

**cgreaterpignore** (*char\_1*, *char\_2*) Function

Like `cgreaterp` but ignores case.



|                                                                                                                                                                                                                                                    |          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <b>charp</b> ( <i>obj</i> )                                                                                                                                                                                                                        | Function |
| Returns <b>true</b> if <i>obj</i> is a Maxima-character. See introduction for example.                                                                                                                                                             |          |
| <b>cint</b> ( <i>char</i> )                                                                                                                                                                                                                        | Function |
| Returns the ASCII number of <i>char</i> .                                                                                                                                                                                                          |          |
| <b>clessp</b> ( <i>char_1</i> , <i>char_2</i> )                                                                                                                                                                                                    | Function |
| Returns <b>true</b> if the ASCII number of <i>char_1</i> is less than the number of <i>char_2</i> .                                                                                                                                                |          |
| <b>clesspignore</b> ( <i>char_1</i> , <i>char_2</i> )                                                                                                                                                                                              | Function |
| Like <b>clessp</b> but ignores case.                                                                                                                                                                                                               |          |
| <b>constituent</b> ( <i>char</i> )                                                                                                                                                                                                                 | Function |
| Returns <b>true</b> if <i>char</i> is a graphic character and not the space character. A graphic character is a character one can see, plus the space character. ( <b>constituent</b> is defined by Paul Graham, ANSI Common Lisp, 1996, page 67.) |          |
| <pre>(%i1) for n from 0 thru 255 do ( tmp: ascii(n), if constituent(tmp) then sprint(tmp) ), newline()\$ ! " # % ' ( ) * + , - . / 0 1 2 3 4 5 6 7 8 9 : ; &lt; = &gt; ? @ A B C D E F G H</pre>                                                   |          |
| <b>cunlisp</b> ( <i>lisp_char</i> )                                                                                                                                                                                                                | Function |
| Converts a Lisp-character into a Maxima-character. (You wont need it.)                                                                                                                                                                             |          |
| <b>digitcharp</b> ( <i>char</i> )                                                                                                                                                                                                                  | Function |
| Returns <b>true</b> if <i>char</i> is a digit.                                                                                                                                                                                                     |          |
| <b>lcharp</b> ( <i>obj</i> )                                                                                                                                                                                                                       | Function |
| Returns <b>true</b> if <i>obj</i> is a Lisp-character. (You wont need it.)                                                                                                                                                                         |          |
| <b>lowercasep</b> ( <i>char</i> )                                                                                                                                                                                                                  | Function |
| Returns <b>true</b> if <i>char</i> is a lowercase character.                                                                                                                                                                                       |          |
| <b>newline</b>                                                                                                                                                                                                                                     | Variable |
| The character newline.                                                                                                                                                                                                                             |          |
| <b>space</b>                                                                                                                                                                                                                                       | Variable |
| The character space.                                                                                                                                                                                                                               |          |
| <b>tab</b>                                                                                                                                                                                                                                         | Variable |
| The character tab.                                                                                                                                                                                                                                 |          |
| <b>uppercasep</b> ( <i>char</i> )                                                                                                                                                                                                                  | Function |
| Returns <b>true</b> if <i>char</i> is an uppercase character.                                                                                                                                                                                      |          |

## 68.4 Definiciones para cadenas

**sunlisp** (*lisp-string*) Function  
 Converts a Lisp-string into a Maxima-string. (In general you wont need it.)

**lstringp** (*obj*) Function  
 Returns **true** if *obj* is a Lisp-string. (In general you wont need it.)

**stringp** (*obj*) Function  
 Returns **true** if *obj* is a Maxima-string. See introduction for example.

**charat** (*string*, *n*) Function  
 Returns the *n*-th character of *string*. The first character in *string* is returned with *n* = 1.

```
(%i1) charat("Lisp",1);
(%o1) L
```

**charlist** (*string*) Function  
 Returns the list of all characters in *string*.

```
(%i2) charlist("Lisp");
(%o2) [L, i, s, p]
(%i3) %[1];
(%o3) L
```

**parsetoken** (*string*) Function  
**parsetoken** converts the first token in *string* to the corresponding number or returns **false** if the number cannot be determined . The delimiter set for tokenizing is {space, comma, semicolon, tab, newline}.

```
(%i1) 2*parsetoken("1.234 5.678");
(%o1) 2.468
```

For parsing you can also use function `parse_string`. See description in file '`\Maxima-5.9.2\share\maxima\5.9.2\share\contrib\eval_string.lisp`'.

**sconc** (*expr\_1*, ..., *expr\_n*) Function  
 Evaluates its arguments and concatenates them into a string. **sconc** is like **sconcat** but returns a Maxima string.

```
(%i1) sconc("xx[" , 3, "]" : ", expand((x+y)^3));
(%o1) xx[3]:y^3+3*x*y^2+3*x^2*y+x^3
(%i2) stringp(%);
(%o2) true
```

**scopy** (*string*) Function  
 Returns a copy of *string* as a new string.

**sdowncase** (*string*) Function

**sdowncase** (*string*, *start*) Function

**sdowncase** (*string*, *start*, *end*) Function

Like **supcase**, but uppercase characters are converted to lowercase.

**sequal** (*string\_1*, *string\_2*) Function  
 Returns **true** if *string\_1* and *string\_2* are the same length and contain the same characters.

**sequalignore** (*string\_1*, *string\_2*) Function  
 Like **sequal** but ignores case.

**sexplode** (*string*) Function  
**sexplode** is an alias for function **charlist**.

**simplode** (*list*) Function

**simplode** (*list*, *delim*) Function  
**simplode** takes a list of expressions and concatenates them into a string. If no delimiter *delim* is used, **simplode** is like **sconc** and uses no delimiter. *delim* can be any string.

```
(%i1) simplode(["xx[" ,3,"]:" ,expand((x+y)^3)]);
(%o1) xx[3]:y^3+3*x*y^2+3*x^2*y+x^3
(%i2) simplode(sexplode("stars")," * ");
(%o2) s * t * a * r * s
(%i3) simplode(["One","more","coffee."]," ");
(%o3) One more coffee.
```

**sininsert** (*seq*, *string*, *pos*) Function  
 Returns a string that is a concatenation of **substring** (*string*, 1, *pos* - 1), the string *seq* and **substring** (*string*, *pos*). Note that the first character in *string* is in position 1.

```
(%i1) s: "A submarine.$"
(%i2) sconc(substring(s,1,3),"yellow ",substring(s,3));
(%o2) A yellow submarine.
(%i3) sininsert("hollow ",s,3);
(%o3) A hollow submarine.
```

**sinvertcase** (*string*) Function

**sinvertcase** (*string*, *start*) Function

**sinvertcase** (*string*, *start*, *end*) Function

Returns *string* except that each character from position *start* to *end* is inverted. If *end* is not given, all characters from *start* to the *end* of *string* are replaced.

```
(%i1) sinvertcase("sInvertCase");
(%o1) SiNVERTcASE
```

**slength** (*string*) Function

Returns the number of characters in *string*.

**smake** (*num*, *char*) Function

Returns a new string with a number of *num* characters *char*.

```
(%i1) smake(3,"w");
(%o1) www
```

**smismatch** (*string\_1*, *string\_2*) Function  
**smismatch** (*string\_1*, *string\_2*, *test*) Function

Returns the position of the first character of *string\_1* at which *string\_1* and *string\_2* differ or **false**. Default test function for matching is **sequal**. If **smismatch** should ignore case, use **sequalignore** as test.

```
(%i1) smismatch("seven","seventh");
(%o1) 6
```

**split** (*string*) Function  
**split** (*string*, *delim*) Function  
**split** (*string*, *delim*, *multiple*) Function

Returns the list of all tokens in *string*. Each token is an unparsed string. **split** uses *delim* as delimiter. If *delim* is not given, the space character is the default delimiter. *multiple* is a boolean variable with **true** by default. Multiple delimiters are read as one. This is useful if tabs are saved as multiple space characters. If *multiple* is set to **false**, each delimiter is noted.

```
(%i1) split("1.2 2.3 3.4 4.5");
(%o1) [1.2, 2.3, 3.4, 4.5]
(%i2) split("first;;third;fourth",";",false);
(%o2) [first, , third, fourth]
```

**sposition** (*char*, *string*) Function

Returns the position of the first character in *string* which matches *char*. The first character in *string* is in position 1. For matching characters ignoring case see **ssearch**.

**sremove** (*seq*, *string*) Function  
**sremove** (*seq*, *string*, *test*) Function  
**sremove** (*seq*, *string*, *test*, *start*) Function  
**sremove** (*seq*, *string*, *test*, *start*, *end*) Function

Returns a string like *string* but without all substrings matching *seq*. Default test function for matching is **sequal**. If **sremove** should ignore case while searching for *seq*, use **sequalignore** as test. Use *start* and *end* to limit searching. Note that the first character in *string* is in position 1.

```
(%i1) sremove("n't","I don't like coffee.");
(%o1) I do like coffee.
(%i2) sremove ("D0 ",%,,'sequalignore);
(%o2) I like coffee.
```

**sremovefirst** (*seq*, *string*) Function  
**sremovefirst** (*seq*, *string*, *test*) Function  
**sremovefirst** (*seq*, *string*, *test*, *start*) Function  
**sremovefirst** (*seq*, *string*, *test*, *start*, *end*) Function

Like **sremove** except that only the first substring that matches *seq* is removed.

**sreverse** (*string*) Function

Returns a string with all the characters of *string* in reverse order.

**ssearch** (*seq, string*) Function  
**ssearch** (*seq, string, test*) Function  
**ssearch** (*seq, string, test, start*) Function  
**ssearch** (*seq, string, test, start, end*) Function

Returns the position of the first substring of *string* that matches the string *seq*. Default test function for matching is **sequal**. If **ssearch** should ignore case, use **sequalignore** as test. Use *start* and *end* to limit searching. Note that the first character in *string* is in position 1.

```
(%i1) ssearch("~s","~{~S ~}~%",'sequalignore);
(%o1) 3
```

**ssort** (*string*) Function  
**ssort** (*string, test*) Function

Returns a string that contains all characters from *string* in an order such there are no two successive characters *c* and *d* such that **test** (*c, d*) is false and **test** (*d, c*) is true. Default test function for sorting is **clessp**. The set of test functions is {**clessp**, **clesspignore**, **cgreaterp**, **cgreaterpignore**, **cequal**, **cequalignore**}.

```
(%i1) ssort("I don't like Mondays.");
(%o1) '.IMaddeiklnnoosty
(%i2) ssort("I don't like Mondays.",'cgreaterpignore);
(%o2) ytsoonnMlkIiedda.'
```

**ssubst** (*new, old, string*) Function  
**ssubst** (*new, old, string, test*) Function  
**ssubst** (*new, old, string, test, start*) Function  
**ssubst** (*new, old, string, test, start, end*) Function

Returns a string like *string* except that all substrings matching *old* are replaced by *new*. *old* and *new* need not to be of the same length. Default test function for matching is **sequal**. If **ssubst** should ignore case while searching for *old*, use **sequalignore** as test. Use *start* and *end* to limit searching. Note that the first character in *string* is in position 1.

```
(%i1) ssubst("like","hate","I hate Thai food. I hate green tea.");
(%o1) I like Thai food. I like green tea.
(%i2) ssubst("Indian","thai",%, 'sequalignore,8,12);
(%o2) I like Indian food. I like green tea.
```

**ssubstfirst** (*new, old, string*) Function  
**ssubstfirst** (*new, old, string, test*) Function  
**ssubstfirst** (*new, old, string, test, start*) Function  
**ssubstfirst** (*new, old, string, test, start, end*) Function

Like **subst** except that only the first substring that matches *old* is replaced.

**strim** (*seq,string*) Function

Returns a string like *string*, but with all characters that appear in *seq* removed from both ends.

```
(%i1) "/* comment */"$
(%i2) strim(" /*",%);
(%o2) comment
(%i3) slength(%);
(%o3) 7
```

**striml** (*seq, string*) Function  
 Like **strim** except that only the left end of *string* is trimmed.

**strimr** (*seq, string*) Function  
 Like **strim** except that only the right end of *string* is trimmed.

**substring** (*string, start*) Function

**substring** (*string, start, end*) Function

Returns the substring of *string* beginning at position *start* and ending at position *end*. The character at position *end* is not included. If *end* is not given, the substring contains the rest of the string. Note that the first character in *string* is in position 1.

```
(%i1) substring("substring",4);
(%o1) string
(%i2) substring(%,4,6);
(%o2) in
```

**supcase** (*string*) Function

**supcase** (*string, start*) Function

**supcase** (*string, start, end*) Function

Returns *string* except that lowercase characters from position *start* to *end* are replaced by the corresponding uppercase ones. If *end* is not given, all lowercase characters from *start* to the end of *string* are replaced.

```
(%i1) supcase("english",1,2);
(%o1) English
```

**tokens** (*string*) Function

**tokens** (*string, test*) Function

Returns a list of tokens, which have been extracted from *string*. The tokens are substrings whose characters satisfy a certain test function. If *test* is not given, *constituent* is used as the default test. {*constituent*, *alphacharp*, *digitcharp*, *lowercasep*, *uppercasep*, *charp*, *characterp*, *alphanumericp*} is the set of test functions. (The Lisp-version of **tokens** is written by Paul Graham. ANSI Common Lisp, 1996, page 67.)

```
(%i1) tokens("24 October 2005");
(%o1) [24, October, 2005]
(%i2) tokens("05-10-24",'digitcharp);
(%o2) [05, 10, 24]
(%i3) map(parsetoken,%);
(%o3) [5, 10, 24]
```



## 69 unit

### 69.1 Introducción a Units

The *unit* package enables the user to convert between arbitrary units and work with dimensions in equations. The functioning of this package is radically different from the original Maxima units package - whereas the original was a basic list of definitions, this package uses rulesets to allow the user to chose, on a per dimension basis, what unit final answers should be rendered in. It will separate units instead of intermixing them in the display, allowing the user to readily identify the units associated with a particular answer. It will allow a user to simplify an expression to its fundamental Base Units, as well as providing fine control over simplifying to derived units. Dimensional analysis is possible, and a variety of tools are available to manage conversion and simplification options. In addition to customizable automatic conversion, *units* also provides a traditional manual conversion option.

Note - when unit conversions are inexact Maxima will make approximations resulting in fractions. This is a consequence of the techniques used to simplify units. The messages warning of this type of substitution are disabled by default in the case of units (normally they are on) since this situation occurs frequently and the warnings clutter the output. (The existing state of *ratprint* is restored after unit conversions, so user changes to that setting will be preserved otherwise.) If the user needs this information for units, they can set *unitverbose:on* to reactivate the printing of warnings from the unit conversion process.

#### 69.1.0.1 Loading *unit*

*unit* is included in Maxima in the *share/contrib/unit* directory. It obeys normal Maxima package loading conventions:

```
(%i1) load("share/contrib/unit.mac");

* Units version 0.50 *
* Definitions based on the NIST Reference on *
* Constants, Units, and Uncertainty *
* Conversion factors from various sources including *
* NIST and the GNU units package *

Redefining necessary functions...
STYLE-WARNING: redefining TOPLEVEL-MACSYMA-EVAL in DEFUN
STYLE-WARNING: redefining MSETCHK in DEFUN
STYLE-WARNING: redefining KILL1 in DEFUN
STYLE-WARNING: redefining NFORMAT in DEFUN
Initializing unit arrays...
Done.
(%o1) /usr/local/share/maxima/5.9.1.1cvs/share/contrib/unit/unit.mac
```

The STYLE-WARNING messages are expected and not a cause for concern - they indicate the *unit* package is redefining functions already defined in Maxima proper. This is



necessary in order to properly handle units. The user should be aware that if other changes have been made to these functions by other packages those changes will be overwritten by this loading process.

The *unit.mac* file also loads a lisp file *unit-functions.lisp* which contains the lisp functions needed for the package.

### 69.1.0.2 Basic Usage

By default, the *unit* package does not use any derived dimensions, but will convert all units to the seven fundamental dimensions using MKS units.

```
(%i2) N;
(%o2)
$$\frac{\text{kg m}}{\text{s}^2}$$

(%i3) dyn;
(%o3)
$$\frac{1}{100000} \frac{\text{kg m}}{\text{s}^2}$$

(%i4) g;
(%o4)
$$\frac{1}{1000} (\text{kg})$$

(%i5) centigram*inch/minutes^2;
(%o5)
$$\frac{127}{18000000000000} \frac{\text{kg m}}{\text{s}^2}$$

```

In some cases this is the desired behavior. If the user wishes to use other units, this is achieved with the `setunits` command:

```
(%i6) setunits([centigram,inch,minute]);
(%o6) done

(%i7) N;
(%o7)
$$\frac{18000000000000}{127} \frac{\text{\%in cg}}{\text{\%min}^2}$$

(%i8) dyn;
(%o8)
$$\frac{18000000}{127} \frac{\text{\%in cg}}{\text{\%min}^2}$$

(%i9) g;
(%o9) (100) (cg)

(%i10) centigram*inch/minutes^2;
(%o10)
$$\frac{\text{\%in cg}}{\text{\%min}^2}$$

```

2  
%min

The setting of units is quite flexible. For example, if we want to get back to kilograms, meters, and seconds as defaults for those dimensions we can do:

```
(%i11) setunits([kg,m,s]);
(%o11) done
(%i12) centigram*inch/minutes^2;
(%o12) (-----) (----)
 127 kg m
 1800000000000 2
 s
```

Or, alternately, if one wishes to restore the default behavior for a particular dimension:

```
(%i13) setunits([centigram,inch,minute]);
(%o13) done
(%i14) centigram*inch/minutes^2;
(%o14) %in cg

 2
 %min
(%i15) forget([cg,%in,%min]);
(%o15) [done, done, done]
(%i16) centigram*inch/minutes^2;
(%o16) (-----) (----)
 127 kg m
 1800000000000 2
 s
```

Derived units are also handled by these commands:

```
(%i17) setunits(N);
(%o17) done
(%i18) N;
(%o18) N
(%i19) dyn;
(%o19) (-----) (N)
 1
 100000
(%i20) kg*m/s^2;
(%o20) N
(%i21) centigram*inch/minutes^2;
(%o21) (-----) (N)
 127
 1800000000000
```

Notice that the *unit* package recognized the non MKS combination of mass, length, and inverse time squared as a force, and converted it to Newtons. This is how Maxima works in general. If, for example, we prefer dyne to Newtons, we simply do the following:

```
(%i22) setunits(dyn);
(%o22) done
(%i23) kg*m/s^2;
```

```
(%o23) (100000) (dyn)
(%i24) centigram*inch/minutes^2;
127
(%o24) (-----) (dyn)
18000000
```

To discontinue simplifying to any force, we use the forget command:

```
(%i26) forget(dyn);
(%o26) done
(%i27) kg*m/s^2;
kg m

2
s
(%i28) centigram*inch/minutes^2;
127 kg m
(%o28) (-----) (-----)
1800000000000 2
s
```

This would have worked equally well with `forget(N)` or `forget(%force)`. `forget` operates on dimensions, not units, so any unit of a particular dimension will work. The dimension itself is also a legal argument.

### 69.1.0.3 convert

When resetting the global environment is overkill, there is the `convert` command, which allows one time conversions. It can accept either a single argument or a list of units to use in conversion. When a convert operation is done, the normal global evaluation system is bypassed, in order to avoid the desired result being converted again. As a consequence, for inexact calculations "rat" warnings will be visible if the global environment controlling this behavior (`ratprint`) is true. This is also useful for spot-checking the accuracy of a global conversion. Another feature is `convert` will allow a user to do Base Dimension conversions even if the global environment is set to simplify to a Derived Dimension.

```
(%i2) kg*m/s^2;
kg m

2
s
(%i3) convert(kg*m/s^2,[g,km,s]);
g km

2
s
(%i4) convert(kg*m/s^2,[g,inch,minute]);

'rat' replaced 39.37007874015748 by 5000//127 = 39.37007874015748
18000000000 %in g
(%o4) (-----) (-----)
127 2
```

```

 %min
(%i5) convert(kg*m/s^2,[N]);
(%o5)
N
(%i6) convert(kg*m^2/s^2,[N]);
(%o6)
m N
(%i7) setunits([N,J]);
(%o7)
done
(%i8) convert(kg*m^2/s^2,[N]);
(%o8)
m N
(%i9) convert(kg*m^2/s^2,[N,inch]);

‘rat’ replaced 39.37007874015748 by 5000//127 = 39.37007874015748
5000
(%o9)
(-----) (%in N)
127

(%i10) convert(kg*m^2/s^2,[J]);
(%o10)
J
(%i11) kg*m^2/s^2;
(%o11)
J
(%i12) setunits([g,inch,s]);
(%o12)
done
(%i13) kg*m/s^2;
(%o13)
N
(%i14) forget(N);
(%o14)
false
(%i15) kg*m/s^2;
50000000 %in g
(%o15)
(-----) (-----)
127 2
s

(%i16) convert(kg*m/s^2,[g,inch,s]);

‘rat’ replaced 39.37007874015748 by 5000//127 = 39.37007874015748
50000000 %in g
(%o16)
(-----) (-----)
127 2
s

```

TODO : dimension functionality, handling of temperature, showabbr and friends. Show examples with addition of quantities containing units.

#### 69.1.0.4 User defaults and resetting *unit* behavior

If a user wishes to have a default unit behavior other than that described, they can make use of *maxima-init.mac* and the *usersetunits* variable. The *unit* package will check on startup to see if this variable has been assigned a list. If it has, it will use *setunits* on that list and take the units from that list to be defaults. *forget* will revert to the behavior defined by *usersetunits* over its own defaults. For example, if we have a *maxima-init.mac* file containing:

```
usersetunits : [N,J];
```

we would see the following behavior:

```
(%i1) load("./unit.mac");

* Units version 0.50 *
* Definitions based on the NIST Reference on *
* Constants, Units, and Uncertainty *
* Conversion factors from various sources including *
* NIST and the GNU units package *

Redefining necessary functions...
STYLE-WARNING: redefining TOPLEVEL-MACSYMA-EVAL in DEFUN
STYLE-WARNING: redefining MSETCHK in DEFUN
STYLE-WARNING: redefining KILL1 in DEFUN
STYLE-WARNING: redefining NFORMAT in DEFUN
Initializing unit arrays...
Done.
User defaults found...
User defaults initialized.
(%o1) ./unit.mac
(%i2) kg*m/s^2;
(%o2) N
(%i3) kg*m^2/s^2;
(%o3) J
(%i4) kg*m^3/s^2;
(%o4) J m
(%i5) kg*m*km/s^2;
(%o5) (1000) (J)
(%i6) setunits([dyn,eV]);
(%o6) done
(%i7) kg*m/s^2;
(%o7) (100000) (dyn)
(%i8) kg*m^2/s^2;
(%o8) (6241509596477042688) (eV)
(%i9) kg*m^3/s^2;
(%o9) (6241509596477042688) (eV m)
(%i10) kg*m*km/s^2;
(%o10) (6241509596477042688000) (eV)
(%i11) forget([dyn,eV]);
(%o11) [done, done]
(%i12) kg*m/s^2;
(%o12) N
(%i13) kg*m^2/s^2;
(%o13) J
(%i14) kg*m^3/s^2;
(%o14) J m
(%i15) kg*m*km/s^2;
(%o15) (1000) (J)
```

Without `userunits`, the initial inputs would have been converted to MKS, and `forget` would have resulted in a return to MKS rules. Instead, the user preferences are respected in both cases. Notice these can still be overridden if desired. To completely eliminate this simplification - i.e. to have the user defaults reset to factory defaults - the `dontusedimension` command can be used. `forget` can restore user settings again, but only if `usedimension` frees it for use. Alternately, `kill(userunits)` will completely remove all knowledge of the user defaults from the session. Here are some examples of how these various options work.

```
(%i2) kg*m/s^2;
(%o2) N
(%i3) kg*m^2/s^2;
(%o3) J
(%i4) setunits([dyn,eV]);
(%o4) done
(%i5) kg*m/s^2;
(%o5) (100000) (dyn)
(%i6) kg*m^2/s^2;
(%o6) (6241509596477042688) (eV)
(%i7) forget([dyn,eV]);
(%o7) [done, done]
(%i8) kg*m/s^2;
(%o8) N
(%i9) kg*m^2/s^2;
(%o9) J
(%i10) dontusedimension(N);
(%o10) [%force]
(%i11) dontusedimension(J);
(%o11) [%energy, %force]
(%i12) kg*m/s^2;
kg m
(%o12) ----
 2
 s

(%i13) kg*m^2/s^2;
kg m
(%o13) ----
 2
 s

(%i14) setunits([dyn,eV]);
(%o14) done
(%i15) kg*m/s^2;
kg m
(%o15) ----
 2
 s

(%i16) kg*m^2/s^2;
kg m
(%o16) ----
 2
 s
```

```

(%o16) -----
 2
 s

(%i17) forget([dyn,eV]);
(%o17) [done, done]
(%i18) kg*m/s^2;
 kg m
(%o18) ----
 2
 s

(%i19) kg*m^2/s^2;
 2
 kg m
(%o19) -----
 2
 s

(%i20) usedimension(N);
Done. To have Maxima simplify to this dimension, use setunits([unit])
to select a unit.
(%o20) true
(%i21) usedimension(J);
Done. To have Maxima simplify to this dimension, use setunits([unit])
to select a unit.
(%o21) true
(%i22) kg*m/s^2;
 kg m
(%o22) ----
 2
 s

(%i23) kg*m^2/s^2;
 2
 kg m
(%o23) -----
 2
 s

(%i24) setunits([dyn,eV]);
(%o24) done
(%i25) kg*m/s^2;
(%o25) (100000) (dyn)
(%i26) kg*m^2/s^2;
(%o26) (6241509596477042688) (eV)
(%i27) forget([dyn,eV]);
(%o27) [done, done]
(%i28) kg*m/s^2;
(%o28) N
(%i29) kg*m^2/s^2;
(%o29) J
(%i30) kill(usersetunits);
(%o30) done

```

```
(%i31) forget([dyn,eV]);
(%o31) [false, false]
(%i32) kg*m/s^2;
 kg m

 2
 s
(%i33) kg*m^2/s^2;
 2
 kg m

 2
 s
```

Unfortunately this wide variety of options is a little confusing at first, but once the user grows used to them they should find they have very full control over their working environment.

One other significant customization option available is the `setunitprefix` command. Normally, abbreviations used in this package are as close to those used in standard texts as possible. Some people, however, prefer to use those symbols for normal work and have units labeled in some other fashion. `setunitprefix` is provided for this case. Here is an example of its use:

### 69.1.0.5 Bugs

Undoubtedly. Probably lots of them. Let me know. `float` and `numer` don't do what is expected.

### 69.1.0.6 Authors

Clifford Yapp is the primary author. He has recieved valuable assistance from Barton Willis of the University of Nebraska at Kearney (UNK), Robert Dodier, and other intrepid folk of the Maxima mailing list.

## 69.2 Definiciones para Units

**metricexpandall** (*x*) Function

Rebuilds global unit lists automatically creating all desired metric units. *x* is a numerical argument which is used to specify how many metric prefixes the user wishes defined. The arguments are as follows, with each higher number defining all lower numbers' units:

```
0 - none. Only base units
1 - kilo, centi, milli
(default) 2 - giga, mega, kilo, hecto, deka, deci, centi, milli, micro, nano
3 - peta, tera, giga, mega, kilo, hecto, deka, deci, centi, milli, micro, nano, pico, femto
4 - all
```



Normally, Maxima will not define the full expansion since this results in a very large number of units, but `metricexpandall` can be used to rebuild the list in a more or less complete fashion. The relevant variable in the *unit.mac* file is *%unitexpand*.

|                                                                                                        |          |
|--------------------------------------------------------------------------------------------------------|----------|
| <b>%unitexpand</b>                                                                                     | Variable |
| Default value: 2                                                                                       |          |
| This is the value supplied to <code>metricexpandall</code> during the initial loading of <i>unit</i> . |          |
| <b>functionname</b> ( <i>arg1, arg2, ..., argn</i> )                                                   | Function |
| <b>variablename</b>                                                                                    | Variable |
| Default value: <code>true</code>                                                                       |          |

## 70 zeilberger

### 70.1 Introducción a zeilberger

`zeilberger` is a implementation of Zeilberger's algorithm for definite hypergeometric summation, and also Gosper's algorithm for indefinite hypergeometric summation.

`zeilberger` makes use of the "filtering" optimization method developed by Axel Riese.

`zeilberger` was developed by Fabrizio Caruso.

`load (zeilberger)` loads this package.

#### 70.1.0.1 The indefinite summation problem

`zeilberger` implements Gosper's algorithm for indefinite hypergeometric summation. Given a hypergeometric term  $F_k$  in  $k$  we want to find its hypergeometric anti-difference, that is, a hypergeometric term  $f_k$  such that  $F_k = f(k+1) - f_k$ .

#### 70.1.0.2 The definite summation problem

`zeilberger` implements Zeilberger's algorithm for definite hypergeometric summation. Given a proper hypergeometric term (in  $n$  and  $k$ )  $F(n, k)$  and a positive integer  $d$  we want to find a  $d$ -th order linear recurrence with polynomial coefficients (in  $n$ ) for  $F(n, k)$  and a rational function  $R$  in  $n$  and  $k$  such that

$$a_0 F(n, k) + \dots + a_d F(n + d, k) = \text{Delta}_K(R(n, k) F(n, k))$$

where  $\text{Delta}_k$  is the  $k$ -forward difference operator, i.e.,  $\text{Delta}_k(t_k) := t(k+1) - t_k$ .

#### 70.1.1 Verbosity levels

There are also verbose versions of the commands which are called by adding one of the following prefixes:

**Summary**     Just a summary at the end is shown

**Verbose**     Some information in the intermediate steps

**VeryVerbose**  
                More information

**Extra**        Even more information including information on the linear system in Zeilberger's algorithm

For example:    `GosperVerbose`,    `parGosperVeryVerbose`,    `ZeilbergerExtra`,  
`AntiDifferenceSummary`.

## 70.2 Definiciones para zeilberger

### AntiDifference ( $F_k, k$ )

Function

Returns the hypergeometric anti-difference of  $F_k$ , if it exists. Otherwise `AntiDifference` returns `no_hyp_antidifference`.

### Gosper ( $F_k, k$ )

Function

Returns the rational certificate  $R(k)$  for  $F_k$ , that is, a rational function such that  $F_k = R(k+1)F_{k+1} - R(k)F_k$  if it exists. Otherwise, `Gosper` returns `no_hyp_sol`.

### GosperSum ( $F_k, k, a, b$ )

Function

Returns the summation of  $F_k$  from  $k = a$  to  $k = b$  if  $F_k$  has a hypergeometric anti-difference. Otherwise, `GosperSum` returns `nongosper_summable`.

Examples:

```
(%i1) load (zeilberger);
(%o1) /usr/share/maxima/share/contrib/Zeilberger/zeilberger.mac
(%i2) GosperSum ((-1)^k*k / (4*k^2 - 1), k, 1, n);
```

Dependent equations eliminated: (1)

```
(n + -) (- 1)
 3 n + 1
 2
(%o2) - ----- - -
 2 4
```

```
(%i3) GosperSum (1 / (4*k^2 - 1), k, 1, n);
 3
 - n - -
 2 1
```

```
(%o3) ----- + -
 2 2
```

```
(%i4) GosperSum (x^k, k, 1, n);
 n + 1
 x
 x
```

```
(%o4) ----- - -----
 x - 1 x - 1
```

```
(%i5) GosperSum ((-1)^k*a! / (k!*(a - k)!), k, 1, n);
 n + 1
```

```
(%o5) - ----- - -----
 a! (n + 1) (- 1) a!
 a (- n + a - 1)! (n + 1)! a (a - 1)!
```

```
(%i6) GosperSum (k*k!, k, 1, n);
```

Dependent equations eliminated: (1)

```
(%o6) (n + 1)! - 1
```

```
(%i7) GosperSum ((k + 1)*k! / (k + 1)!, k, 1, n);
```

```

(%o7)

$$\frac{(n+1)(n+2)(n+1)!}{(n+2)!} - 1$$

(%i8) GosperSum (1 / ((a - k)!*k!), k, 1, n);
(%o8) nonGosper_summable

```

**parGosper** ( $F_{\{n,k\}}$ ,  $k$ ,  $n$ ,  $d$ )

Function

Attempts to find a  $d$ -th order recurrence for  $F_{\{n,k\}}$ .

The algorithm yields a sequence  $[s_1, s_2, \dots, s_m]$  of solutions. Each solution has the form

$[R(n, k), [a_0, a_1, \dots, a_d]]$

**parGosper** returns [] if it fails to find a recurrence.

**Zeilberger** ( $F_{\{n,k\}}$ ,  $k$ ,  $n$ )

Function

Attempts to compute the indefinite hypergeometric summation of  $F_{\{n,k\}}$ .

**Zeilberger** first invokes **Gosper**, and if that fails to find a solution, then invokes **parGosper** with order 1, 2, 3, ..., up to **MAX\_ORD**. If **Zeilberger** finds a solution before reaching **MAX\_ORD**, it stops and returns the solution.

The algorithm yields a sequence  $[s_1, s_2, \dots, s_m]$  of solutions. Each solution has the form

$[R(n, k), [a_0, a_1, \dots, a_d]]$

**Zeilberger** returns [] if it fails to find a solution.

**Zeilberger** invokes **Gosper** only if **gosper\_in\_zeilberger** is true.

## 70.3 General global variables

**MAX\_ORD**

Global variable

Default value: 5

**MAX\_ORD** is the maximum recurrence order attempted by **Zeilberger**.

**simplified\_output**

Global variable

Default value: false

When **simplified\_output** is true, functions in the **zeilberger** package attempt further simplification of the solution.

**linear\_solver**

Global variable

Default value: linsolve

**linear\_solver** names the solver which is used to solve the system of equations in **Zeilberger**'s algorithm.

**warnings**

Global variable

Default value: true

When **warnings** is true, functions in the **zeilberger** package print warning messages during execution.

**gospers\_in\_zeilberger** Global variable  
Default value: `true`  
When `gospers_in_zeilberger` is `true`, the `Zeilberger` function calls `Gospers` before calling `parGospers`. Otherwise, `Zeilberger` goes immediately to `parGospers`.

**trivial\_solutions** Global variable  
Default value: `true`  
When `trivial_solutions` is `true`, `Zeilberger` returns solutions which have certificate equal to zero, or all coefficients equal to zero.

## 70.4 Variables related to the modular test

**mod\_test** Global variable  
Default value: `false`  
When `mod_test` is `true`, `parGospers` executes a modular test for discarding systems with no solutions.

**modular\_linear\_solver** Global variable  
Default value: `linsolve`  
`modular_linear_solver` names the linear solver used by the modular test in `parGospers`.

**ev\_point** Global variable  
Default value: `big_primes[10]`  
`ev_point` is the value at which the variable  $n$  is evaluated when executing the modular test in `parGospers`.

**mod\_big\_prime** Global variable  
Default value: `big_primes[1]`  
`mod_big_prime` is the modulus used by the modular test in `parGospers`.

**mod\_threshold** Global variable  
Default value: `4`  
`mod_threshold` is the greatest order for which the modular test in `parGospers` is attempted.

## 71 Índice de Funciones y Variables



## Apéndice A Índice de Funciones y Variables

(No existe el Índice)





## Resumen del Contenido

|                                         |     |
|-----------------------------------------|-----|
| .....                                   | 1   |
| 1    Introducción a Maxima .....        | 3   |
| 2    Reporte y Detección de Bugs .....  | 7   |
| 3    Ayuda .....                        | 9   |
| 4    Línea de Comandos .....            | 15  |
| 5    Operadores .....                   | 27  |
| 6    Expresiones .....                  | 49  |
| 7    Simplificación .....               | 77  |
| 8    Gráficos .....                     | 85  |
| 9    Entrada y Salida .....             | 97  |
| 10   Aritmética de punto flotante ..... | 121 |
| 11   Contextos .....                    | 125 |
| 12   Polinomios .....                   | 131 |
| 13   Constantes .....                   | 153 |
| 14   Logaritmos .....                   | 155 |
| 15   Trigonometría .....                | 159 |
| 16   Funciones Especiales .....         | 165 |
| 17   Funciones elípticas .....          | 173 |
| 18   Límites .....                      | 179 |
| 19   Diferenciación .....               | 181 |
| 20   Integración .....                  | 191 |
| 21   Ecuaciones .....                   | 209 |
| 22   Ecuaciones Diferenciales .....     | 227 |
| 23   Métodos numéricos .....            | 231 |
| 24   Estadística .....                  | 239 |
| 25   Tablas y Arreglos .....            | 241 |
| 26   Matrices y Álgebra Lineal .....    | 245 |
| 27   Afines .....                       | 267 |
| 28   itensor .....                      | 269 |
| 29   ctensor .....                      | 301 |
| 30   atensor .....                      | 327 |
| 31   Series .....                       | 331 |
| 32   Teoría de Números .....            | 343 |
| 33   Simetrías .....                    | 351 |
| 34   Grupos .....                       | 367 |

|    |                                           |     |
|----|-------------------------------------------|-----|
| 35 | Entorno de Ejecución . . . . .            | 369 |
| 36 | Opciones Misceláneas . . . . .            | 373 |
| 37 | Reglas y patrones . . . . .               | 381 |
| 38 | Listas . . . . .                          | 397 |
| 39 | Conjuntos . . . . .                       | 403 |
| 40 | Definición de Funciones . . . . .         | 421 |
| 41 | Programación . . . . .                    | 443 |
| 42 | Depurado . . . . .                        | 451 |
| 43 | augmented_lagrangian . . . . .            | 459 |
| 44 | bode . . . . .                            | 461 |
| 45 | cholesky . . . . .                        | 463 |
| 46 | descriptive . . . . .                     | 465 |
| 47 | diag . . . . .                            | 487 |
| 48 | distrib . . . . .                         | 495 |
| 49 | eval_string . . . . .                     | 529 |
| 50 | f90 . . . . .                             | 531 |
| 51 | ggf . . . . .                             | 533 |
| 52 | ifactor . . . . .                         | 535 |
| 53 | impdiff . . . . .                         | 537 |
| 54 | interpol . . . . .                        | 539 |
| 55 | lindstedt . . . . .                       | 543 |
| 56 | linearalgebra . . . . .                   | 545 |
| 57 | lsquares . . . . .                        | 557 |
| 58 | makeOrders . . . . .                      | 561 |
| 59 | mnewton . . . . .                         | 563 |
| 60 | numericalio . . . . .                     | 565 |
| 61 | opsubst . . . . .                         | 569 |
| 62 | orthopoly . . . . .                       | 571 |
| 63 | plotdf . . . . .                          | 583 |
| 64 | simplex . . . . .                         | 587 |
| 65 | simplification . . . . .                  | 589 |
| 66 | solve_rec . . . . .                       | 597 |
| 67 | stirling . . . . .                        | 601 |
| 68 | stringproc . . . . .                      | 603 |
| 69 | unit . . . . .                            | 615 |
| 70 | zeilberger . . . . .                      | 625 |
| 71 | Índice de Funciones y Variables . . . . . | 629 |
| A  | Índice de Funciones y Variables . . . . . | 631 |

# Tabla de Contenidos

|          |                                                               |
|----------|---------------------------------------------------------------|
| .....    | <b>1</b>                                                      |
| <b>1</b> | <b>Introducción a Maxima ..... 3</b>                          |
| <b>2</b> | <b>Reporte y Detección de Bugs ..... 7</b>                    |
| 2.1      | Introducción a la detección de Bugs y su reporte ..... 7      |
| 2.2      | Definiciones para la detección de Bugs y su reporte:: ..... 7 |
| <b>3</b> | <b>Ayuda ..... 9</b>                                          |
| 3.1      | Introducción a la Ayuda ..... 9                               |
| 3.2      | Lisp y Maxima ..... 9                                         |
| 3.3      | Recolector de Basura ..... 11                                 |
| 3.4      | Documentación ..... 11                                        |
| 3.5      | Definiciones para la Ayuda ..... 11                           |
| <b>4</b> | <b>Línea de Comandos ..... 15</b>                             |
| 4.1      | Introducción a la Línea de Comandos ..... 15                  |
| 4.2      | Definiciones para la Línea de Comandos ..... 16               |
| <b>5</b> | <b>Operadores ..... 27</b>                                    |
| 5.1      | n-arios ..... 27                                              |
| 5.2      | no-fijos ..... 27                                             |
| 5.3      | operadores ..... 27                                           |
| 5.4      | postfijos ..... 27                                            |
| 5.5      | prefijos ..... 27                                             |
| 5.6      | Definiciones para Operadores ..... 28                         |
| <b>6</b> | <b>Expresiones ..... 49</b>                                   |
| 6.1      | Introducción a las Expresiones ..... 49                       |
| 6.2      | Asignación ..... 49                                           |
| 6.3      | Expresiones complejas ..... 49                                |
| 6.4      | Nombres y verbos ..... 50                                     |
| 6.5      | Identificadores ..... 51                                      |
| 6.6      | Desigualdades ..... 52                                        |
| 6.7      | Sintaxis ..... 52                                             |
| 6.8      | Definiciones para Expresiones ..... 54                        |
| <b>7</b> | <b>Simplificación ..... 77</b>                                |
| 7.1      | Definiciones para Simplificación ..... 77                     |

|           |                                                     |            |
|-----------|-----------------------------------------------------|------------|
| <b>8</b>  | <b>Gráficos</b>                                     | <b>85</b>  |
| 8.1       | Definiciones para Gráficos                          | 85         |
| <b>9</b>  | <b>Entrada y Salida</b>                             | <b>97</b>  |
| 9.1       | Introducción a Entrada y Salida                     | 97         |
| 9.2       | Archivos                                            | 97         |
| 9.3       | Definiciones para Entrada y Salida                  | 97         |
| <b>10</b> | <b>Aritmética de punto flotante</b>                 | <b>121</b> |
| 10.1      | Definiciones para aritmética de punto flotante      | 121        |
| <b>11</b> | <b>Contextos</b>                                    | <b>125</b> |
| 11.1      | Definiciones para Contextos                         | 125        |
| <b>12</b> | <b>Polinomios</b>                                   | <b>131</b> |
| 12.1      | Introducción a los Polinomios                       | 131        |
| 12.2      | Definiciones para Polinomios                        | 131        |
| <b>13</b> | <b>Constantes</b>                                   | <b>153</b> |
| 13.1      | Definiciones para Constantes                        | 153        |
| <b>14</b> | <b>Logaritmos</b>                                   | <b>155</b> |
| 14.1      | Definiciones para Logaritmos                        | 155        |
| <b>15</b> | <b>Trigonometría</b>                                | <b>159</b> |
| 15.1      | Introducción a la Trigonometría                     | 159        |
| 15.2      | Definiciones para Trigonometría                     | 159        |
| <b>16</b> | <b>Funciones Especiales</b>                         | <b>165</b> |
| 16.1      | Introducción a las Funciones Especiales             | 165        |
| 16.2      | specint                                             | 165        |
| 16.3      | Definiciones para las Funciones Especiales          | 166        |
| <b>17</b> | <b>Funciones elípticas</b>                          | <b>173</b> |
| 17.1      | Introducción a las funciones e integrales elípticas | 173        |
| 17.2      | Definiciones para funciones elípticas               | 174        |
| 17.3      | Definiciones para integrales elípticas              | 176        |
| <b>18</b> | <b>Límites</b>                                      | <b>179</b> |
| 18.1      | Definiciones para Límites                           | 179        |
| <b>19</b> | <b>Diferenciación</b>                               | <b>181</b> |
| 19.1      | Definiciones para Diferenciación                    | 181        |

|           |                                                    |            |
|-----------|----------------------------------------------------|------------|
| <b>20</b> | <b>Integración</b>                                 | <b>191</b> |
| 20.1      | Introducción a la Integración                      | 191        |
| 20.2      | Definiciones para Integración                      | 191        |
| <b>21</b> | <b>Ecuaciones</b>                                  | <b>209</b> |
| 21.1      | Definiciones para Ecuaciones                       | 209        |
| <b>22</b> | <b>Ecuaciones Diferenciales</b>                    | <b>227</b> |
| 22.1      | Definiciones para Ecuaciones Diferenciales         | 227        |
| <b>23</b> | <b>Métodos numéricos</b>                           | <b>231</b> |
| 23.1      | Introducción a los métodos numéricos               | 231        |
| 23.2      | Series de Fourier                                  | 231        |
| 23.3      | Definiciones para los métodos numéricos            | 231        |
| 23.4      | Definiciones para las series de Fourier            | 236        |
| <b>24</b> | <b>Estadística</b>                                 | <b>239</b> |
| 24.1      | Definiciones para Estadística                      | 239        |
| <b>25</b> | <b>Tablas y Arreglos</b>                           | <b>241</b> |
| 25.1      | Definiciones para Arreglos y Tablas                | 241        |
| <b>26</b> | <b>Matrices y Álgebra Lineal</b>                   | <b>245</b> |
| 26.1      | Introducción a las Matrices y al Álgebra Lineal    | 245        |
| 26.1.1    | Operador punto                                     | 245        |
| 26.1.2    | Vectores                                           | 245        |
| 26.1.3    | Paquete eigen                                      | 245        |
| 26.2      | Definiciones para las Matrices y el Álgebra Lineal | 246        |
| <b>27</b> | <b>Afines</b>                                      | <b>267</b> |
| 27.1      | Definiciones para Afines                           | 267        |
| <b>28</b> | <b>itensor</b>                                     | <b>269</b> |
| 28.1      | Introducción a itensor                             | 269        |
| 28.1.1    | Notación tensorial                                 | 269        |
| 28.1.2    | Manipulación indexada de tensores                  | 270        |
| 28.2      | Definiciones para itensor                          | 272        |
| 28.2.1    | Trabajando con objetos indexados                   | 272        |
| 28.2.2    | Simetrías de tensores                              | 281        |
| 28.2.3    | Cálculo tensorial indexado                         | 283        |
| 28.2.4    | Tensores en espacios curvos                        | 287        |
| 28.2.5    | Sistemas de referencia móviles                     | 289        |
| 28.2.6    | Torsión y no metricidad                            | 293        |
| 28.2.7    | Álgebra exterior                                   | 295        |
| 28.2.8    | Exportando expresiones en TeX                      | 298        |
| 28.2.9    | Interactuando con ctensor                          | 299        |
| 28.2.10   | Palabras reservadas                                | 300        |

|           |                                                 |            |
|-----------|-------------------------------------------------|------------|
| <b>29</b> | <b>ctensor</b> .....                            | <b>301</b> |
| 29.1      | Introducción a ctensor .....                    | 301        |
| 29.2      | Definiciones para ctensor .....                 | 303        |
| 29.2.1    | Inicialización y preparación .....              | 303        |
| 29.2.2    | Los tensores del espacio curvo .....            | 305        |
| 29.2.3    | Desarrollo de Taylor .....                      | 308        |
| 29.2.4    | Campos del sistema de referencia .....          | 311        |
| 29.2.5    | Clasificación algebraica .....                  | 311        |
| 29.2.6    | Torsión y no metricidad .....                   | 313        |
| 29.2.7    | Otras funcionalidades .....                     | 314        |
| 29.2.8    | Utilidades .....                                | 317        |
| 29.2.9    | Variables utilizadas por <b>ctensor</b> .....   | 321        |
| 29.2.10   | Nombres reservados .....                        | 325        |
| <b>30</b> | <b>atensor</b> .....                            | <b>327</b> |
| 30.1      | Introducción a atensor .....                    | 327        |
| 30.2      | Definiciones para atensor .....                 | 328        |
| <b>31</b> | <b>Series</b> .....                             | <b>331</b> |
| 31.1      | Introducción a las Series .....                 | 331        |
| 31.2      | Definiciones para las Series .....              | 331        |
| <b>32</b> | <b>Teoría de Números</b> .....                  | <b>343</b> |
| 32.1      | Definiciones para Teoría de Números .....       | 343        |
| <b>33</b> | <b>Simetrías</b> .....                          | <b>351</b> |
| 33.1      | Definiciones para Simetrías .....               | 351        |
| <b>34</b> | <b>Grupos</b> .....                             | <b>367</b> |
| 34.1      | Definiciones para Grupos .....                  | 367        |
| <b>35</b> | <b>Entorno de Ejecución</b> .....               | <b>369</b> |
| 35.1      | Introducción al Entorno de Ejecución .....      | 369        |
| 35.2      | Interrupciones .....                            | 369        |
| 35.3      | Definiciones para el Entorno de Ejecución ..... | 369        |
| <b>36</b> | <b>Opciones Misceláneas</b> .....               | <b>373</b> |
| 36.1      | Introducción a Opciones Misceláneas:: .....     | 373        |
| 36.2      | Share .....                                     | 373        |
| 36.3      | Definiciones para Opciones Misceláneas .....    | 373        |
| <b>37</b> | <b>Reglas y patrones</b> .....                  | <b>381</b> |
| 37.1      | Introducción a reglas y patrones .....          | 381        |
| 37.2      | Definiciones sobre reglas y patrones .....      | 381        |

|           |                                                     |            |
|-----------|-----------------------------------------------------|------------|
| <b>38</b> | <b>Listas</b>                                       | <b>397</b> |
| 38.1      | Introducción a las listas                           | 397        |
| 38.2      | Definiciones para listas                            | 397        |
| <b>39</b> | <b>Conjuntos</b>                                    | <b>403</b> |
| 39.1      | Introducción a los conjuntos                        | 403        |
| 39.1.1    | Utilización                                         | 403        |
| 39.1.2    | Iteraciones con elementos                           | 405        |
| 39.1.3    | Fallos                                              | 406        |
| 39.1.4    | Autores                                             | 407        |
| 39.2      | Definiciones para los conjuntos                     | 407        |
| <b>40</b> | <b>Definición de Funciones</b>                      | <b>421</b> |
| 40.1      | Introducción a la definición de funciones           | 421        |
| 40.2      | Funciones                                           | 421        |
| 40.3      | Macros                                              | 422        |
| 40.4      | Definiciones para la definición de funciones        | 425        |
| <b>41</b> | <b>Programación</b>                                 | <b>443</b> |
| 41.1      | Introducción a la Programación                      | 443        |
| 41.2      | Definiciones para la Programación                   | 443        |
| <b>42</b> | <b>Depurado</b>                                     | <b>451</b> |
| 42.1      | Depuración del código fuente                        | 451        |
| 42.2      | Claves de depuración                                | 452        |
| 42.3      | Definiciones para Depurado                          | 454        |
| <b>43</b> | <b>augmented_lagrangian</b>                         | <b>459</b> |
| 43.1      | Definiciones para augmented_lagrangian              | 459        |
| <b>44</b> | <b>bode</b>                                         | <b>461</b> |
| 44.1      | Definiciones para bode                              | 461        |
| <b>45</b> | <b>cholesky</b>                                     | <b>463</b> |
| 45.1      | Definiciones para cholesky                          | 463        |
| <b>46</b> | <b>descriptive</b>                                  | <b>465</b> |
| 46.1      | Introducción a descriptive                          | 465        |
| 46.2      | Definiciones para el tratamiento de datos           | 467        |
| 46.3      | Definiciones de valores descriptivos                | 470        |
| 46.4      | Definiciones de valores descriptivos multivariantes | 477        |
| 46.5      | Definiciones para gráficos estadísticos             | 481        |
| <b>47</b> | <b>diag</b>                                         | <b>487</b> |
| 47.1      | Definiciones para diag                              | 487        |



|           |                                                  |            |
|-----------|--------------------------------------------------|------------|
| <b>48</b> | <b>distrib</b> .....                             | <b>495</b> |
| 48.1      | Introducción a distrib .....                     | 495        |
| 48.2      | Definiciones para distribuciones continuas ..... | 498        |
| 48.3      | Definiciones para distribuciones discretas ..... | 520        |
| <b>49</b> | <b>eval_string</b> .....                         | <b>529</b> |
| 49.1      | Definiciones para eval_string .....              | 529        |
| <b>50</b> | <b>f90</b> .....                                 | <b>531</b> |
| 50.1      | Definiciones para f90 .....                      | 531        |
| <b>51</b> | <b>ggf</b> .....                                 | <b>533</b> |
| 51.1      | Definiciones para ggf .....                      | 533        |
| <b>52</b> | <b>ifactor</b> .....                             | <b>535</b> |
| 52.1      | Definiciones para ifactor .....                  | 535        |
| <b>53</b> | <b>impdiff</b> .....                             | <b>537</b> |
| 53.1      | Definiciones para impdiff .....                  | 537        |
| <b>54</b> | <b>interpol</b> .....                            | <b>539</b> |
| 54.1      | Introducción a interpol .....                    | 539        |
| 54.2      | Definiciones para interpol .....                 | 539        |
| <b>55</b> | <b>lindstedt</b> .....                           | <b>543</b> |
| 55.1      | Definiciones para lindstedt .....                | 543        |
| <b>56</b> | <b>linearalgebra</b> .....                       | <b>545</b> |
| 56.1      | Introducción a linearalgebra .....               | 545        |
| 56.2      | Definiciones para linearalgebra .....            | 547        |
| <b>57</b> | <b>lsquares</b> .....                            | <b>557</b> |
| 57.1      | Definiciones para lsquares .....                 | 557        |
| <b>58</b> | <b>makeOrders</b> .....                          | <b>561</b> |
| 58.1      | Definiciones para makeOrders .....               | 561        |
| <b>59</b> | <b>mnewton</b> .....                             | <b>563</b> |
| 59.1      | Definiciones para mnewton .....                  | 563        |
| <b>60</b> | <b>numericalio</b> .....                         | <b>565</b> |
| 60.1      | Introducción a numericalio .....                 | 565        |
| 60.2      | Definiciones para numericalio .....              | 565        |

|           |                                                |            |
|-----------|------------------------------------------------|------------|
| <b>61</b> | <b>opsubst</b> .....                           | <b>569</b> |
| 61.1      | Definiciones para opsubst .....                | 569        |
| <b>62</b> | <b>orthopoly</b> .....                         | <b>571</b> |
| 62.1      | Introducción a polinomios ortogonales .....    | 571        |
| 62.1.0.1  | Getting Started with orthopoly .....           | 571        |
| 62.1.0.2  | Limitations .....                              | 573        |
| 62.1.0.3  | Floating point Evaluation .....                | 575        |
| 62.1.0.4  | Graphics and orthopoly .....                   | 576        |
| 62.1.0.5  | Miscellaneous Functions .....                  | 576        |
| 62.1.0.6  | Algorithms .....                               | 578        |
| 62.2      | Definiciones para polinomios ortogonales ..... | 578        |
| <b>63</b> | <b>plotdf</b> .....                            | <b>583</b> |
| 63.1      | Definiciones para plotdf .....                 | 583        |
| <b>64</b> | <b>simplex</b> .....                           | <b>587</b> |
| 64.1      | Introducción a simplex .....                   | 587        |
| 64.2      | Definiciones para simplex .....                | 587        |
| <b>65</b> | <b>simplification</b> .....                    | <b>589</b> |
| 65.1      | Introducción a simplification .....            | 589        |
| 65.2      | Definiciones para simplification .....         | 589        |
| <b>66</b> | <b>solve_rec</b> .....                         | <b>597</b> |
| 66.1      | Introducción a solve_rec .....                 | 597        |
| 66.2      | Definiciones para solve_rec .....              | 597        |
| <b>67</b> | <b>stirling</b> .....                          | <b>601</b> |
| 67.1      | Definiciones para stirling .....               | 601        |
| <b>68</b> | <b>stringproc</b> .....                        | <b>603</b> |
| 68.1      | Introducción al procesamiento de cadenas ..... | 603        |
| 68.2      | Definiciones para entrada y salida .....       | 605        |
| 68.3      | Definiciones para caracteres .....             | 607        |
| 68.4      | Definiciones para cadenas .....                | 609        |

|                                                            |                                                           |            |
|------------------------------------------------------------|-----------------------------------------------------------|------------|
| <b>69</b>                                                  | <b>unit</b> .....                                         | <b>615</b> |
| 69.1                                                       | Introducción a Units .....                                | 615        |
| 69.1.0.1                                                   | Loading <i>unit</i> .....                                 | 615        |
| 69.1.0.2                                                   | Basic Usage .....                                         | 616        |
| 69.1.0.3                                                   | <code>convert</code> .....                                | 618        |
| 69.1.0.4                                                   | User defaults and resetting <i>unit</i> behavior<br>..... | 619        |
| 69.1.0.5                                                   | Bugs .....                                                | 623        |
| 69.1.0.6                                                   | Authors .....                                             | 623        |
| 69.2                                                       | Definiciones para Units .....                             | 623        |
| <b>70</b>                                                  | <b>zeilberger</b> .....                                   | <b>625</b> |
| 70.1                                                       | Introducción a zeilberger .....                           | 625        |
| 70.1.0.1                                                   | The indefinite summation problem ...                      | 625        |
| 70.1.0.2                                                   | The definite summation problem ....                       | 625        |
| 70.1.1                                                     | Verbosity levels .....                                    | 625        |
| 70.2                                                       | Definiciones para zeilberger .....                        | 626        |
| 70.3                                                       | General global variables .....                            | 627        |
| 70.4                                                       | Variables related to the modular test .....               | 628        |
| <b>71</b>                                                  | <b>Índice de Funciones y Variables</b> .....              | <b>629</b> |
| <b>Apéndice A Índice de Funciones y Variables</b><br>..... |                                                           | <b>631</b> |